普通高等教育“十一五”国家级规划教材

软件技术基础

[第三版]

黄迪明　主编

李玉柏　许家珆　黄迪明　廖建明　蔡洪斌　编著

电子科技大学出版社

图书在版编目（CIP）数据

软件技术基础 / 黄迪明主编．—3 版．—成都：电子科技大学出版社，2009.7
普通高等教育“十一五”国家级规划教材
ISBN 978-7-5647-0172-7

Ⅰ.软... Ⅱ.黄... Ⅲ.软件－高等学校－教材 Ⅳ.TP31

中国版本图书馆 CIP 数据核字（2009）第 077637 号

普通高等教育“十一五”国家级规划教材

软件技术基础

［第三版］

黄迪明 主编

李玉柏 许家珆 黄迪明 廖建明 蔡洪斌 编著

出　　版： 电子科技大学出版社（成都市一环路东一段 159 号电子信息产业大厦 邮编：610051）
策划编辑： 吴艳玲
责任编辑： 吴艳玲
主　　页： www.uestcp.com.cn
电子邮箱： uestcp@uestcp.com.cn
发　　行： 新华书店经销
印　　刷： 成都中铁二局永经堂印务有限责任公司
成品尺寸： 185mm×260mm **印张** 23.5 **字数** 571 千字
版　　次： 2009 年 7 月第一版
印　　次： 2009 年 7 月第一次印刷
书　　号： ISBN 978-7-5647-0172-7
定　　价： 32.90 元

■ 版权所有 侵权必究 ■

◆ 本社发行部电话：028-83202463；本社邮购电话：028-8328003
◆ 本书如有缺页、破损、装订错误，请寄回印刷厂调换。

内 容 提 要

本书为高校计算机基础教育第二层次的教材，是第一层次“计算机应用基础”和“程序设计语言”的后继课程用书。本书的第一版是国家电子信息类规划教材，本书是普通高等教育“十一五”国家级规划教材。全书共五章，主要内容包括：数据结构、操作系统、软件工程方法、数据库技术、网络技术基础。每章之后有小结和习题。全书内容紧凑、翔实，简明扼要，深入浅出，注重实用。

本书内容是按国家教育部高教司颁发的“工科非计算机专业计算机基础教学指南”中有关软件技术基础课程的教学要求编写的。本书可作为工科非计算机专业大学本科生、研究生教材，也可作为应用软件人员的培训教材或工程技术人员的参考教材。

前　言

本书为高校计算机基础教育第二层次的教材,是第一层次的后继课程用书。本书的第一版是国家电子信息类规划教材,第三版是普通高等教育“十一五”国家级规划教材。本教材由电子科技大学黄迪明主编。

《软件技术基础》是为非计算机专业的大学生和研究生学习计算机软件基础知识而编写的一本综合性教材。本教材内容符合国家教育部高教司颁发的“工科非计算机专业计算机基础教学指南”中有关软件技术基础课程的教学要求。

作为“计算机应用基础”和“程序设计语言”的后继课程，本教材以应用为目的，从计算机软件课程中选取了数据结构、操作系统、软件工程方法、数据库技术、网络技术基础五部分内容，简明扼要地介绍了计算机软件中的一些重要概念，以及软件技术的基础知识和方法，以培养学生利用计算机解决问题的意识和能力，为计算机在各专业中的应用奠定基础。

本教材的参考学时数为68学时(含上机)。各部分内容相对独立，自成体系，讲授时可根据专业需要及教学对象，酌情取舍(有的内容可供自学)。各部分内容之后附有小结和习题。

本教材由李玉柏编写第一章，许家珀编写第二章，黄迪明编写第三章，蔡洪斌编写第四章，廖建明编写第五章。全书由黄迪明统筹并修改定稿。电子科技大学王正智教授、兰家隆教授、俞永康教授对本书的编写提出了各种有益的建议，电子科技大学教务处及出版社对本书的出版给予了大力的支持，在此表示诚挚的感谢。由于编者水平有限，书中难免还存在一些缺点和错误，殷切希望广大读者批评指正。

编　者

2009年7月于电子科技大学

目　录

第 1 章　数据结构 1

1.1　数据结构的基本概念 1
1.1.1　什么是数据结构 1
1.1.2　数据结构中的基本概念 2
1.1.3　C语言的数据类型 5
1.1.4　算法的基本概念与算法效率 6
小结 8
1.2　线性结构 9
1.2.1　线性表 9
1.2.2　栈与队列 19
1.2.3　数组 30
1.2.4　串 33
小结 35
1.3　非线性结构 36
1.3.1　树结构及其基本概念 36
1.3.2　二叉树结构 37
1.3.3　图 48
1.4　查找与排序 56
1.4.1　查找 56
1.4.2　排序 64
小结 73
习题 74

第 2 章　操作系统 78

2.1　操作系统概述 78
2.1.1　操作系统的形成与发展 78
2.1.2　操作系统的功能 81
2.1.3　操作系统的特征 82
2.1.4　操作系统的分类 83
小结 87
2.2　处理机管理 87
2.2.1　进程的概念 87
2.2.2　进程控制 92
2.2.3　进程的互斥与同步 94

2.2.4 信号量机制与P、V操作 96
2.2.5 经典的同步问题 100
2.2.6 进程调度 103
2.2.7 死锁 106
2.2.8 进程通信 109
2.2.9 线程 112
小结 114
2.3 作业管理与用户接口 115
2.3.1 作业的概念 115
2.3.2 作业调度 116
2.3.3 操作系统接口 119
小结 121
2.4 存储管理 122
2.4.1 存储管理的功能 122
2.4.2 分区存储管理 124
2.4.3 覆盖与交换技术 127
2.4.4 虚拟存储管理 128
2.4.5 分页存储管理 128
2.4.6 段式存储管理 135
2.4.7 段页式存储管理 138
小结 140
2.5 设备管理 141
2.5.1 设备管理概述 141
2.5.2 数据传送控制方式 142
2.5.3 缓冲技术 145
2.5.4 设备分配 147
2.5.5 虚拟设备管理与SPOOLing技术 150
2.5.6 I/O管理 151
小结 152
2.6 文件管理 153
2.6.1 文件系统的概念 153
2.6.2 文件的组织 154
2.6.3 文件目录 156
2.6.4 文件的共享、保护和保密 159
2.6.5 文件存储空间的管理 161
2.6.6 文件的使用 162
小结 163
习题 163

第 3 章　软件工程方法 165
3.1　软件工程概述 165
3.1.1　软件工程学的形成与发展 165
3.1.2　软件工程及软件工程学 167
小结 168
3.2　软件与软件生存周期 168
3.2.1　软件 168
3.2.2　软件生存周期 168
3.2.3　软件开发过程模型 169
小结 172
3.3　软件的需求分析 173
3.3.1　需求分析概述 173
3.3.2　结构化分析方法 174
3.3.3　数据流图 176
3.3.4　数据词典 179
小结 183
3.4　软 件 设 计 183
3.4.1　软件设计概述 184
3.4.2　软件设计准则 186
3.4.3　结构化设计方法 188
3.4.4　详细设计方法 197
小结 205
3.5　软件编程 206
3.5.1　软件编程概述 206
3.5.2　程序设计语言 206
3.5.3　编程风格 210
小结 213
3.6　面向对象的分析和设计 214
3.6.1　面向对象的基本概念 215
3.6.2　面向对象分析（Object-Oriented An alysis，OOA） 217
3.6.3　面向对象设计（Object-Oriented Design，OOD） 221
3.6.4　统一建模语言（UML）简介 224
3.6.5　面向对象的程序设计概念 228
3.6.6　面向对象系统分析的一个实例——ATM系统 229
小结 236
3.7　软件测试 237
3.7.1　软件测试概述 237
3.7.2　软件测试策略 239

3.7.3 常用的测试方法 243
小结 247
3.8 软件维护 247
3.8.1 软件维护的概念 248
3.8.2 软件维护的步骤与方法 249
3.8.3 软件维护的副作用 250
小结 251
习题 252
第 4 章 数据库技术 253
4.1 数据库技术概论 253
4.1.1 数据库的基本概念 253
4.1.2 数据管理技术的发展 254
4.1.3 DBMS的主要功能 259
4.1.4 数据库系统 260
小结 262
4.2 数 据 模 型 262
4.2.1 概念模型 262
4.2.2 常见数据模型 266
4.3 关系数据库 270
4.3.1 关系数据结构 271
4.3.2 关系的完整性 272
4.3.3 关系代数 273
小结 276
4.4 SQL结构化查询语言 276
4.4.1 SQL语言概述 277
4.4.2 数据定义 277
4.4.3 数据查询 281
4.4.4 数据更新 287
4.4.5 数据控制 289
小结 290
4.5 数据库设计 290
小结 292
4.6 数据库保护 292
4.6.1 恢复技术 292
4.6.2 并发控制 296
4.6.3 完整性 298
4.6.4 安全性 300
小结 302

习题 303
第 5 章　网络技术基础 304
5.1　计算机网络概述 304
5.1.1　计算机网络概念 304
5.1.2　计算机网络分类 306
5.1.3　计算机网络体系结构 310
5.1.4　网络交换技术 315
小结 317
5.2　INTERNET网络及应用 318
5.2.1　Internet概述 318
5.2.2　TCP/IP协议 318
5.2.3　IP地址与域名 323
5.2.4　Internet服务 326
5.2.5　接入Internet方法 333
小结 339
5.3　基于TCP/IP协议的网络编程 339
5.3.1　客户/服务器模型 339
5.3.2　Socket及其调用的一般流程 341
5.3.3　Windows环境下的网络编程应用 346
小结 349
5.4　HTML语言与网页制作 350
5.4.1　HTTP和Web服务器 350
5.4.2　HTML和XML 351
5.4.3　网页设计与制作 353
小　　结 355
5.5　计算机网络安全 356
5.5.1　网络安全概述 356
5.5.2　数据安全技术 356
5.5.3　通信安全技术 357
小结 361
习题 362

第1章 数 据 结 构

随着计算机应用领域的不断扩大，大数据量的数值运算和非数值数据的处理变得尤为重要，这些数据的元素之间大多是相互有关的，它们的关系决定了对它们进行处理的方法，以及最后程序实现的方式。因此，讨论数据元素之间的逻辑关系、数据元素在计算机中的存储方式及在数据元素集合上设立的运算如何实现，这是研究数据处理的前提，是算法设计的基础，是程序编写的组成部分。本章首先介绍数据结构的相关概念，着重讨论线性结构、树型结构和图形结构三类数据结构，最后介绍数据结构中常用的查找和排序等基本运算。

1.1 数据结构的基本概念

1.1.1 什么是数据结构

早期的计算机多用于进行简单的数值运算，输入和输出的数据量不大，数据元素间的关系较为简单。但随着计算机应用的扩展，目前计算机被更多地用于大数据量的数值运算和非数值处理，如矩阵运算、管理与控制操作，合理安排数据元素之间的关系直接影响计算机运算的效率和使用的存储空间大小。正是如此，才产生了一门重要的计算机课程——数据结构。数据结构已成为学习和设计计算机系统软件、应用程序必不可少的基础知识。

顾名思义，数据结构是讨论计算机系统中数据的组织形式及其相互关系。在计算机系统中，数据不仅包含了通常数值的概念，还有更广泛的含义。它把客观事物采用计算机进行识别、存储和加工所进行的描述，统称为数据。例如，十进制、二进制数；字母、字符；程序段、图形图像、语言等数据信息。数据的基本单位为数据元素（有些书上直接称为数据节点）。

结构是指事物间的相互关系和约束。以一定存储方式存储在计算机系统中的数据元素，其排列并非杂乱无章，而是具有某种组织形式，包括数据元素内在的逻辑关系和存储形式上外在关系。数据元素的相互关系最终反映在对数据元素的一些运算和操作上。因此，研究数据结构，就是要研究以下三方面的内容：

（1）数据元素之间的逻辑关系是什么？

（2）适宜选用什么样的存储结构？

（3）采用什么样的操作实现算法效率更高？

为了增加对数据结构的感性认识，下面举例来具体说明上述概念。

例 1　学生成绩表为：

学号	姓名	数学	物理	电路基础	C 语言
200701101	丁一	87	90	86	84
200701102	马二	80	81	86	90
200701103	张三	91	88	90	87
200701105	李四	93	88	92	85
200701106	王五	67	66	70	65
⋮	⋮	⋮	⋮	⋮	⋮

这张学生成绩表就可称为一个数据结构，表中的每一行为一个数据元素。它是由学号、姓名、各科成绩等数据项组成。该表以学生学号顺序说明数据元素之间的相互关系，是一个线性表结构：对表中任一元素，与它相邻且在它前面的数据元素（亦称为直接前趋）最多只有一个；与表中任一数据元素相邻且在它后面的数据元素（亦称直接后继）也最多只有一个。表中只有第一个元素没有直接前趋，故称为开始数据元素。同时，也只有最后一个元素没有直接后继，故称之为终点数据元素。例如，表中“马二”所在元素的直接前趋和直接后继元素分别是“丁一”和“张三”所在的数据元素。上述元素间的学号顺序关系构成了这张学生成绩表的逻辑关系。

这张学生成绩表的存储方式则是指在计算机存储器中如何表示数据元素之间的这种关系，即表中的数据元素是顺序地邻接存储在一片连续的单元之中，还是用指针将这些元素链接在一起？在这张表中，可能要经常查看某一学生的成绩，当学生退学时要删除相应元素，当招收新学生时要添加新的数据元素，等等。那么，究竟要怎样进行查找、删除、插入，这就是数据高效操作所涉及的问题。搞清楚了上述三个问题也就弄清了学生成绩线性表这种数据结构。

数据结构的基本单位是数据元素，数据元素的类型可以是基本类型，如 int 整型、float 实数型，也可以是构造类型。在数据结构的讨论中，往往并不注重数据元素的类型，而是把它看成一个节点，着重讨论节点之间的关系。在上述学生成绩表中，数据元素的类型可用一个结构类型表示：

```
typedef struct student
    { int id;
      char name［20］;
      float score［4］;
    } elemtype;
```

这样就可以用 elemtype 来表示抽象了的数据元素类型。

1.1.2 数据结构中的基本概念

通常把运用数据结构来描述的数据元素之间的逻辑关系、数据在计算机系统中的存储方式和数据的运算抽象成数据结构的三个层次：数据的逻辑结构、数据的存储结构和数据操作集合。

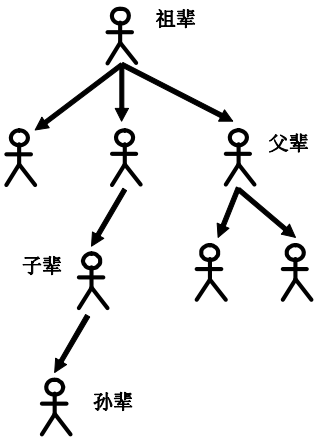
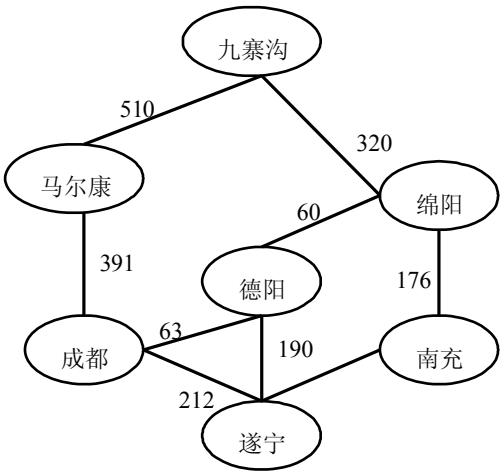
反映数据元素之间关系的数据逻辑结构可分为两大类：

（1）线性结构：线性结构的逻辑特征是，有且仅有一个开始数据元素和一个终点数据

元素，并且所有数据元素都最多只有一个直接前趋和一个直接后继。线性表就是一个典型的线性结构。

（2）非线性结构：非线性结构的逻辑特征是，该结构中一个数据元素可能有多个直接前趋和直接后继。非线性结构中最一般的结构是图结构，在图结构中，对任何数据元素的直接前趋和直接后继的个数都不作限制。在非线性结构中有一类较特殊的结构，我们称为树结构，它的逻辑特征是，有且仅有一个称为根的元素无直接前趋，其他元素有且仅有一个直接前趋，所有数据元素（除根元素外）都存在一条从根元素到该元素的路径。

根据逻辑结构对数据结构的分类可以总结如下：

<table>
<tr><th colspan="2">分类</th><th>逻辑关系特征</th><th>实例</th></tr>
<tr><td>线性结构</td><td>线性表</td><td>● 元素间为严格的一对一关系
● 元素最多只有一个直接前趋和直接后继</td><td>例 1 中的学生成绩表；</td></tr>
<tr><td rowspan="2">非线性结构</td><td>树结构</td><td>● 元素间为严格的一对多关系
● 元素最多只有一个直接前趋，但部分元素有多于一个的直接后继</td><td>比如组织关系和家族关系：
 
</td></tr>
<tr><td>图结构</td><td>● 元素间为多对多关系
● 元素有多于一个的直接前趋和直接后继</td><td>比如电信网络和公路交通：
 
</td></tr>
</table>

而反映数据元素在计算机中的存储方法就是数据的存储结构。数据的存储结构，有时也称为数据的物理结构，它是数据的逻辑结构在存储器里的实现。

数据的存储方法可分为如下四类：

（1）顺序存储方法

该方法是把逻辑上相邻的数据元素存储在物理位置上相邻的存储单元里，元素间的逻辑关系由存储单元的邻接关系体现。由此得到的存储表示称为顺序存储结构。顺序存储方法主要应用于线性的数据结构，如线性表、数组等。非线性的数据结构也可以通过某种线性化的方法来实现顺序存储。

（2）链接存储方法

该方法不要求逻辑上相邻的元素其物理位置上亦相邻，元素间的逻辑关系是由附加的指针字段表示的。由此得到的存储表示称为链式存储结构。链式存储结构要借助于程序语言的指针类型来描述元素的存储地址，即在此存储方法中，每个数据元素所占存储单元分成两部分：一部分为元素本身数据项；而另一部分为指针项，指出其后继或前趋元素的存储地址，从而形成一个链。

（3）索引存储方法

该方法通常是在存储元素信息的同时，还建立附加的索引表，索引表中的每一项称为索引项，索引项的一般形式是：（关键字、地址）。关键字是能唯一标识一个元素的数据项。若每个元素在索引表中都有一个索引项，则该索引表称为稠密索引（Dense Index）；若一组元素在索引表中只对应一个索引项，则该索引表称之为稀疏索引（Sparse Index）。稠密索引中索引项的地址指示元素所在的存储位置，而稀疏索引中索引项的地址则指示一组元素的起始存储位置。

（4）散列存储方法

该方法的基本思想是根据元素的关键字直接计算出该元素的存储地址。即在数据元素的字段中有一个或几个字段的值，通过某一散列函数唯一地确定该元素的存储地址。有时又称散列存储方法为“关键字－地址”转移法。

把数据以一定的逻辑结构组织起来，并以适当的方法存储在计算机系统的存储器里，其最终目的是为了有效处理数据，提高数据处理的运算速度。

在数据结构中，要讨论的常用数据处理与运算有下列几种：

（1）遍历：在数据结构的各个元素中移动，或查看所有数据元素。

（2）插入：往数据结构中添加新的元素。

（3）更新：修改或替代数据结构中指定元素的一个或多个数据项（字段值）。

（4）删除：把指定的数据元素从数据结构中去掉。

（5）查找：在数据结构中查找满足一定条件的数据元素。

（6）排序：在保持数据结构中数据元素个数不变的前提下，把元素按指定的顺序重新排列。排序一般是建立在线性逻辑结构的基础上。

值得指出的是，很多教科书上是将数据的逻辑结构和数据的存储结构定义为数据结构，而将数据的运算定义为数据结构上的操作。但是，无论怎样定义数据结构，都应该将数据的逻辑结构、数据的存储结构及数据的运算这三方面看成一个整体，希望读者学习时，不

要孤立地去理解一个方面，而要注意它们之间的联系。

上述四种基本的存储方法也可以组合起来对数据结构进行存储映像。同一种逻辑结构采用不同的存储方法，可以得到不同的存储结构。选择何种存储结构来表示相应的逻辑结构，主要是使其运算方便及根据算法的时空要求来具体确定。

正是因为存储结构是数据结构不可缺少的一个方面，所以我们常常将同一逻辑结构的不同存储结构冠以不同的数据结构名称来标识它们。例如，线性表是一种逻辑结构，若采用顺序方法的存储表示，则称该结构为顺序表；若采用链接方法的存储表示，则称该结构为链表；若采用散列方法的存储表示，则可称其为散列表。

同理，由于数据的运算也是数据结构不可分割的一个方面，所以给定了数据的逻辑结构和存储结构，若定义的运算集合及其运算的性质不同，也可能导致完全不同的数据结构。例如，若对线性表上的插入、删除运算限制仅在表的一端进行，则该线性表称之为栈；而插入限制在表的一端进行，删除限制在表的另一端进行的线性表称为队列。更进一步，若线性表采用顺序表和链表作为存储结构，则对插入和删除运算做了上述限制之后可分别得到顺序栈和链栈、顺序队列和链队列。

1.1.3 C 语言的数据类型

基于本书是使用 C 语言来进行算法的描述，为了便于读者阅读程序范例，了解各种常见的数据元素类型，在这里我们简要回顾一下 C 语言一些重要的数据类型。

（1）C 语言的基本数据类型

C 语言支持三种基本数据类型：int 型、float 型和 char 型。int 型可以有三个限定词：short、long 和 unsigned，对应短整型、长整型和无符号整型。float 型也有三种类型：float，double 和 longdouble，对应浮点型、双精度浮点型和增长的双精度浮点型。char 型为字符型，只占一个字节的存储空间。

（2）C 语言的指针类型

C 语言允许直接对存放变量的地址进行操作。如定义 int a，b，则&a 表示变量 a 的地址，也称为指向变量 a 的指针。存放地址的变量称为指针变量，指针变量指向的对象称为目标变量。如再定义 int *pa，则语句 pa = &a，表示 pa 为指向目标变量 a 的指针变量，目标变量 a 的值可用*pa 表示，则语句 b = *pa 等效于 b = a。另外，这里 pa 的含义不只是指针，而且是确定的指向整型类型的指针。

C 语言中指针的一个重要作用是在函数间传递数值。对一个形参为简单变量的函数，参数传递的是值传递，即把实参的值拷贝给形参，这样函数内形参的值被修改时实参的值不会跟着变化。若函数采用指针作形参，则参数传递采用地址传递，即把实参的地址传送给形参，或者说实参、形参共用一个存储单元。这时若函数内形参的值改变，则实参中的值将跟着变化。利用指针形参可设计函数的输出参数和变参。

在以后的程序举例中，我们会多次使用指针变量作为函数参数来实现在函数调用时，数据的双向传输。有时，还会使用指向指针的指针变量来作为函数的参数，以实现一个地址（指针）的双向传输。读者应特别留心。

（3）数组类型与字符串

在 C 语言中，数组是同一类型的一组有序数据的集合。数组名标识了这一组数，同时

代表了这一组数的起始地址，也就是说，数组名是一个指针变量。数组元素下标指示了数组元素在该数组中的顺序位置。可以有一维数组和多维数组，例如 int a［100］；定义了一个一维整数数组，下标为 0～99，一共 100 个元素；而 float b［10］［10］；定义一个二维浮点型数组，两个下标分别从 0～9 取值，一共也是 100 个元素。二维数组的两个下标分别称为行下标（第一维）和列下标（第二维）。

两维数组和多维数组的多下标结构，只是一种逻辑上的关系表示，因为，计算机的内存是一维的，只能一个一个元素顺序存放。C 语言中其物理存储结构是行主序存放方法实现的，即第一行的所有元素存完后，再顺序存放第二行的元素，以此类推。

C 语言中没有单独的字符串类型。字符串定义成字符数组。每个字符串由转义符′\0′指示其结束。一个字符串常量由一对双引号指示。一个字符串常量被存入内存，系统自动在其末尾添加字符串结束标志‘\0’。如 char name［20］；说明了一个 char 型的一维数组 name，其数据范围是 20 个字符。语句 name="Li Si"；则是对 name 赋一个字符串常量，字符串长为 5，在其末尾还有一个串结束标志‘\0’。

C 语言子程序函数提供有关字符串操作的各种函数，如字符串长度函数 strlen（name），字符串的拷贝函数 strcpy（str1，str2），字符串的连接函数 strcat（str1，str2），等等，读者可参考有关 C 语言的资料。

（4）C 语言的结构类型

结构类型由一组称为结构成员的项组成，每个结构成员都有自己的标识符。在前面定义 elemtype 类型时，就使用了结构类型的定义。

在 C 语言中定义一个结构变量由两步组成：一是定义结构类型；二是利用该类型来说明结构类型变量，如：

elemtype list［100］；

说明了每个元素为类型 elemtype 的一个数组 list。

除了以上所列举的数据类型，C 语言还支持诸如枚举类型、联合类型等其他类型，这些类型的应用相对较少，这里就不一一介绍，读者可参考有关 C 语言的资料。

1.1.4 算法的基本概念与算法效率

最初算法这个术语是对一种人类思维方式的定义，即从具体的操作规范入手，通过操作过程的构造与实施来解决给定问题的思维方法。这表明，算法既可以用来描述人解决给定问题的操作过程，也可以用来描述计算机解决给定问题的操作过程，后者正是我们编写程序的基础，是计算科学意义下的算法。一般地讲，计算机中科学的算法，是指为解决给定问题的有穷操作规则的有序集合。

一个算法必须具备有穷性、确定性、数据输入、信息输出以及可行性五项基本特征。

（1）有穷性：有始有终是算法最基本的特征。有始无终的过程决不是算法。换言之，一个算法必须在它所涉及的每一种情形下，都能在执行有穷步操作之后而告结束。

（2）确定性：算法的每一步操作，其顺序和内容都必须唯一地确定，不得有任何模糊。如果一个过程的某操作步骤有“既可这样，又可那样”的非确定性，那么它不是算法。

（3）数据输入：一个算法有零个或多个从约定对象集合中取出的原始数据输入。

（4）信息输出：既然算法是用来解决给定问题的，所以一个算法必须将人们关心的信息输出来。也就是说，一个算法至少有一个已获得的有效信息输出。

（5）可行性：一个算法的任何一步操作都必须是可以付诸实施并能完成的基本操作。换言之，各步骤不仅在理论上而且在实践上均能由人们用笔和纸做有穷次运算来完成。

在设计和构造算法时，要确保所得算法真正同时具备这五项基本特征，缺一不可。既然算法必须解决给定问题，算法的处理对象就必然是所论问题的有关数据。算法与数据结构之间存在密切关系，算法是建立在数据结构基础之上的。没有确定对数据进行何种操作，就无法决定如何构造数据结构。同样，确定算法也依赖于作为基础的数据结构。只有明确了问题求解的算法，才能够较好地设计数据结构，但是要设计好的算法，又常常依赖于合理的数据结构，数据结构是设计算法的基础。

对于一些复杂的问题，常常因为数据结构的差异，问题的求解算法也会完全不同，设计合理的数据结构常常可以有效地化简算法。例如，要在具有 100 万个用户的电话簿中查找某人的电话号码，根据数据结构的不同有如下两种算法来实现：一是电话簿中所有用户电话记录都是随机存放的，则只能按顺序依次查找，找到一个用户电话号码平均需要查看 50 万个用户记录；二是电话簿中所有用户电话记录按某个规则排序（序号顺序、字母排序等）存放，则可以用二分查找，平均只需要 28 次查找。两种查找算法的速度和效率相去甚远，主要是数据构造有明显差别。

在本章后面数据结构分析中，常常需要分析在特定数据结构上的算法的效率（由于本教材中算法举例都是很简单的，一个算法的效率往往就是一个基本运算的执行情况，所以会直接分析运算的效率来表征整个算法的效率。在更大型的程序和较复杂的处理中算法的效率和基本运算的效率是不同的两个概念），这里就给出算法效率的定义和计算方法。

算法的效率可分为时间效率和空间效率。算法执行时间需通过依据该算法编制的程序在计算机上运行时所消耗的时间来度量。空间效率主要考虑除存储数据结构本身外，实现算法所需要的辅助空间有多少。

算法执行时间需通过依据该算法编制的程序在计算机上执行时所消耗的时间来度量。而算法所需的运行时间与书写算法的程序设计语言、算法选用何种策略、编译后所产生的机器代码的质量、机器执行指令的速度以及待解决的问题规模大小有关。在这些因素中，前 4 个是与具体的机器或语言有关，在本章不予考虑，仅考虑算法本身效率高低。可以认为，一个特定算法的效率是问题规模的函数。假设整个算法的执行时间为 T，问题的规模为 n，一般情况下，算法中基本操作的重复执行次数是问题规模 n 的某个函数 f(n)，则算法的时间量度记为

$$T(n) = O[f(n)]$$

式中，O[f(n)] 称为语句频度，它表示随问题规模 n 的增大，算法执行时间的增长率和 f(n) 的增长率相同，称为算法的渐近时间复杂度，简称时间复杂度。语句的频度是指语句重复执行的次数，例如：

表达式	频度数量级
x = 0;	O[1]
for(i＝0; i＜n; i+) { a++ = i;}	O[n]
for(i＝0; i＜n; i++) for(j＝0; j＜n; j++) { a++ = (i+1)*(j+1);}	O[n^2]

除上述三种典型的时间复杂度外，算法可能呈现的复杂度还有：对数阶O[log (n)]、指数阶O[2^n]以及立方阶O[n^3]等。

类似于算法的时间复杂度，算法所需的内存空间称为空间复杂度。记做

$$S(n)＝O[f(n)]$$

一个算法所需的存储空间，除了存储程序本身所用的指令、常数、变量和输入数据外，也需要一些对数据进行操作和一些为实现算法所需信息的辅助存储空间。如果输入数据所占空间只取决于问题本身，且与算法无关，则只需分析除输入和程序之外的额外空间，否则应同时考虑输入本身所需的空间。

最后，我们来说明一下程序、算法与数据结构的关系。要使计算机能够完成人们预定的工作，首先必须根据任务和处理对象特征选择合适的数据结构，在此基础上设计一个算法，然后再根据算法和数据结构编写程序。计算机程序要对问题的每个对象和处理规则给出正确详尽的描述，其中程序的数据结构和变量用来描述问题的对象，程序结构、函数和语句用来描述问题的算法。算法和数据结构是程序的两个重要方面。

程序可以定义为：为解决给定问题的计算机语言有穷操作规则（即低级语言的指令，高级语言的语句）的有序集合。显然，当采用低级语言（机器语言和汇编语言）时，程序的表述形式为“指令的有序集合”，当采用高级语言时，程序的表述形式为“语句的有序集合”。著名的 PASCAL 语言发明者 N．沃斯教授进一步提出了如下公式：

程序 ＝ 算法＋数据结构

这个公式的重要性在于：不能离开数据结构去抽象地分析程序的算法，也不能脱离算法去孤立地研究程序的数据结构，而只能从算法与数据结构的统一上去认识程序。换言之，程序就是在数据的某些特定表示方式和结构方式基础上对抽象算法的计算机语言的具体表述。

小　　结

本节对数据结构的基本概念和它主要研究的内容进行了简要的介绍，尤其是对数据和数据结构中的一些基本名词和术语进行了解释；同时对算法的基本概念进行了说明；并回顾了 C 语言中数据类型说明的方法，这将有助于后续内容和程序范例的阅读和理解。

1.2 线 性 结 构

线性结构是数据结构中最简单且最常用的一种数据结构。线性结构的基本特点是数据元素有序并有限。正如上一节所述：线性结构的逻辑特征是在其结构中，有且仅有一个无直接前趋而仅有一个直接后继的数据元素为起始元素；有且仅有一个无直接后继而仅有一个直接前趋的数据元素为终点元素；其余均为内部元素，它们各有一个直接前趋和直接后继。因此，线性结构的数据元素可排成一个线性的序列：

a_1，a_2，a_3，…，a_n

其中，a_1为起始元素，a_n为终点元素，a_i为索引号为i的数据元素。

线性结构有各种类型，如线性表、堆栈、队列、数组、串等。

1.2.1 线性表

线性表是n（n≥0）个相同类型的元素a_1，a_2，…，a_n所构成的有限线性序列，通常表示为（a_1，a_2，…，a_n），其中n为线性表的长度。a_i（1≤i≤n）是线性表中第 i 个序号的数据元素。这里，a_i是抽象表示，在不同的情况下含义不同。例如：一个整数序列：（1，12，123，1234，321，21，22）是一个线性表，表中元素a_i是一个整数，表长为 7。而一周的七天：（SUN，MON，TUE，WED，THU，FRI，SAT）也是一个线性表，表中元素a_i是一个字符串，表长为 7。在前面例 1 中的学生成绩表中的数据元素a_i包括了姓名、学号、各门功课的成绩多个数据项，其数据类型需要用一个结构类型来描述。

线性表主要的基本操作有以下几种：

（1）INITIATE（L）{初始化}：设定一个空的线性表 L。

（2）LENGTH（L）{求表长}：对给定的线性表 L，函数返回值为其数据元素的个数。

（3）GET（L，i）{取元素}：若给定的数据元素序号i满足 1≤i≤LENGTH（L），则函数返回值为给定线性表L的第i个数据元素a_i，否则返回空元素。

（4）LOCATE（L，x）{定位}：对给定值 x，若线性表L中存在某个数据元素a_i等于 x，则函数返回该数据元素a_i在线性表L中的索引号i；若L中存在多个等于 x 的数据元素，则函数返回索引号最小的i值；若L中不存在等于 x 的数据元素，则函数返回一个特殊值（比如-1），以说明不存在的位置。

（5）INSERT（L，i，x）{插入}：对给定的线性表 L，若索引号 i 满足 1≤i≤LENGTH（L）+1，则在线性表的第 i 位置上插入一个新的数据元素 x，使原来表长为 n 的线性表变为表长为 n+1 的线性表，并使函数返回值为 true；否则函数返回值为 false。

（6）DELETE（L，i）{删除}：对给定的线性表L，若索引号i满足 1≤i≤LENGTH（L），则把线性表L中第i个元素a_i删除，使原来表长为 n 的线性表变成表长为n－1 的线性表，并使函数返回值为true；否则函数返回值为false。

对线性表有一些更为复杂的操作，如：将两个线性表合并成一个线性表；将一个线性表中的元素按值的大小重新排列等。这些操作都可以通过上面介绍的基本操作来实现。

上面定义了线性表的逻辑结构和基本操作。要使其成为计算机能够处理的对象，就必须把线性表的数据元素及数据元素之间的逻辑关系存储到计算机的存储器中。线性表有两

种存储方式，对应地把线性表分成了两类：顺序存储结构的顺序表和链式存储结构的链表。

一、顺序表

在顺序表的存储结构中，数据元素按其逻辑次序依次存放在一组地址连续的存储单元里。由于逻辑上的相邻的元素存放在内存的相邻单元中，所以顺序表的逻辑关系蕴含在存储单元的邻接关系中。

设在顺序表中每个元素占用k个存储单元，索引号为 1 的数据元素a_1的内存地址为$Loc(a_1)$，则索引号为i的数据元素a_i的内存地址为：

$$Loc(a_i)=Loc(a_1)+(i-1)*k$$

显然，顺序表中每个元素的存储地址是该元素在表中索引号的线性函数。只要知道某元素在顺序中的索引号就可以确定其在内存中的存储位置。所以，顺序表的存储结构是一种可以随机存取数据元素的结构。

顺序表的存储结构示意图如图 1.1 所示。显然这样一种存储结构，适合使用数组来表征。

存储地址	内存状态	元素索引号
Loc（a_1）	a_1	1
Loc（a_1）+k	a_2	2
…	…	⋮
Loc（a_1）+（i-1）*k	a_i	i
…	…	⋮
Loc（a_1）+（n-1）*k	a_n	n

图1.1　顺序表存储示意图

下面以顺序表的插入和删除算法为例，讨论在顺序存储结构下的运算实现及其算法的描述。顺序表可以用结构类型来说明：

```
typedef struct list_type
   { elemtype data ［MAXNUM］;
     int num;     /* 线性表表长 */
   } listtype;
listtype list;
```

其中，elemtype 为顺序表中元素的类型；MAXNUM 为顺序表最大元素个数，对 list 这个顺序表进行初始化：

```
void initiatelist（listtype *l）
     { l－＞num=0; }
```

【算法 1 】　顺序表的插入算法：

假设顺序表中已有 num（1≤num≤（MAXNUM－1））个数据元素，要在第 i 个位置上（0≤ i ≤num）插入一个新的数据元素，构成一个 num+1 长的顺序表。实现方法是：将原表中第 i 个元素到第 n 个元素后移一个位置，将要插入的元素 x 插在第 i 个位置上，再将表

长 num 加 1。

```
# define true 1
# define false 0
int insertl（listtype  *l，int  i，elemtype  x）
  { int j;
     if（l－>num>＝MAXNUM）
       {  printf（"the list is full，can not insert."）;
          return（false）;
       }
     if ((i<0）‖( i >l－>num))
       {  printf（"i is invalid value"）;
          return （false）;
        }
     for （j=l－>num－1；j>=i；j－－）
        l－>data［j+1］=l－>data［j］;
     l－>data［i］=x;
     l－>num++;
     return（true）;
  }
```

【算法 2】 顺序表的删除算法：

假设顺序表中已有 num（1≤num≤MAXNUM）个数据元素，要删除第 i（0≤i≤num－1）个元素，需将第 i 个元素到第 num 个元素依次前移一个位置，并使当前表长减 1。

```
# define true 1
# define false 0
int delete_ l（listtype * l，int i）
 { int j;
     if（(i<0）‖（i>l－>num－1））
       {  printf（"not exist"）;
          return （false）;
       }
     for（j=i+1；j<l－>num；j++）
     l－>data［j－1］=l－>data［j］;
     l－>num －－；
     return（true）;
 }
```

从上面的算法中不难看出：由于顺序表的随机存取特性，使得顺序表中对每一个元素的存储所需时间都很短，并且与其位置无关，基本运算为 1 次，算法效率为 O[1]；顺序表的插入和删除操作所耗费的主要时间是在移动元素上。如果在顺序表的任何位置上插入和删除一个数据元素时，平均约需移动 num/2 个数据元素，算法效率为 O[n]。

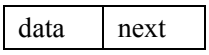
二、线性链表

采用顺序存储结构的线性表，其数据元素是用一组地址连续的存储单元来依次存放的，无须为表示数据元素间的逻辑关系而增加额外的存储空间，并且可以方便地随机存取表中的任一元素。但是从它的插入和删除算法可以看出，顺序表的运算效率较低，需要大量的数据元素的移位。同时，数据元素最大个数需要预先确定，以便计算机系统预先分配够相应的存储空间，这使得计算机存储器使用效率也不高。

而采用链式存储结构的链表是用一组任意的存储单元来存放线性表的数据元素，这组存储单元既可以是连续的，也可以是不连续的，甚至可以是零散分布在内存中的任何位置上，从而可以大大提高存储器的使用效率。

1．单向链表

单向链表是链式存储结构中最简单和最基本的结构。因为在链表中数据元素的逻辑次序与物理存储次序不一定相同，为了能正确表示数据元素间的逻辑关系，在存储每个元素值的同时，还必须存储其直接后继（或直接前趋）的存储位置，这一部分称为链（link）。因此，单向链表的每个数据元素都由两部分组成：存储元素值的数据域（data）和存储直接后继元素存储地址的指针域（next）。其结构如下：



此结构的 C 语言描述为：

```
struct node
  { elemtype data;
     struct node * next;
  };
```

一个线性表a_1，a_2，…，a_n，对应的单链表可用逻辑示意图 1.2 表示。其中，h为链表的头指针，用以确定线性表中第一个元素对应的存储位置。单链表可以用头指针的名字来命名。链表的终点元素无直接后继，故其指针域为空NULL，在图中用∧表示。

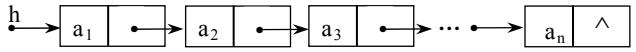


图 1.2　单链表的逻辑示意图

为了算法实现上的方便，通常在单链表的第一元素之前附加一个称为头节点的元素。头节点的数据域可以不存任何数据，也可以存放像线性表的表长的数据信息。一个带头节点的单链表的逻辑示意图如图 1.3 所示。此时，单链表的头指针指向头节点。

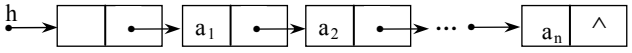


图 1.3　带头节点的单链表逻辑示意图

带头节点的单链表为空时，其逻辑示意图如图 1.4 所示。显然，带头节点的单链表把空表和非空表的处理统一起来了。对单链表初始化，就是建立一个空的单链表。

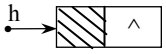


图 1.4　带头节点的空表

其函数如下：

```
void initiatesl（node **h）
    {  *h=（node*）malloc（sizeof（node））;
       （*h）－＞next=NULL;
    }
```

下面来介绍单链表的基本运算：

（1）链表的访问

若要访问带头节点的单链表 h 中的第 i（1≤i≤n）个数据元素，需首先在链表中找到第 i 个节点，然后返回第 i 个节点的数据元素值。

【算法 3】　访问单链表的算法：

```
elemtype access（node *h，int i）
  { node *p;
    int j;
    p=h;  j=0;
    while（p－＞next !=NULL && j<i）
      { p=p－＞next;
        j ++;
      }
    if （p!=NULL && j=i）
      return（p－＞data）;
    else  return（NULL）;
  }
```

（2）单链表的插入运算

设有线性表a_1，a_2，…，a_n，用带头节点的单链表存储，现要求在数据元素a_i的节点之前插入一个数据元素为x的节点。首先应找到数据元素a_{i-1}的节点，以便打开a_{i-1}与a_i两数据元素之间的链；然后申请一个新的节点空间，放上数据值x；最后，重新建立a_{i-1}与x、x与a_i两两数据元素之间的链，构成新的链表。算法示意图如图 1.5 所示。

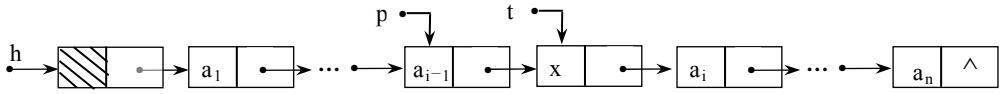


图 1.5　在带头节点的单链表中插入节点 t

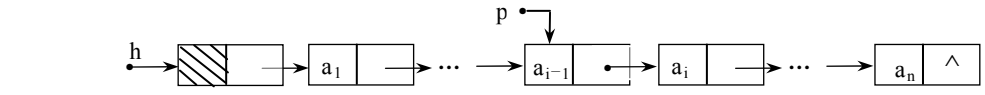
【算法 4】　单链表的插入算法：

```
void insertsl（node * h，int i）
 { node *p，*t;
   int j;
   p=h；j=0;
   while（p!=NULL && j<i－1）
     { p=p－＞next;
       j ++;
     }                    /*  寻找第 i－1 个节点  */
```

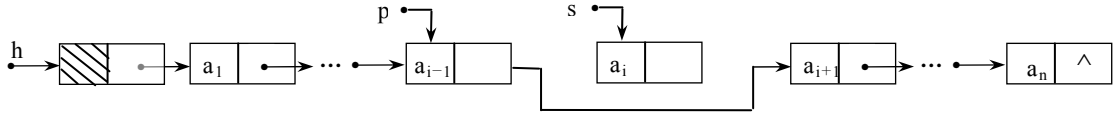
```
    if （j!=i－1）
      { printf（" i is invalid"）;
        return;
      }                          /* 插入位置 i 不合理 */
    t=（node *）malloc（sizeof（node））;
    t－＞data=x;                 /* 节点数据域赋值 */
    t－＞next=p－＞next;
    p－＞next=t;                 /* 链的重新连接 */
  }
```

（3）单链表的删除运算

删除操作和插入操作一样，首先要搜索单链表以找到指定删除节点的前趋节点（假设为 p），然后将待删除节点的指针域内容赋予 p 节点的指针域，最后释放删除节点所占的存储空间。图 1.6 显示了删除算法的单链表变化。



（a）



（b）

图 1.6　单链表的节点删除运算

（a）带头节点的单链表，寻找a_{i-1}节点　（b）删除i节点后的单链表

【算法 5】　单链表的删除算法：

```
void deletesl（node *h，int i）
 { node *p，*s;
   int j;
   p=h；j=0;
   while（p－＞next!=NULL && j<i−1）
     { p=p－＞next;
       j ++;
     }                          /* 寻找第 i−1 个节点 */
   if （j!=i-1）
     { printf（" i is invalid"）;
       return;
     }                          /* 判定 i 是否是有效节点 */
   s=p－＞next;
```

```
    p－＞next=s－＞next;        /* 链的重新连接 */
    free（s）;                  /* 释放被删除节点存储空间 */
  }
```

从上面的算法中不难看出：由于单链表以链接方式存放数据，所有数据元素是离散存放，可以充分利用存储空间；在进行插入和删除操作时不需要进行数据搬移，大大提高了处理速度，基本运算为 1 次，算法效率为 O[1]。但是，单链表在进行任何操作时需要从头节点开始访问数据元素，找到要进行操作的元素位置，平均访问的元素个数为链表长度的 1/2，即数据元素访问的基本运算为 n/2 次。此外，在单链表操作时，必须随时保存当前操作的元素指针，否则操作出错。

（4）单链表的建立

为建立链表，首先要为每个数据元素动态申请一个节点空间，然后通过指针将这些节点依次相连。链表是一种动态存储结构，每个节点所需的存储空间是在程序执行时动态申请的。利用前面已介绍过的链表插入新节点的算法，不难得到创建链表的算法：首先建立一个空链表，然后不断向链表中插入新的数据元素，形成各节点。

根据插入新节点的位置不同，可以形成两种创建新键表的方法：一是每插入的新节点作为链表的第一节点，此时，建立链表是从表的最后一个元素开始，从后向前依次插入节点的；二是每插入的节点是当前链表的最末节点，此时，建立链表是从表的第一个元素开始，从前向后依次插入节点的。下面就给出第二种方式的算法描述：

【算法 6】　单链表生成的算法：

```
void createsl（node **h）
 { node *p，*s;
    int j;
    elemtype x;
    *h=（node*）malloc（size of （node））;
    （*h）－＞next=NULL;              /* 初始化空表*/
    pread（x）;
    while（x!=-1）                   /* ′－1′为线性表结束标志 */
    {
      s=（node *）malloc（size of（node））;
      s－＞data=x;
      if （（*h）－＞next==NULL）
         （*h）－＞next=s;
      else   p－＞next=s;            /* 判定是否建立第一个节点 */
      p=s;                           /* p 始终指向最后一个节点 */
      pread（x）;
    }
```

```
    p－＞next=NULL；
  }
```

上面程序中函数 pread（x）是用于输入一个数据元素。它随着元素类型不同，实现方法及函数内容有所不同。另外，函数中使用的′－1′是作为无效数据元素，表明数据元素输入结束。

2．双链表

在单链表中，每个节点的指针域只放了一个指针，该指针指向该节点元素的直接后继元素的节点，每查找一个数据元素，都必须从头节点开始，从前向后单方向地搜寻。如果在链表的每个节点上再增加一个指向表中每个元素的直接前趋的指针域，就可以实现线性链表从后向前的搜寻，即可双向查找，这种指针域包括指向该节点的直接前趋和直接后继的两个指针的存储结构，称为双向链表，简称双链表。双链表的节点结构如图 1.7 所示。

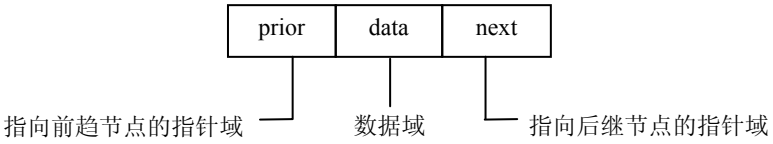


图 1.7　双链表节点结构

此结构的 C 语言描述为：

```
struct node
  { elemtype data；
    struct node * prior，*next；
  }；
```

一个带头节点的双链表，其逻辑示意图如图 1.8 所示。显然，在双链表中，并不需要用头指针 h 来命名这个链表，因为，知道任一节点的指针，就可以访问整个链表的所有节点，包括头节点。

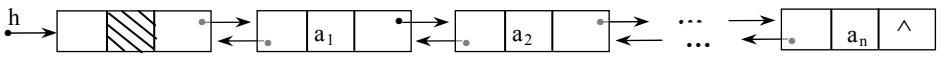


图 1.8　双链表的逻辑示意图

在双链表中，若 p 是指向表中某一节点的指针，则有：

p－＞next－＞prior=p

p－＞prior－＞next=p

下面给出双链表的节点插入和删除算法。

【算法 7】　双链表的节点插入：

要求在一双链表的节点 i 之前插入一个新节点，p 是指向节点 i 的指针，首先应该申请一个新节点所需存储空间（由指针 t 指向新节点），然后重新建立节点间的双向链，就可实现双链表的节点插入。图 1.9 说明了局部链表的结构变化情况。

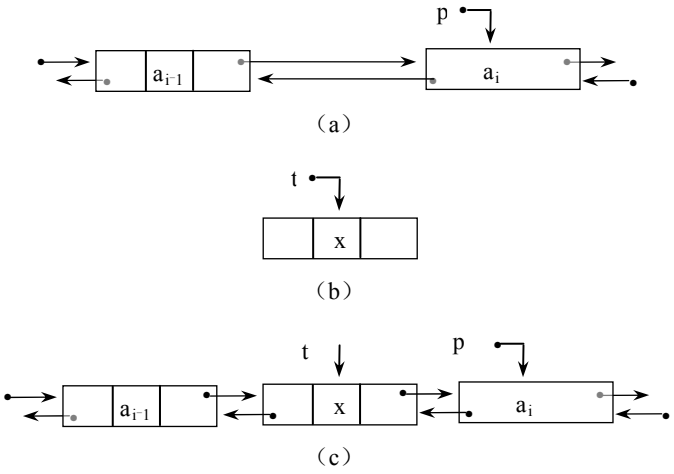


图 1.9　双链表的节点插入

（a）插入前局部链表结构　（b）申请新节点　（c）插入后局部链表结构

```
void insertdl（node *p，elemtype x）
 { node   *t;
   t=（node *）malloc（size of（node））;
   t－＞data=x;
   t－＞prior=p－＞prior;          /* 确定新节点的前趋 */
   p－＞prior－＞next=t;           /* 确定 i －1 节点的后继 */
   t－＞next=p;                    /* 确定新节点的后继 */
   p－＞prior=t;                   /* 重新定义 i 节点的前趋 */
 }
```

【算法 8】　双链表的节点删除：

要求删除双链表的节点 i，指针 p 指向节点 i，只需重新建立 i 节点前面节点的链，然后释放 i 节点的存储空间。图 1.10 说明了双链表节点删除的结构变化。

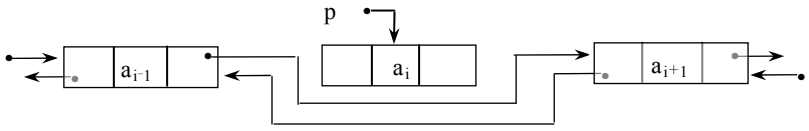


图 1.10　双链表的节点删除

```
void deletedl（node *p）
 {
   p－＞prior－＞next=p－＞next;
   p－＞next－＞prior=p－＞prior;
   free（p）;
 }
```

双链表的使用虽然给计算带来了方便，但是它是以占用较多内存空间为代价的。所以选择这种数据结构时，要根据问题的需要而定。

3．循环链表

循环链表是一种首尾相接的链表。在单链表中最后一个节点的指针域指向空（NULL），如果将它指向头节点（带头节点的链表）或第一个元素节点（不带头节点的链表），使整个链表形成一个环，这就构成了单向循环链表。同样，在双链表中，将双链表的最后一个节点与头节点（带头节点的双链表）或第一个元素节点（不带头节点的双链表）链接起来，就可构成双向循环链表。

将链表构造成循环链表可以实现在不需要增加存储量的情况下，仅对表的链接方式稍作变化，就使得表的处理更加灵活方便。比如，将单链表的尾节点与头节点相连所构成的单向循环链表，可以从表中任一节点处开始，遍历所有链表的其他节点。考虑到各种操作实现的方便性，循环链表仍为带头节点的循环链表，图 1.11 给出了循环链表的结构示意图。

与单链表常用头指针来命名不同，循环单链表更常用尾指针来命名。改用尾指针rear（简写r）来表示循环单链表，查找链表开始节点a_1和终端节点a_n都很方便，它们的存储位置分别是r－＞next－＞next和r，如图 1.12 所示。因此，实用中多采用尾指针命名循环单链表。

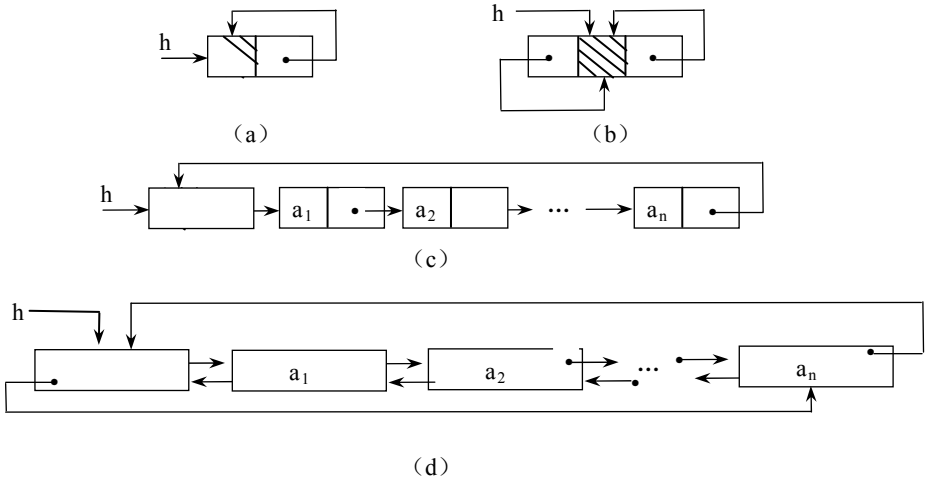


图 1.11　循环链表的结构示意图

（a）循环单链空表　（b）循环双链空表　（c）循环单链表　（d）循环双链表

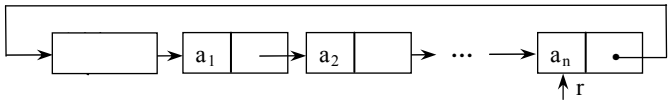


图 1.12　用尾指针表示的循环单链表

下面通过一个例子来说明循环链表的使用。

例 2　在链表上实现将两个线性表（a_1，a_2，…，a_n）和（b_1，b_2，…，b_m）链接成一个线性表（a_1，a_2，…，a_n，b_1，b_2，…，b_m）上的运算。

【说明】　在使用头指针表示的单链表和循环单链表的数据结构上来完成这种链接操作，都需要遍历第一个链表，找到节点a_n，然后将第二个链表的第一个节点b_1链到a_n后面。而在使用尾指针表示的循环单链表的数据结构上来实现，则只需修改一些指针域，无需对第一个链表进行遍历，因为使用尾指针可以很方便地查找循环单链表的首尾节点。下面connect_a是采用单链表实现链表连接的函数，connect_b是采用循环单链表实现的函数。

```
void connect_a（node **ha，node *hb）
 { node *p；
   p=*ha；
   while（p－＞next!=NULL）
     { p=p－＞next；
     }                              /* 遍历第一个链表，找到an */
   p－＞next=h b －＞next；          /* 将链表 hb 的第一节点与链表 ha 的尾节点链接 */
   free（hb）；                     /* 释放链表 hb 的头节点 */
 }
void connect_b（node *ra，node *rb）
 { node *p；
   p=rb－＞next；                   /* 保存 rb 的头节点地址 */
   rb－＞next=ra－＞next；          /* 链表 rb 的尾节点与链表 ra 的头节点链接 */
   ra－＞next=p－＞next；           /* ra 尾节点与 rb 的头节点链接 */
   free（p）；                      /* 释放 rb 的头节点 */
 }
```

循环单链表的指针修改过程如图 1.13 所示。

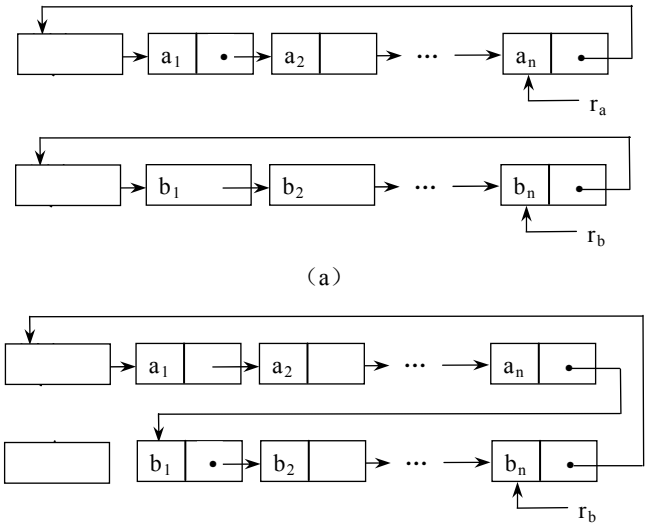


图 1.13　用尾指针说明的循环单链表的合并

（a）合并前的 ra，rb 循环单链表　（b）合并后的循环单链表

1.2.2　栈与队列

栈与队列是两种特殊的线性表。即它们的逻辑结构和线性表相同，只是其插入、删除运算仅限制在线性表的端点进行。栈和队列是程序设计中经常用到的两种数据结构。

一、栈

栈是限制仅在表的一端进行插入和删除运算的线性表。通常称插入、删除的这一端为栈顶，另一端称为栈底。当表中没有元素时称为空栈，栈的逻辑示意图如图 1.14 所示。

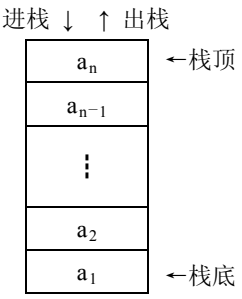


图 1.14　栈示意图

由于栈中元素的插入和删除只能在栈顶进行，所以总是后进栈的元素先出来，即后进先出（LIFO）。

栈的基本运算有五种：

（1）SETNULL（s）{置空栈}：将栈 s 置成空栈。

（2）EMPTY（s）{判空栈}：这是一个布尔函数。若栈 s 为空栈，则返回值为“真”，否则返回值为“假”。

（3）PUSH（s，x）{进栈}：在栈 s 的顶部插入（亦称压入）元素 x。

（4）POP（s）{出栈}：若栈 s 不空，则删除（亦称弹出）顶部元素。

（5）TOP（s）{取栈顶}：取栈顶元素，并不改变栈中内容。

由于栈是运算受限的线性表，因此线性表的存储结构对栈也适用。因此，栈也可以分成采用顺序存储结构的顺序栈和采用链式结构的链栈。

1．顺序栈

顺序栈是利用一组地址连续的存储单元依次存放从栈底到栈顶的若干数据元素。可以用一维数组来定义：

```
typedef struct stack_type
{ elemtype stack［MAXNUM］;
  int top;
} stacktype;                    /* 顺序栈结构定义 */
stacktype s;                    /* 定义一个堆栈 s */
```

其中，elemtype 为栈的数据元素类型；MAXNUM 为栈的最多元素个数，对栈 s 的初始化算法为：

```
    void initiatest （stacktype *s）
        { s－＞top = －1;
        }
```

下面以进栈和出栈为例，说明顺序栈的算法实现。

【算法 9】　顺序栈的元素插入算法：

```
# define true 1
# define false 0
int pushs（stacktype *s，elemtype x）
    { if（s－＞top>=MAXNUM－1）
        return（false）;                    /* 栈满，不能再插入 */
        else
          { s－＞top++;
            s－＞stack［s－＞top］=x;    /* 进栈 */
            return（true）;
```

```
        }
    }
```

【算法 10】　顺序栈的元素删除：

```
elemtype pops （stacktype *s）
 { if（s－＞top<0）
      return（NULL）;            /* 栈空 */
   else  { s－＞top--;
           return（s－＞stack［s－＞top+1］）;
         }                        /* 栈顶元素出栈 */
 }
```

2．链式存储结构的链栈

链栈是运算受限的单链表，其插入和删除操作仅限制在表头位置上进行。链栈中每个数据元素用一个节点表示，栈顶指针作为链栈的头指针，常用来命名一个链栈，不再增设头节点。链栈的数据结构可以定义为：

```
struct node
    { elemtype data;
      struct node * next;
    }
```

图 1.15 是链栈结构示意图。图中 top 为 node 类型的指针，指向栈顶，当 top=NULL 时，该链栈为空栈。

链栈中的节点是动态产生的，一般可以不考虑上溢出问题。下面给出链栈上的进栈和出栈算法。

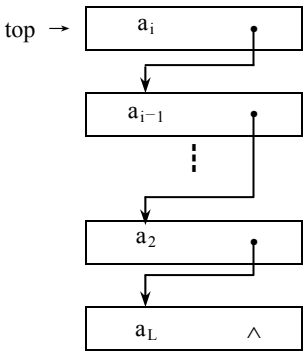


图 1.15　链栈示意图

【算法 11】　链栈的进栈算法：

```
void pushs（node** top，elemtype x）
 { node * p;
   p=（node *）malloc（sizeof（node））;
   p－＞data=x;
   p－＞next=（*top）;
   （*top）=p;
 }
```

【算法 12】　链栈的元素删除（出栈）算法：

```
elemtype pops（node **top）
 { node *p;
   elemtype x;
   if（(*top）=NULL）
        { printf（" stack is underflow"）;
```

```
        return（NULL）;
      }
  x=（*top）－>data;
  p=（*top）;
  （*top）=（*top）－>next;
  free（p）;
  return（x）;
}
```

二、队列

队列是另一种操作受限的线性表，它只允许在线性表的一端进行数据元素插入操作而在另一端才能进行数据元素删除操作。其中，允许插入的一端称为队尾，允许删除的另一端称为队头。根据队列的特点，队列需要两个队列指针来说明：一个是队头指针 front，它总是指向队头元素的前一个位置；另一个是队尾指针 rear，它总是指向队尾元素所在的存储位置。队列的结构示意图如图 1.16 所示。

由于队列中元素的插入和删除分别在队尾和队头进行，所以总是先入队列的元素先出队列，即队列具有先进先出的特性。故而队列又称为先进先出表（FIFO）。

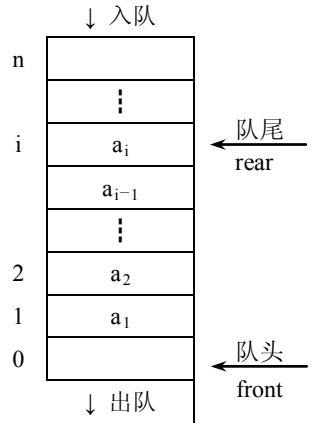


图 1.16　队列结构示意图

同栈的操作类似，队列的基本操作也有五种：

（1）SETNULL（Q）{置空队列}：将队列 Q 初始化为空。

（2）EMPTY（Q）{判队列空}：若队列 Q 为空队列，则函数返回 true；否则返回 false。

（3）ENTER（Q，x）{入队列}：若队列 Q 未满，在原队尾后加入数据元素 x，使 x 成为新的队尾元素。

（4）DELETE（Q）{出队列}：若队列 Q 未空，则将队列的队头元素删除。

（5）GETHEAD（Q）{取队头元素}：若队列 Q 未空，则函数返回队头数据元素，但不改变队列中的内容。

1．队列的顺序存储结构

同栈一样，可以用一组地址连续的空间存放队列中的元素。此时队列的结构可描述如下：

```
typedef struct queue_type
 { elemtype queue［MAXNUM+1］;
   int front;
```

```
    int rear;
  } queuetype;                /* 队列结构定义 */
queuetype Q;                  /* 定义一个队列 Q */
```

其中，MAXNUM 为队列最大长度，queue［MAXNUM+1］为一维数组，用于存放队列的节点元素。其中 queue［0］空着不用，queue［1］～queue［MAXNUM］为 MAXNUM 个队列元素。

初始化一个队列时，队头指针和队尾指针都置为 0，当元素插入队列时，队尾指针随之增加，当它为 MAXNUM 时表示队列已满。下面给出顺序队列入队与出队的算法。

【算法 13】　入队列操作：

```
# define true 1
# define false 0
int enter（queuetype *q，elemtype x）
    { if （q－＞rear>=MAXNUM）
      return（false）;
      else {
           q－＞rear++;
           q－＞queue［q－＞rear］=x;
           return（true）;
           }
    }
```

【算法 14】　出队列操作：

```
elemtype delete （queuetype * q）
  { if（q－＞font==q－＞rear）
        return（NULL）;          /* 队列已空 */
    else {
         q－＞front ++;
         return （q－＞queue［q－＞front］）;
        }
  }
```

顺序队列在进行入队列操作时，会产生假溢出现象，图 1.17 是队列假溢出的示例。图（a）为队列的初始状态，队列的最大长度为MAXNUM=6；图（b）为a_1，a_2，a_3进队列后的状态；图（c）为a_1，a_2出队列后的状态；图（d）为a_4，a_5，a_6进队列后的状态，此时rear=MAXNUM，表示队列已满，若再有a_7进队列就会溢出，这种溢出就是假溢出，因为队列中实际还存在可用空间；图（e）为a_3，a_4，a_5，a_6出队列后的状态，此时，会产生矛盾的结果：一方面rear=MAXNUM表示队列满，而另一方面front=rear表示队列空。要解决顺序队列假溢出问题，需要使用循环队列。

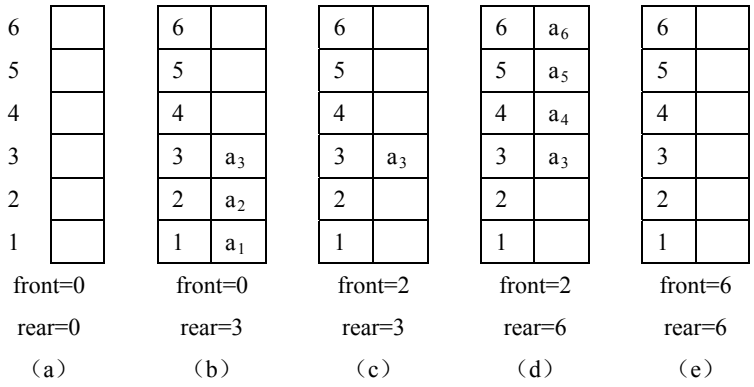


图 1.17　顺序队列的假溢出示例

循环队列是队列的结构空间首尾相连。这样，队列的最后一个单元已占用时（rear=MAXNUM），只要第一个单元是空就可以加入入队节点元素。如图 1.18 所示，像图 1.17（d）所示的情况，对于循环队列，要插入节点元素a_7，可在第一个单元中加入a_7，且把队尾指针指向 1 即可。队列删除时，则按a_3，a_4，a_5，a_6，a_7顺序进行。

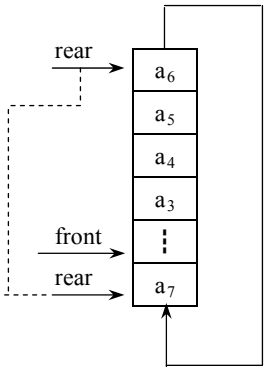


图 1.18　循环队列

从图 1.18 中可以看出：循环队列为空时，front=rear 成立，而循环队列为满时 front=rear 也成立（此时 rear 并不一定为 MAXNUM）。为了区别这两种状态，在循环队列中约定，队列仅还有一个空位时，队列就为满状态，即 front=rear+1 时约定循环队列为满。由此，可以判定循环队列的状态如下：

front=rear=0:　　初始的空队列

front=rear:　　操作过程中队列已空

front=rear+1:　　操作过程中队列已满

循环队列中还有一个问题需要解决，那就是队头指针和队尾指针在到达 MAXNUM 后的后移。这可以通过对地址进行 MAXNUM 的求模运算来实现，即

q－＞front=（q－＞front+1）% MAXNUM；

q－＞rear=（q－＞rear+1）% MAXNUM；

%是 C 语言的求模运算法。由此，可以改写循环队列为空的条件为

q－＞front==q－＞rear

循环队列为满的条件为

（（q－＞rear）+1）% MAXNUM==q－＞front

下面给出循环队列的入队与出队算法。

【算法 15】　循环队列的入队操作：

```
# define true 1
# define false 0
```

```
int enter（queuetype * q，elemtype x）
 { if（((q－＞rear）+1）% MAXNUM==q－＞front
          return （false）；      /* 队列已满 */
   else {
        q－＞rear=（q－＞rear+1） % MAXNUM；
        q－＞queue ［q－＞rear］=x；
        return（true）；
        }
 }
```

【算法 16】 循环队列的出队操作：

```
elemtype （queuetype * q）
 { if（q－＞rear==q－＞front）
          return （NULL）；      /* 队列已空 */
   else {
        q－＞front=（q－＞front+1） % MAXNUM；
        return （q－＞queue ［q－＞front］）；
        }
 }
```

2．队列的链式存储结构

利用带头节点的单链表可以作为队列的链式存储结构。由前所述，一个队列需要两个分别指向队头和队尾的指针才能唯一确定。在链队列里，将队头指针指向单链表的头节点，而将队尾指针指向单链表的最后一个节点。链队列的结构定义如下：

```
struct node
   { elemtype data；
     struct node * next；
   }；
struct qtype
   { node * front；
     node * rear；
   }
```

同链栈一样，链队列一般也不考虑队列的长度问题，节点进队列所需存储空间是通过动态申请来完成的，一般不会出现没有可用空间的情况。下面给出链队列的主要操作算法。

【算法 17】 进链队列操作算法：

```
void enter（qtype * q，elemtype x）
 { node * p；
   p=（node *）malloc（sizeof（node））；
   p－＞data=x；
   p－＞next=NULL；              /* 新的队尾节点 */
   q－＞rear－＞next=p；          /* 链入 队列中 */
```

```
    q－＞rear=p;                    /* 修改队尾指针 */
  }
```

【算法 18】 出链队列操作的算法：

出链队列操作是在队列的队头端口进行，在删除队头节点元素后，需要修改链队列的头指针（front）。如果队列长度仅为 1，即只有一个元素时，在进行删除操作时还要修改链队列的尾指针（rear），使它也指向头节点，这时链队列头、尾指针都指向头节点，说明链队列已空。

```
elemtype delete（qtype *q）
 { node *p;
   elemtype x;
   if （q－＞front－＞next==NULL）
        return （NULL）;          /* 队列空 */
   else {
         p=q－＞front－＞next;
         x=p－＞data;
         q－＞front－＞next=p－＞next;      /* 修改队头指针 */
         if（p－＞next==NULL）
              q－＞rear=q－＞front;        /* 如果队长为 1 时，修改队尾指针 */
         free（p）;
         return（x）;
        }
 }
```

三、栈和队列的应用

栈和队列是应用非常广泛的数据结构之一。在程序设计中，它们常用来保存一些尚未处理而又等待处理的数据。如果处理这些数据具有 FIFO 的特性，即先来的数据先处理，则使用队列结构；如果处理这些数据具有 LIFO 的特性，即后来的数据先处理，则使用栈结构。

栈结构最典型的应用是程序设计中子程序调用与返回的断点和现场数据的处理。当主程序调用子程序时，为保证执行它后能返回到主程序，必须将主程序调用与返回的断点和现场数据保存起来。如果子程序还嵌套调用其他子程序，又必须依次保存各级调用与返回的断点和现场数据，直到最深一级调用的子程序被执行。从子程序返回时，需要弹出返回断点地址和现场数据，其操作顺序与保存相反，即最后保存的断点与现场数据最先弹出。因此，利用栈结构，当子程序一级一级地返回，调用断点与现场数据可以很方便地一层一层弹出和处理。

递归算法是计算方法中非常重要、应用非常广泛的一种算法，在计算机应用系统中，递归算法是利用过程的递归调用来实现的。过程的递归调用是指某一过程在过程内部通过调用语句直接或间接地调用过程本身。实现过程递归调用的功能，需要使用栈结构来处理过程调用前的过程参数值与过程调用后的过程返回地址，以及一些现场数据，以确保过程参数传递正确和返回无误。

例 3　计算 n 的阶乘

根据阶乘的定义：$n!=\begin{cases}1 & 当n=0时\\ n\times(n-1)! & 当n\neq 0时\end{cases}$，可知为了计算 n 的阶乘，必须先计算 (n－1) 的阶乘；为了定义(n－1) 的阶乘，又必须先计算(n－2) 的阶乘……这种用自身的简单情况来计算自己的方式就是一个递归计算。编写程序可以采用递归调用的方式。

```
long int fact(int n)
 { if (n==0)
     then return 1;
     else return(n*fact(n-1));
 }
```

图 1.19 给出了 fact(3) 递归调用的情况，函数 fact(3) 的执行要调用 fact(2) ，函数 fact(2) 的执行要调用 fact(1)，递归调用直到 fact(0)，函数执行产生结果 1，然后依次返回。显然调用次序和返回顺序是相反的，最后调用的函数最先返回，这样的特征适合设置一个函数调用栈来保存返回地址，当递归调用函数时，将返回地址进栈，而函数执行完将当前栈顶的返回地址出栈，并返回。

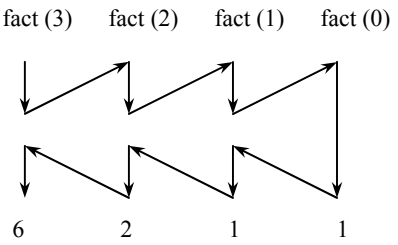


图 1. 19　递归调用与返回

栈结构另一个典型应用是编译系统中的表达式计算。在编译系统中，要把一个人便于理解的表达式翻译成能正确求值的机器指令序列，通常需要先把表达式变成机器便于理解的形式，这就需要变换表达式的表示序列。在高级语言编程中，表达式的描述是采用中缀表达式形式，即表达式中双操作运算符总是出现在两操作数之间，如 A+B、A*B、A*B+C/D 等。这种形式便于人的理解，但由于要处理括号和运算符的优先级，中缀表达式的运算符出现的顺序将不是实际执行的顺序，因而不便于机器的理解。在编译系统中对一个中缀表达式进行计算前，总是先将中缀表达式变换为相应的后缀表达式。后缀表达式是表达式中的运算符出现在操作数后，如 AB+、AB*、AB*CD/+。后缀表达式不含括号，它的操作数与中缀表达式的操作数的先后次序相同，只是运算符的先后次序有所改变。

下面给出利用栈结构把中缀表达式变换为相应的后缀表达式，并根据后缀表达式最终计算出表达式值的具体实现方法。首先，建立两个栈结构，一个称为操作数栈（DS），一个称为运算符栈（OS）。然后从左至右扫描中缀表达式，遇到操作数，则送入 DS 栈中，遇到运算符则将它与 OS 中栈顶运算符的优先级进行比较，若大于栈顶运算符的优先级，则将此运算符压入栈，否则退栈，并以此弹出的运算符与 DS 弹出的两个操作数进行操作形成中间结果压入 DS 栈，再继续扫描直到表达式结束。图 1.20 是对表达式 A*B+C/D 进行处理的过程图示。

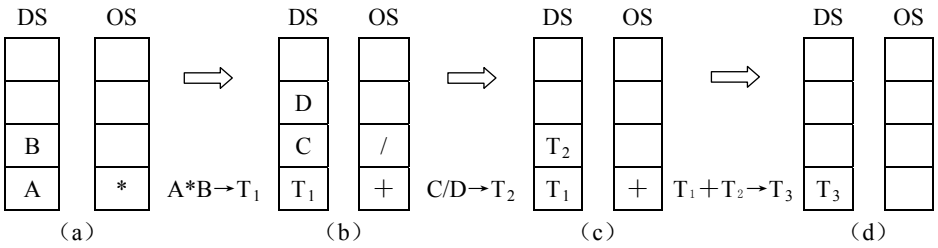


图 1.20　表达式转换的堆栈结构

（1）从左向右扫描表达式，将操作数 A、B 送入数据栈 DS，运算符*送入运算符栈 OS。

（2）扫描到+号时，因该运算符的优先级较*的优先级低，故+号暂不入栈，先弹出运算符*和操作数A、B，完成A*B，其结果T_1压入数据栈中，再将+号入栈。

（3）继续扫描将 C、D 送入数据栈。运算符/的优先级比当前栈顶运算符+的优先级高，故直接压入运算符栈中。

（4）扫描表达式结束，依次弹出运算符/和操作数C、D进行C/D运算，其结果T_2压入数据栈；再弹出运算符+和操作T_1，T_2，完成$T_1＋T_2$，生成的结果T_3就是表达式的值。

根据以上操作，不难写出计算机编译系统进行计算的后缀表达式为 AB*CD/+。

同栈结构一样，队列结构的应用也非常广泛。例如，在允许多道程序运行的计算机系统中，同时有几个作业运行。如果运行的结果都需要通过通道输出，那么就可以按请求输出的先后次序，将这些作业排列一个队列，按照“先来先服务”的原则输出这些结果。又比如在计算机系统中用键盘输入字符时，为解决主机处理与键盘输入数据速度匹配问题需设置键盘输入缓冲区，主机处理其他任务时，用户敲入的内容不能在屏幕上显示出来，而被存入到缓冲区的队列中，直到当前工作的进程结束后，主机再从缓冲区读取那些已键入的字符，并按队列方式处理。这里键盘输入缓冲区可采用循环队列，这样既由队列结构保证了输入字符先敲先存的要求，又由循环结构有效地避免了溢出问题。

队列结构还常常用于应用系统中的事件规划、事件模拟、非线性结构问题中的一些非递归搜索算法，也可以用于一些具体计算和操作。下面以杨辉三角系数打印为例说明队列的此类应用。

例 4　打印二项展开式$(a+b)^n$的系数（n = 1, 2, 3, … , N）

将二项式$(a+b)^n$展开，其系数构成杨辉三角形，题目要求按行将展开式系数的前N行打印出来，如图 1.21（a）所示。

从三角形的形状可知，除第一行以外，第 n 行的系数可以利用 n−1 行的系数来生成，即在打印 n−1 数据的过程中同时来计算和生成第 n 行的系数；在打印 n 行系数时，又可以为 n+1 行生成系数，依次类推。打印过程是读出数据，生成系数的过程是插入数据，显然，先插入的数据先输出打印，为此建立一个队列来完成要打印的数据管理：新一行系数的计算和插入队列，以及要打印的数据的读出。其具体操作步骤如下：

（1）在队列里插入第一行数据，并插入一个 0 作为该行系数结束；

（2）对 s 赋初值为 0，然后开始打印操作和新的一行系数计算和插入队列。其具体步骤是，读出队列值赋给 t，如果 t 不为 0 则打印 t，并计算 s+t 得到下一行系数插入队列；然后 s=t；继续从队列读出系数，重复进行操作。

（3）如果读出队列值赋给 t，而 t=0，则计算下一行系数最后一个系数 s＋t 插入队列；再向队列插入一个 0 作为下一行系数结束，同时打印机换行。

（4）重复（2）、（3）操作。

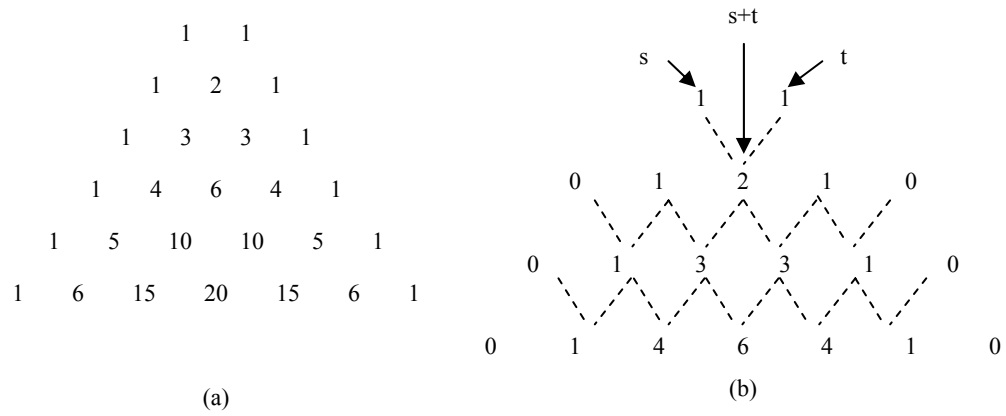


图1.21　杨辉三角系数的说明

	1	1	0	1	2	1	0	1	3	3	1	0	1	

图1.22　利用队列打印杨辉三角系数

可以画出需要建立的队列情况如图 1.22 所示，程序实现如下：

```
void yanghui(int n)
   {   queuetype *q;
       int s,t;
       int i,j;
       /*  在队列插入第一行数据；*/
       enter(* q, 1); enter(* q, 1);
       for (i=1; i<=n; i++)
          {   s=0;
              enter (* q, 0);   /*  在队列插入一行系数的结束标志数 0 ；*/
              printf ("\n");     /*  换行处理  */
              for (j=1; j<=i+2; j++)
                  {   t = delete(*q);   /*  从队列读出一数来打印  */
                      printf ("    %d", t);
                      enter (* q, s+t); /*  计算下一行系数并插入队列；*/
                      s=t;
                  }
          }
       }
```

1.2.3 数组

数组应是读者十分熟悉的数据类型，几乎所有的高级程序设计语言都支持数组这种数据类型，在前面讨论的各种线性表结构的顺序存储结构时都是借用一维数组来描述的。数组本身也是一种数据结构，一维数组是一种顺序表结构，多维数组是一种特殊的线性结构，是线性表的推广。基于二维数组应用最为广泛，像数学中的矩阵、生活中常见的报表都是二维数组，下面就以二维数组为例进行讨论，其结果不难推广到三维、多维数组。

二维数组

$$A_{mn}=\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

从逻辑结构上看，二维数组的每一行都是一个线性表，同行元素间的关系满足线性表的关系，即a_{11}是a_{12}的“行前趋”，而a_{12}是a_{11}的“行后继”。同理每一列也是一个线性表，元素间也满足线性表的关系。整个数组A_{mn}可以看成是由m个行向量组成的线性表，也可以看成是由n个列向量组成的线性表。前者称为数组的行主序，后者称为数组的列主序。线性表中的数据元素本身也是一个线性表，从这种意义上讲，数组可以看成是线性表的扩展。

数据通常有两种最基本的操作：

（1）给定一组下标，找到与之相应的数据元素。

（2）给定一组下标，存取或修改与其相应的数据元素中某个数据项的值。

这两种运算，可以归结为给定一组下标，确定与之相应的数据元素的存储地址。而这个问题是与数组的存储结构有关的。

1．数组的顺序存储结构

在计算机内是用一组连续的内存单元来存储数组的。常用的方法有两种，一种是行优先顺序存储：数组元素按行向量排列，即i+1 行向量紧接在第i个行向量之后。对于二维数组A_{mn}按行优先顺序存储的数据组元素次序为：

a_{11}，a_{12}，…，a_{1n}，a_{21}，a_{22}，…，a_{m1}，a_{m2}，…，a_{mn}

在 PASCAL、C 等高级语言中，数组就是按行优先顺序组织存储的。

设每个数组元素占S个存储单元，则在行优先存储中，二维数组A_{mn}的每个元素的存储地址可用下列计算公式算出：

$$Loc(a_{ij})=Loc(a_{11})+((i-1)*n+(j-1))*S$$

另一种数组顺序存储的方法是按列优先顺序存储：数组元素按列向量排列，即j+1个列向量紧接在第j个列向量之后，对于二维数组A_{mn}按列优先顺序存储的数组元素次序为：

a_{11}，a_{21}，…，a_{m1}，a_{12}，a_{22}，…，a_{m2}，a_{1n}，a_{2n}…，a_{mn}

在 FORTRAN 语言中，数组是按列优先顺序组织存储的。

设每个数组元素占S个存储单元，则在列优先顺序存储中，二维数组A_{mn}的每个元素的存储地址可用下列计算公式算出：

$$Loc(a_{ij})=Loc(a_{11})+((j-1)*m+(i-1))*S$$

上述二维数组的两种顺序存储方式不难推广到三维数组A_{1mn}。对于按行优先的顺序存放，则以左下标为主序，即先排右下标，最后排左下标；而按列优先顺序的存放，则以右下标为主序，即先排左下标，最后排右下标。对于三维数组A_{1mn}的每一个元素，存储地址的计算公式为：

按行优先的顺序存放

$$Loc(a_{ijk})=Loc(a_{111})+((i-1)*m*n+(j-1)*n+(k-1))*S$$

按列优先的顺序存放

$$Loc(a_{ijk})=Loc(a_{111})+((k-1)*l*m+(j-1)*l+(i-1))*D$$

采用顺序存储方式的优点是可方便地随机存取数组的数据元素。因为只要知道数组元素的一组下标值，就可以计算出该元素的存储位置。但是，顺序存储方式要求计算机存储系统提供能存放所有数组元素的一块连续的存储空间。在实际数值运算中，经常要处理的矩阵阶数很高，同时在矩阵中有许多具有相同值的元素或零元素。此时，顺序存储方式就不太合适了，会造成大量的存储单元浪费。因此，需要考虑一种更有效的存储结构，这种存储结构就是压缩存储结构。

2．矩阵的压缩存储结构

矩阵压缩存储结构的实质是尽量不为零元素分配存储空间，并使多个值相同的元素只占用一个存储空间。下面来介绍两类矩阵的压缩存储。

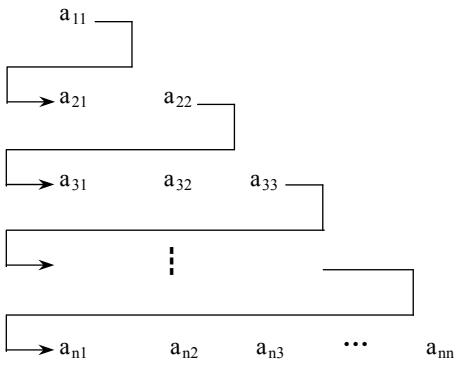


图 1.23　对称矩阵的压缩存储

（1）特殊矩阵的压缩存储

特殊矩阵是指值相同的元素或者零元素的分布有一定规律的矩阵，如对称矩阵、三角矩阵、对角矩阵等。

首先来看对称矩阵的压缩存储。对称矩阵中的元素关于主对角线对称，故只要存储矩阵中上三角或下三角中的元素，让每一对对称的元素共享一个存储空间。这样，能节约近一半的存储空间。不失一般性，可按行优先顺序存储方式对 n 阶对称矩阵A_{nn}主对角线（包括主对角线）以下的元素进行存储，其存放形式可用图 1.23 表示。

在这个下三角矩阵中，第i行共存储了i个元素，存储元素总数为$\frac{1}{2}n(n+1)$个。在计算机的存储器中对称矩阵A_{nn}被压缩成为一个顺序表，表长为$\frac{1}{2}n(n+1)$。矩阵A_{nn}的数组元素a_{ii}与顺序表的对应关系为：

当i≥j，即a_{ij}本身就是在下三角矩阵中。它对应在计算机存储系统中的存储位置为：

$$Loc(a_{ij})=Loc(a_{11})+((i(i-1)/2)+(j-1))*S$$

当i<j，即a_{ij}是在上三角矩阵中，它必须通过$a_{ij}=a_{ji}$找到对称元素a_{ji}的位置来得到数据元素本身。故它在计算机存储系统中的存储位置为

$$Loc(a_{ij}) = Loc(a_{11}) + ((j(j-1)/2) + (i-1)) * S$$

对称矩阵的压缩存储方式也同样适合于三角矩阵。

其次来看三对角矩阵的压缩存储。三对角矩阵中，所有的非零元素都集中在以主对角线为中心的带状区域中。即除了主对角线和主对角线邻近的上、下方的元素外，所有其他的元素均为零。n阶三对角矩阵A_{nn}

$$\begin{bmatrix} a_{11} & a_{12} & 0 & \cdots & 0 & 0 \\ a_{21} & a_{22} & a_{23} & \cdots & 0 & 0 \\ 0 & a_{32} & a_{33} & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn-1} & a_{nn} \end{bmatrix}$$

共有 3n−2 个非零元素。若按行优先顺序将三对角矩阵压缩存储到一个线性表中，则数组元素在该表中的存储次序是

a_{11}，a_{12}，a_{21}，a_{22}，a_{23}，a_{32}，…，a_{nn-1}，a_{nn}

设三对角矩阵上的非零元素为a_{ij}，这里下标i和j满足：i=j或i=j−1 或i=j+1，按行优先顺序压缩存储在计算机系统中的位置为：

$$Loc(a_{ij}) = Loc(a_{11}) + ((i-1)*2+j-1) * S$$

（2）稀疏矩阵

如果一个矩阵中绝大多数为零，非零元素很少且分布无一定的规律，这类矩阵称之为稀疏矩阵。

在实际应用中，经常会遇到处理大量的稀疏矩阵情况。因此设法压缩零元素占据的存储空间，不仅可节省内存空间，还可避免零元素进行无意义的运算，提高处理的效率。按照压缩存储的概念，稀疏矩阵只需存储矩阵中的非零元素。这可以通过使用三元组存储法来实现。这里三元组指非零元素所处的行值和列值，以及非零元素本身的值。

设 A 是一个 m×n 的稀疏矩阵，其中非零元素的个数为 I，三元存储法可采用一个（I+1）×3 的 T 来存储 A。矩阵 T 的第一行是 m，n，I；后面的每一行对应 A 矩阵中的一个非零元素，如某一行为 i，j，x；对应的是矩阵 A 中第 i 行，第 j 列的那个值为 x 的非零元素。矩阵 A 中各非零元素对应的三元组可按其行的递增次序（相同行按列的递增次序）排序，这称为行优先的三元组存储方式。当然也可以按列优先的三元组存储方式排序。

例如，按上述三元组原则，下列 5×6 稀疏矩阵按行优先的三元组存储表示为它右侧的$T_{6\times3}$。

$$A_{5\times6} = \begin{bmatrix} 1 & 0 & 0 & 0 & 7 & 0 \\ 0 & 0 & 12 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -3 & 0 & 0 & 0 & 0 \end{bmatrix} \Leftrightarrow T_{6\times3} = \begin{bmatrix} 5 & 6 & 5 \\ 1 & 1 & 1 \\ 1 & 5 & 7 \\ 2 & 3 & 12 \\ 3 & 1 & 3 \\ 5 & 2 & -3 \end{bmatrix}$$

三元组存储方法仍是基于线性表结构，而这种结构适合于存取和修改的运算，因为这种运算不改变矩阵的稀疏程度。如果要对稀疏矩阵进行相加、相乘等就要改变矩阵稀疏程度，用三元组存储方式，运算就很麻烦。这时可用链接结构方法来解决，这里就不讨论了。

1.2.4 串

串是字符串的简称，它是由零到多个字符组成的连续有限序列，一般记为：

$$S='a_1a_2a_3\cdots a_n'$$

其中，S为串名，引号中为串值（单引号本身不是串值）。a_i是串元素，它由字母或其他符号组成。n（n≥0）是串的长度，当串长度为 0（即n＝0）时称为空串，记为S＝′ ′ 。空格也可作为串字符，由于各个串的串值是不等长的，大多数高级语言在实现串操作时都要作专门处理以确定串的长度，为此应特别注意不要将串值为空格字符的空格串等同成空串。为了明确起见，一般用′ □′ 表示一个空格的串。

一个串中任意多个连续的字符组成的子序列称为该串的子串，包含该子串的串称为主串，一个字符在串中的序号称为该字符在串的位置。当一个字符在串中多次出现时，该字符在串中的位置指的是该字符在串中第一次出现的位置。子串在主串中的位置指的是该子串的第一个字符在主串中的位置。称两个串是相等的即是指两个串中的字符序列一一对应相等。

例如有如下一些串：

S1＝′ I am a student′ ；

S2＝′ teacher′ ；

S3＝′ a′ ；

S4＝′ student′ ；

S5＝′ student□′ ；

在上面这五个串中，可以说 S1 是 S3 和 S4 的主串，子串 S3 在主串 S1 中的位置为 3，子串 S4 在主串 S1 中的位置为 8。S4 和 S5 为不同的两个串，S4 的长度为 7，S5 的长度为 8，它不是 S1 的子串。

对比串的定义和线性表的定义可知，串是一种其数据元素固定为字符的线性表。因此，仅就数据结构而言，串归属于线性表这种数据结构。但是，串的基本操作和线性表上的基本操作相比却有很大的不同。线性表上的操作主要针对线性表中的某个数据元素进行，而串上的操作主要是针对串的整体或串的某一部分子串进行，下面是串的基本操作：

（1）ASSIGN（S，T）{赋值}：把串 T 的值赋给串 S。

（2）EQUAL（S，T）{判相等}：S，T 为串名或串值，若 S 和 T 相等，则函数返回 true，否则函数返回 false。

（3）LENGTH（S）{求串长}：求串 S 的长度，返回值为串 S 中字符的个数。

（4）SUBSTR（S，start，len）{取子串}：从串 S 中的第 start 个字符开始，抽取出 len 个字符构成一个新的串。其中，参数应满足：1≤start≤LENGTH（S）且 1≤len≤LENGTH（S）－start＋ 1 。

（5）CONCAT（S1，S2）{连接}：将 S2 的串值紧接着放在 S1 串值的末尾而组成一个

新的串，新的串值名为 S1。

（6）INDEX（S1，S2）{定位}：若在主串 S1 中存在与 S2 相等的子串，则函数返回 S1 中第一个与 S2 相等的子串在主串 S1 中的位置，否则函数值返回为 0。

（7）REPLACE（S1，S2，S3）{替换}：该函数完成一个置换操作，操作结果是用串 S3 替换串 S1 中所有与串 S2 相等且不重叠的子串。

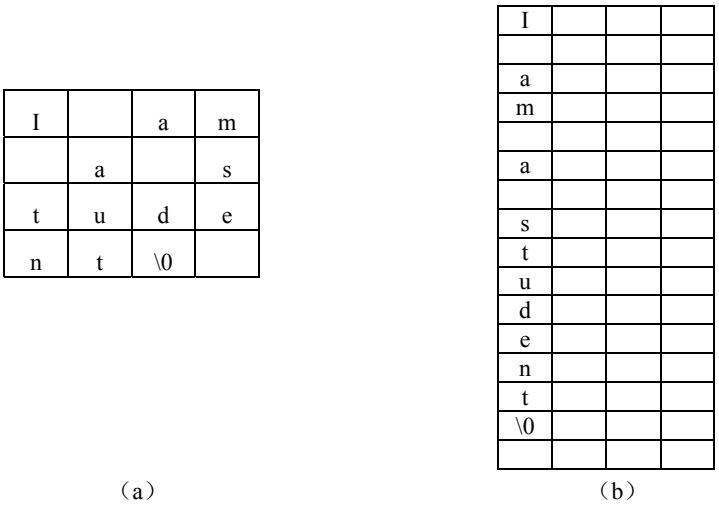
与线性表的存储结构类似，字符串的两种基本存储结构是顺序结构和链式存储结构。

顺序存储结构是用一块地址连续的存储单元存储串的字符序列。由于大多数计算机的存储器地址采用的是以字编址，一个字占多个字节（一般为 4 个字节），而一个字符只占 1 个字节，所以为了节省存储空间，顺序存储结构存储串值时允许采用紧缩格式，即一个字节存放一个字符，而把一个字存储单元中放一个字符称为非紧缩格式。显然，非紧缩格式浪费存储空间，但操作方便；紧缩格式节省存储空间，但若要分离某一部分字符时，操作比较麻烦。图 1.24（a）是串值的紧缩格式存储示例；图 1.24（b）是串值的非紧缩格式存储示例。通常系统在存储串值时自动在串值的末尾处添加一特殊符号作为当前串值的结束标记。在 C 语言中串值结束标记采用转义字符′\0′。

C 语言用字符数组存放字符串，其定义格式如下：

```
# define MAXNUM 80                    /* 串允许的最大字符个数 */
typedef struct str_type
    { char str［MAXNUM］;      /* 串值 */
        int len;                        /* 串长 */
    } strtype;                      /* 串类型定义 */
strtype s;                          /* 串变量定义 */
```

C 语言中字符数组在内存中是使用紧缩格式存放，即仅支持紧缩格式；而一些其他语言，如 PASCAL 语言允许非紧缩格式和紧缩格式两种方式。在 PASCAL 语言中常规定义的字符数组采用非紧缩格式存储。使用保留字 PACKED 来定义的字符数组采用紧缩格式存储。



（a）　　（b）

图 1.24　（a）串值的紧缩格式存储　（b）串值的非紧缩格式存储

串的顺序存储结构有两个缺点：一是需要预先定义一个串允许的最大字符个数，当该值估计过大时存储效率就降低，浪费存储空间；二是当需要在字符串中插入或删除字符时，要花许多时间移动字符。解决这个问题可以使用链表存储结构来存储串。串的链式存储结构的定义如下：

```
struct node
  { char data;
    struct node *next;
};
```

设 S＝′ student′，则该字符串的链表存储结构示意图如图 1.25 所示。

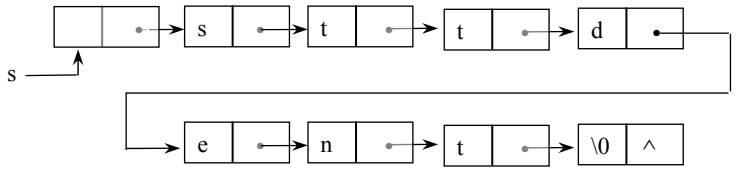


图 1.25　串的链表存储结构

作为链式存储结构的共同特点，存储一个串的字符数据节点的存储空间可以动态申请。不需要预先定义；同时对串进行字符数据节点插入和删除时都不需要作大量的字符移动。

小　　结

线性结构是最简单且最常用的一种数据结构。线性结构的基本特点是数据元素有序并有限。本章对常用的几种线性结构：线性表、堆栈、队列、数组和串进行了讨论。

线性表的逻辑结构特征是数据元素之间存在着线性关系。即：有且仅有一个开始元素没有直接前趋，而仅有一个直接后继；有且仅有一个终点元素没有直接后继，而仅有一个直接前趋；其余的内部节点元素都有且仅有一个直接前趋和一个直接后继。在计算机中，用顺序存储方式来表示这种线性关系，则得到线性表的顺序存储结构（即顺序表）；用链表存储方式来表示这种线性关系，则得到线性表的链式存储结构（即链表）。

通常顺序表是用一个一维数组和一个指示当前表中终点元素位置的指针 last 来描述，若数组的下界是 i，则当前表的长度应是 last－i＋1。

链表是由指针类型描述的，节点元素的存储空间的分配和释放可以动态执行，节点元素的插入和删除很方便，不需大量数据元素的移动。链表可以有单链表、循环链表及双链表等多种形式，通常一个链表是由一个头指针唯一确定的。

对于顺序表最基本的操作是插入和删除节点元素；对于链表最基本的操作包括链表的建立、节点元素的查找、插入和删除等操作。

栈和队列是两种特殊的线性表，它们的逻辑结构和线性表相同，但其运算规则受限制。栈是限制仅在表的一端进行插入和删除运算的线性表。进行插入、删除的一端称为栈顶，另一端称栈底。栈中元素总是最后进栈的元素，最先退栈，具有后进先出（LIFO）的特征；队列却只允许在表的一端进行插入，而在另一端进行删除。允许删除的一端称为队头，允

许插入的一端称为队尾。队列中元素总是先进入队列的元素先离开队列，具有先进先出（FIFO）的特征。

同一般线性表一样，栈和队列也可以采用两类存储结构存储：一是顺序结构方式存储，二是链式结构方式存储。顺序队列中会产生“假溢出”现象，因此，在实际应用中，顺序队列常被循环队列来代替。

栈和队列的最基本运算有五种：初始化、判空、进栈 / 队列、出栈 / 队列、取栈/队列顶元素。基于这五种基本操作，堆栈和队列在数值计算、程序设计、多任务管理、编译系统等多方面完成数据序列的特定变换。

数组是应用最广泛的一种数据结构，由于多维数组的逻辑特征是一个数组元素可能有多个直接前趋和直接后继，因此是一种非线性结构。但是多维数组可以理解其数据元素本身就是一种数据结构的线性表，从这种意义上，数组可以看成是线性表的扩展。

数组的存储结构一般采用顺序存储结构方式。对于二维数组及多维数组有两种顺序存储方式：一种是行优先顺序，即在计算机存储系统中，数组元素按行向量排列；另一种是列优先顺序，即在计算机存储系统中，数组元素按列向量排列。对于一些特殊矩阵，如对称矩阵、三对角矩阵、三角矩阵等，由于它们的非零元素或零元素的分布是有一定规律的，可以采用压缩存储方式进行存储。对于稀疏矩阵，非零元素很少，可以用三元组表示法来存储矩阵。

不论是采用顺序存储方式还是压缩存储方式，在进行数组元素的存取时，都要进行下标变换或地址换算。

串是零个或多个字符组成的有限序列，是一种特殊的线性表，它的每个节点元素仅由一个字符组成。另外，串上的操作主要是针对串的整体或串的某一部分子串进行，不同于一般线性表是针对表中某一个节点元素进行的。

1.3　非线性结构

在前面已经介绍过，非线性结构的逻辑特征是一个节点元素可能有多个直接前趋和多个直接后继。最主要的非线性结构是树结构和图结构。

1.3.1　树结构及其基本概念

树结构是一类重要的非线性结构。树结构是节点之间有分支、层次关系的结构。在客观世界中，树结构是大量存在的，例如家谱、行政组织机构都可用树形象地表示。在计算机领域中，树结构也被广泛应用，如计算机磁盘文件的管理，是一种从根目录到各级子目录的分层结构；在数据库系统中，常采用树来组织数据信息。

树结构非常类似于自然界中的树，也有树根、树叶及联系它们的枝干。不过这里讨论的树结构是一种倒生树，可以用递归的方式定义如下：

树是一个或多个节点元素组成的有限集合 T，且满足如下条件：

（1）有一个特定的节点元素，称为根节点（Root）；

（2）其余节点元素分成m个（m≥0）互不相交的有限集T_1，T_2，…，T_m，其中每个集又都是一棵树，这些树称为Root的子树。

在树中，一个节点元素常简称节点。采用递归方式定义树结构，揭示出树的固有特性。实际上，树中的每个节点都是该树中某一子树的根。图 1.26 给出了一棵树的逻辑结构示意图，结合该示意图，下面介绍一些有关树结构的重要术语与概念。

（1）叶子　没有后继的节点称为叶子（或终端节点），如图 1.26 中的节点 D、E、F、G、H、J。

（2）分支节点　非叶子节点称为分支节点（或非终端节点）。

（3）节点的度　一个节点的子树数目就称为该节点的度，如图 1.26 中的节点 B 的度为 2；节点 C 的度为 3；节点 D、J 的度为 0。

（4）树的度　树中各节点的度的最大值称为该树的度，如图 1.26 所示的树的度为 3。

（5）子节点　某节点子树的根称为该节点的子节点。

（6）父节点　相对于某节点子树的根，称该节点为子树根的父节点。

（7）兄弟　具有同一父节点的子节点称为兄弟。如图 1.26 中节点 C 是节点 G、H、I 的父节点；节点 G、H、I 是节点 C 的子节点；节点 J 是节点 I 的子节点；节点 G、H、I 互为兄弟。

（8）节点的层次　根节点的层次为 1，其他任何的层数等于它的父节点的层数加 1。

（9）树的深度　一棵树中，节点的最大层次值就是树的深度。如图 1.26 所示的树的深度为 4。

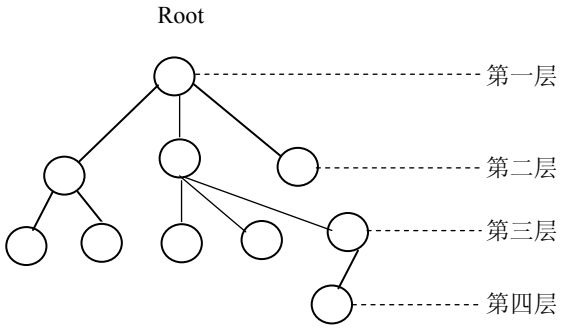


图 1.26　树结构的图形表示

（10）有序树和无序树　如果一棵树中节点的各子树从左到右是有序的，即若交换了某节点各子树的相对位置，则构成了不同的树，称这棵树为有序树；反之，则为无序树。

（11）森林　森林是 n 棵树的集合（n≥0）。任何一棵树，删去根节点，树就变成了森林。对树中的每个节点来说，其子树的集合就是一个森林。

1.3.2　二叉树结构

二叉树结构同一般树结构一样，也是非线性结构中重要的一类，许多实际问题抽象出来的数据结构往往是二叉树的形式，即使是一般的树结构也能简单地转换为二叉树，因此，二叉树结构显得特别重要。

一、二叉树的基本概念

二叉树的定义如下：

二叉树是 n 个节点的有限集合（n≥0）；这个集合可以是空（即 n＝0），此时称为空二叉树；或者由一个根节点和两棵互不相交的被称为根的左子树和右子树组成。左子树和右子树分别又是一棵二叉树。

这也是一种递归定义。二叉树可以是空集，同样，根可以有空的左子树或右子树，或者左、右子树皆为空。由此，二叉树可以有五种基本形态，如图 1.27 所示。



图 1.27　二叉树的五种基本形态

（a）空二叉　（b）仅有一个根节点的二叉树　（c）根的左子树非空、根的右子树为空的二叉树
（d）根的右子树非空、根的左子树为空的二叉树　（e）根的左、右子树皆为非空的二叉树

必须注意一般树与二叉树概念上的不同。树至少应有一个节点，而二叉树可以是空；其次，二叉树的节点的子树要区分为左子树和右子树，即使某节点只有一棵子树，也要明确指出该子树是左子树还是右子树，而树中则无此区分。

前面 1.3.1 小节中介绍的有关树的基本术语和概念在二叉树中同样适用。这里再介绍几个有关二叉树的基本概念：

（1）满二叉树，在一棵二叉树中，如果所有分支节点都存在左子树和右子树，并且所有叶节点都在同一层上，这样的一棵二叉树称作满二叉树。

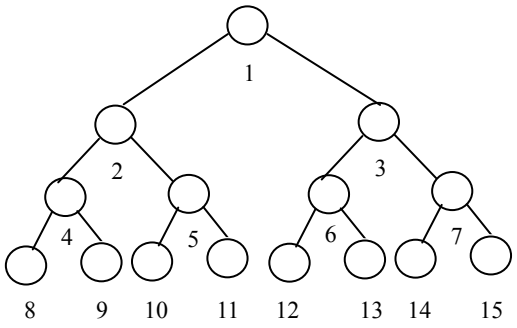


图 1.28　满二叉树

图 1.28 是一个满二叉树的图示说明，图中节点内的字符代表该节点本身的数据信息，节点下面的数字为该节点的顺序编号，这里约定编号顺序是按从上到下、从左到右的顺序进行。

（2）完全二叉树，若一棵二叉树至多只有最下面的两层上节点的度数可以小于 2，并且最下一层上的节点都集中在该层最左边的若干位置上，则此二叉树称为完全二叉树。图 1.29 是完全二叉树和非完全二叉树的图示说明。图中（b）的二叉树的节点 F 没有左子树而有右子树，故它不是一棵完全二叉树。显然一个 n 个节点的二叉树，其所有节点的编号和一个满二叉树的 1～n 个节点编号完全一致，则它肯定是完全二叉树。

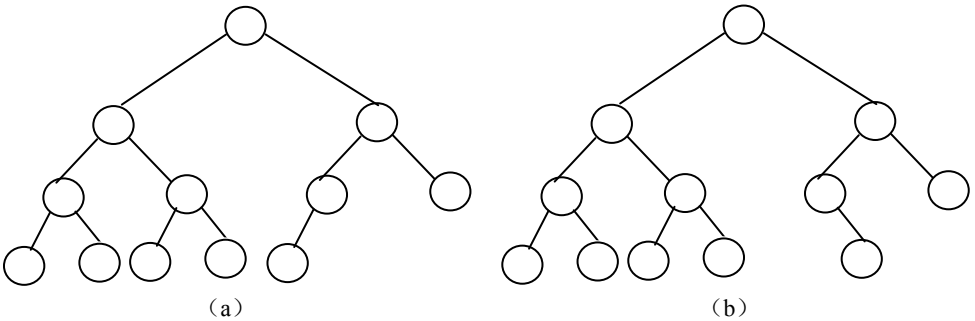


图 1.29　特殊形态的二叉树

（a）完全二叉树　（b）非完全二叉树

（3）二叉排序树，二叉排序树可以通过递归方式来定义，它或者是空二叉树，或者是具有如下性质的二叉树：左子树上所有节点的关键字均小于根节点的关键字；右子树上所有节点的关键字均大于等于根节点的关键字。左子树和右子树本身又各是一棵二叉排序树。图 1.30 是一棵二叉排序树的图示，图中每一个节点上的数字是节点数据元素的关键字。

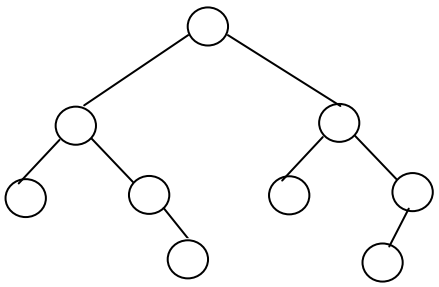


图 1.30　二叉排序树的图示

二、二叉树的存储结构

二叉树的存储方式也可以分为顺序存储和链式存储两种。顺序存储方式是把二叉树的所有节点按照一定的次序存储到一片连续的存储单元中，因此，必须把节点安排成一个适当的线性序列，使得节点在这个序列中的相互位置能反映出节点之间的逻辑关系。

对于满二叉树和完全二叉树，具有如下性质：对于有 n 个节点的满二叉树和完全二叉树，如果从上至下和从左到右的顺序对二叉树中的所有节点从 1 开始顺序编号，则对于任意序号 i 的节点有：

（1）如果 i＞1，则序号为 i 的节点的父节点的序号为 i / 2（取整）；如果 i＝1，则节点是根节点，无父节点。

（2）如果 2i≤n，则序号为 i 的节点的左子节点的序号为 2i。

（3）如果 2i＋ 1 ≤n，则序号为 i 的节点的右子节点的序号为 2i＋ 1 。

显然，满二叉树和完全二叉树中节点的序号可以唯一地反映出节点之间的逻辑关系，所以可以用顺序存储方式按从上到下、从左到右的顺序存储树中所有节点的数据信息。

图 1.31 给出了如图 1.29（a）所示的完全二叉树的顺序存储示意图。

图中给出了数组下标，它是二叉树中节点序号减 1。利用数组下标关系就可以反映满二叉树和完全二叉树中节点之间的逻辑关系。

然而对于一般的二叉树，如果仍按照从上到下、从左到右的顺序将树中的节点顺序存储在一维数组中，则数组元素下标之间的关系不能够反映二叉树中节点之间的逻辑关系，这时可以将一般二叉树进行改造，人为增添一些并不存在的空节点（其元素值为“空”），使之成为一棵完全二叉树的形式，然后再用一维数组顺序存储。显然这可能会造成大量的存储空间的浪费，尤其是那些右单支二叉树，此时应考虑使用链式存储方式存储。

	A	B	C	D	E	F	G	H	I	J	K	L
数组下标	0	1	2	3	4	5	6	7	8	9	10	11

图 1.31　完全二叉树的顺序结构

根据二叉树的特性，任何一个节点最多有左、右两个子树，也就是说，对于二叉树中任何一个节点，我们可以采用如图 1.32 所示的结构来描述。

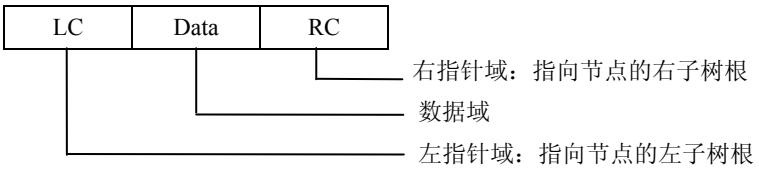


图 1.32　二叉树链表的节点结构

这种存储结构称为二叉树链表（简称二叉链表）。当使用二叉链表方式存储一棵二叉树时，如果某节点的左子树或右子树为空，则相应的指针域为“空”，除此之外，还可设置一指针变量指向二叉树的根节点，称为头指针。和单链表中头指针的作用相似，二叉链表中的头指针可以唯一地确定一棵二叉树。在二叉链表上，可以比较方便地从某个节点出发，找到它的一子节点。但如果要从某个节点找其父节点就比较麻烦了。所以，在有些情况下，可以采用一种称为三叉链表的存储结构，即在二叉链表的节点上再增加一个指向父节点的指针（Parent）。三叉链表的节点结构如图 1.33 所示。

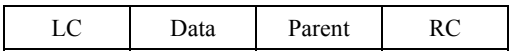


图 1.33　三叉链表的节点结构

例 5　画出给定二叉树的二叉链表存储结构和三叉链表存储结构，结果如图 1.34 所示。

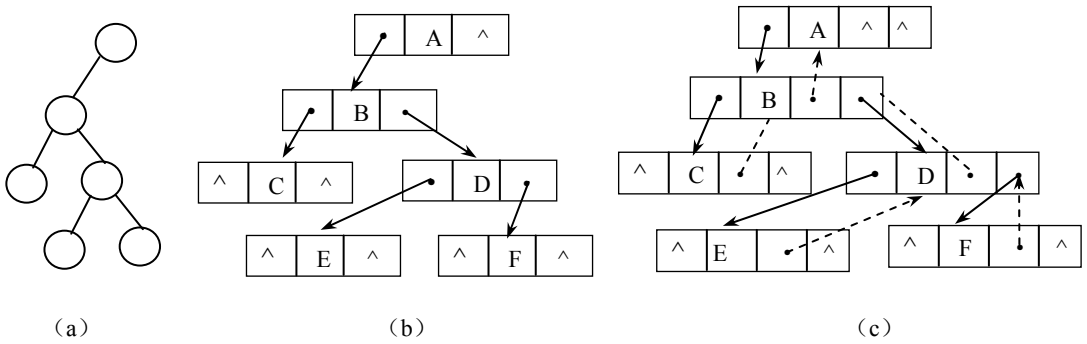


图 1.34　二叉树的链式存储结构

（a）二叉树　（b）二叉链表结构　（c）三叉链表结构

三、二叉树的遍历

遍历是树结构的基本操作。树遍历的含义是指用一定的规律走遍树的每一个节点，使每个节点被访问一次而且只被访问一次。访问该节点时，可对该节点进行操作。

任一棵二叉树都由三部分组成：即根节点（记作 D）、左子树（记作 L）、右子树（记作 R）。这样，遍历一棵二叉树的次序有六种：DLR、LDR、DRL、RDL、LRD、RLD。若限定对左右子树的访问是先左后右，则只有三种：DLR（称为先序遍历）、LDR（称为中序遍历）和 LRD（称为后序遍历）。

1．先序遍历

先序遍历的递归定义为：

若二叉树为空，遍历结束，否则：

（1）访问根节点；

（2）按先序遍历方式遍历根节点的左子树；

（3）按先序遍历方式遍历根节点的右子树。

下面给出先序遍历的算法。假定二叉树的存储结构为二叉链表，其节点的类型定义如下：

```
typedef struct btreenod
        { elemtype data;
          struct btreenode *LC;
          struct btreenode *RC;
        } bnode;
bnode *BT;
```

【算法 19】　二叉树的先序遍历算法：

```
void preorder（bnode *BT）
    { if（BT==NULL）
            return;
    else
      { visit（BT）;          /* 访问 BT 指向的根节点 */
        if（BT－>LC!=NULL）preorder（BT－>LC）;
        if（BT－>RC!=NULL）preorder（BT－>RC）;
      }
    }
```

上述算法是用递归完成的，这是与二叉树的递归定义相符合的。递归算法简单，但读者不容易看出算法的执行过程。利用一个辅助堆栈，可以将递归算法转化为等价的非递归算法，有兴趣的读者可以试着编写一下二叉树的先序遍历非递归算法的程序。

2．中序遍历

中序遍历的递归算法可描述如下：

（1）按中序遍历的方法遍历根节点的左子树；

（2）处理根节点；

（3）按中序遍历的方法遍历根节点的右子树。

【算法 20】　二叉树的中序遍历算法：

```
void inorder（bnode *BT）
    { if（BT==NULL）
            return;
    else
      { if（BT－>LC!=NULL）inorder（BT－>LC）;
        visit （BT）;              /* 访问 BT 指向的根节点 */
```

```
        if（BT－>RC!=NULL）inorder（BT－>RC）;
      }
    }
```

3．后序遍历

后序遍历的递归定义和算法为：

（1）按后序遍历的方法遍历根节点的左子树；

（2）按后序遍历的方法遍历根节点的右子树；

（3）访问根节点。

【算法 21】　二叉树的后序遍历算法：

```
void postorder（bnode *BT）
   { if（BT==NULL）
        return;
     else
     { if（BT－>LC!=NULL）postorder（BT－>LC）;
       if（BT－>RC!=NULL）postorder（BT－>RC）;
       visit（BT）;        /* 访问根节点 */
     }
    }
```

例如，对于如图 1.35 所示的二叉树，按先序遍历方法遍历的次序为：

ABDGECFH

按中序遍历方法遍历的次序为：

DGBEAFHC

按后序遍历方法遍历的次序为：

GDEBHFCA

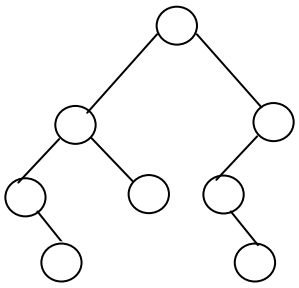


图 1.35　一个二叉树示例

四、根据遍历序列构造二叉树

由一棵给定的二叉树可以获得三种遍历序列，同样，也可以由这些遍历序列来重新构造二叉树。单用一个遍历序列是无法构造二叉树的，因为无法从遍历序列中区分二叉树的左、右子树。但利用中序遍历序列，并结合先序遍历序列和后序遍历序列中任何一个序列就能重新构造二叉树。

根据二叉树节点遍历操作定义，先序遍历序列和中序遍历序列的节点分布特点是：

先序遍历序列：〈根〉〈左子树上所有节点〉〈右子树上所有节点〉

中序遍历序列：〈左子树上所有节点〉〈根〉〈右子树上所有节点〉

可以得到二叉树的构造步骤：

第一步：从先序遍历序列中取出第一个节点，该节点一定是二叉树的根。然后在中序遍历序列中找出根节点，根节点前面的节点序列就是左子树的中序遍历序列，根节点后面的节点序列就是右子树的中序遍历序列。

第二步：对根的左子树先序遍历序列和中序遍历序列及右子树的先序遍历序列和中序

遍历序列。再执行第一步，直到得出所有叶子节点为止。

例 6 已知一棵二叉树的先序遍历序列为 ABCDEFGHIJ。中序遍历序列为 CBDEAFHIGJ，试构造这棵二叉树。构造过程如图 1.36 所示。

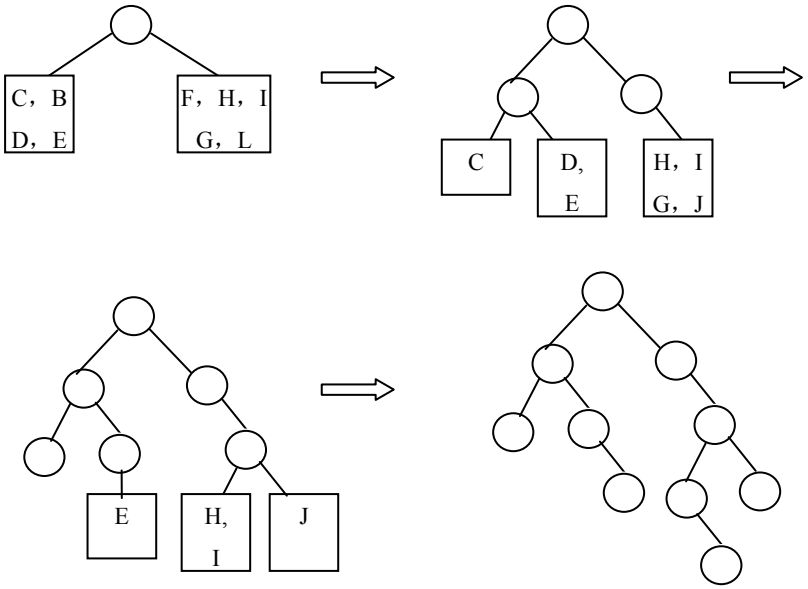


图 1.36 重构二叉树的过程

通过同一棵二叉树的后序和中序序列也能构造出唯一的一棵二叉树，其构造步骤同利用先序和中序遍历序列构造二叉树的步骤相类似，这里不再重述。

五、二叉排序树的生成

前面已给出了二叉排序树的定义，从二叉排序树的定义可以得出二叉排序树的一个重要性质：按中序遍历该树所得的中序序列是一个递增有序列，因此，二叉排序树常用于对数据排序。利用二叉排序树来组织数据，可以减少数据查找次数，提高效率。

由给定的数据序列生成二叉排序树的过程是在二叉排序树上插入节点的过程。对一序列 $\{k_1, k_2, \cdots, k_n\}$，先设一棵空二叉排序树，然后将序列中的元素顺次生成节点后逐个插入。插入步骤如下：

第一步：k_1作为二叉排序树的根。

第二步：若$k_2 < k_1$，则k_2所在节点应插入到k_1的左子树上；否则，插入到k_1的右子树上。

第三步：读入k_i，若$k_i < k_1$（根），则进入左子树，否则进入右子树，继续与子树之根比较，直到某节点k_j，若有$k_i < k_j$且节点k_j的左子树为空，则节点k_i插入到节点k_j的左子树；若有$k_i \geqslant k_j$且节点k_j的右子树为空，则节点k_i插入到节点k_j的右子树。

【算法 22】 设有 n 个无序的数据元素序列存放在一维数组 A［1. . n］中，二叉排序树采用二叉链表存储结构，根节点指针为 proot。生成二叉排序树的算法如下：

```
void Creat_Binary_Sort_tree （bnode ** proot）
```

```
{ bnode *p，*q；
  elemtype k；
  int i，n；
  *proot=NULL；          /* 初始化空树 */
  printf（"input n:"）；  /* 输入数据元素个数 */
  scanf（"%d"，&n）；
  for（i=0；i<n；i++）
    { p=（bnode *）malloc（sizeof（bnode））；
      p－>LC=NULL；
      p－>RC=NULL；
      printf（"input k:"）；
      scanf（"%d"，&k）；
      if（*proot==NULL）
          *proot=p；
      else
        {q=*proot；
          while（q!=NULL）
            { if（q－>data>k）
                if（q－>LC!=NULL）       /* 若左子树不为空，继续沿 LC 链向下查找 */
                    q=q－>LC；
                else
                    { q－>LC=P；
                      q=NULL；      /* 插入左子树 */
                    }
              else
                 if （q－>RC!=NULL）    /* 若右子树不为空，继续沿 LC 链向下查找 */
                     q=q－>RC；
                 else
                     { q－>RC=P；
                       q=NULL；
                     }
             } /* end while */
        }
    } /* end for */
} /* end function */
```

例 7　有 15 个数构成的序列｛190，381，12，40，410，394，540，760，85，476，800，146，9，445，600｝，根据算法的基本思想构造一个二叉排序树。

生成过程如图 1.37 所示。

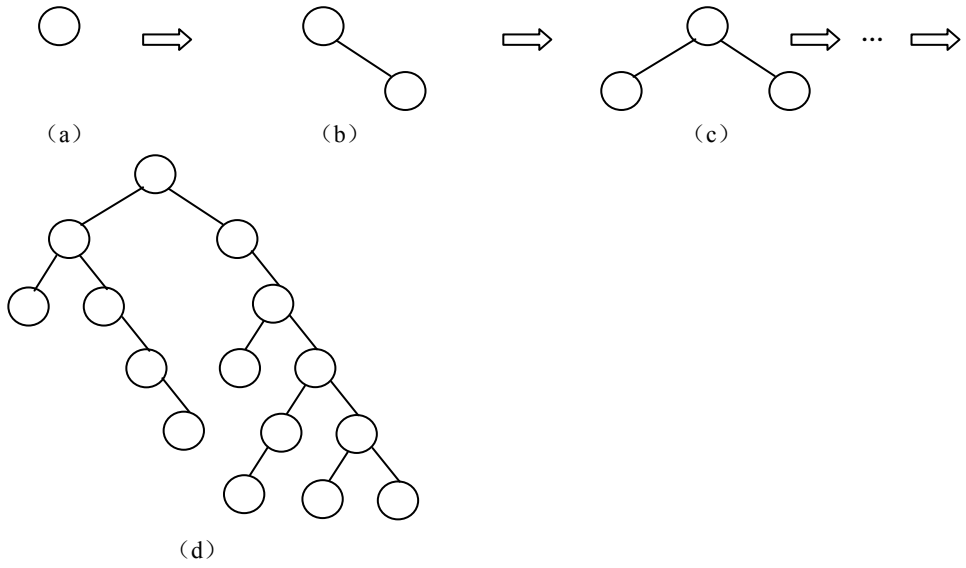


图 1.37　二叉排序树的构造过程

六、树、森林与二叉树的转换

由于一般树中各个节点的度可能相差很多（即节点的子树个数不规则），若用链表结构表示时无法确定指针域的数目。若每个节点采用最大指针域个数，则浪费存储空间；若按每个节点的实际需要分配指针域又增加了编程的困难。如果能把树或森林转换为二叉树，不但可以采用二叉树的存储结构，而且可以利用二叉树的有关算法来实现树的有关操作。

1．把一棵树转换成对应的二叉树

转换的基本思想是：树所对应的二叉树里，一个节点的左子节点是它在树中的第一个子节点（最左边的一个子节点），取树中此节点的兄弟作为前一个兄弟节点的右子节点。下面是将树转换成二叉树的步骤：

（1）在兄弟节点之间加一连线；

（2）对每一个节点，只保留它与第一个子节点的连线，与其他子节点的连线全部抹掉；

（3）以树根为轴心，顺时针旋转 45°。

使用上述方法，如图 1.38（a）所示的树变为如图 1.38（c）的形式，此时已是一棵二叉树；再按顺时针方向将它旋转约 45°就更清楚地变成如图 1.38（d）所示的二叉树。利用这种方法将树换成二叉树后，树根没有兄弟，故转化为二叉树后的根节点的右子树必为空。

2．把森林转换成对应的二叉树

森林是树的有限集合，利用树的转换思想，可以实现森林的转换，其基本方法是：先将森林中每一棵树变为二叉树，然后将各二叉树的根节点看做兄弟连在一起。图 1.39 说明了把森林转换成对应的二叉树的过程：图（a）是森林，图（b）是将每棵树转换成对应的二叉树的结果，图（c）是将森林中每棵树的根节点相连，构成一棵二叉树，图（d）是将图（c）旋转后的二叉树。

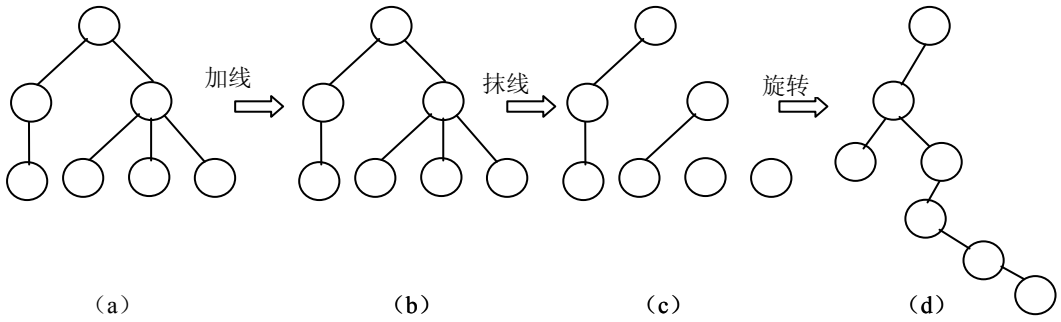


图 1.38　树转换成对应的二叉树过程示意

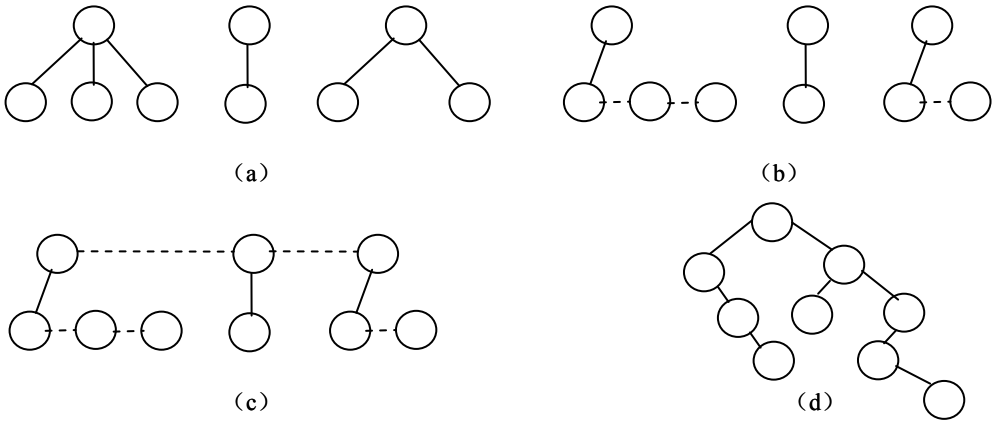


图 1.39　森林转换成相应的二叉树过程示意

七、二叉树的应用举例——最优二叉树与霍夫曼编码

假设在某数据通信和数据传输系统中，用到 n 个字符。通过大量实验的统计得知这些字符在通信传输中出现的频率不同。现在的问题是如何对这些通信字符集设计一套二进制编码，使得在采用这种编码进行通信时，总的信息传输量最省。为了在字符之间省去不必要的间隔符号，希望所给的每个字符的编码在系统识别时可以唯一确定，即任一字符的二进制码不可能解释为其他字符编码的前缀（否则译码时会出现判断上的二义性）。霍夫曼（Huffman）为解决这类问题给出了著名的霍夫曼算法，通过建立最优二叉树 —— 一类带权路径长度最短的二叉树来进行字符编码。

下面首先通过一个简单的例子来说明最优二叉树的形式和构造方法。

已知一个数据通信系统中使用的字符为 a，b，c，d，e，f，g。传输信息中它们出现的频率为：每传输 1 千字中 a，b，c，d，e，f，g 平均分别出现 115，11，14，35，516，254，55 次。霍夫曼算法首先把每个字符看成是一个只有一个节点的二叉树（见图 1.40(a)），把它们的频率看成各个根节点的权，或者简单地说就是这颗二叉树的权；然后从这 7 个二叉树中选出两个权最小的二叉树，用它们分别作为一棵新二叉树的左、右子树，新二叉树的权定义为两个子树的权的和，从而得到 6 个带权的二叉树（见图 1.40(b)），重复这个过程直到最后只剩下一棵二叉树为止（见图 1.40(c)）。

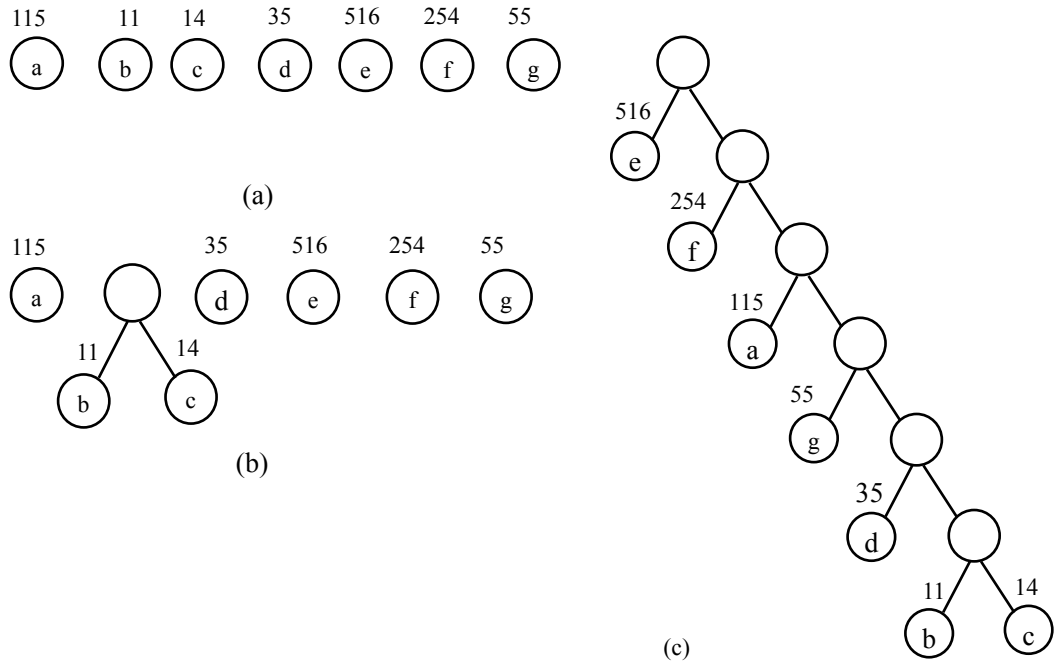


图1.40　最优二叉树的构造

霍夫曼算法得到的二叉树总是满二叉树，7 个叶子的霍夫曼树要经过 6 次合并，产生 6 个新节点，共有 13 个节点。如果把二叉树中每条到左子树的边记为 0，到右子树的边记为 1，那么从根到每个叶节点的路径就可以用一个 0、1 组成的串识别，霍夫曼证明了用这个二进制的串作为对应叶节点字符的编码，就是能满足总的信息传输量最省的一种编码。具体对这个例子而言，所得到的编码为：

A—110　　b—111110　　c—111111　　d—11110

e—0　　f—10　　g—1110

霍夫曼编码是一种被广泛应用而且非常有效的数据压缩技术，根据待压缩数据的特征，一般可压缩掉 20%~90%。比如上面例子中，传统的对 a，b，c，d，e，f，g 共 7 个字符进行二进制编码，每个字符需要 3 比特。传输 1 千字，共需要：

3×1000＝3000 比特

经过霍夫曼编码后，同样传输 1 千字，只需传输：

3×115+6×11+6×14+5×35+1×516+2×254+4×55=1914 比特

数据压缩比为：(3000−1914) / 3000 =36.2%。

霍夫曼算法得到的二叉树给出了一种能保证信息通信时最少的编码量，也引入了最优二叉树的概念。下面就给出最优二叉树（又称为霍夫曼树）的基本概念，以及构造最优二叉树（也就是霍夫曼编码）的具体编程实现。

（1）叶子节点的权值：叶子节点的权值是对叶子节点赋予的一个有意义的数值量。在霍夫曼编码应用中，叶节点代表的字符出现的频率就是该节点的权值。

（2）节点的带权路径长度：节点的带权路径长度为从该节点到树根之间的路径长度与该节点权的乘积。

（3）树的带权路径长度（WPL）：树的带权路径长度为树中所有叶子节点的带权路径长度之和。

（4）最优二叉树：是指所有叶子节点的二叉树中带权路径长度最小的二叉树。

霍夫曼编码过程就是构建最优二叉树的过程。具体编程实现是，首先定义一个大小为2n−1 的数组来存储最优二叉树中的节点，其存储结构为：

```
typedef struct
  { float weight;      /* 权值，不妨设权值均大于零*/
    int lchild，rchild，parent;  /* 左右孩子及双亲指针（数组下标）*/
  } HTNode;
HTNode T[MAXNUM];  /* T 时 HuffmanTree 是节点向量数组 */
```

因为 C 语言数组的下界为 0，故用−1 表示空指针。树中某节点的 lchild、rchild 和 parent 不等于−1 时，它们分别是该节点的左、右孩子和双亲节点在向量中的下标。这里设置 parent 域有两个作用：其一是使查找某节点的双亲变得简单；其二是可通过判定 parent 的值是否为−1 来区分根与非根节点。

在上述存储结构上实现的霍夫曼算法可大致描述为：

第一步：初始化，将要构成的最优二叉树 T[0. . m−1]中 2n−1 个节点里的三个指针均置为空(即置为−1)，权值置为 0。

第二步：输入，读入 n 个叶子的权值存于向量的前 n 个分量（即 T[0. . n−1]）中。它们是初始森林中 n 个孤立的根节点上的权值。

第三步：合并，对森林中的树共进行 n−1 次合并，所产生的新节点依次放入向量 T 的第 i 个分量中(n≤i≤2n−2)，总共有 2n−1 个节点。下面是具体的程序：

```
void CreateHuffmanTree (T)
{   int i，p1，p2；
    InitHuffmanTree(T)；  /* 将 T 初始化 */
    Input Weight(T)；      /* 输入叶子权值至 T[0. . n−1]的 weight 域 */
    for ( i=n；i<2n−1；i++ )
        {   /* 共进行 n−1 次合并，新节点依次存于 T[i]中 */
            /* 在 T[0…i−1]中选择两个权值最小的根节点，设序号为 p1 和 p2 */
            SelectMin(T，i−1，&p1，&p2)；
            T[p1].parent = T[p2].parent = i；
            TIi].1child=p1；  /* 最小权值的根节点是新节点的左孩子 */
            T[j].rchild=p2；  /*次小权值的根节点是新节点的右孩子 */
            T[i].weight=T[p1].weight+T[p2].weight；
        }
}
```

1.3.3 图

图是另一种重要的、比树更复杂的非线性数据结构。在树中，每个节点只与上层的父节点有联系，并可以与其下层的多个子节点有联系，而同一层的节点之间没有任何横向联

系。但在图中，节点之间的联系是任意的，每个节点都可以与其他的节点相联系。

图的应用范围非常广泛，诸如电网络分析、交通、管道线路、集成电路布线图、工程进度安排等实际问题的处理都可以归纳为图的问题。

一、图的基本概念

图 G 由两个集合 V 和 E 组成，记为 G=（V，E），其中 V 是顶点的有穷非空集合，E 是 V 中顶点偶对（称为边）的有穷集。通常也将图 G 的顶点集和边集分别记为 V（G）和 E（G）。E（G）可以是空集，若 E（G）为空集，则图 G 只有顶点而没有边。

若图G中的每条边都是有方向的，则称G为有向图。在有向图中，一条有向边是由两个顶点组成的有序对，通常用尖括号表示。例如〈V_i，V_j〉表示一条有向边，V_i是边的始点，V_j是边的终点。因此，〈V_i，V_j〉和〈V_j，V_i〉是两条不同的有向边。有向边也称为弧，边的始点称为弧尾，终点称为弧头。若图G的每条边都是没有方向的，则称G为无向图。无向图中的边均是顶点的无序对，用圆括号表示。因此无序对（V_i，V_j）和（V_j，V_i）是同一条边。

如图 1.41 所示是有关图的两个例子G_1和G_2。G_1为无向图，G_2为有向图。



（a）无向图G_1　　（b）有向图G_2

图 1.41　图的表示形式

集合表示形式：

G_1=（V，E）

V（G_1）=｛1，2，3，4，5｝

E（G_1）=｛(1，2)，(1，3)，(2，3)，(3，4)，(4，5)｝

G_2=（V，E）

V（G_2）=｛1，2，3，4，5，6｝

E（G_2）=｛〈1，2〉，〈2，1〉，〈2，3〉，〈2，4〉，〈3，5〉，〈5，6〉，〈6，3〉｝

可见，按照习惯说法，图是一种对节点的前趋和后继个数不加限制的数据结构。

下面介绍一些有关图结构的重要术语和概念。

（1）邻接点：对于无向图，如果边（v，u）∈E，则 v 和 u 互为邻接点，亦即 u 是 v 的邻接点，v 也是 u 的邻接点。

对于有向图，如果弧〈v，u〉∈E，则 u 是 v 的邻接点。

（2）顶点的度：常用 D（V）表示，在无向图中，顶点的度就是以该顶点为一个端点的边的条数。

在有向图中，以某顶点为弧头的弧的数目，称为此顶点的入度，常用 ID（V）表示；以某顶点为弧尾的弧的数目，称为此顶点的出度，常用 OD（V）表示。有向图顶点的度是此顶点的入度与出度之和，即 D（V）=ID（V）+OD（V）。

无论是有向图，还是无向图，顶点数 n 、边数 e 和度数之间有如下关系：

$$e=\frac{1}{2}\sum_{i=1}^{n}D(V_i)$$

（3）路径：在图G中，从顶点v至顶点u的一条路径是顶点的序列（v，v_1，v_2，…，v_i，u），并且（v，v_1），（v_1，v_2），…，（v_i，u）（无向图）或〈v，v_1〉，〈v_1，v_2〉，…，〈v_i，u〉（有向图）都属于集合E。路径上弧的数目称为该路径的长度。

（4）在无向图中，若每一对顶点之间都有路径，则称此图为连通图。

在有向图中，若每一对顶点 u 和 v 之间都存在从 v 到 u 及从 u 到 v 的路径，则称此图为强连通图。

（5）网络：如果图 G（V，E）中每条边都赋有反映这条边的某种特性的数据，则称此图是一个网络，其中与边相关的数据称为该边的权。

二、图的存储结构

图的存储结构有多种表示法，本节讨论两种最常用的表示方法：邻接矩阵表示法和邻接表表示法。

1．邻接矩阵

根据图的定义可知，一个图的逻辑结构的说明分两部分：第一部分是组成图的顶点集合 V；第二部分是顶点之间的联系，即顶点偶对集合 E。因此，对一个图的计算机表示只要分别解决集合 V 和集合 E 的存储表示即可。

集合 V 中的所有顶点可以利用一个一维数组表示；而集合 E 用一个二维数组来表示。此二维数组称其为邻接矩阵。邻接矩阵反映了图中各顶点之间的相邻关系。设 G＝（V，E）是有 n（n≥ 1 ）个顶点的有向图，则 G 的邻接矩阵是具有如下性质的 n×n 矩阵：

$$A[i,j]=\begin{cases}1 & 若\langle V_i,V_j\rangle\in E\\ 0 & 若\langle V_i,V_j\rangle\notin E\end{cases}$$

对于 G＝（V，E）是有 n（n≥ 1 ）个顶点的无向图，则 G 的邻接矩阵是这样一个 n×n 矩阵：

$$A[i,j]=A[j,i]=\begin{cases}1 & 若\langle V_i,V_j\rangle\in E\\ 0 & 若\langle V_i,V_j\rangle\notin E\end{cases}$$

以图 1.41 中的图G_1和图G_2为例，其邻接矩阵分别为

$$A_1=\begin{bmatrix}0&1&1&0&0\\1&0&1&0&0\\1&1&0&1&0\\0&0&1&0&1\\0&0&0&1&0\end{bmatrix}\qquad A_2=\begin{bmatrix}0&1&0&0&0&0\\1&0&1&1&0&0\\0&0&0&0&1&0\\0&0&0&0&0&0\\0&0&0&0&1&0\\0&0&1&0&0&0\end{bmatrix}$$

借助于邻接矩阵，可以很容易的求出图中顶点的度，因而有下述结论：

（1）对于无向图，邻接矩阵第 i 行（或第 i 列）的元素之和则是顶点V_i的度；

（2）对于有向图，邻接矩阵第 i 行的元素之和为顶点V_i的出度；邻接矩阵第 i 列的元素之和为顶点V_i的入度。

网络的邻接矩阵，只要相应的有向图或无向图的邻接矩阵的元素A［i，j］＝1 改为从顶点V_i到V_j的边上的权值w_{ij}，元素A［i，j］＝0 改为∞即可。

图的邻接矩阵存储结构的 C 语言描述形式如下：

```
# define MAXNODE ×××          / * 其中×××为图中顶点的最大个数 * /
typedef struct graph_type
    { nodetype nodes［MAXNODE］;        / * nodetype 是顶点的数据类型 * /
      int arcs ［MAXNODE］［MAXNODE］;
    } graph;
```

graph 为图的邻接矩阵存储结构，其中一维数组 nodes 用来表示与顶点有关的信息，二维数组 arcs 用来表示图中顶点之间的关系。

采用邻接矩阵存储结构，很容易实现图的基本操作。下面给出在图中增加一条边操作的算法：

【算法 23】　在图 G 中增加一条从顶点 V 到顶点 W 的边：

```
void ins_arc（graph *g，int V，int W）
    { g－>arcs［V］［W］＝1；
      return；
    }
```

2．邻接表

邻接矩阵的邻接存储方法实际上是图的一种静态存储方法，建立这种存储结构时需要预先知道图中顶点的个数。如果图结构本身需要在解决问题的过程中动态地产生，则每增加或删除一个顶点都需要改变邻接矩阵的大小，显然，这样做的效率是很低的。除此之外，邻接矩阵占用的存储单元数目只与图的顶点个数有关，而与弧的数量无关，若图的邻接矩阵是一个稀疏矩阵，必然会造成存储空间的大量浪费。

邻接表就是图的一种链式存储结构。在邻接表中，对图中每个顶点建立一个单链表，第 i 个单链表中的节点包含顶点 i 的所有邻接顶点。这样，邻接表中，每个节点由三个域组成（如图 1.42 所示）。

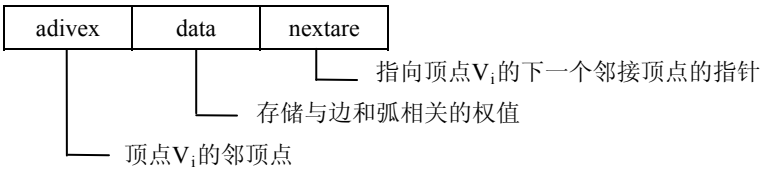


图 1.42　邻接表的表节点（邻接节点）结构

为了便于邻接表操作，在每个单链表上附设一个头节点，在头节点中，除了一个指向顶点V_i的第一个邻接顶点的指针域外，另外还设一个存储顶点V_i有关信息的数据域。头节点结构如图 1.43 所示。

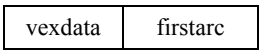


图 1.43　邻接表的头节点结构

为了利用顺序存储结构的随机访问特性，邻接表中将每个单链表的头节点以顺序结构的形式存储，以便能随机访问任一顶点的单链表。

综上所述，邻接表的存储结构可以用 C 语言描述如下：

```
# define MAXNODE ×××              / * 其中×××为图中顶点的最大个数 * /
typedef struct st_arc
    { int adivex;
      weighttype date;             / * weight_type 为弧的权值类型 * /
      struct st_arc * nextarc;
    }starc
typedef struct head_node
    { nodetype vexdata;            / *nodetype 为图顶点数据类型 * /
      struct st_arc * firstarc;
    } headnode;
typedef headnode adjlist［MAXNODE］;
```

图 1.44 说明了图 1.41 中图G_2的一个邻接表存储结构。

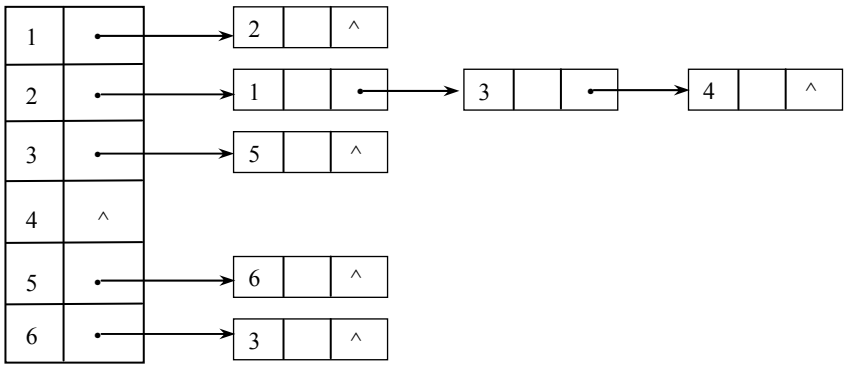


图 1.44　一个图的邻接表表示

显然，当图中顶点数很多而弧数较少时，采用邻接表存储结构可以节省存储空间。从邻接表也可以很容易确定图中顶点的度。对于无向图，图中第 i 个顶点的度等于邻接表中第 i 条链表中邻接节点的个数；对于有向图，图中第 i 个顶点的出度等于邻接表中第 i 条链表中邻接节点的个数，但入度则需扫描整个邻接表，统计各条链中各个邻接节点的 adivex 域出现 i 值的次数。

三、图的遍历

和树的遍历类似，图的遍历也是从某个顶点出发，沿着某条搜索路径对图中所有顶点各作一次访问。若给定的图是连通图，则从图中任一顶点出发顺着边可以访问到该图中所有的顶点。然而，图的遍历比树的遍历复杂得多，这是因为图中的任一顶点都可能和其他

点相邻接，故在访问了某个顶点之后，可能顺着某条回路又回到了该顶点。为了避免重复访问同一个顶点，必须记住每个顶点是否被访问过。为此，可设置一个标志向量 visited［1．．n］，以便说明哪些顶点被访问了。根据搜索路径的方向不同，有两种常用的遍历图的方法：深度优先搜索遍历和广度优先搜索遍历。

1．深度优先搜索遍历

深度优先搜索（Depth-First Search）遍历类似于树的前序遍历。假设给定图G的初态是所有顶点均未曾访问过，在G中任选一顶点 v_i 为初始出发点，则深度优先搜索可如下进行：首先访问出发点 v_i，并将其标记为已访问过，然后依次从 v_i 出发搜索 v_i 的每一个邻接点 v_j，若 v_j 未曾访问过，则以 v_j 为新的出发点继续进行深度优先搜索。

显然上述搜索法是递归定义的，它的特点是尽可能先对纵深方向进行搜索，故称之为深度优先搜索。例如，设 x 是刚被访问过的顶点，按深度优先搜索方法，下一步将选择一条从 x 出发的未检测过的边（ x ， y ）。若发现顶点 y 已被访问过，则重新选择另一条从 x 出发的未检测过的边。若发现顶点 y 未曾访问过，则沿此边从 x 到达 y ，访问 y 并将其标记为已访问过，然后从 y 开始搜索。直到搜索完从 y 出发的所有路径，才回溯到顶点 x ，然后再选择一条从 x 出发的未检测过的边，上述过程直至从 x 出发的所有边都已检测过为止。此时，若 x 不是初始出发点，则回溯到在 x 之前被访问过的顶点；若 x 是初始出发点，则整个搜索过程结束。显然这时图 G 中所有和初始出发点有路径相通的顶点都已被访问过。因此，若 G 是连通图，则从初始出发点开始的搜索过程结束也就意味完成了对图 G 的遍历。

因为深度优先搜索是递归定义的，故很容易写出其递归算法。在该算法中，标志数组 visited（用于标志图中的顶点是否已被访问）规定如下：

$$\text{visited}[i]=\begin{cases}0 & \text{图中的第i个顶点未被访问过}\\ 1 & \text{图中的第i个顶点已被访问过}\end{cases}$$

【算法 24】　深度优先搜索遍历的算法：

```
void dfe（adjlist G，int v0）
/* 从顶点v0出发，深度优先搜索图G */
{ int w，visited［MAXNODE］;
  visited［v0］＝1；
  printf（"%d＼n"，v0）;
  w=FIRST_VERTEX（G，v0）;
  while（w!=0）
  { if（visited［w］==0）
      {dfs（G，w）；}
    w=NEXT_VERTEX（G，v0，w）;
  }
}
```

在dfs（G，v_0）函数中，用到了两个函数来完成图的基本操作：求第一个邻接点函数FIRST_VERTEX（G，v_0）和求下一个邻接点函数NEXT_VERTEX（G，v_0，W），当图采用不同的存储结构时，这两个操作的实现算法是不同的。

对于如图 1.45 所示的无向图及其邻接表，从顶点 V_1 出发，采用深度优先搜索算法所得

的遍历结果序列为：

$$V_1，V_2，V_4，V_5，V_6，V_3$$

2．广度优先搜索遍历

广度优先搜索（Breadth-First Search）遍历类似于树的按层次遍历。设图G的初态是所有顶点均未访问过，在G中任选一顶点 v_i 为初始出发点，则广度优先搜索的基本思想是：首先访问出发点 v_i，接着依次访问 v_i 的所有邻接点 w_1，w_2，…，w_t，然后再依次访问与 w_1，w_2，…，w_t 邻接的所有未曾访问过的顶点，以此类推，直至图中所有和初始出发点 v_i 有路径相通的顶点都已访问到为止。此时从 v_i 开始的搜索过程结束，若G是连通图则遍历完成。

显然，上述搜索法的特点是尽可能先对横向进行搜索，故称之为广度优先搜索。设 x 和 y 是两个相继被访问过的顶点，若当前是以 x 为出发点进行搜索，则在访问 x 的所有未曾访问过的邻接点之后，紧接着是以 y 为出发点进行横向搜索，并对搜索到的 y 的邻接点中尚未被访问的顶点进行访问。也就是说，先访问的顶点其邻接点亦先被访问。为此，需引进队列保存已访问过的顶点。下面给出当图采用邻接表存储结构时的广度优先搜索算法。

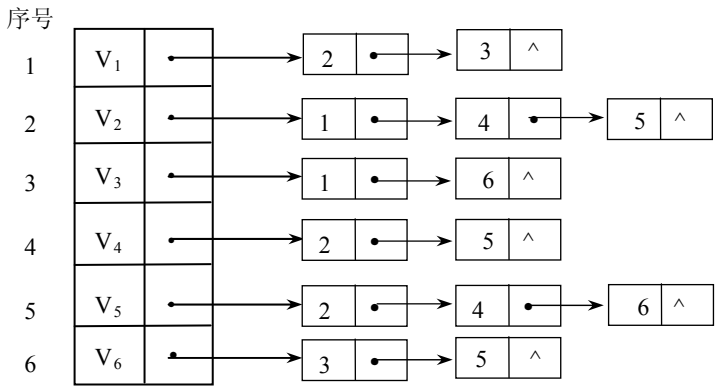


图 1.45　无向图及其邻接表

（a）无向图　（b）邻接表

【算法 25】　广度优先搜索一个采用邻接表方式存储的图 G：

```
void bfs（ADJLIST g，int k，int visited［］）
/* 当图采用邻接表存储结构时，从顶点 k 出发，广度优先搜索图 G */
{ arcnode *p；
  int w；
  visited［k］=1      /* 访问顶点 k，并将它入队 */
  printf（"%d＼n"，k）；
  enter（g_queue，k）；
  while（elmtype（g_queue）==0）          /* 当队列不为空时，取出队头元素，并访问队头元素的所
                                             有邻接点 */
  {
```

```
    w=delete（g_queue）;
    p=g［w］.firstarc;
    while（p!=NULL）
      { if （visited［p－>adjvex］==0）
          { printf（"%d"，p－>adjvex）;
            visited［p→adjvex］=1;
            enter（g_queue，p－>adjvex）;          /* 将访问的顶点入队 */
          }
        p=p－>nextarc;
      }
    }
}
```

采用广度优先搜索方法对图 1.45 进行遍历，所得的节点序列为：

$$V_1，V_2，V_3，V_4，V_5，V_6$$

值得注意的是，不论采用深度优先搜索法还是广度优先搜索法进行图的遍历，如果选定的出发点不同或者是所建立的存储结构不一致，则可能得到不同的遍历结果。只有当选取的出发点、采用的存储结构以及遍历方式都是确定的，遍历的结果才是唯一的。此外，若一个图是非连通图，则从图中任意一个顶点出发进行深度优先搜索或广度优先搜索都不能访问到图中所有顶点，而只能访问到初始出发点所在的连通分量中的所有顶点。若从每个连通分量中都选一个顶点作为出发点进行搜索，便可访问到整个非连通图中所有顶点。因此非连通图的遍历必须多次调用深度优先搜索或广度优先搜索算法。

小　　结

本章讨论的是非线性数据结构，在这类结构中至少存在一个数据元素，它具有两个或两个以上的直接后继或直接前趋。树、二叉树和图是非线性数据结构中最典型的结构形式。树型结构用于描述数据元素之间的层次关系，如人类社会的族谱和各社会组织机构的表示等。由于树形结构本身的固有特性，树形结构都可以递归定义。树结构的定义是：树是由一个或多个节点组成的有限集合T，且满足条件：有一个没有直接前趋的特定节点，称为根节点；其余节点分成m个互不相交的有限集T_1，T_2，T_3，…，T_m，其中每个集又都是一棵树，这些树称为根节点的子树。

树结构中有一系列重要的术语，它们是叶子、分支节点、节点的度、树的度、树的深度、节点的层次、森林、子节点、父节点、兄弟。

二叉树是另一种非线性结构，它与一般树概念不同，它可以是为空二叉树；并且二叉树最多只有两个也是二叉树的子树，被称为左、右二叉子树。满二叉树、完全二叉树是二叉树的两个特殊情形。利用满二叉树、完全二叉树从上到下、从左到右的编号可以推知某一节点的父节点，左、右子节点，兄弟节点的位置。因此，它们可以用顺序存储结构方便地存储。对于一般的二叉树可以采用二叉链表的存储结构来存储。二叉排序树是二叉树又一个特殊情形。利用二叉排序树可以对一组无序的数据排序，利用二叉排序树来组织数据，可以减少数据查找时间。

对二叉树的遍历方法有三类：先序遍历、中序遍历和后序遍历，遍历结果为先序遍历序列、中序遍历序列和后序遍历序列。利用中序遍历序列，再加先序遍历序列或后序遍历序列可重构二叉树。

图是更复杂的一种非线性结构，由两个集合 V 和 E 组成，其中 V 是顶点的有穷非空集合，E 是 V 顶点偶对的有穷集。图分为有向图和无向图两种，图中涉及的一些重要术语包括：邻接点、顶点的度、路径、路径长度、网络、边的权等。

图的存储结构有多种形式，最常用的有邻接矩阵和邻接表。图的顶点数很大，而边数很少时，采用邻接矩阵会造成存储空间的浪费。

图的遍历方法一般有两种：深度优先遍历和广度优先遍历。对于连通图，这两种方法都能一次遍历图的所有节点，但遍历的节点序列次序可能不同。对于非连通图，则需要多次调用深度优先遍历或广度优先遍历算法才可能遍历完图的全部节点。

图涉及许多算法，如 Kruskar_Prim的最小生成树算法，Dijkstra_Floyd 的最短路径算法，AOV 网的拓扑排序算法，AOE 网的关键路径算法等，受篇幅限制，本节未加以讨论，感兴趣的读者可进一步阅读有关数据结构的书籍。

1.4 查找与排序

查找和排序是非常重要的运算。查找的功能是从大量数据中寻找出所需的数据来，而排序是将一组记录按其关键字的递增或递减的次序排列，以便进行数据查找、数据处理。

1.4.1 查找

要查找某一个数据或记录，必须依据预先确定的某种标志，这就是关键字的概念。通常，关键字是指数据元素中可以唯一标识一个数据元素的数据项。如学生的学号、居民身份证号码等。查找时，根据给定的关键字值，在一组数据元素中确定一个其关键字值等于给定值的数据元素。若存在这样的数据元素，则称查找是成功的；否则称查找失败。一组待查数据元素的集合又称为查找表。

显然，查找运算依赖于查找表中各数据元素在计算机存储系统的组织与存储结构。按照数据元素的组织与存储方式决定了采用什么样的查找方法；反过来，为了提高查找方法的效率，又要求数据元素采用某些特殊的存储方式。下面就介绍几种常用的查找方法，它们各有特点，能满足不同要求的应用场合。

一、简单查找方法

1．顺序查找

顺序查找是最普通也是最简单的查找技术。其基本思想是：从第一个数据元素开始，逐个把数据元素的关键字值和给定值比较，若某个元素的关键字值和给定值相等，则查找成功；否则，若直至第 n 个数据元素都不相等，说明不存在满足条件的数据元素，查找失败。

顺序查找方法既适用于以顺序存储结构组织的查找表的查找，也适用于以链式存储结构组织的查找表的查找。下面以顺序表结构为例说明顺序查找的算法。

【算法 26】 顺序查找的算法：

查找表的各数据元素存储在顺序表 S 中，S 的类型说明如下：

elemtype S［MAXNUM］;

其中，elemtype 为数据元素的数据类型；MAXNUM 为可能有的数据元素的最大个数。

```
int seq_search（elemtype S［］，keytype k，int n）
  { int i;
    S［n］. key= k ;            /* 设置监视哨 */
    i=0;
    while （S［i］. key!=k）
        i++;
    if（i<n）
        { printf（"searching success"）;
          return（i）;
        }
    else
        { printf（"searching failed"）;
          return（－1）;
        }
  }
```

在算法中 n 个待查数据元素放在顺序表 S［0］~S［n－1］中；在其后增设了一监视哨 S［n］，并使 S［n］. key 的值等于 k，这样能使判断查找是否结束变得简单容易。由于 while 语句执行频率最高，增设监视哨可提高 while 语句的执行速度，从而加快查找速度。

衡量查找运算执行效率的标准是平均查找长度。对于含有 n 个数据元素的查找表，查找成功时的平均查找长度为：

$$ASL = \sum_{i=1}^{m} P_i C_i$$

其中，P_i为查找第 i 个数据元素的概率；C_i为查到第i个数据元素时，需进行的比较次数。

对于顺序查找算法，假设查找表中每个数据元素的查找概率相同，即$P_i = 1/n$（i＝1，2，…，n）；查找表中第i个数据元素所需进行的比较次数为$C_i = i$。故顺序查找运算查找成功的平均查找长度为：

$$ASL_{Sq} = \sum_{i=1}^{n} P_i C_i = \sum_{i=1}^{n} (\frac{1}{n} \cdot i) = \frac{1}{2}(n+1)$$

即成功查找的平均比较次数约为线性表长度的一半，算法效率为 O[n]。当要查找的元素不在查找表中，则需进行 n＋1 次比较后才确定查找失败。因此，这种查找算法效率不高，仅对短表合适。

2．二分查找

如果按顺序存储结构组织的查找表中的所有数据元素按关键字有序，则可以采用一种高效率的查找方法——二分查找。其基本思路为：由于查找表中的数据元素按关键字有序（假设递增有序），则在查找时可不必逐个顺序比较，而采用跳跃的方式——先与“中间位置”的数据元素的关键字比较，若相等，则查找成功；若给定值大于“中间位置”的关键字，则在查找表的后半部继续进行二分查找，否则在前半部进行二分查找。

二分查找的过程是：先确定待查元素所在区域，然后逐步缩小区域，直到查找成功或失败为止。

设：待查元素所在区域的下界为 low，上界为 hig，则中间位置 mid=（low+hig）/2。

① 若此元素关键字值等于给定值，则查找成功；

② 若此元素关键字值大于给定值，则在区域 mid+1～hig 内进行二分查找；

③ 若此元素关键字值小于给定值，则在区域 low～mid－1 内进行二分查找。

由于二分查找要求数据元素的组织方式应具有随机存取的特性，所以二分查找只适用于以顺序存储结构组织的有序查找表。

【算法 27】　二分查找的算法：

```
int bin_search（elemtype s［］，keytype k，int n）
   { int low，high，mid；
     low=0；
     high=n－1；
     while（low<=high）
     { mid=（low+high）/2；
       if（s［mid］.key==k）
          { printf（"searching success"）；
            return（mid）；
          }
       else if（s［mid］.key<k）
                low=mid+1；
          else
                high=mid－1；
     }          / * while 循环结束 * /
     printf（"searching failed"）；
     return（－1）；
   }
```

二分查找成功的平均查找长度ASL≈$\log_2$（n＋1）－1，算法效率为O[log n]。

二分查找的优点是比较次数少，查找速度快。对于 1000 个数据元素组成的查找表，采用顺序查找，平均查找长度为 500，而二分查找平均查找长度为 9。但是，二分查找所付出的代价是要对数据元素按关键字的大小进行排序，这是很费时的。因此，二分查找适合于数表一经建立就很少变动、可事先进行排序而又经常需要查找的场合。

3．分块查找

分块查找又称索引顺序查找，是介于顺序查找和二分查找之间的一种查找方法。它的基本思想是：

（1）将所有数据元素分成若干块，块内元素按关键字是无序的，但块间按关键字是有序的。即第一块中所有元素大于（或小于）第二块中所有元素；第二块中所有元素均大于（或小于）第三块中所有元素；以此类推。

（2）建立一个块的最大（或最小）索引关键字表，该表是有序的。

（3）查找时分两步进行：

第一步：先用待查的关键字对索引关键字表进行二分查找或顺序查找，确定待查数据元素可能在哪一块。

第二步：在已确定的那一块中进行顺序查找。

如图 1.46 所示为 12 个数据元素构成的查找表。按分块查找的思想：（1）将它分成三块；（2）建立一个最大顺序的索引关键字表；（3）根据待查的关键字进行查找，设待查的关键字为 50 的元素，则从索引关键字表中查到该 50 应在第二块中，因 30<50<65。再在第二块内用顺序查找到关键字为 50 这个元素。

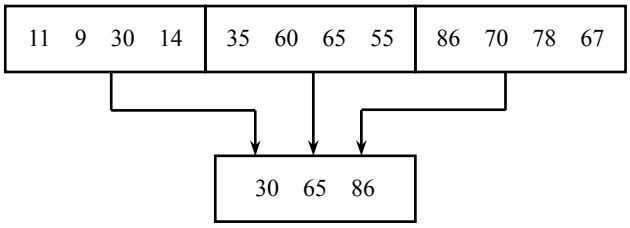


图 1.46　查找表及其索引

二、树表查找

在前面介绍的三种简单查找方法（顺序查找、二分查找、分块查找）中，以二分查找的效率最高，但二分查找要求查找表中的数据元素按关键字有序，且不能用链表作存储结构。当对查找表中的数据元素进行插入和删除时，为了保持查找表的有序性，势必要移动很多记录，造成新的时间开销，当插入和删除频繁进行时，这种额外开销就会抵消二分查找的优点。

使二分查找效率降低的根本原因是查找表的顺序组织方式，采用二叉排序树作为查找表中数据元素的结构，就能避免二分查找方法的不足。

根据二叉排序树的定义，在二叉排序树中，左子树上所有节点的关键字都小于根节点的关键字；右子树上所有节点的关键字都大于根节点的关键字。因此，二叉排序树查找的基本思想是：

（1）当二叉排序树不空时，首先将给定值 k 与根节点的关键字进行比较，若相等则查找成功；

（2）若给定值 k 小于根节点的关键字，则下一次与左子树的根节点的关键字进行比较，若给定值 k 大于根节点的关键字，则与右子树的根节点的关键字进行比较。如此递归地进行下去直到某一次比较相等，查找成功。如果一直比较到树叶都不等，则查找失败。

下面来讨论这个递归查找算法的 C 语言编程实现。首先给出二叉排序树中节点数据类型定义和节点的结构类型定义：

```
typedef struct node_data
    { ┆
      keytype key;          /* 关键字类型说明 */
       ┆
    } datatype;
typedef struct bs_node
    { datatype data;
      struct bs_node *LC;
      struct bs_node *RC;
    } bsnode;
```

【算法 28】　在二叉排序树上进行查找的算法：

```
bsnode * bs_search（bsnode *t，keytype k）
    { bsnode *s;
      if（t==NULL）
        { printf（"empty tree"）;
          return（t）;
        }
      s=t;
      if（（s－>data）.key==k）
        { printf（"searching success"）;
          return（s）;
        }
      else if （（s－>data）.key>k）          /* 在左子树上继续查找 */
              return（bs_search（s－>LC，k））;
           else                              /* 在右子树上继续查找 */
              return （bs_search（s－>RC，k））;
    }
```

例 8　一组数据元素的关键字序列是 45，24，53，12，37，95，30，40，80。这一组数据元素按二叉排序树组织后，所构成的查找表的结构如图 1.47 所示。

如果要查找关键字是 37 的节点，第一步：37 与根节点P_1之关键字比较，因 37<45，查找左子树；第二步：37 与左子树根节点P_2之关键字比较，因 37>24，查找P_2的右子树；第三步：37 与节点P_3的关键字比较，因 37＝37，故查找成功。

如果要查找关键字是 50 的节点：第一步：50 与根节点P_1之关键字比较，因 50>45，

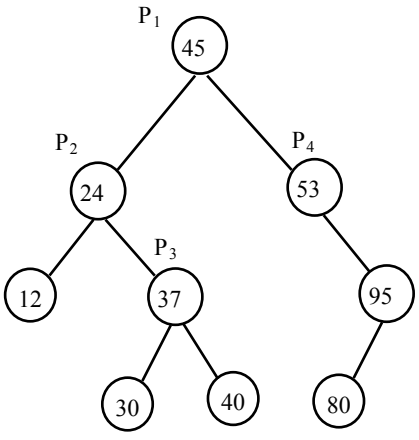


图 1.47　例 8 所构造的二叉排序树

查找P_1的右子树；第二步：50 与节点P_4的关键字比较，因 50<53，查找P_4的左子树；第三步：P_4的左子树为空。故查找失败。

由上例可以看出：在二叉排序树中查找成功时走了一条从根节点到所查节点的路径。因此二叉排序树查找的平均查找长度取决于二叉排序树的深度。

当二叉排序树左、右子树的深度不超过 1 时平均查找长度与二分查找相同，ASL≈$\log_2 n-1$，此时二叉排序树查找的效率最高。但是，值得注意的是：由于二叉排序树是动态生成的，树的节点随插入的先后次序不同而不同，最恶劣的情况是会蜕化成一棵单支树，此时二叉排序树查找长度与顺序查找相同，$ASL \approx \frac{1}{2}(n+1)$，效率最低。因此，总体来说，二叉排序树查找算法的效率介于O[n] 与O[log n]之间。

三、哈希查找

哈希查找又称散列查找，它不像前面讨论的按关键字的比较来确定被查找的数据元素，而是通过对关键字作某些运算后直接确定数据元素的存储地址。这个运算通过一个称之为哈希函数（或叫做散列函数）的函数来表示。

设a_i是长度为 n 的查找表中的一个数据元素（$1 \leqslant i \leqslant n$），其关键字为$k_i$，则可建立$k_i$和地址$a_i$的函数关系：

$$Addr(a_i) = H(k_i)$$

其中，$Addr(a_i)$为a_i的地址（或称为a_i的哈希地址），$H(k_i)$为哈希函数。通过哈希函数，可把关键字集合中的元素映像到地址集合中的元素，即由关键字空间映像到地址空间。一般来说，关键字集合往往远大于地址集合，不可能按照一一对应的关系实现集合元素的映射。如居民身份证号码有十位数字，一般计算机系统不可能提供如此大的存储空间给查找表使用，实事上也完全没有必要要如此大的存储空间。在大多数情况下，哈希函数是一种“压缩映像”，它把数据元素的关键字取值很大的数据集合映射到一个范围确定的地址表中。如以学号作为学生数据元素的关键字，哈希函数得到学号的后两位作为地址，可得到如表 1-1 所示的关键字和地址的对照表。该表的哈希函数H（k）=k mod 100（或k－int（k/100）*100）。

表 1-1

Addr	04	25	09	01	30	27	02	05	11	25	08	16	…
Key	95104	95125	95203	95201	95130	95127	95102	95105	95111	95225	95108	95216	…

显然，采用“压缩映像”后产生的地址可能会发生冲突，如表 1-1 中关键字 95125 和 95225 经过哈希函数运算得到的地址都是 25。所以哈希查找方法中，既要选择一个计算简单、地址分布均匀的哈希函数，以避免或减少冲突发生；又要在出现冲突时，建立解决冲突的方法。

构造哈希函数的方法很多，如上面例子中采用的是除余法，即选择一适当的正整数整除关键字，取其余数作为地址。当需要给定地址范围时，可再将上述系数加上一个常数。除除余法之外，还有一些常用的哈希函数构造方法，如直接定址法（又称自身函数法）、数字分析法、折叠法、平方取中值法等。

解决哈希函数产生的地址冲突的方法也很多，但常用的只有两种：链地址法和开放地址法。

（1）链地址法：链地址法处理冲突的基本思想是：将所有具有相同哈希地址的 i 个数据元素存储在同一个单链表中。这一点与邻接表的基本思想非常类似。

链地址法处理冲突产生的哈希表的每个存储单元需增加一个指针域，用来指向下一个具有相同哈希地址的数据元素。

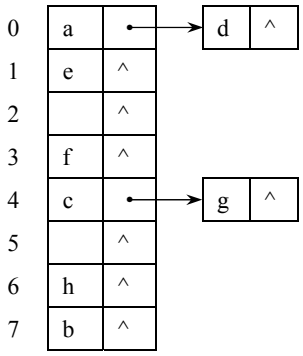


图 1.44　用链地址法解决冲突的哈希表

例 9　设有 8 个元素，{a，b，c，d，e，f，g，h}，通过某哈希函数得到的哈希地址范围为 0～7，采用链地址法得到的哈希表如图 1.44 所示。

（2）开放地址法：开放地址法的基本思想是在发生冲突时，按照某种方法继续探测哈希表中的其他存储单元，直到找到空位置为止。可以用下式描述：

$$H_i(k) = (H(k) + d_i) \bmod m \quad (i=1, 2, \cdots, m)$$

式中，H（k）为关键字k的直接哈希地址；m为地址空间长度（即哈希表的长度）；d_i为每次再探测时的地址增量。

用这种方法时，首先计算出关键字的直接哈希地址H(k) ，若该单元已被其他数据元素先占用，继续查看地址为H(k) $+d_1$的单元，若该单元已被占用，再继续查看地址为H（k）$+d_2$的单元，如此下去，直到发现某个单元为空时，将关键字为k的记录存放到该单元中。增量d_i可以有不同的取法：

（1）$d_i=1, 2, 3, \cdots$

（2）$d_i=1^2, -1^2, 2^2, -2^2, \cdots$

（3）d_i＝伪随机序列

当d_i采用上述三种不同的取值法时，分别称为线性探测再散列、二次探测再散列和伪随机再散列。

例 10　设查找表关键字序列为（32，13，49，55，22，20，21），哈希函数 H(k)＝k mod T，按下述两种解决冲突的方法分别构造出哈希表：

（1）线性探测再散列；（2）二次探测再散列

【说明】　先来看按照线性探测再散列法处理冲突的哈希表构造：

32　MOD　7＝4　存入地址空间 4；

13　MOD　7＝6　存入地址空间 6；

49　MOD　7＝0　存入地址空间 0；

55　MOD　7＝6　发生冲突，处理冲突如下：

　　下一个存储地址是（6＋1）　MOD　7＝0　仍发生冲突

　　再下一个存储地址（6＋2）　MOD　7＝1　不冲突，存储

22　MOD　7＝1　发生冲突，处理冲突如下：

　　下一个存储地址是（1＋1）　MOD　7＝2　不冲突，存储

38　MOD　7＝3　存入地址空间 3；

21　MOD　7＝0　发生冲突，处理冲突如下：

下一个存储地址是（0+1）　MOD　7＝1　仍发生冲突

再下一个存储地址是（0+2）　MOD　7＝2　仍发生冲突

再下一个存储地址是（0+3）　MOD　7＝3　仍发生冲突

再下一个存储地址是（0+4）　MOD　7＝4　仍发生冲突

再下一个存储地址是（0+5）　MOD　7＝5　不冲突，存入该地址

所得哈希表如表 1-2 所示。

表 1-2　用线性探测再散列法处理冲突产生的哈希表

0	1	2	3	4	5	6
49	55	22	38	32	21	13

用同样的方法可以分析得到用二次探测再散列法处理冲突产生的哈希表，如表 1-3 所示。

表 1-3　用二次探测再散列法处理冲突产生的哈希表

0	1	2	3	4	5	6
49	22	21	38	32	55	13

建立了哈希表，在哈希表上再来查找关键字为给定值 k 的数据元素的过程就很简单了。其基本思想是：根据 k 值求出其直接哈希地址，若该单元为空，则查找失败；若该单元不为空，将 k 与该单元的关键字相比较，若二者相等，则查找成功，否则按照所采用的构造哈希表的方法，继续在“下一个存储地址”中查找。

下面以采用链地址法构造出一个长度为m的哈希表 a 为例，来说明在表 a 上进行哈希查找的算法。在哈希链表中节点的类型定义如下：

```
typedef struct chain
    { keytype key;                    /* 关键字类型说明 */
      elemtype data;                  /* 数据元素类型说明 */
      struct chain * next;
    } chainnode;
```

【算法 29】　若哈希函数为 H，那么在表 a 中查找关键字为 k 的数据元素的算法如下：

```
chainnode * hash_search（chainnode * a［］，keytype k）
  { chainnode *p;
    int i;
    i=H（k）;
    p=a［i］;
    while（p!=NULL && p－>key!=k）
        p=p－>next;
    if（p==NULL）
      { printf（"searching failed"）;
        return（NULL）;
```

```
    }
  else
    { printf（"searching success"）;
      return（p）;
    }
}
```

哈希查找是一种直接计算地址的方法，在查找过程中所需的比较次数较少，属于高效查找算法之一。由查找算法可以看出，在进行哈希查找时，首先要由哈希函数、解决冲突的方法以及关键字值计算出哈希地址，因此在考虑哈希查找的效率时，不但要考虑查找时所需的比较次数，还要考虑求得哈希地址所需时间。哈希查找的效率分析比较复杂，这里不详细分析，一般在不考虑哈希地址计算的复杂度的情况下，可以近似认为哈希查找算法效率为 O[1]。

1.4.2 排序

排序又称为分类，它是数据处理中经常使用的一种重要的运算。如何进行排序，特别是高效率地进行排序是计算机应用中的重要课题之一。

排序是将一组数据元素（有时又称为记录）按其排序码进行递增或递减排列的运算操作。假设含有n个数据元素的序列为｛R_1，R_2，…，R_n｝，其相应的排序码为｛k_1，k_2，…，k_n｝。按排序码的某种次序排列得到一种排列p，使

$$k_{i(P)} \leqslant k_{i+1(p)} \quad 或 \quad k_{i(P)} \geqslant k_{i+1(p)}$$

由此得到的一个按其排序码线性有序的序列｛$R_{1(p)}$，$R_{2(p)}$，…，$R_{n(p)}$｝就是排序。下面介绍几个有关排序的基本概念：

（1）排序码：排序的依据是排序码，它是数据元素中的一个或多个字段；排序码可以是关键字，也可以不是关键字。

（2）排序算法的稳定性：如果待排序的序列中，存在多个具有相同排序码的数据元素，若经过排序这些数据元素的相对次序保持不变，则称这种排序算法是稳定的，若经过排序，这些数据元素的相对次序发生了改变，则称这种排序算法是不稳定的。

（3）内部排序和外部排序：内部排序指对数据元素序列的整个排序过程都是在计算机内存中进行的排序；外部排序是指待排序的数据元素太多，不能同时存放在内存中，所以排序操作不仅要使用内存，而且还要使用外部存储器存储数据的排序。

衡量内部排序算法和外部排序算法的运算效率，其方法是不同的。因为对内部排序而言，时间的主要开销花在数据元素的比较和移动上，可用算法执行时要进行的比较次数来衡量；而对外部排序而言，既要进行数据元素的比较和移动，还要不断地读、写外部存储器，读、写外部存储器的速度是很慢的，因此，外部排序的时间代价更主要的是用读、写外部存储器的次数来衡量。

一、简单排序算法

下面介绍几种最常用、最简单的排序方法。为了简单起见，采用顺序存储结构存放所排序的数据元素序列。存储结构说明如下：

```
typedef struct
    {   ┆
        keytype key;                /* 顺序码说明 */
        ┆
    } elemtype;                     /* 元素类型说明 */
elemtype x［MAXNUM］;               /* 排序序列说明 */
```

1．简单插入排序：简单插入排序的基本思想是把 n 个数据元素的序列分为两部分，{R_1，…，R_{i+1}｝为已排好序的有序部分，{R_i，R_{i+1}，…，R_n｝为未排序部分。这时，把未排序部分的第一个元素R_i依次与R_1，…，R_{i-1}比较，并插入到有序部分的合适位置上，使得{R_1，…，R_i｝变为新的有序部分。

初始时，令i＝2。因为一个元素自然有序，故{R_1｝自然成为一个有序部分，未排序部分是{R_2，…，R_n｝，然后依次将R_2，R_3，…，R_n插到有序部分中去，就可得整个有序序列。

例 11　待排序数据元素序列的排序码序列是（18，12，10，12，30，16），写出简单插入排序每一趟执行后的序列状态。

【说明】　图 1.45 说明了简单插入排序的过程。

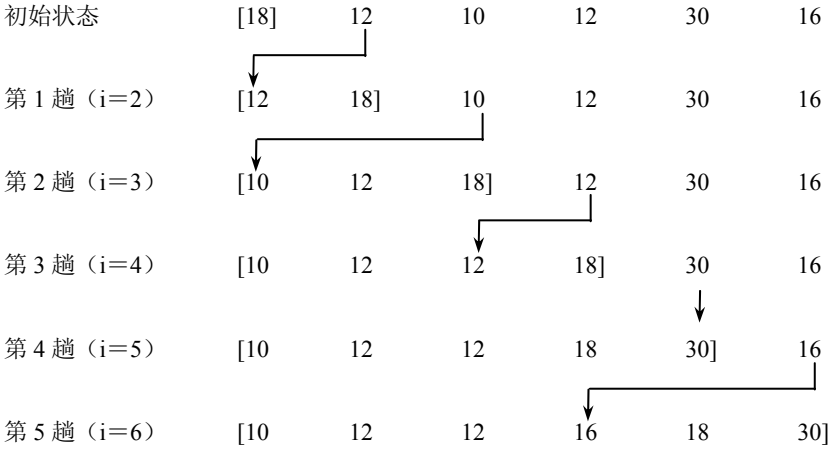


图 1.45　简单插入排序示例

【算法 30】　下面给出简单插入排序算法的 C 描述：

```
insertsort（elemtype x［］，int n）
    {   int i，j；
        elemtype a；
        for（i=0；i<n－1；i++）                    /* 对 x［0］～x［n－1］进行排序 */
          {   a=x［i+1］；
              j=i；
              while （j>－1 && a．key<x［j］．key）
                  {   x［j+1］=x［j］；
                      j－－；
                  }
```

```
            x［j+1］=a;                    /* 插入 a 元素 */
        }
    }
```

简单插入排序是稳定的排序算法，它不含改变原序列中排序码相同的元素的相对次序。

下面分析一下简单插入排序算法的效率，排序算法通常需要比较、交换或者移动数据项等基本运算，除非特殊应用情况，一般衡量排序算法执行效率的标准是平均数据元素比较次数。在上面简单插入排序函数insertsort中，外层for循环执行n−1 次，该循环包含一个内层的while循环，此循环最多执行j次，进行数据元素的比较和移动（j的值从 1 到n−1 变化）。因此在最坏情况下，算法需要进行的数据元素比较次数为：1+2+…+ n−1＝n(n−1)/2。算法效率为O[n^2]。

需要说明的是，简单插入排序算法在插入元素时要进行数据移动，在最坏情况下，需要进行 1+2+…+ n−1＝n(n−1)/2 次数据移动，这是该算法的不足。

2. 简单选择排序：简单选择排序的基本思想是：第一趟排序是在无序的{R_1，R_2，R_3，…，R_n｝中按排序码选出最小的元素，将它与R_1交换；第二趟排序是在无序的｛R_2，R_3，…，R_n｝中选出最小的元素，将它与R_2交换；而第i趟排序时｛R_1，R_2，…，R_{i-1}｝已排好序，在当前无序的｛R_i，…，R_n｝中再选出最小的元素，将它与R_i元素交换，使｛R_1，R_2，…，R_i｝成为有序。因为每趟排序都使有序区中增加了一个数据元素，且有序区中的数据元素的排序码均不大于无序区中元素的排序码，所以经过n－1 趟排序后，整个数据元素序列就递增有序。

例 12　设待排序数据元素序列的排序码序列为（2，7，2，2，3，1），写出按递增排序的简单选择排序每一趟执行后的序列状态。

【说明】　简单选择排序过程如图 1.46 所示。

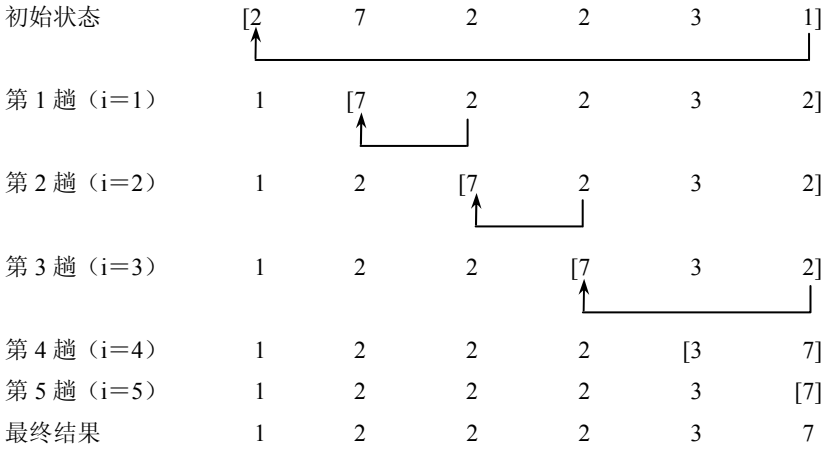


图 1.46　简单选择排序示列

【算法 31】　下面给出简单选择法的 C 描述：

```
selectsort （elemtype x［］，int n）
  { int i，j，small;
    elemtype swap;
```

```
for （i=0；i<n-1；i++）
  { small=i；
    for （j=i+1；j<n；j++）          /* 找出排序码最小的数据元素 */
      { if（x［j］  key<x［small］  key）
            small=j；
      }
    if（small!=i）                   /* 把无序区中最小元素与无序区中第一个元素交换 */
      { swap=x［ i ］；
        x［i］=x［small］；
        x［small］=swap；
      }
  }
}
```

简单选择法排序会改变排序码相同的数据元素的相对位置，因此，简单选择法排序是不稳定的排序算法。

在采用简单选择排序算法中，函数selectsort中的外层for循环要执行n−1 次。因此，找出排序码最小的数据元素的内for循环和数据元素交换swap各要执行n−1 次。对于找出排序码最小的数据元素的内for循环，最坏情况要执行n−i次数据比较。因此，简单选择排序selectsort函数在最坏情况下需要总共执行：(n−1) + (n−2) + … + 1＝n(n−1)/2 次。算法效率为$O[n^2]$。

需要说明的是，简单选择排序算法不需要数据的移动，但需要 n−1 次数据元素交换操作，这是该算法的特点。

3．冒泡排序：冒泡排序是一种常用的方法，它的基本思想是：从R_1开始，两两比较R_i和R_{i+1}（i＝1，2，…，n−1）的排序码的大小，若$k_i > k_{i+1}$则交换R_i和R_{i+1}的位置。第一趟全部比较完毕后R_n是序列最大的数据元素。

再从R_1开始两两比较R_i和R_{i+1}（i＝1，2，…，n−2）的排序码的大小，若$k_i > k_{i+1}$则交换R_i和R_{i+1}的位置。第二趟全部比较完毕后R_{n-1}是序列中次大的元素。如此反复，进行n－1趟冒泡排序后所有待排序的n个元素序列按排序码有序。

下面来看一个例子。

例 13　待排序元素序列的排序码序列是（65，97，76，13，27，49，58），写出冒泡排序每一趟执行后的序列状态。

【说明】　图 1.47 说明了冒泡排序的过程。

初始状态	[65	97	76	13	27	49	58]
第 1 趟（j＝1～6）	[65	76	13	27	49	58]	97
第 2 趟（j＝1～5）	[65	13	27	49	58]	76	97
第 3 趟（j＝1～4）	[13	27	79	58]	65	76	97
第 4 趟（j＝1～3）	[13	27	49]	58	65	76	97
第 5 趟（j＝1～2）	[13	27]	49	58	65	76	97
第 6 趟（j＝1）	[13]	27	49	58	65	76	97

图 1.47　冒泡排序示例

【算法 32】　下面给出冒泡排序的算法的 C 描述：

```
bubblesort （elemtype x[ ]，int n）
  { int i，j，flag；
    elemtype swap；
    flag=1；
    for（i=0；i<n－1 && flag==1；i++）
      { flag=0；
        for（j=0；j<n-i；j++）
          if （x［j］. key>x［j+1］. key）
            { flay=1；
              swap=x［j］；
              x［j］=x［j+1］；
              x［j+1］=swap；
            }
        if（flag==0）return；
      }
  }
```

在该算法中使用了一个标记变量 flag，用于标记在每一趟执行时是否有数据元素交换，如果有一趟冒泡排序“空跑”，表示此时数据元素序列已经有序，应结束排序过程，如上例中执行完第 3 趟冒泡排序后元素序列已经有序。后面的 3 趟冒泡空跑是不必要的。

冒泡排序算法是稳定的。对于加标记变量flag的冒泡排序能“判别”数据元素序列的状态，所以当待排序序列是基本有序时，采用冒泡排序方法的效率是很高的。在待排序序列非常无序时，冒泡排序算法的效率并不高。因为在最坏情况下，bubblesort函数需要执行n−1趟（外层for循环）。第 1 趟需要n−1 次比较和最多n−1 次交换（执行一次内层for循环）；第 2 趟需要n−2 次比较和最多n−2 次交换，依此类推，第i趟需要n−i次比较和最多n−i次交换。因此在最坏情况下，冒泡排序算法总共需要：(n−1) + (n−2) + … +1＝n (n−1)/2 次比较。算法效率为O[n^2]。

需要说明的是，冒泡排序算法需要进行大量的数据元素交换，在最坏情况下，需要进行 1+2+…+ n−1＝n(n−1)/2 次数据交换，这会影响算法执行速度，是该算法的不足。

二、快速排序

快速排序又叫做分区交换排序，是目前已知的平均速度较快的一种排序方法，它是对冒泡排序方法的一种改进。快速排序方法的基本思想是：从待排序的n个数据元素中任意选取一个元素R_i（通常选取无序序列中的第一个元素）作标准，调整序列中各个元素的位置，使排在R_i前面的元素的排序码都小于R_i.key，排在R_i后面的元素的排序码都大于R_i.key。通常把这样的一个过程称为一次快速排序。在第一次快速排序中，确定了元素R_i最终在序列中的排列位置，同时也把剩余的数据元素分成了两个子序列。对两个子序列再分别进行快速排序，又确定了两个元素在序列中应处的位置，并将剩余元素分成了四个子序列，如此重复下去，当各个子序列的长度为 1 时，全部元素排序完毕。

可见，快速排序中的基本操作是子序列的划分操作。设当前处理的元素序列的下界是low，上界是 high，划分操作的步骤如下：

（1）让两个指针 i，j 分别指向序列的第一个元素和最后一个元素（即 i＝low；j＝high），并将第一个元素的排序码保存在 k 中；

（2）用k与j指向的元素的排序码key比较，若k≤R_j.key，则再比较j的前一个元素的排序码（j＝j－1）；否则将元素R_j和R_i互换位置。此过程又称为从右向左扫描过程。

（3）用k与i指向的元素的排序码key比较，若k≥R_i.key，则再与i的后一个元素排序码比较（i＝i－1）；否则将元素R_i和R_j互换位置。此过程又称为从左向右扫描过程。

（4）比较 i 和 j，i＜j，则重复上面（2）、（3）操作，直到 i＝j 时，划分操作结束。

下面用例子进行具体的说明。

例 14　一组待排序元素的排序码序列为（49，38，60，90，70，15，30，49），试画出在第一趟快速排序中元素的交换示意图。

【说明】　快速排序第一趟中元素交换过程如图 1.48 所示。其中第一个元素排序码R_1＝49 为划分操作的依据。

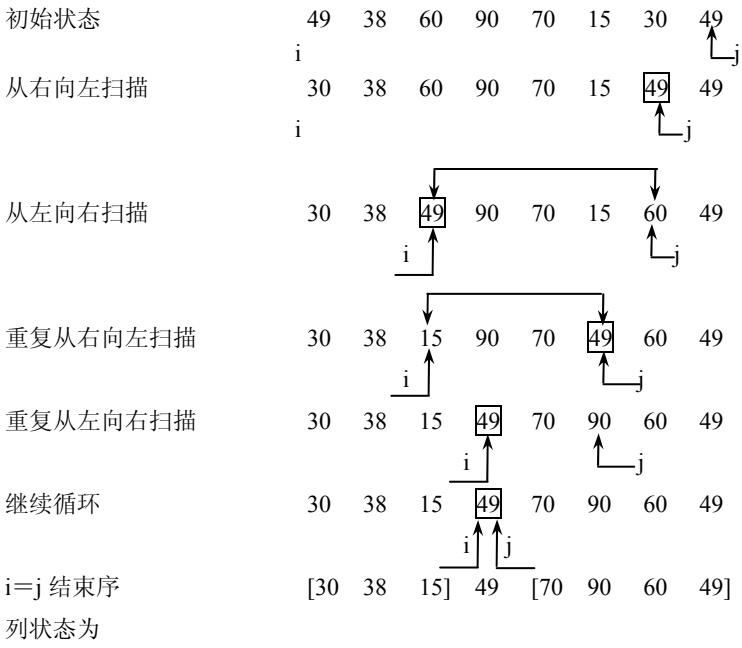


图 1.48　快速排序示例

【算法 33】　下面给出快速排序的算法的 C 语言描述：

```
quicksort（elemtype x［］，int l，int r）
  { int i，j；
    elemtype swap；
    i=l；j=r；
    swap=x［l］
    while（i<j）
```

```
    { while（i<j && swap.key=<x[j].key）
          j－－；                          /* 从右向左扫描 */
      if（i<j）
        { x［i］=x［j］;
          i++;
        }
      while（i<j && swap    key>=x[i].key）
          i++;                            /* 从左向右扫描 */
      if（i<j）
        { x［j］=x［i］;
          j－－；
        }
    }
  x［i］=swap;
  if（l<i）quicksort（x，l，i-1）;        /* 对所得到的子序列再进行快速排序 */
  if（i<r）quicksort（x，j+1，r）;
}
```

上述函数中，待排序元素序列顺序存放在 x[l]，x[l+1]，…，x[r]中，变量 i、j 指示左右两端当前扫描到的元素的序号，变量 swap 为存放当前标准元素的中间变量。该函数采用递归的方法来实现。

快速排序是不稳定的排序，并且对于同一待排序列，如果选取不同的基准元素的话，某排序速度可能不同。如果选择的基准元素恰好是序列中所有元素的中位数，则所划分的前后两个子序列的元素数目近乎相等。这时排序速度最快。在这种情况下，我们可以简单估算一下快速排序的效率。在第一次扫描中大概需要n−1 次比较，然后待排序数据元素序列会分为两个大小大致为n/2 的子序列。这两个子序列每个大致需要n/2 次比较，然后生成 4 个大小大致为n/4 的子子序列。依次分解，进行$\log_2 n$次划分之后，生成n个大小为 1 的子序列。这样排序所需的总的比较次数为：

$$\underbrace{(n-1)+2\cdot(n-1)/2+\cdots(n-1)\cdot 1}_{\log_2 n}=(n-1)\log_2 n$$

因此，算法效率为$O[n\log^n]$。

三、归并排序

归并排序又称作表排序，归并的含义是把两个（或两个以上）有序的序列合并为一个有序的序列，若是对两个有序序列进行归并又称为 2-路归并。

归并排序的基本思想是：将有 n 个元素的原始序列看作 n 个有序子序列，每个子序列的长度为 1，然后从第一个子序列开始，把相邻的子序列两两合并，得到 n/2 个长度为 2 或 1 的子序列，这一过程称为一次归并排序，对第一次归并排序后的 n/2 个子序列采用上述方法，继续顺序成对归并，如此重复，当最后得到长度为 n 的一个子序列时，该子序列便是

原始序列归并排序后的有序序列。

由归并排序的基本思想可以看出：在归并排序过程中，最基本的问题是如何把两个位置相邻的有序子序列合并为一个有序子序列。

设两个有序序列存储在一维数组 R 中，有序子序列 1 是（R[1]，R[l＋1]，…，R[m]），有序子序列 2 是（R[m＋1]，…，R[r]）。归并后产生的有序序列存放在另一个一维数组 x 中，该有序序列是（x[1]，x[l＋1]，…，x[r]）。归并方法如下：

（1）设置三个变量i_1，i_2，j。其中i_1，i_2分别表示序列 1 和 2 中当前要比较的元素的位置号（下标），初始值为 1，m＋1；j表示数组 x 中当前元素应放置的位置号（下标），初始值为 1。

（2）反复比较R[i_1]和R[i_2]排序码的值，若R[i_1].key≤R[i_2].key，则R[i_1]存储到x[j]中，然后，i_1＝i_1＋1，j＝j＋1；若R[i_1].key＞R[i_2].key，则R[i_2]存储到x[j]中，然后i_2＝i_2＋1，j＝j＋1。

（3）当序列 1 或 2 中的全部元素已归并到数组x中，即i_1＝m或i_2＝r时，比较结束，然后将另一序列中剩余的所有元素依次放到数组x中，这样就完成了有序序列 1 和 2 的合并。

下面进一步来讨论一次归并排序的算法。

所谓一次归并排序，是把若干个长度为 k 的相邻有序子序列，从前向后两两进行合并，得到若干个长度为 2k 的相邻有序子序列。这里有一问题，若元素的个数 n 为 2k 的整数倍，序列的两两归并正好完成 n 个元素的一次归并，否则当归并到一定位置时，剩余的元素个数不足 2k 个，这时的处理方法是：

（1）若剩余的元素个数大于 k 而小于 2k，把前 k 个元素作为一个子序列，其他剩余的元素作为另一个子序列，显然，它们是分别有序的。采用上面所讲的方式对这两个子序列进行归并。

（2）若剩余的元素个数小于等于 k，可以直接把它们顺序放到数组 x 中去，因为它们已经有序了。

【算法 34】　下面给出一次归并排序的算法：

```
merge（elemtype R［］，elemtype x［］，int k，int n）
 /* 对序列 R[0]～R[n－1]进行归并排序，每个子序列长度为 k，结果放在 x[0]～x[n－1]中 */
 { int i1，i2，l1，r1，l2，r2，j;
   l1＝0；j=0;
   while（l1＋k＜n－1）
     { l2=l1＋k;
       r1＝l2－1;
       r2＝（l2＋k－1＜＝n－1）?l2＋k－1：n－1;
       for（i1＝l1，i2＝l2；i1＜＝r1＆＆i2＜＝r2；j＋＋）
         { if（R[i1].key＜＝R[i2].key）
             { x[j]＝R[i1];
               i1＋＋;
             }
```

```
        else
            { x[j]=R[i2];
              i2++;
            }
        }                          /* end for */
        while（i1<=r1）        /*序列 2 已归并完，将序列 1 中剩余元素顺序放到数组 x 中 */
            { x[j]=R[i1];
              i1++; j++;
            }
        while（i2<=r2）        /*序列 1 已归并完，将序列 2 中剩余元素顺序放到数组 x 中 */
            { x[j]=R[i2];
              i2++; j++;
            }
        l1=r2+1;              /* 从前向后继续进行 */
    }                          /* end while */
    for（i1=l1; i1<n; i1++, j++）      /* 将原始序列中剩余的不足一组的元素顺序放到数
组
                                                x 中 */
        x[j]=x[i1];
}
```

归并排序方法就是调用“一趟归并排序”的算法，经过多次归并排序来实现。对于 2-路归并排序，每调用一次一趟归并排序的函数，待排序列中有序子序列的长度扩大一倍。第一趟归并时，有序子序列的长度 k 为 1，当有序子序列长度 k≥n 时排序完成。

【算法 35】 2-路归并排序算法如下：

```
mergesort （elemtype R［］, int n）
  { int i, k;
    elemtype x［MAXNUM］;
    k=1;
    while（k<n）
      { merge （R, x, k, n);
        for（i=0; i<n; i++）
            R［i］=x［i］;
        k=2*k;                 /* 子序列长度加倍 */
      }
  }
```

下面举例来说明归并排序的过程。

例 15 待排序元素序列的排序码为（25，57，48，37，12，92，86），写出 2-路归并排序每一趟执行后的序列状态（见图 1.49）。

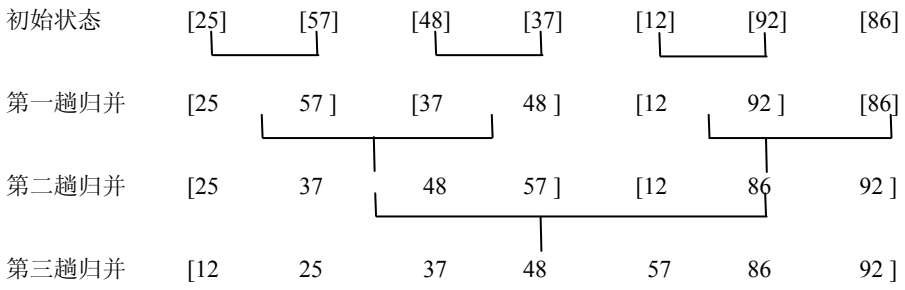


图 1.49　2-路归并排序示列

2-路归并排序是稳定的，其运行效率与快速排序是同一数量级，算法效率也为O[n log^{n}］。归并排序在计算中需要一个与原始数组一样大小的辅助数组，从而占用存储空间较多，这是归并排序算法的不足。

小　　结

本节介绍了在数据结构上的两类重要操作：查找与排序。查找又称为检索，它的目的是从确定的数据集合中找出某个特定的元素。最常用的查找方法有线性表的三种简单查找方法（顺序查找、二分查找和分块查找）、有利用二叉排序树组织数据的树表查找、有建立哈希表的哈希查找等。

顺序查找的基本思想是从线性表的一端顺序扫描表中各元素，直到找到关键字与给定值相等的数据元素，在等概率情况时其平均查找长度为（n+1）/2。

二分查找是建立在已排序的线性表上，取给定值与当前线性表的中间元素的关键字比较，以确定待查元素可能在表中上半段、下半段，还是中间元素，如此方法循环使用，直到找到待查元素或查找失败。在等概率情况下，二分查找平均查找长度为 $\log_2{}^{n}-1$。

分块查找是结合二分查找和顺序查找思想，将无序的待查表分成几个块间有序、块内无序的子表；确定待查元素属于哪一块可用二分查找；在块内的查找可用顺序查找。

二叉排序树的查找是利用二叉排序树来组织数据时，可提供一种数据有序的事实（用中序遍历二叉排序树各节点，可得到各节点元素升序排列的序列）。当构造的二叉排序树任一节点的左右子树的高度最多相差 1 时（此时称二叉树为平衡二叉树），二叉排序树的查找效率同二分查找一样。

哈希表查找是通过一个哈希函数，由数据元素的关键字直接得到哈希地址来查找元素。使用哈希表查找时，数据查找表的数据元素是按哈希表存储的。为了建立哈希表，应构造一个计算简单且均匀映射的哈希函数，采用合适的解决冲突的方法。哈希表查找的效率取决于查找时数据比较的次数和利用哈希函数以及解决冲突的方法来求得哈希地址所需的时间两方面因素。

排序也是数据处理中经常使用的重要运算，研究高效排序方法是计算机应用中的重要课题之一。在本节，介绍了排序算法涉及的一些基本概念和常用的排序算法。

简单插入排序、简单选择排序、简单冒泡排序是最常用的三种排序算法。简单插入排序的基本思想是顺序地把待排序列中的各个元素按其排序码的大小，插入到已排序的序列的适当位置上；简单选择排序的基本思想是不断从待排序的序列中选取排序码最小的数据

元素放到已排序的元素序列的后面，直到序列中所有记录都已排序为止；简单冒泡排序的基本思想是每次对相邻的两个元素的排序码进行比较，当不符合次序即交换位置，这样，顺序码值大的（或小的）像冒气泡一样逐渐升起，直到所有相邻两元素的次序合理为止。

快速排序是在冒泡法排序基础上发展起来的一种排序算法，它每一步都将待排序列中第一个元素放到最终位置上，并同时将待排序列中其他元素分成两个子序列，一个子序列中元素的顺序码均小于第一个元素的排序码，另一个子序列中元素的顺序码均不小于第一个元素的排序码。如此循环处理，直到每个子序列只剩下一个元素，即完成了排序。

归并排序是利用归并技术来进行排序，把待排的数据元素看作长度为 k 的有序序列（开始 k 为 1），把相邻的序列两两归并成长度为 2k 的有序序列，如此循环，直至整个待排序列只剩下一个有序序列时为止。归并排序算法的两个重点是两个有序子序列的归并算法和 2-路归并排序进行一趟的算法。

习　　题

1．简述下列术语：数据、数据元素、数据结构。

2．数据结构主要研究哪些内容？

3．什么是数据逻辑结构？逻辑结构主要可分为哪几种？

4．什么是数据的存储结构？主要的存储方法有哪几种？

5．常见的操作包括哪些？

6．C 语言的数据类型有哪些？

7．试说明数据结构的概念与 C 语言中数据类型的概念有何异同。

8．试描述头指针、头节点、开始节点的区别，并说明头指针和头节点的作用。

9．设线性表存放在向量 A［1.．MAXNUM］的前 elenum 个分量中，且递增有序。试写一算法，将 x 插入到线性表的适当位置上，以保持线性表的有序性。

10．设有多项式：

$$A（x）＝1＋5x＋7x^{3}－9x^{7}＋5x^{16}$$

$$B（x）＝4x＋7x^{4}＋4x^{7}－3x^{15}＋x^{16}$$

① 用单链表给出 A（x）和 B（x）的存储表示；

② 以上述两个单链表为基础，通过插入和删除等运算给出 A(x)＋B(x) 的存储表示。

11．设有编号为 1，2，3，4 的四辆列车，顺序进入一个栈式结构的站台，具体写出两辆列车开出站的所有可能的顺序。

12．设两个栈共占一维数组空间 S[m]，它们的栈底分别设在一维数组的两端，且进栈的每个数据元素只占一个分量。试编写这两个栈公用的栈操作算法，取栈顶元素 get（i），其中 i 为 0 或 1（用以指示堆栈号）。

13．假设以一维数组 Sq[m] 存储循环队列的元素，若要使这 m 个存储空间全部都得到利用，需另设一个标志 tag，以标志 tag 为 0 或 1 来区分队头指针和队尾指针相同时队列是空还是满。编写与此结构相对应的入队列和出队列的算法。

14．对于下列稀疏矩阵

$$A_{6\times7}=\begin{bmatrix}1&0&0&0&0&2&0\\0&0&0&0&0&0&0\\0&-3&0&0&0&0&0\\0&-4&0&0&0&0&0\\0&0&0&0&0&-1&15\\0&0&0&7&0&0&0\end{bmatrix}$$

写出它的三元组表示法。

15．设有下三角矩阵$A_{4\times4}$为

$$A_{4\times4}=\begin{bmatrix}3&0&0&0\\2&-1&0&0\\5&4&1&0\\1&0&1&3\end{bmatrix}$$

$A_{4\times4}$采用压缩存储方式存储于一维数组S_a中，试求：

（1）一维数组S_a的元素个数；

（2）矩阵元素a_{43}在一维数组S_a中的下标。

16．在高级语言中，通常把串的连接操作 CONCAT（s，t）表示成字符加（+）运算。设有子串分别为：A="□"； B ＝"MULE"；C="□OLE"；D="MY"，写出下列运算结果：

（1）A+B；

（2）D+C+A。

17．二维数组 A 的元素是 6 个字符组成的串，行下标 i 的范围从 0 到 8，列下标 j 的范围从 1 到 10。从供选择的答案中选出正确答案填入括号中。

（1）存放 A 至少需要（　　）个字节；

（2）A 的第 8 列和第 5 行共占（　　）个字节；

（3）若 A 按行存放，元素 A［8，5］的起始地址与当 A 按列存放时的元素（　　）的起始地址一致。

【供选择的答案】

（1）a．90　　b．180　　c．240　　d．270　　e．540

（2）a．108　　b．114　　c．54　　d．60　　e．150

（3）a．A[8，5]　　b．A[3，10]　　c．A[5，8]　　d．A[0，9]

18．画出具有 3 个节点的树的所有形态。

19．画出具有 3 个节点的二叉树的所有形态。

20．给出下面树结构的先序遍历序列、中序遍历序列、后序遍历序列，如图 1.50 所示。

21．试找出分别满足下面条件的所有二叉树：

（1）先序遍历序列和中序遍历序列相同；

（2）中序遍历序列和后序遍历序列相同；

（3）先序遍历序列和后序遍历序列相同。

22．采用顺序存储方法和二叉链表存储方法来存储如图 1.51 所示的二叉树，试分别画出相应的存储结构。

23．对 n 个顶点的无向图 G，采用邻接矩阵表示，如何判别下列有关问题：

（1）图中有多少条边？

（2）任意两个顶点 i 和 j 是否有边相连？

（3）任意一个顶点的度是多少。

24．已知如图 1.52 所示的一邻接表，试求：

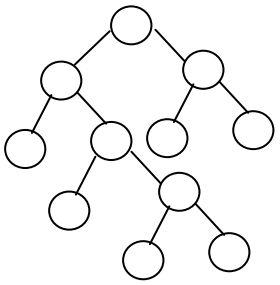


图 1.50

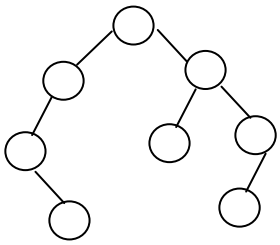


图 1.51

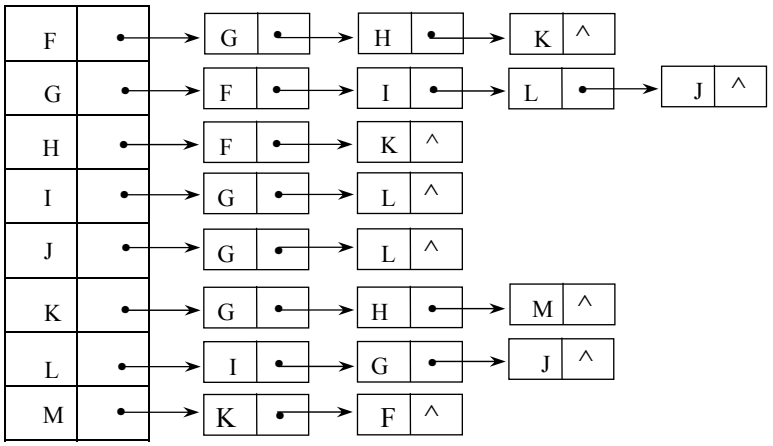


图 1.53

（1）给出此邻接表对应的图的邻接矩阵；

（2）给出由 G 开始的深度优先遍历序列；

（3）给出由 G 开始的广度优先遍历序列。

25．若线性表中各节点的查找概率不等，则可用如下策略提高顺序查找的效率：若找到指定的节点，则将该节点和其前趋（若存在）节点交换，使得经常被查找的节点尽量位于表的前端。试对线性表的顺序存储结构和链式存储结构写出实现上述顺序查找的算法。

26．设有序序列中数据元素的关键字分别为 3，10，13，17，40，43，50，70。要求查找的关键字 k 为 43 的元素（成功）和关键字为 5 的元素（不成功）。

使用二分查找，写出其计算步骤及每步的 low，high，mid 的值。

27．对以下关键字序列构造两个哈希表（每个哈希表的地址空间为 0 ～16）：（Jan，Feb，

Mar，Apr，May，Jun，Jul，Aug，Sep，Oct，Nov，Dec）。H(x) =i/2，其中 i 为关键字 x 中第一个字母在字母表中的序号。（1）用线性探测再散列法处理冲突；（2）用链地址法处理冲突，并分别求这两个哈希表在等概率情况下查找成功的平均查找长度。

28．对于给定的一组关键字（513，87，512，61，908，170，897，275，653，462），分别写出简单插入法、简单冒泡法、简单选择法、快速排序法、归并排序法来处理上述数据的各趟结果。

29．不难看出，对 n 个元素组成的线性表进行快速排序时，所进行的比较次数依赖于这 n 个元素的初始排列。

（1）n＝7 时，在最好的情况下需进行多少次比较？

（2）对 n＝7 给出一个最好情况的初始排列实例。

30．试设计一个算法，使得在尽可能少的时间内重排数组，将所有取负值的关键字放在所有取非负值的关键字之前。

第2章　操 作 系 统

操作系统是计算机的大脑、中枢神经，是计算机的灵魂。用户所有的工作都是在操作系统的支撑环境中进行的，在计算机系统中，各种软、硬件资源也是由操作系统来管理的。本章将从资源管理和用户使用两个角度对计算机操作系统的基本概念、工作原理及实现机制进行讨论，分别叙述操作系统的处理机管理、存储管理、设备管理、文件管理和用户接口五大功能。

2.1　操作系统概述

计算机操作系统由硬件和软件两部分组成，计算机硬件资源主要包括中央处理机（CPU）、存储器和外部设备；软件资源则是指信息文件，是指充分发挥计算机效率的操作系统、各种应用程序及其数据，如各种语言的编译和解释程序、链接、编辑程序、数据库管理系统等。

操作系统（Operating System，OS）是计算机最重要的系统软件，在计算机系统中占据了特殊、重要的地位，同时 OS 也是计算机领域中最活跃学科之一，其更新、发展的速度十分迅速。

在所有软件中，操作系统是紧挨着硬件的第一层软件（见图 2.1），是对硬件功能的首次扩充，所有其他软件都必须在操作系统的支撑下才能建立和运行。因此，操作系统不仅是硬件与所有其他软件的接口，而且是整个计算机系统的控制和管理中心，起到“中枢神经”的作用。操作系统是现代计算机系统必不可少的关键组成部分。

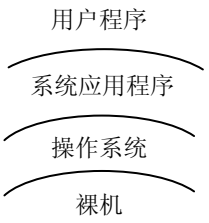


图2.1　操作系统与软、硬件的关系

本节概述操作系统的作用、功能、组成、特性及类型等，给出操作系统的宏观概貌。

2.1.1　操作系统的形成与发展

为了提高计算机资源的利用率和方便用户使用操作，从 20 世纪 50 年代至今，操作系统逐渐形成与发展，经历了从简单到复杂，从低级到高级的发展过程。

一、手工操作阶段

早期的计算机工作时，编程人员使用机器语言或汇编语言书写程序，通过人工操纵计算机控制台上一系列复杂的开关，控制和监视程序的装载、运行及输入输出等操作，使计算机的操作繁琐、易错、低效，难于掌握。

特别是装入和运行一个程序全部在人工控制下执行，且用户独占计算机的全部资源直至作业完成，因此 CPU 常要等待人工操作，90%以上的时间空闲，利用率很低。

为了解决计算机资源利用率低的突出矛盾，出现了早期的批处理系统。

二、早期批量处理阶段

批量处理的基本思想是减少两个作业间的人工干预，事先把用户作业组合排序成一批作业，利用常驻内存的监督程序，将作业逐个投入连续运行。由于实现了作业的自动转换，从而缩短了作业之间的交接和准备时间，这种系统称为联机单道批处理系统。

由于 I/O 设备是联机的，输入和输出均由中央处理机控制完成。因此联机单道批处理系统并没有解决慢速的输入/输出和快速的中央处理机之间串行工作的矛盾，因等待输入/输出使处理机大部分时间仍处于空闲状态。为了提高处理机的利用率，在批处理系统中引入脱机输入/输出技术，形成了脱机批处理系统。

脱机批处理系统由卫星机和主机组成。低档的卫星机完成输入/输出的工作，作业在主机上运行。这种工作方式使中央处理机与输入/输出等外部设备可以并行工作，从而提高了处理机的利用率。其工作原理如图 2.2 所示。

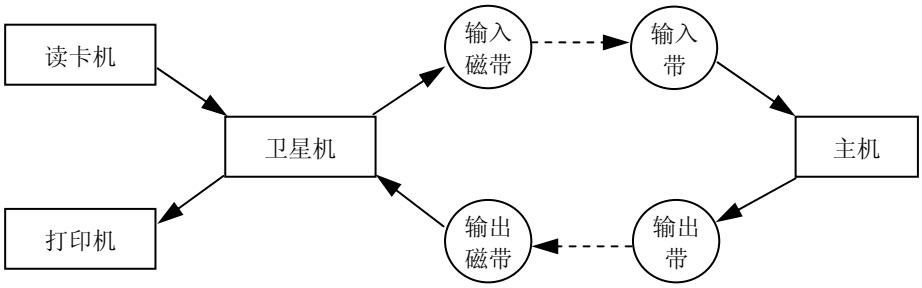


图 2.2　脱机批处理工作方式

三、管理程序阶段

脱机批处理虽缓解了 CPU 与 I/O 设备速度不匹配的矛盾，提高了 CPU 的利用率，但脱机处理增加了用户的等待时间。

20 世纪 60 年代，计算机硬件由于通道（Channel）技术的引入和中断（Interrupt）技术的发展而取得了突破性进展。通道是一种专用于控制输入输出设备的小型处理机，又称 I/O 处理机。与中央处理机相比通道的速度较慢，价格便宜，可借助于缓冲技术实现通道与中央处理机之间的并行工作。

图 2.3 描述了 CPU 的有限中断处理过程。中断是指当中断源向 CPU 发出中断请求（如 I/O 请求），CPU 接到外部信号后，进行中断响应：一是立即停止原来的工作，并保存 CPU 现场，以便处理完中断以后能恢复原来的断点继续工作；二是要识别中断类型，将中断交给不同类型的中断处理程序去处理。之后，CPU 退出中断，恢复 CPU 现场。

由于中央处理机与 I/O 通道的并行操作需要一个软件系统来协调，这即是中断处理程序

和 I/O 控制系统（IOCS），又称为管理程序或执行程序。它同时还起着负责作业运行的自动调度，控制管理各种编译程序、装配程序及子程序库及用户程序和 I/O 操作的作用。



图 2.3　CPU 的有限中断处理过程

所以管理程序是扩展的监督程序，增强了系统的保护能力，常驻内存，亦称为执行程序。即是操作系统的雏形。

四、多道程序设计与多道批处理系统

虽然管理程序解决了 CPU 与 I/O 的并行操作，但由于作业仍然是串行执行的，在单道程序系统中，内存中只允许存在一道程序。例如，执行一个计算量很少而需要输入大量数据的作业，CPU 仍然由于要等待通道完成输入任务而处于空闲状态，如图 2.4 所示。

为解决上述矛盾，出现了多道程序设计技术，即在内存中允许同时存放若干道相互独立的程序，当一道程序的输入、输出由 CPU 交给通道执行后，CPU 则去运行其他程序，因此各道程序“走走停停、交替执行”，共享各种软硬件资源。图 2.5 以 4 道程序为例，描述了多道程序的执行情况。

从宏观上看，多道程序“同时”处于运行状态，称为“多道程序”并发执行，而从微观上来说，任一特定的时刻，单 CPU 系统只能运行一道程序。由于应用通道、中断技术及多道程序设计技术，大大提高了 CPU 的利用率，这种并发性和共享性要求有更完善、更复杂的管理系统，并为用户提供更方便的操作，于是便形成了现代操作系统。

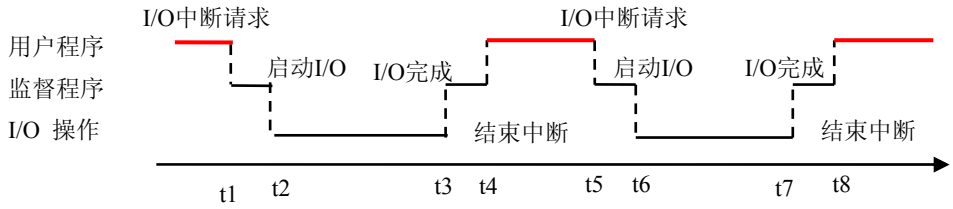


图 2.4　单道程序系统的运行

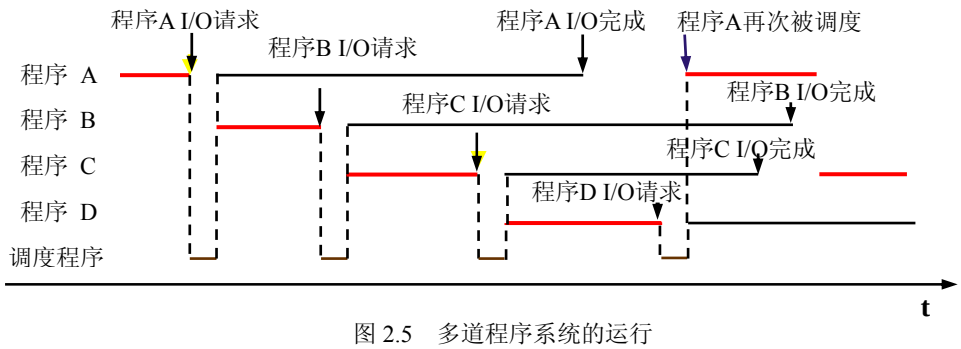


图 2.5　多道程序系统的运行

2.1.2 操作系统的功能

从两个不同的角度讨论操作系统的功能；一是从资源管理的角度看，操作系统的功能是协调、管理计算机的软硬件资源，提高它们的利用率；二是从用户的角度看，操作系统为用户提供良好的用机环境和服务。

下面从资源管理和用户接口的观点出发说明操作系统的主要功能。

一、处理机管理

计算机系统资源按性质分为四大类：CPU、内存、外部设备及信息文件，其中 CPU 是最重要、最宝贵的资源。尤其在多道程序系统中，CPU 是各道程序争夺使用的资源，因而处理机管理的基本任务是按一定的调度策略，对处理机进行分配及资源回收。在多道程序系统中，处理机的分配和运行是以进程为单位的，因此处理机管理又可归结为对进程的管理。有关进程的概念在 2.2.1 小节中介绍。

进程管理应具有以下功能：

①进程控制　基本功能为建立、撤销进程，控制进程在不同状态间的转换。

②进程同步　协调系统中并发执行的进程，控制它们以互斥方式或同步方式访问共享资源，协调进程的运行使其合作完成同一任务。

③进程通信　对相互合作完成同一任务的进程，彼此间必须交换信息，实现进程间通信。

④进程调度　控制协调各进程对 CPU 的竞争使用，按照某种调度策略实现对 CPU 的分配和回收。

二、存储器管理

存储器管理主要涉及内存管理，其任务是为多道程序的执行提供必要和良好的环境，提高存储器的利用率，并为用户提供足够大的存储空间。具体功能有：

①内存分配与回收　为各道程序分配内存空间，在使用完后对内存空间进行回收。这是存储管理最基本的功能。内存分配方式分为静态分配和动态分配两种。

②存储保护　保证各道程序只能在自己的存储区内运行，互不干扰，保护用户的程序和数据不被破坏。

③地址映射　内存空间是所有内存物理地址（可访问的实地址）的集合，所有的程序地址都是逻辑地址，必须将逻辑地址转换为物理地址程序才能执行，这一转换过程称为地址映射。

④内存扩充　为保证用户程序对大存储空间的要求，引入虚拟存储管理技术，将内、外存结合起来管理，用大容量的外存对内存进行逻辑扩充，为用户提供一个容量比实际内存空间大得多的虚拟存储空间。

⑤设备管理　计算机系统所连接的物理设备种类繁多、特性各异，设备管理的基本任务是为用户提供统一的与设备无关的接口，对各种外部设备进行调度、分配，实现设备的中断处理及错误处理等。采用虚拟设备技术和缓冲技术，尽可能发挥设备和主机的并行工作能力。

三、文件管理

文件管理是对计算机软件资源的管理，故又称信息管理。文件管理的基本任务是：实现文件符号名到物理存储空间的映射，负责文件的物理存储空间的组织分配与回收，实现文件的建立、读写及修改、删除等操作，提供对文件的保护和保密措施。

四、用户接口

从用户的角度考虑，操作系统是用户与计算机系统之间的接口，因此用户接口的任务是为用户提供一个使用系统的良好环境。操作系统的用户接口，通常有三种形式：命令接口、程序接口（即系统调用）和图形用户接口。

综上所述，操作系统的主要功能可归纳为以下方面：

1．提供用户与计算机硬件之间的接口。

2．有效地控制和管理计算机的软、硬件资源。

3．合理地调度计算机的工作流程，改善计算机系统的性能。

2.1.3 操作系统的特征

多道程序系统的引入，提高了 CPU 的利用率，但也给操作系统的设计和实现带来了许多复杂问题，为了进一步了解现代操作系统的原理和实现机制，从资源管理和用户服务的观点出发，讨论操作系统应该具有的特征：

一、并发性（concurrency）

并发性是指多个事件同时发生。在多道程序系统中，多道程序同时驻留内存，它们是轮流交替地被 CPU 所调用，从宏观上看，它们“同时”处于运行状态，称为“多道程序”并发执行。因此，并发性是一种逻辑的宏观上的同时概念。

操作系统是管理并发系统的程序集合。操作系统的并发性体现在操作系统自身各程序之间，与用户程序以及系统应用程序之间的并发执行。操作系统能有效地控制和管理系统中的各种并发活动。

二、共享性（sharing）

共享性是指多道程序或多个用户共同使用有限的资源。共享性是现代操作系统的一个最大特点，是操作系统所追求的主要目标之一，操作系统的主要职能之一就是组织好对资源的共享，使系统资源得到高效的利用。

通常共享有两种方式：

①互斥共享

也称为顺序共享，所有系统资源均可顺序共享，即在一段时间内只允许一个进程访问该资源，只有当访问结束，以及资源释放后，才允许另一个进程访问。

②并发访问

又称为同时访问，允许在一段时间内有多个进程“同时”使用某资源，但在某一时刻该资源只能被一个进程访问，即多个进程对该资源的访问是交替进行的，例如，对磁盘的

访问。

并发与共享是操作系统两个最基本的特征，资源共享是程序并发执行的必然结果，同时只有对资源共享实施有效管理，才能实现和保证“程序”的并发执行。

三、虚拟性（virtual）

虚拟的本质含义是把物理设备的一个变为逻辑上的多个，例如，在分时系统中，将一个物理 CPU 虚拟为多个逻辑上的 CPU，在存储管理中，使用虚拟技术，将一个统一编址的物理存储器变为多个逻辑上独立编址的存储器等。虚拟性是操作系统的奇妙功能，对于不同的对象实现虚拟的方法不同。

四、不确定性（nondeterministic）

不确定性是指在操作系统控制下各程序的执行顺序和每个程序的执行时间是不确定的。系统内部的各种活动是错综复杂的，如各种中断发生时间的随机性，程序运行错误及系统故障发生的随机性等。这些随机事件都造成了操作系统的不确定性。

操作系统的不确定性也是并发与共享的必然结果，操作系统必须具备随时响应和正确处理各种随机事件的能力。

并发、共享是操作系统最基本的特征，资源共享是进程并发执行的必然结果，同时只有对资源实施有效管理，才能实现和保证进程的并发执行。

2.1.4 操作系统的分类

随着计算机软硬件技术的发展，已经形成了各种类型的操作系统以满足不同应用的要求。按照操作系统的使用环境和对作业处理方式来考虑，操作系统的基本类型有：

- 批处理操作系统；
- 分时操作系统；
- 实时操作系统；
- 网络操作系统；
- 分布式操作系统；
- 嵌入式操作系统。

一、批处理操作系统（Batch Processing System）

在批处理系统中，各用户将作业提交给系统操作员（或计算机），由操作系统将作业按规定的格式组织存入磁盘的某个区域（输入井），然后按照某种调度策略选择一个或几个作业调入内存中进行处理；内存中各个作业交替执行，处理的步骤可由用户预先设定，作业输出结果也由操作系统存入磁盘的某个区域（输出井）后，再由操作系统控制输出。

1．批处理系统分类和特点

批处理系统分为单道批处理系统、多道批处理系统和远程批处理系统。下面主要介绍多道批处理系统。

作业的装入和结果的输出通常由 SPOOLing 系统（Simultaneous Peripheral Operations On Line）完成。批处理系统具有以下主要特点：

优点：系统吞吐量大，操作系统开销小，资源利用率高。

缺点：平均周转周期长，无交互功能。

“多道”和“成批”是批处理系统的两大特点。“多道”是指系统内允许有多个作业，这些作业存放在外存中，组成一个后备队列，系统按照一定的调度策略从后备队列中选取一个或多个作业进入内存运行。“成批”的特点是指在作业运行过程中不允许用户直接干预操作，作业的装入、运行，以及结果的输出都由系统自动实现，从而大大压缩了两个作业之间的转换时间，在系统中形成了一个自动转接的连续作业流。批处理操作系统所追求的目标是资源利用率高，作业吞吐量大及操作流程的自动化。

2．多道批处理系统（Batch Processing System）

在批处理系统中，为解决 CPU 利用率低的问题，也引入了多道程序设计技术。多道批处理系统具有以下特点：

（1）多道性。内存中同时存放多道相互独立的程序，宏观上多道程序并行，微观上交替执行。

（2）共享性。多道程序共享软、硬件资源，使计算机四类资源（CPU、内存、外设、信息文件），尤其是 CPU 得到充分利用。

3．多道批处理系统需要解决的问题

实现多道批处理系统需要解决以下问题：CPU 的共享、分配和回收，内存的分配与回收，各种 I/O 设备的分配与回收，文件的组织、管理，安全保护，以及作业管理。

二、分时操作系统（Time Sharing System）

分时系统允许多个用户同时联机使用计算机。“分时”即多个用户对系统资源进行时间上的分享，一台分时计算机系统联有若干台本地或远程终端，多个用户可在各自的终端上向系统发出服务请求，以交互方式使用计算机，如图 2.6 所示。

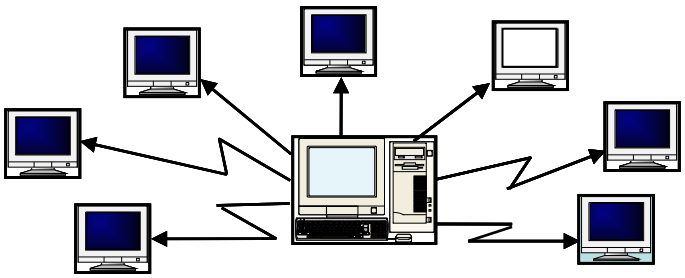


图 2.6　分时系统

分时系统具有以下三个特点：

（1）多路性　分时系统支持多个终端用户同时工作，系统对这些用户同时服务，采用按时间片轮转的方式处理用户的服务请求，即按照某个轮转次序给每个用户分配“时间片”（一个 CPU 的时间间隔，通常以毫秒计），每个作业一次只在 CPU 上执行一个时间片，这样从宏观上讲系统是同时响应每个用户作业的服务请求。

（2）交互性　系统采用联机服务方式，每个用户都可以在终端上通过与系统交互会话

请求系统服务，等待计算机的处理结果并决定下一步的处理。交互性为用户提供了比较理想的程序开发环境，因此交互性是分时系统的主要特性。

（3）独立性　尽管分时系统是轮流为多个用户服务，但用户在各自的终端上工作互不干扰，用户程序相互独立。

（4）及时性　用户请求能得到及时响应。即只要所规定的“时间片”合适，每个用户的服务请求都在可容忍的时间内得到响应，每个用户都感到“独占”计算机。

分时系统的响应时间是指用户发出终端命令到系统响应所需时间，是衡量分时系统性能的主要指标。

三、实时操作系统（Real Time System）

所谓“实时”就是“立即”或“及时”，其具体含义是指系统能及时响应随机发生的外部事件，并以足够快的速度完成对该事件的处理。

实时操作系统是为了支持如工业过程控制、飞行物及火炮发射等自动控制系统等有实时要求的应用系统而设计的。实时系统应具有以下主要特点：

（1）及时性

实时系统的主要特点是“及时响应”，对响应时间有较高要求。即每一个信息接收分析处理和发送的过程必须在严格的时间限制内完成，响应时间将直接影响到现场的过程控制质量或服务质量，特别是对过程控制，若不能及时响应则可能导致严重后果。

（2）高可靠性

由于实时系统的控制处理对象往往是重要的经济或军事目标，同时又是由现场直接控制和处理，一旦发生错误或丢失信息，往往会造成重大经济损失甚至导致灾难性后果，因此实时系统要求有高可靠性。

（3）多路性

对多路现场信息进行采集，对多个对象或执行机制进行控制，并要求对信息的采集及对对象的控制相互独立，互不干扰。

（4）简单的交互功能

实时系统支持用户使用终端，提供联机服务，但与分时系统比较实时系统提供的交互能力较弱。人与系统的交互，仅限于对某些专用服务程序的访问。

四、网络操作系统（Network OS）

计算机网络是通过通信设施将若干本地或远程的独立的计算机系统互连起来，实现信息交换、资源共享、互操作与协作处理的系统。网络操作系统除了具有基本类型的操作系统所具备的资源管理和服务功能外，还在原来各自计算机操作系统上，按照网络体系结构的各个协议标准进行开发。主要应具有以下两大功能：

（1）提供高效、可靠的网络通信功能。由于网络上各计算机（或计算机系统）的特性不同，数据表示格式及其他方面的要求不同，在相互通信时，应有许多约定，称之为通信协议或规程。

（2）提供多种网络服务功能。如远程作业录入并进行处理的服务功能，文件传输服务功能，电子邮件服务功能，远程打印服务功能等。

总之，要为用户提供网络资源共享和系统安全及多种网络应用服务。

网络操作系统可以从不同的角度进行分类，从网络地理范围上，可分为广域网操作系统和局域网操作系统。按提供的服务方式或控制方式，可把操作系统分为客户／服务器结构的网络操作系统和对等结构的网络操作系统。

五、分布式操作系统

与NOS一样分布式系统也是通过通信网络将物理上分布的具有自治功能的数据处理系统或计算机系统互连起来，实现信息交换和资源共享，协作完成任务。分布式系统是一个不共享公共存储器的处理机的集合，每个处理单元都包含有处理机和局部存储器。分布式操作系统强调的是分布处理、计算。

分布式操作系统的主要特点：

1．系统的处理控制分散到系统各处理单元。系统中的所有任务可动态分配到各处理单元。

2．系统要求“统一”的操作系统，也称为高级操作系统，实现系统操作的统一性。分布式操作系统管理分布式系统中的所有的资源分配和调度，将大任务划分为可并行执行的小任务并动态地分配到各处理单元，控制、协调信息的传输。

3．系统具有透明性，为用户提供统一界面、标准接口，实现互连计算机的协调工作。用户通过这一界面实现所需要的操作和使用系统资源，至于操作定在哪台计算机上执行，使用的是哪台机器的资源均由操作系统控制完成，用户不必知道。

4．系统以网络为基础，是逻辑上的紧密耦合，而不是物理上的松散耦合。

由于分布式系统更强调分布式计算和处理，因此对于多机合作和系统重构、坚强性和容错能力有更高的要求。更短的响应时间、高吞吐量、高可靠性是分布式操作系统所追求的目标。

由于网络操作系统与分布式操作系统有很多相似之处，特别要注意它们的区别。表2-1对它们之间的主要特性进行了比较。

表2-1　网络操作系统与分布式操作系统比较

	网络操作系统	分布式操作系统
网络支持	硬件连接，系统拓扑结构，通信控制基本相同	
分布性	OS的控制、处理功能相对集中（C/S）	OS处理控制功能是分布式，均匀分布在各站点
并行性	无任务分配功能，按照“协议”开发	统一的操作系统，多处理单元，将多个任务分配到各处理单元，并行执行
透明度	一定的透明度	隐藏内部细节，如物理位置并发控制，任务分配执行
共享性	各站点资源主要独享。共享资源一般在网络服务器中	各站点资源全系统共享，集中管理
健壮性	由于系统资源集中，潜在的不可靠	健壮性强，可用性、可靠性好，系统重建功能强

六、嵌入式操作系统

嵌入式操作系统（Embedded Operating System）是运行在嵌入式系统环境中，对整个嵌入式系统以及它操作、控制的各种部件装置等资源进行统一协调、调度、指挥和控制的系统软件。

近年来随着嵌入式系统的应用不断扩大，对嵌入式操作系统的研究已成为操作系统的热门研究课题。

小　　结

本节介绍了操作系统的发展历史、特点及功能。从 20 世纪 50 年代至今，操作系统从早期的批处理系统发展到分时、实时、网络及分布式等现代操作系统。

从资源管理及用户的角度看，操作系统的功能一是协调、管理计算机系统中的所有软、硬件资源，合理地组织计算机的工作流程，提高资源利用率；二是为用户提供一个使用计算机的良好环境和友好的接口。

具体可从五个方面来说明操作系统的功能：（1）处理机管理；（2）存储管理；（3）设备管理；（4）文件管理；（5）用户接口。

现代操作系统具有并发性、共享性、虚拟性和不确定性等基本特征。按照操作系统的使用环境和对作业的处理方式考虑，操作系统的类型有：批处理操作系统、分时操作系统、实时操作系统、网络操作系统、分布式操作系统及嵌入式操作系统。各种类型的操作系统满足不同的应用要求。

2.2　处理机管理

在现代计算机系统中，为了提高系统的资源利用率，CPU 将不为某一程序所独占，而为多道程序共享，通常采用多道程序设计技术。在多道程序系统中，处理机管理归结为对进程的管理。“进程”是操作系统最基本，也是最重要的概念之一。进程的概念对操作系统的理解、描述和设计都具有极其重要的意义。

2.2.1　进程的概念

在引入进程概念之前，先讨论在单道程序环境下程序的顺序执行和在多道程序环境下程序的并发执行所具有的不同特性，从而理解引入进程的必要性和重要性。

一、程序的顺序执行与并发执行

在单道程序系统中，一段时间内只有一个用户程序运行，我们把一个具有独立功能的程序独占 CPU 直至得到最终结果的过程称为程序的顺序执行。

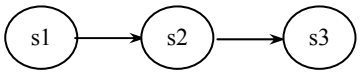
如：有三条语句：

s1： a=x+y

s2： b=a-5

s3： c=b+1

执行顺序用有向图表示：



当程序顺序执行时具有以下特征：

（1）顺序性

处理机操作严格按照程序所规定的顺序执行，每个动作的执行都以前一个动作的结束为前提条件。即程序执行在时间上严格按前后相继的次序操作。

（2）封闭性

程序执行的最终结果只由给定的初始条件决定，不受外界因素的影响。程序独占资源，只有程序本身所规定的操作才能改变这些资源的状况。

（3）可再现性

顺序执行的最终结果与程序运行的速度无关。无论何时执行程序，只要输入的初始条件相同，都能得到相同的结果，与执行过程和时间无关。

多道程序系统中，为增强系统的处理能力，提高系统资源的利用率，广泛采用多道操作技术。程序的并发执行是指在逻辑上相互独立的一组程序在执行时间上的相互重叠，即一个程序段的执行尚未结束，另一个程序段的执行已经开始。程序的并发执行具有与程序的顺序执行完全不同的特征，主要表现在：

（1）并发执行的程序间相互约束性

由于程序并发执行，在任一时刻系统中存在着多个同时进行的活动。各并发执行的程序共享计算机的资源，所以本来并无逻辑关系的程序之间产生了相互制约的关系。而有些并发执行的程序需要相互通信共同协作完成同一任务，程序间的相互依赖、相互制约的关系使并发执行的程序具有“执行—暂停—执行”的活动规律，相互穿插交替运行。程序并发执行中，一个共享程序段允许多个用户作业调用，从而形成多个“计算”，不存在程序顺序执行时程序与计算的一一对应关系。

（2）资源的争夺与共享

系统中的资源不再为单个程序所独占，而由若干道并发执行的程序所共享。由于每个并发程序是作为一个独立的实体去获得系统资源，在执行过程中可能有多个并发程序申请同一资源，这就带来了资源分配的动态性，因此资源的状态是由多道程序的活动所共同决定的，程序的运行不再具有封闭性和可再现性，并发程序的执行结果与其执行的相对速度有关。

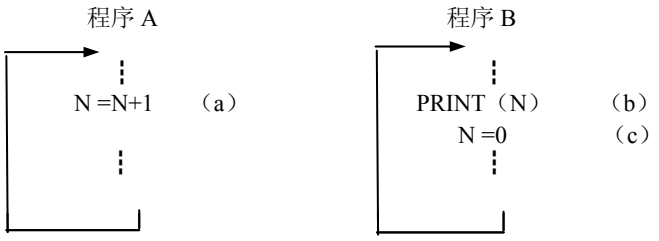
因此，程序并发执行时具有以下特征：

（1）间断性　各道程序“走走停停、交替执行”，不再是一道程序顺序执行完成。

（2）无封闭性　是指对资源的使用已经不由某道程序独占，资源的分配由多道程序共同决定。

（3）不可再现性　不可再现性是程序并发执行时的间断性及失去封闭性的必然结果，即程序的执行结果与其执行速度有关。

例如，有以下程序段，N 是程序 A 和程序 B 的公共变量，初值为 N=n，（a）、（b）、（c）为三条语句的标号。



若分别按照以下不同顺序执行，得到不同的结果：

① 执行顺序：（a），（b），（c）　N 值：n+1，n+1，0

② 执行顺序：（b），（a），（c）　N 值：n，n+1，0

③ 执行顺序：（b），（c），（a）　N 值：n，0，1

即多道程序在并发执行时出现不可再现性，这显然是不允许的。从上述程序并发执行的特点可见，程序的概念是一个静态概念，已无法描述程序动态执行过程中的并发活动，故需引入一个新的概念来描述，这就是“进程”。

二、进程的概念

进程的概念是在 20 世纪 60 年代初期由 MIT 的 Multics 系统和 IBM 的 TSS / 360 系统首先引用，从那以后人们对进程下过各种定义，例如：

（1）进程是一个可以并发执行的计算部分（S.E.Madnick，J.D.Donovan）；

（2）进程是一抽象的实体，它在执行的过程中需要分配和释放各种资源（P.Denning）；

（3）进程是一次可以进行调度的独立的执行活动（E.Cohen，D.Jofferson）。

归纳各种定义的本质，给出进程的定义：

进程是可并发执行的程序在给定数据集合上的一次执行过程，是系统进行资源分配和调度的一个独立的基本单位和实体，是执行一个映像程序的总环境，是一个动态的概念。

进程是为了更好地描述并发执行程序的动态特性而引入的，而进程与程序既有区别又有联系，主要表现在以下方面：

① 进程是指一次运行的活动，是执行程序的动态过程，是一种动态概念。程序是一组有序的指令，是进程运行的静态文本，是一种静态的概念。若把程序比喻为电影的拷贝，则进程就是电影的放映过程。

② 一个进程可以执行一个或多个程序。如编译 C 程序时，同时执行预处理、语法和语义分析、代码生成和优化等几个程序；反之，一个程序也可能被多个进程执行。

③ 程序可以作为一种资源以文件的形式长期保存；而进程只是一次执行过程，具有生命期。

进程具有以下基本特征：

（1）动态性

进程是程序的一次执行过程，具有生命期，它可由系统创建并独立地执行，在执行过程中可能由于某种原因而被暂时挂起，当条件满足时又可继续执行，直至完成而被撤销，所以进程是一个动态的概念。

（2）并发性

进程是可并发运行的基本单位，不同进程之间的动作在时间上可以重叠，即一个进程的第一个动作可以在另一个进程的动作结束之前开始。由于共享资源，进程间相互约束，各个进程都以“走走停停”的状态执行。从宏观上看，它们可以“同时”执行。由于共享资源，进程间相互约束，相互依赖。

（3）独立性

进程是程序和数据集合的实体，是系统分配资源、能被处理机调度运行的基本单位，各进程之间相对独立。

（4）异步性

各进程按照各自独立的、不可预知的速度异步向前推进。

三、进程控制块

一个进程是一个程序对某个数据集的执行过程，是分配资源的基本单位。对进程的描述应该反映出进程的存在及其变化。通常进程由程序及其相应的数据集合、进程控制块三部分组成。

进程控制块 PCB（Process Control Block）是操作系统为了反映进程的动态特性，便于系统控制和描述进程的活动过程而专门定义的一种数据结构。用于记录和描述进程执行情况和状态变化。

PCB 可包含四类信息：

（1）进程标识信息　可包括外部标识信息、内部标识信息、进程家族标识。

（2）处理机状态信息　包括通用寄存器、指令计数器、程序状态字（PSW）和用户栈指针等。

（3）进程调度信息　包括进程状态、进程优先级、其他调度信息和等待事件等。

（4）进程控制信息　包括程序数据地址、进程同步及通信信息、资源清单、链接指针等。

PCB 所包含的信息类型和数量随操作系统而异，但一般都包括两类信息：调度信息和现场信息。

1．调度信息描述进程的当前状况：包括进程名或进程标识、当前状态、优先级、资源清单、家族关系、队列指针及其他调度信息，供进程调度时使用。

2．现场信息刻画了进程运行的情况：记录那些进程在运行过程中会改变的寄存器，如程序状态字、时针、基址寄存器等，一旦进程的执行过程被中断，必须立即将中断时的各类信息的内容记入现场信息，以便下一次进程再被 CPU 调度时能继续运行。

PCB 是进程存在的唯一标志。当系统创建一个进程时，即是在系统区中为该进程创建一个 PCB，对进程的管理实际上就是对 PCB 进行控制和管理，当进程撤销时，系统回收它的 PCB。

四、进程状态及其转换

1．进程状态

进程在运行过程中，处于不同的状态，描述了进程的当前行为。进程的动态性也表明进程在其生命周期中需要经历一系列离散状态。运行中的进程可以有三种基本状态：执行

状态、就绪状态和等待状态。

（1）就绪状态（Ready）

处于就绪状态的进程已经获得了除 CPU 以外的其他所需要的资源，具备了执行的条件，但由于 CPU 正被其他进程占用而暂时不能执行。只要一旦被调度，获取了处理机的使用权，便可立即投入运行。往往系统中处于就绪状态的进程可以有多个。

（2）执行状态（Executing）

当前进程（Current Process）占有 CPU，并正在执行。显然，在单 CPU 系统中只有一个进程处于执行状态。

在某些操作系统中，一个进程在其生命期内的执行过程中，总要涉及用户程序和操作系统内核程序两部分。因此，进程的执行状态又可进一步划分为用户执行状态（简称用户态或目态）及系统执行态（简称为核心态或管态）。

（3）等待状态（Wait）

又称为阻塞状态（Blocked），是指进程由于等待某个事件（条件）的发生而暂时不能运行的状态。例如，两个进程同时申请使用某个资源，未占用该资源的进程处于等待状态，必须等到该资源被释放后才可以去使用它。等待状态还可根据等待事件的不同种类分为不同的子状态，例如，内存等待、I/O 等待、文件等待等。系统中常有多个等待进程。

2．进程状态转换

进程的状态反映进程执行过程的变化。进程在运行过程中，由于它自身的进展情况及外界条件的改变，三种基本状态会自动转化，图 2.7 表示了三种基本状态之间的转换及其典型的转换原因。

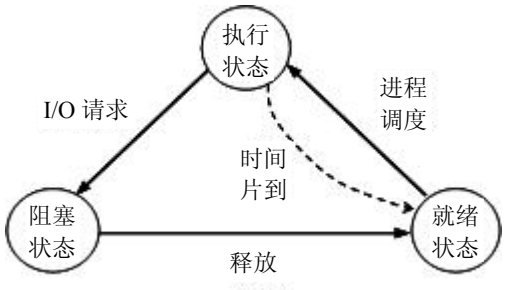


图 2.7　进程状态转换

进程一经创建并具备运行条件后即处于就绪状态；处于就绪状态的进程排在就绪队列中，等待处理机调度。进程一旦被处理机调度便处于执行状态，在执行过程中，若进程因请求输入/输出或其他服务则转入等待状态。一旦所等待的条件完成，具备了执行条件就进入就绪状态。此外，处于执行状态的进程若用完了所分配的时间片，而未完成执行任务就从运行状态转变为就绪状态，等待处理机的下一次调度。

3．细化的进程状态

系统在运行过程中，如果发生系统故障，或工作负荷达到高峰，需要调整负荷，又或需要对系统进行检测，系统将自动停止某些进程执行使系统恢复正常。有时需要人为停止某进程执行，以便对其进行分析研究。出于上述需要，要对进程状态作进一步细化。通常增加“挂起”（Suspend）状态。

所谓“挂起”状态，是使被挂起的进程处于静止状态。挂起状态分为静止就绪（Readys）和静止阻塞（Blockeds），处于“挂起”状态的进程不能实施它所在状态的操作。未被挂起的进程处于活动状态，分为活动就绪（Readya）和活动阻塞（Blockeda）。被挂起的进程一般有三种情况：

（1）被挂起的进程如果正在执行，则立即停止执行，变为静止就绪（Readys）状态。

（2）被挂起的进程若处于就绪状态，则停止调度，并回收资源，转变为静止就绪（Readys）状态。

（3）被挂起的进程如果处于阻塞状态，则停止等待条件的发生，转变为静止阻塞（Blockeds）状态。

图 2.8 描述增加了“挂起”状态的进程状态，以及引起状态改变的典型事件。

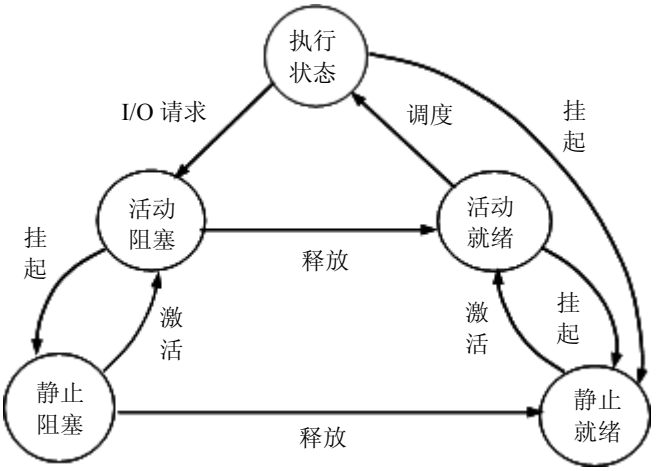


图 2.8　细化的进程状态图

2.2.2　进程控制

进程有一个从创建到撤销的生命周期，进程控制就是对进程在其生命期的各种活动及状态转变实施有效的控制。对进程的控制一般是由组成操作系统内核的各种原语（primitive）来实现的。所谓“原语”是机器指令的延伸，是非进程模块，原语操作的执行过程不可间断，现代操作系统中为保证系统的“稳定性”和“安全性”，原语用微代码实现。

用于进程控制的原语包括：创建进程、撤销进程、挂起进程、激活进程、阻塞进程、唤醒进程及改变进程优先级等。

一、创建原语create（ ）

一个进程的创建通常有两种方式：由系统程序模块统一创建和由父进程创建。无论用哪种方式创建，都必须调用创建原语来实现。

创建一个进程的主要任务就是为进程建立一个进程控制块（PCB），将调用者提供的信息填入该 PCB，具体算法步骤是：

1．向 PCB 集合申请一个空闲的 PCB，并获取其内部标识；

2．将有关信息填入 PCB；

3．将该进程置为就绪状态，并插入就绪队列。

二、撤销原语destroy（）

当进程完成任务或产生异常中断，应立即撤销，并释放其占有的所有资源。其具体算法步骤是：

1．根据进程的外部标识检索 PCB 集，找到要撤销的 PCB；

2．从队列中撤销该进程及其所有子、孙进程。通常是在一递归过程中实现；

3．释放被撤销进程所占的所有资源，撤销进程控制块 PCB。

三、阻塞原语block（）

进程在执行过程中，需要执行 I/O 操作或等待某些条件的发生，则由该进程调用阻塞原语把进程从执行状态转换为阻塞状态，算法步骤是：

1．中断 CPU 执行，将 CPU 的当前状态保存到 PCB 的现场信息中；

2．将进程的当前状态置为阻塞状态；

3．将进程的 PCB 插入到该事件的阻塞队列中。

四、唤醒原语wakeup（）

当处于阻塞状态的进程所等待的事件发生后，使用唤醒原语将进程从等待状态转换为就绪状态，具体算法步骤是：

1．在等待队列中找到该进程，将其 PCB 的当前状态置为就绪；

2．将进程从等待队列中撤销，并插入到就绪队列中。

五、挂起原语suspend（）

进程在执行过程中，根据用户或系统的某种需要，调用挂起原语将进程从活动状态转换为静止状态。

算法步骤：

1．根据外部标识检索 PCB 集，找到要挂起的 PCB 块；

2．检查被挂进程的当前状态：

① 若当前状态为执行态，将其变为静止就绪，插入静止就绪队列，并转调度程序，重新调度；

②若当前状态为活动就绪，将其变为静止就绪，插入静止就绪队列；

③若当前状态为活动阻塞，将其变为静止阻塞，插入静止阻塞队列。

六、激活原语active（）

当处于静止状态的进程所等待的激活事件发生后，使用激活原语将进程从静止状态转换为活动状态。

算法步骤为：

1．确定要激活的进程；

2．检查被挂进程的当前状态；

3．将进程从静止状态转换为相应的活动状态。

2.2.3 进程的互斥与同步

并发性、共享性及异步性是进程的主要特征。进程的并发执行使系统资源得到了充分利用，提高了系统的效率，但由于资源有限又导致了进程之间的资源竞争和共享。进程又以各自独立的、不可预知的速度异步运行，因而各自使用资源的时间是不确定的，这种共享资源往往引起与时间有关的错误，为了防止这种与时间有关的错误，必须对进程的异步运行在时间上施加某种限制，对并发执行的进程进行协调和控制。这种协调和控制主要表现在进程的互斥和同步两个方面。

一、并发进程间的约束关系

并发执行的进程之间，通常有两种关系：

1．互斥关系　进程之间彼此无关，但是由于竞争使用同一共享资源而产生了相互约束的关系，称为进程的互斥。这是进程之间一种间接的相互制约关系。

2．同步关系　多个并发执行的进程，在共同协作完成一项任务的过程中，相互约束，进程之间通过在执行时序上的某种限制而达到相互合作。这种约束关系称为进程的同步，是一种直接的相互制约关系。

进程的同步与互斥虽然是两个既有区别又有联系的概念，但从本质上看并发进程的异步执行都必须按一定的相互约束的时序进行，因此统称为“进程同步”。

显然，必须解决好进程的同步问题，才能保证并发进程的正常执行。

二、进程的互斥与临界区

进程的互斥是由多个进程竞争同一共享资源而产生的相互制约的关系。例如，A、B 两进程共享一台打印机，若 A 进程在使用打印机的过程中，B 进程又插进来打印，将会出现 A、B 两进程的打印结果混杂在一起的错误。因此对打印机这种共享资源，只能为一个进程所独用，只有当该进程打印结束并释放打印机后，其他进程才能使用。

这种因共享资源而产生的制约关系称为进程的互斥（mutual exclusion）。

以互斥关系使用的共享资源称为临界资源（critical source）。临界资源具有一次只允许一个进程使用的属性。临界资源既可以是硬件资源，如打印机、显示器、软驱等，又可以是软件资源（公共变量、数据等），共享临界资源的进程必须相互排斥。

每个进程中访问临界资源的那段代码称为临界区。进程对临界资源互斥访问的代码由以下部分组成：

① 进入区（entry section）	申请进入临界区
② 临界区（critical section）	访问临界资源
③ 退出区（exit section）	退出对临界资源的访问
④ 剩留区（remainder section）	执行进程其他部分

三、进程的同步

多个并发进程在协作完成某项任务的过程中，相互间需传递某种信息，一个进程在没有获得合作进程提供的必要信息之前，不能超越某一执行点。例如，有 A、B 两个进程，A

进程负责从键盘输入数据到缓冲区，B 进程负责从缓冲区读出数据进行计算。显然，B 进程只有等待 A 进程把数据输入到缓冲区，并向 B 进程发出信号后才能进行计算，而 A 进程只有等待 B 进程取走缓冲区的数据，向 A 进程发出信号后才能继续从键盘向缓冲区中送数据。只有 A、B 两进程协同才能完成读数据及计算的工作。A、B 两进程并发执行过程如图 2.9 所示。

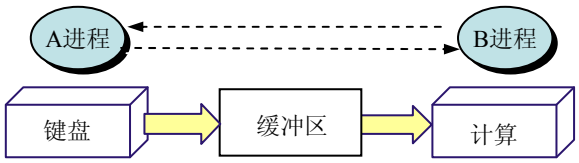


图 2.9　两进程并发执行

也可以用如图 2.10 所示的算法模型来描述 A、B 两进程并发执行的情况。

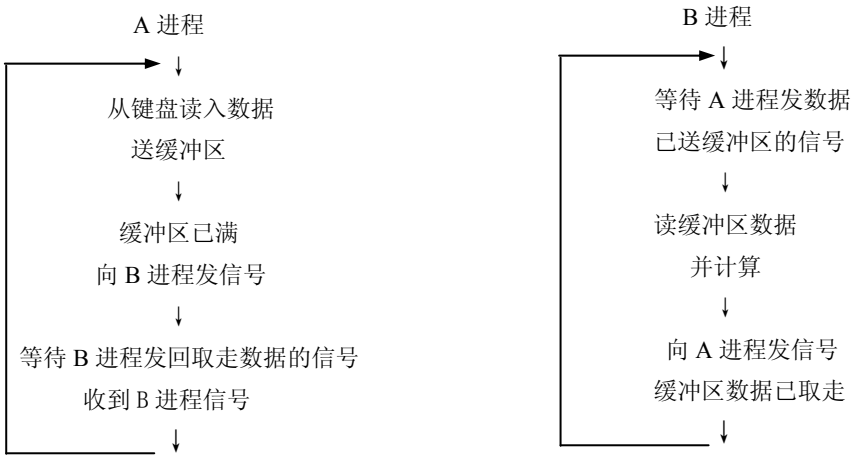


图 2.10　两进程并发执行的算法模型

进程之间通过在执行时序上的某种限制而达到相互合作。这种因相互合作而产生的限制关系称为进程的同步。这是一种直接的协同工作关系。

进程的同步与互斥是两个既有区别又有联系的概念，但从本质上看，并发进程的异步运行都必须按一定的相互约束的时序进行，因此，从广义上说，互斥也是一种同步关系。

四、进程的同步机制

通常把实现进程同步与互斥的机制称为同步机制。因此进程的同步机制就是要解决在进程异步运行时，在时间上施加某些限制，使其共享资源的操作与时间无关。或称在执行时序上施加某种限制，达到彼此间的合作。实现进程的同步机制的基本思想是：对临界资源“加锁”。

系统的同步机制为了解决对临界资源的访问，应遵守以下同步机制的准则：

① 空闲让进　无进程处于临界区内，可让一申请进入临界区的进程进入；
② 忙则等待　若临界区内有进程，其余申请进入临界区的进程必须等待；
③ 有限等待　进程进入临界区的要求必须在有限时间内得到满足；
④ 让权等待　等待进入临界区的进程，若占有 CPU 必须立即释放。

同步机制可以由硬件实现，也可由软件实现，例如，可借助一条硬件指令 Test and Set（简称 TS 指令）来实现同步机制，但该同步机制虽然实现了对临界资源的互斥访问，但执行过程却未“让权等待”，即进程在等待进入临界区时，始终在对 TS 指令进行测试，称为“忙等待”。

2.2.4　信号量机制与 P、V 操作

信号量机制是一种比用 TS 指令实现的同步机制更加优越的同步机制，解决了“忙等待”的问题。

一、什么是信号量

并发进程间的相互制约关系从本质上说是由于争夺和共享资源而产生的。1986 年荷兰著名计算机科学家 Dijkstra 把互斥的关键含义“资源”抽象为信号量（Semaphore），引入在信号量基础上的同步操作原语： P 操作、V 操作。提出了一种典型的同步机制，即信号量机制，这是一种比 TS 指令更加完善的同步机制。P 和 V 分别是荷兰语“等待”和“发信号”两词的首字母。

设信号量为 S，用信号量 S 的值表示共享资源的使用情况。S 可以取整数值，也可是记录类型。以记录类型的信号量机制为例进行讨论。

记录类型的信号量的定义：

```
type    semaphore = record
        value：integer;                信号量值
        L：list  of  process           信号量等待队列指针
        end;
```

其中：

信号量值——表示某种资源的数量。

等待队列指针——当信号量值为负时，表示该类资源已分配完，信号量值的绝对值表示排在等待队列中等待该类资源的进程数。

L——为指向该信号量等待队列的指针。

二、P、V操作

P 操作和 V 操作都是原语，定义如下：

定义：VAR　S：Semaphore；

1．P 操作（wait 原语）

P（S）——表示对信号量 S 进行 P 操作，每作一次 P 操作，申请分配一个单位的资源。

（1）S.value = S.Value－1；

（2）若 S.Value ≥0，则进程继续执行。

若 S.Value＜0，则进程阻塞，并进入等待队列（L）。

2．V 操作（Signal 原语）

V（S）——表示对信号量 S 进行 V 操作，每作一次 V 操作，释放一个单位的资源。

（1）S.value＝S.value+1；

（2）若 S.Value > 0，则进程继续执行。

若 S.Value ≤ 0，则释放等待队列中的一个进程，使之获取资源，转为就绪状态。其中，信号量 S 的值表示资源的数量，当 S＞0 时，S 的值表示资源可分配的数量；当 S.Value ≤0 时，S 的绝对值表示等待队列中进程的数目。

按照 P、V 操作的定义，每执行一次 P 操作，表示要申请分配一个单位的资源，而每执行一次 V 操作，表示释放一个单位的资源。

三、用P、V操作实现进程间的互斥

例 1 设有 A、B 两进程，竞争使用某临界资源，利用信号量机制实现进程间互斥。

解 设 mutex 为公共信号量，即互斥信号量。初值：mutex.Value = 1。

图 2.11 给出利用 P、V 操作实现进程互斥的算法模型：

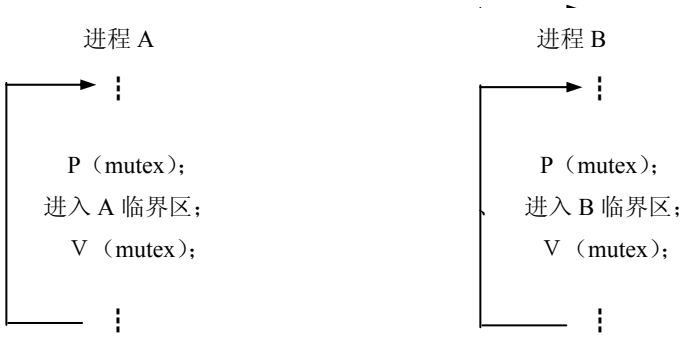


图 2.11 P、V 操作实现进程互斥

由于公共的互斥信号量 mutex 初值为 1，任一进程先执行 P 操作，将能申请到该资源，进入临界区。若系统中另一进程再申请该临界资源，则因 mutex≤ 0 而被阻塞，并进入该临界资源的等待队列，直到申请到该资源的进程执行 V 操作后，释放资源并唤醒等待队列中的一个进程。由于每次只允许一个进程进入临界区，有效地保证了对临界资源的互斥操作。执行过程中 mutex.Value 的值，在 1，0，－1 之间变化。下面是利用 P、V 操作实现进程互斥的算法：

```
Var   mutex : semaphore:=1;
  Begin
     parbegin
        process   1: begin
           repeat
              P(mutex);
              critical   section;
```

```
            V(mutex);
              remainder  section;
        until    false
    end
  Process 2:begin
        repeat
            P(mutex);
            critical  section;
            V(mutex);
              remainder  section;
        until    false
    end
  parend
End.
```

其中，parbegin 与 parend 表示它们之间的进程并发执行。

四、用P、V操作实现进程间的同步

利用信号量同样可以方便地实现合作进程间的同步。先考察一个进程间同步的实例：

例 2　如图 2.12 所示，设有一个打印进程（IOP）和一个计算进程（CP），当缓冲区为空时，计算进程才能不断将计算结果放入缓冲区，打印进程必须在缓冲区中有计算结果时才能不断取出数据进行打印。

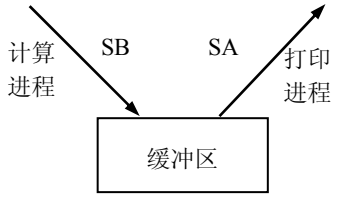


图 2.12　进程同步问题

分析：打印进程与计算进程之间有两个约束：

1）计算进程只有当缓冲区为空时，才能放入计算结果。

2）打印进程只有当缓冲区有结果时，才能从缓冲区取出计算结果打印。

设信号量 SB 和 SA 分别为计算进程和打印进程的私有信号量，初值均为 0，用于控制打印进程与计算进程之间两个约束。SB 用于判断缓冲区是否为空，SA 用于判断缓冲区中是否有数据。

SB——控制计算进程向缓冲区送计算结果。当 SB = 0 表示可以放入计算结果。

SA——控制打印进程从缓冲区取计算结果打印。当 SA≥0 时表示可以打印。

下面给出利用 P、V 操作实现计算进程与打印进程同步的算法模型（见图 2.13）。

图 2.13 中 T1、T2、L1、L2 为语句标号，显然，只有 CP 执行到 T2，IOP 才能执行 T1，即 T2 是 T1 的直接前趋。只有 IOP 执行到 L2，CP 才能执行 L1，即 L2 是 L1 的直接前趋。

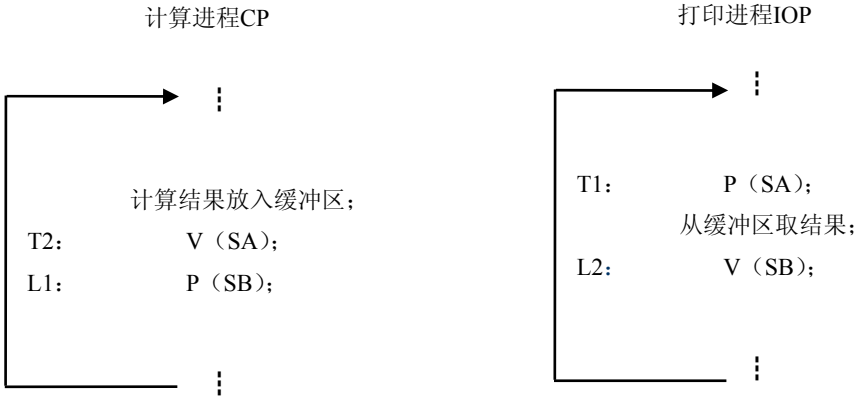


图2.13　计算进程与打印进程同步的模型

计算进程与打印进程同步的算法描述如下：

```
Var SA,SB:semaphore:=0,0;
      begin
         parbegin
       CP: begin
              repeat
                  computer   next   number ;
                  add     to   buffer;
                  count := count−1;
T2:                   V(SA);
L1:                   P(SB);
                   if   count=0   then
                        destroy(cp);
              until     false
              end
IOP : begin
              repeat
T1:                          P(SA);
               take from buffer;
L2:                           V(SB);
                  print   last   number;
                  if   count = 0 then
                        destroy(IPO);
               until     false
```

end

parend

end

例 3　如图 2.14 所示，有三个进程，进程 get 从输入设备上不断读数据，并存入缓冲区 buffer1；进程 copy 不断将 buffer1 的内容复制到缓冲区 buffer2，进程 put 则不断取出 buffer2 的内容在打印机上输出。为了使三个进程并发执行，协调工作。建立这三个进程并发执行的同步模型。

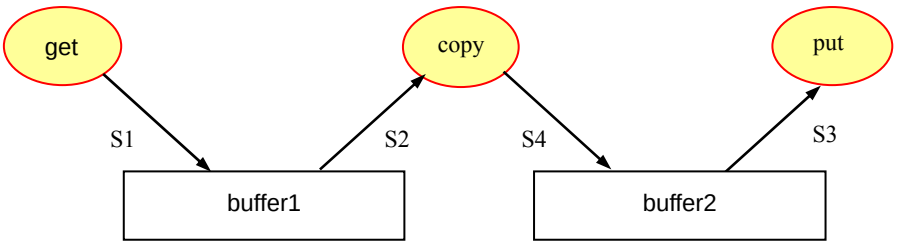


图 2.14　三个进程间的同步问题

分析：只有当缓冲区 buffer1 为空时，进程 get 才能将数据输入 buffer1。而进程 copy 则只有当 buffer1 有数据而 buffer2 为空时才能完成复制工作。进程 put 只有当缓冲区 buffer2 中有数据时才能打印输出。分别为三个进程设置 4 个信号量：S1，S2，S3，S4。

其中，S1、S4 分别表示 buffer1 和 buffer2 是否为空；S2、S3 分别表示 buffer1 和 buffer2 是否装满数据。

其初值分别为：S1＝ 1 ； S2＝ 0 ； S3＝ 0 ； S4＝ 1 。

该三个并发进程的同步模型如图 2.15 所示。

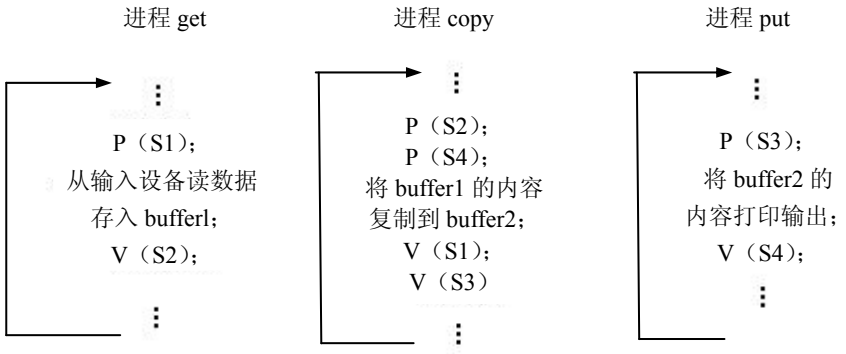


图 2.15　三个进程并发执行的同步模型

2.2.5　经典的同步问题

把并发进程的同步和互斥问题一般化，可以抽象出一类同步模型，称为经典的同步问题。本节介绍两类经典的同步问题。

一、生产者—消费者问题（Producer Consumer Problems）

在系统中各进程都申请使用和释放各种软、硬件资源。我们把使用某一类资源的进程称为该资源的消费者，而把释放同类资源的进程称为该类资源的生产者。

生产者—消费者问题描述为：

有一组消费者进程 P i（P1，P2，…，Pk）和一组生产者进程 Ci（C1，C2，…，Cm），互斥地使用由 n 个相等的缓冲区组成的缓冲池（如图 2.16 所示）。每个单元可放一个“产品”，当缓冲区有空时，生产者进程不断地将“产品”放入缓冲区内。消费者进程则不断地从缓冲区内取出“产品”。显然，生产者进程与消费者进程在使用缓冲区时有以下制约关系：

（1）至少有一个缓冲区为空，生产者进程才能做放入“产品”的操作。当缓冲池放满产品时生产者必须等待。

（2）至少有一个缓冲区是满的，消费者进程才能执行取出“产品”的操作。当缓冲池空时，消费者进程必须等待。

（3）各消费者进程与各生产者进程只能互斥地使用临界资源缓冲区。

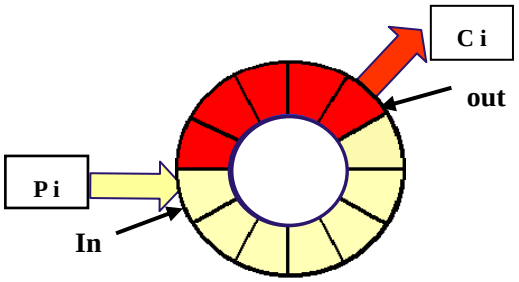


图 2.16　生产者-消费者问题

根据上述分析，设置以下信号量：

①公用信号量 mutex：初值为 1，用于实现对临界区互斥访问；

②生产者私有信号量 empty：初值为 n，表示空缓冲区数；0≤empty≤n。

生产者进程

```
生产一个产品；
    ⋮
P(empty)；
P(mutex)；
将产品放入 in 所指向的缓冲区；
in=(in+1) mod n;
V(mutex);
V(full);
    ⋮
```

消费者进程

```
    ⋮
P(full);
P(mutex);
从 out 所指向的缓冲区取产品；
out=(out+1) mod n;
V(mutex);
V(empty);
    ⋮
```

③消费者私有信号量 full：初值为 0，表示有产品的缓冲区数；0≤full≤n。

④in、out 分别指向当前第一个空缓冲区和第一个满缓冲区。in、out 类型定义为整型的子界型，其作用相当于指针。

下面给出生产者—消费者进程的算法模型：

```
Var    mutex，empty，full：semaphore :=1, n , 0；
       buffer：array  [0 ..n−1] of message;
       in, out : 0..n−1:= 0, 0；
```

注意：在生产者和消费者进程中，两个 P 操作的次序不允许颠倒，否则可能造成死锁。请读者自己分析为什么。

二、读者—写者问题

问题描述：有两组并发进程，读者进程（只读），写者进程（只写），它们共享一个文件（资源）对象。读、写规则如下：

①允许多个“读者”同时执行读操作。

②任一“写者”访问文件时，不允许其他“读者”或“写者”工作。

定义：

Readcount——记录当前正在访问文件的“读者”个数。

Rmutex——读互斥信号量。

Wmutex——写互斥信号量。

读者—写者问题算法：

```
var  Rmutex，Wmutex：semaphore：=1，1；
     Readcount ：integer：=0；
  Begin
   Parbegin
    Reader:begin
     repeat
   P(Rmutex)；                                  / 读者申请使用文件 /
       if  Readcount=0  then  P（Wmutex）；      / 阻塞写者进程 /
Readcount:= Readcount +1；
          V(Rmutex)；                           / 读者释放文件让其他读者进入 /
            ┆
          Perform  read  operation；
            ┆
          P(Rmutex)；
          Readcount:=Readcount −1；
          if  Readcount=0  then  V(Wmutex)；     / 读者退出 /
          V(Rmutex)；
     Until  false；
     End
   Writer: begin
```

```
        repeat
P(Wmutex);                              / 写者申请使用文件 /
         Perform  write  operation;
         V(Wmutex);                     / 写者释放文件 /
        Until  false;
        End
     Parend
end
```

2.2.6 进程调度

一、调度的基本概念

作业从进入系统到完成，可能要经历三级调度过程：

1. 高级调度。又称为作业调度，它决定将哪些在外存上处于后备状态的作业调入内存，准备执行。因此，有时把它称为接纳调度。

2. 低级调度。又称为进程调度，它决定就绪队列中哪个进程将获得处理机，并实际执行将处理机分配给进程的操作。执行分配处理机的程序称为分派程序。

3. 中级调度。中级调度的主要作用是在内存和外存之间进行进程交换，以解决内存紧张的问题。如它将内存中处于等待状态的某些进程调至外存对换区，以腾出内存空间，而将外存对换区上已具备运行条件的进程重新调入内存，准备运行。故又称为交换调度。

显然，作业调度是进程调度的基础，作业被调入内存后，是以进程的形式执行的。在一个 OS 中进程调度与作业调度的策略是一致的。

进程调度即处理机调度。在多道程序系统中，由于有多个进程争夺处理机，进程调度的任务是协调和控制各进程对 CPU 的使用，按照某种策略动态地将处理机分配给处于就绪状态的某个进程。由于 CPU 是计算机最重要的资源，因此进程调度的性能直接影响到 CPU 的利用率及系统性能。本节主要讨论进程调度。

二、进程调度的原因及方式

进程的调度一般是在下述情况下发生的：正在执行的进程已运行完毕，正在执行的进程由于等待某种条件的发生（如 I/O 请求），或是分时系统中执行进程的时间片已用完，或是就绪队列中出现高优先级的进程申请使用 CPU 等都可能出现进程调度。

进程调度的方式分剥夺式和非剥夺式，所谓剥夺式即就绪队列中一旦有高优先级的进程出现立即剥夺正在执行的进程的 CPU 使用权，将 CPU 转让给高优先权的进程；而非剥夺式，即一旦把 CPU 分配给某个进程，该进程将一直占有 CPU，直到由于时间片到或是进程自己进入阻塞状态时才让出 CPU。

三、进程调度的功能

进程调度应实现以下功能：

（1）记录系统中所有进程的执行情况

进程管理模块必须将系统中各进程的执行情况和状态特征记录在各进程的 PCB 中。并根据各进程的状态特征和资源需求等将各进程的 PCB 排成相应队列。进程调度程序根据 PCB 的变化来掌握系统中存在的所有进程的执行情况和状态特征，以便在适当的时机从就绪队列中选择出一个进程占据处理机。

（2）确定分配处理机的原则

这是进程调度的主要功能，即按照一定的调度策略（又称为调度算法）选择一个处于就绪状态的进程，使其获得处理机执行。不同的系统设计有各种调度选择的策略。例如，适合于分时系统的轮转法（Round Robin）、系统开销较少的静态优先级法等。这些调度策略决定了调度的性能。有关算法将在下面介绍。

（3）处理机的分配与回收

将 CPU 分配给按调度策略所选中的就绪队列中的进程。即将被调度进程的 PCB 中有关现场的信息（如程序状态字、通用寄存器等的内容）送入 CPU 相应的寄存器中。

当进程执行状态结束后，回收处理机。

四、进程调度的算法

进程调度算法应以尽可能提高资源利用率，减少 CPU 空闲时间为原则，解决以何种次序对各就绪进程进行处理机分配的问题。因此，对进程调度性能的衡量是操作系统设计的一个重要指标。评价调度算法的优劣，通常考虑以下两个指标：

①周转时间 TT（Turnaround Time）或平均周转时间 ATT（Average Turnaround Time）。周转时间是指从进程第一次进入就绪队列到进程运行结束的时间间隔，而平均周转时间是指系统中各进程的 TT 平均值。

$$\text{平均周转时间：}\ ATT = \frac{1}{n}\sum_{i=1}^{n} Ti$$

$$\text{带权平均周转时间：}\ W = \frac{1}{n}\sum_{i=1}^{n} \frac{Ti}{Tri}$$

其中，Ti——各进程的 TT，Tri——实际 CPU 执行时间。

②响应时间 RT（Response Time）。所谓响应时间是指从提交一个请求开始到计算机作出响应、显示结果的一段时间间隔。

常用的调度算法有：

1．先来先服务（FCFS）调度算法

该算法将就绪进程按进入的先后次序排成队列，并按先来先服务（First Come First Serve）的方式进行调度，即每当进行进程调度时，总是选择就绪队列的队首进程运行。

这是一种非剥夺式的调度算法，算法简单，容易实现。在一般意义下该算法是公平的，但对于那些执行时间较短的进程来说，如果它们在某些执行时间很长的进程之后到达，则将等待很长时间，服务质量差。因此，该算法一般只作为辅助的调度算法。

2．最短 CPU 运行期优先（SCBF）算法

该算法针对 FCFS 算法对短进程服务质量差，等待时间长，平均周转时间 ATT 长的缺点，最先调度 CPU 运行期短的进程。该算法 ATT 短，但因算法依赖于各进程的下一个 CPU

周期，实现较困难，通常采用近似估算的方法。例如：

设t_n为第n个实际的CPU时间，τ_n为估计的第n个CPU时间，进程的下一CPU时间为：

$$T_{n+1}=\alpha t_n+(1-\alpha)\tau_n$$

其中，系数α为控制值，$0\leqslant\alpha\leqslant 1$，为使$\tau_{n+1}$在$\tau_n$与$t_n$之间，常取$\alpha=0.5$。

根据该估算公式算出每个就绪进程的τ_{n+1}，优先调度τ_{n+1}最小的进程。该算法优于FCFS算法，但对长进程等待时间较长，且对τ_{n+1}的估算有较大误差。

3．时间片轮转（RR）算法

该算法主要用于分时系统，其基本思想是：按照公平服务的原则，将 CPU 时间划分为一个个时间片，若一个进程在被调度选中后执行一个时间片，当时间片用完后，强迫执行进程让出 CPU 而排到就绪队列的末尾，等待下一次调度。同时调度程序又去调度当前就绪队列中的第一个进程。

轮转法中，关键是时间片长度的选取。时间片太大，以至于每个进程的 CPU 周期都能在一个时间片内执行完，则轮转法蜕化成了 FCFS 法；若时间片太小，则 CPU 调度过于频繁，由于每次调度涉及保存运行现场和装入新进程的现场，增加了 CPU 的额外开销。

影响时间片设置的主要因素有：系统的响应时间，就绪进程的数量及计算机的处理能力等。时间片长度 q 的选择可表示为：

$$q=R\ /\ N_{max}$$

其中，R为系统规定的响应时间，N_{max}为就绪队列中所允许的最大进程数。

4．最高优先级（HPF）调度算法

这是多道程序系统中广泛使用的算法，即进程调度每次都将 CPU 分配给就绪队列中具有最高优先级（Highest Priority）的进程。该算法的核心是确定进程的优先级。

进程的优先级算法分为静态优先级算法和动态优先级算法。

（1）静态优先级是在进程创建时根据进程初始特性或用户要求而确定的。例如，按进程的类型；系统进程的优先级高于用户进程。进程的静态优先级一经确定，在进程的生命期中就不可改变。显然，静态优先级算法简单，易实现，系统开销小，但可能导致某些优先级低的进程无限期地等待，使调度性能不高，系统效率较低。静态优先级一般用于实时进程。

（2）动态优先级则是在进程创建时先确定一个初始的优先级，又称为基本优先级，在进程运行过程中按照某种原则使各进程的优先级随进程特性的改变而变化。例如：

①根据就绪进程等待 CPU 的时间长短来决定。一个就绪进程在就绪队列中等待的时间越长，则它获得动态优先级提升越大。

②根据进程占有 CPU 时间的长短来决定。一个进程占有处理机的时间愈长，则在被阻塞之后动态优先级下降越大。一般非实时进程采用动态优先级的调度策略。但由于动态优先级随时间的推移而改变，系统要经常计算各进程的优先级，因此，系统要付出一定的开销。

进程优先级的确定通常按照如下原则：

（1）按进程的类型　对不同类型的进程赋予不同的优先级，如系统进程的优先级高于用户进程，前台用户进程高于后台用户进程的优先级，实时进程的优先级高于一般进程的

优先级等。因为这些进程或者使用频繁，或者要求快的响应速度，因此要求优先调度。

（2）按对资源的需求量及类型　例如，对占用 CPU 时间短的，使用内存资源少的进程，赋予高的优先级。

（3）按作业到达系统的时间顺序　先到者赋予较高的优先级，即“先到先服务”。

（4）按用户类型和要求　例如，由用户自己按需要确定优先级，对优先级高的用户进程高额收费。

5.　高响应比优先调度算法（HRN）

HRN（Highest Response　ratio Next）算法将短进程优先与动态优先级相结合。所谓高响应是指进程获得调度的响应，即优先数 R，CPU 总是先调度优先数 R 高的进程。

$$R = (W+T)/T = 1+W/T$$

其中，T 为估计进程执行的时间，W 为进程等待的时间。

该算法具有以下特点：

① 随着进程等待时间的增加，优先数动态增加。

② 对等待相同时间的短进程比长进程优先数增加得多。

③ 长进程随着等待时间增加也会被调度。

该算法是一种综合调度算法，既考虑了短进程优先，减少各进程的平均周转周期，同时也考虑对长进程的公平服务。

6．多级反馈队列

在实际的操作系统中，所采用的调度算法往往是将几种基本算法相结合的综合调度。多级队列反馈法就是一种考虑了 HPF 法、RR 法和 FCFS 法的综合调度算法，其基本思想是：

① 按优先级分别设置 n 个就绪队列，且优先级愈高的队列分配的时间片愈小。

② 某个进程并非固定在某一队列中，即采用动态优先级，进程的优先级在运行过程中按进程动态特性进行调整。

③ 系统总是先调度优先级高的队列，仅当高优先级队列为空时，才调度下一高优先级队列中的进程。

④ 同一优先级队列中的进程按到达先后次序排列，即按 FCFS 法与 RR 法相结合的策略调度。

需要说明的是：在实际的操作系统中一般采用综合调度策略，以提高系统的调度性能。

2.2.7　死锁

在系统中多个进程并发执行并共享系统软硬件资源的情况下，通常采用动态分配策略（即随时申请，随时分配），将各类资源分配给申请资源的进程。若对资源的管理使用不当，在一定条件下会导致系统发生随机故障，出现进程被阻塞的现象，即若干进程彼此互相等待对方所拥有且又不放的资源，其结果是谁也无法得到继续运行所需的全部资源，因而永远等待下去。这种现象称为死锁，处于死锁状态的进程称为死锁进程。

系统发生死锁现象不仅浪费大量的系统资源，甚至导致整个系统崩溃，带来灾难性的后果，所以必须高度重视死锁的问题。

一、产生死锁的原因

为了解决死锁的问题，首先要搞清楚产生死锁的原因，通常有以下原因引起死锁：

（1）争夺资源而引起死锁

若干进程因使用共享资源不当而造成死锁。一般系统中的资源分为可再用资源和消耗性资源两类；往往系统所提供的资源个数少于并发进程所要求的该类资源数，因而对资源的争夺可能引起死锁。

如有P_1、P_2两进程都要使用打印机（R_1）和磁带机（R_2）（如图 2.17 所示），若P_1已占有R_1，还申请R_2，而P_2已占有R_2同时又申请R_1。由于R_1与R_2均为非剥夺性资源，即将资源分配给某进程后不能强行收回，则P_1、P_2对R_1和R_2的争夺形成僵局，或称为形成环路，因而产生死锁。

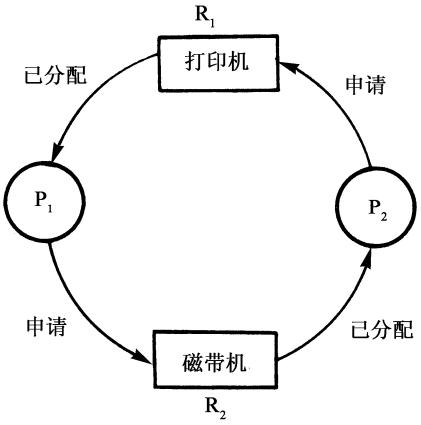


图 2.17　争夺 I/O 设备的死锁

（2）进程推进顺序不当而引起死锁

进程执行时的操作顺序不当，也可能造成死锁。如在生产者—消费者问题中，若颠倒两个 P 操作的顺序就可能造成死锁。

生产者进程：	消费者进程：
⋮	⋮
P（mutex）；申请使用缓冲区	P（mutex）；申请使用缓冲区；
P（empty）；检查是否有空缓冲区；	P（full）；检查缓冲区是否有信息；
⋮	⋮

在生产者进程中先申请缓冲区后，再检查是否有空的缓冲单元，若此时缓冲区全满，生产者进程被阻塞。而消费者进程也因缓冲区被生产者进程占用又总不释放而被阻塞，造成系统死锁。

通过上述分析得知，造成死锁的原因一是争夺系统资源而系统对资源的分配管理不当而引起的；二是多道程序运行时，进程推进顺序不合理造成的。总之，产生死锁的根本原因是共享资源。因此产生死锁的必要条件是：

①互斥条件

进程互斥使用资源，任一时刻一个资源只为一个进程独占，其他进程若请求一个已被占用的资源，只能等占用者释放后才能使用。

②不剥夺条件

进程所获得的资源在未使用完毕前，不能被其他进程强行剥夺，只能由获得该资源的进程自己释放。

③部分分配条件

进程每次申请它所需要的一部分新资源的同时，继续占用它已分配到的资源。

④环路条件

存在一个循环等待链，链中每一个进程都在等待它的前一个进程所持有的资源。

二、死锁的预防

预防死锁是采用某种策略，限制并发进程对资源的请求，破坏产生死锁的必要条件，从而使得死锁的必要条件在系统执行的任何时间都不满足。一般可采用以下三种预防措施：

（1）采用资源的静态预分配策略，破坏“部分分配”条件。即要求进程必须预先申请其所需的全部资源，只有当进程所需的全部资源满足时，系统才予以一次分配，且在进程整个运行期间不再申请新资源。

静态预分配策略简单、安全、易于实施，但资源利用率低，进程常常被延迟运行。

（2）允许进程剥夺使用其他进程占有的资源，从而破坏“不可剥夺条件”。即在允许进程动态申请资源的前提下，规定一个进程在请求新资源不能立即得到满足而变为等待状态前，必须释放已占有的全部资源。

这种预防方法实施复杂，需要保留和恢复进程现场，开销较大，还可能出现反复申请和释放某些资源的情况。

（3）采用资源顺序使用法，破坏“环路”条件。即将系统中所有资源按类型线性排队，并按递增规则赋予每类资源唯一编号，进程申请资源时，必须严格按资源编号递增顺序分配。

由于资源申请、分配是动态进行的，与静态预分配策略比较，虽然提高了资源利用率，但由于进程使用资源的顺序与系统规定的次序往往不同，因此顺序申请分配可能造成资源浪费，故资源利用率仍然不高。

三、避免死锁

预防死锁实际上是设法破坏产生死锁的必要条件之一，严格限制产生死锁的必要条件存在。而避免死锁则不那么严格地限制产生死锁的必要条件存在，而是在系统运行过程中，一旦有可能出现死锁，尽力避免系统进入不安全状态。

基本方法是系统对每种资源的分配提供一种算法，当进程申请资源时，采用相应算法计算，以确定这一申请是否会造成死锁，若计算结果表明有可能产生死锁，就不予分配资源。

最著名的避免死锁算法是 Dijkstra 提出的银行家算法。该算法是把操作系统比作一个银行家，把操作系统管理的各种资源比作银行的可周转的借贷资金，而把申请资源的进程比喻为借贷的客户。若每个客户的借贷总额不超过银行的资金总数，而且在有限的期限内银行可收回借出的全部贷款，那么银行就可以满足客户的贷款要求，否则银行将拒绝借贷给客户。

四、死锁的检测及解除

预防和避免死锁的目的是为了不让系统发生死锁，它是以对资源的使用设置一些限制及增加额外的 CPU 开销为代价的。

解决死锁的另一途径是检测死锁的方法，该方法对资源的分配不加限制，即允许死锁的发生，但系统定时地运行一个“死锁检测”程序，判断系统内是否出现死锁，若检测到死锁就设法将其解除。解除死锁常采用以下方法：

（1）撤销进程法

按照一定策略，逐个撤销死锁进程，每撤销一个进程就检测死锁是否继续存在，直到死锁消除为止。撤销进程的策略可按最小代价的原则，即撤销路径最短，撤销进程数最少，或按进程优先级小的先撤销。

（2）资源剥夺法

逐个撤销死锁等待链上的一些进程，每撤销一个进程就剥夺它所占有的全部资源，并分配给死锁进程，直到死锁状态解除。

总之，死锁是对计算机系统正常运行危害最大，但又是随机的、不可避免的现象。因此解决“死锁问题”是一个重要的研究课题。现有的各种方法都是以不同程度的牺牲系统效率为代价的。

2.2.8 进程通信

并发执行的进程，需要进行信息的交换，以便协调一致地完成指定的任务，这种联系是通过交换一定数量的信息来实现的。进程同步也是一种简单的通信方式，利用信号量和 P、V 操作只能传递控制信息，称为低级通信。把大批量数据的交换称为高级通信。使用高级通信原语不仅能实现进程间的相互制约、相互协调的关系，同时实现了进程间大量信息的传送。

目前常用的高级通信方式有消息缓冲通信、管道通信和信箱通信。

一、消息缓冲通信

消息缓冲通信是一种直接通信方式，其基本思想是利用内存中公共消息缓冲区实现进程间信息交换。如图 2.18 所示，当一个进程（A）向另一进程（B）发送消息时，首先准备好欲发送的消息，申请一个空的消息缓冲区，使用发送原语 send(receiver，a) 将消息发送出去，填入空缓冲区，并将其插入到接收进程的消息缓冲队列中。发送原语的参数 receiver 为接收消息的进程名，a 为发送者进程的内存工作区中存放该消息的地址。

接收进程则利用接收原语 receive(b)，从消息队列中摘取一则消息。参数 b 为接收进程内存工作区中接收消息的内存地址。在接收消息后，释放消息缓冲区。

由于消息缓冲机制中所使用的是公用缓冲区，所以在传送消息时，两通信进程应满足如下约束：

（1）在发送进程把消息写入缓冲区及把缓冲区插入消息队列时，应禁止其他进程对该消息队列的缓冲区访问，否则将引起消息队列的混乱。同理，当接收进程从消息队列中读取消息时，也禁止其他进程对消息队列的访问。

（2）当缓冲队列为空时，接收进程不能接收任何消息。

在消息缓冲队列通信机制中，为了实现上述的通信过程，必须定义以下数据结构：

①消息缓冲区

type

message　buffer = record

sender：发送进程标识

size：消息长度

② PCB 块中通信数据

type

process control bloke　= record

mg ：消息队列首指针

mutex：消息队列互斥信号量

text：消息正文　　sm：消息队列资源信号量

next：指向下一消息的指针

end；　　end；

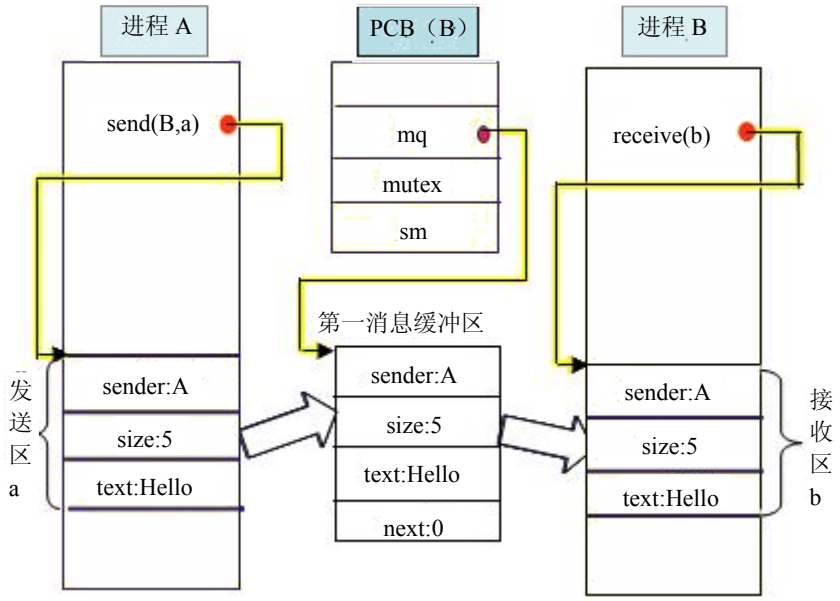


图 2.18　消息缓冲队列通信

定义 mutex 为公共的互斥信号量，用于控制进程对缓冲区的互斥访问，初值为 1。设 sm 为接收进程的私有的资源信号量，表示消息队列中等待接收的消息个数，初值为 0。

将发送与接收过程描述如下：

```
send(receiver,a);
  begin
    向系统申请一个消息缓冲区;
    P(mutex);
    将发送区消息送入消息缓冲区;
    将消息缓冲区插入消息队列;
    V(mutex);
    V(sm);
  end

Receive(b);
  begin
    P(sm);
    P(mutex);
    取消息队列中的消息;
```

将消息从缓冲区复制到接收区；

释放缓冲区；

V(mutex)

end

由于消息缓冲通信的实质是利用内存缓冲来实现进程间数据交换，每个缓冲块大小相同且固定，每则消息的长度有限，因此，每次传送的数据量有限。

二、管道通信

管道（pipe）通信由 Unix 首创，是一种有效的通信技术，因而管道通信成为进程通信的一种重要方式。

管道通信与消息缓冲通信不同，它是以文件系统为基础的。所谓管道是一个连接读、写进程的共享文件，如 Unix 系统中的 Pipe 文件，以字符流的形式传送。

管道文件在使用前由使用者建立并打开，即是在系统的打开文件表中建立两个表目，一个表目用于控制该管道的写操作，另一个表目控制该管道的读操作。通常一个管道为两个进程专用，一个进程只用于写入端，另一进程只用于读出端（如图 2.19 所示）。

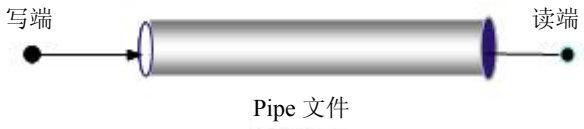


图 2.19　管道通信的结构

发送进程可以源源不断地从管道的写入端写入数据流，接收进程在需要时从管道的读出端读出信息。在对管道文件进行读、写操作过程中要对发送进程和接收过程实施正确的同步和互斥，以保证通信的正确性。读、写操作必须互斥地使用管道。

由于管道通信的实质是利用外存来进行数据通信，所以具有传送数据量大的优点，但通信速度较慢。

三、信箱通信

信箱是进程间另一种通信机制，是一种间接通信方式。以发送、接收、回答信件作为通信的基本方式。由发送进程申请建立一个与接收进程链接的邮箱。发送进程把消息送往邮箱，接收进程从邮箱中取出消息，从而完成进程间信息交换。

邮箱由邮箱头和邮箱体组成。其中，邮箱头描述邮箱名、大小及拥有该邮箱的进程名等，邮箱体主要用于存放消息（如图 2.20 所示）。

对只有一个发送进程和一个接收进程使用的邮箱，进程间通信应满足以下条件：

（1）发送进程发送消息时，邮箱中至少要有一个空格能够存放消息；

（2）接收进程接收消息时，邮箱中至少要有一个消息存在。

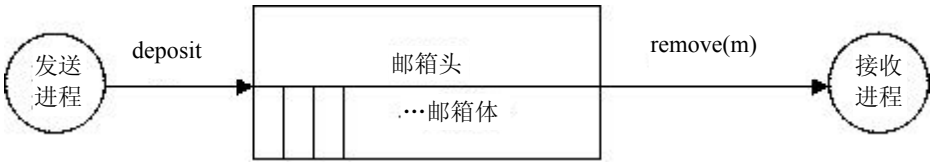


图 2.20　邮箱通信结构

为了实现邮箱通信，必须提供相应的原语，如创建信息的原语、撤销信息原语、发送原语、接收原语等。如进程之间的通信使用相应的发送和接收原语实现：

send（mailbox，message）；

receive（mailbox，message）；

一个邮箱可以考虑成发送进程与接收进程间的私有数据结构，而不是像消息缓冲队列那样被系统内所有进程共享。使用邮箱通信最大的好处就是发送和接收没有处理时间上的限制。

2.2.9　线程

20 世纪 80 年代中期，为了改善系统性能，人们提出了比进程更小的、能独立运行的基本单位——线程（Thread）。线程的引入提高了系统内程序并发执行程度，从而进一步提高了系统的吞吐量。近年来线程得到更加广泛的应用，不仅在 OS 中引入了线程，而且在数据库管理系统和其他软件中，都纷纷引入线程来改善系统的性能。

一、线程的概念

1．为什么引入线程

在操作系统中引入进程的目的，是为了使多个进程并发执行，以改善资源利用率及提高系统的吞吐量。由于进程既是资源的拥有者，也是调度的基本单位，因而在进程的创建、撤销和切换中，系统必须为之付出较大的时空开销。正因为如此，系统中设置的进程数目不宜过多，进程的切换也不宜过高，这也限制了并发程度的进一步提高。

为了减少进程并发执行时所付出的时空开销，提高系统性能，将资源分配与调度分开而引入线程。

2．什么是线程

在引入线程的 OS 中，线程是进程的一个实体，是能被系统独立调度和分配的基本单位。只有进程才是申请资源的基本单位。线程自己基本上不拥有系统资源，但它可与同属一个进程的其他线程之间并发执行，共享所属进程的资源，因此又将线程称为轻型进程。

由于线程是在进程内用于调度和占有处理机的基本单位，它由线程控制表、存储线程上下文的用户栈以及核心栈组成。

3．线程的描述

一个进程可由一个或者多个线程构成。进程用 PCB 块描述，类似地，线程用 TCB 块（Thread Control Block）描述。

线程中所使用的同步控制机制与进程的同步控制机制相同。线程有三种基本状态，即

执行、就绪和阻塞。线程没有挂起状态，也就是说线程是只与内存和寄存器相关的概念，它的内容不会因为交换而进入外存。使用 5 种基本操作来对线程的 3 种基本状态进行转换。

（1）派生：线程是由进程内派生出来的，它既可以由进程派生，也可以由线程派生。在创建进程时，就创建了一个主线程。用户一般用系统调用或相应的库函数派生自己的线程。

（2）阻塞：如果一个线程在执行过程中需要等待某个事件发生，则被阻塞。阻塞时，寄存器上下文、程序计数器以及堆栈指针都会得到保证。

（3）激活：阻塞线程的事件发生，则该线程被激活并进入就绪队列。

（4）调度：选择一个就绪线程进入执行状态。

（5）结束：线程结束，它的寄存器上下文以及堆栈内容等将被释放。

二、线程的特性

线程具有许多传统进程所具有的特征，故又称为轻型进程或进程元，而把传统的进程称为重型进程。在引入了线程的 OS 中，通常一个进程都有若干个线程，至少也有一个主线程。下面我们通过对线程与进程的比较来进一步认识线程的特性。

（1）进程是申请资源的基本单位，它拥有自己的地址空间和各种资源，例如，内存空间、外部设备等；线程只是处理机调度的基本单位，它只能与其他线程一起共享进程资源，但自己不能申请资源。

（2）同一进程的多个线程具有并发执行的特性，线程之间相互约束，线程执行过程呈现间断性。也允许不同进程的线程并发执行，线程的调度算法与进程类似，对 CPU 的分配也分抢占式和非抢占式。一般不会引起进程切换。

（3）线程和进程一样，也具有就绪、阻塞和执行三种基本状态，不过，由于线程没有单独的数据和程序空间，因此，线程不能像进程的数据和程序那样，交换到外存存储空间，从而线程没有挂起状态。

（4）进程的调度、同步等控制大多由操作系统内核完成，而线程的控制既可以由操作系统内核完成，也可以由用户完成。

（5）线程的系统开销小于进程。可从以下方面比较：

① 创建—撤销的开销　由于 PCB 比 TCB 复杂，因此创建—撤销进程的开销明显比线程大。

② 调度开销　同一进程内的线程切换的开销小于进程切换开销。以进程为单位进行处理机切换和调度时，由于涉及资源转移以及现场保护等问题，将导致处理机切换时间较长，资源利用率降低。而以线程为单位进行处理机切换和调度时，由于不发生资源变化，特别是地址空间的变化，处理机切换时间较短，从而提高了处理机效率。

③ 资源开销　线程一般不拥有资源，但可访问所属进程的资源。

④ 通信开销　进程通信具有独立的地址空间，通过共享文件等实现，线程通信是任务通信，所以开销小。

三、线程的分类

线程有两种基本类型：用户级线程和系统级线程（核心级线程）。同一操作系统中，有的使用纯用户级线程，有的使用纯核心级线程，例如，Windows NT 和 OS/2 就是这样的系统；有的则混合使用用户级线程和核心级线程，例如，Solaris 系统。

1．用户级线程（User-level Threads）

用户级线程管理过程全部由用户程序完成，操作系统内核只对进程进行管理。

为了对用户级线程进行管理，操作系统提供一个在用户空间执行的线程库。该线程库提供创建、调度、撤销线程等功能。同时，该线程库也提供线程间的通信，线程的执行以及存储线程上下文的功能。用户级线程只使用用户堆栈和分配给所属进程的用户寄存器。当一个线程被派生时，线程库为其生成相应的线程控制块 TCB 等数据结构，并为 TCB 中的参量赋值和把该线程置于就绪状态。其处理过程与进程创建过程大致相似。

不同的是：

（1）用户级的调度算法和调度过程全部由用户自行选择和确定，与操作系统内核无关。在用户级线程系统中，操作系统内核的调度单位仍是进程。

（2）用户级的调度算法只进行上下文切换而不进行处理机切换，且其线程上下文切换是在内核不参与的情况下进行的。也就是说，线程上下文切换只在用户栈、用户寄存器之间进行切换，不涉及处理机的状态，新线程通过程序调用指针使得程序计数器变化而得以执行。

（3）因为用户级线程上下文切换与内核无关，所以可能出现以下情况：当一个进程由于 I/O 中断或者时间片用完等原因造成该进程退出处理机，而属于该进程的执行中线程仍处于执行状态。也就是说，尽管相关进程的状态是阻塞的或者等待的，但所属线程状态却是执行的。

2．核心级线程（Kernel-level Threads）

核心级线程由操作系统内核进行管理。操作系统内核给应用程序提供相应的系统调用和应用程序接口 API，以使用户程序可以创建、执行、撤销线程。

与用户级线程不同，核心级线程既可以被调度到一个处理机上并发执行，也可以被调度到不同的处理机上并行执行。操作系统内核既负责进程的调度，也负责进程内不同线程的调度工作，因此核心级线程不会出现进程处于阻塞或者等待状态，而线程处于执行状态的情况。另外，核心级线程技术也可以用于内核程序自身，从而提高操作系统内核进程的执行效率。与用户级线程比较，核心级线程的上下文切换时间要大于用户级线程的上下文切换时间。有些操作系统，比如，Linux 或者 Solaris 2.x，提供用户级线程和核心级线程两种功能。这些操作系统中，线程的创建、调度和同步仍在用户空间完成，而这些线程也可被映射到系统空间，并转化为系统级线程执行。

小　结

进程是操作系统中最重要、最基本的概念之一，它是可并发执行的程序在给定的数据集合上的执行过程。进程是系统分配资源的基本单位。

进程具有动态性、并发性、独立性及异步性等基本特征。其动态性反映在从创建到撤销进程的生命期。对进程生命期的各种活动及就绪、执行、阻塞三种基本状态的转换和控制，由构成操作系统内核的各种原语实现。进程的并发特性反映在对资源进行竞争而产生的相互约束上。因共享资源而产生的间接制约关系称为互斥，以互斥关系使用的共享资源称为临界资源。而由于协同完成某项任务而产生的直接制约关系称为同步。进程的同步和互斥关系，可通过 P、V 操作原语实现。

处理机的调度可归结为进程的调度，这是处理机管理的另一重要内容。当多个进程竞争处理机时，进程调度程序按照某种调度算法，从进程的就绪队列中挑选一个进程使之占有处理机运行。常用的基本调度算法有 FCFS、SCBF、RR、HPF、HRN 及多级反馈队列等。在实际操作系统中所采用的调度算法往往是一种综合调度算法。

进程间通信常用的方式有消息缓冲通信、管道和邮箱通信，它们是能实现大批量数据交换的高级通信方式。

由于争夺资源或进程推进顺序不当都可能造成死锁。死锁产生的四个必要条件是：互斥条件、不剥夺条件、部分分配及环路条件，只要破坏这四个条件之一便可防止死锁的产生。

最后，讨论了为了提高系统运行效率而引入的线程的概念；讨论了线程的特性、分类及进程与线程的区别与联系。

2.3 作业管理与用户接口

进程是计算机系统中的基本工作单位，而作业则是用户请求系统服务的最大单位。与进程一样，作业也有从提交到完成的生命期，作业管理就是对其生命历程进行宏观控制。具体任务是对作业的控制和调度。

2.3.1 作业的概念

一个作业，就是用户请求计算机系统执行的一次独立的上机任务，是能共享公共资源区域的一族有关进程（进程家族）。

从静态观点看，作业由控制命令序列、程序集及数据集三部分构成。作业控制命令序列反映用户对其作业的控制意图，例如：程序是否要进行修改、编译、对程序的错误是否要进行修改，程序如何处理，运行作业需要哪些资源，运行结果如何处理等。

一个作业一般又可分为若干顺序处理的作业步。作业步相互独立又互相关联，上一作业步的执行结果，往往是下一作业步的输入数据或输入文件。一般只有上一作业步完成才能继续执行下一作业步。例如上机调试运行一个程序的过程；编辑输入是一个作业步，它产生源程序文件；编译也是一个作业步，它产生目标代码文件；同样，链接、运行也是相应的作业步。

一、作业控制块

一个作业也由作业控制块 JCB（Job Control Block）来描述。JCB 是记录类型的数据结构，用于记录作业的有关信息，通常包括：作业名、优先级、作业建立时间、作业状态，外存中存放作业的首地址、作业长度、内存需求量、作业估计的执行时间等。作业控制块 JCB 与作业之间具有一一对应的关系。JCB 是作业在系统中存在的唯一标志。

当系统把有关作业的信息输入到外存输入井时，就为该作业建立了 JCB，同时将作业有关信息填入 JCB，并使 JCB 插入到作业的后备队列。JCB 存在于作业的整个生命期，只有当作业完成时，对应的 JCB 才被撤销。

二、作业状态

一个作业从进入系统到运行结束，一般要经历进入、后备、运行和完成四个阶段，相应的作业也有四种状态：

1．进入状态

用户以操作说明书或作业控制序列的形式将作业的信息从输入设备上输入，此时称为作业处于进入状态，亦称为提交状态。

2．后备状态

当作业的全部信息都已输入并由操作系统将其存放于外存的输入井中，同时为其建立一个作业控制块 JCB，系统将该 JCB 插入到作业的后备队列中等待作业调度程序调度运行，此时作业处于后备状态。

3．运行状态

作业调度程序按照某种调度策略，从作业后备队列中挑选一个或多个作业，分配必要的资源，将其装入到所分配的内存区，并为其创建一个或多个进程。从作业进入内存开始到作业完成，称为作业处于运行状态。

系统为处于运行状态的作业建立的进程，处于就绪状态，其活动由进程调度程序管理。因此处于执行状态的作业，是以进程的形式参与运行的。

4．完成状态

当进程正常运行结束或非正常终止时，称为作业处于完成状态，此时，系统将回收该作业所占用的全部资源，将作业运行情况及输出结果以文件形式存入外存的输出井中，并清除有关的 PCB 和 JCB，作业从系统中消亡。

作业状态及其转换过程如图 2.21 所示。

2.3.2 作业调度

作业从用户提交开始到真正占有处理机而被执行，要经过多级调度才能实现。§2.2 节中所讨论的进程调度称为处理机的低级调度，而作业的调度称为处理机的高级调度或宏调度。作业调度的关键是作业调度的算法。

一、作业调度的功能

作业调度由作业调度程序完成，作业调度的主要任务是：按照某种调度算法，从作业

的后备队列中挑选一批合理搭配的作业进入内存，实现作业从后备状态转变为运行状态。同时为选中的作业分配内存和外设资源，并为其建立有关的进程。

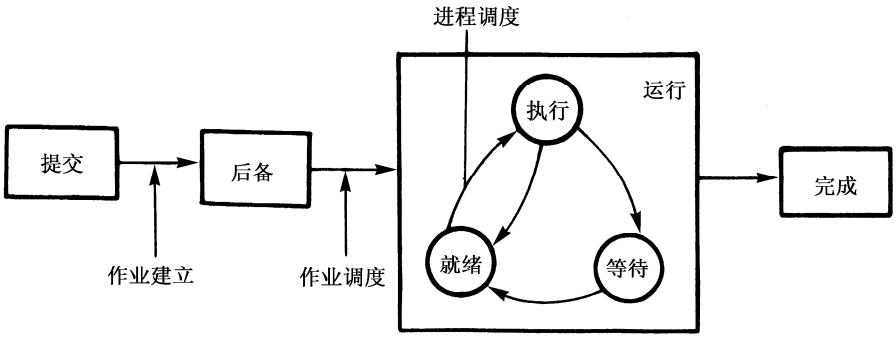


图 2.21　作业的生命周期

当作业运行结束后，进入完成状态时，做好作业的善后处理工作，回收作业所占有的资源。作业调度程序是根据作业的 JCB 对作业的调度过程进行控制的。

作业调度与进程调度是既有区别又紧密联系的两个概念。作业调度是确定哪些作业有资格调入内存，而进程调度则是确定内存中的进程哪一个能被 CPU 调度。也就是说作业调度是进程调度的基础，而进程调度是作业在运行状态下的执行过程。

二、作业调度算法

作业调度算法决定了系统资源能否获得高效的利用，决定了作业调度性能的优劣。

1．作业调度算法的性能评价

调度算法的选择一般要考虑三个基本目标：尽量提高系统作业的吞吐量；应尽量使 CPU 和外设处于“忙”状态，以提高资源利用率；应对所有作业的服务公平合理，使所有用户都满意。具体有以下度量指标：

（1）CPU利用率U_P

$$U_P = \frac{\text{CPU有效工作时间}}{\text{CPU总运行时间}}$$

其中，CPU 的总运行时间＝CPU 有效工作时间+空闲时间。一个好的作业搭配，例如计算量大的作业与输入、输出量大的作业搭配，通常可以提高 CPU 的利用率。

（2）作业平均周转时间 T

作业i从提交的时刻t_{si}到完成时刻t_{ci}所经历的时间称为作业的周转时间T_i，即

$$T_i = t_{ci} - t_{si}$$

作业的平均周转时间为：

$$T = \frac{1}{n}\left(\sum_{i=1}^{n} T_i\right)$$

其中，n为作业数；T_i为每一作业的周转时间。

作业的平均周转时间用于衡量不同的调度算法对同一作业流的调度性能。

（3）平均带权周转时间 W

作业i的周转时间T_i与其实际运行时间T_{ri}之比称为作业 i 的带权周转时间W_i。

则作业的平均带权周转时间为：

$$W=\frac{1}{n}(\sum_{i=1}^{n}W_i)=\frac{1}{n}(\sum_{i=1}^{n}\frac{T_i}{T_{ri}})$$

其中，n为作业数；W_i为每一作业的平均带权周转时间。

通常用平均带权周转时间 W 来衡量同一调度算法对不同作业的调度性能。

2．作业调度算法

在同一个操作系统中，作业调度算法与进程调度算法一致。

（1）先来先服务算法（FCFS）

FCFS（First Come First Serve）的算法按照作业提交的先后次序来调度作业。具体由登录程序按作业登录的次序建立后备队列。先进入系统的作业被先选择，放入“现行作业队列”并将其状态置为“运行”，为该作业建立相应的就绪进程。

先来先服务的算法简单，易于实现，利于长作业的运行，但对短作业可能等待时间较长，因而平均周转周期较长。

在有“脱机”和“联机”两种控制方式的多道批处理系统中，以“联机”队列的调度优先。

（2）最高优先级算法（HPF）

HPF（Highest Priority First）算法是根据作业的优先级进行调度，每次总是选择优先级高的作业。作业的优先级可由用户自己规定，亦可由系统根据作业的轻重缓急程度，作业长短、等待时间多少，资源使用要求等情况计算优先级。

（3）最短作业优先算法（SF）

SF（Shortest job First）算法优先选择估计运行时间最短的作业，算法的平均周期时间优于 FCFS 算法，有利于短作业的调度。

但对较长的作业可能会长期等待。

（4）最高响应比优先算法（HRN）

HRN（Highest Response ratio Next）算法是先来先服务算法与最短作业优先算法的折中，响应比 R 定义为系统对作业响应时间与作业运行估计时间的比值。即

$$R=\frac{\text{响应时间}}{\text{估计运行时间}}=1+\frac{\text{作业等待时间}}{\text{估计运行时间}}$$

其中，响应时间为作业进入系统后的等待时间加上估计的运行时间。首先计算后备作业的响应比 R，响应比高的作业被优先调度。从式中可见，一个作业的 R 会随着它的等待时间的增加而增大，因而等待时间长的作业总可能由于响应比高而获得调度。

该算法虽然调度性能不如 SF 法，但它同时兼顾了用户到来的先后次序和要求服务的时间。但由于每次调度时都需计算作业的响应比。系统开销也相应增加。

（5）资源搭配法

又称均衡调度法。选取作业的原则是使进入运行状态的各作业获得最好的搭配，使系

统资源得到高效均衡的使用。

在多道程序系统中，由于各类作业对资源共享及各类作业对资源的种类、数量要求不同，情况更为复杂，因此对作业的调度，通常采用综合调度策略。

2.3.3 操作系统接口

一、用户与操作系统的接口

操作系统是用户与计算机之间的接口，用户通过操作系统提供的两个接口来使用计算机，一是系统为用户提供的各种命令接口。用户利用这些操作命令来组织和控制作业的执行或管理计算机。另一个接口是系统调用。编程人员使用系统调用来请求操作系统提供服务。

1．命令接口

又称为作业控制级接口，这是系统为用户在作业控制级获得系统服务而设置的接口，用户利用作业控制级接口对作业在系统中运行的全过程进行控制。

这类接口又进一步分为联机接口和脱机接口。联机接口通常由一组键盘操作命令组成，它向联机用户提供了以交互方式请求系统服务的手段。脱机接口由一组作业控制命令（或称作业控制语言）组成，供脱机用户使用。脱机用户不能直接干预作业的运行，而把对作业控制的要求及对系统服务的要求用作业控制语言写成命令过程，连同作业的程序和数据一起提交给系统，系统在执行该作业时，对命令过程的命令逐条解释执行。

2．系统调用

又称为程序级接口，由一组系统调用命令（又称广义指令）组成，是操作系统为用户在程序一级提供的服务。编程人员利用系统调用在源程序一级动态请求和释放系统资源。

系统调用命令可分为有关进程控制、进程通信、存储管理、文件管理、设备管理等几类。不同的系统提供了不同的系统调用命令。一般每个系统为用户提供了几十到几百条系统调用。通常这些系统调用是事先编写好的特殊的子程序，显然这些子程序是操作系统程序模块的一部分，不允许用户直接访问。

系统调用过程是一个低级过程，是通过系统调用命令中断现行程序的执行而转去执行相应的子程序完成特定的功能。

在系统中为控制系统调用服务的机构称为陷入（TRAP）或异常处理机构。把由于系统调用引起处理机中断的指令称为陷入或异常指令。此外还必须为实现各种系统调用功能的子程序编制入口地址表，使每个入口地址都与相应的系统子程序名对应。系统调用的处理过程如图 2.22 所示。

由于在系统调用结束后，用户程序还需使用系统调用结果继续执行，因此在进入系统调用处理前应保存处理机现场。

3．图形用户接口

在用户界面中，用图形、图标、对话框及窗口等来描述 OS 与用户的接口，将能够更加形象地为用户提供丰富的、有用的信息，以达到可视化的目的。具有直观、形象、使用方便等特点，典型的是 Windows 操作系统的用户接口。

构成图形用户界面的主要成分有：

（1）对话框

对话框是在需要时，显示在屏幕上一个矩形区域内的图形和正文信息。通过对话，实现系统和用户之间的通信。

对话框显示的方式与弹出式菜单类似，即瞬时弹出。同时，系统还应将对话框所覆盖的原图像进行保存，以便在对话结束后能立即恢复。

（2）窗口

窗口在图形学中称为视图区（Viewport），视为虚拟屏幕。一个实用窗口，可以包含以下部件：

菜单区（menu bar）、图标区（icon bar）、标题区（title bar）、移动区（move bar）、大小区（size bar）、退出区（quit bar）、用户工作区（user's work bar）、横向滚动区（horizontal scroll bar）、纵向滚动区（vertical scroll bar）。

事实上，用户接口一直是计算机科学技术中发展最快和研究最多的一部分。因为计算机用户接口的好坏，直接影响到这类计算机的兴衰。总之，操作系统的用户接口既在不断地发展、演变，同时也是衡量一个操作系统好坏的重要标志。

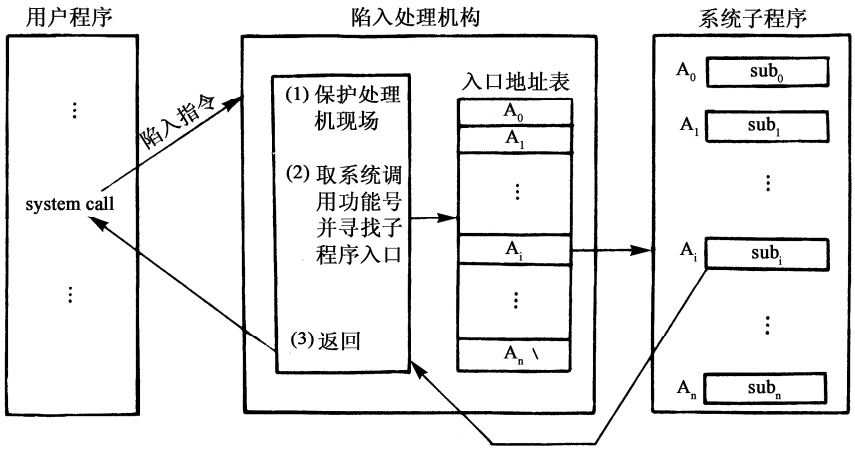


图 2.22　系统调用的处理过程

二、作业控制方式

作业控制方式有联机作业控制和脱机作业控制两种，分时系统一般采用前一种，批处理系统主要采用后一种。

1．联机控制方式

联机控制采用人机对话方式来控制作业的运行，操作系统向用户提供联机操作命令，直接控制作业的执行。操作系统每接到一条命令后，按命令要求控制作业的执行，并及时反馈作业的运行情况给用户。不同系统提供了不同的联机控制方式。通常有命令驱动、菜单驱动和窗口环境三种。

（1）命令驱动方式

操作系统向用户提供一组键盘命令实现对作业运行的控制，键盘命令一般包括：系统

访问命令、过程命令、文件管理命令、查询命令、系统管理及控制转换等命令。

命令处理程序对用户敲入的命令解释执行，用于控制用户程序的执行过程。这种方式的典型例子是 MS DOS 所提供丰富的联机操作命令，即 DOS 命令。用户从终端键入 DOS 命令，若为内部命令，则由命令处理程序 COMMAND.COM 解释执行，若为外部命令，对应的可执行命令由系统直接装入执行。

（2）菜单驱动方式

由于不同的操作系统提供了不同的众多的键盘命令，一般的终端用户难于熟练掌握。因此许多操作系统提供了菜单驱动方式，即系统将各种联机操作命令进行分类，组成分级菜单，用户通过选择菜单执行操作命令。

菜单驱动方式为用户提供了方便，但有的菜单项要经过多级菜单选择后才能定位。因此有的系统同时提供了命令驱动和菜单驱动两种方式。

（3）窗口环境

窗口环境提供了全新的操作系统人机界面。是通过对窗口、菜单、对话框、图标等图形界面（GUI）和符号操作来实现的。用户通过屏幕上的当前窗口对应用程序交互操作。还可通过同时打开多个窗口对多任务的运行进行监控。各窗口中的多个任务间既可方便地切换，又可方便地交换信息，窗口环境以其友好的用户界面和众多的功能受到广大用户的青睐，成为当前最为流行的用户界面。

采用这种方式的典型系统有 X-Window（在 UNIX 下运行），Microsoft Windows 等。

2．脱机控制方式

脱机控制方式通常用于批处理系统。由系统提供一组作业控制语言 JCL（Job Control Language）供脱机用户使用。JCL 由一组作业控制命令组成。

在脱机控制方式下，用户不能直接干预其作业的运行，而把对作业的控制意图，对系统的请求、控制过程用作业控制语言所提供的各种语句，对作业加以定义并提供给操作系统，由操作系统的作业管理软件负责组织、调度和控制各用户作业的运行。

若干作业定义的集合称为作业流。一个批处理系统中可以有多个作业流。

小　　结

作业是用户请求计算机执行的一个独立的程序任务，是用户请求计算机服务的最大单位。作业管理是对作业的生命期进行宏观控制。作业从进入系统开始要经历进入、后备、运行及完成四种状态。

作业管理的主要任务是作业控制和作业调度。作业控制是通过操作系统为用户提供的命令接口来组织和控制作业的执行。通过程序接口获得必要的系统服务。作业调度则是按照一定的调度策略，从后备队列中调度一个作业投入运行并创建相应的进程，作业在内存中以进程的形式并发执行。常用的作业调度算法有先来先服务（FCFS）、最高优先级算法（HPF）及最短作业优先（SF）、最高响应比优先（HRN）和资源搭配算法等。作业调度性能直接影响到系统性能。

2.4 存 储 管 理

存储器是计算机系统的重要资源，存储管理是操作系统研究的中心问题之一。存储管理的效率直接影响计算机系统性能，也最能反映一个操作系统的特点。

存储管理主要指对主存储器空间的管理。内容包括：内存的分配和释放，虚拟存储管理技术，地址变换，内存保护与共享。

2.4.1 存储管理的功能

存储管理的目的是尽可能地方便用户和提高主存储器的效率。因此，存储管理有以下几方面功能。

一、内存分配和回收

内存空间分配是多道程序共享的基础，当作业或进程要求使用内存资源时，系统根据内存单元的当前状态，按照分配策略，为申请者分配足够的空闲区域，若无法满足申请者的要求，则让其处于等待内存资源的状态，直到有足够的内存空间时再分配给它。内存分配的功能涉及以下问题：

1．管理内存分配表

表中记录存储空间的已分配情况、空闲情况及其他有关内存分配的信息，并随时修改有关表项。

2．制定分配策略

内存分配策略涉及：①放置策略：如何选择空闲区域的原则；②调入策略：确定信息装入内存的时机。通常有两种调入策略，即请调策略和预调策略，前者是根据需要才将所需信息装入内存，后者是预先将全部信息调入内存；③淘汰策略：使用请调策略，需要将外存的信息调入内存而内存又无空闲区时，需要确定将内存中的哪些信息调出而腾出内存空间的策略。

3．内存区域的划分方式

系统以块为单位分配空闲内存，应确定如何对内存空间分块，一般有两种划分块的方式：划分为大小不等的块和大小相等的块。可变分区管理和段式管理划分为大小不等的块，而页式存储管理采用第二种划分方式。

当作业或进程因完成或阻塞而主动或被动地释放内存资源时，存储管理应执行一系列操作回收所释放的存储空间，使它们成为又可供分配的空闲区域。

二、内存空间的共享

内存空间的共享是为了提高内存空间的利用率，共享的含义包括两方面：一是共享主存资源，在多道程序系统中，若干个程序同时进入内存，各自占有一定数量的存储空间，

实现多道程序的并发执行；二是共享内存的某些区域的信息，将若干公用的子程序、编译程序、链接程序及数据等放在某个内存区域中，各道程序在执行时都可以访问。

三、存储保护

在内存中不仅存放有操作系统本身的程序和应用程序，还有用户作业的程序。为了避免内存中的程序相互干扰，必须对内存中各程序、数据或信息区进行保护。同时还要防止受某些错误的破坏和干扰。如发生越界错误时，系统产生保护中断，中断当前执行的程序，而转去进行出错处理，特别要防止用户程序侵犯系统内存区。

四、地址映射

1．地址空间与存储空间

通常用户在使用高级语言或汇编语言编程时，由于程序中的数据和指令都用符号表示，源程序所在的空间称为符号名空间。经过编译或汇编后所形成的目标代码，通常采用相对地址，即其首地址为零，其他指令中的地址都是相对首地址而定。因此，这个相对地址也称逻辑地址或虚拟地址。目标程序所限定的相对地址的全体称为程序的地址空间或逻辑空间、虚地址空间。

显然，逻辑空间不是实际的物理存储空间，逻辑地址也不是内存中的物理地址。在内存中不能用逻辑地址存取信息。只有物理地址才是内存中存储单元的实际地址，也称绝对地址，是可识别的、可寻址并实际存在的。

物理地址的全体称为存储空间或实地址空间。

2．重定位

在多道程序系统中程序装入内存前通常为逻辑地址，为保证程序的执行，操作系统需要为它分配一个合适的存储空间，并将程序执行时要访问的地址空间中的逻辑地址转换成内存空间中对应的物理地址。这种地址的转换过程称为地址映射或重定位。

重定位可以分为两种：

①静态重定位

静态重定位也称静态地址映射，是在用户程序运行前，在程序装入内存的过程中一次完成从逻辑地址到物理地址的转换，且在程序运行过程中地址不再改变。

静态重定位的优点是无需硬件支持，通常用软件实现，可在任何机器上执行。缺点是必须使用连续的内存空间，程序一经重定位就不能再移动，不易实现程序共享。

②动态重定位

也称动态地址映射。动态重定位的时间发生在程序执行过程中，在访问指令或数据时，才进行地址变换，把逻辑地址转换为物理地址，需要依靠硬件地址变换机构实现。一般是设置一个公用的基地址寄存器（BR），也称重定位寄存器（RR），用于存放现行程序在内存中的起始地址，当 CPU 访问内存时，映射机制自动把逻辑地址与重定位寄存器中的内容相加后形成绝对地址。因此，只要改变重定位寄存器中的内容，便可把程序定位在新的内存空间。图 2.23 显示了动态地址映射的过程。

动态地址映射的过程可执行内存的动态分配，内存使用灵活，提供了实现虚拟存储器的基础，提供较大的地址空间。还可以对内存进行非连续的分配，提高内存利用率，有利

于程序段的共享。但由于需要硬件支持，实现起来较静态地址映射复杂。

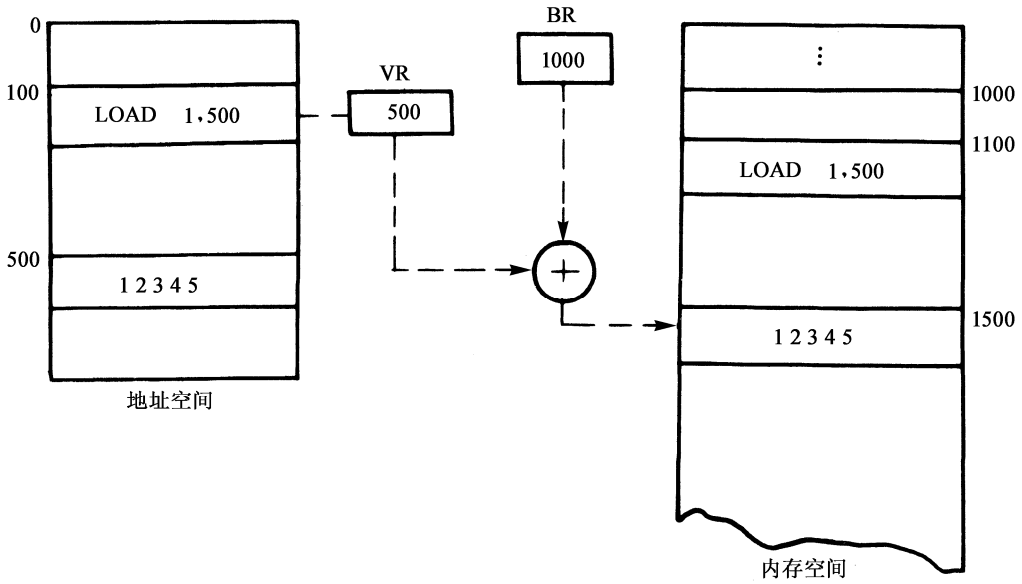


图 2.23　动态地址重定位

五、内存空间的扩充

为了解决内存的实际容量远小于多道程序所需的内存容量的矛盾，利用大容量的外存空间来逻辑扩充内存，暂时不执行的程序和数据先存放在外存中，等到需要执行访问它们时再将其从外存调入内存。这样既充分发挥内存资源的利用率，使系统能够有效地支持多道程序的并发运行，也对用户提供了一个巨大的存储空间，从而极大地方便了用户。

内存空间扩充的功能是通过覆盖与对换技术，主要是虚拟存储技术实现的。

2.4.2　分区存储管理

分区存储管理是实现多道程序的存储管理最简单的存储管理方案。其基本思想是把内存划分成若干个大小不等的连续区域，称为分区。每个作业可占用一个或多个分区。按照不同的内存分配方式，又分为固定分区和可变分区。

一、固定分区（Fixed Partitions）

固定分区法是将内存区固定地划分为若干个大小相等或不等的区域，分区划分的原则一般由系统操作员或操作系统确定。除操作系统占用一个区域外，其余分区由用户作业共享。分区一旦划分后，在整个执行过程中每个分区的长度和内存的总分区个数将保持不变。

系统设置一张内存分配表，记录每个分区的区号、起始地址、大小及占用情况（状态）。当作业申请内存资源时，系统为该作业选择一个或多个大小适当的分区，并将其装入内存运行。固定分配的内存划分和占用情况及对应的内存分配表如图 2.24 所示。

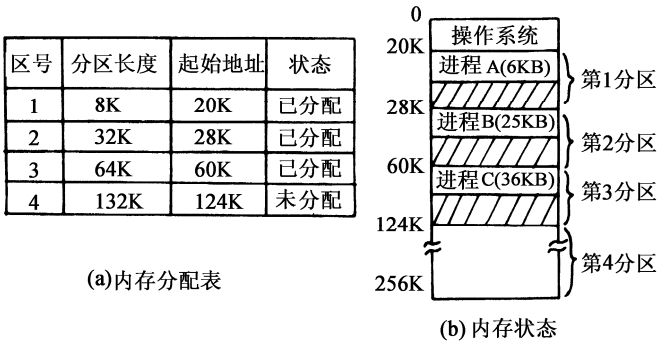


图 2.24　固定分区

图中，操作系统占用低地址部分的 20KB，其余空间被划分为 4 个分区，其中 1，2，3 号分区已分配，4 号分区未分配。

由于固定分区的大小是预先确定的，因此常常出现用户作业的地址空间大小与存储分区不匹配的情况，当用户作业的地址空间小于分区大小时，常常产生许多不可使用的存储碎片，即“内零头”，造成一些存储区间的浪费。但这种分区方法由于简单、可靠，因而在一些微机多用户系统中使用，操作系统的存储管理较简单。

二、可变分区（Variable Partitions）

为了克服固定分区造成的存储空间的浪费，引入了可变分区存储管理方式，即在存储分配的过程中按作业的大小来划分分区，使分区大小可以随作业对内存的要求而改变，同时分区的个数也可改变。这就改变了固定分区中那种即使是小作业也可能占据大分区的情况，从而提高了内存的利用率。

为了方便分区的分配与回收，内存分配表由两张表组成，一张为已分配分区表，另一张为未分配分区表，两张表的结构与固定分区分配表类似；每张表的表项，用一个称为存储控制块 MCB（Memory Control Block）的数据结构描述。通常包括分区号、起始地址、分区大小及状态等字段。描述已分配分区的表项称为分配区控制块 AMCB（Allocation Memory Control Block），描述未分配分区的表项称为空闲区控制块 FMCB（Free Memory Control Block）。通常空闲区控制块按某种次序构成 FMCB 链结构。

当一个作业申请分配大小为 X（KB）的存储空间时，系统在未分配分区表中查看内存中是否有足够的空闲区域，若该空闲区比需要的大，则将其分为两部分，一部分等于作业所需空间大小，另一部分则为空闲区，并修改两个分配表的信息。当回收一个被作业释放的分区时，要将其与邻接的空闲区合并为一个大的空闲区。以上过程如图 2.25 所示。

可变分区分配方式提高了存储空间的利用率。但这种存储管理方案，在经过一段时间后会产生一些称为“外零头”的小空闲区无法利用。解决“外零头”的办法是：移动某些已分配分区，使那些分散的小空闲区合并为一个大的连续空闲区并集中在存储器的一端，称为分区的“拼接”或“紧缩”技术。但移动一个作业分区后，必须利用重定位技术，对作业中有关地址部分进行调整。

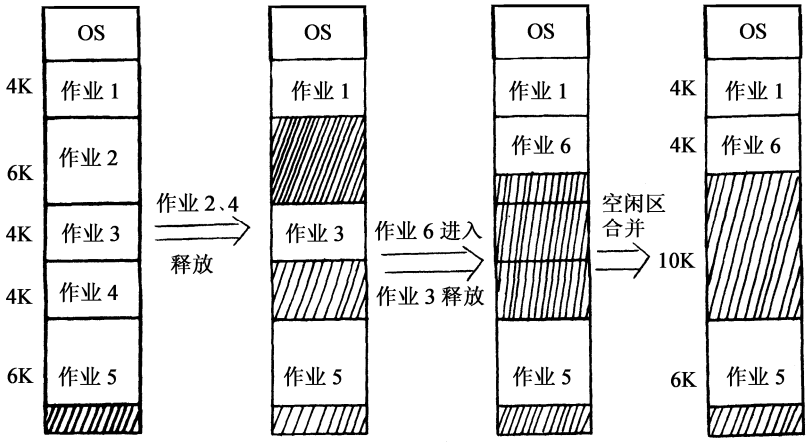


图 2.25　可变分区

采用“拼接”技术的可重定位分区分配虽然克服了碎片问题，但作业大规模移动时处理机要停止作业运行，增加了处理机的额外开销。

三、存储分配策略

在分区管理中，内存分配常采用以下几种策略：

（1）首次适应算法 FF（First Fit）

未分配分区按地址从小到大排列，每次分配时顺序查找分区分配表，选择所遇到的第一个足以满足请求容量的内存空闲区进行分配。这种算法优先使用了低地址部分空闲区，从而使高地址部分保持一个较大的空闲区，有利于后面大作业的装入。

该算法简单，搜索速度快，回收被释放区方便；无论被释放区是否与空闲区相邻，都不用改变该区在可分配分区表（或 FMCB 链）中的位置，只需修改其大小或起始地址。所以首次适应算法是使用最多的算法。

（2）最佳适应算法 BF（Best Fit）

将空闲区按其大小从小到大的次序排列，每次分配时总是从头顺序查找未分配分区表，找到第一个能满足要求的最小空闲区进行分配，这种分配算法是节约存储空间，保证一个小作业不会分割一个大的空闲区，即用最佳适应算法所选出的是最适合用户要求的可用空闲区。保证了其后面的大作业的存储分配要求容易得到满足。这种算法的缺点是每次分区释放都要重排空闲区表，且容易产生无法使用的很小的碎片。

（3）最坏适应算法 WF（Worst Fit）

最坏适应算法要求空闲区按照从大到小的顺序排列，每次分配时总是挑选一个最大的空闲区分配给作业，这样剩余的空闲区不会太小，还能用来装下新的信息，但后面的大作业可能申请不到空闲分区。该算法适于中、小作业的存储分配。

总之，以上分配算法各有特点，针对不同的请求队列，它们的效率和功能是不一样的。因此，最佳适应算法不一定是最优的，而最坏适应算法也不一定是最坏的。

2.4.3 覆盖与交换技术

分区管理方案对作业的大小有严格限制，当作业要求运行时，系统将作业的全部信息一次装入内存，直到运行结束。若作业的存储量大于内存的可用空间，该作业就无法运行。为了解决大作业与小内存的矛盾，常采用“覆盖”和“交换”两种存储管理技术，其实质是对内存进行逻辑扩充。

一、覆盖（Overlay）

所谓覆盖就是一个作业的若干程序段，或几个作业的某些部分共享某一内存区域，即是一个内存区可以先后被不同的程序段重复使用，当某程序段不再需要该内存区域时另一程序段可占用。为此，把内存区划分为常驻区（非覆盖区）和覆盖区。常驻区用于驻留该作业的总控程序及常用的公用子程序。把可以进行覆盖的内存区称为“覆盖区”，可以相互覆盖的程序段称为“覆盖”。

可将大作业划分为若干个覆盖，把作业执行时不要求同时装入内存的覆盖组成一组，称为“覆盖段”。覆盖段与覆盖区一一对应。图 2.26 显示了一种典型的覆盖结构。

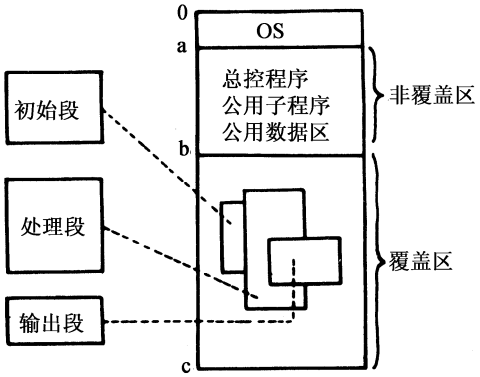


图 2.26　典型的覆盖结构

作业的主控程序、公用子程序及数据区被预先调入起始地址为 a 的非覆盖区。作业运行时先将初始段调入起始地址为 b 的覆盖区，再将处理段调入该覆盖区，覆盖了初始段，最后输出段又覆盖了处理段。

覆盖技术的关键问题是分析和建立程序的覆盖结构，但这对大型或较复杂的程序往往是困难的。在采用覆盖技术的存储管理中，系统自动实现覆盖区的管理和覆盖程序段的调入调出等功能。覆盖技术不仅用于用户程序，而且主要用于系统程序的内存管理。如用于 MS-DOS 系统的外部命令。

二、交换（Swapping）

在多道程序系统或分时系统中，为了缓解内存不够的矛盾而引入了交换技术。交换技术是指在内外存之间交换程序和数据。

例如，在分时系统中，不可能把几十个甚至上百个终端用户的作业全部装入内存，实

际上只能将一部分用户作业保留在内存中，为了保证对每个用户的进程都能作出及时响应，引入了中级调度机制，又称交换调度。系统在高速大容量的外存中（通常是磁盘）开辟一个进程交换区，作为内存的直接延伸。所有用户进程的实体都存放在交换区中。

当需要将交换区中的进程实体调入内存时，若内存空间不够，则按某种调度策略挑选内存中的一个或多个进程，将其实体调出内存，换出到交换区。通过交换，系统得以在有限的内存空间上支持多个进程并发执行，并能在合理的时间内对用户进程作出及时响应。

从存储管理的角度看，交换并不是一种独立的存储管理方案，常与分区、分页及分段等存储管理技术相结合。

2.4.4 虚拟存储管理

分区存储管理所采用的实存管理技术，主要存在以下问题：一是用户编程的地址空间（逻辑地址空间）不能超过实际的内存空间，无法运行很大的应用程序。即使采用覆盖技术，虽然能解决这一问题，但程序覆盖段的划分相当复杂，用户编程难度大；二是内存分配是连续分配方式，不可避免地会产生存储碎片，内存空间利用率低，当消除碎片而进行分区拼接会花费太高的代价。

虚拟存储技术有效地解决了这两个问题。虚拟存储的基本思想是用大容量的外存来对内存进行逻辑扩充，利用虚拟技术为用户提供一个比有限的实际内存空间大得多的虚拟内存空间，以便能够有效地支持多道程序系统中大型作业的需要，增强系统的处理能力。

对于像分区管理这样的实存管理方案，要求作业执行代码一次性装入内存，而且分区内部的地址必须连续，既浪费内存空间，也不便于共享程序。

而虚拟存储管理技术，则采用“部分装入”，“部分交换”的策略，即当大型作业运行时，不是将它的全部信息装入内存，而是将其一部分先装入内存，另一部分暂时存放在外存（磁盘）上，在作业运行过程中，根据需要动态地装入。当内存已占满而又需将外存上信息装入时，则按一定策略进行内外存的交换。

虚拟存储方式分为请求分页存储管理、请求分段存储管理和请求段页式存储管理。

2.4.5 分页存储管理

可重定位分区分配虽然可以解决碎片和共享问题，但由于信息的大量移动而损失了许多宝贵的 CPU 时间，且存储用户的作业受到实际存储容量的限制。多重分区分配虽在一定程度上解决了这些问题，但由于要求各分区设置分段的重定位寄存器，存储分配策略较复杂，且需较多的硬件支持。

分页式管理技术通过地址转换机制，能明显消除内、外存之间的差别，将外存看成内存的扩充和延伸，并能很好解决“外零头”的问题。

分页存储管理中，将程序的逻辑地址空间分为大小相同的页，称为虚页面（Page），通常页面大小为 2 的整数次幂（512B～4KB）。将物理内存空间也划分为与页面大小相等的块，称之为存储块或页框（Page frame）。连续划分的逻辑地址空间的页面，通过页面地址转换

机构可以映射到不连续的内存块中。

对逻辑地址空间的分页是由系统自动进行的，其逻辑地址由相对页号和页内位移量（页内地址）两部分组成（图 2.27（a））。图中设逻辑地址长度为 32 位，可表示 4096MB 的逻辑地址空间。物理地址也由块号及块内位移量（块内地址）两部分组成（图 2.27（b））。



图 2.27　逻辑地址与物理地址

（ａ）逻辑地址（LA）　（ｂ）物理地址

页式存储管理通常是由页表实现逻辑地址到物理地址的转换。

一、页面地址转换

在分页式管理系统中，作业装入时，以页为单位分配存储块。由于虚存管理的主要技术是部分装入和部分交换，在作业运行前，不是装入全部页面，而是只装入部分页面，在执行过程中再根据需要动态调入其他页面，所以又称请求式分页管理，当内存占满时要按某种算法进行页面的置换。

为了实现将作业的逻辑地址转换为实际内存地址，需建立以下数据结构：

1．存储分块表 MBT（Memory Block Table）

表中记录内存中每个存储块的使用情况，其中状态是指存储块是否空闲（图 2.28（a）），整个系统一张表，表目数等于内存块的总数。

2．页表 PT（Page Table）

也称页面映象表 PMT（Page Map Table），如图 2.28（b）所示。为每个运行的作业进程建立一张页表，表中的每一个表项对应该作业虚拟地址空间的一页，表项按页的顺序排列。每个表项（又称为页描述子）的内容包括页号、标志（特征）、块号等。块号表示该页面所对应的存储块，标志为 0 表示该页面不在内存，标志为 1 表示该页面在内存。

存储分块表 MBT

块号	作业号	状态
0	OS	忙
1	J1	忙
2	J1	忙
⋮		
n	JK	闲

（a）

页表 PT

页号	标　志	⋮	块号
0	1	⋮	2
1	1	⋮	1
2	0	⋮	—
3	1	⋮	8
4	0	⋮	—
⋮	⋮	⋮	⋮
m	1	⋮	17

（b）

图 2.28　存储分块表与页表

为了减少硬件开销，常将页表放在主存的系统区内。

当进程被调度时，该页表的起始地址及大小装入特定的页表控制寄存器（PTCR）中（见图 2.29）。

页表始址	页表大小

图 2.29　页表控制寄存器 PTCR

3．作业表 JT（Job Table）

整个系统设置一张作业表，每个作业为一个表项，用于记录该作业的页表在内存中的起始地址、大小及状态等（见图 2.30）。

作业号	作业大小	页表始址	状态
J1	16	1025	已占用
J2	18	⋮	⋮
⋮	⋮	⋮	⋮

图 2.30　作业表

页式存储管理的地址转换机制如图 2.31 所示，逻辑地址映射到物理地址的过程如下：

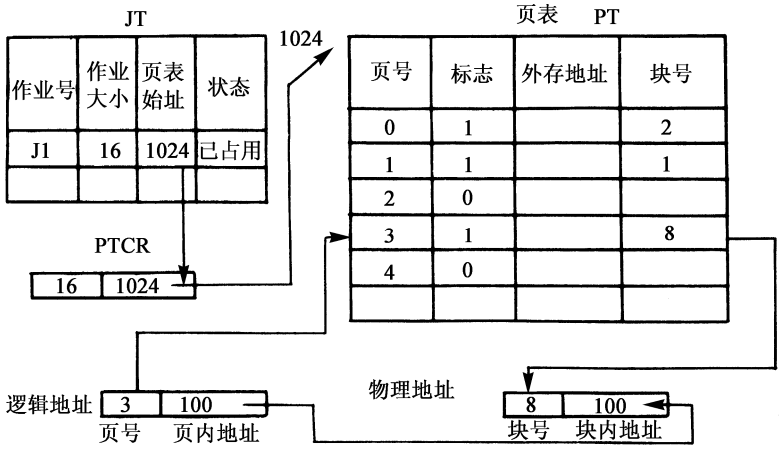


图 2.31　页式地址转换机构

①当作业进程被调度时，系统从作业表（JT）中查找相应页表的起始地址及大小，并装入页表控制寄存器。

②按页表起始地址查找页表，首先将当前访问的逻辑地址的页号与页表寄存器中当前页表的大小进行比较，确定访问合法性。若页号超过页表大小则为非法访问，产生越界中断。若访问合法则从相应页描述子中找到相应块号。

③将块号与逻辑地址中页内地址相拼接，即将页内地址作为块内地址，形成物理地址。

二、缺页中断处理

在地址转换过程中，若当前访问页的标志位为 0，即该页不在内存，则产生缺页中断。此时用户作业暂停执行，CPU 转去执行缺页中断处理程序，该程序的功能是把所需的页面从外存调入内存，并把实际块号填入页表，将标志位改为 1，然后继续执行被中断的程序。

若内存中无空闲块时，则按照某种页面淘汰算法选择一页面换出内存，再从外存调入所需页面。图 2.32 给出了缺页中断的处理过程。

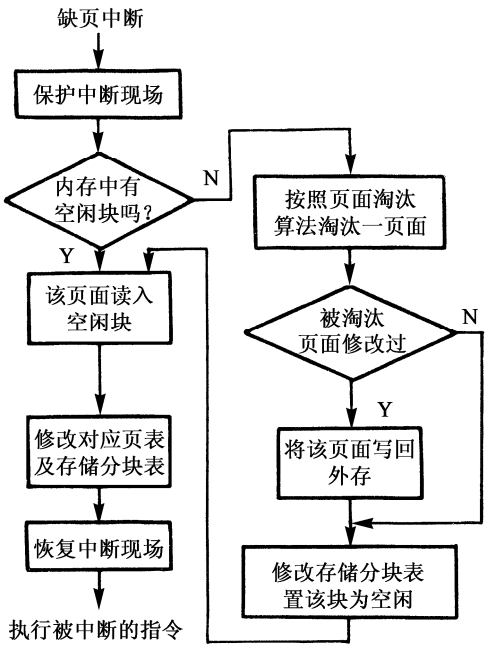


图 2.32　缺页中断处理

页号	块号	标志	页面访问	页面修改	存取控制	外存地址

其中：页面访问 1　页面被访问过；0　页面未被访问过

页面修改 1　页面被修改过；0　页面未被修改过

图 2.33　请求式分页系统的页描述子

这种按需调入页面的请求式分页系统中，为了实现页面交换及存储保护，通常在页描述子中增加页面访问位和页面修改位及存取控制字段（见图 2.33）。

在页面置换过程中，若所选择的淘汰页面的页面修改位为 1，则应将该页面写回外存，替换该页面在外存中的副本，再修改存储分块表，将该块置为空闲。

关于页面淘汰的算法将在下面介绍。

三、快表

从地址转换过程可见，每存取一个数据需要访问两次内存，第一次访问页表，得到数

据的物理地址，第二次才能访问要存取的数据。这就增加了访问时间，降低了系统的速度。为了提高查找页表的速度，通常在地址变换机构中增加一个小容量的联想寄存器，它是一专用的高速缓冲存储器（Cache memory），用来存放页表中当前访问最频繁的表项，又称为快表。

在具有高速缓冲存储器的分页式管理系统中，需要进行地址转换时，首先根据逻辑地址中的页号查找快表，若找到该页所对应的块号，立即形成物理地址，否则再到内存页表中去查找。图 2.34 给出了具有高速缓存的地址转换过程。

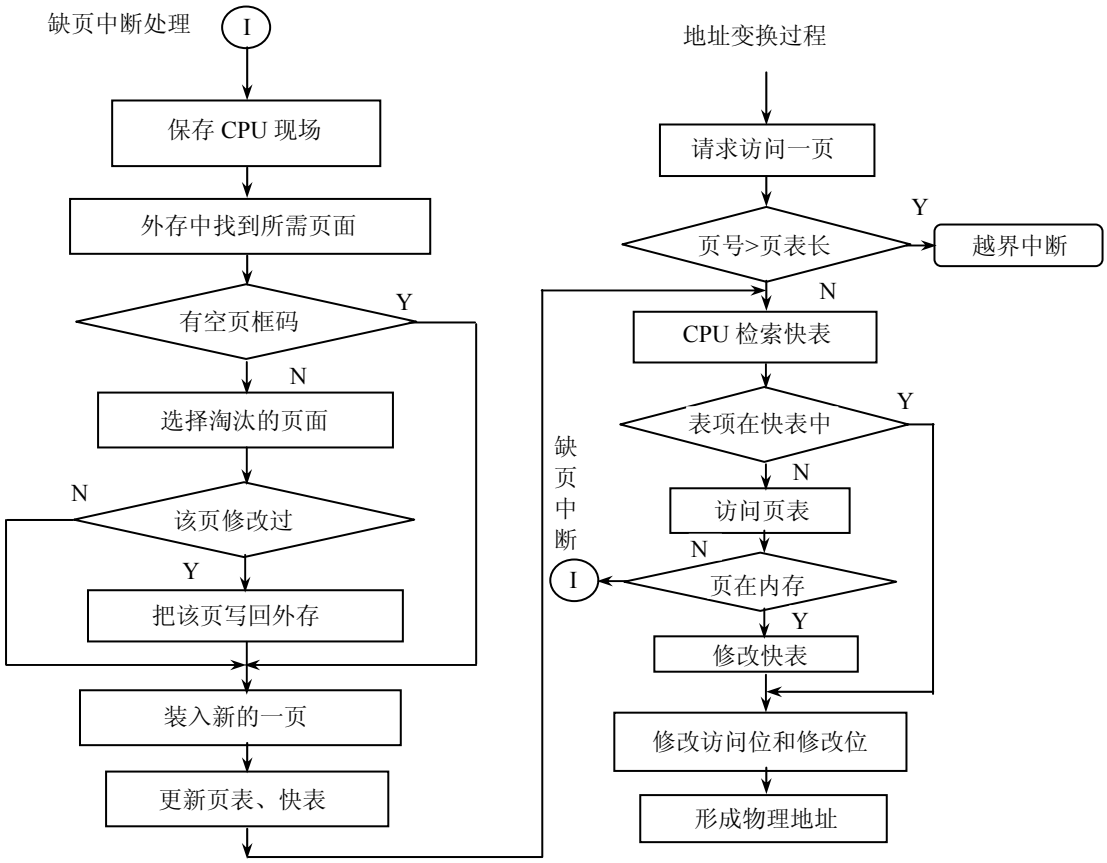


图 2.34　具有快表的请求分页地址转换

在访问内存的过程中，随着进程的推进，快表的内容不断动态更新，页表的少数表目需动态装入快表。当进程交替时，快表被清零。

高速缓存虽然容量小，但由于能按内容关联存放在一段时间内最频繁使用的页表表项，所以访问快表的命中率高达 90%以上，这样可大大提高地址转换速度。快表是硬件对分页存储管理的支持。

四、页面淘汰算法

在处理缺页中断的过程中，当内存无空闲块又需调入新页面时，需将内存中的一些页面置换出去。淘汰页面的算法，又称页面置换策略，是虚拟存储管理的核心问题，将直接

影响到存储管理的性能。若淘汰不当，可能会出现刚被淘汰的页面立即又被调入内存，频繁的内外存交换可能引起“抖动”现象（thrashing），以致大部分 CPU 时间都用于频繁的页面调度上，降低了系统的速度和效率。

常见的淘汰算法有：

（1）FIFO 算法

先进先出 FIFO（First In First Out）算法是一种最简单的淘汰算法。首先淘汰在主存中驻留时间最长的页面。该算法的依据是，由于每个作业装入是依次进行的，一般来说，页号相邻的页之间逻辑关系最密切，所以最早调入主存的页不再被使用的可能性比最近调入主存的页要大。

该算法容易实现，但算法仅在按线性顺序访问地址空间时才是理想的，那些在内存中驻留很久的页面却往往是被经常访问的。此外，该算法还有可能产生异常现象，即当分配给一进程的页面增多时，缺页中断的次数反而增加。

（2）LRU 算法

LRU（Least Recently Used）算法，即最近最久不用页面的淘汰算法，算法是根据一个作业在执行过程中的过去页面踪迹来推测未来的情况。一个已在内存的页面如果在本次缺页中断前的最近一段时间内未被使用的时间最长，那么最近的将来，它也可能不被使用，故应被淘汰。

该算法比 FIFO 算法性能好，但实现起来比较困难。

（3）LFU 算法

LFU（Least Frequently Used）算法，即最近最少使用页面淘汰算法，是将最近一段时间内，使用得最少的页面淘汰掉。实际上 LFU 算法可以看成是 LRU 的一种近似算法。

五、分页式存储管理的特点

请求式分页存储管理是一种虚拟存储管理方式，其主要特点是：

（1）存储效率高

存储效率高的原因是调页方式比其他方式允许更多的程序驻留在主存储器中。由于主存储器是动态分配的，只有当前被访问的那些页面才被调入主存储器中，因此即使大型映象也不需采用覆盖技术或其他方法便很容易被执行。

此外，在分页虚拟存储中不会产生“碎片”问题，因为所有虚页面和存储块的大小相同，任何程序的一个页面都正好能放入一存储块中。

（2）具有较强的存储保护功能

由于用户作业使用各自的页表（PT）在进行地址转换时，首先进行是否“越界”的检查，因此避免了作业间的相互影响。同时页表中的存取控制字段对每个页面进行了保护，每当进行页访问时，系统首先按保护码对进程的访问权进行校验，以确定是否允许访问，这是一种面向自己的保护方式。

（3）方便的重定位

对页面进行重定位时，只需修改页表表目中的块号，就可建立起虚页面与存储块之间的对应关系，方便地将虚地址（逻辑地址）转换为实地址（物理地址）。

（4）易于共享页面

在分页式虚拟存储管理中，通过页表可以很容易地实现页面共享。只要在每个作业的页表表目中设置相同的字段或块号即可。

（5）开销（Overhead）较大

采用分页式存储管理，产生的额外开销有三类：

①比非虚拟系统使用更多的硬件（寄存器）。

②由于地址变换数据结构（如页表）的设置，对存储区域有所损失。

③地址变换过程对运行时间有所损失。

但寄存器的价格不贵，维护地址变换所造成的存储区损失，也可由扩充主存予以补偿。由地址变换所产生的时间延迟，对大多数应用来说并不重要。

六、“抖动”（thrashing）现象与工作集

“抖动”现象是由于在主存与外存之间出现信息过量传送或过多的频繁“调页”而引起的。具体表现在程序执行过程中发生连续缺页，用于等待页面调入的时间超过了 CPU 执行时间，由于系统花费大量时间忙于频繁的页面交换，致使系统效率降低，严重时甚至导致系统瘫痪。因此，一个虚存管理系统必须能有效地防止“抖动”现象的发生。

“抖动”现象与内存中并发的用户进程数及系统给每个进程分配的存储块数有关。图 2.35 给出了 CPU 利用率与用户进程数之间的关系，图 2.36 给出了进程的实页面数与缺页中断率的关系。一般情况下，随着进程数目的增加，各进程所分配的存储块数减少，缺页率增高，从而导致频繁的页面交换。若减少并发进程数，原则上可为每个进程多分配存储块，虽可降低缺页率，但 CPU 利用率也随之下降。因此应考虑选取一个最佳值，兼顾并发进程数及缺页率。

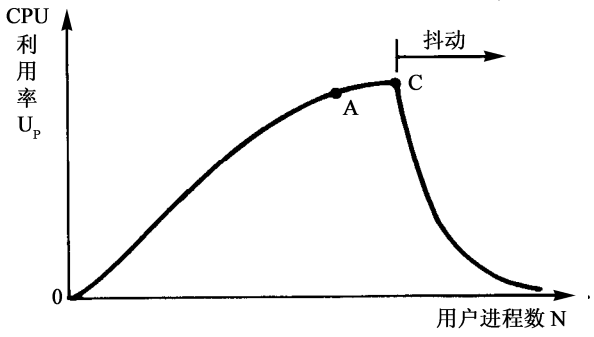


图 2.35　CPU 利用率曲线

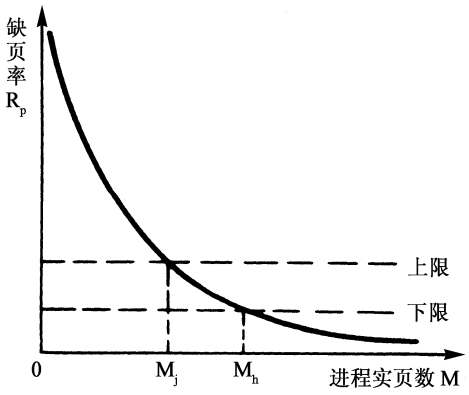


图 2.36　进程的缺页率曲线

采用工作集，可以较好地解决抖动问题。试验表明，任何程序对内存都有一个临界值要求，当内存分配的存储块小于这个临界值时，内存和外存之间的页面交换次数会急剧增加。而内存分配大于该临界值时，再增加内存分配也不能显著减少交换次数。这个内存分配的临界值称为工作集。

将工作集概念用于页面调度，即保证调度一个进程时，内存中应给予分配一定大小的工作集。为了防止过多的进程进入内存，又对工作集的总量进行了限制，例如 VMS 中引入

了平衡集的概念，所谓平衡集是所有在内存中的工作集的总和，且工作集的大小可按照当前内存的负荷进行动态调整。

总之，分页存储管理有效地实现了内存分配的非连续性，解决了碎片问题，从而大大提高了内存利用率。

2.4.6 段式存储管理

分页存储管理的进程逻辑空间结构是线性的。分页是对连续的地址空间的一种固定长度的连续划分，一页通常不是一个完整程序或数据逻辑段。而实际上，用户的作业是按逻辑上有完整的段来划分的；例如主程序、子程序、数据等都可各成一段，每段对应于一个过程、一个程序模块或一个数据集合。

为了给用户提供一个方便灵活的程序设计环境，将作业的地址空间按逻辑意义分段，以段作为内外存交换的单位。这样既利于程序设计，又可方便地按段名访问，易于实现对模块特别是共享模块的保护和共享。

另外，从链接的角度看，分区和页式管理只能采用静态链接方式。由于一个大作业可能包括数百甚至上千个程序模块，对它们进行静态链接需花费大量的 CPU 时间，但实际执行过程中可能某一段时间只用到其中一个子集。而段式存储管理可使用动态链接的方式。

一、分段存储分配

在分段存储管理系统中，一个作业分为若干段，每段都有自己的段名，程序经编译链接后，段名转换为段号，以段号唯一标识一个段，各段长度不等，其长度取决于对应程序模块的大小。各段都是从 0 开始相对编址的连续地址空间。可见作业的地址空间是二维的（见图 2.37）。

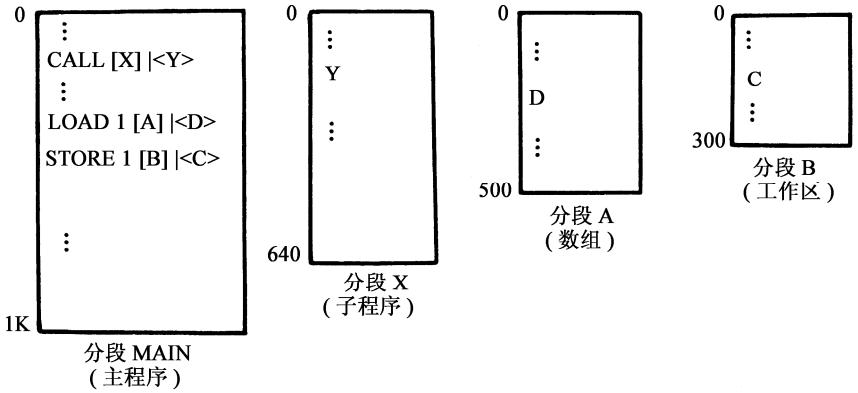


图 2.37　分段地址空间

其逻辑地址由段号及段内地址（段内位移量）两部分组成（见图 2.38）。

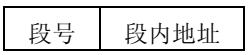


图 2.38　分段逻辑地址

由于段是信息的逻辑单位，它有完整和相对独立的意义，在以段为单位进行存储分配

时，一个段必须分配在内存的一个连续空间中，但各段可以不连续。段长由程序决定并且可以动态延伸。分配策略与可变分区分配类似。

二、动态地址转换

段式管理中，逻辑地址到物理地址的转换机构与分页存储管理类似。系统为每个作业在系统区内建立一张段表 ST（Segment Table）。其典型的表目结构如图 2.39 所示。段表表项按段号顺序从小到大排列，每张段表包含一个作业的所有段。

段号	段长	起始地址	特征位	存取控制	修改位	增补位

图 2.39　段表表目

其中，段长为该段的长度，起始地址是该段在内存的物理起始地址，即绝对地址。

• 特征位——为 1 表示该段在内存，为 0 表示该段不在内存。

• 存取控制——规定对该段的存取权限。以 R 表示允许读，W 表示允许写，E 表示允许执行等。

• 修改位——为 1 表示该段在内存中修改过，为 0 表示该段未修改过。

• 增补位——为 1 表示允许该段动态增长，为 0 表示不允许该段增长。

为了便于查找段表，系统为正在运行的作业建立一个段表始址寄存器，用于存放作业的段表始址。段表的始址寄存器是公用的，整个系统只有一个。作业运行之前，系统将作业的段表始地址及段长填入段表始址寄存器。

段式动态地址转换过程如图 2.40 所示。

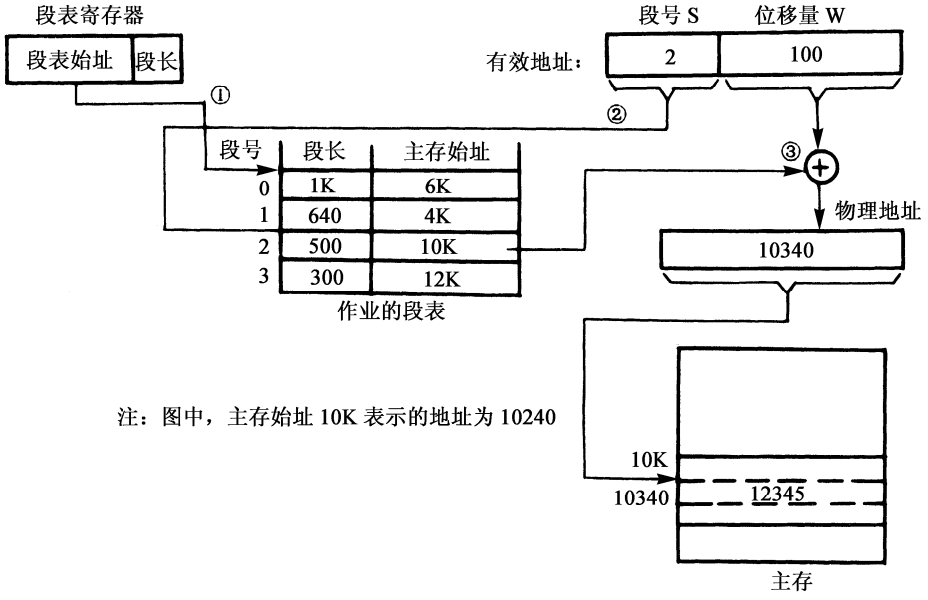


图 2.40　分段地址变换过程

（1）将逻辑地址中的段号与段表寄存器中的段长比较，验证段号合法性。

（2）由段表始址寄存器的段表始址找到段表位置。根据逻辑地址中的段号查找段表，得到该段在主存的起始地址。

（3）同时还应判断该段的特征位，若为 1 表示该段在内存；若为 0 产生缺段中断，应将相应段调入内存；若内存空间不够，应合并空闲区，或按某种淘汰算法淘汰部分段。图 2.41 为缺段处理的流程。

（4）根据“存取控制”字段验证访问合法性后，将该段在主存的起始地址与段内位移量相加，形成物理地址。

为了提高地址转换速度，通常使用快表。

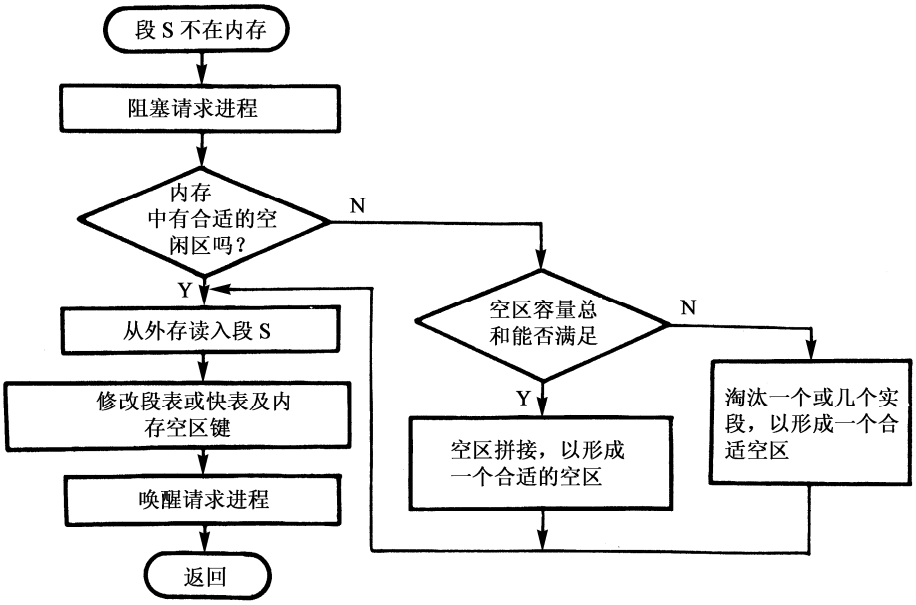


图 2.41　缺段中断处理

三、分段的共享与保护

由于分段存储管理是按逻辑意义来分段，按段名（符号名）进行访问的，因此可以方便地实现分段的共享与保护。

1．段的共享

在多道程序环境中，许多应用程序或子程序等由多个用户作业共享。而且随着多窗口系统的发展，共享的程序或数据越来越多。若共享某程序的所有作业都各自保留一个运行副本，将造成内存空间的极大浪费。实际上只需保留共享段的一个副本。

分段的共享是通过将共享段作为表项登记到共享该段的各作业的段表中实现的。即各作业可以使用不同的段名，只要在各自的段表中填入已在内存中的共享段的起始地址，并设置相应的存取控制权，就可共享一个内存段的信息。

由于共享段可能被多个作业同时调用或被反复调用，因此要求在执行过程中，该段的指令和数据不能被修改。为了协调分段的共享，整个系统建立了一张现行分段表（AST）。如图 2.42 所示，它记录了共享段的状态，其中状态位表示该段是否在主存。当一共享段调入主存时，均在表上登记。

段名	段长	主存始址		状态
共享本段的作业数				
状态	作业名	作业号	段号	存取控制

图 2.42　现行分段表（AST）

2．段的保护

在多道程序系统中，为了保证段的共享及程序的正常运行，必须实行对段的保护。

段式管理保护方法主要有两种：

（1）地址越界保护　在进行地址变换时，利用段号与段表大小比较以及段内位移量与段长比较进行段的保护。

（2）存取控制保护　设置段表中的存取控制字段，对程序的访问权限进行各种必要的限制。

2.4.7　段页式存储管理

分页式存储管理对存储空间的利用率较高，而分段存储管理从程序的逻辑结构出发进行分段，便于实现段的共享、保护及动态链接。但每个分段必须占用主存的连续区域，且内存空间利用率不高，易产生碎片。分页存储管理则较好地克服了分段管理的缺点。因此为将分页和分段管理结合起来，发挥各自优点，克服各自缺点，而引入了段页式存储管理。

一、逻辑地址结构

在段页式存储管理中，对用户作业的地址空间进行分段，段的划分是由程序员或编译程序根据程序的逻辑结构而定。对每个段内再进行分页，即用分页的方法来分配和管理内存。将内存也划分为位置固定、大小相等的若干物理块。分页是由系统自动完成的。

因此在段页式系统中，作业的地址空间仍是二维的，可以看成由段号 S 及段内地址 D 组成，而段内地址又自动分为页号 P 和页内地址 W，其逻辑地址结构如图 2.43 所示，由段号 S、段内页号 P、页内地址 W 三部分组成。

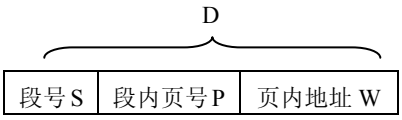


图 2.43　段页式逻辑地址

在一个作业的内存空间中，不仅各逻辑段的物理空间可以是不连续的，而且因为段被进一步分页，所以同一段中各页面也可以是不连续的。

二、地址变换

为实现段页式存储管理的地址变换，系统需要建立段表、页表及控制寄存器。系统为每个作业建立一张段表，而每个段又必须建立一张页表。段表及页表的表目与分段和分页系统中的段表、页表相似。另外，由于在段页式管理中，页表不再属于进程而是属于某个段，因此段表中应有专项指出该段所对应页表的页表始地址和页表长度。段页式管理中段表、页表及内存的关系如图 2.44 所示。

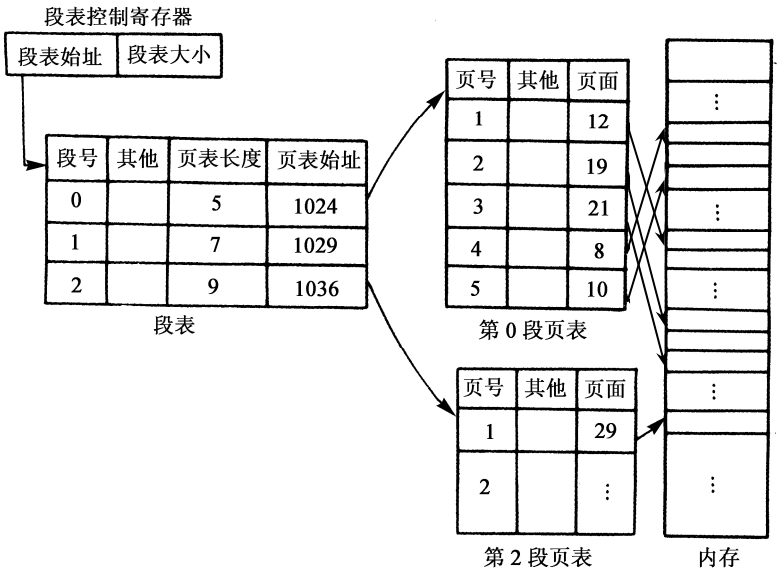


图 2.44　段页式管理中段表、页表与内存的关系

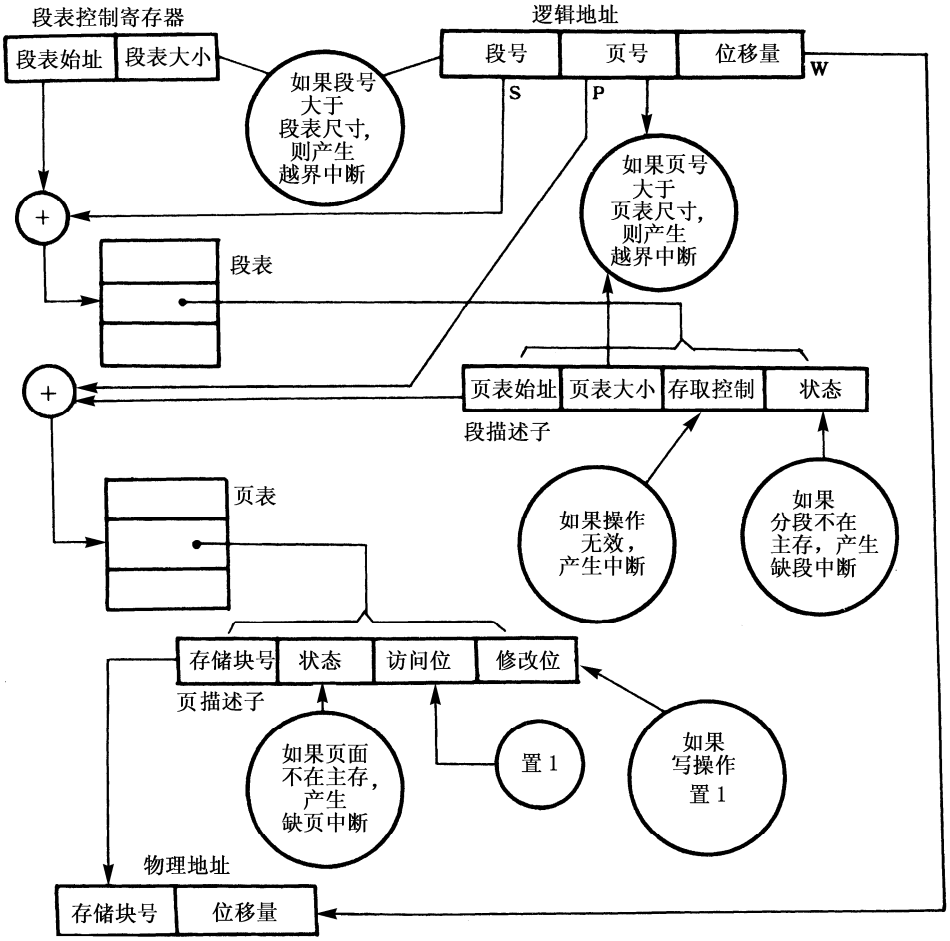


图 2.45　段页式系统地址变换过程

如图 2.45 所示，段页式存储管理的地址变换过程如下：

（1）由段表控制寄存器，查找段表在主存中的始址；

（2）查找段表，根据段号 S 查找页表在内存中的位置；

（3）访问页表，根据页号 P 查找该页所在的物理块号；

（4）将块号与逻辑地址中的页内地址 W 拼接形成实际的物理地址。

在地址变换过程中，若段号大于段表长度或页号大于相应页表长度时，产生保护性越界中断。

在段页式管理系统中，要对内存中指令或数据进行一次存取，至少需要访问三次以上内存。显然降低了地址转换速度，因此在实际的段页式管理系统中通常都引入了快表。

段页式存储管理虽然既方便了用户又有效地利用了内存，但由于增加了软硬件的开销，使操作系统更为复杂。

小　　结

内存管理的基本目的是提高内存的利用率及方便用户使用，具体涉及四个基本问题：内存分配、地址变换、内存保护及内存扩充。内存分配的目标是实现多道程序共享内存，为用户作业分配所需的内存空间。分配方式通常分为静态分配和动态分配。地址变换即是将用户编程的逻辑地址转换为内存空间中的物理地址，保证用户进程的执行。内存保护是保证内存中的各程序间互不干扰。

本节还介绍了各种常用的内存管理方法：分区管理、分页式管理、分段式管理及段页式管理。表 2-2 对这几种存储管理方式进行了比较。

表　2-2

方法 / 功能	分区式（固定分区、可变分区）	页式	段式	段页式
适用环境	多道	多道	多道	多道
虚拟空间	一维	一维	二维	二维
重定位方式	静态、动态	动态	动态	动态
分配方式	静态、动态 分配连续区	动态，页为单位 非连续	动态，段为单位 非连续	动态，页为单位 非连续
释　放	执行完成后全部释放 分区释放	淘汰与执行 完后释放	淘汰与执行 完成后释放	淘汰与执行 完成后释放
保　护	越界保护及保护键	越界保护与 控制权保护	越界保护与 控制权保护	越界保护与 控制权保护
内存扩充	覆盖与交换技术	内外存统一 管理的虚存	内外存统一 管理的虚存	内外存统一 管理的虚存
共　享	不　能	一　般	方　便	方　便
硬件支持	保护用寄存器 地址变换机构	地址变换机构 中断机构 保护机构	段式地址变换 机构，保护与中 断机构，链接机构	段页式地址 变换机构， 保护与中断 机构，链接机构

2.5 设 备 管 理

设备管理是操作系统重要而又基本的组成部分，特别是在一个多用户、配备有多种输入输出设备的计算机系统中，更需要对设备进行有效管理。本节主要讨论设备管理的基本任务、功能，I/O 数据传输方式，缓冲技术及虚拟设备管理技术。

2.5.1 设备管理概述

在计算机系统中，除中央处理机和主存外的所有设备都称为外部设备。这些外部设备种类繁多，特性各异，操作、控制方式差异较大，导致了设备管理的复杂性和多样化。

一、设备的分类

对设备进行分类的目的在于简化设备管理程序。对于同类设备来说，由于它们的硬件特性十分相似，管理程序可以相同或只需作很少的修改。

按工作特性，可以把外设分为输入输出设备和存储设备两大类。

（1）I/O 设备

I/O 设备用于接收来自计算机的外部信息和将计算机的内部信息向外传送，由于这类设备上的信息，往往是以字符为单位组织的，并以字符为单位存取，故也称这类设备为字符设备。如键盘、终端、打印机等都是字符设备。

输入设备是把模拟信号通过模数（A/D）转换部件变为计算机可接收的数字化电信号。输出设备则将计算机的数字化信号通过数模（D/A）转换为模拟量。

（2）存储设备

用于保存信息的设备，由于这类设备上存储的信息在物理上往往是按字符块组织并以块为单位存取的，因此也称这类设备为块设备。如磁盘、磁带等都属块设备。这类设备通常又由存取控制和信息载体两部分构成。

而从资源分配的角度又可将外部设备分为：

（1）独占设备　这类设备多为慢速的 I/O 设备，一旦设备分配给某作业后便由该作业独占，直到该作业完成并释放后，才能被其他作业使用。此类设备管理简单，但利用率低。

（2）共享设备　允许多个作业同时使用的设备。通常为高速大容量的设备，如磁盘等允许多个进程交替访问，共享设备利用率高，但管理较独占设备复杂。

（3）虚拟设备　为了克服独占设备的缺点，引入了虚拟设备管理技术，如 SPOOLing 系统。利用该技术，可以将一台独占设备虚拟为多台虚拟设备。虚拟设备技术的使用有效地提高了设备的利用率。

二、设备管理的任务和功能

设备管理的主要任务是：

①向用户提供使用外设的方便接口。按照用户的要求和设备的类型，控制设备的工作，完成用户的输入输出请求。

②充分发挥设备的使用效率，提高 CPU 与设备之间、设备与设备之间的并行工作程度。在多道程序环境下，按一定策略对设备进行分配和管理，保证设备高效运行。

为了完成上述任务设备管理应具备下述功能：

①建立统一的且独立于设备的接口。

②按照设备类型和相应算法，进行设备的分配与回收。

③进行设备驱动，实现真正的 I/O 操作及设备间的并行操作。

④实现输入输出缓冲区管理，解决高速 CPU 与慢速设备速度不匹配的问题。

⑤实现虚拟设备管理。

2.5.2 数据传送控制方式

设备管理的主要任务之一是控制设备与内存或 CPU 之间的数据传送。选择控制方式的原则是：保证在足够的传送速度下数据的正确传送。要尽可能减少系统开销，充分发挥硬件资源能力，即使 I/O 设备尽量忙，而 CPU 等待时间少。

外设与内存间常用的数据传送方式有以下几种：

（1）中断控制方式；

（2）DMA 方式；

（3）通道方式。

一、中断控制方式

为了使用中断（Interrupt）方式控制外围设备和内存与 CPU 之间的数据传送，要求 CPU 与设备间有相应的中断请求线，设备控制器的状态寄存器有相应中断位。

当进程需要输入数据时，通过 CPU 发出指令启动外设，进程应放弃处理机，等待输入完成。在启动外设后，进程调度程序转去调度别的进程占有 CPU。当数据输入完成后，I/O 控制器发出中断请求信号，CPU 在接收到中断信号之后，转向中断处理程序对数据进行处理。图 2.46 给出了中断控制方式的处理过程。

中断控制方式虽实现了 CPU 与设备间的并行操作，但由于 CPU 直接控制 I/O 操作，每传送一个字符，控制器就要向 CPU 发一中断信号，CPU 就要作一次中断响应，这将耗去大量的 CPU 处理时间。若多种外设都采用中断方式，则中断次数急剧增加，造成 CPU 无法响应中断及数据丢失，所以中断控制方式不适宜需要交换大量数据的情况。

二、DMA方式

为了减少中断次数，提高 CPU 利用率及设备的并行工作能力，常采用 DMA（Direct Memory Access）方式，即直接内存访问方式。由于绝大多数的小型、微型机都采用总线结构（见图 2.47），DMA 方式的基本思想是：“窃取”或“挪用”CPU 总线的控制权。它要求 CPU 暂停使用若干总线周期，由 DMA 控制器占用总线来进行 CPU 与设备间的数据交换。

在数据块传送期间，不产生中断，不需要 CPU 干预。当数据块传输完后，DMA 控制器才归还 CPU 控制权，并向 CPU 发出中断请求信号。

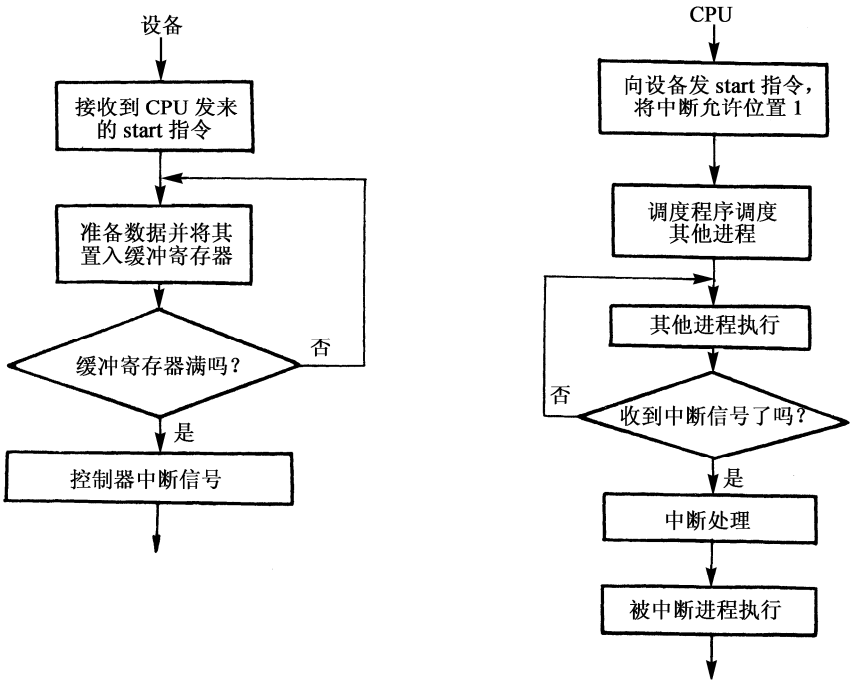


图 2.46　中断处理过程

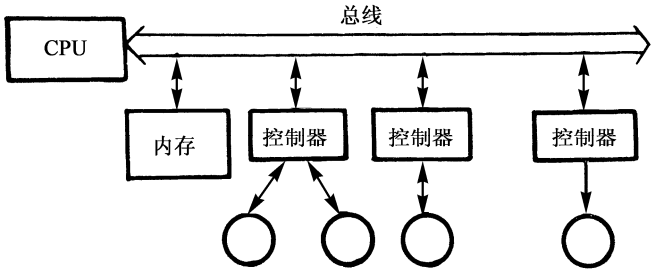


图 2.47　总线结构的计算机硬件组织

块设备控制器一般都支持 DMA 方式，DMA 方式传送数据的单位是数据块，有效地减少了中断次数。以磁盘的数据输入为例（见图 2.48），DMA 方式进行数据输入的过程如下：

①预置 DMA 的寄存器

数据输入进程在要求磁盘控制器输入一批数据之前，将传送的字数送入 DMA 控制机构的传送字数计数器，将接收数据的内存起始地址送入 DMA 控制机构的内存地址寄存器。

②启动 DMA 传送一个数据块

将 DMA 硬件控制机构中控制状态寄存器的允许中断位和启动位置为 1，从而启动 DMA 工作。此时 DMA 硬件控制机构“接管”系统总线控制权，将缓冲器中的数据通过系统总线，

一个字一个字地传送到内存地址寄存器所指定的内存单元，每传送一个字，内存地址寄存器加 1，传送字数寄存器减 1。

③睡眠并等待 DMA 传送结束

启动 DMA 工作后，数据输入进程调用 Sleep 过程进入睡眠状态，等待数据传送结束。同时进程调度程序 Switch 挑选其他进程投入 CPU 运行。

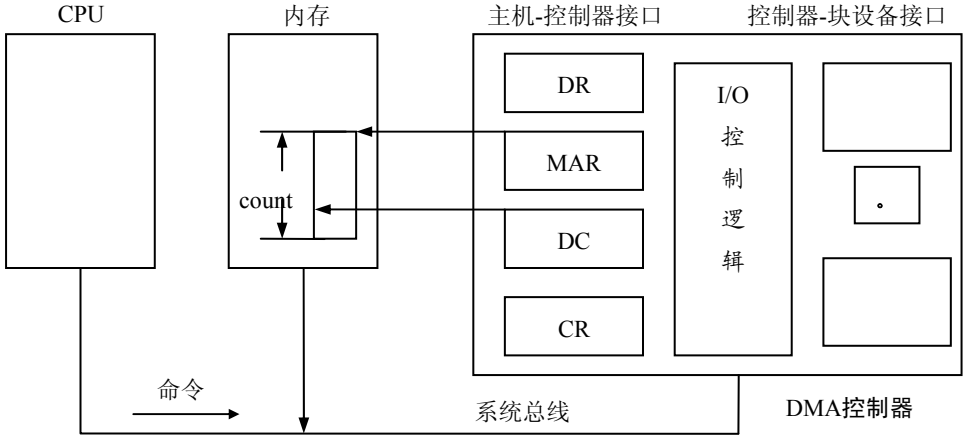


图2.48　DMA控制机构

④I/O 中断处理

当一批数据传送完毕，即传送字数计数器为 0 时，磁盘控制器产生中断请求，中断响应后，中断正在运行的进程并转向中断处理程序，中断处理程序在保护现场后调用过程 Wakeup，唤醒处于睡眠的数据输入进程，然后恢复中断现场，返回被中断的进程。

⑤唤醒数据输入进程

由于执行 Wakeup 调用，使输入进程从阻塞状态变为就绪状态，在就绪队列中排队等待再次被调度。

三、通道方式

通道（Channel）是比 DMA 控制机构更完善，功能更强的 I/O 控制机构。与 DMA 一样，也是一种以内存为中心，实现设备和内存直接交换数据的控制方式，每次可传送一组数据块。

但与 DMA 方式不同，通道是一种专门控制 I/O 工作的简单的处理机，也称 I/O 处理机，它有自己的简单指令系统，其指令称为通道控制字（CCW）。由 CCW 编制成通道程序存放在内存，用于实现对外设的 I/O 操作的控制。

通道结构的三级控制过程如下：

①CPU 启动通道

通道程序的起始地址存放在一个称为通道地址字（CAW）的固定内存单元中，由 CPU 执行“启动 I/O”的指令，启动通道程序，若通道可用，启动成功，则 CPU 可转去执行其他任务。

②执行 CCW

通道被启动后，根据 CAW 访问通道程序，逐条执行 CCW，向设备控制器发出 I/O 操作命令。

③控制器执行 I/O 操作命令，控制设备

设备控制器启动设备，经通道在内存与 I/O 设备之间传送数据；输入设备读入数据经通道送往指定的内存区，或将指定内存区的数据经通道送输出设备输出。

在通道方式下，主机与 I/O 设备间数据交换是经通道进行的。通道独立执行 I/O 任务，无需 CPU 干预，同时实现了多部件的并行处理，即 CPU 与设备之间，设备与设备之间的并行处理，从而显著提高了 CPU 的利用率，也提高了设备的利用率。

通道按信息交换的方式分为三类（见图 2.49）。

①字节多路通道

用于连接大量中、低速的并以字节为传送单位的 I/O 设备。通道以多路方式执行 I/O 操作，即当一个设备传送完一个字节后，便立即腾出通道让另一个设备使用。

②选择通道

用于连接如磁盘这样的高速外存，它一次连续执行一道程序，只有执行完一道程序后才能转去执行另一道通道程序，因此，一段时间只能控制一台设备，但能连续、高速地以块为单位传送一批数据。

③数组多路通道

用于如磁带机等中速块设备。兼有字节多路通道的并行性和选择通道的高速成批传送的特点。

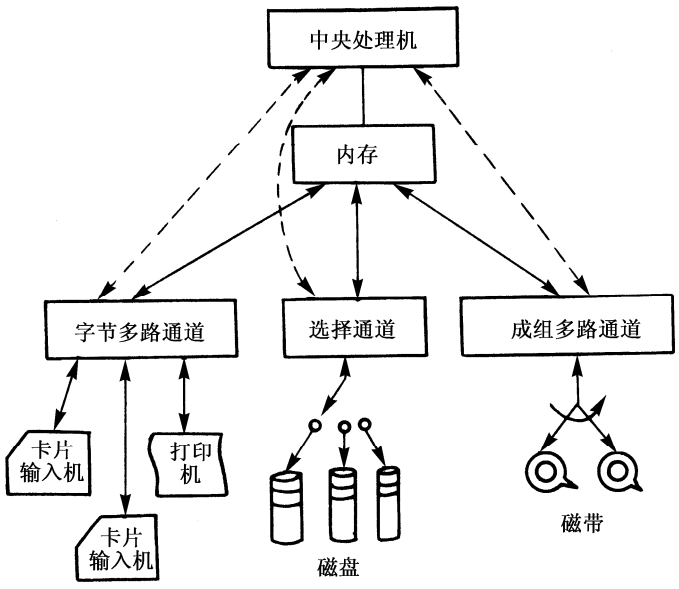


图 2.49　通道结构

2.5.3　缓冲技术

中断、DMA 及通道技术的引入为 CPU 与外设之间的并行操作提供了可能性。但 CPU 与外设之间的速度不匹配，以及外设频繁地中断 CPU 的运行，仍会降低 CPU 的利用率，限

制了与 CPU 联结的外设数量。

为了解决外设与 CPU 速度的匹配问题，减少中断次数和 CPU 的中断处理时间，也为了解决 DMA 或通道方式时的瓶颈问题，在设备管理中引入了用于暂存数据的缓冲技术。

缓冲技术的基本思想是：在内存中开辟一个或多个专用的区域，即缓冲区，作为 CPU 与 I/O 设备之间信息传输的集散地。

一、缓冲区的组织

按照使用缓冲区的方式可分为专用缓冲区和公用缓冲区。专用缓冲区为某台设备专用。公用缓冲区则为多个或所有设备而设置。

在操作系统中常按开辟缓冲区的数量分为单缓冲区、双缓冲区、多缓冲区及缓冲池几类。

（1）单缓冲区

在设备和处理机之间设置一个缓冲区。当处理机与设备间传递数据时，先将数据写入缓冲区，需要数据的处理机或设备再从缓冲区中取走数据。由于缓冲区属于临界资源，不允许多个进程同时对缓冲区操作，因此单缓冲区只能起到匹配处理机与设备的速度的作用，而不能实现设备间的并行操作。

（2）双缓冲区

所谓双缓冲区，即设置两个大小相等的上、下缓冲区。可以解决两台外设，例如打印机和终端之间的并行操作。CPU 将输出到打印机的数据放入其中一个缓冲区，让打印机慢慢打印；另一缓冲区则存放从终端输入的数据，CPU 又从中读取所需要的输入数据。

两个缓冲区交替进行输入、输出，实现 I/O 设备的并行操作。

（3）多缓冲区与缓冲池

为了实现多个外设的并行工作及解决与 CPU 的速度匹配问题，现代计算机系统中一般使用多缓冲区或缓冲池结构。

多缓冲区是将多个缓冲区连接起来组成两部分，一部分专门用于输入，而另一部分专门用于输出。缓冲池则是把多个大小相等的缓冲区连接起来，统一管理，既可作为输入也可以用于输出。缓冲池的使用更有效地利用了缓冲资源。

显然，无论是对多缓冲区还是缓冲池的管理，都涉及对其缓冲区的申请、释放和互斥的问题。

二、缓冲池的管理

以缓冲池为例，介绍缓冲管理技术。

1．缓冲池的结构

为了讨论缓冲池的管理，应先了解缓冲池的组成。缓冲池由多个缓冲区组成，每个缓冲区又由缓冲区首部和缓冲体两部分组成。首部通常包括缓冲区号、设备号、设备上的数据块号及缓冲队列的链指针等有关缓冲区标识和管理的信息（见图 2.50）。缓冲体用于存放数据。

设备号
缓冲区号
缓冲区地址
缓冲区状态（忙/闲）
数据块号
互斥标识位
设备缓冲队列指针

图 2.50　缓冲区首部

根据缓冲区的使用情况，将缓冲区连接成三种队列：

（1）空缓冲区队列（emq）。

（2）满输入缓冲区队列（inq）。

（3）满输出缓冲区队列（outq）。

2．缓冲池的管理

图 2.51 给出了一种典型的缓冲池管理方式，对缓冲池管理包括以下操作：

①收容输入　输入进程调用 getbuf（emq），从空白队列 emq 中申请一空白缓冲区，作为收容输入数据的工作缓冲区（hin）。当输入设备输入的数据将 hin 装满后，调用 putbuf（inq，hin），将缓冲区 hin 插入满输入队列的队尾。

②提取输入　当需要处理输入数据时，计算进程调用 getbuf（inq），从满输入队列的队首取出一缓冲区（其中装满了已输入的数据）作为提取输入的工作缓冲区（sin），计算进程从中提取所需的数据进行处理，当 sin 中的数据处理完后，调用 putbuf（emq，sin）释放工作缓冲区 sin 并将其插入空闲缓冲区队尾。

同理，不难得到收容输出和提取输出的处理过程。其中 sout 和 hout 为提取输出工作缓冲区和收容输出工作缓冲区。

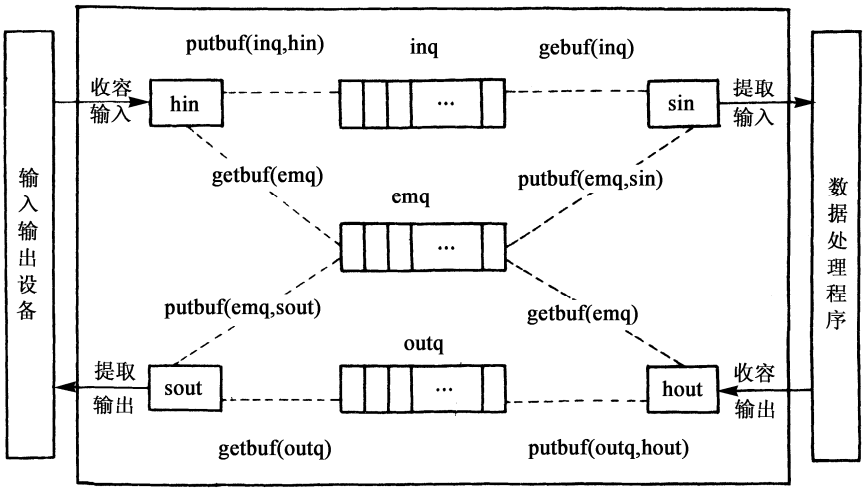


图 2.51　缓冲池管理

缓冲池的使用进一步改善了 CPU 和外设工作的并行程度，提高了资源的利用率。

2.5.4　设备分配

前面讨论 I/O 数据传送控制方式及相关的中断与缓冲技术时，都是假设进程已经申请到所需要的外设。事实上，在多道程序系统中，由于资源是有限的，不是每个进程都随时能申请到资源，因此设备的分配问题是设备管理的重要问题。下面我们讨论与设备分配相关的数据结构、分配原则及分配策略和算法。

一、用于设备分配的数据结构

设备的分配管理通过以下数据结构进行：

1．设备控制表 DCT（Device Control Table）

设备控制表 DCT 反映设备的特性，设备和 I/O 控制器的连接情况，通常包括以下字段：

（1）设备标识，用于区别设备。

（2）设备类型，反映设备的特性，表示磁盘、磁带或是打印机、终端等。

（3）设备状态，表示设备是忙或是空闲。

（4）设备地址或设备号，每个设备的地址可以统一编址，也可单独编址。

（5）设备等待队列的指针，其队首和队尾指针都存放在 DCT 中。

（6）I/O 控制器指针，该指针指向与该设备相连的 I/O 控制器。

系统中的每个设备都有一张 DCT，表中的内容在系统执行时可动态修改。

2．系统设备表 SDT（System Device Table）

整个系统一张 SDT，用于记录已被连接到系统中的所有物理设备的情况，包括系统中各设备的状态、数量等。每个物理设备在 SDT 中占有一个表项。表项的内容包括：

（1）DCT 指针，指向该设备的 DCT 表。

（2）正在使用该设备的进程标识。

（3）设备类型、标识符。

3．控制器表 COCT（Controler Control Table）

每个控制器一张表，反映 I/O 控制器的使用情况及与通道的连接情况（DMA 方式时无此项）。

4．通道控制表 CHCT（Channel Control Table）

每个通道一张，用于描述通道的使用情况，包括通道标识符、通道状态（忙 / 闲），以及等待用该通道的进程的等待队列。

数据结构表见图 2.52。

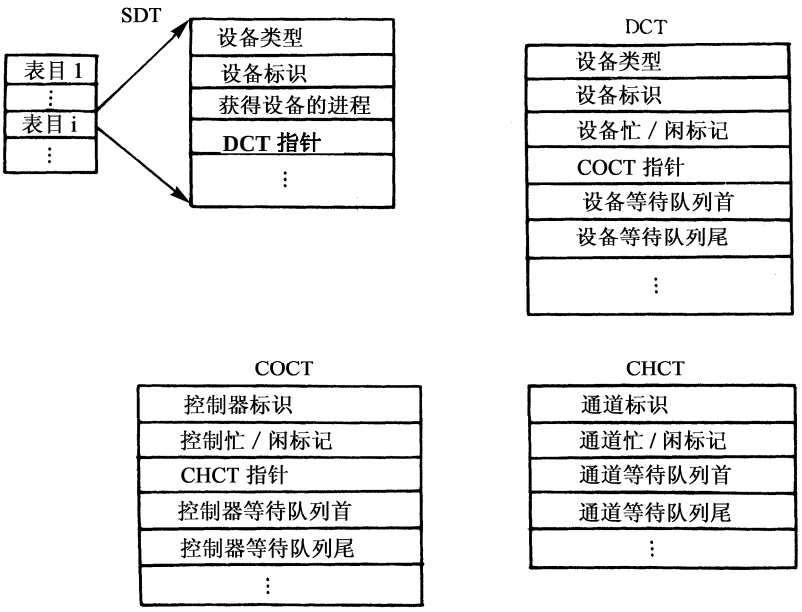


图 2.52　数据结构表

二、设备分配的原则及方式

设备分配的原则由设备特性及用户请求、系统中设备使用情况而定。总的原则是既要充分发挥设备的效率，又要防止由于不合理分配造成死锁。

设备分配的方式通常分为静态分配和动态分配。静态分配方式是在用户作业执行前将其所需要的全部设备及控制器、通道等一次分配，并由该作业占用直至作业被撤销。这种方式虽不会出现死锁且分配策略简单，但设备利用率不高。

动态分配则是在作业执行过程中根据需要，按照一定策略进行设备分配。当设备使用完后立即释放。动态分配方式有利于提高设备的利用率，但分配算法较静态分配复杂，并应注意防止产生死锁。设备分配的一般过程如图 2.53 所示。

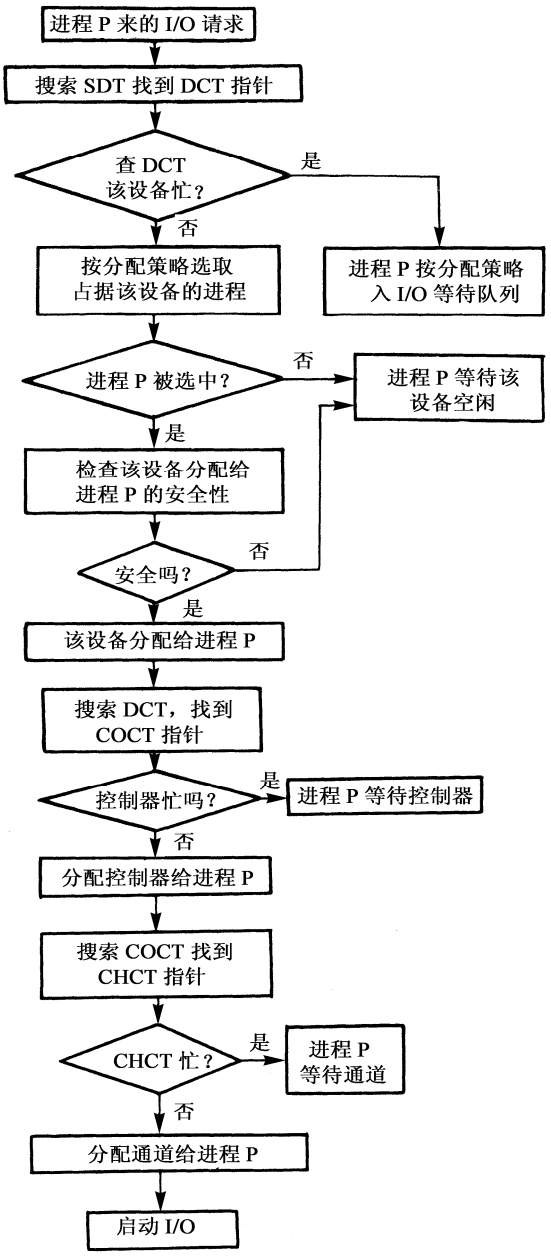


图 2.53　设备分配流程图

由于设备使用方式不同，又可采用不同的分配方式。

（1）独占设备的分配

独占设备通常采用静态分配方式。分配策略可使用先来先服务的算法或最高优先权算法。由于这种管理方式利用率较低，也可采用动态分配方式，即把设备分配推迟到进程申请使用设备时，但实施动态分配方式则必须考虑系统的安全性。

（2）共享设备的分配

由于多道程序系统中各进程对共享设备的访问频繁，因而对共享设备的使用是否合适直接影响着系统的效率。

因共享设备通常采用动态分配方式，操作系统应随时准备进行设备的分配和回收，还要保证多个进程互斥地使用设备及改善系统性能，因此共享设备的管理较复杂。

2.5.5 虚拟设备管理与 SPOOLing 技术

由于多道程序系统中对外设需求频繁，但设备数量不足尤其是独占设备不仅数量有限且利用率低。即使采用缓冲技术，也只能缓解速度不匹配的问题，加之输入输出信息的时间、数量上的随机性，更难于满足诸多进程对设备的需求。因而采用虚拟设备技术，其核心思想是用大容量的快速设备（如磁盘）模拟慢速度的独占设备，使一台独占设备，成为若干台可并行操作的虚拟设备，即把独占设备变成逻辑上的共享设备。

SPOOLing 是一种典型的虚拟设备技术，SPOOLing 是 Simultaneous Peripheral Operations On Line（外围设备同时联机操作）的缩写。SPOOLing 系统的组成如图 2.54 所示。

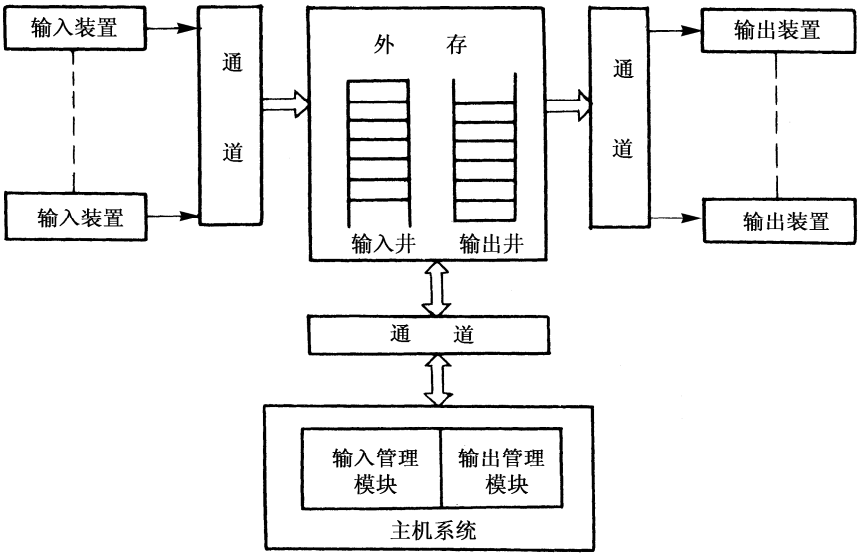


图 2.54　SPOOLing 系统

当用户作业进入系统时，由 SPOOLing 系统的预输入程序将作业信息从物理输入设备送入输入井；当用户进程运行过程中需数据时，从输入井读入内存；当输出数据时，先将要输出的数据送到输出井；然后，缓冲输出程序再依次将输出井上的数据送到物理的输出设备上。

以打印机为例，说明 SPOOLing 系统的工作原理。当某进程要求打印输出时，操作系统不是把实际的打印机分配给它，而是在磁盘上的输出井中为其分配一块区域，将要打印的数据送入。输出井上的区域相当于一台虚拟的打印机。在输出井中形成各进程的打印队列，再由 SPOOLing 系统的缓冲输出程序依次输出到打印机打印。从用户角度看，他似乎独占打印机；从系统角度看，同一台打印机又可以分时地为每个用户服务。

由于从外观上看，SPOOLing 系统中信息的输入输出好像是在硬盘和输入输出设备之间脱机进行的，因此又将 SPOOLing 系统称为假脱机系统。

SPOOLing 技术不仅缓冲了 CPU 与设备速度的不均匀性，同时也提高了 CPU 与设备的并行性。

2.5.6 I/O 管理

I/O 管理是操作系统中最庞杂，最琐碎的部分，而又是与用户关系最密切的部分。

一、I/O控制系统

我们把从用户进程的输入/输出请求开始，给用户进程分配设备和启动有关设备进行 I/O 操作，以及在 I/O 操作完成之后响应中断，进行善后处理的全过程称为 I/O 控制过程。I/O 过程是由输入输出控制系统（IOCS）完成。但由于 I/O 设备品种繁多、特性各异，加之不同计算机系统的 I/O 控制硬件也可能不同。因此，从 I/O 管理的目标及实现 I/O 控制过程的角度，给出一种典型的 I/O 控制系统的层次模型（见图 2.55）。

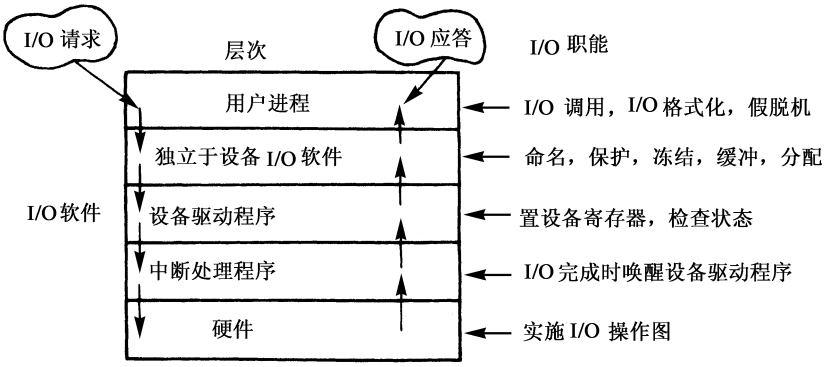


图 2.55 I/O 系统的层次模型

当用户进程向 IOCS 发出 I/O 请求时，独立于设备的 I/O 处理程序查找系统的逻辑设备表（LUT），将用户进程的逻辑 I/O 请求转换为物理请求。独立于设备的 I/O 处理程序向用户提供一个统一的接口，并实现设备保护、缓冲区管理、设备分配和释放等功能。

设备驱动程序是用于控制某个或某类外设 I/O 操作的子程序的集合，驱动设备进行输入输出操作。

当数据传送结束后，外设发出中断请求，I/O 控制过程将调用中断处理程序和做出中断响应。对于不同的中断，其处理不同。例如处理中断结束时，要释放相应的设备、控制器和通道，并唤醒正在等待该操作完成的用户进程。

二、I/O设备驱动程序

设备驱动程序是 I/O 系统中与物理设备密切相关的软件，也是最繁杂的部分，是 IOCS 的主体。一般系统为每类设备设置一个驱动程序。

设备驱动程序一般分为启动过程和继续过程。启动过程的功能是启动一个 I/O 操作，继续过程完成数据传送及调用中断处理程序进行中断处理。一个典型的设备驱动程序包括以下具体过程：

（1）服务请求校验

确定 I/O 请求的类型，检验 I/O 操作是否有相应硬件支持。若不符合要求，则返回相应信息，并决定是中断或继续执行进程。

（2）确认设备状态

CPU 与 I/O 设备间通常有一接口，接口包括数据寄存器、状态寄存器、操作方式寄存器等。驱动程序应在传送数据前，将设备状态寄存器的内容读入 CPU 进行状态确认，并将设备状态返回发出 I/O 请求的进程。确认设备状态也即是确定设备是否可用。

（3）启动 I/O 请求

驱动程序确认“设备状态”为可用后，通过向设备控制寄存器写入一个或多个控制字，启动 I/O 操作。

（4）中断处理

在 I/O 操作过程中，外设发出中断信号，CPU 应作出中断响应，此时驱动程序应保存处理器的当前状态，以便在中断处理后，重新执行被中断的进程。

（5）I/O 请求完成

当 I/O 请求完成后，驱动程序应能识别，并将控制返回 IOCS。将被中断用户进程从等待状态转换到就绪状态，并插入就绪队列等待 CPU 调度。

小　　结

由于计算机的外设种类繁多、特性各异，导致了设备管理的复杂性和多样化。设备管理的主要任务是：（1）向用户提供使用外设的方便接口，完成用户的 I/O 请求。（2）控制 CPU 与设备之间的操作，提高其并行工作程度，充分发挥设备的使用效率。

常用设备与 CPU 间数据传送分为中断控制方式、DMA 方式和通道方式。中断控制方式虽然实现了 CPU 与设备间的并行操作，但由于中断次数多，CPU 需花费较多时间处理中断，对并行操作的设备的台数也受到限制。DMA 方式和通道方式较好地解决了上述问题，这两种方式采用了外设和内存直接交换数据的方式。只有在一数据块传送结束后才发出中断信号，大大提高了 CPU 利用率。DMA 方式与通道控制方式的区别是：DMA 方式要求 CPU 执行设备驱动程序启动设备，给出存放数据的内存地址、操作方式和传送字节长度等；而通道是一种专门控制 I/O 工作的简单处理机，有自己的指令系统，当 CPU 启动通道后，I/O 操作由通道控制完成。

I/O 系统结构从低到高大致分为三层：I/O 中断处理程序，设备驱动程序，独立于设备的 I/O 软件（即逻辑 I/O 系统）。底层和中层与设备的物理特性有关，设备驱动程序是 I/O 系统的主体，它们实现具体的 I/O 操作，驱动设备完成主机与设备之间的数据交换。逻辑 I/O 系统是与设备特性无关的部分，其功能为处理用户进程的 I/O 请求，实现逻辑 I/O 请求到物理请求的转换，进行设备与缓冲区的分配。

I/O 系统所使用的基本数据结构包括：系统设备表（SDT），设备控制表（DCT），控制器表（COCT）及通道控制表（CHCT）等。

为解决外设与 CPU 速度的匹配问题引入了缓冲技术。缓冲区的管理是 I/O 系统的基本功能。缓冲区按组织方式分为单缓冲区、双缓冲区、多缓冲区及缓冲池。公用缓冲池是被广泛采用的一种缓冲区管理方案。

设备分配是逻辑 I/O 系统的基本功能之一，分静态分配和动态分配两种方案。动态分配有利于提高设备利用率，但要考虑分配的安全性。

SPOOLing 是一种虚拟设备管理技术，是用可共享的、高速的硬盘来模拟独享设备，从

而提高设备利用率。

2.6 文 件 管 理

计算机的重要作用之一是快速处理大量信息，因此，信息的组织、存取和保管就成为一个极重要的内容，文件系统是计算机组织、存取和保存信息的重要手段。本节讨论文件的组织结构、存取结构、保护以及文件系统空间管理等问题。

2.6.1 文件系统的概念

操作系统的主要功能之一是对计算机资源的管理。计算机资源分为硬件资源与软件资源两大类。前面所讨论的处理机管理、存储管理及设备管理都是对硬件资源的管理，主要解决硬件资源的有效和合理利用问题。而对软件资源的管理则是对各种系统程序（包括操作系统本身）、应用程序、库函数及用户程序等进行组织、管理。由于软件资源是以文件形式存储的，因此，对软件资源的管理也就导致了文件系统的出现。

一、文件的分类

为了便于对文件进行管理和控制，常从不同角度对文件进行分类。

（1）按文件的性质

可分为系统文件、库文件、用户文件等。系统文件只允许用户通过系统调用来执行它们，而不允许对其进行读写和修改。这些文件主要由操作系统的核心模块和各种系统应用程序及数据组成。库文件允许用户进行读、运行，但不允许修改。库文件主要由各种标准的子程序库组成。用户文件的访问权通常可由系统缺省给出或由用户自己确定。

（2）按文件内容

可分为普通文件、目录文件及特殊文件。普通文件是指组织格式为系统中所规定的一般格式的文件，既包括系统文件，也包括用户文件及应用程序文件等。而目录文件是由文件的目录信息构成的文件，其内容是用来检索普通文件的目录信息。特殊文件通常是指输入、输出的设备，从广义上可将 I/O 设备都看成是文件。

（3）按文件的保护级别

分为只读文件、读写文件及可执行文件等，使其对文件的访问权限进行有效的控制。

此外，还可以按照文件的存取方式分为顺序存取文件和随机存取文件或按信息流向分为输入文件、输出文件或输入输出文件等。

二、文件系统的功能

文件系统是操作系统中负责管理和存放文件信息的软件机构，它向用户提供了一种简便、统一的存取和管理信息的方法。

具体来说，文件系统应具有以下功能：

（1）统一管理文件的存储空间（外存），实施存储空间的分配与回收。

（2）确定文件信息的存放位置及存放形式。

（3）实现文件从名字空间到外存地址空间的映射，即实现文件的按名存取。

（4）实现对文件的控制操作（如建立、撤销、打开、关闭等）和存取操作（如读、写、修改、复制等）。

（5）实现文件信息的共享，并提供可靠的文件保护与保密措施。

总之，文件系统提供了用户与外存的界面，用户可通过文件名，按照文件的逻辑结构，使用简单直观的操作存取和管理所需的信息。

2.6.2 文件的组织

文件组织是指文件中信息的配置和构造方式。通常从两种角度来研究文件的组织，从用户的角度，研究用户的抽象文件形式，即逻辑文件或文件的逻辑结构；另一种是从实现的角度出发，研究逻辑文件在文件存储介质上的存放形式，即物理文件或文件的物理结构。

一、文件的逻辑结构

文件的逻辑结构是用户可见的结构。文件的逻辑结构可分为两大类：字符流式的无结构文件和记录式的有结构文件。

1．流式文件

无结构的流式文件是相关的有序字符的集合。比如源程序文件、目标代码文件等属于流式文件。流式文件可视为一有序的字符流。流式文件中包含的字符个数即是该文件的长度。

流式文件的管理简单，将对文件的解释留给用户，因而用户可以不受约束地灵活组织其文件的内部逻辑结构。

2．记录式文件

记录式文件是一种有结构的文件，一个记录式文件由若干逻辑记录组成，每个逻辑记录由彼此相关的数据项（域）构成。例如一个班某门课程的成绩可用一个记录式文件表示。一个学生的成绩可以用一条逻辑记录表示，每个逻辑记录又由学号、姓名、成绩等数据项组成。

记录式文件又分为定长记录式文件和变长记录式文件。定长记录式文件的每个逻辑记录长度相等，而变长记录式文件的各逻辑记录长度不等。

从操作系统管理的角度看，逻辑记录是存取信息的基本单位，它包含一个记录链和其他属性，记录式文件可把记录按各种不同的方式排列，以便用户对文件中的记录进行修改、追加、查找和管理等操作。

3．文件的存取方式

用户通过对文件的存取来完成对文件的各种操作。有两种基本的文件存取方式：顺序存取方式和随机存取。

顺序存取是指严格按照文件基本单位的逻辑排列顺序，依次逐个地存取。对记录式文件是按记录的排列顺序来存取。例如，若当前读取的记录为R_i，则下一次读取的记录被自动确定为R_{i+1}。对流式文件，顺序存取时读写指针自动移动变化。

随机存取则允许用户根据记录键存取文件的任一记录，或者根据存取命令把读写指针

移到指定位置。

二、文件的物理结构

文件的物理结构和组织是指逻辑文件在物理存储空间中的存放方法和组织关系，它主要分为连续、链接和索引三种存储结构。通常，文件的物理结构以块为单位进行组织。由于文件的物理结构决定了文件信息在存储设备上的存储位置，因此，文件信息的逻辑块号（逻辑地址）到物理块号（物理地址）的转换也是由文件的物理结构决定的。

1．连续文件

也称顺序文件，是一种最简单的物理结构，是将逻辑文件的信息依次存放在连续编号的物理块中。图 2.56 给出了连续文件结构。

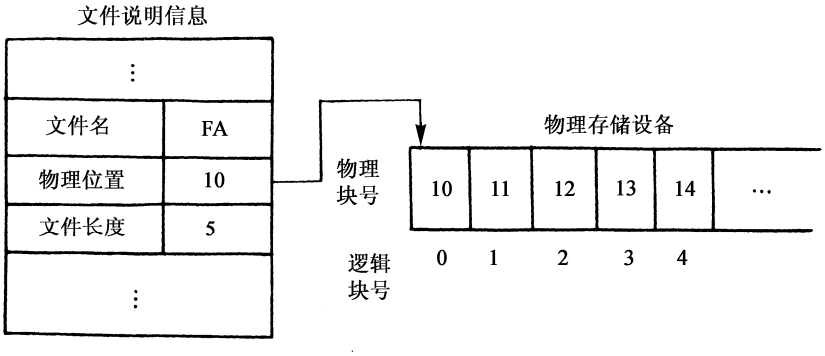


图 2.56　连续文件结构

连续文件的优点是结构简单，且成批顺序存取时速度较快，缺点是建立文件前需要预先确定文件长度才能分配外存空间，而且对文件进行插入、删除、修改等操作较困难。

这种结构特别适宜在顺序存取设备如磁带上组织文件。

2．链接文件

链接文件把一个逻辑上连续的文件分散地存放在不连续的物理块中。在链接文件中，块与块之间通过每个物理块所设置的指针连接起来，只要指明该文件的第一个块号，就可以按链指针检索整个文件。链接文件的结构如图 2.57 所示。

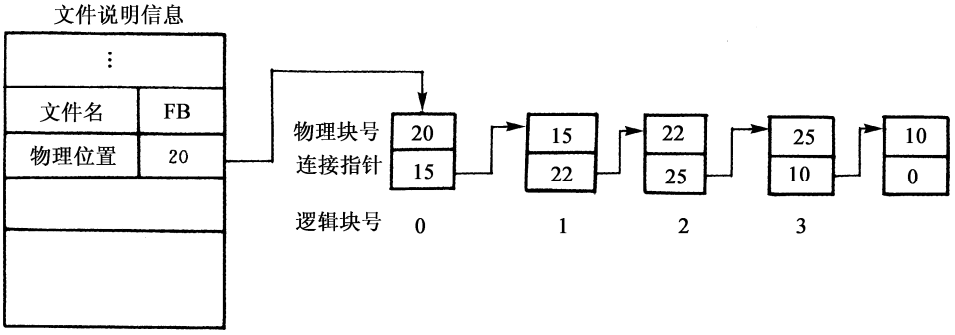


图 2.57　链接文件结构

链接文件克服了连续文件进行插入、删除、修改等操作困难的缺点，允许文件长度动态增长或缩短。由于不必连续分配外存，所以外存空间利用率高。但与连续文件一样，只适于顺序存取。

3．索引文件

为了实现对文件的随机存取及不连续的存储，引入了索引文件。系统为每个索引文件建立一张索引表。索引表的表目指出文件信息所在的逻辑块号和与之对应的物理块号。索引表也以文件形式存放在磁盘上。给出索引表的地址，就可以查找与已知文件逻辑块号相应的物理块号。存放文件的各物理块可不必占用连续的外存空间。索引文件结构如图 2.58 所示。

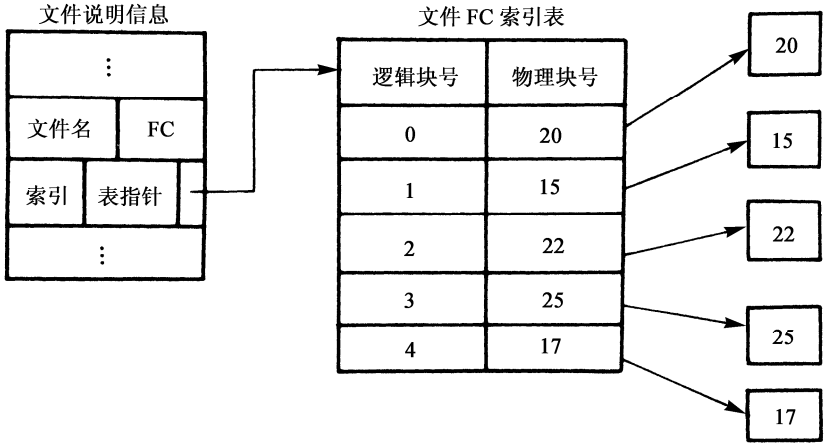


图 2.58　索引文件结构

索引文件除具有链接文件的优点外，更主要的是可以较方便、迅速地实现随机存取。但是索引表要占用额外的外存空间，尤其当文件较大时，索引表也相对较大，不仅占用较大的外存空间还会影响随机访问的速度。可以采用多级索引表来提高查找的速度。

2.6.3　文件目录

计算机系统中存在着许许多多的文件，为了便于对文件的管理和方便、快速地查找文件，建立了文件目录。文件系统的主要功能是“按名存取”，用户只要给出文件的符号名就能够方便地存取存放在外存空间的文件信息，而不必关心文件的具体物理地址。实现文件符号名到文件物理地址的映射的主要环节是检索文件目录。系统为每个文件设置了一个描述性的数据结构——文件控制块 FCB，文件目录就是文件控制块的有序集合。

一、文件控制块

从文件管理的角度看，一个文件包括两部分：文件说明和文件体。文件体指文件本身的信息。文件说明也称为文件控制块 FCB（File Control Block），是用于描述一个文件的文件名、物理位置及其他有关说明信息与控制信息的数据结构。文件控制块 FCB 通常应包含以下内容：

（1）文件名：即文件的符号名。

（2）文件类别：指明文件属于哪一类。

（3）文件的物理位置：指出文件内容在外存的起始块号及所占的块数。

（4）文件长度及逻辑结构：指出是流式文件或记录式文件。对于记录式文件，给出记录个数和每个记录长度；对于流式文件，则长度以字符为单位。

（5）文件的物理结构。

（6）文件存取权限：指出各类用户对文件分别具有读、写、执行等哪类操作权限。

（7）文件建立的日期、时间，最近一次修改文件的日期和时间，最近一次读文件的日期和时间等。

二、文件目录结构

文件控制块的有序集合称为文件目录。因此，文件目录实际上是文件名及其存放地址的对照表，还包括关于文件说明和控制的有关信息。文件目录本身也以文件形式存放，称为目录文件。

文件目录的组织和管理是文件管理中的一个重要方面，一般有一级目录结构、二级目录结构和多级目录结构。

1．一级目录结构

一级目录结构是最简单的目录结构，它是由文件控制块（目录项）构成的线性表（目录表）。每个文件控制块都指向一个非目录文件（见图 2.59）。

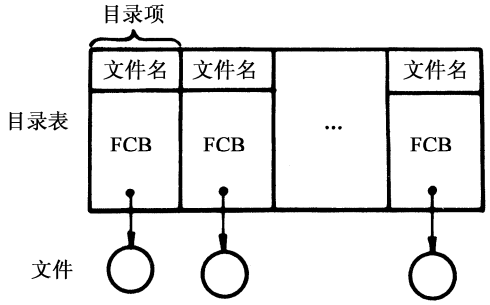


图 2.59　一级目录结构

这种结构的优点是处理简单。创建或删除一个文件，只需在目录表中添加或删去该文件的目录项（FCB）。存取文件时，只需在目录表中顺序查找相应的目录项，并根据目录项中给出的访问权限，判定访问的合法性，如合法，便可从目录项给定的物理位置及本次存取的逻辑块号找到相应的物理块。

但一级目录结构的缺点是限制了用户对文件的命名，不允许文件重名，一旦发生重名文件就会产生冲突和混乱，导致错误结果。而要求各用户文件不同名是很难做到的。同时当文件很多时，查找速度慢。为了克服这些缺点，引入了二级和多级目录结构。

2．二级目录结构

将文件目录分为两级：第一级是主文件目录 MFD（Master File Directory），第二级是用户文件目录 UFD（User File Directory）。如图 2.60 所示，系统的每个用户在主文件目录中都

有一个登记项，记录了用户名和该用户的文件目录的物理地址。每个用户有一个 UFD，并列存放属于该用户的所有 FCB。

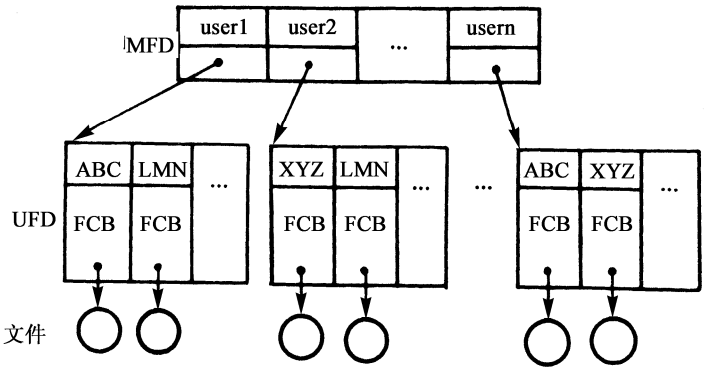


图 2.60　二级目录结构

对文件的访问必须首先从主目录 MFD 开始搜索，按用户名从主目录中找到相应的用户目录 UFD，再按一级目录操作进行搜索，操作简便。由于每个用户有自己的 UFD，既防止了其他用户的非法访问，也允许两个不同用户具有同名文件，系统会把它们区分开来。

与单级目录相比，如果单级目录表的长度为 n，则单级目录的搜索时间与 n 成正比；而在二级目录中，由于将目录项 n 划分为 m 个子集（m 为用户个数），则二级目录的搜索时间与 m+r 成正比，其中 r 为每个用户的文件个数，显然 m+r＜＝n，从而对相同的文件数，二级目录的搜索速度明显高于单级目录。

使用二级目录不仅可以解决文件重名和文件共享问题，还可以获得较高的搜索速度。

3．多级目录结构

为了给用户带来更多的方便，引入多级文件目录结构，它是二级文件目录的层次关系的推广。如图 2.61 所示，多级目录结构又称树型目录结构，叶子节点表示普通文件（用圆圈表示），中间节点表示目录文件（用矩形框表示），分支则代表目录项。从根节点到任一中间节点或叶子节点都有且仅有一条路径，该路径上的全部分支组成一个全路径名。用户使用文件的路径允许访问文件，而系统根据这个路径名，从根到叶子节点一级级进行查找，从而确定文件的位置。

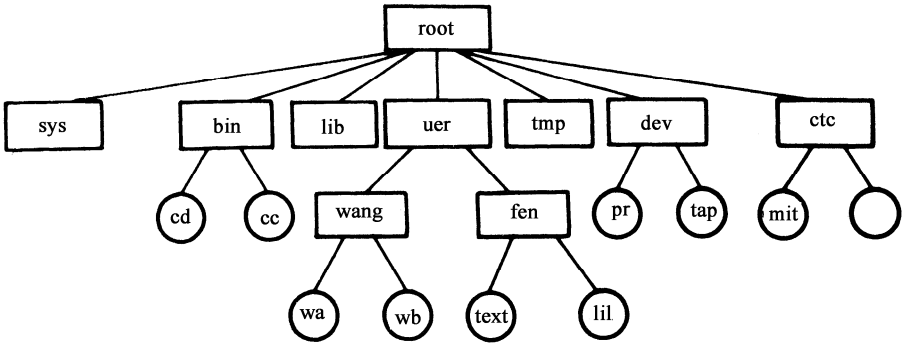


图 2.61　多级目录结构

多级目录结构使得对文件系统的管理和使用十分方便，且可以更确切地反映系统内文件的分支结构，可将部分文件构成子树，与其他部分分开，规定不同层次或子树文件具有不同的存取权限以便于文件的保护和保密。

三、文件目录管理

在一个多层的树形文件目录结构中，如果每次都从根节点开始检索，就很不方便。通常各目录文件往往放在外存，每次存取文件，都要先到外存中去查找目录，影响访问速度。尤其当层次较多时，检索要耗费很多时间。由于任何一个用户在一段时间内仅仅使用众多文件中的少数几个，仅涉及少量目录，因此，引入“工作目录”或称“当前目录”的概念解决上述问题。

具体做法是：当使用文件打开命令打开某个文件时，将该文件涉及的目录调入指定的内存缓冲区，在文件操作期间，该目录一直驻留在内存，直到使用关闭命令关闭该文件为止。这样，每一个用户有一个当前工作目录，访问某个文件时，就从当前目录开始向下顺序检索，路径名只要给出当前目录开始到所要访问文件的那部分路径即可。这样检索路径缩短，检索速度提高。

2.6.4 文件的共享、保护和保密

文件的共享和保护、保密是密切相关的，可以把文件的保护和保密看成文件共享的必然需要，其实质是实施有条件的文件共享。

一、文件的共享

文件的共享是指一个文件可以让多个用户共同使用。文件共享不仅是完成共同任务所必需，而且还可以减少用户的大量重复性劳动、节省文件所占用的存储空间。

在多级目录结构中，连接法是常用的共享技术。对文件共享有两种连接方式：

1．将目录项连接到一文件目录的节点上，即可共享被连接的目录及其各子目录所包含的全部文件。这种结构，通常把所有要共享的文件放在一个目录中，要共享这些文件的用户可以建立自己的子目录，并连接该共享目录。

这种连接方式便于共享，但维护和控制较困难，容易造成环路连接使目录管理混乱。

2．只对单个普通文件连接，即连接到叶子节点上。可通过不同路径来访问同一共享文件。这种方式较可靠，易于管理，所以通常采用这种连接方式。

具体连接的方法一般不采用在目录项中包含共享文件的物理地址的方法（直接指向），因为这样做，当共享文件的物理地址改变时，则无法对其进行访问。

通常采用的具体连接方法是：①将共享文件的物理地址记录到与该文件相关联的文件控制块或文件索引表中，再将目录项中的指针指向该文件控制块或文件索引表。②采用符号连接，即在目录项中存放共享文件的路径名而不是物理地址；并在目录项中增加一个字段 LINK，用于区分是共享文件的路径名还是正常的目录指针（物理地址）。

二、文件的保护和保密

文件的共享与保护保密是矛盾的两个方面。文件系统必须提供安全可靠的保护与保密

措施，以防止由于误操作而对文件造成的破坏，防止未经授权的用户对文件的访问。

文件保护因系统而异，常用三种方法。

1．存取控制表

文件的保护、保密实际上是对用户的存取控制权进行限制。一般为文件的存取设置两级控制：第一级是访问者的识别，即规定哪些人可以进行共享；第二级是存取的识别，即有权参与共享者可对文件执行哪种操作。

例如，为实施一级控制，可将用户分为四类：

S（SYSTEM）：系统管理员或特权用户；

O（OWNER）：文件主人，即文件创建者；

G（GROUP）：文件组的同组用户或合作用户；

W（WORLD）：其他用户。

为实施二级控制，可设置以下几种基本的存取权限：

R（READ）：读文件；

W（WRITE）：写文件，修改文件；

E（EXECUTE）：执行映像文件；

D（DELETE）：删除文件；

N（NOACCESS）：不允许任何操作。

系统为每个文件设置一张存取控制表（见图 2.62），为每类用户规定了存取权限。

存取控制表由文件主提供，系统将其记录在文件的 FCB 中。每当要访问文件时，系统就检查相应的存取控制表，验证存取的合法性。一般文件主或系统用户可以修改存取权。

用户类	存取权
S	RWED
O	RWED
G	RE
W	E

用户名	存取权
fen	RWED
wang	RE
zhang	R
GON	E

图 2.62　存取控制表

2．口令

比较简单的方法是为文件设置口令。口令方式有两种。一种是用户在进入系统时，验证所建立的终端进程的口令（password），若用户输入口令与原设置的口令不一致，则用户不能进入系统。另一种方式，是用户在建立一个文件的同时，为该文件规定一个口令，并将口令保存在文件控制块中。用户在存取文件时必须先提供口令，只有当口令与对应文件目录项中的口令相同时才能进行存取。

但若要收回某用户的使用权，只有修改口令。此外口令也容易被他人窃取，故安全保密性不高。

3．密码

加密是一种保密性较强的方法。在创建文件时对文件进行编码加密，在读文件时对其进行译码解密。只有知道密码的用户才能解密读出文件信息，故有较强的保密作用。

对文件进行加密解密都需要用户提供一个代码键（KEY）。加密程序根据代码键对用户文件进行编码变换，然后写入存储设备。在读文件时应先确认用户输入的代码键才能解密。

密码方式虽然保密性好，但编码解码过程要耗费大量处理时间，增加系统的额外开销。

2.6.5 文件存储空间的管理

存储空间管理是文件系统的重要任务之一。只有对存储空间进行有效管理，才能保证多个用户共享文件存储设备和实现文件按名存取。常用的存储设备有磁盘、光盘、磁带等。这里主要介绍磁盘空间的管理。

由于磁盘空间分为大小相等的若干个物理块，并以块为单位来交换信息，因此对磁盘空间的管理包括对空闲块的组织、分配及回收等几个问题。常用的磁盘空间管理方法有以下几种：

一、位示图

利用一串二进制位的值来反映磁盘空间的分配情况。每一个物理块对应一个二进制位。如果相应的物理块空闲，对应二进制位为 0；如果物理块已分配，对应的二进制位为 1。

位示图描述能力较强，用一个二进制位就可描述一个物理块的状态，因而位示图所占用存储空间较小。可以将其放入内存，既方便查找又提高查找速度。位示图适用于各种物理结构的文件系统。

二、空闲文件目录表

这种管理方法是把所有空闲块的块号统一放在一个称为空闲文件目录的物理块中，即建立一个空闲文件目录表，每个表项记录一个空闲文件的起始块号和大小（以块计）。一个空闲文件（又称空闲区）由若干个连续的空闲块组成。各空闲文件在目录表中排列的顺序取决于所采用的分配算法。常用的分配算法有首次适应算法、最佳适应算法、最坏适应算法等。

当系统建立新文件时检索空闲文件目录表，寻找合适的表项，将空闲块分配给新文件。当用户删除文件时，系统回收它所占用的物理块，必要时进行相邻空闲区的合并，并修改相应表项。

三、空闲块链接法

空闲块链接法是一种常用的空闲块管理方法。将系统中所有空闲物理块链接成一个队列，并将头指针指向第一个空闲块，其余各个空闲块中包含有下一个空闲块的块号。最后一空闲块的指针为 NULL，表示链尾。

当申请者需空闲块时，分配程序从链首摘取所需要的空闲块，并调整首指针；反之，当回收空闲块时，把释放的空闲块逐个插入链尾。

空闲块链接法节省内存，但效率较低。因为在增加或移动空闲块时需对空闲块链作较大的调整，需耗费一定的系统开销。

一种改进的办法是采用空闲块成组链接法，该法是把所有空闲块按固定数目分组，例如 50 块划分为一组。组的划分为从后向前顺序划分，每组的第一块用于存放前一组的块号

和总块数。组与组之间形成链接关系，最后一组的块号（可能不足 50 块）通常放在内存的一个专用栈结构中。空闲块的分配与回收可在内存中完成，且按组进行。UNIX 系统就是采用这种方法。

2.6.6 文件的使用

用户在使用文件系统时，要由具体的操作来实现，这些操作一般由系统调用或广义指令来完成。用户利用系统调用可以完成某项文件操作，具体的系统调用命令因系统而异。下面介绍几种常用的文件系统调用功能：

一、创建文件

创建文件的系统调用一般格式为

create （文件名，参数表）；

用户提供所创建的文件名及若干参数，系统为新文件建立一个文件控制块，并根据所给出的参数填写文件控制块 FCB 中各项。

二、打开文件

用户进程在访问一个文件之前，必需先执行打开文件的操作，建立用户与文件的联系。

打开文件的系统调用的一般格式为

open（文件名，读写方式）；

读写方式可为读、写和既读又写等。打开文件的基本思想是：按指定文件名检索目录结构，将检索到的需打开文件的 FCB 读入并保存在内存中，从而加速对文件的访问速度。

系统具体完成以下步骤：

（1）检查该文件名所对应的文件控制块是否已在内存，若不在，则将其调入内存，该文件控制块 FCB 称为“活跃的”。

（2）检查操作的合法性，如果根据文件保护码为只读，而打开读写方式为写，则不能打开文件。

三、读文件

可以读取已打开文件的信息。读文件的系统调用的一般格式为

read（文件名，键值，内存位置）；

将所读取文件（或文件描述符）中给定键值的几个字节或记录读入内存中指定单元。

四、写文件

写文件操作与读文件相反，系统调用的一般格式为

write（文件名，键值，内存位置）；

表示把内存中指定单元开始若干个字节（字符流文件）的信息或记录写入指定文件名（或文件描述符）的文件中。系统还将为其分配物理块，以便把信息写到外存上。

五、关闭文件

当文件暂时不用时，应将其关闭。若需再次访问时，应将文件重新打开。关闭文件的系统调用的一般格式为

close（文件名）；

系统根据用户所提供的文件名或文件描述符，修改该文件的文件控制块。撤销活动文件表中相应表目，将其文件控制块复制到外存。

小　　结

文件系统是指操作系统中与文件管理有关的软件和数据。文件系统的功能包括：文件存储空间的管理，文件名到文件存储空间的映射，实现文件按名存取，实现对文件的各种操作，支持文件的共享，提供文件的保护与保密措施。

文件目录是文件系统的关键数据结构，用来组织文件卷上的文件以及对文件进行检索。每个目录项是一个文件控制块（FCB），它记录了文件说明和控制信息。广泛使用的是多级目录结构，即目录树，它有利于实现文件共享及加快目录检索的速度。设置当前工作目录可以缩短检索的路径，提高检索的速度。

可从两个角度看待文件组织；从用户角度看文件的逻辑结构，可分成无结构的流式文件和有结构的记录式文件，对文件的存取分为顺序存取和随机存取；从实现的角度看，文件的物理结构可分为连续文件、链接文件和索引文件，后面二者都可非连续存放。

文件存储空间的管理是实现连续空间或非连续空间的分配。常用的管理方法有：位示图、空闲区表及空闲块链接法。

文件的共享与文件的保护和保密是同一个问题的两个方面，保护与保密的实质是有条件地共享。存取控制表方法集保护与保密于一体，口令和密码则是用于保密。

习　　题

1．设置操作系统的主要目的是什么？操作系统有哪些基本功能？

2．操作系统有哪些基本类型？它们各自的主要特点是什么？

3．网络操作系统与基本类型的操作系统有何不同？

4．进程的基本特征是什么？它与程序有何区别与联系？进程有哪几种基本状态？

5．什么是临界资源？什么是临界区？解决进程互斥的基本方法是什么？

6．设有一组计算进程和一组打印进程，它们共享具有 n 个缓冲区的有界缓冲池。每个缓冲区存放一个计算结果，各进程对缓冲池进行互斥访问。利用信号量机制写出进程间的同步算法，并说明各信号量的意义、作用。

7．高级调度与低级调度的主要职能是什么？有何区别与联系?为什么要引入中级调度？

8．有哪几种进程调度的方式？简述常用的调度算法及其特点。

9．何谓死锁？产生死锁的原因和必要条件是什么？如何预防和解除死锁？

10．进程之间有哪几种通信方式？低级通信与高级通信方式的主要区别是什么？

11．作业管理主要包括哪些内容?作业调度的主要功能及算法是什么？

12.何谓用户接口?通常在用户和操作系统之间提供哪几种类型的接口?它们的主要特点及功能是什么？

13．什么是系统调用？系统调用有哪些主要功能？

14．在多道程序系统中，一个作业从提交给系统到运行结束退出系统，要经历哪些阶段和状态?说明各状态转变的原因及过程。

15．一个通用的操作系统，对用户作业提供哪几种控制方式？

16．解释下列术语：地址空间、存储空间、逻辑地址、物理地址、地址变换。

17．什么是重定位？静态重定位和动态重定位有何区别?试比较它们的优缺点。

18．为什么要引入虚拟存储技术？虚拟存储技术的基本思想有何特点？

19．固定分区分配有何缺点？可变分区中采用何种方法减少存储碎片？简述分区管理中常用的内存分配策略及其特点。

20．什么是请求式分页存储管理？说明页式动态地址变换机构，并画出地址变换过程的流程图。

21．请求式分页管理中有哪几种常用的页面置换算法？试比较它们的优缺点。

22．什么是快表?快表在虚拟存储系统中起到什么作用？

23．为什么要引入段式存储管理？它与页式管理有什么本质上的区别？段式管理如何实现段的共享和存取控制?

24．什么是静态链接？什么是动态链接?在段式管理中如何实现动态链接？

25．为什么要引入段页式存储管理？请说明在段页式系统中动态地址变换过程。

26．何谓系统的“抖动”现象?如何防止“抖动”现象的产生？

27．I/O 设备管理的主要任务和功能是什么？

28．数据传送有哪几种控制方式?比较它们的优缺点。

29．什么是 SPOOLing 技术？它的作用是什么？举例说明 SPOOLing 技术的应用。

30．为什么要引入缓冲技术？缓冲器的作用是什么？

31．什么是文件系统？文件系统的基本功能是什么？

32．什么是文件目录？与一级目录相比，二级目录、多级目录有什么特点？使用当前目录的作用是什么？

33．什么是文件的逻辑结构？有哪两种基本的文件存取方式？

34．什么是文件的物理结构？常用的存储空间管理方式有哪几种？各有何特点？

35．页式系统中地址结构长度为 16 位，页面大小为 2KB，作业地址空间为 6KB，该作业的各页依次存放在 2，3，6 号物理块中，相对地址 2500 处有一条指令 Store 1，4500，请给出该作业的页表、该指令的物理单元和数据存放的物理单元。

36.　在一个请求页式存储系统中，一个程序的页面走向为 4，3，2，1，4，3，5，4，3，2，1，5，并采用 LRU 页面置换算法。假设分配给该程序的存储块数 M 分别为 3 和 4 时，求出在访问过程中发生的缺页次数和缺页率，比较所得结果。

37．一个请求分页存储管理系统中，可以使用大小为 512B 的页，也可以使用大小为 2048B 的页，试分析分别选择使用 512B 的页面和选择使用 2048B 页面的主要优缺点。

第 3 章　软件工程方法

随着计算机应用领域的扩展，对计算机软件的需求越来越大。为了保证软件开发的质量和满足软件市场的需求，软件也应当像其他产业的产品一样，以系统的、工程的方法进行开发制作，并提供售后服务。本章首先简要介绍软件工程的相关概念；然后，重点介绍结构化的软件开发方法并简要介绍了面向对象的方法。

3.1　软件工程概述

3.1.1　软件工程学的形成与发展

软件工程的提出源于20世纪60年代末期出现的“软件危机”，并在较短的时间内发展成为一个完整的学科方向，30多年来，科研人员已在理论研究和工程实践两个方面做了大量的工作。各种方法、技术的研究取得了长足的发展，其中有些已比较成熟并广泛应用于软件开发之中。软件的工程化生产已形成一个独立的产业部门，成为软件产业。软件已成为计算机工业最重要的产品之一。

自从1945年第一台电子数字计算机诞生以来，软件开发方法从机器码编程到软件工程方法，经历了三个阶段：

1．程序设计时期（1946年到20世纪60年代中期）

在这个时期将软件看作是按具体问题编制的专用程序。程序设计凭个人的经验和编程技术单独进行，使用的工具是机器语言、汇编语言、服务性程序；程序的运行和维护等工作也全部由程序设计者承担。生产方式完全是手工生产、个体劳动。这个时期只有程序的概念，没有软件的概念。

2．软件时期（20世纪60年代中期至70年代中期）

在这一时期，随着计算机技术的发展，计算机硬件已进入晶体管、小规模集成电路时代，计算机内存容量增大、运算速度加快、外部设备配置增加，因而急需大量的编程人员。由于程序规模增大，个体生产方式已经不能适应生产要求，需要多人分工，共同协作来编制一个程序，即采用了所谓的“作坊式”生产方式。同时，人们已开始认识到程序可以使计算机发挥巨大的灵活性，程序不再是硬件的附属。这时已提出“软件”的概念，并且在软件开发中，开始强调软件工程师的作用，强调开发者的相互通信和协作，软件技术取得一定的进展。随着计算机应用领域的不断扩大，软件的规模及结构的复杂程度也不断增加，作坊式的软件生产方式已难以满足软件生产的质量和数量上的要求，从而出现了所谓的“软件危机”。

软件危机主要有以下一些表现：

（1）对软件开发成本和进度的估计常常很不准确，有时竟然出现实际费用超过预算一

个数量级以上、进度推迟几个月甚至几年的情况。

（2）用户往往对已完成的软件不满意。

（3）软件的质量常被怀疑。

（4）现有的软件极难维护。要在许多程序中纠正潜在的错误是很困难的，要使已有的软件适用于新的计算机或者增加用户新的需求，实际上也难以做到。

（5）缺乏良好的软件文档。软件文档往往与当前的软件产品不符，不能保证在软件开发过程中有合格的文档作为软件开发者相互通信的工具，不完整的文档给软件维护带来许多困难。

（6）软件开发生产率提高的速度远远跟不上计算机应用迅速普及深入的趋势。软件生产的低效率使人类不能充分利用现代计算机硬件提供的巨大潜力。

例如，IBM公司于20世纪60年代对OS/360操作系统的开发过程中遭受到的挫折是一个典型的例子。OS/360由4000个模块组成，共约有100万条指令，工作量是5000个人年（一人年表示一个人工作一年的工作量），开发费用达数亿美元，但人们在使用中从操作系统的程序中发现了2000个以上的错误。使该系统的开发未达到预期的效果。在经济上蒙受重大损失。

软件危机产生的根本原因是什么？它与软件产品的特征和软件开发与维护的方法不正确有关。

其一，软件是逻辑的系统部件而不是物理的系统部件，它以程序和文档形式存在，具有无形性。因此不应根据多个实体部件的质量来衡量软件的成败，而应将软件作为一个完整的产品来衡量。软件在运行过程中不会因为使用时间过长而被“用坏”，运行中如果发现错误往往是开发时期引入而在测试阶段未能发现的故障，在软件维护中要换掉有毛病的部件，不能采用简单的更换备份的方法，而只能修改原来的设计。同时，软件开发过程是一种智力活动，其开发过程的进展情况较难衡量，软件开发的质量也较难评价，因此，管理和控制软件开发过程相当困难。

其二，软件规模越来越大，功能越来越强，导致软件结构非常复杂。例如，以美国宇航局的软件系统为例，1963年的水星计划有二百多万条指令，1967年的双子星座计划达到四百万条指令，1973年阿波罗计划有一千多万条指令，1979年的哥伦比亚航天飞机达到四千多万条指令。在大型软件开发中，如何保证每个人完成的工作合在一起确实能构成一个高质量的大型软件系统是一个极端复杂的问题，它既涉及技术和方法问题，也涉及管理问题。早期软件开发的个体化特点导致了对软件开发和维护没有统一的标准可以遵循，由于没有注意软件开发方法和技术的研究，使软件技术的发展跟不上解决软件复杂性的要求，这些都使得软件开发的管理不善或技术效果不好。

如何解决软件危机呢？

认识软件开发中的问题和造成这些问题的原因只是解决软件危机的开始。更重要的是要充分吸取和借鉴人类长期以来从事各种工程项目所积累的行之有效的原理、概念、技术和方法，并应用于软件开发的实践中，将软件开发变成一种组织良好、管理严密、各类人员协同配合，共同完成的工程项目。

为了实现上述目标，不仅要有协调、控制、管理软件开发的理论，还应该有支持软件开发各个阶段的技术和工具。如果有标准接口完成标准功能的构件块大量存在，也将会大大促进计算机软件的设计；如果能从软件需求规格说明直接生成程序而实现自动程序生成，则显然会大大提高软件的生产效率；如果能出现真正的程序正确性证明器，软件危机就将消失。

由此可见，解决软件危机的根本出路在于将软件开发方法学（包括管理、控制、复审等方法）与支持软件开发各个队段的技术和各种支持工具结合起来。

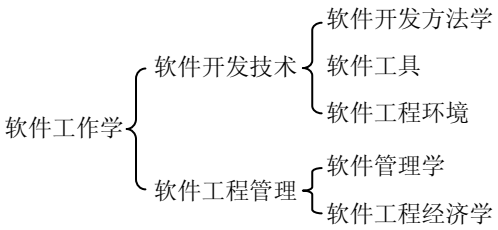
3．软件工程时期（20世纪70年代至今）

针对20世纪60年代后期使用传统的软件开发方法已不能适应大型软件的生产，难以满足软件生产的质量和数量上的要求而出现的“软件危机”，1968年、1969年北大西洋公约组织成员国的软件工作者召开了两个研讨会，讨论如何摆脱这一困境，同时，提出了“软件工程”这一术语，其根本目的在于克服“软件危机”中所遇到的困难问题，从此软件生产进入软件工程时代。

3.1.2 软件工程及软件工程学

“软件工程”是软件人员谋求解决出现的软件危机而产生的。人们曾从不同的角度给软件工程下过一些定义。例如，Boehm定义为：“运用现代科学技术知识来设计并构造计算机程序及为开发、运行和维护这些程序所必需的相关文件资料”；1983年IEEE定义为：“软件工程是开发、运行、维护和修复软件的系统方法”，等等。但其核心思想都是“采用工程化的原理与方法对软件进行计划、开发和维护”，以按预期的进度和经费完成软件生产计划，提高软件的生产率和可靠性。

自“软件工程”出现以后，人们围绕着实现软件优质高产的目标进行了大量的理论研究和实践上的努力，逐渐形成了“软件工程学”这一新兴学科，它所包括的重要内容如下：



软件开发方法学包括了软件开发“如何做”的技术，其任务是对软件开发步骤和文档格式提出了规范化的要求，使软件生产摆脱了个人生产方式，进入了有章可循，按结构化和标准化方式进行“工程化”的开发。

各种不同的软件工具对提高软件的生产率，促进软件生产的自动化，减轻软件生产者劳动强度起到了重要的作用。不同的软件工具可以支持不同的具体的软件生产任务。

工具和方法总是密切相关的，当一种方法提出并证明有效后，往往随之研制出相应的工具，并将新方法融于工具之中，从而为用户提供更多的集成开发工具。方法与工具相结合，再加上配套的软、硬件支持，就形成了软件工程环境。一个理想的软件工程环境，应

该同时具备支持开发和支持管理两个方面的工具。

软件工程管理的目的是为了按照进度及预算完成软件计划，实现预期的经济和社会效益。它包括成本估算、进度安排、人员组织、质量保证等多方面的内容，涉及管理学、经济学等多项学科。

小　结

软件工程是从工程角度来研究软件开发的方法和技术，它是在克服软件危机的过程中产生而发展起来的。

软件工程学是自软件工程出现以后形成的一门新兴学科。它包括的重要内容有：软件开发技术——软件开发方法学、软件工具、软件工程环境；软件工程管理——软件管理学、软件工程经济学。

采用工程化的方法和技术开发软件，可以提高软件的可靠性、可维护性、可移植性，其目标是在给定的时间和费用的条件下开发出一个满足用户功能和性能要求的可靠的软件。

3.2　软件与软件生存周期

3.2.1　软件

从狭隘的观点理解软件这个概念，通常认为软件即程序。这样简单的理解不仅不能表达软件的确切含义，而且会将软件开发技术与程序设计方法等同起来，从而忽略了软件开发技术中许多重要的其他方面。

随着克服软件危机而产生的各种软件开发方法、技术及其相对应的软件工作模型，人们对软件这个概念进行了各种新的定义，其中较全面的定义为：“软件是程序以及开发、使用和维护程序所需的各种文档”。对于一个计算机系统来说，软件是计算机系统中与硬件相互依存的另一部分。通常，软件部分的完整配置应包括四个主要部分：

（1）应用程序　直接面向用户，为解决各种特定问题而编写的程序，如实时控制、工程研究、科学计算、信息管理、人工智能等。

（2）系统程序　为应用程序服务所编制的程序汇总，面向计算机硬件，是应用程序的支撑部分。

（3）面向用户的文档　如何使用和维护应用程序的资料，如用户手册、操作手册以及维护手册等。

（4）面向开发者的文档　记载了应用程序的设计和开发过程，以便进一步开发和修改。如：可行性研究报告、项目开发计划、需求规格说明书、设计说明书、程序设计说明、测试报告等。

3.2.2　软件生存周期

软件生存周期的概念是从工业产品生存周期概念借用过来的。一种产品从订货开始，

经过设计、制造、调试、使用维护，直到该产品被淘汰不生产为止，这就是所谓产品生存周期。

一个软件从用户提出开发要求，到软件在使用中消退的全过程，称为软件的生存周期。软件工程采用的生存周期方法就是从时间角度对软件开发和维护的复杂问题进行分解，将软件的生存周期划分为若干个阶段，每个阶段有相对独立的任务，便于不同人员分工协作，使软件开发过程按有秩序、能管理的方式组织起来，从而降低了整个软件开发的难度。

软件生存周期一般可分为三个阶段：制订计划、开发和运行维护，其中开发可进一步分为需求分析、设计、编码和测试。

计划时期的主要任务是分析用户需求，分析软件系统所追求的目标，分析开发该系统的可行性。这一时期需要用户与系统分析员的交互和配合。

• 问题定义　收集、分析、理解、确定用户的要求，系统分析员在与用户讨论的基础上共同提出“软件系统目标与范围的说明书”。

• 可行性研究　确立对问题定义阶段确定的问题是否有可行的解决办法，并对各种可能方案做出成本/效益分析，系统分析员据此提出“可行性论证报告”，作为使用部门负责人是否继续进行这项工程的依据。

开发时期包括设计和实现两个任务，其中设计包括需求分析和设计两个阶段，实现包括编程和测试两个阶段。

• 需求分析　确定用户对软件系统的全部需求，并以“需求说明书”的形式表达，其目的是明确软件系统“做什么”。

• 设计　设计软件系统的模块层次结构，设计数据库的结构，设计模块的控制流程，其目的是明确软件系统“如何做”。这个阶段通常可分为两个步骤：概要设计和详细设计。概要设计完成软件系统的模块的划分和模块的层次结构以及数据库设计；详细设计完成每个模块的控制流程的设计。这个阶段结束应交付“设计说明书”。

• 编程　按照选定的程序设计语言为设计说明书中每个模块的控制流程编出相应的程序，得到软件系统的源程序。

• 测试　检查并排除软件中的错误，提高软件的可靠性。测试分为如下三个步骤：模块测试，测试程序的每个模块是否有错误；组装测试，测试模块之间的接口是否正确；确认测试，测试整个软件系统是否满足用户功能性要求。本阶段结束应交付“测试报告”，说明测试的对象，测试数据的选择，测试结果是否符合预期结果。如果测试发现问题，经过调试，找出出错原因，然后进行改正。

运行与维护时期的主要任务是软件维护。其任务是修改软件系统在使用过程中发现的隐含错误，扩充在使用过程中用户提出的新功能要求，其目的是维持软件系统的正常运行。本阶段的文档是“软件维护报告”。

3.2.3　软件开发过程模型

如前所述，软件开发方法学和软件工具都是软件工程的重要内容。方法学是指怎么定义和开发软件，而工具是指对方法提供自动或半自动的支持。按照软件工程的观点，软件

开发既要采用适当的方法和工具，还应遵循某种合理的“过程”。在不同的阶段宜使用什么方法与工具？需要完成哪些文档？怎样才能在所有阶段，包括软件更改后保证产品的质量？这些问题都应该在“过程”中一一规定。所谓软件开发模型，实际上就是方法、工具和过程三者的有机结合。软件过程描述了软件开发的一组具有时间约束的活动，通常包括需求分析、设计、编码、测试等活动，软件过程模型则以模型化的手段来定义开发的框架。

最早的软件开发模型是瀑布模型。它奠定了20世纪80年代初软件工程学的基础。以后，人们又在实践中创建了快速原型和智能模型等。开发者可按照软件的应用领域、规模和复杂性，自由选择所需要的开发模型。

一、瀑布模型

瀑布模型指出软件开发包括制订计划、需求分析、设计、编码、测试和运行及维护。这些活动自上而下，相互间有固定的次序，如同瀑布流水的逐级下落，如图3.1所示。

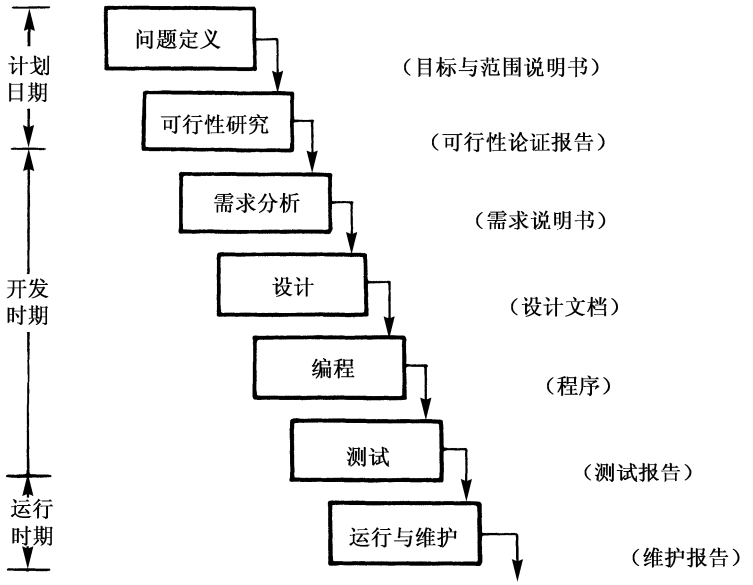


图3.1　瀑布模型

实际上，上述各项活动之间并非具有完全的线性关系，每项活动都具有如下特征：

（1）从上一项活动（如果有的话）接受该项活动的工作对象，作为输入；

（2）利用这一输入实施该项活动应完成的内容；

（3）提交该项活动的工作成果，作为输出传给下一项活动；

（4）对该项活动的工作进行评审，如果得到确认则进行下一个活动，否则返回前项活动，甚至更前项活动。

需要说明的是，软件生存周期虽然按瀑布模型可划分为若干阶段，事实上各阶段不可能理想化地进行分割，在实际开发过程中充满了回溯、反复和交叉过程。例如，在设计阶段发现需求说明书有不完整或不正确之处，就必须进行“再分析”，测试阶段发现模块界

面有错误，就必须进行“再设计”，在运行阶段为了扩充系统的功能又必须进行“再分析”、“再设计”、“再编程”等。

二、快速原型

按瀑布模型开发软件，只有当系统分析员作出准确的需求分析时，才能够得到预期的结果。但由于系统分析员和用户在专业上的差异，在计划时期定义的用户需求，常常不完全和不准确，快速原型方法正是为解决上述问题而提出的。其方法是：首先建立一个能反映用户主要需求的原型，使用户通过使用这个原型来提出对原型的修改意见，然后根据用户意见对原型进行改进，如此反复多次，最后建立起符合用户要求的新系统。这种原型相当于工业产品的样机。它的特点不仅是快速，而且用户与分析员之间的交互从抽象变为具体，避免了许多由于理解的不同而造成的需求分析的错误。图3.2是快速原型模型的工作流程图。

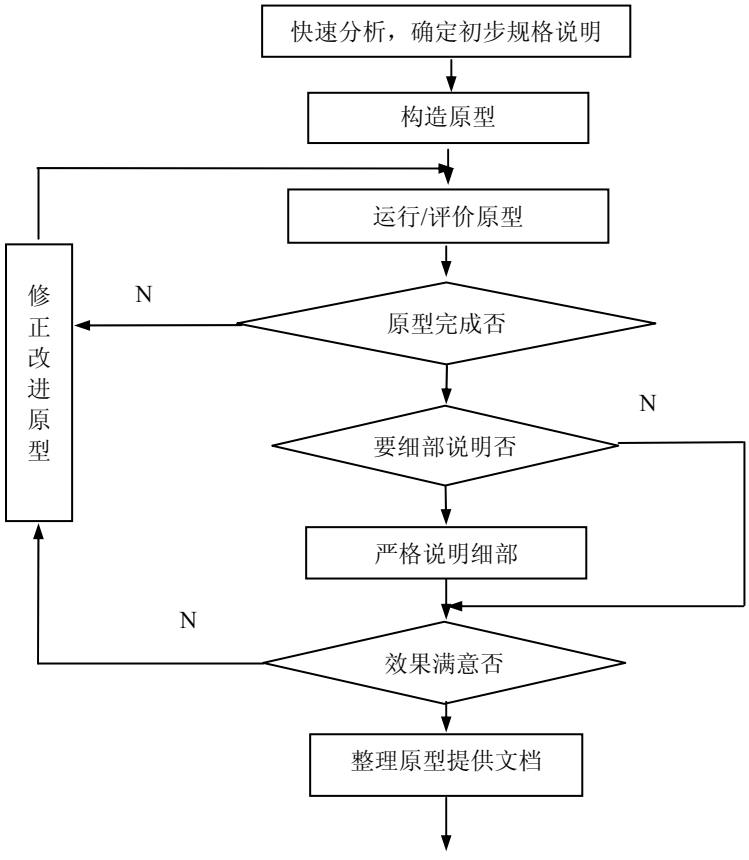


图3.2　快速原型模型的工作流程图

在实现原型系统时，为了减少原型系统的成本，原型系统可以只包括新系统的主要功能和系统的主要界面，推迟考虑系统的性能需求。

使用该方法时，是否能够快速开发出原型和原型系统，是否可以通过补充与修改而获得最终的实际系统是使用快速原型法能否成功的关键问题。

三、智能模型

智能模型也称为基于知识的软件开发模型，是知识工程与软件工程相结合的软件开发模型。其主要特点是必须建立知识库，并将模型本身、软件工程知识、特定领域知识放入知识库。具体描述可以使用形式功能规约，也可以使用知识处理语言描述等。其模型如图3.3所示。

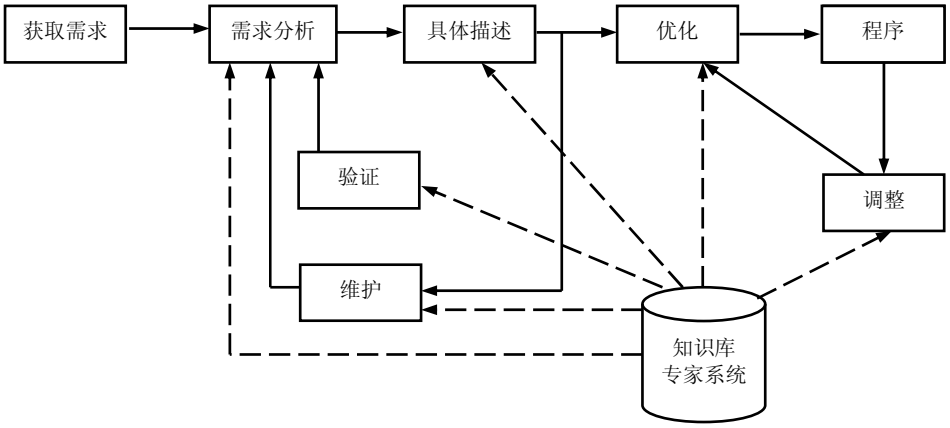


图3.3　智能模型

从软件开发模型的讨论中，不难看出，在“定义——开发——维护”三个阶段的活动中，开发活动总是处于核心的地位。软件开发的目的是在规定的投资和时间限制内，实现符合用户需求的高质量软件，必须用软件工程的方法和技术来指导软件开发的全过程。当前软件开发的主要方法有：结构化开发方法、面向对象的开发方法和基于构件的开发方法。本章重点介绍常用的结构化开发方法，然后简要介绍面向对象的开发方法。

小　　结

软件不完全等同于程序，对软件的较全面的定义为：“软件是程序以及开发、使用和维护程序所需的各种文档”。通常，软件的完整配置应包括：应用程序、系统程序、面向用户的文档、面向开发者文档四个部分。

一个软件从用户提出开发要求，到废弃不用为止的全过程，称为软件的生存周期。

软件工程采用的生存周期方法学就是从时间角度对软件开发和维护的复杂问题进行分解，将软件的生存周期划分为若干阶段（如：需求定义、软件设计、编程、测试、运行维护等），每个阶段有相对独立的任务，便于分工协作，使软件开发过程按有秩序、能管理的方式组织起来，从而降低软件开发的难度。

对软件生存周期的不同划分，形成了不同的软件生存周期模型，常用的模型有：传统的瀑布模型、快速原型模型和智能模型等。

3.3　软件的需求分析

3.3.1　需求分析概述

软件的需求分析是软件生存周期中非常重要的一个阶段。对于大型软件而言，在进行需求分析前，首先要进行问题定义和可行性研究。开发人员在对用户的要求及现实环境进行调查了解的基础上，从技术、经济和社会因素等方面进行研究，探索若干种可供选择的解决方案，并对每种方案进行可行性论证，论证的主要内容有：

- 技术可行性　使用现有的技术能实现这个系统吗？
- 经济可行性　系统的经济效益能超过其开发成本吗？
- 操作可行性　系统的操作方式在这个用户内可行吗？

开发人员在对种种可能方案做出成本/效益分析后，经过对可行性研究报告的评审和审批，决定项目是否可以开发，对确定要开发的项目制定出初步的项目开发计划。

在进行了可行性分析的基础上，需求分析的任务是：用户和软件人员双方进一步理解用户的需求，并将双方的共同理解表达成双方都能接受的“需求说明书”。需求分析是在软件生存周期各阶段中唯一面向用户“问题”的阶段，它主要对用户的业务活动进行分析，明确在用户的业务环境中，软件系统应该“做什么”。所谓用户需求是指用户要求软件系统必须满足的所有功能、性能和限制。用户需求通常包括功能要求、性能要求、可靠性要求、安全保密要求以及开发费用、开发周期、可使用的资源等方面的限制，其中功能要求是最基本的，它又包括数据要求和加工要求两方面。

需求分析是用户和软件人员双方讨论协商的阶段。一方面由于用户往往不熟悉计算机技术，难以用计算机术语表达他们的要求，同时，对于一个大型而复杂的软件系统，用户往往也很难精确完整地提出其功能及性能要求，一开始只能说一个大概或者一些模糊的功能；另一方面开发人员往往又不熟悉用户的具体业务，与用户缺乏共同语言，增加了交流的难度。所以这个阶段的工作应由能在需求分析中起到组织和协调作用的系统分析员主持。系统分析员应具有以下能力：熟悉计算机技术；了解用户业务领域的相关知识；能在用户和软件人员之间借助于数据处理的概念进行交流。同时从个体素质上，系统分析员应善于领会一些抽象的概念，将其重新整理使之成为各种逻辑成分，并根据各种逻辑成分综合出问题的解决办法；善于从相互冲突的原始材料中汲取适当的论据，具备良好的用书面或口头形式进行讨论及交换意见的能力。

参加需求分析的用户人员通常应有三个层次，即企业负责人，各部门负责人，具体工作人员。需求分析时，首先要召开调查会与三个层次的用户人员讨论，了解收集业务过程和业务环境，然后收集与各业务有关的资料、材料、报告、报表、文件、记录等文字或图表材料，必要时应到现场去参观了解。这种调查研究应反复进行几次，直到用户的功能要求都已搞清楚为止。然后对用户的要求进行分析、理解，明确该项目有些什么输入信息，它们之间的关系和结构是怎样的，对这些信息要进行什么样的处理和加工，最后应该得到

什么信息、什么样的统计报表等，最后确定用户的功能要求，用文档形式把它们表达出来，这就是编写需求说明书。

需求说明书主要有三个作用：作为用户和软件开发人员之间的合同，为双方相互了解提供基础，这是双方对开发的软件系统共同理解的结果；作为开发人员进行设计和编程的根据；作为软件开发完成后验收的依据。

编写需求说明书时，应该完整、一致、精确、无二义性，同时又要简明、易懂、易修改。作为软件设计和编程基础的需求说明书，只有精确、无二义性，才能不致被人误解。它越精确，以后出现错误、混淆、反复的可能性就越小。如“系统能处理所有信息，能进行各种组合的查询统计”，又如“系统查询等待时间很短”等词语，是含糊不清的描述，验收时无法检查，而“查询等待时间不超过5秒”就是精确的描述，验收时就可检查是否达到这个要求。

需求说明书最终要得到开发者和用户双方的认可，所以用户要能看得懂，并且还能发现和指出其中的错误，这对于保证软件系统的质量有很大的作用。这就要求说明书尽量少用或不用计算机领域的概念和术语。

需求说明书的内容应包括如下部分：

1．概述　说明开发本软件的目的、意义和背景。

2．需求说明

（1）功能说明，逐项列出各种功能需求的序号、名称和简要说明。

（2）性能说明，说明处理速度、响应时间、精度等。

3．数据描述

（1）数据流图。

（2）数据字典。

（3）接口说明。

4．运行环境

（1）设备，说明软件运行所需的硬件设备。

（2）支持软件，说明软件运行所需的系统软件和软件工具。

5．限制

说明软件开发的成本、进度、设计和实现方面的限制。

必须指出，需求分析对整个开发阶段都具有重大的影响，它是软件开发的基础，一旦需求分析出现错误，将导致整个软件开发的失败，就如同高楼大厦的地基不稳，将导致大厦的倒塌。如果在需求分析中产生一个错误，这个错误发现越晚，则花的代价越高，甚至给整个软件开发造成重大损失。

3.3.2　结构化分析方法

需求分析最常用的方法是结构化分析方法。结构化分析方法（STRUCTURED ANALYSIS）简称为SA方法，它适用于分析大型数据处理系统，它与设计阶段的结构化设计方法（STRUCTURED DESIGNE，简称为SD）一起联合使用，能够较好地实现一个软件

系统的研制。SA方法通常用数据流图表达需求，并以数据词典的形式记录数据的逻辑定义。其方法简单实用，易于理解，使用广泛。

软件工程中控制复杂性的基本手段是“分解”和“抽象”。对于一个复杂的系统，为了将复杂性降低到可以掌握的程度，可以把大问题分割成若干个小问题，然后分别解决，这就是所谓的“分解”。分解也可以分层进行，即先考虑问题最本质的属性，暂把细节略去，以后再逐层添加细节，直至涉及最详细的内容，这种用最本质的属性表示一个子系统的方法就是“抽象”。SA方法也是采用了这两个基本手段。为了理解和表达一个系统的功能，SA方法使用了自顶向下逐层分解的方式，如图3.4所示。假设图中x是一个复杂的系统，为了理解它，可以把它分解为子系统1，子系统2，子系统3。若子系统1和2还很复杂，可以将它们再细分为1.1、1.2、1.3、1.4、2.1、2.2……子系统，一直到各部分都足够简单、容易理解、能清楚地表达为止。在合理的逐层分解之后，我们就可以为每个细部（如图中的1.1、1.2等）写出说明，再将所有这些“小说明”组织起来，就获得了整个系统x的系统说明书。

可见按照逐层分解的方式，无论一个系统有多复杂，分析工作都可以有计划、有步骤、有条不紊地进行，系统规模再大，分析工作的复杂程度不会随之增大，而只是多分解几层，所以SA方法有效地控制了复杂性。

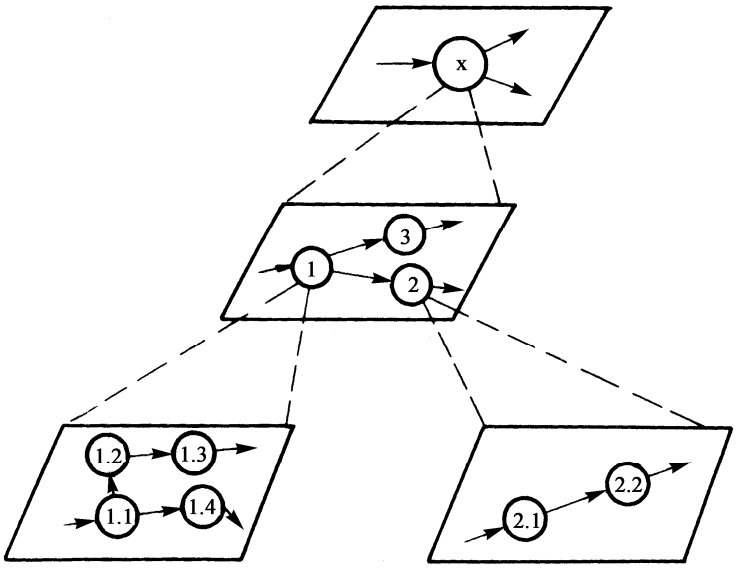


图3.4　逐层分解

SA方法在表达问题时尽可能用图形的方法，因为图形比较形象、直观、容易理解。用SA方法来描述软件将要处理的信息时，使用数据流图和数据词典的描述工具。数据流图表示了软件的信息流向和信息的加工，而数据词典对这些信息和加工进行更详细的描述。还可以使用结构英语、判定表、判定树等其他方法对信息加工的加工逻辑进行描述。

使用SA方法进行软件需求分析时，可按如下步骤进行：

1．建立当前系统的“具体模型”。当前系统可能是需要改进的某个已在计算机上运行的数据处理系统，也可能是一个人工处理过程。在这一步首先分析、理解当前系统和现实

环境，描述当前系统是怎样进行工作的，了解当前系统的机构、输入输出、资源利用情况和自己对当前系统的理解。这一模型应客观地反映现实世界的实际情况。

2．抽象出当前系统的逻辑模型。在理解当前系统“怎样做”的基础上，抽取其“做什么”的本质，从而从当前系统的具体模型抽象出当前系统的逻辑模型。

在具体模型中有许多物理因素，随着分析工作的深入，有些非本质的物理因素就成为不需要考虑的因素，因而需要对具体模型进行分析，区分哪些是本质的、哪些是非本质的因素，去掉那些非本质的因素即可获得反映系统本质的逻辑模型。

3．建立目标系统的逻辑模型。这是SA方法实质性的一步，是SA的最终目标。所谓目标系统是指将要开发的由计算机处理的数据处理系统，它是在分析当前系统逻辑模型与目标系统逻辑模型的差别基础上建立起来的。其方法如下：

首先在数据流图上把目标系统与当前系统在逻辑上的不相同部分找出来，这些部分就是要改变的部分，从而可将当前系统分为两部分：要改变部分和不改变部分。然后将要改变部分抽象为一个加工，再对要改变部分重新进行分解，就可获得目标系统的逻辑模型。

4．为了对目标系统做完整的描述，还需要考虑人-机界面和其他一些问题

一个计算机系统最终也是某个企业的一个组成部分，所以设计新系统时不但要考虑新系统本身，还要考虑它所处的环境及它与外界环境的相互联系，即软件系统的人-机界面。考虑的细节问题还应包括：出错处理，即在各种出错情况下系统将如何处理；系统的启动和结束，即说明系统如何进入工作状态和如何结束工作；新系统的输入输出格式；其他性能方面的要求等。

3.3.3 数据流图

SA方法采用“分解”的方式来描述一个复杂的系统，数据流图是描述系统中数据流程的图形工具，它标识了一个系统的逻辑输入和逻辑输出以及把逻辑输入转换为逻辑输出所需要的加工处理。图3.5是表示银行取款过程的数据流图的例子。

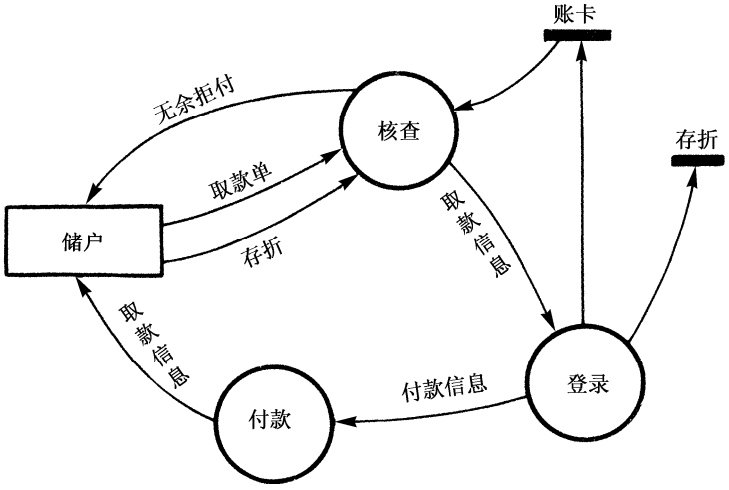


图3.5　取款过程的数据流图

数据流图以图形的方式表达系统中的信息变换和传递的过程，数据流图的基本符号只有下列四种（如图3.6所示）：



图3.6　数据流图的基本成分

（1）数据流　数据流是具有名字且具有流向的数据，用标有名字的箭头表示，例如取款单、取款信息、付款信息等。每条数据流表示在这条流水线上有一组由一定成分组成的数据流动。如取款单，它由取款人姓名、存折号、取款时间、取款金额等组成。

（2）加工　表示对数据进行的加工或变换。数据流图中以标有名字的圆圈代表加工，以名字表示加工的含义，指向加工的数据流是该加工的输入数据，离开加工的数据流是该加工的输出数据。在图3.5中，核查、登录和付款都是加工。

（3）数据存储　文件是数据存储形式的统称。它可以是程序设计语言中的文件，也可以是数据库系统中的数据库，或其他某种能起数据保存作用的数据结构。在数据流图中，它表示了数据流在加工过程中需要存储或查找的信息。文件以标有名字的粗线表示，如图中的账卡、存折，对文件的存和取分别以指向文件的箭头及离开文件的箭头表示，由于文件的内容能顾名思义，因而对文件的存取数据流不一定要用名字标记。

（4）数据源及数据终点　标明数据处理过程的数据来源和数据去向。数据流图中以有名的方框表示，它只起到注释作用，补充说明系统与其他外界环境的联系。如图中的储户。

下面用取款数据流图说明取款的实际工作过程。该数据流图的数据源为“储户”，它产生数据流“取款单”和“存折”并流向“核查”加工，核查加工把输入的这两个数据流与从“账卡”文件中读出的数据流进行加工后，决定是否可以取款，如不能取款，则产生“无余拒付”的输出数据流通知储户；否则产生“取款信息”的输出数据流，并流向“登录”加工，经加工处理，产生两个输出数据流分别修改“存折”和“账卡”文件，同时产生输出数据流“付款通知”，并流向“付款”加工，经加工处理，向数据终点的“储户”付款。

一个复杂的软件往往有几十个甚至更多的加工过程，不可能用一个数据流图来表示所有这些处理过程，按SA方法，这时要采用“先全局后局部，先整体后细节，先抽象后具体”的逐步求精的原则，先画出总体的数据流图，在总体的数据流图上局部的数据流图可能只抽象为一个加工，然后将总体的数据流图逐步分解，最终对每一个具体的数据处理过程都由一个局部的数据流图表示出来。通常可以把这种分层的数据流图，用顶层、中间层、底层来划分。

（1）顶层　是一个高度抽象的软件系统的逻辑模型，它决定系统的范围，决定输入输出数据流，它说明系统的边界，把整个系统的功能抽象为一个加工，顶层数据流图只有一张。

（2）中间层　顶层之下是若干中间层，某一中间层既是它上一层的加工的分解结果，又是它下一层若干加工的抽象，即它又可进一步分解。

（3）底层　若一张数据流图的加工不能进一步分解，这张数据流图就是底层的数据流图了。故底层数据流图的加工是由基本加工构成的，所谓基本加工是指不能再进行分解的加工。

在画各层数据流图时应注意以下问题：

（1）父图和子图的平衡　在分层数据流图中，直接相邻的两层，上层称为下层的父图，下层称为上层的子图。通常父图中有几个加工，就对应下层有几个子图，如果父图中有基本加工，它就没有子图，此时子图的个数少于父图中加工个数。任何一个子图与其父图上的一个加工对应，两者的输入数据流和输出数据流必须一致。

（2）子图的编号　在顶层和底层之间的中间层的数据流图描述了某个加工的分解，而它的组成又可能要进一步分解。较小的系统可能没有中间层、而大的系统中间层可达8、9层之多。为了便于组织，应给加工对应的子图编号，其规则如下：子图的编号即为父图中相应加工的编号；子图中加工的编号由子图号、小数点、局部号构成。顶层只有一张，只有一个加工，不必编号。第一层编号为0，图中加工的编号为1、2、3……。第二层对应的子图编号为1图、2图、3图……，该层图中每个子图的加工编号为1.1、1.2……，2.1、2.2……。第三层中对应的子图为1.1图、1.2图……，2.1图、2.2图……，该层图中每个子图的加工编号1.1.1、1.1.2……，2.1.1、2.1.2……，等等。这样可以根据子图编号确定该子图在哪一层上。关于子图编号的例子如图3.7所示。

（3）数据守恒　所谓数据守恒是指加工的输入输出数据流是否匹配，应成对出现，即每一个加工既有输入数据流又有输出数据流。若有输入无输出，或有输出无输入都是数据不守恒的。

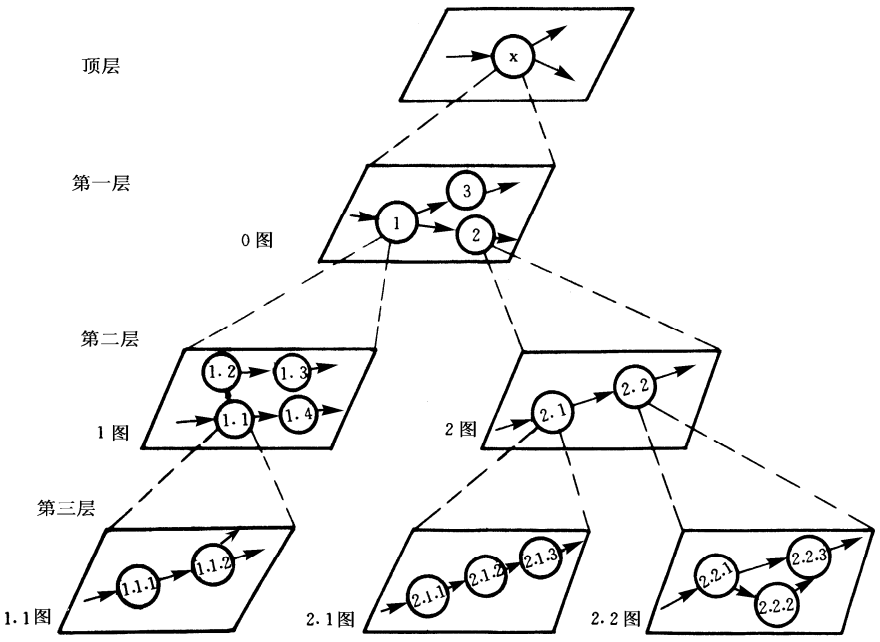


图3.7　子图的编号

下面以建立一个“职业培训中心管理系统”为例，说明在需求分析中使用数据流图的具体方法。

职业培训中心的主要任务是为就业人员举办各种专业和技能培训班，提高就业人员的业务素质，为就业作准备。要求培训的学员可以通过信函、电报、电话来联系各种事项，询问培训班的种类，开设的课程，讲授内容，学习时间，费用等；也可能在看到招生简章后，直接来报名学习；也可能正在中心学习一些课程，由于某种原因不能继续学习，要注销；也可能是要交纳费用。中心管理系统的工作任务就是要对上述各种情况进行处理。通常的处理过程是：

将学员发来的电报、信件、电话收集分类后，按几种不同情况处理。如果是报名的则将报名数据送给负责报名事务的职员，他们要查阅课程文件，检查某门课程是否满额，然后在学生文件、课程文件上登记，并开出报名单交财务部门，财务人员再开出发票经复审后通知学员。如果是付款的，则由财务人员在账目文件上登记，再经复审后，也给学员一张通知单。如果是注销原来已选修的课程，则由注销人员在课程、学生、账目文件上作相应修改，经复审后通知学员。如果是查询的，则交查询部门查阅课程文件后给出答复。对一些要求不合理的函电，中心将拒绝处理。

首先画顶层图，把整个系统功能抽象为“职业培训中心管理系统”，数据源和终点都是学员，输入数据流是函电，输出数据流是回答信息和通知单，如图3.8（a）所示。

然后把顶层图的加工处理分解为“处理函电”、“查询处理”、“综合处理”三个加工过程，得到0图，其输入输出数据流图如图3.8（b）所示。对0图中“处理函电”的加工进行分解，得到1图，如图3.8（c）所示。对0图中“综合处理”加工进行分解得到2图，如图3.8（d）所示。对2图的“报名处理”加工进行分解得到2.1图，如图3.8（e）所示。对于其他部分的进一步分解，限于篇幅，不再列举。

3.3.4 数据词典

数据流图描述软件系统的信息流和加工，尽管对所有图形元素都进行了命名，但它们都是对一些属性和内容抽象的概括，没有直接参加定义的人对每个名字可能有不同的理解。对一个大型软件项目来说，对数据流图上命名的不同理解，将会严重影响开发和维护工作。SA方法使用数据词典对数据流名、数据存储名、数据项名、基本加工名进行详细说明。对数据流图中包含的所有元素的定义的集合构成了数据词典。其作用正如用英语词典可以帮助查寻不认识单词的语义一样。数据词典是SA方法的主要工具之一，它与数据流图共同构成系统的逻辑模型，没有数据流图则数据字典难以发挥作用，没有数据字典则数据流图就不严格。

数据词典中有四种类型的条目：数据流、文件、数据项和加工。

（1）数据流条目

数据流条目给出某个数据流的定义，它通常是列出该数据流的各组成数据项。

在进行数据流定义时，可采用如下符号：

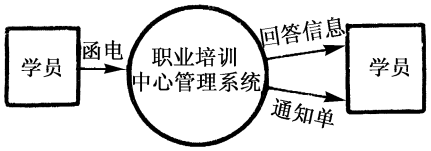
“＝”表示定义为，“＋”表示“与”。例如：报名单＝姓名＋单位名＋年龄＋性别

＋课程名，表示数据流“报名单”由上述各数据项组成。

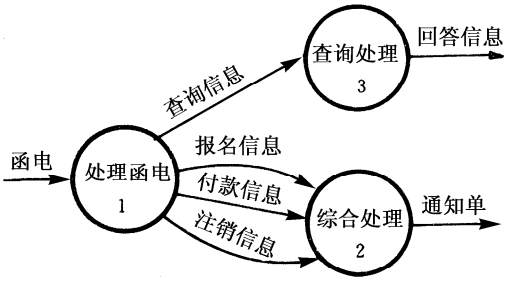
“［｜］”表示“或”，即选择括号中的某一项，例如：培训期＝［1｜3｜5］表示培训期可选1个月、3个月或5个月。

“｛　｝”表示“重复”，即括号中的项要重复若干次，重复次数的上、下限也可在括号边上标出。例如：发票＝｛发票行｝，表示一张发票有若干行。

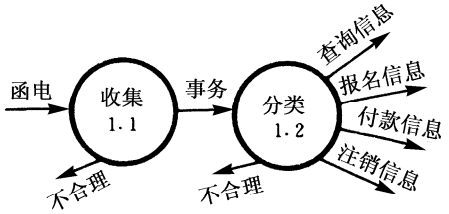
“（　）”表示“可选”，即括号中的项可能没有。



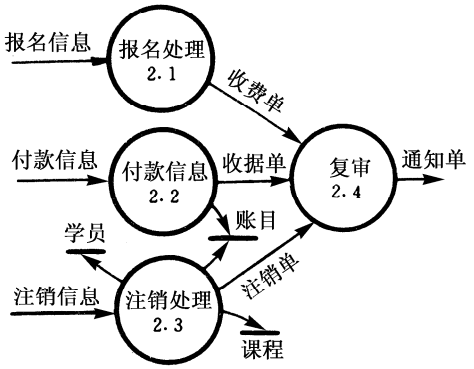
（a）顶层图



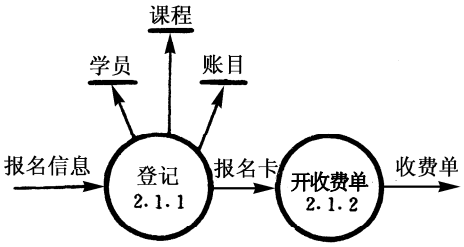
（b）0 图



（c）1 图



（d）2 图



（e）2.1 图

图3.8　培训中心管理系统数据流图

有些数据项组成比较复杂，可以采用自顶向下分解的方法将它表示为更低层成分的组合，以利于项目有关人员的了解。例如，某学校管理系统中，有名为“课程”的一个数据流，它由“课程名”、“教员”、“教材”和“课程表”组成，而“课程表”又由“星期

几”、“第几节”和“教室”组成，则在词典中可以建立“课程”及“课程表”两个条目，它们分别是：

课程＝课程名＋教员＋教材＋课程表

课程表＝｛星期几＋第几节＋教室｝

（2）文件条目

文件条目给出某个文件的定义，同数据流一样，文件的定义通常是列出文件记录的组成数据项，此外，文件条目还可以指出文件的组织方式，如按单号递增次序排列等。例如，在某个销售系统中有一个订单文件，则在数据字典的文件条目中对它应作如下的描述：

订单文件＝订单编号＋顾客名称＋产品名称＋订货数量＋交货日期

（3）数据项条目

数据项条目给出某个数据单项的定义，通常是数据项的值类型，允许的取值范围等，如，

学员编号＝1～9999

（4）加工条目

加工条目又称小说明。对数据流图中每一个不能再进一步分解的基本加工都必须有一个小说明给出这个加工的精确描述，而对数据流图中的其他非基本加工则不必。小说明可以按加工名的词典顺序排列，也可按加工的编号排列，作为数据词典的加工条目。

小说明中应精确地描述用户要求某个加工做什么，其中包括加工的激发条件、加工逻辑、优先级、执行频率和出错处理等，其中最基本的部分是加工逻辑。加工逻辑是指用户对这个加工的逻辑要求，即该加工的输出数据流同输入数据流之间的逻辑关系，描述的重点放在对某个加工中用户要求做什么，而不是用编程语言来描述具体的加工过程。

对基本加工说明有三种描述方式，采用的描述工具分别为：结构英语、判定表、判定树。

结构英语是描述基本加工条目中的加工说明的常用工具，它是介于自然语言和形式语言之间的一种语言。如果用自然语言来描述加工逻辑，其优点是容易理解，但是它不能精确描述，有二义性，很难把加工说明描述得简捷、清楚、精确；如果采用像程序设计语言那样的形式语言来描述加工说明，优点是简捷、清楚、精确，但是对于用户来说，是难于理解的，它涉及严格的语法定义，书写和理解都有困难，它还涉及严格的细节描述和具体存储安排，也不适于描述加工说明。而结构英语综合了这两者的优点，克服了这两种语言的缺点。它采用程序设计语言的控制结构作为加工说明的控制结构的外部框架，用自然语言来描述加工说明的内部处理，描述控制结构的外部框架称为外语法，描述加工逻辑的内部处理称为内语法，因此，结构英语有内外两种语法：外语法采用程序设计语言中的三种控制结构，即顺序、选择、重复，作为外部框架，把各种内部处理联系起来。内语法是采用自然语言英语描述框架内的局部处理，只用祈使句型，明确地叙述做什么事情。在我国，自然语言用汉语较好。外部控制结构框架的关键字用大写英文单词表示，而且按固定的标准格式书写，框架内部的局部处理，如判定条件、处理过程等用汉语的祈使句书写。例如：

（1）顺序结构

MOVE　单科成绩　TO　学生单科成绩表

MOVE　学费　TO　账目文件

（2）选择结构

IF　存货小于标准点

　DO　订货

ELSE　（不小于标准点）

　SET　标志

（3）重复结构

FOR　每一个学员号码

PUT　学员成绩

REPEAT

　FROM　消息线　GET　下一个消息

　WRITE　消息正文

　WRITE　消息头和长度

UNTIL　消息头和长度

在描述加工说明时，如果有一系列逻辑判断，用结构英语描述就不直观，也不简捷，这时可用另外两种工具：判定表和判定树来描述。判定表是用表格的形式列出了在什么条件下做什么处理，一目了然。判定树是以一棵从左向右生长的树形表示来描述在各种条件下要做的事情，树的各个分支表示某种条件，分支的端点表示该分支对应的条件要做的处理。图3.9给出一个判定树的例子。图3.10是把图3.9中的判定树表示成判定表图解，图中Y表示满足该项条件，N表示不满足该项条件，X表示执行该项操作。

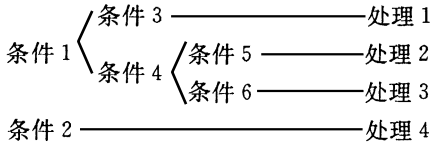


图3.9　判定树示例

条件1	Y	Y	Y	N
条件2	N	N	N	Y
条件3	Y	N	N	N
条件4	N	Y	Y	N
条件5	N	Y	N	N
条件6	N	N	Y	N
处理1	X			
处理2		X		
处理3			X	
处理4				X

图3.10　判定表图解

目前数据词典的实现有全人工过程、全自动化过程和人机混合过程三种途径。全自动化过程是利用专用的数据词典处理程序，人机混合过程则是用正文编辑程序等实用程序帮助人工过程。全人工过程则可用一叠卡片构造，其建立过程如下：为每个要定义的名字准备一张卡片，在卡片上注明名字和类型（数据流、数据存储数据项或加工），写上这个名字的定义及其他限制，最后将卡片按名字的字典顺序排列起来。图3.11中给出了数据流条目和数据存储条目的示例。

无论采用哪种途径实现的数据字典都应有下述特点：通过名字能方便地查阅数据的定义；没有冗余；尽量不重复其他部分已说明的信息；能单独处理描述每个名字的信息；书写方法简单方便而且严格，容易更新和修改等。

名字	报名单
种类	数据流
简述	培训人员概况
别名	无
组成	姓名+单位名+年龄……
数据	每天人数
…	
注	

（a）

名字	课程目录
种类	数据存储
简述	开课概况
别名	课程文件
组成	课程编号+课程名……
数据	按编号递增排列
…	
注	

（b）

图3.11　数据词典卡片示例

（a）数据流条目　（b）数据存储条目

小　　结

软件的需求分析是软件生存周期中一个重要而又困难的阶段，需求分析是用户和软件人员双方讨论协作的阶段，软件开发人员的任务是收集、理解、分析用户对软件系统的功能和性能要求，然后以需求说明书的形式准确无误地表达出来。

需求分析最常用的方法是结构化分析方法（SA方法），SA方法适于分析大型数据处理系统，使用的主要工具有数据流图和数据词典。数据流图以图形形式表示软件信息流向和信息加工，而数据词典对这些信息和加工进行更详细的描述。SA方法简单实用、易于理解、使用广泛。

3.4　软 件 设 计

对任何工程项目来说，在施工之前，总要先完成设计，设计往往是保证开发质量的重要环节。软件设计与其他领域的工程设计一样，也需要有好的方法、好的分析策略等。把软件设计看成仅仅是程序设计或者编制程序，这是很片面的。实际上，程序设计只是软件设计的实现，不能把它们混同起来。

3.4.1 软件设计概述

一、任务和目标

经过需求分析阶段得到了软件需求说明书。它是目标系统的逻辑模型，解决软件系统“做什么”的问题。进入软件设计阶段，要把目标系统的逻辑模型变为物理模型，解决软件系统“如何做”的问题。因此软件设计的任务可概括为：将分析阶段获得的需求说明转换为计算机中可实现的系统，完成系统的结构设计，包括数据结构和程序结构，最后应该得到软件设计说明书。

软件设计阶段是软件生存周期的中心，设计质量直接影响软件质量，因此在设计阶段应达到的目标是：

1．提高可靠性。可靠性包含正确性和健壮性两个方面，正确性指软件系统本身没有错误，健壮性指在输入数据不合理或异常时，软件系统还能适当工作，不造成严重的损害，在设计阶段必须考虑一定的容错措施。软件可靠性是一个重要的目标，它涉及软件系统能否投入使用，使用后效率是否好的问题。

2．提高可维护性。软件工程按阶段进行开发，但各阶段相互有影响，由于软件维护费用极高，因此在设计阶段需要考虑软件的可维护性，它体现在可读性、可扩充性、可修改性。

3．提高可理解性。软件设计提高可理解性的具体要求是：软件实体有明显的层次结构，软件实体应是模块化的，模块具有完全独立的功能；软件实体与环境的界面清晰；设计规格说明清晰、简洁、完整和无歧义性。

4．提高效率。效率是指软件设计能否有效地使用计算机资源，如时间和空间等。需要说明的是，效率虽然是衡量软件设计质量的一个重要方面，但在硬件价格下降、人工费用上升的情况下，设计者有时也宁可牺牲一定的效率换取其他方面的收益。

实现一个系统有多种方案，在设计阶段应在充分使用资源条件下，从可靠性、可维护性、可理解性、效率高等方面考虑。对多种设计方案进行比较，全面评价，从中选择一种较好的方案。

二、设计方法和步骤

软件设计是一个把软件需求变换成软件表示的过程。其方法是首先描绘出软件的总体框架，然后进一步细化，在此框架中填入细节，把它加工成在程序上非常接近于源程序的软件表示。从工程管理的角度来看，软件设计分为两步完成，总体设计和详细设计，其工作流程如图3.12所示。

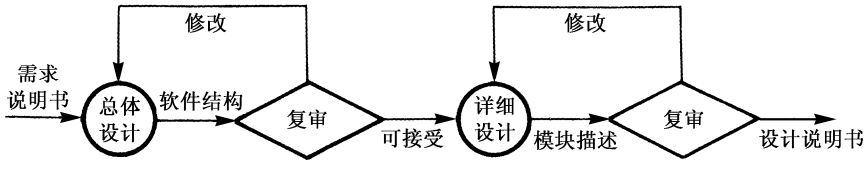


图3.12　设计流程图

总体设计也称为概要设计，其任务是将软件需求转化为数据结构和软件的系统结构，主要工作有：制定软件开发组在设计时应共同遵守的标准，以便协调各成员的工作；设计软件系统结构，进行模块划分，建立模块层次结构，确立模块间的调用等；设计全局数据结构和数据库，设计系统接口及人-机界面，设计数据安全性等。

总体设计的方法有：人们常用的模块化方法，功能分解方法，60年代后期提出的面向数据流的设计方法，面向数据结构的设计方法，近年来提出的面向对象的设计方法等。

详细设计是根据总体设计中对每个模块的功能描述，设计出每个模块的内部的实现算法，实现这些算法的逻辑控制流程，设计这些模块所需的局部数据结构。由于在后面的阶段，程序员将根据详细设计结果写出实际的程序代码，因而详细设计的结果基本上决定了最终的程序代码质量。

详细设计的方法主要是用结构程序设计方法，详细设计的表示工具有图形工具和语言工具。图形工具有程序流程图、NS图（由Nassi 和Shneiderman开发的，简称NS图）、PAD图（Problem Analysis Diagram）；语言工具有伪码等，伪码也称为PDL语言（Program Design Language）。

三、设计文档

设计阶段结束要交付的文档是设计说明书。设计说明书前面部分在总体设计后完成，后面部分在详细设计后完成。设计说明书是软件设计的最终成果，其主要作用是：对于编程和测试人员，提供工作指南；对于维护人员，在软件交付使用后，提供帮助。设计说明书的框架和内容如下：

1．概述。描述设计工作总的范围，包括系统目标、功能、接口及设计约束及限制等。

2．参考文档。现有的软件文档，系统的文档资料，外购件和软件文档，技术参考资料等。

3．系统结构。用软件结构图说明本系统的模块划分，扼要说明每个模块的功能，分层次地给出各模块之间的控制关系。

4．数据结构及数据库设计。对整个系统使用的数据结构及数据库进行设计，包括概念结构设计，逻辑结构设计、物理设计。用相对应的图形和表格把设计结果描述出来。

5．接口设计。包括人-机界面设计，说明向用户提供的命令以及系统的返回信息；外部接口设计，说明本系统与外界的所有接口安排，包括软件与硬件之间的接口、本系统与支撑软件之间的接口关系等。

6．模块设计。这是详细设计的结果，根据模块的功能，描述每个模块的流程及所用的数据结构，确立数据的组织形式。

7．测试的准备。拟定测试大纲、整体策略及具体考虑。

四、设计复审

每个设计步骤完成之后，都应进行复审，对软件设计来说，考虑周到、计划周密的复审与设计一样重要。目前，有两种不同的设计复审方法。一种是非正规复审，只召集少数设计人员和有关用户参加开会来复查所有技术文档。另一种是正规复审，要组织一个审查

小组，查看设计文档，听取设计者介绍情况，然后进行审议，以书面形式表示复审的结果。

对设计进行复审的目的是可以比较早地发现软件的缺陷，从而可以使软件设计中的缺陷在进行编程、测试和交付之前予以纠正，防止错误蔓延扩大，明显地降低开发阶段和维护阶段的费用。为了进行复审，开发者必须耗费时间、工作量和资金。但是，在早期付出的代价是为了避免以后付出更多代价。

3.4.2 软件设计准则

刻画一门工程学科属性的方法之一是确定其有意义的度量方式，而计算机软件可用的度量仍处于认识的开始，目前使用的软件设计阶段的基本度量还是一种定性的度量，但它为软件工程的科学方法提供了一个基础。下面分别讨论从软件设计实践经验中总结出的与软件结构的准则、模块化的准则、模块独立性的准则有关的几个问题。

一、软件结构的准则

软件可以从结构上和过程上进行表示，这两种表示是按总体设计和详细设计的要求实现的。软件结构表示软件的系统结构，它是软件模块间关系的表示。尽管软件模块间的关系可有多种多样（如调用、包含、从属、嵌套等），但无论哪种关系，均可表示为层次关系。如图3.13所示，位于最上层的是顶层模块，与其联系的有若干下属模块，各下属模块还可以进一步引出更下一层的下属模块。这种层次结构体系，它不考虑时间的先后和执行的顺序，而只给出各软件模块之间的关系和相互作用。从图3.13中可看出，模块A可以调用模块B、C、D，但没有指明调用B、C、D的次序和条件。软件结构只提供了软件模块间组成关系的表示。它不提供模块间实现控制关系的操作细节，更不能提供模块内部的操作细节。软件过程是用以描述每个模块的操作细节，同时也包括一个模块对下一层模块控制的操作细节。从图3.14中可以看出，执行P_1或P_2取决于一个条件判断，P_4要在P_3之后执行。所以过

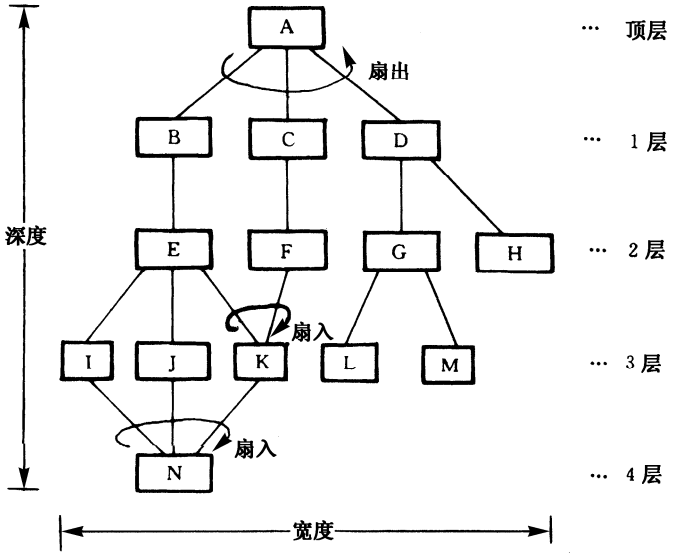


图3.13 软件结构

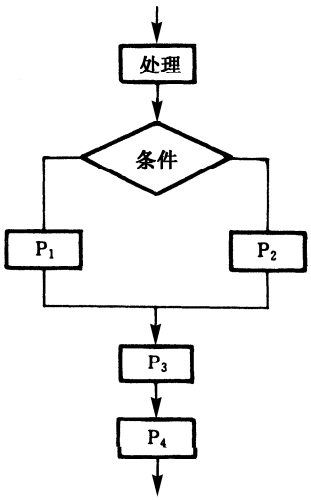


图3.14 软件过程

程的描述就是关于某个模块算法的详细描述。它应该包括处理的顺序、精确的判定位置、重复的操作以及数据组织和结构等。

层次结构通常用结构图表示，由于层次结构具有结构清晰、可理解性、易维护性好等特点，使它获得了广泛应用。为了描述软件结构的形态特征，下面介绍几个有关的术语。下属和上属关系：在软件结构的层次中上层模块调用下层模块，同一层模块之间不发生关系。在图3.13中模块A调用模块B、C、D，则模块A是模块B、C、D的直接上属，模块B、C、D是模块A的直接下属。同时模块A也是模块E、F、G、H的上属……。深度：表示软件结构中从顶模块到最底层模块的层数；宽度：表示控制的总分布；扇出数：指一个模块直接控制下属的模块个数；扇入数：指一个模块的直接上属模块个数。可见，一个好的软件结构的形态准则是：软件结构应具有倒水缸形，即顶部宽度最小，中部宽度最大，底部宽度次之；在结构顶部有较高的扇出数，在底部有较高的扇入数。

与刻画软件结构有关的另一准则涉及“影响范围”和“控制范围”两个概念。所谓一个模块的控制范围是指一个模块本身及所有下属模块构成的集合。在图3.13中，模块B的控制范围是模块B、E、I、J、K、N。所谓一个模块的影响范围是指一个模块中若有一个判定条件，所有受判定条件影响的模块的集合。假设图3.13中，模块G中有一个判定条件，受该判定条件影响的模块有L和M。则模块G的影响范围是L和M。所谓受某个判定条件影响的模块是指那些模块的执行与否与判定条件的成立与否有密切的关系。一个好的软件结构的第二个准则是：判定的影响范围应该是控制范围的一个子集。若不满足这个准则，应把判定条件移到上属模块中去，使其影响在控制范围之内，或者将受影响的模块移入到控制范围内。假设在图3.13中，模块H也受到模块G中判定条件的影响，则模块G的影响范围是G、H、L、M，而模块G的控制范围是L和M，这样一来，影响范围不是控制范围的子集，这时可把模块H移到G的下面，也可以把G中的判定条件移到模块D中。

二、模块化准则

对于模块这个术语目前尚无统一的定义，通常认为模块是数据说明、可执行语句等程序元素的集合，它是指单独命名的可通过名字来访问的过程、函数、子程序或宏调用。不分模块的程序是无法理解、无法管理、无法维护的程序。所谓模块化就是将程序划分成若干模块，每个模块完成一个子功能，把这些模块集中起来组成一个整体，以满足所要求的整个系统的功能。模块化的根据是，如果一个问题由多个问题组合而成，那么这个组合问题的复杂程度将大于分别考虑每个问题时的复杂程度之和。这个结论使得使用计算机编程的人员乐于利用模块化的方法来将复杂的问题分解成许多容易解决的简单问题。

一般程序设计语言都提供有建立模块的机制，以利于开发程序的模块化。但是模块化方法并不等于分解的模块越多越好。因为随着模块的增多，虽然开发单个模块的工作量减少了，但是设计模块间接口所需的工作量也将增加。因此，只有选择合适的模块数目才会使整个系统的开发成本最小。此外，选取模块大小要适当，模块太长人们难于理解，编程和测试效率都不高；模块太短又会使系统过于零碎。同时增加了执行模块时系统的资源开销。经验表明，一个模块的语句数量最好在30～50个语句之间，也就是一页纸能写下模块

的所有语句。

三、模块独立性准则

按照模块化的方法，把软件划分为模块后，如何评价模块结构的好坏呢？为解决这一问题提出了模块独立性概念。模块独立性的含义是指开发具有功能专一，模块之间无过多相互作用的模块。具有独立性的模块只涉及该软件要求的一个具体子功能，而且与软件结构的其他部分的接口比较简单。这种类型的模块可以并行开发，开发容易，能减少错误的影响，使模块容易组合、修改及测试。为了更好地定性度量模块独立性，下面引入模块耦合性和内聚性概念。耦合性是模块之间联系紧密程度的度量，内聚性是模块内各部分之间联系紧密程度的度量。

内聚和耦合是相互关联的。在程序结构中各模块的内聚程度越高，模块间的耦合程度就越低。软件设计的目标是按照模块独立性准则，力求增加模块的内聚，尽量减少模块间的耦合，即：使模块功能单一，模块之间连接关系简单。必须指出，增加内聚比减少耦合更重要，应当把更多的注意力集中到提高模块的内聚程度上来。

3.4.3 结构化设计方法

结构化设计（Structured Design，简称SD）方法，它是由美国IBM公司的Constantine等人研制的。这种方法用于软件系统的总体设计，它可以同分析阶段的SA方法衔接起来使用。SD方法的目标是建立结构良好的软件系统，基本思想来源于模块化、自顶向下逐步求精的功能划分、模块独立性准则、结构化程序设计等程序设计技术。SD方法提供了总体设计时从数据流图导出软件结构的方法和规则，并采用软件结构图描述软件结构。

用SD方法设计的软件结构，由于模块之间相对独立，所以模块可以独立地被理解、编程、测试、排错和修改，这就使得复杂的研制工作得以简化，此外，模块的相对独立性也能有效地防止错误在模块之间扩散蔓延，因而提高了系统的可靠性、可理解性和可维护性。

一、面向数据流图的设计过程

使用SD方法的基础是数据流图。正如前面所述，几乎所有软件在分析阶段都可以表示为数据流图，所以SD方法基本上可适用于任何软件的开发工作。用SD方法进行总设计的过程大致如下：

（1）研究、分析和审查数据流图，从软件的需求说明书中弄清数据流加工的过程；

（2）根据数据流图确定数据流图的类型；

（3）从数据流图导出系统的初始软件结构图；

（4）改进初始软件结构图，直到符合要求为止；

（6）复查。

二、数据流图的类型

通常可以把数据流图分为两大类型，即转换处理型和事务处理型。由于大多数数据流图都可看成是对输入数据进行转换而得到输出数据的处理，因此可把这类处理抽象为转换

处理型。转换处理过程大致分为输入数据、变换数据和输出数据三步；它包含的数据流有输入流、转换流、输出流三个部分。在输入流中，信息由外部数据转换为内部形式进入系统；在转换流中，对内部形式的信息进行一系列加工处理，得到内部形式的结果；在输出流中，信息由内部形式的结果转换为外部形式数据流出系统。转换处理型的数据流图如图3.15所示。

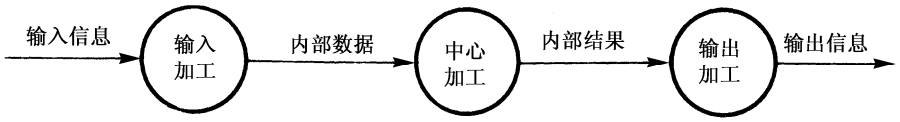


图3.15　转换处理型数据流图

另一类数据流图可看成是对一个数据经过某种加工后，按加工的结果选择一个输出数据流继续执行的处理。这种类型的处理可以抽象为事务处理型。在事务处理中，输入数据流称为事务流，加工称为事务中心，若干平行数据流称为事务路径。当事务流中的事务送到事务中心后，事务中心分析每一事务，根据事务处理的特点和性质选择一个事务路径继续进行处理。

事务处理型的数据流图如图3.16所示。

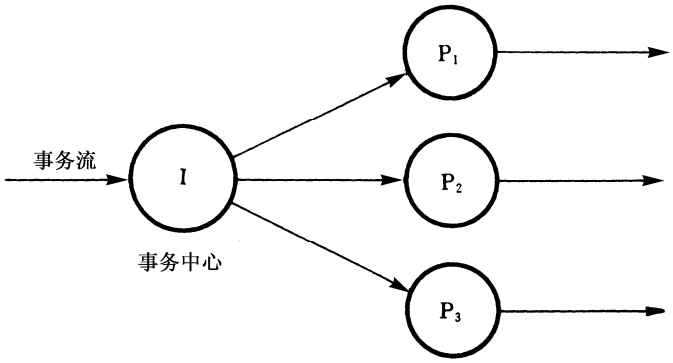


图3.16　事务处理型数据流图

通常，一个系统的数据处理问题从总体上看，总能表示为转换处理型，可采用转换分析建立系统的结构图。但也可能遇到明显的事务数据处理问题，这时可采用事务分析设计方法。因此，设计人员应当根据数据流图的主要问题类型，选择一个面向全局的，即涉及整个软件范围的问题处理类型。此外，在局部范围内是变换处理型，还是事务处理型，可具体研究，区别对待。

三、软件结构的描述方式

在SD方法中，软件结构用一种结构图来描述，它是设计说明书的一部分。结构图描述了软件模块结构，并反映了模块和块间联系等特性。在结构图中，用方框表示模块，框内用文字标记模块的名字，模块的名字还常用以反映模块的功能；用模块之间的箭头表示模块间的调用关系；用调用箭头边上的小箭头表示调用时模块间传送的数据，小箭头的方向表示传送信息的方向；用模块之间的菱形符号表示上属模块有条件地调用下属模块；用弧

形箭头表示上属模块重复地调用下属模块。图3.17是某结构图中一部分的示例。

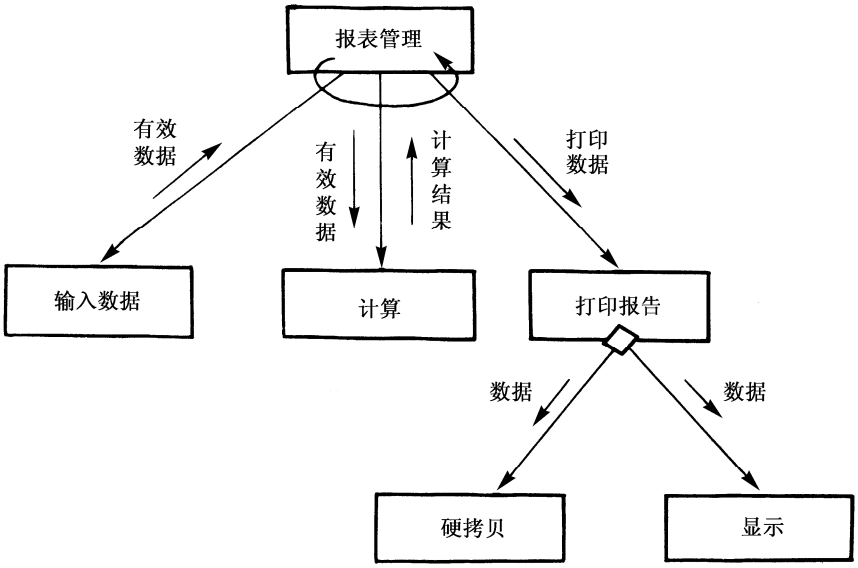


图3.17　结构图示例

必须指出，“结构图”同“程序流程图”（在下一节中介绍）是不同的。一个软件有层次性和过程性两个方面的特点，通常“层次性”反映的是整体性质，“过程性”反映的是局部性质，“结构图”描述软件结构的层次特性，表示模块与模块相互间的隶属和调用关系。“程序流程图”描述软件的过程特性，表示程序中先执行哪一部分，再执行哪一部分等。总体设计时，设计者关心的是程序的层次结构而不是执行过程，所以用结构图作为描述手段，而程序流程图一般在详细设计时才使用。

四、转换分析

转换分析是系统结构设计的一种策略，使用转换分析技术可把转换型处理数据流图转换为初始的标准结构，有了标准结构再根据软件结构的度量准则、模块化准则、模块独立性准则来修改完善软件结构图，从而得到结构良好的最终结构图。

转换分析步骤为：确定数据流图的类型；确定输入流、转换流、输出流的流界；进行一级分解，设计上层模块；进行二级分解，设计中下层模块；进一步细化。

（1）确立数据流图的类型

对分析阶段获得的数据流图进行分析，根据其主要特性，确立数据流图是否是转换处理型。

（2）确立流界

这一步主要是要确定输入流和输出流的边界，孤立出中心加工。首先要找出输入流的终点，输入流是把物理输入转换为逻辑输入的数据流，即从输入设备上获得未经加工的外部信息，经过格式转换、合理性检查等而得到的内部信息。还要找出输出流的起点，即经过中心加工以后得到的结构信息，经过格式转换，组成物理块和各种需要的图表，并经过

物理设备输出。要找出中心加工，一般可根据经验，由于中心加工是系统的转换部分，因此，对于设计人员来说是不难解决的。

（3）进行一级分解，设计上层模块

下面以图3.18（a）为例来说明分解的过程，首先，在中心加工位置设计一个主模块（顶层模块），命名为M，其功能是控制整个软件结构，完成系统所要做的各项工作，其次为输入流设计一个输入控制模块CI，功能是为主模块提供逻辑输入信息。为输出流设计一个输出控制模块CO，功能是为主模块提供的数据输出，为中心加工设计一个转换控制模块CT，功能是控制所有内部数据的操作。上层模块设计结果如图3.18（b）所示。

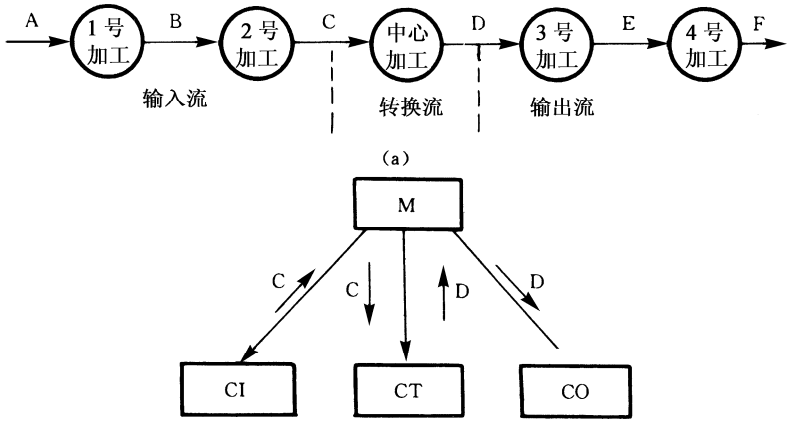


图3.18　转换分析

（a）转换处理型数据流程图　（b）顶层结构图

（4）进行二级分解，设计中下层模块

这一步工作是自顶向下，逐步细化，为第一层的每一个输入模块、输出模块、转换模块设计它们的从属模块，设计下层模块的顺序一般从设计输入模块的下层开始。首先为每一个输入流加工设计一个输入模块，向上属模块提供输入信息，而这个模块又需要有两个下属模块，一方面本身需要信息来源，另一方面又需将接收到的信息转换为上属模块所需的信息，即为“取信息模块”和“转换信息模块”。对于接收数据模块又是输入模块的情况，又要重复上述过程，如果输入模块已是物理输入端，则细化工作停止。因此，也可以说，对于每一个逻辑输入，在数据流图上采用了向物理输入端方向逆向移动的方式，只要还有加工框，就在相应输入模块下面建立一个子输入模块和子转换模块。在图3.18的数据流图中，C是逻辑输入数据流，它是CI模块向M提供的输入，而CI模块又需要两个下属模块。因此，对数据流图中的2号加工，设计“取B”和“转换B”两个模块，对1号加工设计“取A”和“转换A”两个模块。“转换A”的输出信息是B，传送到上属模块“取B”，模块“转换B”的输出信息是C，传送到上属模块CI，如图3.19（a）所示。

为每个逻辑输出流设计输出模块时，同样，也应当有两个下属模块，一个是将上属模块提供的数据变换成输出的形式；另一个是将它们输出。因此，对于每一个逻辑输出，在

数据流图上可采用向物理输出端方向正向移动的方式，只要还有加工框，就在相应输出模块下面建立一个子变换块和一个子输出模块。因此对于图3.18中的3号加工设计了“转换D”、“送E”两个模块，对4号加工设计了“转换E”、“送F”模块。由于数据流F已是物理输出数据流，所以分解到此为止。图3.19（b）是分解的输出模块结构图，图中的CO模块用于输出M模块送来的数据流D。

为中心加工设计转换模块，一般可根据中心加工的子加工来建立模块，对图3.18中的数据流图的逻辑输入C和逻辑输出D设计出相应的转换模块，如图3.19（c）所示。

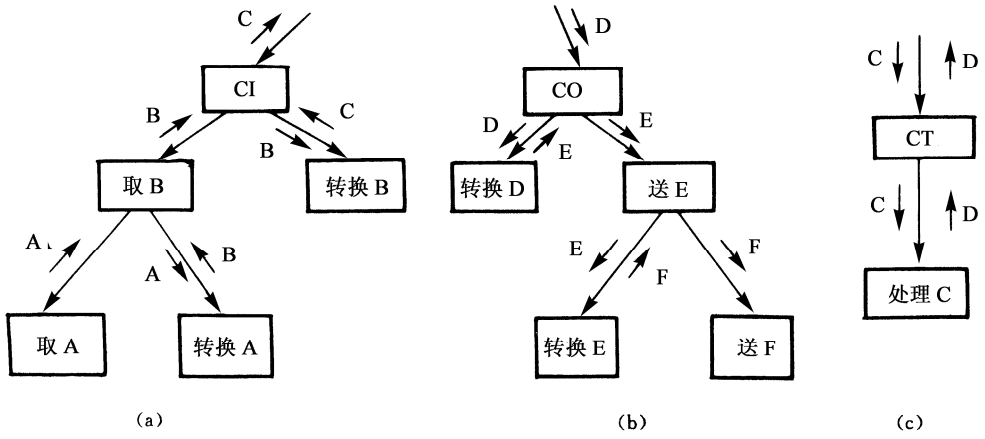


图3.19　中下层结构图

（a）输入流结构图　（b）输出流结构图　（c）转换流结构图

（5）进一步细化

由上述步骤得到的软件结构图是采用转换分解技术方法得到的标准结构图，这只是一个初步的设计，其中可能有不符合软件设计准则的地方，需要对初始结构图作进一步修改和完善。例如，可以取消仅仅是传送信息而没有处理工作的模块；对功能多、还比较复杂的模块需要进一步分解；对某个模块的判定的影响范围不在该模块的控制范围内时，需对该模块作适当调整，消除重复功能等。

五、事务分析

通常一个大型软件系统是转换处理型结构和事务处理型结构的混合结构。所以，设计者常利用以转换分析为主、事务分析为辅的方式进行软件结构设计。当数据流图明显呈现出事务型特征时，就要用事务分析技术，从相应的数据流程图导出标准结构图，其步骤与转换分析相同，下面以图3.20（a）中的数据流图为例予以说明。

（1）决定数据流图类型

事务处理型数据流图的显著特点是有一事务流，经过事务中心后，发散为若干事务路径，某一事务只按其中一条事务路径处理下去。在分析数据流图的基础上，确定它是否是事务处理型数据流图。

（2）确定流界

事务处理型的流界比转换处理型的流界容易确立。首先从数据流图中找出事务流、事

务中心和事务路径。事务中心前是接受事务，事务中心后是事务路径，而事务中心本身有显著特点，即一股数据流流入，有若干数据流流出。

（3）进行一级分析，设计上层模块

事务分析的任务是从数据流图中导出具有接受分支和发送分支的软件结构。对事务中心应设计“事务控制”模块；对事务流应设计“接受事务”模块；对事务路径，应设计“发送控制”模块。对应的上层模块如图3.20（b）所示。

（4）进行二级分解，设计中下层模块

对于接受分支，可用类似于转换处理型数据流图中对输入数据流的分析方法设计中下层模块。对于发送分支，在发送控制模块下为每条事务路径设计一个事务处理模块，这一层称为事务层。在事务层模块下，沿各事务路径进行分析，进一步细化，设计动作层模块及细节层模块等等。其标准结构图如图3.20（b）所示。

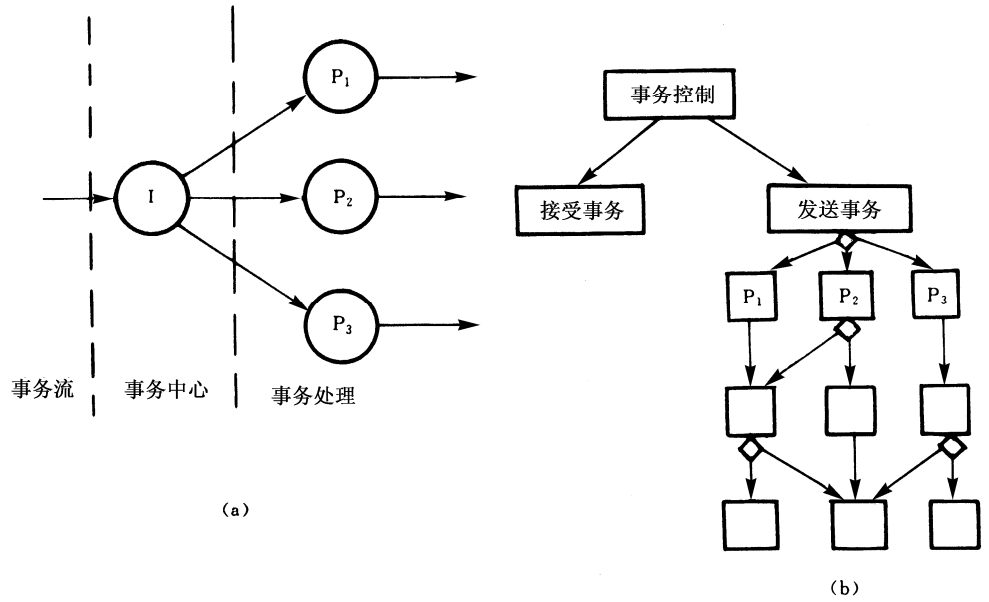


图3.20　事务分析

（a）事务处理型数据流图　（b）标准结构图

六、设计后处理

在完成转换分析和事务分析以后，设计者对从数据流程图得到的标准结构图还要进行修改完善，以获得满意的最终结构图。根据总体设计的要求，设计者还应做好以下几项工作：为每一个模块写一份处理说明；为每一个模块提供一份接口说明，确定全局数据结构和局部数据结构，指出所有的设计约束和限制；进行设计评审；必要时进行设计优化。

总体设计的最后工作是交付设计文档，文档的主要内容包括：

（1）结构图；

（2）每个模块的描述：功能、界面、过程、注释；

（3）数据库、文件结构和全程数据的描述；

（4）测试计划。

在讨论完本节内容以后，我们以汽车仪表板的设计为例，说明用SA方法和SD方法进行需求分析和概要设计的过程。

由微处理器控制的汽车仪表板是具有“智能”的仪表板，在这类产品中，把设计的软件做在只读存储器中，作为设备的一部分，从而使设备具有某些“智能”。这类产品的设计都包含有软件开发任务。

1．需求分析

假设仪表板将完成下述功能：

（1）汽车发动机经传动齿轮、底盘和转换器，经A/D变换后实现与微计算机接口；

（2）在发光二极管面板（LED）上显示数据；

（3）指示每小时英里数（mph）、行驶的里程（m）、每加仑油行驶的英里数（mpg）等；

（4）指示加速或减速；

（5）超速警告，即车速与标准速度55里/每小时相比较，超速则发出警告铃声。

在软件需求分析时应对上述每条要求以及系统的其他特点进行全面的分析评价，建立起必要的文档资料。假设系统规格说明、软件计划说明和软件需求规格说明都已具备，根据对上述功能的分析，可画出如图3.21所示的数据流图。

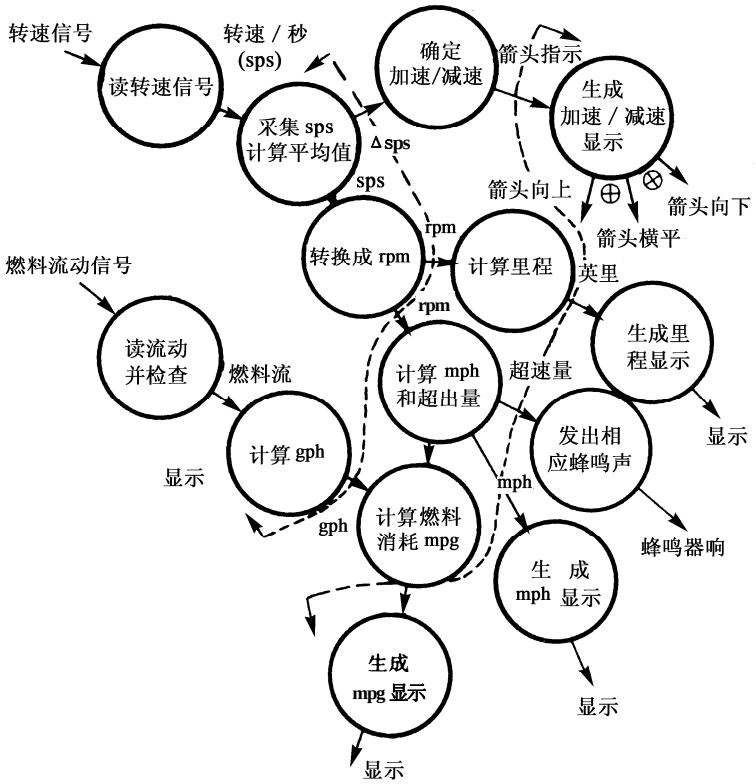


图3.21　数字仪表系统的数据流图

2．总体设计

图3.21是经细化的数据流图。数据流图中的第一条输入流（左上方位置）是经过转换过的转速信号读入并转换成数字量（每秒脉冲数），计算出每秒平均值sps和每秒增量Δsps并用之驱动加/减速度指示器，以给出加速、减速和等速指示。把sps转换成每分钟转速rpm，用rpm指示里程并计算超越量，驱动蜂鸣器。用rpm生成时速mph指示，用mph计算燃料消耗。

数据流图中的第二条输入流是读入的燃料流量信号，计算每小时消耗加仑数，计算每加仑里程（mpg）并显示。

从该数据流图看出，输入数据沿着两条输入通路进入系统，然后沿着五条通路流出，没有明显的事务中心，因此可以认为这个数据流图具有中心变换型的结构。对中心变换型结构需要确定其输入流和输出流，从而找出中心加工来。输入流和输出流的边界和对它们的解释有关，也就是说不同设计人员可能会在数据流内选取不同的点作为边界的位置。

根据本题特征，在寻找中心变换单元时，可以采取由输入/输出边界向内搜索的办法，即从输入流和输出流的边界向中心推进，直到可以明显地鉴别出输入和输出分界为止。分界所包围的单元即为中心处理单元。图3.21中的虚线即为I/O分界，分界内包围了四个中心处理单元，即，确定加速/减速、计算里程、计算mph、计算超速量和燃料消耗。

进行一级分解时，首先设计上层模块，为了保证模块的内聚性，开始时可以把四个中心处理单元看成是一个模块，所有输入看成是一个模块，所有输出看成是一个模块。初始的模块结构图如图3.22中的上面部分所示，主控模块是“数字仪表板控制”，输入控制模块是“接受传感器信号”，转换控制模块是“数据变换控制”，输出控制模块是“驱动仪表板”。进行二级分解时，把三个加工模块再细分为适当的加工模块，其映射的结果得到了如图3.21所示的软件结构图。为了产生合理的分解，得到尽可能高的内聚性，尽可能低的耦合性，对软件结构图总可以根据模块独立性准则进行完善。为了使生成的软件结构易于实现、易于测试、易于维护，应对上面映射出的软件结构进行再合并或分解。可能进行的修改有：

（1）输入部分的模块“转换成rpm”和“采集sps”可以合并以提高模块的内聚；

（2）变换部分的模块“确定加/减速”可以放在模块“计算mph”下面，以减少耦合；

（3）模块“加速/减速显示”可以相应放在模块“显示mph”的下面。

经过上述修改后的软件结构图如图3.23所示。

由此可见，在进行软件设计时，软件结构图是描述软件结构的常用工具，在进行软件结构设计时，应该遵循的最主要的准则是模块独立性准则，也就是说软件应该由一组完成相对独立的子功能的模块组成，这些模块彼此之间的接口关系应尽量简单。在进行详细设计和编程之前，先进行软件结构设计，其好处在于可以在软件开发的早期站在全局高度对软件结构进行优化，在这个时期进行优化付出的代价不高，却可以使软件质量得到重大改进。

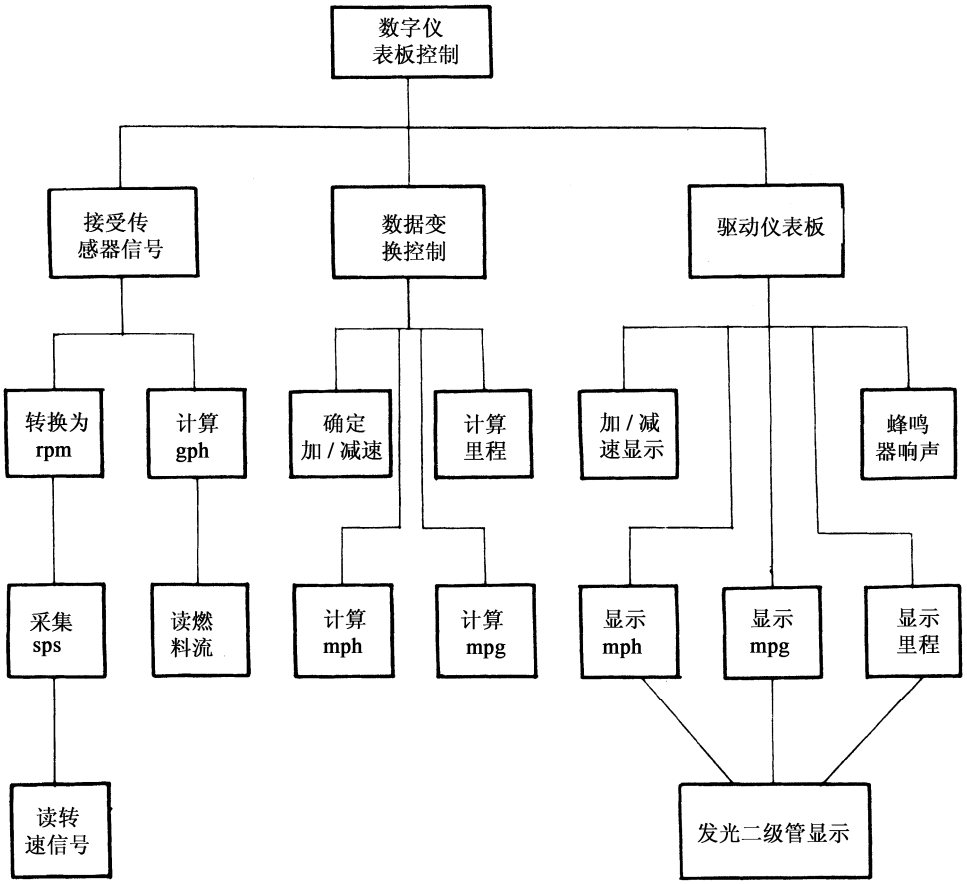


图3.22　数据流图映射成的软件结构图

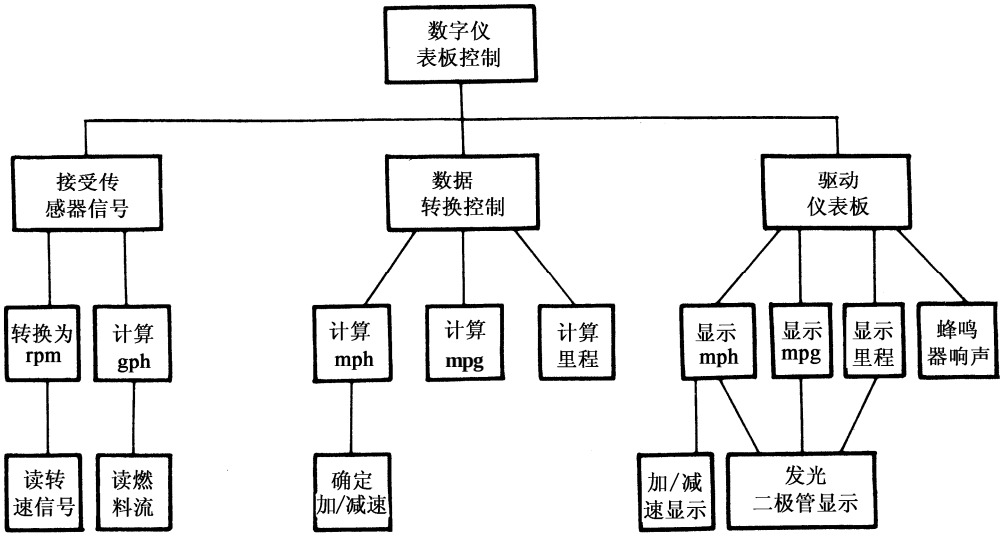


图3.23　修改后的软件结构图

3.4.4 详细设计方法

详细设计是软件设计阶段的第二步，在总体设计阶段，采用结构化设计方法，已经确定了软件系统的总体结构，给出了系统中各个组成模块的功能和模块间的接口。本步骤的工作，就是要在上述结果的基础上，考虑如何实现这个软件系统，直到对系统中的每个模块给出足够详细的过程描述。

在编程阶段。程序员将根据详细设计结果写出实际的程序代码，因而，详细设计的结果基本上决定了最终的程序代码质量。由于在软件的生存周期中，设计测试方案，诊断程序错误，修改和改进程序等工作都必须首先读懂程序，因此可读性是衡量程序质量的一个重要指标，为了提高程序的可读性，详细设计的任务就不仅仅是逻辑上正确地实现每个模块的功能，更重要的是设计出的处理过程应尽可能简单明了。

一、详细设计的任务

详细设计的任务，是为软件结构图中的每一个模块确定采用的算法和块内数据结构，用某种选定的表达工具给出清晰的描述，表达工具应具有描述过程细节的能力，而且在编程时能够使程序员直接翻译成程序设计语言的源程序。

详细设计阶段将产生详细设计说明书，为此，设计人员必须做到：为各个模块写出实现算法的详细过程性描述；为每个模块确定使用的数据结构；为每个模块确定接口细节，包括对系统外部的接口和用户界面，与系统内部其他模块的接口，以及关于模块输入数据、输出数据以及局部数据的全部细节。在详细设计结束时，应该把上述结果写入详细设计说明书，并且通过复审形成正式文档，作为编程阶段的工作依据。

二、结构程序设计方法

结构化程序设计（Structured Programming，简称SP）方法是在对程序质量标准的探讨中逐步形成的技术。在程序质量标准的讨论中，计算机科学家E.W.Dijkstra首先提出“GOTO语句是有害的”，他认为由于使用GOTO语句造成程序结构混乱，致使程序质量下降，它是造成程序难于控制其复杂性的根源。1966年C.Bohm和G.Jacopinl从理论上证明了只用“顺序”、“分支”、“循环”三种基本控制结构就能实现任何单入口、单出口的程序。于是针对是否禁用GOTO语句的问题在计算机界展开了一场激烈的争论，经过争论大家认为，问题不只是要不要使用GOTO语句，问题的实质是对程序应该提出什么样的质量要求，应该采用什么样的程序设计思想和方法。在这种形势下，20世纪60年代末至70年代中逐渐形成了结构化程序设计技术。

结构化程序设计的根本出发点是：为了保证程序设计的质量，应使所设计的程序具有易读性、易理解性、通用性好，而且执行时具有较高的效率。

结构化程序设计方法的基本要求是：在详细设计阶段为了确保模块逻辑清晰，就应该使所有的模块只用单入口、单出口和顺序、选择和循环三种基本控制结构。这样，不论一个程序包含多少个模块，每个模块包含多少个基本的控制结构，整个程序仍能保持结构清晰。结构化程序设计还要求在设计过程中采用自顶向下逐步求精的设计方法。

所谓逐步求精就是把整个程序设计过程分为明确的层次，把一个复杂的过程分解为多个简单的过程，逐步加以解决。在解决过程中，每一步是在前一步的基础上进行的，是前

一步设计的细化和具体化，细化过程一直进行下去，直到一系列的处理步骤能方便地使用某种高级语言或机器指令写出为止。这样的求精过程可以保证设计过程条理清楚、不重复、不丢失、逻辑性强。

使用结构化程序设计技术的优点是：

（1）自顶向下、逐步求精的方法符合人们解决复杂问题的普遍规律，可以显著地提高软件开发的成功率和生产率；

（2）用先全局后局部，先总体后细节，先抽象后具体的逐步求精的过程开发程序，使程序有清晰的层次结构，并容易理解和阅读；

（3）使用逐步求精的方法进行详细设计，由于相邻步骤之间变化甚小，不难验证其内容是否等效，易验证设计的正确性；

（4）控制结构只有确定的几种逻辑模式，编写程序代码直截了当，使程序易读易懂而且容易测试；

（5）程序层次分明和模块化的特点，使得修改和重新设计软件时可重用的代码量增加。

结构化程序设计的缺点是：需要的存储容量和运行时间都有一定的增加，但是随着计算机硬件的飞速发展，这已经不是严重的问题。必须指出，尽管结构化程序设计技术只用三种控制结构来编程序，虽然也从理论上证明了只用这三种控制结构能编制任何程序，但实际应用问题是复杂的，多种多样的，有时可能只用一条GOTO语句就能解决的问题，由于限制使用GOTO而采用其他方法，反而使程序复杂化了，所以通过实践，扩充了重复结构的类型，还增加了新的一类控制结构，即出口结构，它是一种有限制的GOTO结构，允许从循环体中转移出来，从而终止循环等。

三、详细设计使用的工具

描述程序处理过程的工具称为详细设计工具，这些工具应该能支持对详细设计无二义性的描述，以便在编码阶段能把所描述的详细处理过程直接翻译成程序代码。

1．程序流程图

程序流程图也称为程序框图，它是软件开发者熟悉的一种算法表达工具。它独立于任何一种程序设计语言，比较直观，清晰，易于掌握。因此，程序框图是软件开发者最普遍采用的一种进行详细设计的工具。人们在需要了解别人开发软件的具体实现方法时，也常常需要借助程序框图，来理解其思路及处理方法。

程序框图的表示方法很简单，框图中只能使用规定的图形符号，如用方框表示一个处理步骤，菱形框表示逻辑判定条件，箭头表示控制流向等。图3.24给出了国际标准化组织提出，并已为中国国家技术监督局批准的程序流程图中的基本符号。

按结构化程序设计技术的要求，由规定的图形符号构成的基本控制结构如图3.25所示，不同类型的基本控制结构的主要特征如下：

顺序结构：在这种结构中，几个连续的加工依次排列，并按序执行。

选择结构（IF-THEN-ELSE结构）：这种结构是根据所给定的判定条件是否满足，从两个加工任务中选择执行其中的一个任务。

多分支选择结构（CASE结构）：在这种结构中，根据控制变量的取值，从列举的多种加工任务中，选择执行其中一个任务。

先判定型循环结构（WHILE结构）：这种结构中先测试判定条件，只要条件为真就重复执行一个循环加工任务。

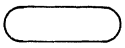
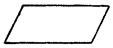
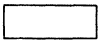
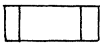
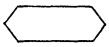
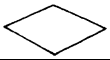
符号名	使用图形	意义
端点符		表示流程图的开始或结束
输入/输出符		表示数据的输入/输出
处理符		表示对数据处理
特定处理符		表示调用子程序等
准备符号		表示初始状态
判断符号		表示条件判断，由判断结果决定如何执行
循环开始符		表示循环开始
循环结束符		表示循环结束
连接符		指出流程图中的转移处
流线符号		用以连接流程图中各符号

图3.24　程序流程图中最基本的符号

后判定型循环结构（UNTIL结构）：这种结构是先执行循环加工任务，然后测试判定条件，只要条件为真就重复执行一直到该判定条件不成立为止。

任何复杂的程序流程图都可由这些不同类型的基本控制结构组合或嵌套而成。图3.26是流程图的一个示例，它的特点是简单清晰。

必须指出的是：由于在程序流程图中，设计人员用箭头可以实现向任意位置的转移，如使用正确的话，流程图是简单易懂又很灵活的，但使用不当时，流程图就可能非常难懂，而且无法进行维护。对箭头的使用必须小心谨慎。

2．方框图（N-S图）

为了避免程序流程图在描述程序逻辑时用箭头代表控制流的随意转移性与灵活性，1973年Nassi和Shneiderman提出了一种符合结构化程序设计原则的图形描述工具——方框图，以代替传统的程序流程图，人们也把这种图称为N-S图。

用方框图描述五种基本控制结构的方法如图3.27所示。

任何一个方框图都是由这五种基本控制结构相互组合与嵌套构成的。为了说明方框图的使用方法，这里仍以图3.26提供的程序流程图为例，将其用方框图表示，如图3.28所示。

从方框图的实例中可见，每个处理步骤都是用一个方框表示，这些处理步骤可以是语句或语句序列，在需要时，方框中还可以嵌套另一个方框，嵌套深度一般没有限制，只要整张图可以在一张纸上容纳下就行。由于只能从上边进入方框然后从下边走出，除此之外没有其他的入口和出口，所以NS图限制了随意的控制转移，保证了程序的良好结构。

实际使用情况说明方框图与传统的流程图相比具有明显的优点：其一，它强制设计人

员按结构化程序设计方法进行思考并描述设计方案，因为除了表示几种标准控制结构的符号之外，它不再提供其他描述手段，这就保证了程序设计和质量；其二，方框图形象直观、易于表达嵌套结构，容易确定局部数据和全程数据的作用域，具有良好的可见度，容易理解设计意图，为编程、复审选择测试用例、维护都带来了方便。由于方框图的上述优点，它已被许多软件开发人员接受，成为一种流行的描述方式。

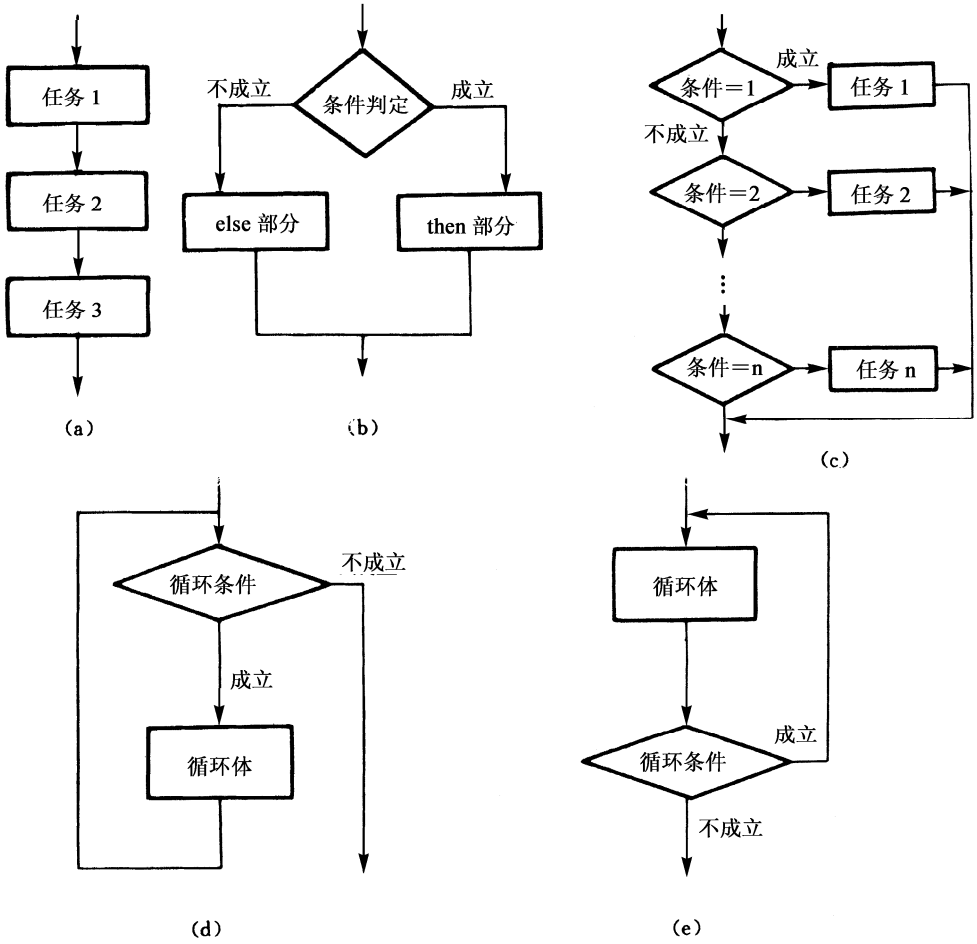


图3.25　程序流程图的基本控制结构

(a)顺序结构　(b)选择结构　(c)多分支选择结构　(d)先判定型循环结构　(e)后判定型循环结构

3．问题分析图（PAD图）

问题分析图（Problem Analysis Diagram，简称为PAD）是日本日立公司二村良彦等人于1979年提出的又一种主要用于描述软件详细设计的图形表示工具。现已成为受到许多人欢迎的有前途的表达工具之一。

PAD图使用的基本控制结构如图3.29所示。其中分支结构由两部分组成，带锯齿的框为条件判定，每一个锯齿表示该判定中的一种条件，与该齿相连的处理框表示该分支要执行的处理。循环结构也由两个部分组成，左边的一个框（右部多一道竖线）表示循环的类型和终止条件，右边的一个框表示要重复执行的循环体。

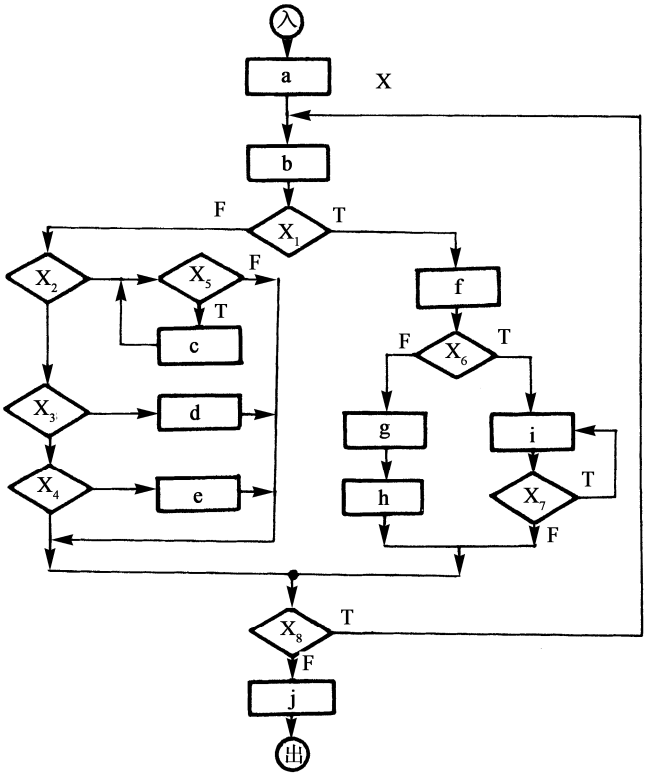
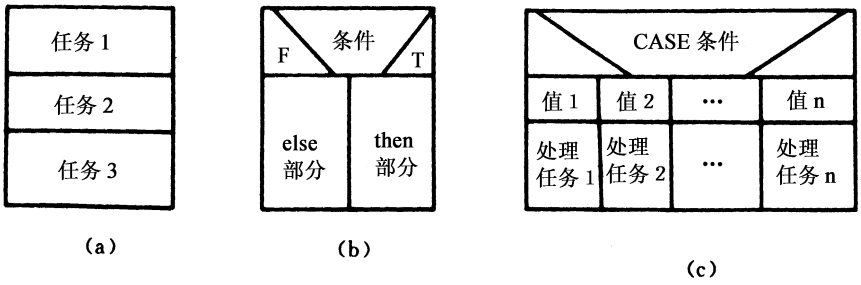


图3.26　程序流程图示例



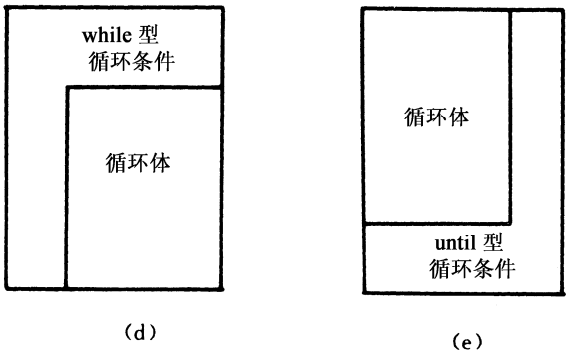


图3.27　用方框图表示的基本控制结构

(a)顺序结构　(b)选择结构　(c)多分支选择结构　(d)先判定型循环结构　(e)后判定型循环结构

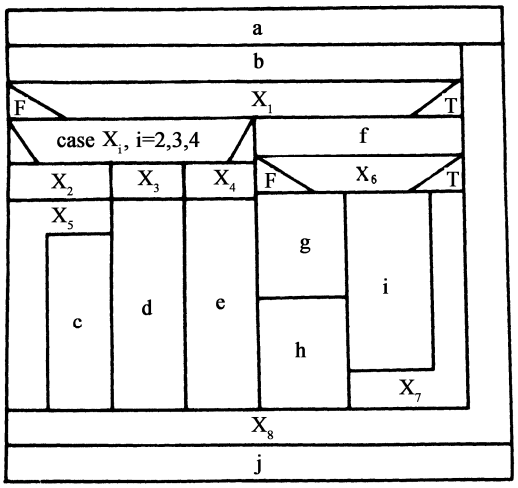


图3.28　方框图示例

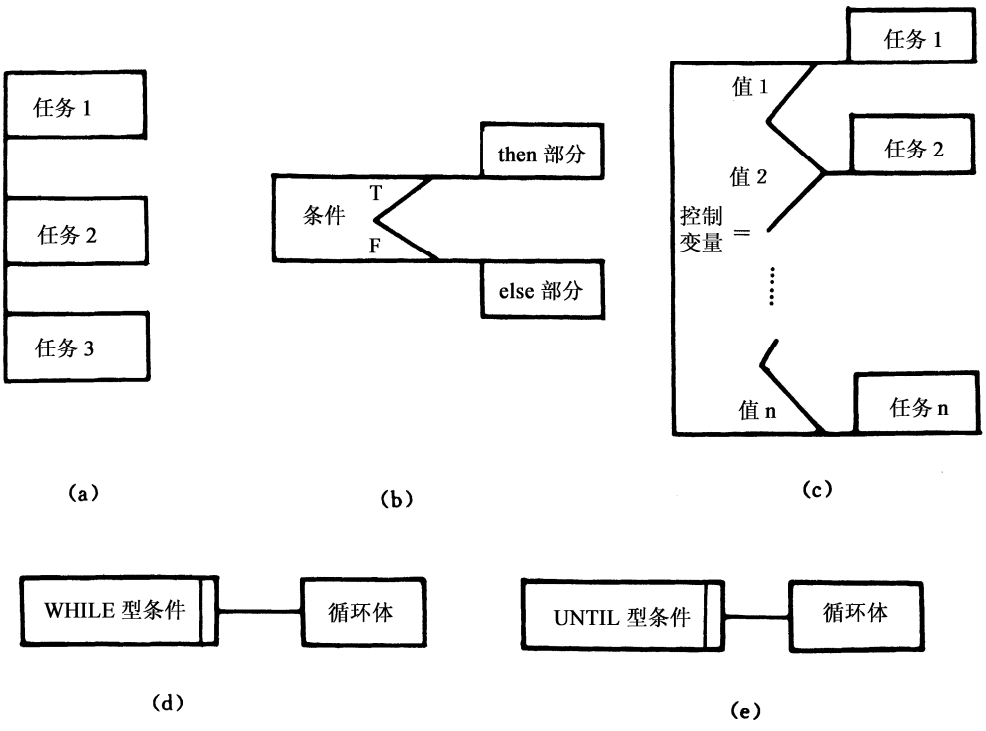


图3.29　PAD图的基本控制结构

(a)顺序结构　(b)选择结构　(c)多分支选择结构　(d)先判定型循环结构　(e)后判定型循环结构

任何一个PAD图都是由上述的基本控制模块构成的。作为PAD图应用的一个例子，图3.30给出了表示图3.26程序的PAD图。

PAD图也是一种二维平面图形，它是从左向右逐渐延伸的树形结构图，每增加一个层

次，图形就向右扩展一条竖线，PAD图中竖线的总条数就是程序的层次数。由基本控制结构组成的PAD图和方框图一样比程序流程图直观，结构更清晰，其最大的优点是能够反映和描述自顶向下逐步求精的过程。首先可以描述出一个模块有几部分构成，然后再求精每一个部分是什么结构，再进一步描述每种结构的细节部分，直至求精结束完成详细设计。而程序流程图做不到这一点，我们所看到的程序流程图已是最后结果，中间的求精过程无法保存，无法描述。

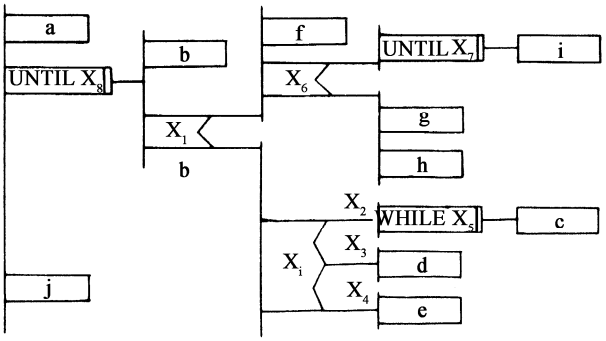


图3.30　PAD图示例

4．伪码（PDL语言）

伪码也称为程序设计语言（Program Design Language，简称为PDL），是一种非形式的比较灵活的语言，它用语言的方式描述模块内部的具体算法和加工细节。一般说来，伪码的语法规则分成外语法和内语法。外语法应当符合一般程序设计语言常用的程序语句的语法规则；而内语法是没有定义的，它可以用自然语言的一些简洁的句子、短语和通用的数学符号，来描述程序应执行的功能。

下面举一个使用PDL描述冒泡排序算法的例子。

众所周知，冒泡排序是一种分类技术，它可以对任意一组数据的序列，通过将每两个邻接数的比较，按升序或降序的要求交换这两个数或不交换这两个数，直到排序成功。最后，数据序列中的最大值或最小值数据被放置在最高序号的位置上，次大或次小值数据被放置在次高序号的位置上……

```
Read array elements
Echo （Write） unsorted array elements
K=（number of items in array）－1
DO WHILE K>=1
    Set pair pointer （J） to 1
    DO WHILE （J）<=K
        IF array element （J）>array element（J+1）
        Hold=array element （J）
        Array element （J）=array element（J+1）
        Array element （J+1）=Hold
```

```
            ENDIF
            Add 1 to （J）
        ENDDO
        Subtract 1 from K
    ENDDO
    Print sorted array
```

从上面的例子可以看出，PDL语言具有正文格式，很像一种高级语言。任何一个设计人员，均可利用所掌握的程序设计语言的控制语句结构写出PDL伪码程序。这种伪码程序在编写时可依赖于某种程序设计语言（如类PASCAL语言），但在实现时可以用任何一种计算机语言来实现。例如，用结构化程序设计中的基本控制结构编写的伪码程序，既可用PASCAL语言、C语言来实现，也可用汇编语言来实现。PDL作为一种用于描述软件详细设计的工具，具有以下特点：

（1）严格的关键字外语法，用于提供全部结构化控制结构、数据说明和模块特征。属于外语法的关键字是有限的词汇集，它们能对PDL伪码程序进行结构分割，使之变得易于理解。

（2）内语法使用自然语言描述处理特性，语法灵活，只要写清楚就行，易写易读。

（3）应有包括说明简单的与复杂的数据结构的数据说明机制。

（4）应有子程序定义与调用方法，用来表达各种方式的接口说明。

用于进行设计的PDL语言应该是与编程语言无关的。一种典型的PDL是仿照PASCAL特征的语言，其基本的PDL组成应包括：数据说明；用于构造程序块的方法；顺序结构；选择结构；循环结构；子程序定义；接口说明；输入/输出结构。

下面是把图3.26中的流程图用PDL语言描述的一个例子。

```
    ┆
execute process a
REPEAT UNTIL condition X8
    execute process b
    IF condition X1
        THEN BEGIN
            execute process f
            IF condition X6
                THEN
                    REPEAT UNTIL condition X7
                    execute process i
                ENDREP
                ELSE BEGIN
                    execute process g
                    execute process h
                END
```

```
            ENDIF
        ELSE CASE OF Xi
          WHEN condition X2   SELECT
            DO WHILE condition X5
              execute process c
            ENDCASE
          WHEN condition X3   SELECT process d
          WHEN condition X4   SELECT process e
        ENDCASE
      ENDIF
    ENDREP
    execute process j
    END
```

PDL有如下优点：

（1）由于PDL对处理的描述接近于自然语言，可以使用普通的文字编辑器完成伪码程序的编写，同时，也便于程序员理解；

（2）PDL伪码程序的总体结构与一般程序相同，这样从伪码翻译成程序语言的源程序非常方便；

（3）伪码可以作为注释直接插在源程序中间，这样做能促使维护人员在修改程序代码的同时也相应地修改PDL注释，因此有助于保护文档和程序的一致性，提高文档的质量。

PDL的缺点是不如其他的图形工具形象直观，描述复杂的条件组合与动作间的对应关系时，不够明了。

上面已经介绍了支持结构化程序设计的四种描述工具，即结构化流程图、方框图（NS图）、问题分析图（PAD图）、伪码（PDL语言）。它们各有优缺点，应根据实际情况选择使用。但是，无论使用哪一种详细设计的描述工具，所描述出的每个模块的控制结构都是准确无二义性的，均可转换为程序设计语言的语句，到编程阶段只要按照对应关系可方便地编写出源程序。

小　　结

软件设计的任务是将需求分析阶段获得的需求说明转换为计算机中可实现的系统，完成系统的结构设计，包括数据结构和程序结构，最后得到软件设计说明书。

软件设计可分为总体设计和详细设计。总体设计是进行数据结构和数据库、输入输出、模块划分和模块层次结构的设计。详细设计是进行各个模块的控制结构和控制流程的设计。在设计时，要遵循模块化准则、模块独立性准则、软件结构形态特征准则。

总体设计通常使用结构化设计方法，它与需求分析阶段的结构化分析方法配合使用，按照一组映射规则将数据流图映射为软件结构图。详细设计是根据结构化程序设计原则进行的，只使用顺序、分支、重复三种结构来设计模块的控制流程，使用的表示工具有程序流程图、方框图、PAD图、伪码（PDL语言）等。

3.5 软 件 编 程

按照软件工程方法，前面所介绍的软件生存周期的各个步骤都是为了一个共同目标：将“软件表示”变换成计算机能够“理解”的形式。本节介绍关于把“设计”变换成用程序设计语言编写的程序有关的一些问题。

3.5.1 软件编程概述

软件工程的方法论指出，软件编程是在软件进行总体设计和详细设计之后进行的工作，其任务是为每个模块编写程序，即将详细设计的结果转换成某种程序设计语言编写的程序。尽管程序的质量基本上取决于设计的质量，但是，编程使用的语言、编写程序的风格和途径对程序的质量均有相当大的影响。

编程阶段相当于机械工程中图纸设计完成后的“制造”阶段，编程阶段应交付的文档就是可在计算机中运行的程序。

在软件生存周期中，编程阶段相对来说是比较容易的一个阶段，有了详细设计的方案，又有可选择的高级语言，编程已不困难，工作量约占整个开发阶段的20%左右。

在相当长的一段时间，人们认为程序主要是给机器执行而不是供人阅读的，所以只追求程序的正确性，而忽略了程序本身是否容易阅读和理解。随着软件规模日益庞大，结构越来越复杂，人们逐步发现在软件生存周期中，程序经常需要被阅读，例如，发现隐含错误要修改时，或要扩充功能时，都需要由程序的开发者和其他人来阅读、理解和分析程序。人们意识到读程序是软件开发工作中的一个重要的组成部分，有时需要花费许多时间，因此，程序的可读性、可理解性是非常重要的。具有良好可读性的程序，容易理解、容易维护，而且隐含错误的可能性将大大降低，可靠性也就高了，因此程序的可读性对软件质量有深刻影响。

在编程阶段对详细设计说明书的任何一点误解，必然会导致出现错误的源程序，而程序设计语言的性能不仅影响软件开发人员的思想方法和对设计方案的理解，甚至对软件设计的质量和效能、数据结构的选择，以及软件的测试和维护性能也产生影响和制约。由于编程所采用的语言的特性对编码的质量和效率的影响甚大，因此正确地评价和选择编程语言对于顺利完成编程任务十分重要。下面介绍与程序设计的效能和质量有关的两个问题：程序设计语言和编程风格。

3.5.2 程序设计语言

程序编码阶段的任务是将软件的详细设计转换成用程序设计语言实现的程序，因此程序设计语言的性能和设计风格对于程序设计的质量有着直接的关系。目前用于软件开发的程序设计语言有数百种之多，从理论上来说，任何一种程序设计语言均可作为编程工具完成上述任务，但是，它们对问题的处理及解决方式却不尽相同。因此，我们不仅需要了解某种语言，而且还应该了解不同语言之间的关系。

程序设计语言可以分为机器语言、汇编语言和高级语言三类。机器语言和汇编语言是

面向机器的语言，一般来说，它们随计算机型号不同而不同。

机器语言是计算机硬件系统能够识别、执行的一组指令。指令的全体称为计算机的指令系统。指令通常分为操作码和操作数两大部分。操作码表示计算机执行什么操作，操作数表示参加操作的数本身或数所在的地址。机器语言程序不易编制也不易理解。用机器语言编写的程序对不同种类的机器没有通用性、难于交流和移植。

汇编语言是用助记符来代替操作码，用地址符号来代替地址码的符号式语言。这些助记符通常使用指令功能的英文单词的缩写（如用ADD表示“加”，SUB表示“减”，MOV表示“传送”等），这样，每条指令都有明显的特征，易于理解和记忆。汇编语言的语句基本上是与机器指令一一对应的。汇编语言通常还提供定义常数、条件汇编、宏汇编等广义指令，并允许在程序中加上注释。虽然编制汇编语言程序不必用二进制来编程，比起机器语言来说要方便，但是程序的代码量却和机器语言程序差不多。把汇编语言翻译成机器语言的程序称为汇编程序，翻译的过程称为汇编。相应地，把机器语言程序转换成汇编语言程序称为反汇编。

高级语言是在克服面向机器的语言存在的缺点的基础上，于20世纪50年代中期开始发展起来的。高级语言使用接近自然语言和数学公式的表达方式，也称为算法语言。高级语言有相应的语法，它独立于具体的机器。为了使计算机能读懂高级语言源程序，需要把源程序翻译成相应的机器语言程序。翻译程序有两种类型：一种是编译程序，另一种是解释程序。

编译程序的功能是将源程序翻译成相应的机器语言的目标程序，然后，目标程序通过连接程序被连接成可执行程序，再直接运行可执行程序得到运行结果。编译之后的目标程序和连接之后的可执行程序可以保存起来，以后可直接运行可执行程序，不必对源程序重新编译和连接。

解释程序是将送入计算机中的源程序逐条解释，逐条执行。执行后即得到运行结果，但不保存执行后的机器码，这样，下次运行时，还要重新对源程序解释执行。

显然，使用编译方式，直接运行程序比运行解释程序要快得多。

高级语言可以分为两大类：通用语言和专用语言。专用语言面向特定的应用领域，语言结构与问题密切相关。通用语言适应性强、应用面广。下面简要介绍几种通用的程序设计语言。

1．BASIC语言

BASIC是“初学者通用符号指令代码”的英文缩写，初始开发出的BASIC语言只有17条基本语句，它是一种具有会话功能，便于人机交互的语言，这种语言简单易掌握。目前BASIC语言在基本BASIC的基础上已有很大发展，扩展BASIC功能很强，具有图形功能、音乐功能等。Turbo BASIC和Visual BASIC不仅具有结构化语言的基本控制结构和能力，还为用户提供了编辑、编译、连接、运行为一体的可视化集成环境，使用户可以既快又简便地开发出在Windows环境下的各种应用程序。

2．FORTRAN语言

FORTRAN语言是一种利用公式的表达方式和英语语句的组合形式来编写源程序的语

言，又称为公式翻译语言。它是一种适用于科学和工程计算的程序设计语言。FORTRAN语言自1956年出现以来，已经进行过多次修改和补充。目前比较通用的FORTRAN77就是美国国家标准协会于1978年正式公布的新版本，它引入了字符类型和结构化成分，能方便地用于非数值计算领域。

FORTRAN语言由一个主程序和一组子程序（包括函数）组成。主程序和各个子程序是独立编译的，运行前必须把独立编译所得到的各个目标程序连接起来。

FORTRAN语言语法规定严格并作了许多限制，编译过程容易优化。因此，用FORTRAN程序算题不仅速度快，又有精度保证，适宜于大型的科学与工程计算。

全世界流行的科学和工程计算软件包中，大约90%是用FORTRAN语言编写的。许多计算方法都有可直接使用的FORTRAN子程序库。

3．COBOL语言

COBOL是“通用面向商业语言”的英文缩写。它是一种主要用于商业、金融业等数据处理的程序设计语言，适用于编写各种商务管理程序。

COBOL语言采用自然语言的程序设计风格，其书写格式接近英语表达形式。COBOL语言源程序由标识部分、环境部分、数据部分和过程部分组成。其中引入了不依赖于机器的数据描述方法，将数据的描述和数据的操作分离开来。COBOL程序结构严谨，层次分明，可读性好。

随着COBOL语言的发展，其功能不断扩大，出现各种版本。例如，现有排序和制表功能的COBOL 65，有数据通信能力的COBOL 69，有含有数据库功能的COBOL 76，有结构程序设计功能的COBOL 78及COBOL 85等。COBOL通用性强、容易移植，并提供了与事务处理有关的大范围的过程化技术。但是它缺乏简洁性。

4．PASCAL语言

PASCAL语言是一种结构化程序设计语言。它是以历史上著名的法国数学家PASCAL的名字来命名的。

PASCAL语言的程序结构由程序首部、说明部分和程序体等几部分组成，它提供了较丰富的数据类型和构造数据结构的方法。同时，它可以满足结构化程序设计的要求，用PASCAL语言编写的程序易读、易懂，并易于进行程序验证。PASCAL语言提供了一种系统的、精确的、合理的方式来表达程序设计的基本概念和结构，所以，它适合作为程序设计课程的教学语言。

目前，在微机中广为流行的是Turbo PASCAL语言，它提供了一种全新的集成开发环境，支持鼠标、多文件编辑和多重叠窗口，并配备有增强型调试工具和功能完备的嵌入式汇编器及其他高级语言的调用接口。

5．C语言

C语言是一种结构化的程序设计语言。C语言最初是作为设计操作系统的语言而研制的，UNIX操作系统就是用C语言实现的。

C语言程序具有层次化和模块化的程序结构。C语言除了具有丰富的数据类型、灵活多样的运算符、语句表达能力强等优点外，还具有能直接访问物理地址和寄存器，进行系统

调用等。由于C语言可以完成与汇编语言类似的功能，这就有利于充分发挥计算机硬件的潜在功能，提高编程效率。因此C语言已成为在系统软件和应用软件开发中广为使用的程序设计语言。目前流行的C语言版本有Turbo C和Quick C等。它们都支持ANSI C标准，支持集成环境和命令行调试。在编程过程中提供有性能优越的查错和调试工具，还具有丰富的库函数，为应用开发提供了极大的方便。

6．ADA语言

ADA语言是由美国国防部研制的。该语言力图作为一种用于嵌入式实时计算机设计的标准语言。ADA语言在结构和使用的符号方面类似于PASCAL语言，但其功能更强、更复杂。ADA语言提供一组丰富的实时特性，包括多任务处理、中断处理、任务间同步与通信，并且它号称是基于对象的语言，并提供许多ADA程序包。

7．LISP语言

LISP语言是一种符号处理语言。这种语言主要用来解决人工智能领域的问题。例如专家系统、自然语言理解、模式识别、计算机辅助学习和系统程序设计等。

在计算机应用方面一个重要的领域是非数值处理。LISP就是处理非数值数据的程序设计语言。它特别适用于组合问题中常见的符号运算和表处理。特别适用于在定理证明、树的查找以及其他一些问题求解活动中。子程序主要用递归函数来实现。因为每个LISP函数是一个独立的实体，通过生成基本函数库，就可以得到程序的可复用性。LISP在定义知识库系统时，对事实、规则和对应推理的定义比较简单、容易。

8．PROLOG语言

PROLOG是另一种广泛用于专家系统构造的程序设计语言。和LISP一样，PROLOG提供了支持知识表示的特性。它是一种陈述性语言，只描述要计算机做什么，而并不告诉计算机怎样一步步去运算。这种语言用一种称为term的统一的数据结构来构造所有的数据和程序。每一个程序都由一组代表事实、规则和推理的子句组成。LISP和PROLOG都特别适合用于处理对象及其相互关系的问题，因此有人把LISP和PROLOG称为面向对象的语言。

9．SMALLTALK语言

SMALLTALK是首先实现真正面向对象的程序设计语言之一。它引入了与传统程序设计语言根本不同的控制结构与数据结构。SMALLTALK可以定义对象，对象由数据结构和指向一组方法（服务操作）的指针组成。每个对象都是一个类的实例。因而既然新对象继承它的类的所有特性，新对象就很容易创建。此外，它提供的面向对象的方法可以支持程序部件的“可复用性”，从而使大型软件系统的开发时间和以源程序行数计算的程序量大大减少。

10．C++

C++是当今最受欢迎的面向对象的程序设计语言之一。因为它既具有面向对象的特征，又与C语言兼容，保留了C语言的许多重要特性，这就使C程序员不必放弃自己已经十分熟悉的C语言，而只需要补充学习C++提供的那些面向对象的概念。现在常用的是Turbo C++，Borland C ++，Microsoft C++等。

11．JAVA

JAVA是一种功能更强的面向对象的程序设计语言。它的基本功能类似于C++，但对C++进行扩充，可提供更多的动态解决方法。JAVA设计接近C++语言，但做了许多重大修改。它不再支持运算符重载、多继承及许多自动强制等易混淆和较少使用的特性，增加了内存空间自动垃圾收集的功能。JAVA中提供了附加的例程库，通过它们的支持，JAVA应用程序能够自由地打开和访问网络上的对象，就像在本地文件系统中一样。JAVA有建立在公共密钥技术基础上的确认技术，可以构造出无病毒、安全的系统。JAVA提供了简单、高效、强有力的实现方法。它适用于Internet环境并具有较强的交互性和实时性。它提供了网络应用的支持和多媒体的存取，推动了Internet和企业网络的Web的进步。

从上面简介的常用程序设计语言可以看出，对程序设计员来说，尽管选择语言的自由度较大，但由于每一种程序设计语言都有其自身的特点，因此，选择语言时要根据编程任务及语言的特点综合考虑。语言选择的一般准则如下：

（1）项目的应用领域。这是选择语言的关键因素。在其他条件都具备的情况下，尽量选取适合某个应用领域的语言。在科学工程领域中，FORTRAN语言仍然是一种最适用的语言。在商用数据处理领域，通常使用COBOL语言，在实时控制和系统软件中，通常使用汇编C、ADA等语言。在应用软件中，常使用C、C++、SMALLTALK等语言，在人工智能领域主要采用LISP或PROLOG语言，在教学工作中，常选用PASCAL语言。

（2）算法和计算复杂性。一般说来，商用数据处理和系统软件的算法要简单一些，而科学工程计算和模拟、实时计算机系统以及人工智能的算法要复杂得多。要根据不同语言的特点及所能提供的数据类型，选取能够适应软件项目算法和计算复杂性的语言，这与工程特点也是密切相关的。

（3）软件的执行环境。应选取机器上具有的语言处理程序及其相应支撑软件的语言。

（4）性能因素。这是结合工程具体性能考虑的准则。例如，对于要求能即时做出反映的实时系统，当然最好选取汇编语言。

（5）数据结构的复杂性。一般说来，商用数据处理系统软件的数据结构复杂，科学工程计算和模拟，实时计算机系统等数据结构较为简单。要根据不同语言构造数据结构类型的能力选取合适的语言。

（6）软件开发人员的水平。人们一般习惯于使用过去熟练使用过的语言。要选取新的语言，不仅存在软件开发人员学习和熟悉使用的问题，而且还要克服心理因素，以便从原有语言过渡到新的语言。

3.5.3 编程风格

编程风格是指一个程序员在编程时，对程序的结构形式、行文方式及编写特点的要求。编程风格是根据软件技术发展的需要提出来的，也是程序员实际工作经验的总结及素质的体现。

一篇文章只有在主题与采用的体裁吻合，内容性质与表达方式一致，文章层次段落编辑清晰的前提下才有可能称为好文章。类似地，一个高质量的程序必须具有清晰、合理的结构，简明、易懂的编码，直观、标准的格式，这样的程序才可能有好的易读性、易理解

性和易维护性。下面介绍与编程风格有关的一些问题。

一、源程序文件

源程序中各种变量如何命名，如何加注释，源程序按什么格式书写，这些对于源程序文件的编写风格有至关重要的作用。

1．符号名的命名

符号名是程序中的数据对象，符号名命名的好坏对阅读和理解程序有直接影响，因此必须予以足够的重视。各种语言都对符号名的命名作了相应的规定，在符合规定的前提下，应使其直观、易于理解而且不易出错。为此应注意以下几点：

（1）尽量用与实际相同或接近的标识符作为符号名。这样可以“见名知义”，便于理解符号名的作用。例如，用area表示面积，用time表示时间，用line表示直线，用sum表示总和等等。这样的符号名在程序中所代表的意义是很明显的，这无疑会给阅读和理解程序带来方便。

（2）注意照顾日常习惯

人们在学习和工作中已有一些共同的习惯，例如，用U代表电压，用I代表电流，用R代表电阻，用X、Y代表未知数等。在程序中尽量使用人们习惯用的符号名，这将对程序的可理解性带来好处。

（3）要注意的其他问题

符号名的表达方式应统一，所有符号名应采用统一的英语名字，或汉语拼音名字，不要采用两种语言表示的混合符号名。命名时，应注意每种语言对符号名长度的限制不要超过规定的长度，采用过长的名字，不仅容易出错，而且也增加了程序的输入量。在命名时不要采用过于相似的名字，例如，变量名ENN，ENNM，ENMN等，这样容易引起误解或输入错误。命名时的次序应该标准化，最好按照数据结构和数据类型确定说明的次序，这样便于查找。当多个符号名在一个语句中说明时，应该按字母顺序排列。

2．源程序中的注释

源程序中注释是软件开发人员与源程序的使用者之间的通讯方式之一。它也是在软件生存期的维护阶段对维护者理解程序提供的指导。目前常用的程序设计语言，大都允许使用注释行，注释可分为序言性注释和解释性注释。序言性注释是在一个程序或模块的开头对本程序段做必要的说明，它对理解程序起引导的作用。通常包括以下内容：整个模块的功能、接口信息、数据结构、开发历史、设计者、使用方法、修改情况等。解释性注释是插入在程序正文中的注释行。在程序正文中有必要做说明的地方插入注释，以说明其后的语言或程序段要做什么。这里需要注意的是，不要把注释写成对语句功能的描述，因为语句功能的解释仅仅是程序本身的重复，对于程序的阅读和理解不能起到帮助作用。注释本身应提供一些从程序本身难以得到的信息。注释应该与程序一致，所以修改程序时，应同时修改注释。

3．源程序书写格式

尽管目前有的程序设计语言（如C语言）对程序的书写格式要求很宽松，书写比较灵活，

但良好的源程序书写格式有助于对程序的阅读和理解。除了在程序中要充分利用注释行外，还应该根据程序中的各语句组或模块功能和嵌套关系采用缩进编排，使程序的模块和复合关系都变得十分明显。同时，在程序中加上适当的空格或空行（如在变量定义部分和可执行语句部分之间）可使程序更加清晰。例如：下面是一个用C语言编写的打印输出九九乘法表的程序。程序中，采用了缩进编排的方式，复合语句用一对花括号对齐表示，由两个for语句构成的二重循环嵌套关系表示得非常清楚。

```
/ * c4－7.c */
main（ ）
{
    int i，j，p；

    printf（"     *"）；
    for （i=1；i<=9；i++）
        printf（"%4d"，i）；
    printf（"＼n"）；
    for （i=1；i<=9；i++）
    {
        printf（"%4d"，i）；
        for （j=1；j<=i；j++）
        {
            p=i*j；
            printf（"%4d"，p）；
        }
        printf（"＼n"）；
    }
}
```

程序的执行结果是：

```
*   1   2   3   4   5   6   7   8   9
1   1
2   2   4
3   3   6   9
4   4   8   12  16
5   5   10  15  20  25
6   6   12  18  24  30  36
7   7   14  21  28  35  42  49
8   8   16  24  32  40  48  56  64
9   9   18  27  36  45  54  63  72  81
```

此外，在书写每个程序语句时，还应该注意：每个语句应该简单直接，不应该为了提高效率而把语句搞复杂了，应直接反映编程意图，不必过于巧妙或深奥；应尽量少用中间变量，应将计算一气呵成，这样可以防止在以后的修改中将完成一个具体计算的几个语句拆散。适当添加括号可使表达式的计算意图更清楚。

二、输入输出

输入和输出的方式是在需求分析和设计阶段就已经确定了的。系统在用户心目中的形象很大程度上取决于输入和输出的方式，从有利于程序的理解和方便用户的使用两个方面考虑，对输入和输出的风格应注意以下几点：

1．输入和输出格式应尽可能统一

为了提高程序的易读性，而且方便用户的使用，程序中输入和输出的格式必须统一。比如，数据输入时给出必要的提示信息，数据间用相同的分隔符分隔，并以相同的结束符结束数据的输入；输入数据的格式尽可能一样，使数据的总位数、小数点位数都统一。同时，在满足问题要求的前提下，输入格式应尽量简单，而且对输入的数据应提供正确性检查的方法；输出信息应该整齐、清晰、便于理解，给用户以简单、易用的良好界面。

2．输出信息中应该反映输入的数据

程序运行时，用户输入的数据直接决定了程序运行的结果，不同的数据输入对应着不同的输出结果。为了验证程序运行结果的正确性，常常把数据输入情况和运行结果都反映在输出信息中。这是检查程序运行正确性和用户输入数据正确性的需要。否则，无法根据输入的数据核对输出的结果，因而也无法断定程序的运行结果正确与否，也无从检验输入数据是否有误，当然也就不可能判断程序的有效性与可靠性。

3．输入和输出应尽可能集中安排

程序中的输入输出是变化和修改比较多的部分，在程序中设计输入输出时应充分考虑数据的修改和更新的方便。比如，对应不同组的数据要得到不同的结果，这时应该允许用户选定输入数据；当应用场合和应用条件改变时，需要改变数据输入，这时可能要对程序中的输入输出部分进行修改。为修改或扩充的方便，应该把输入输出代码集中在最少的程序段之中，比如，集中放在主程序中，或输入输出模块中。这样使得由于输入条件变化或处理要求不同而引起的修改量最少，从而为程序的修改和扩充带来方便。

小　　结

软件编程的任务是将软件详细设计的结果转换成某种程序设计语言编写的源程序。

编程是软件生存周期中较容易实现的阶段。有了详细设计的方案及各模块，又有可选择的高级语言，可以将各种控制结构与程序设计语言的相关语句一一对应起来，这样的转换工作就变得较为容易了。

编程必须注意编程风格。编程风格是指有关源程序的书写格式、各种名字的命名方法、语句的构造、数据的说明、输入输出的结构等问题。编程风格的原则是简明性和清晰性。

结构化程序设计方法是一种面向数据和过程的设计方法，设计中数据和过程分离为相互独立的两部分，数据代表问题空间中的实体，过程体现了处理这些数据的算法。

3.6 面向对象的分析和设计

面向对象方法是一种把面向对象的思想应用于软件开发过程中，指导开发活动的系统方法，简称OO方法。开发方法的不同其根本区别在于如何看待一个系统。如前所述，结构化分析方法把系统看作一系列的功能节点，节点间的联系通过数据流来实现，而功能节点本身的描述则可以采用功能节点细化和控制逻辑描述实现。结构化方法分析的结果是数据流图和数据字典，设计的结果是模块结构图，实现的结果是由程序模块组成的源程序。这些图中的成分或程序模块不能直接映射到客观世界中系统的实体上，也就是说，解空间的结构与问题空间的结构是不一致的。当用户需求有一些小的改变时，这种不一致性将导致分析、设计的较大变化。而面向对象方法则以客观世界中系统的实体为基础，将客观实体的属性及其操作封装成对象。在分析阶段，识别系统中的对象以及它们之间的关系；在设计阶段，仍沿用分析的结果，并根据实现的需要增加、删除或合并某些对象，或在某些对象中添加相关的属性和操作，同时设计实现这些操作的方法；在实现阶段，则用程序设计语言来描述这些对象以及它们之间的联系。因此，面向对象方法的分析、设计、实现的结果能直接映射到客观世界中系统的实体上，也就是说，解空间的结构与问题空间的结构是一致的。分析、设计、实现一个系统的过程与认识这个问题的过程是一致的。由于面向对象的分析和设计采用同样的图形表示形式，分析、设计和实现都以对象为基础，因此面向对象开发的各阶段之间具有很好的无缝连接。当用户的需求有所改变时，由于客观世界中的实体是不变的，实体之间的联系也是基本不变的，因此面向对象的总体结构也相对比较稳定，所引起的变化大多集中在对象的属性与操作及对象之间的消息通信上。总之，面向对象方法符合人们对客观世界的认识规律，所开发的系统相对比较稳定。

面向对象方法开发的软件系统由对象类组成，对象的封装性很好地体现了抽象和信息隐蔽的特征。对象以属性及操作为接口（界面），使用者只可通过接口访问对象（请求其服务），对象的具体实现细节对外是不可见的。这些特征使得软件系统的体系结构是模块化的，这种体系结构易于理解、扩充和修改。当对象的接口确定以后，实现细节的修改不会影响其他对象，易于维护。同时也便于分配给不同的开发人员去实现，依据规定的接口能方便地组装成系统。

在同一应用领域的不同应用系统中，往往会涉及许多相同或相似的实体，这些实体在不同的应用系统中存在许多相同的属性和操作，也存在一些不同的应用系统所特有的属性和操作。在开发一个新的软件系统时，可复用已有系统中的某些类，通过继承和补充形成新系统的类。在同一个应用系统中，某些类之间也存在一些公共的属性和操作，也含有它们各自私有的属性和操作。这也可以通过继承来复用公共的属性和操作。

面向对象软件开发方法又称为OOSD（Object-Oriented Software Development）。OOSD包括面向对象分析（OOA）、面向对象设计（OOD）和面向对象程序设计（OOP）三个方面。其中OOP是基础，OOA和OOD是应用OOP的机制。随着人们对面向对象技术的广泛应用，面向对象的概念已渗透到软件的各个领域。例如，面向对象数据库，面向对象图形处理，面向对象操作系统，面向对象分析技术，神经元网络中的面向对象技术等。面向对象

技术被誉为20世纪90年代软件的核心技术之一。

3.6.1 面向对象的基本概念

一、对象（object）

对象是客观世界事物的表示或描述，世界上任何有形或无形的事物都可称为对象。每个对象都有它的属性和操作，如电视机有颜色、音量、亮度、灰度、频道等属性，可以有切换频道、增大/减小音量等操作。电视机的属性值表示了电视机所处的状态，而这些属性值只能通过其提供的操作来改变。电视机的各组成部分，如显像管、印制板、开关等都封装在电视机机箱中，人们不知道也不关心电视机是如何实现这些操作的。

在面向对象的系统中，对象是指一组属性以及这组属性上的专有操作的封装体。属性通常是一些数据，有时它也可以是另一个对象。例如，书是一个对象，它的属性可以有书名、作者、出版社、出版年份、定价等属性，其中书名、出版年份、定价是数据，作者和出版社可以是对象，他们还可以有自己的属性（在有些简单的软件系统中可能只用到作者名和出版社名，而不关心作者和出版社的其他信息，那么，他们也可以是数据）。每个对象都有它自己的属性值，表示该对象的状态。对象中的属性只能通过该对象所提供的操作来存取或修改。操作也称为方法或服务，它规定了对象的行为，表示对象所能提供的服务。封装是一种信息隐蔽技术，用户只能看见对象封装界面上的信息，对象的内部实现对用户是隐蔽的。封装的目的是使对象的使用者和生产者分离，使对象的定义和实现分开。一个对象通常可由对象名、属性和操作三部分组成。面向对象的基本思想就是把要构造的系统表示为对象的集合。

二、类（class）

类是一组具有相同属性和相同操作的对象的集合。一个类中的每个对象都是这个类的一个实例（instance）。例如“学生”是一个类，“学生”类的实例“张三同学”、“李四同学”都是对象。也就是说，对象是客观世界中的实体，而类是同一类实体的抽象描述。在分析和设计时，我们通常把注意力集中在类上，而不是具体的对象上。我们也不必为每个对象逐个定义，只需对类作出定义，而对类的属性的不同赋值即可得到该类的对象实例，如图3.31所示。类和对象之间的关系类似于程序设计语言中的类型（type）和变量（variable）之间的关系。通常把一个类和这个类的所有对象称为类及对象或称为对象类。

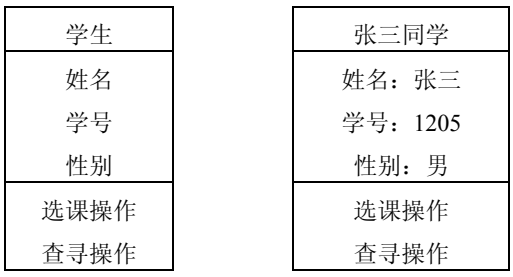


图3.31 类及实例

三、消息（Message）

客观世界中的实体之间存在着大量的交互行为。交互行为的实现有赖于实体之间的通讯联系。如果类之间不能够相互通信，那么类就无法在一起有效地描述问题域，特别是无法描述动态行为。消息是类（对象）间实现通信的手段。

前面指出操作是类与外界的接口，那么外界就是通过消息来申请类提供的服务，因此消息通常会标明要求服务的名称和相应的参数等。类收到消息请求后先分析请求的合法性和消息的合法性，然后由类为请求者服务，具体如何服务则封装在类的内部，由类自行决定，不为外界所知。

发送消息的对象和接收消息的对象之间形成了消息连接。消息连接表明一个对象（消息发送者）的处理依赖关系，它指出对象的某个操作为了履行其职责而需要其他对象的服务。

四、继承

在客观世界中，存在着整体和部分的关系，一般和特殊的关系，继承表达了对象类之间具有的共同性和差异性的关系。用一个例子作比喻，例如，昆虫可以分成有翅类和无翅类两类，有翅类下有蛾、苍蝇、蝴蝶等种类。在试图对一些新的动物或对象进行分类以前，人们关心的问题是：它与其他一般的类有多少相似之处？差别有多大？每一个不同的类都由一组描述它的行为和特征组成，最高层是最普遍的，问题也最简单，每一层都比它之前的层更具体。一旦某个特征定义下来，所有在它之下的种类都要包含该特征。所以，一旦确定苍蝇为昆虫有翅类中的一种，我们就不必指出苍蝇是昆虫，有一对翅膀，因为苍蝇从它的目中继承了这个特征。

在面向对象语言中，类功能支持这种层次结构。存在继承关系的类之间形成父类（基类）和子类（也称为派生类）。如果一个类只有一个父类称为单继承，否则称为多继承。一个子类可以从它的基类那里继承所有的数据和操作，并扩充自己的特殊数据和操作。基类抽象出共同特征，子类表达其差别。有了类的层次结构和继承性，不同对象的共同性质只需定义一次，用户就可以充分利用已有的类，从基类派生一个新类来裁剪、扩充基类，这种方法满足了不断变化的具体应用的要求，达到代码重用和易维护可扩充的目的。继承是实现递增式程序设计的有效途径，这就是引入继承的目的所在。

继承性是面向对象程序设计语言不同于其他语言的最主要的特点。

五、聚合（Aggregation）

人们认识世界有时以分解的方式进行，即先抓住总的框架，再分析它的细节组成；也可以按照组合的方式进行，即先明确单元个体的特性，再分析认识单元个体的组合体。这就要求一方面类要具有被分解的能力（实质上是类的结构可分解能力），另一方面类要具有组合的能力。聚合这种机制就使类具有这两种能力。

聚合是一种描述类之间的整体与部分组成关系的机制，它使得面向对象可以基于分解的方式来分析问题域，并以基于组合的方式来实现问题域的建模。如前所述，书类和作者

类及出版社类之间的关系就是整体与部分的组成关系。即书类中包括有作者类和出版社类的对象。

六、多态性（Polymorphism）

多态性是指对象的某个行为具有多种形态的特性。行为的多种形态封装在对象内部，外界并不知道。一个行为的多种形态从外界看来具有相同的行为名称（也即相同的消息名），因此外界看到的只是一种行为。具体应该执行哪种形态由对象自己根据接收到消息里的相关参数决定。

由此可见，多态性是指同一个操作作用于不同的对象上可以有不同的解释，并产生不同的执行结果。例如，在图形系统中的“画图”操作，作用在“矩形”对象上，则在屏幕上画一个矩形，作用在“圆”对象上，则在屏幕上画一个圆。也就是说，相同操作的消息发送给不同的对象时，每个对象将根据自己所属类中定义的这个操作去执行，从而产生不同的结果。

七、面向对象

常用的“面向对象”这个词语，是建立在上述概念之上的。Peter Coad和Edward Yourdon认为：

面向对象＝对象（object）
＋ 分类（classification）
＋ 继承（inheritance）
＋ 通过消息的通信（communication with messages）

这个等式可以解释为只有应用了对象（属性及服务、行为的封装）去对问题空间的实例进行抽象，并应用分类技术来管理对象，应用继承机制管理对象之间的相似性，并且通过对象间的消息连接来实现对象间通信的方法可称为面向对象方法。因此，可以认为面向对象方法的核心概念就是对象、分类、继承和消息连接。

3.6.2 面向对象分析（Object-Oriented Analysis，OOA）

一、OOA的目标和步骤

面向对象分析的目标是完成对所解问题的分析，确定待建的系统要做什么，并建立系统的模型。为达到这一目标，必须完成以下任务：

（1）在客户和软件工程师之间沟通基本的用户需求。
（2）标识类（包括定义其属性和操作）。
（3）刻画类的层次结构。
（4）表示类（对象）之间的关系。
（5）为对象行为建模。
（6）递进地重复任务（1）至任务（5），直至完成建模。

其中任务（2）至任务（4）刻画了待建系统的静态结构，任务（5）刻画了系统的动态行为。面向对象分析的一般步骤如下：

（1）获取客户对系统的需求：包括标识场景（scenario）和用例（use case），以及建造需求模型。

（2）用基本的需求为指南来选择类和对象（包括属性和操作）。

（3）定义类的结构和层次。

（4）建造对象-关系模型。

（5）建造对象-行为模型。

（6）利用用例 / 场景来复审分析模型。

二、分析过程

通常OOA可以从理解系统的使用方式开始，如果系统是人机交互的，则考虑被人使用的方式；如果系统涉及过程控制，则考虑被机器使用的方式；如果系统协同并控制应用程序，则考虑应用程序的使用方式。

1．获取客户对系统的需求

需求获取必须让客户与开发者充分地交流，这里介绍一种采用用例（use case）来收集客户需求的技术。分析员首先标识使用该系统的不同的行为者（actor），这些行为者代表使用该系统的不同的角色。每个行为者可以叙述他如何使用系统，或者说他需要系统提供什么功能。行为者提出的每一个使用场景（或功能）都是系统的一个用例的实例，一个用例描述了系统的一种用法（或一个功能），所有行为者提出的所有用例构成系统的完整的需求。

注意，行为者与用户是不同的两个概念，一个用户可以扮演几个角色（行为者），一个行为者可以是用户，也可以是其他系统（应用程序或设备）。得到的用例必须进行复审，以使需求完整。

2．标识类和对象

在确定了系统的所有用例后，即可开始标识类以及类的属性和操作。

这里介绍一种类-责任-协作者（class-responsibility-collaborator，简称CRC）技术。CRC实际上是一组表示类的索引卡片，每张卡片分成三部分，它们分别描述类名、类的责任和类的协作者，如图3.32所示。

类名：	
类的类型：（如：设备，角色，场所……）	
类的特征：（如：有形的，原子的，并发的……）	
责任：	协作者：

图3.32　CRC卡片

（1）类

一组具有相同属性和操作的对象可以定义成一个类，因此标识类和标识对象是一致的。标识系统中的对象可以从问题陈述或用例描述着手，通常陈述中的名词或名词短语是可能

的对象，它们以不同的形式展示出来，如：

1）外部实体（如其他系统、设备、人员），他们生产或消费计算机系统所使用的信息；

2）事务（如报告、显示、信函、信号），它们是问题信息域的一部分；

3）发生的事件或事情（如完成一系列遥控动作），它们出现在系统运行的环境中；

4）角色（如管理者、工程师、销售员），他们由与系统交互的人扮演；

5）组织单位（如部门、小组、小队），他们与一个应用有关；

6）场所（如制造场所、装卸码头），它们建立问题和系统所有功能的环境；

7）构造物（如四轮交通工具、计算机），它们定义一类对象，或者定义对象的相关类。

通过上述分析，我们可以得到一些潜在的对象，但并非所有的潜在对象都会成为系统最终的对象。我们可以用以下选择特征来确定最终的对象：

1）保留的信息：仅当必须记住有关潜在对象的信息，系统才能运作时，则该潜在对象在分析阶段是有用的；

2）需要的服务：潜在对象必须拥有一组可标识的操作，它们可以按某种方式修改对象属性的值；

3）多个属性：在分析阶段，关注点应该是“较大的”信息（仅具有单个属性的对象在设计时可能有用，但在分析阶段，最好把它表示为另一对象的属性）；

4）公共属性：可以为潜在的对象定义一组属性，这些属性适用于该对象类所有的实例；

5）公共操作：可以为潜在的对象定义一组操作，这些操作适用于该对象类所有的实例；

6）必要的需求：出现在问题空间中的外部实体以及对系统的任何解决方案的实施都是必要的生产或消费信息，它们几乎总是定义为需求模型中的对象。

对象和类还可以按以下特征进行分类：

1）有形性（tangibility）：类表示了有形的事物（如键盘或传感器），还是表示了抽象的信息（如预期的结果）？

2）包含性（inclusiveness）：类是原子的（即不包含任何其他类），还是聚合的（至少包含一个嵌套的对象）？

3）顺序性（sequentiality）：类是并发的（即拥有自己的控制线程），还是顺序的（被外部的资源控制）？

4）持久性（persistence）：类是短暂的（即它在程序运行期间被创建和删除）、临时的（它在程序运行期间被创建，在程序终止时被删除），还是永久的（它存放在数据库中）？

5）完整性（integrity）：类是易被侵害的（即它不防卫其资源受外界的影响），还是受保护的（该类强制控制对其资源的访问）。

基于上述分类，CRC卡的内容可以扩充，以包含类的类型和特征。

（2）责任

责任是与类相关的属性和操作。简言之，责任是类所知道的或要做的任何事情。

标识属性：从本质上讲，属性定义了对象，阐明了问题空间中的这个对象意味着什么。属性表示类的稳定特征，即为了完成客户规定的目标所必须保存的类的信息，一般可以从问题陈述中提取出或通过对类的理解而辨识出属性。分析员可以再次研究问题陈述，选择

那些应属于该对象的内容，同时对每个对象回答下列问题：“在当前的问题范围内，什么数据项（复合的和/或基本的）完整地定义了该对象？”

定义操作：操作定义了对象的行为并以某种方式修改对象的属性值。操作可以通过对系统的过程叙述的分析提取出来，通常叙述中的动词可作为候选的操作。类所选择的每个操作展示了类的某种行为。

操作大体可分为三类：

1）以某种方式操纵数据的操作（如增加、删除、重新格式化、选择）；

2）完成某种计算的操作；

3）为控制事件的发生而监控对象的操作。

（3）协作者

一个类可以用它自己的操作去操纵它自己的属性，从而完成某一特定的责任，一个类也可和其他类协作来完成某个责任。如果一个对象为了完成某个责任需要向其他对象发送消息，则我们说该对象和另一对象协作。协作实际上标识了类间的关系。

一个类的协作可以通过确定该类是否能自己完成每个责任来标识，如果不能，则它需要和另一个类交互，从而可标识一个协作。

为了帮助标识协作者，可以检索类间的类属关系。如果两个类具有整体与部分关系（一个对象是另一个对象的一部分），或者一个类必须从另一个类获取信息，或者一个类依赖于（depends-upon）另一个类，则它们间往往有协作关系。

（4）复审CRC卡

在填好所有CRC卡后，应对它进行复审。复审应由客户和软件分析员参加，复审方法如下：

1）参加复审的人，每人拿CRC卡片的一个子集。注意，有协作关系的卡片要分开，即没有一个人持有两张有协作关系的卡片。

2）将所有用例/场景分类。

3）复审负责人仔细阅读用例，当读到一个命名的对象时，将令牌传送给持有对应类的卡片的人员。

4）收到令牌的类卡片持有者要描述卡片上记录的责任，复审小组将确定该类的一个或多个责任是否满足用例的需求。当某个责任需要协作时，将令牌传给协作者，并重复4）。

5）如果卡片上的责任和协作不能适应用例，则需对卡片进行修改，这可能导致定义新类，或在现有的卡片上刻画新的或修正的责任及协作者。

这种做法持续至所有的用例都被复审完成为止。

3．定义类的结构和层次

在确定了系统的类后，就可定义类的结构和层次。类的结构主要有两种：一般-特殊（generalization-specialization）结构和整体-部分（whole-part）结构。

一般-特殊结构反映了类间的一般与特殊的关系。例如，交通工具可分成汽车、轮船、飞机。“交通工具”类就是一个一般类，“汽车”、“轮船”、“飞机”等类就是特殊类，一般类与特殊类之间是一种“is a”的关系，如：汽车是一种交通工具。同样，特殊类还可

以分为更特殊的类，如：“汽车”类还可分成“轿车”、“货车”等类。这样可形成类的层次结构。

整体-部分结构反映了类间的整体与部分关系。例如“汽车”代表整体，而“车轮”、“发动机”、“底盘”等都是“汽车”的一部分。值得注意的是，整体-部分关系是对对象而言的，而不是对类的。整体-部分关系是一种“has a”的关系，如“汽车”有“发动机”。同样，整体-部分结构也具有层次结构。

有的面向对象方法中，把互相协作以完成一组紧密结合在一起的责任的类的集合定义为主题（subject）或子系统（subsystem）。主题和子系统都是一种抽象，从外界观察系统时，主题或子系统可看作黑盒，它有自己的一组责任和协作者，观察者不必关心其细节。观察一个主题或子系统的内部时，观察者可以把注意力集中在系统的某一个方面。因此，主题或子系统实际上是系统更高抽象层次上的一种描述。

三、建造对象-关系模型

对象-关系模型描述了系统的静态结构，它指出了类间的关系（relationship）。类间的关系有多种。利用CRC卡的建模步骤如下：

1）利用CRC卡片画出协作者对象的网络图。在一个类的CRC卡中，如果该类的某个责任需要另一个类协作时，这两个类之间用线连接。

2）复审CRC卡片，评估责任和协作者，对连接线命名，以表示是什么样的关系。

3）对连线的每一端评估，以确定基数（cardinality）。基数表示它们之间关联的数目（如1对1、1对多、多对多等）。

四、建立对象-行为模型

对象-行为模型描述了系统的动态行为，它们指明系统如何响应外部的事件或激励。建模的步骤如下：

1）评估所有的用例，以完成理解系统中交互的序列。

2）标识驱动交互序列的事件，理解这些事件如何和特定的对象相关联。

3）为用例创建事件轨迹的活动图。

4）为系统建造状态图。

5）复审对象-行为模型，以验证准确性和一致性。

3.6.3 面向对象设计（Object-Oriented Design，OOD）

面向对象设计是将OOA所创建的分析模型转化为设计模型。与传统的开发方法不同，OOD和OOA采用相同的符号表示，OOD和OOA没有明显的分界线，它们往往反复迭代地进行。在OOA时，主要考虑系统做什么，而不关心系统如何实现。在OOD时，主要解决系统如何做，因此需要在OOA的模型中为系统的实现补充一些新的类，或在原有类中补充一些属性和操作。OOD时应能从类中导出对象，以及这些对象如何互相关联，还要描述对象间的关系、行为以及对象间的通信如何实现。

OOD同样应遵循抽象、信息隐蔽、功能独立、模块化等设计准则。

一、OOD的步骤

面向对象设计的一般步骤如下：

（1）系统设计

1）将子系统分配到处理器；

2）选择实现数据管理、界面支持和任务管理的设计策略；

3）为系统设计合适的控制机制；

4）复审并考虑权衡。

（2）对象设计

1）在过程级别设计每个操作；

2）定义内部类；

3）为类属性设计内部数据结构。

（3）消息设计

使用对象间的协作和对象-关系模型，设计消息模型。

（4）复审

复审设计模型，并在需要时迭代。

二、系统设计

本节讨论与系统设计有关的活动。

1．将分析模型划分成子系统

在OOD系统设计时，我们把分析模型中紧密相关的类、关系等设计元素包装成子系统，这样，分析模型被划分成若干个子系统。

通常，子系统的所有元素共享某些公共的性质，它们可能都涉及完成相同的功能；它们可能驻留在相同的产品硬件中；或者它们可能管理相同的类和资源。子系统由它们的责任所刻画，即一个子系统可以通过它提供的服务来标识。在OOD中，这种服务是完成特定功能的一组操作。

子系统的设计准则是：

1）子系统应具有定义良好的接口，通过接口和系统的其他部分通信；

2）除了少数的“通信类”外，子系统中的类应只和该子系统中的其他类协作；

3）子系统的数量不宜太多；

4）可以在子系统内部再次划分，以降低复杂度。

我们可以用类似于数据流图（DFD）的图来描述子系统之间的通信和信息流，此时，DFD中的每个加工（process）表示一个子系统。

2．标识问题本身的并发性，并为子系统分配处理器

通过对对象-行为模型的分析，可发现系统的并发性。如果对象（或子系统）不是同时活动的，则它们不需并发处理，此时这些对象（或子系统）可以在同一个处理器上实现；反之，如果对象（或子系统）必须对一些事件同时异步地动作，则它们被视为并发的，此时，可以将并发的子系统分别分配到不同的处理器，或者分配在同一个处理器，而由操作

系统提供并发支持。

3．任务管理设计

Coad和Yourdon提出如下管理并发任务对象的设计策略：

1）确定任务的类型；

2）定义协调者任务和关联的对象；

3）将协调者任务和其他任务集成。

通常可通过了解任务是如何被启动的来确定任务的类型，如事件驱动任务、时钟驱动任务。每个任务应该定义其优先级，并识别关键任务。当有多个任务时还可以考虑增加一个协调者任务，以控制这些任务协同工作。

4．数据管理设计

通常数据管理设计成层次模式，其目的是将操纵数据结构的低层需求和处理系统熟悉的高层需求加以分离。

数据管理的设计包括设计系统中各种数据对象的存储方式（如内存数据结构、文件、数据库），以及设计相应的服务，即为要储存的对象增加所需的属性和操作。

5．资源管理设计

OO系统可利用一系列不同的资源（如磁盘驱动器、处理器、通信线路等外部实体或数据库、对象等抽象资源），很多情况下，子系统同时竞争这些资源，因此要设计一套控制机制和安全机制，以控制对资源的访问，避免对资源使用的冲突。

6．人-机界面设计

对大多数应用系统而言，人-机界面本身是一个非常重要的子系统。人-机界面主要强调人如何命令系统，以及系统如何向人提交信息。它包括窗口、选单、报告的设计。

7．子系统间的通信

子系统之间可以通过建立客户/服务器连接进行通信，也可以通过端对端（peer to peer）连接进行通信。我们必须确定子系统间的合约（contract），合约提供了一个子系统和另一个子系统交互的方式。

确定合约的设计步骤如下：

1）列出可以被该子系统的协作者提出的每个请求，按子系统组织这些请求，并把它们定义到一个或多个适当的合约中，务必要标记那些从父类中继承的合约；

2）对每个合约标记操作（继承的和私有的），这些操作被请求以实现被该合约蕴含的责任，务必将操作和子系统内的特定类相关联；

3）每个合约应包含：合约的类型（客户机 / 服务器或端对端），协作者（合约伙伴的子系统名），类（子系统中支持合约蕴含服务的类名），消息格式（实现协作者间交互所需的消息格式）；

4）如果子系统间的交互模式比较复杂，还可以建立子系统协作图。

三、对象设计

对象设计是为每个类的属性和操作做出详细的设计，并设计连接类与它的协作者之间

的消息规约（specification of the messages）。

1．对象描述

对象的设计描述可以采取以下形式之一：

1）协议描述：描述对象的接口，即定义对象可以接收的消息以及当对象接收到消息后完成的相关操作；

2）实现描述：描述传送给对象的消息所蕴含的每个操作的实现细节，实现细节包括有关对象私有部分的信息，即关于描述对象属性的数据结构的内部细节和描述操作的过程细节。

对对象的使用者来说，只需要协议描述就够了。

2．设计算法和数据结构

为对象中的属性和操作设计数据结构和实现算法。

四、设计模式

在许多面向对象系统中，存在一些类和通信对象的重复出现的模式。这些模式求解特定的设计问题，使面向对象设计更灵活，并最终可复用。这些模式帮助设计者复用以前成功的设计，设计者可以把这些模式应用到新的设计中。

一个设计模式或设计构件通常可用四个信息来描述：

1）模式或构件名。设计模式或构件名应具有实际的含义，它能反映模式或构件的适用性和意图。

2）应用该模式或构件时必须存在的环境和条件。

3）设计模式或构件的特征。模式或构件特征指出一些设计的属性，调整这些属性使该模式或构件能适应各种不同的问题。这些属性表示设计的特征，这些特征能被用于检索（通过数据库）以找到合适的模式或构件。

4）应用设计模式或构件的结果（consequences）。对于一个设计模式或构件的使用结果表明设计决策的走向。

3.6.4 统一建模语言（UML）简介

如前所述，面向对象方法是一种把面向对象的思想应用于软件开发过程中，指导开发活动的系统方法，它是建立在对象概念（对象、类和继承）基础上的方法。20世纪60年代后期出现了面向对象的编程语言。自80年代中期起，人们注重于面向对象分析和设计的研究，逐步形成了面向对象方法学。典型的方法有P.Coad、E.Yourdon的面向对象分析（OOA）和面向对象设计（OOD），G.Booch的面向对象开发方法，J.Rumbaugh等人提出的对象建模技术（OMT），Jacobson的面向对象软件工程（OOSE）等。90年代中期，由G.Booch、J.Rumbaugh、Jacobson等人发起，在Booch方法、OMT方法和OOSE方法的基础上推出了统一的建模语言（UML），1997年被国际对象管理组织（OMG）确定作为标准的建模语言。UML作为面向对象方法的重要工具，它的出现很快受到计算机软件界的青睐。

一、UML的主要内容

作为一种建模语言，UML的定义包括语义和语法两部分。UML的语义描述基于UML提供的精确的元模型（Meta —model）的定义。元模型是定义和构造UML模型的必要手段和描述方法，它为UML的所有元素在语法和语义上提供了简单一致、通用的定义性说明。这使得开发者们在语义上取得了一致，消除了因人而异的最佳表达方法所造成的影响。此外，UML还提供了扩展机制，使得人们可以扩展UML的元模型。UML语法定义了UML的概念、元素、符号表示法以及用法，这为开发者或开发工具使用这些图形符号和文本语法进行建模提供了标准和规范。为了使用图表方式更方便地描述面向对象系统，UML共提供了五类图共九种图形，下面逐一加以介绍。

第一类图是用例图（Usecase diagram）。用例图从用户角度描述系统功能，并指出各功能的操作者。用例这一概念直接来自于OOSE中的用例。在UML中，一个用例图由一系列角色（Actors）和用例（Usecase）组成，其中用例被画在一个框内，该框表示系统边界。

第二类是静态图，包括类图、对象图和包图。其中类图描述系统中类的静态结构，包括类的定义和类之间的关系（关联、继承、聚合等）。类图描述的是一种静态关系，在系统的整个生命周期都有效。对象是类的实例，对象图与类图几乎具有完全相同的标识，但是对象图显示类的多个实例。由于对象存在一定的寿命期，因此对象图只能在系统某一时间段内存在。包图用于描述系统的分层结构，用来管理系统规模复杂性。包可由包和类组成，也可以定义包与包之间的关系。

第三类图是行为图，它描述系统的动态模型和组成对象间的交互关系，包括类的状态图和活动图。其中状态图描述类的对象的所有可能状态以及事件发生时状态的转移条件。通常我们可以把状态图作为类图的补充，但实际上我们并不需要对每个类都画出它的状态图，而仅关注那些具有多个状态，其行为受外界环境的影响后会发生改变的类。我们为这些类建立状态图以更好地描述它们的动态特性。活动图描述满足用例要求所要进行的活动以及活动间的约束关系，活动图可用来识别并发活动。

第四类图是交互图，它描述对象间的交互关系，包括顺序图（Sequence diagram）和合作图（Collaboration diagram）。其中顺序图显示对象间的动态协作关系，合作图也描述对象间的动态协作关系，但除表示信息交换外，合作图还显示对象以及它们之间的关系。合作图与顺序图相比较，合作图强调对象间的连接关系，而顺序图强调对象间交互动作的时序特性。

第五类是实现图（Implementation diagram），包括构件图（Component diagram）和配置图（Deployment diagram）。其中构件图用来描述代码构件的物理结构和各构件间的依赖关系。一个构件可以是一个资源代码构件或一个二进制构件或者是一个可执行构件。构件包含类的逻辑和实现方面的有关信息。构件图有助于分析和理解构件之间的相互影响。配置图定义系统中软、硬件的体系结构，它可以描述实际的计算机和设备以及它们之间的连接关系，也可显示连接的类型及构件之间的依赖性。

以上这些图极大地丰富了UML描述问题的能力。UML的描述手段（机制）中的用例图、类图、包图、对象图、构件图和配置图用来建立系统的静态模型，属于静态建模机制；状

态图、活动图、顺序图和合作图用来建立系统的动态模型，属于动态建模机制。

二、UML的主要特点

UML的主要特点可以用以下三点来概括：

（1）它统一了Booch、OMT和OOSE等方法中的概念；

（2）它吸取了面向对象技术领域中其他流派的长处；

（3）它在演变过程中还提出了一些新的概念。

如前所述，UML是在Booch、OMT和OOSE等三种方法的基础上发展起来的，是这些方法的延续。因此关于使用Booch、OMT和OOSE方法所积累的经验绝大部分都可以保存下来在UML过程中应用。但是UML与这三种方法（以及其他的一些方法）相比，除具有表达能力更强和表达更清晰、一致的特点外，它还可以在更广泛的范围内建模，并且清除了不同方法在表示法和术语上的差异。

UML不仅统一了Booch、OMT和OOSE中的概念，同时还引入了一些其他方法中的可取之处。UML在演变过程中还提出了一些新的概念，如：模板、扩展机制、线程与进程、并发和开发模式等。

三、UML应用领域

UML的目标是以面向对象的思想来描述任何类型的系统，因而具有很宽的应用领域。人们常用它来建立信息系统（包括软件系统）的模型，但它还可以用于描述不带任何软件的机械系统、一个企业的机构或企业过程等。如处理复杂数据的信息系统，具有实时要求的工艺系统或工业过程、嵌入式实时系统、分布式系统、系统软件和商业系统等。总之，UML是一个通用的建模语言，可以对任何系统进行分析和建立相应的模型。

UML本身是一个完整的建模语言，因此支持系统开发的不同阶段，从需求描述到系统的测试。在需求分析阶段，UML通过用例模型来捕获用户需求。通过用例图，UML描述对系统感兴趣的外部角色和他们对系统的功能要求。分析阶段主要关心问题域的概念和实体，并得到与问题域直接相关的类和对象，以及它们之间的关系。在系统设计阶段，用户需要定义一些与技术实现相关的类，如用户接口、数据库、通信和并行等问题，UML提供了强大的静态和动态建模机制。在编码阶段，开发人员应用一种语言（最好是支持面向对象的语言）将设计得到的类转换成实际的代码，这时需要把某些UML所支持的概念合适地转化到特定的实现语言中。

同其他的开发方法一样，应用UML进行开发过程中产生的文档（文字和图示文档）是软件测试的重要依据。在单元测试阶段，测试人员主要依据类图和类的规格说明；在集成测试阶段，测试人员主要依据构件图和合作图；在系统测试阶段，测试人员主要依据用例图来验证系统的行为。有关测试的内容将在3.7节软件测试中作较详细的介绍。

总之，UML适用于任何类型的系统，其丰富的描述能力可以精确地描述系统的静态结构和动态行为。同时，UML本身并不限制使用某一种具体的开发过程，但它支持从需求、设计到测试和维护的全生命周期。

四、UML面向对象分析和设计的一般过程

面向对象分析的首要任务是识别用户需求，并用合适的模型工具把识别得到的用户需求记录下来。不仅如此，面向对象分析还要对这些需求进行分析从而得到分析模型。UML提供了如此多的模型元素，实际项目中我们可能并不需要使用其中的全部模型元素。同时由于这些模型元素本身存在内在的逻辑关系，因此我们首先应该掌握UML面向对象分析的一般过程，即分析的工作流描述，这样我们才能做到在实际项目中对UML的模型元素有所取舍。

一般来说，首先我们需要了解、熟悉用户业务的领域问题（正如前面我们对于需求分析过程的讨论中所提到的那样）。通常这时我们需要和用户进行广泛而深入的交流，了解他们真正想要的东西是什么。抓住问题域的本质对于分析来说是最重要的一件任务，如果从一开始就错误地理解了用户的需求，那么最终开发出来的软件必将无法让用户满意。用户需求的捕获可以通过由用户写的一段介绍性的材料、分析人员对用户提问的回答内容等。这个时候用户需求的描述不拘一格，可以是纯文字描述，可以是表格，也可以是一系列图形。

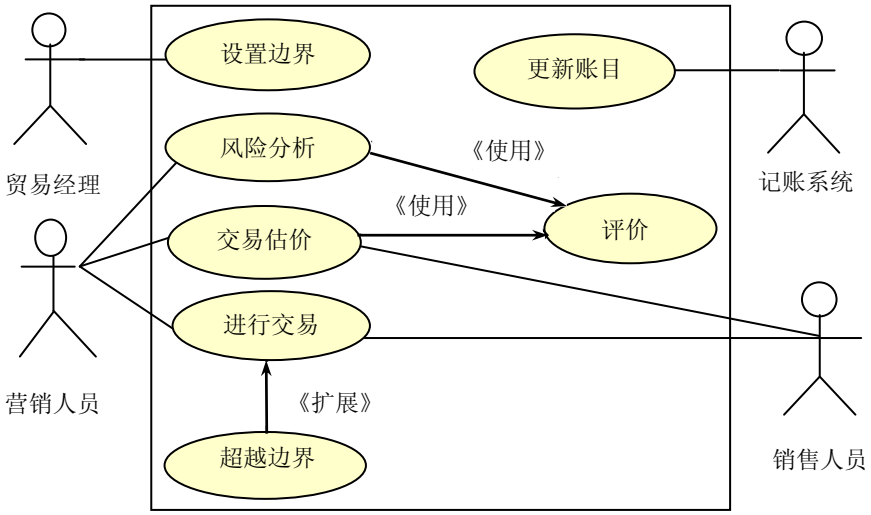
当与用户的交流告一段落后，分析员需要把识别的结果整理出来。通常一段简要、具有概括性的系统问题域描述是必须的。除此以外，还需要确定系统的范围。

UML的用例模型一直被推荐为识别和捕获需求的首选工具。在分析员对于问题域知识整理的基础上，可以应用用例模型确定系统的边界、主要功能和活动。因为用例重点描述的是某个功能组成（即用户所见的功能），一般它并不分析功能的实现。从面向对象分析的角度来说，我们不需要对每个用例都进行进一步的分析。但是如果某个用例所实现的功能是用户功能，那么我们就应该做进一步的分析。顺序图、合作图和活动图都是合适的分析工具。分析用例的目标并不是要实现或设计用例，只是尽量找出那些直接反映问题域本质和用户需求的活动或功能。如图3.33所示为一个金融贸易系统用例图的实例。按照获取执行者的方法，确定了4种执行者：贸易经理、营销人员、销售人员和记账系统。在该系统所确定的用例中，由贸易经理确定“设置边界”，所谓边界，是指金融贸易的范围。基本的用例是“进行交易”，用例“交易估价”与“风险分析”都要使用公共的“评价”动作，所以将“评价”作为一个独立的用例，在交易过程中，可能会出现超越交易范围或超过交易量，这时，允许对用例“进行交易”进行修改，即用例“超越边界”是用例“进行交易”的扩展。虽然大多数执行者是人，而图3.33中执行者“记账系统”是一个外部系统，它需要与用例“更新账目”进行交互。“交易估价”与“风险分析”都要对交易进行评价，因此，“评价”用例要被“交易估价”与“风险分析”使用。

毫无疑问，类及对象在面向对象方法中居于核心地位。面向对象分析需要识别类和对象。但是在这个阶段我们认为只要识别反映问题域本质的类和对象就可以了，尽量不要过早引入设计因素。我们建议在用例模型建立之后就应该进行类和对象的识别工作了，否则将无法进一步对用例进行分析。

在用例模型、类及对象模型和用例的描述模型基础上，通常我们需要找出那些具有复杂状态空间和状态变迁的问题域对象，这些对象可能不会有很多，但是这些类和对象的分

析、设计和实现以及测试将有可能花掉开发总成本中很大一部分，因此我们需要对这些类及对象作进一步的分析，可以使用的工具包括状态图和类图。



3.33　金融贸易系统用例图

面向对象分析的总的策略就是把所有能得到的问题域信息和用户需求记录下来，并用面向对象的观点对这些信息和用户需求进行描述，但不要过早地引入设计因素。虽然面向对象分析和面向对象设计之间并没有明确的界限之分，但是从整个开发来说，在整体需求尚没有明了和确定之前，引入设计内容会使整个模型在细化程度上不一致，同时使返工的可能性也大大地增大了。这些对于开发和管理来说都是风险因素，我们应该尽量减少这样的风险。

从需求分析的角度我们不仅要建立待实现的系统的规格说明，而且我们还要分析建立系统的性能目标以及系统的测试方法。一般在需求分析阶段需要明确的系统性能指标，包括系统可用性（usability）、可靠性（reliability）、失效安全性（safety）、信息安全性（security）、响应时间（response time）和可维护性（maintainability）等。

从UML的角度来说，面向对象分析阶段的测试策略可以根据用例模型来制定。由于需求分析主要用来获取用户的需求并定义系统的用户功能，因此对需求模型的测试目标就是其中的功能项和性能项。UML需求分析模型虽然由很多元素组成，但是并不是所有的模型元素都是用户所关心的，其中某些模型元素的存在是为了解释和说明另外一些模型元素。通常测试的对象包括用例模型和类及对象模型。

3.6.5　面向对象的程序设计概念

如前所述，面向对象软件开发方法OOSD包括面向对象分析（OOA）、面向对象设计（OOD）和面向对象程序设计（OOP）三个方面。其中OOP是基础，所以应用OOSD时，系统分析和设计的结果，应该选择面向对象程序设计语言来实现。随着人们对面向对象技术的广泛应用，面向对象语言的研究也非常活跃。面向对象的概念已渗透到软件的各个领域。

下面对面向对象程序设计与结构化程序设计两种方法进行简单的比较。

1．模块与对象

结构化程序设计中，模块是对功能的抽象，每个模块都是一个处理单位，它有输入和输出。而对象是包括数据、操作的整体，是数据和功能的抽象和统一。可以说，对象包含了模块。

2．过程调用与消息传递

在结构化程序设计中，过程为一独立实体，对于相同的输入参数，每一次过程调用，其输出结果是相同的。

在面向对象程序设计中，方法（过程或操作）是隶属于对象的，它不是独立存在的实体，而是对象的功能体现。从对象实现机制看，对象是一台自动机，其中私有状态表示了对象的状态，该状态只能由对象的操作改变它。在C++语言中用成员函数实现操作。每当需要改变状态的时候，只能由其他对象向该对象发送消息（借助于成员函数调用来实现消息发送）。对象响应消息后，按照消息模式找出匹配的方法，并执行该方法（执行函数调用）。应该注意，发送消息和过程调用的意义是不同的，发送消息只是触发自动机，同样的输入参数，可能因自动机状态不同，其输出结果也不同。因此，同一消息的多次发送可能产生不同的结果。

3．类型与类

类型与类都是对数据和操作的抽象，即定义了一组具有共同特征的数据和可操纵这些数据的一组操作，但类所定义的数据集（包括数据和操作）比常规语言的类型所定义的数据集要复杂得多。例如，在C语言中，要将i定义为一个整型变量，只要使用变量定义语句：int i；就行了。但在C++语言中，要定义一个类必须使用构造函数，用构造函数创建类对象。当创建的类对象在程序中不再使用时，类需要使用析构函数来回收无用的单元。而且，类引入了继承机制，实现了可扩充性。在定义一个类以后，由类定义的对象实例与在C语言中由类型定义的变量是类似的。

4．模块的可重用性

结构程序设计方法的核心是逐步细化。这种自顶向下的方法是通过不断在程序的控制结构中增加细节来开发程序。它开发的模块往往是为了满足特定需要，可重用性较差。

面向对象程序设计以数据结构为中心开发模块，同时一体化地考虑操作的功能，抓住了程序设计中最不易变的部分——数据，因此对象常具有良好的可重用性。

3.6.6　面向对象系统分析的一个实例——ATM系统

一、问题陈述

ATM（Auto Trade Machine）自动出纳机业务是银行网络系统的重要组成部分，包括人工出纳和分行共享的自动出纳机；各分理处用自己的计算机处理业务（保存账户、处理事务等）；各分理处与出纳站通过网络通信；出纳站录入账户和事务数据；自动出纳机与分行计算机通信；自动出纳机与用户接口，接受现金卡，发放现金，打印收据；分行计算机与拨款分理处结账。

要求系统正确处理同一账户的并发访问；网络费用平均摊派给各分理处。图3.34给出了银行网络系统的示意图。

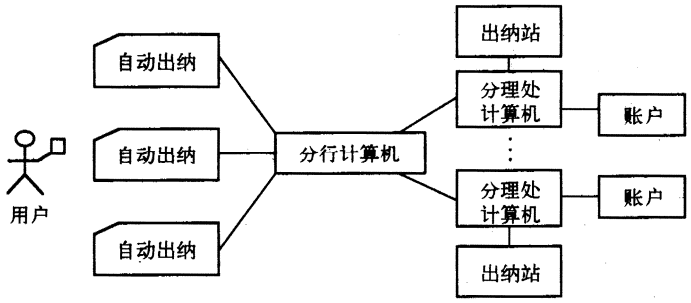


图3.34　银行网络系统的示意图

二、系统需求分析

ATM系统包括软件和硬件控制的部分，因此了解外部设备是如何协调工作的是整个建模的基础。ATM取款机的业务大致分为4部分：查询余额、取款、存款和更改密码。根据具体的业务对系统进行建模，一个功能完整的ATM系统必须包括以下的几个模块：

（1）读卡机模块。在这个功能模块中，允许客户将银行卡插入读卡机，读卡机识别卡的种类并在显示器上提示输入密码。

（2）输入模块。在该功能模块中，客户可以输入密码和取款金额，并选择要完成的事务。通常在键盘上只设置数字键和选择键，目的是方便客户使用。在该功能模块中，需要客户的交互。

（3）IC认证模块。这个功能模块主要用于鉴别卡的真伪。基于IC卡的安全授权系统，要求从技术上严格保证卡的唯一性与防伪性，使基于数字化形式的电子政务和电子商务安全运转，保证网络系统安全。

（4）显示模块。在该功能模块中，显示一切与客户有关的信息，包括客户交互时所需的提示信息和确认信息。

（5）吐钱机模块。该功能模块的功能是按照客户的需求，选择合适面值的钞票给客户，这是比较关键的一步。

（6）打印报表模块。该功能模块提供给客户一张取款凭据，客户可以选择是否打印。打印的主要信息是卡号和金额等。

（7）监视器模块。该功能模块的设置，是为了防止意外事件产生。设置了监控的摄像头，以保证户外交易的安全性。银行有权调查取款记录。

ATM系统功能需求如图3.35所示。

三、建立系统用例模型

面向对象系统分析的切入点是构筑系统的类和对象。使用UML建模的首要工作是建立系统中的用例模型。

1．角色的确定

首先考察ATM系统需要为哪些人服务，可有如下角色：

（1）客户使用ATM系统进行现金交易；

（2）银行有关人员更改ATM的设置，放置现金，维护机器等。

（3）信用系统作为外部的角色参与整个交易过程。

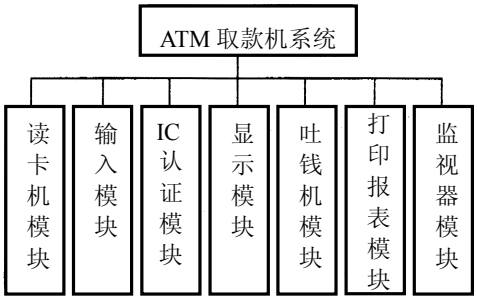


图3.35　ATM系统功能需求

ATM作为一个独立的系统，与客户、银行人员和信息系统这三个角色产生交互，最终确定以下角色：客户；银行人员；信息系统。

2．确定用例

用例是由角色驱动的，基于这样的考虑，ATM系统根据业务流程大致可以分为以下的几个用例：

（1）与客户相关的用例：客户转账；客户查询余额；客户存钱；客户通过信用系统付款；客户取钱；客户修改密码。

（2）与银行人员相关的用例：银行人员改变密码；银行人员维护ATM硬件；银行人员为ATM添加现金。

（3）与信用系统相关的用例：信用启动来自客户的付款。

3．建立系统用例图

系统用例图如图3.36所示。图中方框表示系统边界，用例图从宏观上表示了系统的执行者与用例的交互关系。

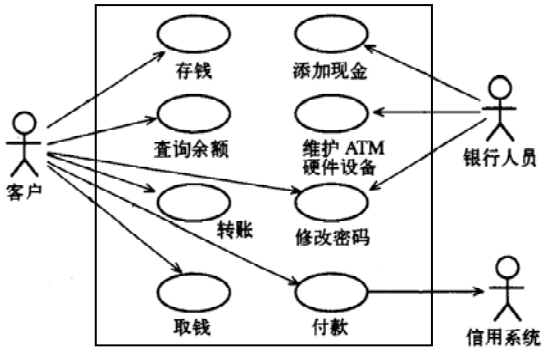


图3.36　系统用例图

四、系统类模型

1．系统包图

在定义具体的类之前，先在宏观的角度上将整个系统分割成多个独立的包。此处把整个ATM系统分成如图3.37所示的包。整个系统分为硬件和逻辑两块，分别控制不同的应用。

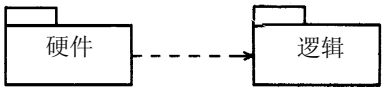


图3.37　系统包图

2．类图

类图是建立各类模型的基础。图3.38和图3.39分别描述了逻辑包、硬件包中的类图。在图3.38中，描述了Account（账户）和Database Connector（数据库连接）两个类之间的关系。在Account类中，有3个属性、4个方法；在Database Connector类中有2个属性和2个方法。

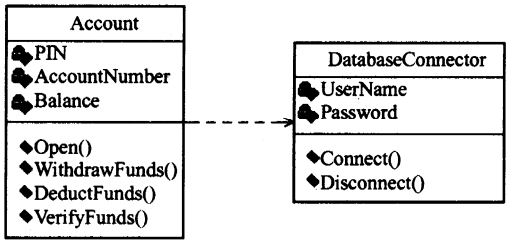


图3.38　逻辑包内的类图

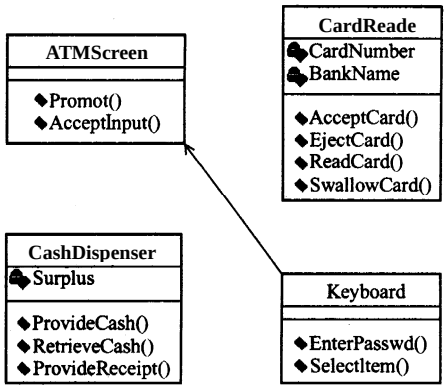


图3.39　硬件包内的类图

从Account指向DatabaseConnector的箭头，说明了Account类要发消息给Database Connector类，请求得到消息。当用户名和密码经过验证后，就可以访问数据库了。

如图3.39所示，在硬件包内，有4个类，其中Keyboard（键盘）类要发送消息来传递客户发出的指令，例如客户输入的密码、选择存钱还是取钱等操作。

类CashDispenser（吐钱机）和CardReader（读卡器）都跟逻辑包中的Account类有联系，但彼此之间没有消息联系，在这个包内是相对独立的。CardReader类的属性是卡号和银行代号，有4个方法：接受、拒绝、读卡和吞卡。CashDispenser类的属性只有Surplus（金额），方法有提供现金、收回现金等。

将逻辑包与硬件包中的类合起来，可建立系统类图。如图3.40所示，共有6个类，其中Account类跟其他4个类都有关联。

五、建立系统动态模型

动态模型包括活动图、时序图、合作图和状态图。这些UML图是对系统动态特征的描述。

1．活动图

活动图展示了系统中的功能流，可以在业务模型中显示业务工作流；可以在收集需求

时显示一个使用案例的事件流。活动图可以分为垂直泳道，每个泳道表示工作流中不同的角色。通过不同泳道中活动之间的过渡，可以了解角色之间的通信流程。图3.41示意了开户过程的活动图。经过这样的可视化建模，读者可以比较清楚地知道整个开户过程的业务流程，以便为将来的细化工作做准备。

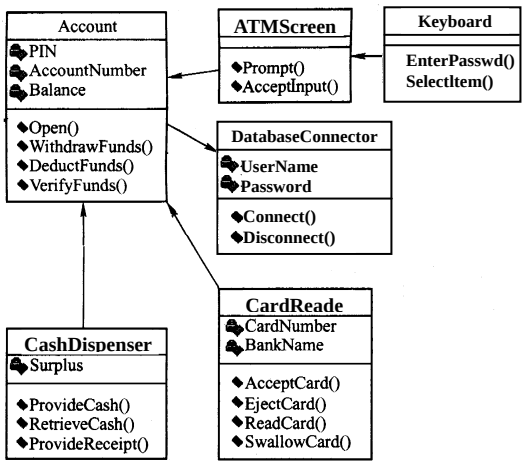


图3.40　系统类图

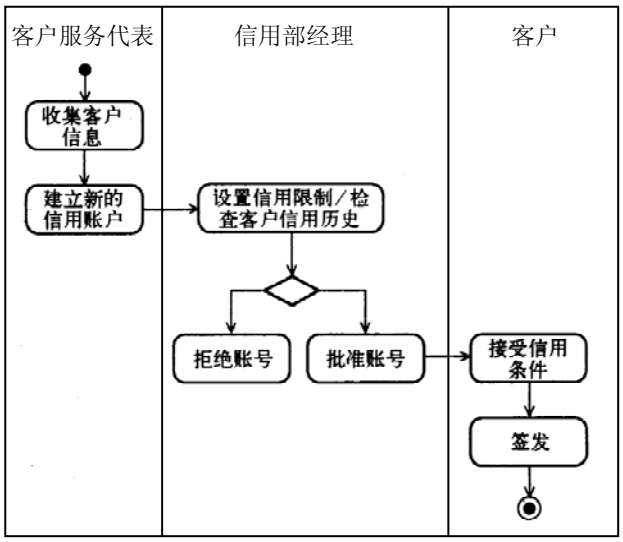


图3.41　“开户”的活动图

2．顺序图

表现系统流程以及系统对象之间的交互关系可以用时序图与协作图，它们能够清晰地表达系统流程，以及系统元素之间的交互关系，在时间与空间顺序上说明系统对象之间的关系。时序图的功能是按时间顺序描述系统对象间的交互。如图3.42所示为客户取款的时序图。

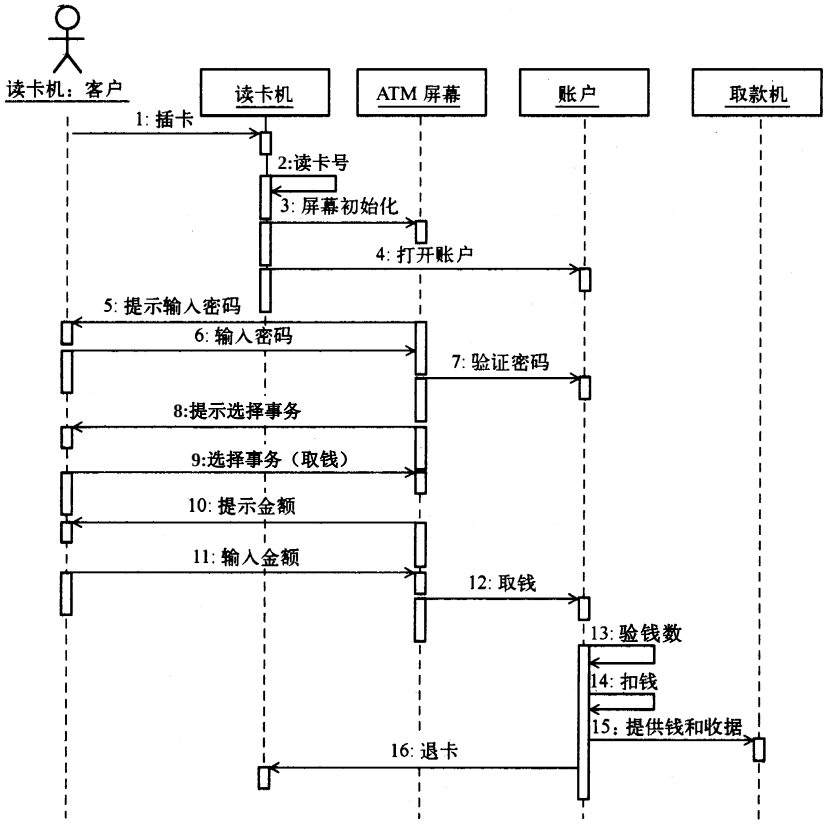


图3.42　客户取款的时序图

3．合作图

图3.43描述了客户取款的合作图。参与交互的对象有ATM显示屏幕、客户的账户、取款机、读卡机。消息的编号给出了取款过程的执行顺序。

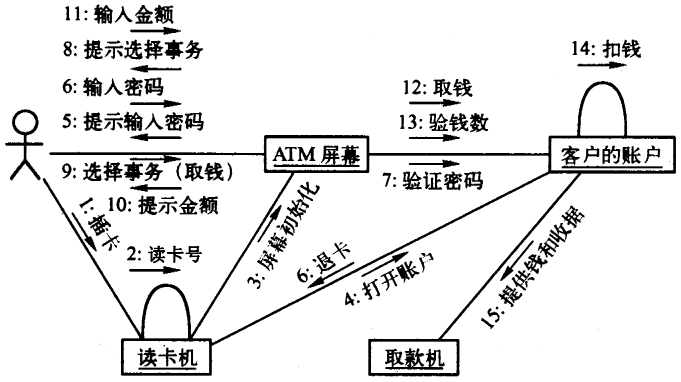


图3.43　客户取款合作图

合作图与时序图都是交互图，但两种图具有不同的作用。时序图强调对象之间交互的时间顺序，而合作图则主要考虑通过对象之间的交互，共同合作完成任务的过程，同时可以有效地利用视图空间。

4．状态图

状态图用来描述一个特定的类对象的所有可能的状态及其引起状态转移的事件。例如，银行的账目对象可能有打开、关闭或者透支三种状态。如图3.44所示，其中矩形框表示对象状态，对象在不同状态下功能不同，箭头表示状态转移，其上标注的是转移条件。

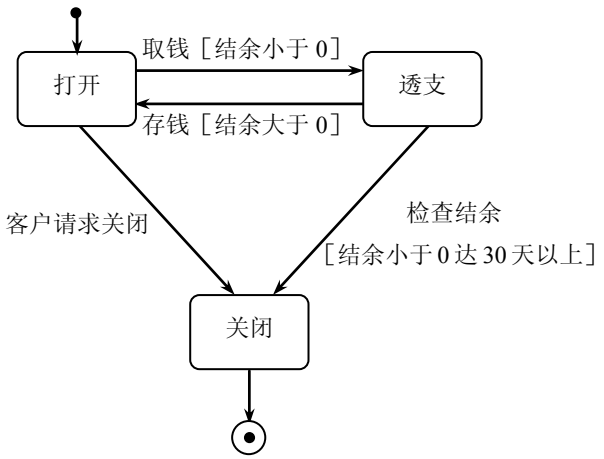


图3.44　账目类对象状态图

六、建立系统部署图

ATM系统部署是整个项目实施过程的最后阶段，其实质就是把该系统中涉及的硬件、软件整合到一起，描述系统的运行情况。在部署中有两种视图，即构件图和配置图。

1．构件图

构件图也称为组件图，它包含了模型代码库、可执行文件、运行库和其他构件的信息。它是代码的实际模块。

构件图分为两部分：Client和Server。

如图3.45所示，ATM系统Server的构件图中，Account类跟Server服务器有紧密的联系。深灰色表示是账目类的一个实例。

ATM系统Client的构件图，如图3.46所示，在Client中包含了ATM取款机的基本硬件。最终将客户端打包成一个exe应用程序，包括3个构件：Card Reader、ATM Screen和Card Dispenser。深灰色表示它们所对应的实例。

2．配置图

ATM系统的配置图如图3.47所示。在该配置图中只画出了一个ATM Client，一个地区的ATM服务器，银行数据库服务器，以及一台打印机，它大致上描述了整个系统的物理部署情况。

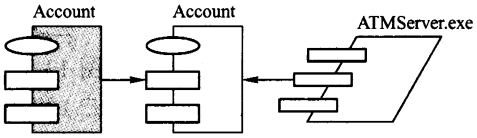


图3.45　Server的构件图

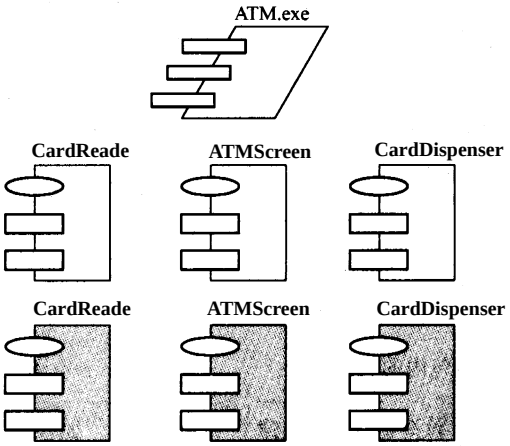


图3.46　Client的构件图

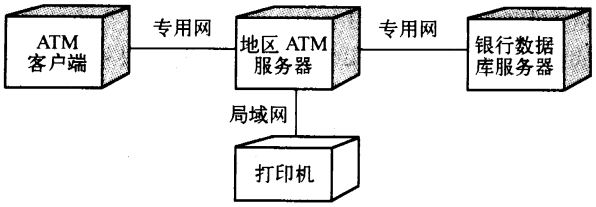


图3.47　ATM系统的配置图

小　　结

面向对象方法是一种把面向对象的思想应用于软件开发过程中，指导开发活动的系统方法，简称OOSD。OOSD包括面向对象分析（OOA）、面向对象设计（OOD）和面向对象程序设计（OOP）三个方面。面向对象的方法是在结构化方法的基础上，力图用更自然的方法反映客观世界。在面向对象的系统中，将数据和使用该数据的一组基本操作或过程封装在一起，用“对象”这个概念来完整地反映客观事物的静态属性和动态属性。“面向对象”的基本思想就是把要构造的系统表示为对象的集合。

面向对象技术是一个非常实用而强有力的软件开发方法。面向对象的程序具有易理解性、易修改性和可重用性，可以明显提高软件开发和维护的效率，降低软件的总成本。随着人们对面向对象技术的广泛应用，面向对象的概念已渗透到软件的各个领域。面向对象技术被誉为20世纪90年代软件的核心技术之一。

UML的目标是以面向对象的思想来描述任何类型的系统。UML是一个通用的建模语言，可以对任何系统进行分析和建立相应的模型。UML支持系统开发的不同阶段，从需求描述到系统的测试。在需求分析阶段，UML通过用例模型来捕获用户需求。通过用例图，UML描述对系统感兴趣的外部角色和他们对系统的功能要求。分析阶段主要关心问题域的概念和实体，并得到与问题域直接相关的类和对象，以及它们之间的关系。在系统设计阶段，用户需要定义一些与技术实现相关的类，如用户接口、数据库、通讯和并行等问题，

UML提供了强大的静态和动态建模机制。在编码阶段，开发人员应用一种语言（最好是支持面向对象的语言）将设计得到的类转换成实际的代码，这时需要把某些UML所支持的概念合适地转化到特定的实现语言中。

面向对象语言也有不足之处。由于传统计算机与面向对象语言的语义相差比较大，这有点类似于在传统计算机上实现函数式和逻辑式语言，此外，消息传递也比过程调用费时间，所以面向对象语言程序的执行效率较低。

3.7 软 件 测 试

为了保证软件的质量和可靠性，应力求在分析、设计等各个开发阶段结束前对软件进行严格的技术评审，以便能够尽早地发现错误。但是，实践证明，由于人们能力的局限性，审查并不能发现所有的错误。如果在软件投入运行之前，没有发现并纠正大部分的错误，那么这些错误将会在软件的生产过程中暴露出来。到这时，不仅要为改正这些错误付出巨大的代价，同时也会造成恶劣的后果。这种情况就迫使人们必须认真地、彻底地进行软件测试。

3.7.1 软件测试概述

一、基本概念

软件测试是在软件投入运行前，对软件需求分析、设计规格说明和编码的最终复审，是保证软件质量的关键步骤。我们可以这样定义软件测试：软件测试是为了发现错误而执行程序的过程；或者说，软件测试是根据软件开发各阶段的规格说明和程序的内部结构而精心设计一批测试用例（即输入数据及预期的输出结果），并应用这些测试用例去运行程序，以发现程序错误的过程。

软件测试在软件生存周期中横跨了两个阶段：通常在编写出每一个模块之后就对它做必要的测试（称作单元测试）。编码与单元测试属于软件生存周期中的同一阶段。在结束这个阶段之后，对软件系统还要进行各种综合测试，这是软件生存周期的另一个独立的阶段，即测试阶段。

二、目标和原则

软件测试不是为了证明软件是正确的，而是立足于软件有错误而进行的。其基本目的是要设法暴露软件中存在的错误和缺陷。软件测试的目标可以归纳为以下几点：

1．测试是为了发现软件中的错误而去运行软件的过程。

2．好的测试方案是尽可能地发现至今尚未发现的错误的测试方案。

3．成功的测试则是发现出至今尚未发现的错误的测试。

测试的目标是以最少的时间和人力找出软件中潜在的各种错误和缺陷。除了发现软件中的错误外，测试还能够证明软件的功能是否同性能和需求说明相符。此外，实施测试所收集到的测试结果数据为可靠性分析提供了依据。

在彻底暴露软件错误的前提下，软件测试应遵循以下基本原则：

1．不要抱有“软件不会有错或认为查不出错”的幻想

由于原始问题的复杂性和软件设计的创造性、抽象性等因素，使得开发的每一个环节都会产生错误。在测试工作开始之前，若抱着“不会有错”的侥幸心理进行测试用例的设计势必会放过错误，达不到测试的目的。

2．设计测试用例时，应同时确定输出结果

测试用例应当包括测试输入数据和与之对应的预期输出结果两部分，否则，由于对输出结果心中无数，会将一些不十分明显的错误结果当作正确的结果。

3．设计测试用例时，应包括合理的输入数据和不合理的输入数据

在测试软件时，人们常常会倾向于重视合理的、有效的输入数据，以检查软件是否完成了它应该完成的工作，而忽略了那些非预期的、不合理的输入数据。事实上，软件在投入运行以后，用户不遵循事先的约定，而使用一些意外的输入是常有的事。因此，软件处理非法命令的能力也必须在测试时受到检验。同时，一些不合理的输入，有可能发现一些在正常的输入条件下没有暴露的问题。所以选择合理的输入数据和不合理的输入数据都是非常重要的。

4．软件设计者应当避免测试自己的程序

出于对人们似乎不愿否定自己的工作这一心理状态等因素的考虑，通常由别人来测试软件将更为客观、冷静。另外，由于程序员对需求说明和规格说明的理解偏差而引入的错误，本人是不易发现的，所以应当避免软件设计者测试自己的程序。

5．严格全面地执行测试计划

要制定明确、详细而完善的测试计划。在执行过程中，不能对测试计划进行随意的解释。一般来讲，测试计划包括：所测软件的功能、输入和输出、测试内容、各项测试的进度安排、资源要求、测试资料、测试工具、测试用例的选择、测试的控制方式和过程、系统组装方式、跟踪规程、调试规程、回归测试的规定以及评价标准等。

在测试的程序段中，若发现的错误多，那么残存的错误数目也较多。这就是“错误群集”现象。所以在测试过程中，不要以为找到几个错误就解决了问题，而不再继续测试。

对每一个测试结果都要做认真、仔细的分析，以免遗漏已暴露出来的错误。

6．测试完成以后，应该妥善保存测试计划、测试用例、出错统计和最终分析报告。

三、测试的方式

软件测试的方式可以分为：人工测试、动态测试和自动测试三种。它们可以互为补充，从而提高软件测试的效率。

1．人工测试

人工测试是由人工对软件进行阅读和检查。人工测试的主要方法有：走查和会审。经验表明，使用这种方法能够有效地发现30%到70%的逻辑设计和编码错误。

走查是测试人员阅读程序，并扮演“计算机”的角色，让测试用例沿程序的逻辑运行一遍，从中发现一些程序中的疏漏和错误。

会审是由软件开发者主持，聘请其他程序员或分析员参加的软件审查会，对源程序进行审查。

人工测试包括的主要内容有：检查软件的结构是否合理，功能是否达到设计说明中的要求；检查各模块之间的接口是否正确；检查各模块内部控制逻辑的正确性；进行数据结构分析和语法分析。

2．动态测试

动态测试就是上机测试，用设计好的测试用例作为输入，执行程序，以发现软件中的错误。

3．自动测试

自动测试就是利用一些测试工具，代替人工进行静态分析和动态测试。这些测试工具分为：静态分析工具、动态分析工具和测试评估工具。

静态分析工具无需执行所测试的软件，它扫描被测试软件的正文，对软件的数据流和控制流进行分析。然后得出测试报告。这类工具包括静态确认工具、符号执行工具、程序验证工具。

动态分析工具帮助人们进行动态测试。通过选择适当的测试用例，实际运行被测软件，比较实际运行结果和预期结果，以发现错误。这类测试工具包括测试数据发生器、覆盖监视器、模块驱动工具、符号查错工具。

综合测试评估工具是集静态分析、功能分析、测试评估等功能为一体的测试工具。

3.7.2　软件测试策略

测试过程是按单元测试、组装测试、确认测试和系统测试四个步骤进行的。图3.48显示了软件测试的过程。

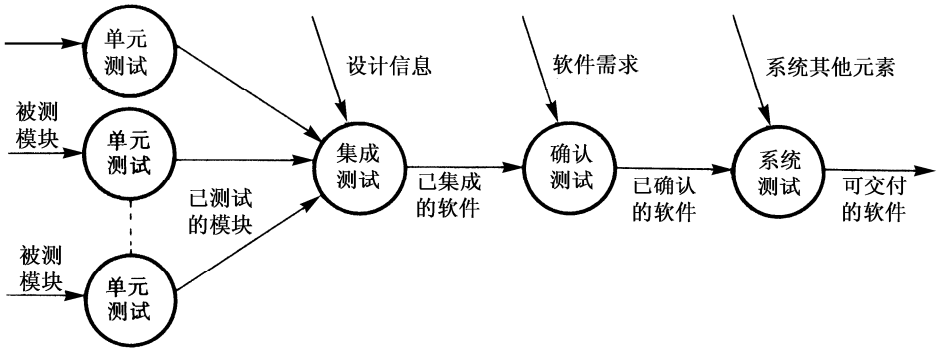


图3.48　软件测试的过程

一、单元测试

单元测试又称模块测试。它的测试目的是发现模块的子程序或过程的实际功能与该模块的功能和接口的描述是否相符，以及是否有编码错误存在。单元测试的主要内容有：模块接口测试；局部数据结构测试；重要路径测试；出错处理能力测试；边界条件测试。严格地讲，单元测试从程序编写完毕，着手对程序模块进行检查的时候就开始了。它包括人

工测试（代码审查）和上机测试两个步骤。

二、组装测试

单元测试之后，应按要求把模块组装起来，进行组装测试。组装测试又称集成测试或联合测试。它的测试目的是为了发现程序结构的错误。组装测试过程中的模块组织方式有非渐增式和渐增式两种。

1．非渐增式组装测试

非渐增式组装测试又称一次性组装方式或整体拼装。这种组装测试方式是先对每个模块分别进行测试，然后再把所有模块组装在一起进行整体测试。其优点是对各模块的测试可以并行进行，有利于充分利用人力，加快测试速度。其缺点是由于程序中不可避免地存在涉及模块间接口、全局数据结构等方面的问题，所以一次试运行成功的可能性不大，结果是发现有错误，但却找不到错误的产生原因。

2．渐增式组装测试

渐增式组装测试又称增殖式组装方式。这种方式是先对一个个模块进行模块测试，然后将这些模块逐步组装成较大的系统。在组装过程中，每连接一个模块便进行一次测试，直到把所有模块集成为一个整体并进行测试，则软件的组装测试完成。很显然，渐增式组装测试的优点是能及时发现各模块接口间的错误，而且对问题的诊断和定位都较非渐增式组装测试容易。另外，在渐增式组装测试方式下，已测试的模块将和新组装进去的模块一起测试，这将在变化条件下对已测试的模块进行新的检验，从而使得测试更为彻底。随之而来的问题是程序规模变大，机时开销增加等。

在渐增测试过程中，将模块结合起来的策略有两种：自底向上测试和自顶向下测试。

（1）自底向上测试

自底向上测试策略是从程序模块结构的最低层模块进行组装和测试。因为模块是自底向上进行组装的，对给定层次的模块的下层模块处理功能总可以得到，所以这种测试策略不必设计桩模块（存根模块），但要设计驱动模块。常见的驱动模块如图3.49所示。

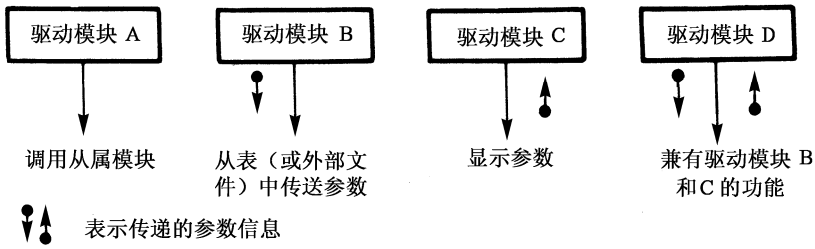


图3.49　常见的驱动模块

下面，我们举例说明采用自底向上测试策略的测试步骤。如图3.50所示的程序结构，采用自底向上测试策略的测试过程如图3.51所示。

图中，模块M_1、M_2、M_3、M_4、M_5、M_6是程序模块；模块D_1、D_2、D_3、D_4、D_5是建立的驱动模块。测试步骤为：

第一步：如图3.51（a）所示，对最低层的模块M_3、M_5、M_6进行测试。设计驱动模块D_1、D_2、D_3，用它们来模拟M_1、M_2、M_4对M_3、M_5、M_6的调用。

第二步：如图3.51（b）所示，将模块M_2和M_5联结起来，设计驱动模块D_4模拟M_1对它们的调用。将模块M_4和M_6连接起来，设计驱动模块D_5模拟M_1对它们的调用。

第三步：如图3.51（c）所示，把已测试的子结构按程序结构联结起来完成程序整体的组装测试。

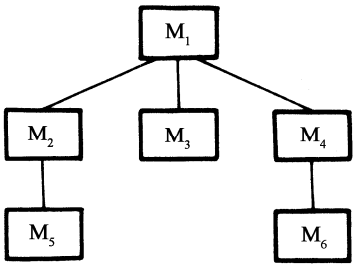


图3.50　程序模块示意图

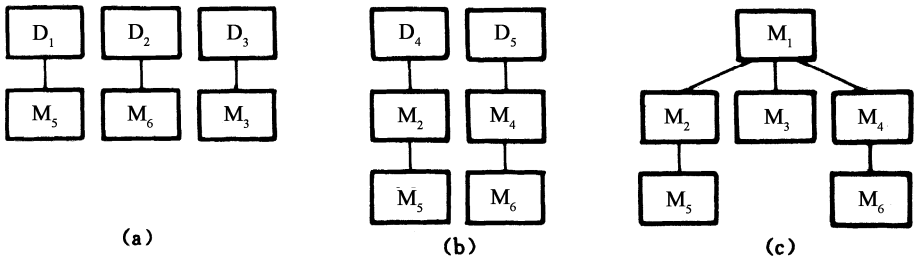


图3.51　自底向上测试过程示意图

（2）自顶向下测试

自顶向下测试策略是将模块按系统程序结构，沿控制层次自顶向下进行组装。由主控模块开始，按照程序的层次结构向下移动，逐渐把各个模块组装起来。对如图3.50所示的有三层结构的程序，采用自顶向下测试策略的测试过程如图3.52所示。

第一步：如图3.52（a）所示，对主控模块M_1进行测试，设计桩模块S_1、S_2、S_3模拟被M_1调用的模块M_2、M_3、M_4。

第二步：依次用M_2、M_3、M_4替代桩模块S_1、S_2、S_3。每替换一个模块的同时进行一次测试。

第三步：如图3.52（b）所示，把模块M_2、M_3、M_4和主控模块M_1结合，设计桩模块S_4、S_5模拟被M_2和M_4调用的模块M_5和M_6，然后进行测试。

第四步：如图3.52（c）所示，依次用模块M_5和M_6替换桩模块S_4、S_5，并同时进行新的测试。此时已包含了该程序的全部模块，组装测试完毕。

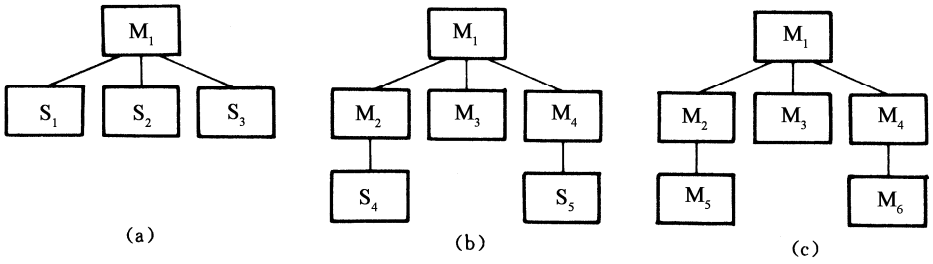


图3.52　自顶向下测试过程示意图

常见的桩模块如图3.53所示。

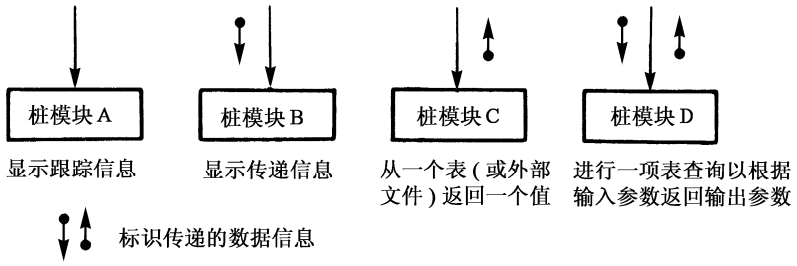


图3.53　常见的桩模块

三、确认测试

确认测试又称有效性测试。组装测试结束后，得到的是一个完整的软件系统。这时需要进行最后的测试，即有效性测试。有效性测试阶段主要进行的测试有：有效性测试（黑盒测试）、软件配置复查、α测试和β测试以及验收测试。

有效性测试是在模拟的环境下，运用黑盒测试的方法，验证被测试的软件是否满足需求规格说明书上的需求。

软件配置复查是为了保证软件配置齐全，既要保证所有文档资料的完整性和正确性，同时还要保证软件各方面的质量都要符合要求。

α测试和β测试是为发现可能只有最终用户才能发现的错误。α测试是一个用户在开发环境下，并且在开发者的陪同下进行的测试。目的在于评价软件的功能、可使用性、可靠性、性能和支持等。β测试是多个用户在一个或多个用户的实际使用环境下进行的测试。进行β测试时，开发者不在现场。β测试的目的在于测试可支持性。β测试要在α测试达到一定的可靠程度时才可进行。

验收测试是以用户为主的测试。目的在于确定系统功能和性能的可接受性。

四、系统测试

系统测试是指将经过确认测试后的软件系统与计算机硬件、外设、其他支持软件以及其他系统元素一起进行测试。测试内容主要有：功能测试、吞吐量测试、可用性测试、保密性测试、安装测试、可恢复性测试、资料测试和程序测试。

3.7.3 常用的测试方法

常用的测试方法有黑盒测试和白盒测试两种。采用不同的测试方法，设计的测试用例也有所不同。“测试用例”是指为测试软件而设计的测试数据（包括输入数据和其对应的预期输出结果）。

一、白盒测试

白盒测试又称结构测试或逻辑驱动测试。所谓“白盒”是指将测试对象看作一个打开的盒子，测试人员可利用程序内部的逻辑结构及有关的信息来设计或选择测试用例。因此白盒测试主要考虑的是测试用例对程序内部逻辑的覆盖程度，而不考虑程序的功能。用例对程序的覆盖程度从低到高分别为：语句覆盖、判定覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖。

下面以如图3.54所示的程序段为例，分别介绍这几种逻辑覆盖。

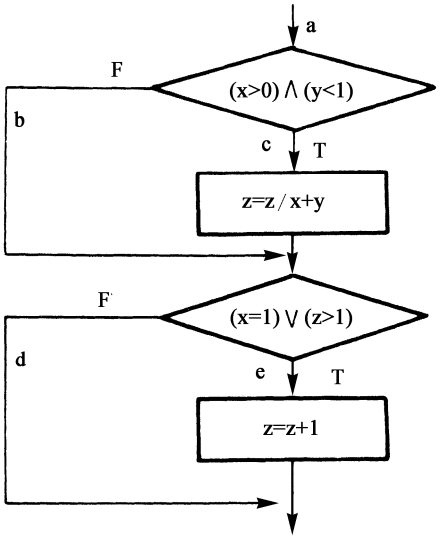


图3.54　程序段流程图

如图3.54所示，该程序段包含4条不同的路径。分别为：l_1（a，c，e），l_2（a，b，e），l_3（a，b，d），l_4（a，c，d）。

1．语句覆盖

语句覆盖是针对程序中的语句执行情况来设计测试用例，使用例的执行能覆盖程序中所有语句，从而使得每一条可执行语句至少执行一次。

在图3.54所给出的例子中，所有可执行语句都在路径l_1上，所以选择路径l_1设计测试用例为：输入：（x，y，z），输出：（x，y，z）⇒ 输入：（1，0.5，2），输出：（1，0.5，3.5），覆盖路径l_1。

这种测试初看起来仿佛是检查了每个语句的正确性，实际上它忽略了对程序分支的判定条件的检查。比如：若将第一个判定条件中的“∧”误写为“∨”或将第二个判定条件

中的“∨”误写为“∧”，上面的测试用例仍可以覆盖4条可执行语句。也就是说，即使做到了语句覆盖，也发现不了判断中逻辑运算出现的错误。因此，语句覆盖准则是比较弱的逻辑覆盖准则。

2．判定覆盖

判定覆盖又称分支覆盖。它是着眼于程序中每个分支的执行情况来设计若干个测试用例，使得程序中的每个判定至少都获得一次“真值”和“假值”的机会。即：使程序中的每一个分支都至少通过一次。

如图3.54所示的例子中，如果选择路径l_1和路径l_3，就可以使程序中的每一个分支都至少通过一次。当然选择路径l_2和路径l_4也可以设计出满足要求的测试用例。

判定覆盖比语句覆盖严格，因为它既注重判定的正确性，又包含了对每个语句的执行，满足判定覆盖的测试同时可以做到语句覆盖。但是判定覆盖仍然不够充分。因为一个判定往往包含多个条件，若其中某一个或几个条件有错，用判定覆盖准则就不能检查出来。

3．条件覆盖

条件覆盖注重每个判定中各个条件的各种可能的取值。也就是要设计若干个测试用例，使程序中的每个判定的每个条件的可能取值至少执行一次。

对图3.54给定的例子，做如下假设：

判定1：x>0　设真值为：T1，假值为：F1

　　y<1　设真值为：T2，假值为：F2

判定2：x=1　设真值为：T3，假值为：F3

　　z>1　设真值为：T4，假值为：F4

则可设计如下的测试用例：

测试用例一：（见表3-1）

表 3-1

条件取值	通过路径	覆盖分支	测试用例	
			输入（x，y，z）	输出（x，y，z）
T1 T2 T3 T4	L_1	c，e	（1，0，2）	（1，0，3）
F1 T2 F3 F4	L_3	b，d	（−1，0，0）	（−1，0，0）
T1 F2 T3 F4	L_2	b，e	（1，2，0）	（1，2，1）

测试用例二：（见表3-2）

表 3-2

条件取值	通过路径	覆盖分支	测试用例	
			输入（x，y，z）	输出（x，y，z）
F1 T2 F3 T4	L_2	b，e	（−1，0，2）	（−1，0，3）
T1 F2 T3 F4	L_2	b，e	（1，2，0）	（1，2，1）

测试用例一覆盖了所有判定的取真值和取假值的分支，并且覆盖了判定中所用条件的

可能的取值。测试用例二满足了判定中所有条件的可能的取值，却只覆盖了第一个判定的取假值的分支和第二个判定的取真值的分支，不满足判定覆盖。这说明了：条件覆盖和判定覆盖不一定是一致的。满足条件覆盖的测试用例不一定能满足判定覆盖。为了兼顾判定中的条件和每个分支，需要把条件覆盖和判定覆盖结合起来。

4．判定-条件覆盖

判定-条件覆盖是兼顾条件和分支的覆盖准则。就是要设计足够多的测试用例，使得判定中的每个条件都能取得各种可能的“真”和“假”值，并且每个判定也都能取到“真”和“假”两种结果。

仍以图3.54程序段为例，设计如表3-3所示的测试用例就可以覆盖所有的条件和分支。

表 3-3

条件取值	通过路径	覆盖分支	测试用例	
			输入（x，y，z）	输出（x，y，z）
T1 T2 T3 T4	L_1	c，e	（1，0，2）	（1，0，3）
F1 F2 F3 F4	L_3	b，d	（－1，2，0）	（－1，2，0）

判定-条件覆盖准则比前三种准则有效，但是它还不够充分。因为一个判定通常包含多个条件，判定的结果是由各个条件的取值进行逻辑运算后决定的。在判定-条件覆盖中考虑了各个条件的取值和判定的取值，但并没有对各个条件可能的取值的逻辑运算（即各种组合情况）进行考虑。因此，采用判定-条件覆盖，逻辑表达式中的错误不一定查得出来。

5．条件组合覆盖

条件组合覆盖是更严格的逻辑覆盖准则。这种覆盖准则是指设计足够多的测试用例，使每个判定的所有可能的条件取值组合至少出现一次。

对于图3.54给出的例子，每个判定的条件取值组合如下，并依此来设计测试用例。

判定1：x>0，y<1

x<=0，y<1

x>0，y>=1

x<=0，y>=1

判定2：x=1，z>1

x=1，z<=1

x<>1，z>1

x<>1，z<=1

条件组合覆盖准则既能满足判定覆盖准则，也能满足条件覆盖准则和判定-条件覆盖准则。

需要说明的是，上述各种覆盖准则的侧重点不同，覆盖的程度也不同。但它们共同的是：任何一种覆盖都不能做到完全测试。

二、黑盒测试

黑盒测试又称为功能测试或数据驱动测试。在这种测试方法中，程序对测试者是完全

透明的。测试者不考虑程序的内部结构和特性，就好像把程序看作一个不能打开的盒子，只根据程序的需求规格说明中的程序功能或程序的外部特性来设计测试用例。

黑盒测试的方法包括：等价分类法、边缘值分析法、因果图法和错误推测法。

1．等价分类法

等价分类法是一种典型的黑盒测试法。使用等价分类法的步骤是：

第一步：划分等价类，并为每个等价类规定一个唯一的编号。

第二步：设计一个测试用例，使其尽可能多地覆盖尚未覆盖的有效等价类，重复此步直到所有有效等价类都被覆盖为止。

第三步：设计一个新的测试用例，使其仅覆盖一个无效等价类，重复此步直到覆盖了所有无效等价类为止。

划分等价类是指把所有可能的输入情况划分成若干个输入域的集合，即等价类。选择每个集合中的一个测试用例即可代表该集合中其他的测试用例。对程序设计要求来讲，那些合理的、有意义的输入数据所构成的集合为有效等价类。对程序设计要求来讲，那些不合理的或无意义的输入数据所构成的集合为无效等价类。根据问题的不同，有效等价类可以是一个，也可以是多个。无效等价类至少应该有一个，也可能有多个。等价类的划分应该遵循以下原则：

①若输入条件规定了取值范围或值的个数，则可以确定一个有效等价类和两个无效等价类。

比如，输入条件是：“……x可以从0到10……”，则有效等价类是“0≤x≤10”；两个无效等价类是：“x<0”和“x>10”。

②若输入条件规定了输入值的集合，或者规定了“必须如何”的条件，则可以确定一个有效等价类和一个无效等价类。即规定的输入值的集合构成有效等价类，而不在此集合内的归于无效等价类。

③若已知某等价类中的各元素在程序中的处理方式不同，则将此等价类划分为更小的等价类。

2．边缘值分析法

在程序设计时，设计者往往不注意输入/输出条件的边界值，而引起程序出错。因此提出边缘值分析法。这种方法是对等价分类法的补充。

采用边缘值分析应遵循的主要原则有：

（1）若输入条件规定了值的范围，则应取刚达到这个范围的边界的值，以及刚刚超越这个范围边界的值作为测试输入数据。

（2）若输入条件规定了值的个数，则用最大个数、最小个数、比最大个数多1、比最小个数小1的数作为测试数据。

（3）根据规格说明的每个输出条件，使用原则（1）和原则（2）。

（4）若程序的规格说明中给出的输入/输出域是有序集合，则选取集合的第一个元素和最后一个元素作为测试用例。

（5）若程序中使用了一个内部数据结构，则选择这个内部数据结构的边界上的值作为

测试数据。

3．因果图法

前两种测试方法只考虑了输入条件单独出现时可能出现的错误情况，而没有对各种输入条件的组合情况做测试。事实上，检查各种输入情况的组合并不是容易的事。因为组合条件数是非常庞大的。

因果图法是针对此而提出的一种系统的设计测试用例的方法。它的基本思想是把输入条件视为“因”，把输出条件视为“果”，将黑盒看成是从因到果的网络图。然后利用因果图导出测试用例。

采用因果图法的基本步骤是：

（1）分析程序规格说明，找出哪些是“因”，哪些是“果”。

（2）根据程序规格说明，画出连接各原因和各结果的因果图。

（3）将因果图转换为判定表。

（4）根据判定表的每一列设计相应的测试用例。

4．错误推测法

测试人员往往可以根据经验和直觉推测程序中可能存在的错误，并以此设计测试用例来检查这些错误。

错误推测法的基本思想是列出程序中可能有的错误和容易发生错误的特殊情况，根据它们选择测试方案。

测试方法还有回归测试、强度测试等等。每一种测试方法都各有所长，在实际测试中应综合使用。一般来讲，通常用黑盒法设计基本的测试方案，再利用白盒法做必要的补充。

小　　结

软件测试是为了发现错误而执行程序的过程。软件测试的目的是要暴露软件系统中的隐含错误，然后通过软件调试找出错误的原因和位置并加以改正。

软件开发中，测试是保证软件正确性的最后一个阶段，测试需要制定测试计划，设计测试用例，然后实施测试，测试后进行分析评价，测试结束后，要给出测试报告。

软件测试方式分为：人工测试、动态测试和自动测试三种。测试过程按单元测试、组装测试、确认测试和系统测试四个步骤进行。

常用的测试方法有黑盒测试和白盒测试两种。对于不同的测试方法，需要设计不同的测试用例。

3.8　软 件 维 护

当软件系统开发完毕并交付用户使用后，便进入运行/维护阶段。软件维护是生存周期的最后一个阶段，也是历时最长的一个阶段。它的工作是保证软件在长时间内能够正常运行。

3.8.1 软件维护的概念

一、进行软件维护的必要性

软件系统在运行过程中，会暴露出一些错误和缺陷，同时由于软件运行环境可能发生变动，而导致要对现有的软件系统做一定的修改，因此，软件维护工作是必不可少的。概括起来，要进行软件维护的主要原因有：

1．对在运行过程中的某些特定条件下暴露出来的程序错误和设计缺陷做修改。

2．由于软件的运行环境发生了改变（如：增加新的硬件），则需要修改软件以适应新的工作环境。

3．为增加新的功能，提高软件的性能而修改软件。

有资料表明，维护阶段的花费约占整个软件生命期花费的67%，因此，只有认识到维护工作的重要性和迫切性，才能使开发的软件发挥其应有的效益。

二、软件维护的种类

根据维护目的不同，软件维护可以分为4类。不同类型的软件维护，其维护策略也有不同。

1．纠错性维护

主要针对在测试阶段未能发现的而在软件投入使用后才逐渐暴露出来的错误。对这类错误的测试、诊断、定位、纠错以及验证、修改的回归测试过程称为纠错性维护。

2．适应性维护

适应性维护就是要使运行的软件能适应运行环境的变动而修改软件的过程。计算机技术发展日新月异，随着新的计算机硬件系统的不断发展，新的操作系统或操作系统的新版本会不断推出。此外，外部设备和其他部件也要经常修改和改进。所以为了使应用软件在与开发环境有很大差异的实际环境中运行，需要不断地进行适应性维护。

3．完善性维护

当一个软件系统投入使用时、用户会根据应用发展的实际情况提出增加新的功能、修改已有功能以及一般的改进要求或建议。为满足这些要求而进行的工作称为完善性维护。完善性维护是维护工作中最主要的部分。

4．预防性维护

预防性维护是为了进一步改善软件的可靠性和易维护性，或者为将来的维护奠定更好的基础而对软件进行的修改过程。

可见，维护不仅仅局限于纠正软件中的错误，更重要的是完善现有的软件系统。根据统计资料显示：完善性维护约占全部维护活动的50%，纠错性维护约占25%，适应性维护约占21%，预防性维护约占4%。

三、提高软件的可维护性

软件的可维护性是指纠正软件出现的错误和缺陷，以及为满足新的要求进行修改、扩充或压缩的容易程度。软件的可维护性是软件开发阶段各个时期的关键目标。

通常可以用可理解性、可测试性、可修改性、可靠性、可移植性、可使用性和效率这七个特性来衡量软件的可维护性。对不同类型的维护，这七种特性的侧重点不同：纠正性维护侧重可理解性、可测试性、可修改性和可靠性；适应性维护侧重可修改性、可移植性和可使用性；完善性维护侧重可使用性和效率。

软件可维护性对于延长软件的生存期具有决定意义。为提高软件的可维护性可以从如下五个方面考虑：建立明确的软件质量目标和优先级、使用提高软件质量的技术和工具、进行明确的质量保证审查、选择可维护的程序设计语言和改进程序的文档。

3.8.2 软件维护的步骤与方法

一、软件维护的工作流程

软件维护的工作流程如图3.55所示。

对任何种类的维护请求，首先是确认维护要求，明确维护类型。这是需要由用户与维护人员反复协商确定的。

对纠错性的维护请求，从评价软件错误的严重性开始。若存在严重的错误，则安排人员在系统管理员的指导下立即进行问题分析和紧急维护；若存在的错误不严重，可根据问题的轻重缓急，进行排队，统一安排时间。

对适应性维护和完善性维护的维护请求，从确定每项申请的优先次序开始。如果某项维护申请的优先级非常高，则立即开始维护工作；否则与其他维护任务一样，进行排队，统一安排时间。

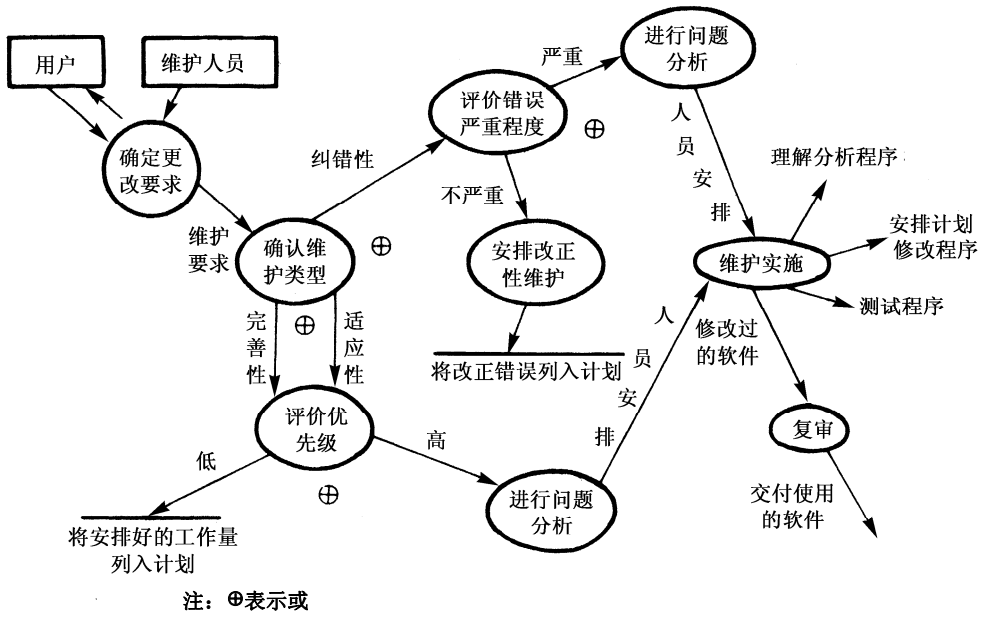


图3.55 软件维护的工作流程

尽管维护请求的类型不同，但都要进行同样的技术工作。这些工作有：修改软件设计、设计复审、对源程序做必要的修改、模块测试、组装测试、系统测试、复审。

二、软件维护的方法

具体的维护工作经过理解分析、修改和重新验证现有软件系统三个过程。下面简单介绍这三个过程中使用的维护方法。

1．理解、分析现有软件系统

理解、分析现有软件系统即理解系统的功能、系统的内部结构以及系统的操作要求。常用的方法有：分析软件结构图、数据跟踪和控制跟踪。

2．修改现有软件系统

修改现有软件系统时，应注意保持系统功能的完整性，有计划地、周密有效地进行修改。修改程序的步骤是：

（1）制定修改计划书：通过研究系统中各个模块、模块的接口和数据库结构使维护人员能从整体观点提出修改计划的方针和策略。

（2）按计划修改程序。

（3）控制修改所带来的副作用。

3．重新验证现有软件系统

软件修改后，在交付用户使用之前需要进行充分的确认和测试，以证实修改后的程序的正确性。常用的方法有：静态确认和计算机确认。

三、软件维护的管理

软件维护不仅是技术性的，同时还需要大量的管理工作与之相配合，才能保证维护工作的质量。虽然不一定要建立正式的维护机构，但确定一个负责任的非正式组织是绝对必要的。这个组织中的人员设置和他们相应的职责如表3-4所示。

表 3-4

人员设置	职　　责
维护管理员	接受维护申请并将申请交给某个系统监督员去作评价
系统监督员	必须熟悉产品，对维护申请作出评价
修改负责人	在系统监督员作出评价后，决定如何进行修改

软件维护的管理流程如图3.56所示。

3.8.3　软件维护的副作用

软件维护的副作用是指因修改软件而造成的错误或其他不希望发生的情况。通常，软件维护的副作用可以分为三类。

1．修改程序的副作用

对源代码的修改有可能引入错误。例如，删去或修改一个子程序、一个语句标号、一个标识符；为改进运行性能所做的修改；改变程序代码的时序关系等都会引入错误。

2．修改数据的副作用

修改数据结构时，有可能造成软件设计与数据结构不匹配，因而导致软件出错。常见的对数据结构的修改有：重新定义局部及全程的常数、重新定义记录和文件的格式、改变

一个数组的大小或改变高层数据结构的大小、改变全局数据、对控制标志或指针重新初始化、重新安排I/O或子程序变量。修改数据而引起的副作用可以通过详细的设计文档加以控制，在此文档中描述一种交叉引用，把数据元素、记录、文件和其他结构联系起来。

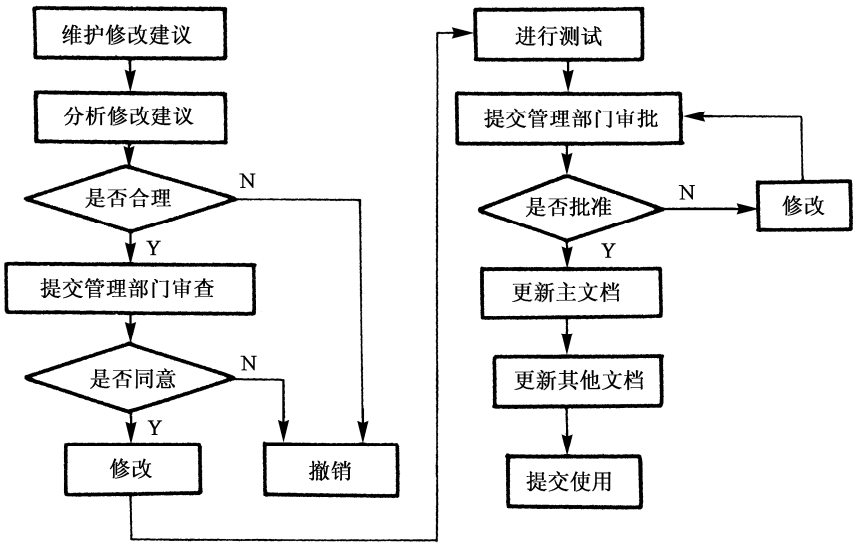


图3.56　软件维护的管理流程

3．文档的副作用

维护工作中，在对数据流、软件结构、模块逻辑等进行修改的同时，还要对相关的技术文档进行相应修改，以保证整个软件配置的一致性。对用户来讲，软件事实上就是文档，如果对可执行软件的改动没有在文档里反映出来，就会产生文档的副作用。例如，对交互输入的顺序或格式进行修改、没有正确地记入文档中等都会产生副作用。

为了控制因修改而引起的副作用，应做到：

（1）按模块把修改分组；

（2）自顶向下地安排所修改模块的顺序；

（3）每次修改一个模块；

（4）对每个已修改的模块，在安排修改下一个模块之前，要确定这个修改的副作用。可以使用交叉引用表、存储映像表、执行流程跟踪等。

小　　结

软件维护是生存周期的最后一个阶段，它的工作是保证软件在长时间内能够正常运行。软件维护可分为四类，即纠错性维护、适应性维护、完善性维护和预防性维护。

软件维护过程中应注意和防止产生软件维护的副作用。

习　题

1．什么是软件危机？产生软件危机的原因是什么？

2．什么是软件工程？什么是软件工程学？

3．什么是软件生存周期？软件生存周期为什么要划分阶段？常用的软件生存周期模型有哪两种？它们的主要特点是什么？

4．需求分析阶段的主要任务是什么？怎样做好需求分析？

5．什么是结构化分析方法（SA方法）？它使用的主要描述工具有哪些？

6．构成数据流图的基本成分有哪些？怎样画数据流图？

7．数据词典有哪些条目？每种条目的作用是什么？

8．软件设计的准则是什么？

9．结构化设计方法（SD方法）的基本思想是什么？它与结构化分析方法（SA方法）衔接使用时，怎样把数据流图转换为软件结构？

10．软件设计分为哪两个步骤？每一步骤的任务是什么？

11．模块基本控制结构有哪些？有哪些表示方法？

12．选择高级程序设计语言的一般原则有哪些？

13．与编程风格有关的问题有哪些？

14．面向对象程序设计方法与面向过程的结构化程序设计方法的主要区别有哪些？

15．在面向对象系统中，什么是对象、类、消息、继承和多态性？

16．面向对象系统的分析和设计的主要步骤有哪些？

17．什么是UML？它在面向对象系统的分析和设计中的主要作用是什么？它包含哪九种视图？其主要功能是什么？

18．软件测试的目标和原则是什么？

19．软件测试阶段分为哪几个步骤？每个步骤与开发各阶段有什么对应关系？

20．什么是白盒法？采用白盒法进行测试时，测试用例覆盖的路径有哪几种？

21．什么是黑盒法？采用黑盒法进行测试时，怎样设计测试用例？

22．为什么要进行软件系统的维护？

23．软件维护的种类有哪些？在维护工作中怎样防止维护的副作用？

第 4 章　数据库技术

随着计算机的应用由科学部门扩展到企业、行政部门，数据处理迅速上升为计算机应用的主要方面。数据库技术是作为数据处理中的一门技术发展起来的。数据库技术是计算机应用软件领域的一个重要分支。

本章首先介绍数据管理技术的发展和数据库中的一些相关概念，然后介绍数据模型、关系数据库、结构化查询语言，最后介绍数据库设计及数据库保护等相关问题。

4.1　数据库技术概论

数据库技术是信息社会的重要基础技术之一，是计算机科学领域中发展最为迅速的分支。数据库技术是一门综合性技术，它涉及操作系统、数据结构、程序设计等方面的知识。

早期的计算机主要用于科学计算，当计算机应用于生产管理、商业财贸、情报检索等领域时，它面对的是数据量惊人的各类数据。为了有效地组织、管理和利用这些数据，就产生了数据库技术。

数据库技术产生于 20 世纪 60 年代末、70 年代初。在计算机的三大主要应用领域（科学计算、数据处理与过程控制）中，数据处理约占 70%的比重。数据库技术主要研究如何存储、使用和管理数据，它是从文件系统的基础上发展起来的，是计算机数据管理技术发展的最新阶段。30 多年来，数据库在理论上、实现技术上均得到很大的发展，不断地有许多数据库管理系统问世，性能越来越好，功能越来越强，这使得计算机应用渗透到各行各业的各类管理工作中。管理信息系统、办公自动化系统、决策支持系统等都是使用了数据库管理系统或数据库技术的计算机应用系统。

4.1.1　数据库的基本概念

在计算机应用中，数据处理和以数据处理为基础的信息系统占据着很大的比重。下面介绍与数据库相关的一些术语。

1．数据（Data）

数据是一种物理符号序列，它的内容是事物特性的反映。数据是对现实世界的事物采用计算机能够识别、存储和处理的方式进行的描述，或者说是计算机化的信息。

数据的概念在数据处理领域中得到不断的发展，目前数据不仅包括数字、字母、文字，而且包含图形、图像、声音等多媒体数据。

数据在空间上的传递就是通常所说的数据通信，而数据在时间上的传递则称为存储。

2．信息

信息是经过加工处理的数据，是人们消化理解了的数据，是数据的具体含义。数据与信息既有联系又有区别。数据是载荷信息的物理符号或称为载体，而信息则是数据的具体

内涵，而且同一数据也可能有不同的解释。数据一般都可以表示某些信息，但并非任何数据都能包含对人们来说有用的信息。信息是抽象的，不随数据设备所决定的数据形式而改变；而数据的表示方式却具有可选择性。

信息是反映客观现实世界的知识，用不同的数据形式可以表示同样的信息。例如，同样一条新闻可能在不同报纸上都有报道，它的文字内容可能不尽相同，但其信息内容可以相同。

3．数据处理

数据处理是指将数据转换成信息的过程。广义地讲，它包括对数据的收集、存储、加工、分类、检索、传播等一系列活动。狭义地讲，它是指对所输入的数据进行加工整理。基本目的是从大量的、已知的数据出发，根据事物之间的固有的联系和规律，通过分析归纳、演绎推导等手段，提取出对人们有价值、有意义的信息，作为决策的依据。数据的加工可以比较简单，也可以相当复杂。简单加工包括组织、编码、分类、排序等；复杂加工可以复杂到使用统计学方法、数学模型等对数据进行深层次的加工。

4．数据库（Database，DB）

数据库是指长期储存在计算机内的、有组织的、可共享的数据集合。数据库中的数据按一定的数据模型组织、描述和存储，具有较小的冗余度、较高的数据独立性和可扩展性，并可为各种用户共享。

5．数据库管理系统（Data Base Management System，DBMS）

数据库管理系统是位于用户和操作系统之间的一层数据管理软件。

数据库在建立、运行和维护时，由数据库管理系统统一管理、统一控制。数据库管理系统使用户方便地定义数据和操纵数据，并能够保证数据的安全性、完整性、多用户对数据的并发使用和故障发生后的系统恢复。

6．数据库系统（Data Base System，DBS）

数据库系统是指在计算机系统中引入数据库后的系统，一般由数据库、数据库管理系统及开发工具、应用软件、数据库管理员和用户组成。

7．数据库管理员（Data Base Administrator，DBA）

数据库的建立、使用和维护等工作需要专门的人员来完成，这些人成为数据库管理员，他们负责定义和存储数据库的内容、实施数据库运行与维护管理。

8．数据字典（Data Dictionary，DD）

在数据字典中存放着对实际数据库各级模式所作的定义，即对数据库结构的描述。它提供了对数据库中数据描述的集中管理手段，对数据库使用的操作都要通过查询数据字典进行。数据字典通常包括数据项、数据结构、数据流、数据存储和处理过程五个部分。其中数据项是数据的最小组成单位，若干个数据项可以组成一个数据结构，数据字典通过对数据项和数据结构的定义来描述数据流、数据存储的逻辑内容。

4.1.2　数据管理技术的发展

计算机数据管理技术经历了由低级到高级的发展过程。数据管理技术随着计算机硬件（主要是外存储器）、软件技术和计算机应用范围的发展而不断发展，多年来大致经历了如下四个阶段：

- 人工管理阶段
- 文件系统阶段
- 数据库系统阶段
- 分布式数据库系统阶段

一、人工管理阶段

20 世纪 50 年代中期以前，计算机主要用于科学计算，一般不需要将数据长期保存，所以数据的管理是靠人工进行的，计算机不保存数据，也没有专门的数据管理软件，只有程序的概念，没有文件的概念，一组数据对应一个应用程序，不同程序的数据之间是相互独立的，彼此无关。数据与应用程序之间的关系如图 4.1 所示。

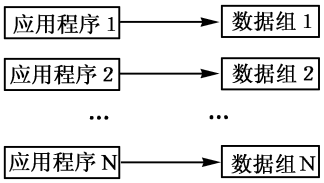


图4.1　人工管理阶段数据与程序的关系

这一时期数据管理的特点如下：

（1）数据与程序没有独立性

一组数据对应一组程序，即数据是面向应用的。这就使得程序依赖于数据，如果数据的类型格式或者数据量、方法、输入/输出方式等改变了，程序必须作相应的修改。而且由于各应用程序处理的数据不会毫无联系，程序之间可能有重复数据。

（2）数据不能长期保存

因为计算机主要用于科学计算，一般不需要保存数据，程序运行时输入数据，计算完成后输出结果。

（3）只有程序的概念，没有文件的概念。

即使有文件，也大多是顺序文件，其他组织方式必须由程序员自行设计。

（4）系统中没有对数据进行管理的软件

数据管理任务包括存储结构、存取方法、输入/输出方式等完全由程序设计人员自负其责。这就给应用程序设计人员增加了很大的负担。

手工管理的主要缺点是数据没有独立性，数据的冗余度比较大。

二、文件系统阶段

20 世纪 50 年代后期至 60 年代中期，计算机开始大量地用于管理。大量的数据存储、检索和维护成为紧迫的需求。在硬件方面，可直接存取的磁鼓、磁盘成为联机的主要外存；在软件方面，出现了高级语言和操作系统。操作系统中的文件系统是专门管理外存的数据管理软件。数据处理方式有批处理，也有联机实时处理。

在这一阶段，程序与数据有了一定的独立性，程序和数据分开存储，有了程序和数据文件的区别。数据文件可以长期保存在外存储器上多次存取，如进行查询、修改、插入、

删除等。数据的存取以记录为基本单位，并出现了多种文件组织形式，如顺序文件、索引文件、随机文件等。

在文件系统的支持下，数据的逻辑结构与物理结构之间可以有一定的差别，逻辑结构与物理结构之间的转换由文件系统的存取方法来实现。数据与程序之间有了一定的独立性，程序只需用文件名访问数据，不必关心数据的物理位置。这样，程序员可以集中考虑数据处理的算法，而不必考虑数据存储的具体细节。

此阶段数据与程序的关系如图 4.2 所示。

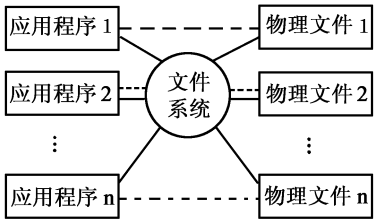


图4.2　文件系统阶段数据与程序的关系

文件系统阶段对数据的管理虽然有了长足的进步，但一些根本性问题还是没有彻底解决，主要表现在以下三个方面：

（1）数据冗余度大

数据冗余是指不必要的重复存储，同一数据项重复出现在多个文件中。在文件系统下，数据文件基本上与各自的应用程序相对应，数据不能以记录和数据项为单位共享。即使有部分数据相同，只要逻辑结构不同，用户必须各自建立自己的文件，这不仅浪费存储空间，增加更新开销，更严重的是，由于不能统一修改，容易造成数据的不一致。

（2）数据缺乏独立性

文件系统中的数据文件是为了满足特定业务领域的专门需要而设计的，服务于某一特定应用程序。由于数据和程序相互依赖，如果改变数据的逻辑结构或文件的组织方法，必须修改相应的应用程序；同样，如果修改应用程序，如改用另一种程序设计语言来编写程序，也将影响数据文件的结构。

文件是无弹性、无结构的数据集合。所谓无弹性是指，由于记录的内部由相应应用程序定义，而不是由系统来统一管理，因此对现有数据文件的应用不易扩充，不易移植，也难以增删数据项以适用新的应用要求。无结构是指各个数据文件之间是独立的，缺乏联系，不能反映现实世界事物之间的联系。

（3）数据无集中管理

除了对记录的存取由文件系统承担以外，文件没有统一的管理机制，其安全性与完整性无法保障。数据的维护任务仍然由应用程序来承担。

这些问题阻碍了数据管理技术的发展，不能满足日益增长的信息需求，这正是数据库技术产生的动力和背景。应用需求和计算机技术的发展促使人们研究一种新的数据管理技术——数据库技术。

三、数据库系统阶段

20 世纪 60 年代后期以来，计算机用于管理的规模越来越大，应用越来越广泛，数据量急剧增长，同时多种应用、多种语言相互共享数据集合的要求越来越强烈。数据库技术应运而生，出现了统一管理数据的专门软件系统——数据库管理系统。目前较流行的数据库管理系统包括 Oracle、DB2、Sybase、SQL Server，它们可以运行于中、小、微型计算机上。

通过数据库管理系统管理大量的数据，不仅解决了数据的永久保存，而且真正实现了数据的方便查询和一致性维护，并且能严格保证数据的安全。从文件系统到数据库系统，标志着数据库管理技术的飞跃。

在数据库管理系统支持下，数据与程序的关系如图 4.3 所示。

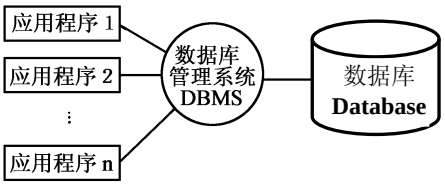


图4.3　数据库系统阶段数据与程序的关系

数据库系统的主要特点如下：

（1）数据结构化

数据结构化是数据库与文件系统的根本区别。

在文件系统中，相互独立的文件的记录内容是有结构的。传统文件的最简单形式是等长同格式的记录集合。这样，可以节省许多存储空间，灵活性也可相对提高。但由于程序与文件独立性较差，所以，文件系统的灵活性仍有局限性。

数据库系统实现整体数据的结构化，不仅描述数据本身，还要描述数据之间的联系。在数据库系统中，不仅数据是结构化的，而且存取数据的方式也很灵活，可以存取数据库中的某一个数据项、一组数据项、一个记录或一组记录。而在文件系统中，数据的最小存取单位是记录，粒度不能细到数据项。

（2）实现数据共享，减小数据冗余

在数据库系统中，对数据的定义和描述已经从应用程序中分离开来，通过数据库管理系统来统一管理。数据的最小访问单位是数据项，既可以按数据项的名称获取数据库中某一个或某一组数据项，也可以存取一条或一组记录。

在建立数据库时，以面向全局的观点来组织库中的数据，而不是像文件系统那样仅仅考虑某一部门的局部应用。数据库中存放全组织（比如企业）通用化的综合性的数据，某一类应用通常仅使用总体数据的子集，这样才能发挥数据共享的优势。

（3）数据独立性高

数据独立性是数据库领域中一个常用术语，包括数据的物理独立性和数据的逻辑独立性。

物理独立性是指用户的应用程序与存储在磁盘上的数据库中数据是相互独立的，也就是说，由 DBMS 管理数据库中的数据在磁盘上的存储方式。应用程序要处理的只是数据的逻辑结构，这样即使当数据的物理存储改变了，应用程序也不用改变。

逻辑独立性是指用户的应用程序与数据库的逻辑结构是相互独立的，也就是说，数据的逻辑结构即使改变了，用户程序也可以不变。

数据与程序的独立是把数据的定义从程序中分离出去，加上数据的存取又由 DBMS 负责，从而简化了应用程序的编写，大大减少了应用程序的维护和修改工作量。

（4）具有统一的数据控制功能

数据库作为多个用户和应用的共享资源，对数据的存取往往是并发的，即多个用户同时使用同一个数据库。数据库管理系统必须提供并发控制、数据的安全性控制和数据的完整性控制等功能。

4. 分布式数据库系统阶段

20 世纪 70 年代后期之前，数据库系统多数是集中式的。分布式数据库系统是数据库技术和计算机网络技术相结合的产物，在 20 世纪 80 年代中期已有商品化产品问世。分布式数据库是一个逻辑上集中、地域上分散的数据集合，是计算机网络环境中各个局部数据库的逻辑集合，同时受分布式数据库管理系统的控制和管理，如图 4.4 所示。

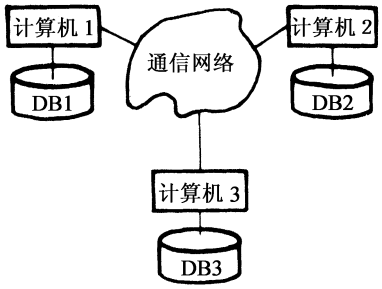


图4.4　分布式数据库系统

分布式数据库系统在逻辑上像一个集中式数据库系统，实际上数据存储在不同地点的计算机上。每个节点有自己的局部数据库管理系统，它有很高的独立性。用户可以由分布式数据库管理系统（网络数据库管理系统），通过网络通信相互传输数据，实现数据的共享和数据的存取。分布式数据库系统有高度透明性，每台计算机上的用户并不需要了解他所访问的数据究竟在什么地方，就像在使用集中式数据库一样。

分布式数据库系统的主要优点是：

（1）局部自主

网络上每个数据库系统都具有独立处理本地事务的能力，而且各个节点之间也能够互相访问，通过相互之间的有效配合，可以对比较复杂的事务进行处理。因此，分布式数据库系统特别适合地理位置上分散的组织机构。例如，银行业务、飞机订票、企业管理等。

（2）可靠性和可用性

分布式数据库系统具有比集中式数据库系统更高的可靠性，在某些节点或者某些通信线路发生故障的情况下仍然能够继续工作。一个局部系统发生故障一般不会导致整个系统停顿或破坏，只要有相应的数据备份可用，数据就可以很快得到恢复。因此，合理的数据冗余能够充分发挥分布式数据库系统的优点。

（3）效率和灵活性

分布式数据库系统分散了工作负荷，缓解了单机容量的压力。数据可以存储在邻近的常用节点上。如果本地的数据子集包含了要查询的全部内容，显然比集中式数据库在全部数据集上查询节省了时间。

（4）系统易于扩展

如要增加新的机构，分布式数据库系统能够在对现有系统影响较小的情况下方便实现扩充。由此，扩大系统规模比集中式系统更加方便、经济、灵活。

4.1.3 DBMS 的主要功能

数据库管理系统作为数据库系统的核心软件，主要目标是使数据成为方便各种用户共享使用的资源，并提高数据的安全性、完整性和可用性。

DBMS 一般具有以下几个方面的功能：

（1）数据库定义功能

提供数据定义语言 DDL（Data Description Language）或者操作命令，以便对各级数据模式进行精确的描述。由此，系统必须包含 DDL 的编译或解释程序。用 DDL 所作的定义将被系统保留在数据字典中，以便在进行数据操纵和控制时使用户可以查阅数据定义以便共享数据库中的数据。

（2）数据操纵功能

数据库管理系统提供数据操纵语言 DML（Data Manipulation Language），可以对数据库中的数据进行更新、插入、修改、删除、检索等操作。不同的 DML 语言的语法格式也不相同，以其实现方法而言可分为两个类型：一类 DML 可以独立交互式使用，不依赖于任何程序设计语言，称为自含型或自主型语言；另一类是宿主型 DML，嵌入到宿主语言中使用，如嵌入 FORTRAN，COBOL，C，PASCAL 等程序设计语言中，在使用高级语言编写的应用程序中，需要调用数据库中的数据时，则要用宿主型 DML 语句来操纵数据。因此，DBMS 必须包含 DML 的编译或解释程序。

（3）数据库运行控制功能

数据库中的数据是提供给多个用户共享使用的，用户对数据进行并发的存取，即多个用户同时使用同一个数据库。因此，DBMS 必须提供以下四方面的数据控制功能：

• 数据的安全性控制

数据安全性控制是对数据库采用的一种保护措施，防止非授权用户存取数据，造成数据泄密或破坏。例如，设置口令、确定用户访问密级和数据存取权限等，系统审查通过后才执行允许的操作。

• 数据的完整性控制

数据完整性是数据的正确性、有效性和相容性。系统应采取一定的措施确保数据有效，确保与数据库的定义一致。例如，当输入或修改数据时，不符合建立数据库时的定义或范围等规定的数据，系统不予接受。然而系统并不能保证所有输入数据绝对准确。例如，将日期 12/08/96 误输成 13/08/96，由于没有 13 月份，系统不予接受，必须重新输入。但是如果误输成 11/08/96，系统则无法控制。

• 并发控制功能

对多用户并发操作加以控制、协调。如果某个用户正在修改某些数据项时，而其他用户同时也在存取这些数据时，就可能导致错误结果。如果两个用户同时修改同一数据时，

先存储的修改就会丢失。数据库管理系统应对要修改的记录采取一定的措施，如加锁，暂时不让其他用户访问，待完成修改存盘之后再开锁。

• 数据库备份与恢复

计算机系统的硬件故障、软件故障、操作员的失误以及故意的破坏也会影响数据库中数据的正确性，甚至造成数据库部分或全部数据的丢失。DBMS 必须具有将数据库从错误状态恢复到某一已知的正确状态的功能，这就是数据库的恢复功能。

上述几方面是一般的 DBMS 所具备的功能。通常在大、中型计算机上实现的 DBMS 功能比较齐全，而在小型机，尤其是微机上实现的 DBMS 功能相应有不同程度的减弱。

4.1.4 数据库系统

一、数据库系统结构

在数据模型中有“型”（type）和“值”（value）的概念。型是指对某一类数据的结构和属性的说明，值是型的一个具体赋值。

模式（schema）是数据库中全体数据的逻辑结构和特征的描述，它仅仅涉及型的描述，不涉及具体的值。模式的一个具体值称为模式的一个实例，同一个模式可以有很多实例。模式是相对稳定的，而实例是相对变动的。模式反映的是数据的结构及其关系，而实例反映的是数据库某一时刻的状态。

可以从多种不同的角度来看数据库系统的结构，从数据库管理系统的角度来看，数据库系统通常采用三级模式结构。在三级模式结构中使用二级映像来实现数据的独立性。

1．数据库系统的三级模式结构

数据库系统的三级模式结构是指数据库系统是由外模式、模式和内模式三级构成，如图 4.5 所示。

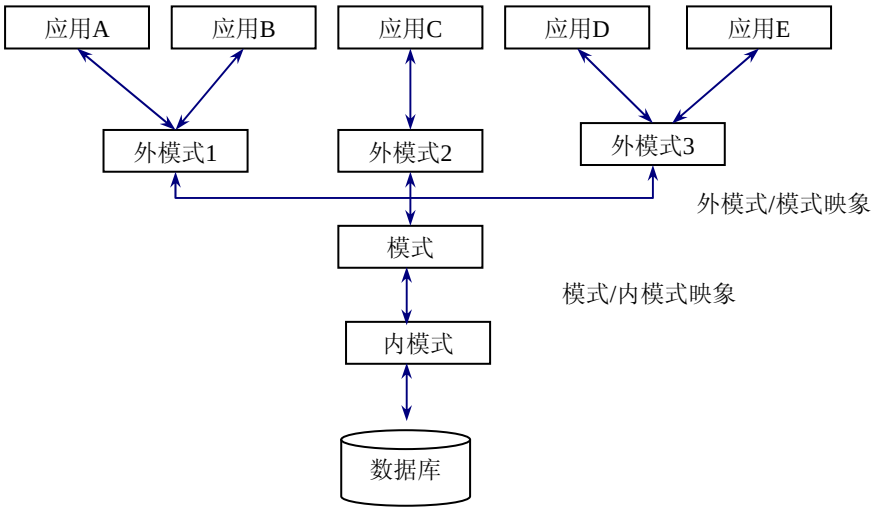


图4.5　数据库系统三级模式结构

模式也称为逻辑模式，是数据库中全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图。它是数据库系统模式结构的中间层，不涉及数据的物理存储细节和硬件环境，与具体应用程序、所使用的应用开发工具及高级程序设计语言无关。

外模式也称为子模式或用户模式，是数据库用户看见和使用的局部数据的逻辑结构和特征的描述，是数据库用户的数据视图，是与某一应用有关的数据的逻辑表示。

内模式也称为存储模式，是数据物理结构和存储结构的描述，是数据在数据库内部的表示方式。

2．数据库的二级映像功能与数据独立性

数据库系统在三级模式之间提供了二级映像：外模式/模式映像和模式/内模式映像，这二级映像保证了数据库系统中的数据的逻辑独立性和物理独立性。

在数据库系统中，对应于一个模式可以有任意多个外模式，对于每一个外模式，都有一个外模式/模式映像，它定义了该外模式与模式之间的对应关系。当模式改变时，由数据库管理员对各个外模式/模式的映像作相应改变，可以使外模式保持不变，从而应用程序不必修改，保证了数据的逻辑独立性。

数据库中只有一个模式，也只有一个内模式。模式/内模式映像是唯一的，它定义了数据全局逻辑结构与存储结构之间的对应关系。当数据库的存储结构改变了，由数据库管理员对模式/内模式映像作相应改变，可以使模式保持不变，从而保证了数据的物理独立性。

二、数据库系统的组成

数据库系统一般由数据库、数据库管理系统及开发工具、应用软件、数据库管理员和用户组成，如图 4.6 所示。

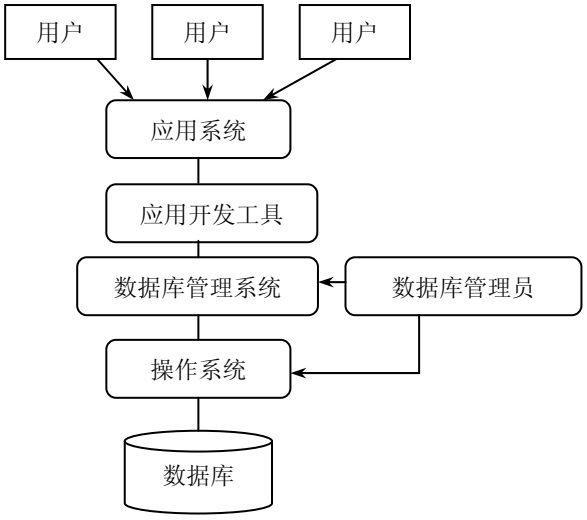


图 4.6　数据库系统组成

数据库是存储数据的集合。数据库管理系统（DBMS）是为数据库建立、使用和维护而设计的软件。它是数据库系统的核心组成部分。除此之外还有支持数据库管理系统运行的操作系统、系统开发软件等。

数据库系统一般需要专人来对数据库进行管理，这些人员称为数据库管理员 DBA。数据库管理员负责数据库系统建立、维护和管理。数据库管理员的职责包括：定义并存储数据库的内容；监督和控制数据库的使用；负责数据库的日常维护，必要时重组或改进数据库。

数据库系统还涉及不同的用户。数据库系统的用户分为两类：一类是最终用户，主要

对数据库进行联机查询或通过数据库应用系统提供的界面来使用数据库。这些界面包括菜单、表格和报表等。另一类是专业用户，即应用程序员，他们负责设计应用系统的程序模块，对数据库进行操作。

小　结

本节主要讨论了以下内容：

数据库的基本概念，包括数据、信息、数据处理、数据库、数据库系统、数据库管理系统。

数据管理技术的发展过程。包括人工管理阶段、文件系统阶段、数据库系统阶段和分布式数据库系统阶段。

数据库管理系统的主要功能包括数据库的定义功能、数据操纵功能、数据库运行控制功能。

数据库系统采用三级模式结构，并使用二级映像保证了数据的独立性。数据库系统组成包括数据库、数据库管理系统及开发工具、应用软件、数据库管理员和用户。

4.2　数 据 模 型

数据库中的数据是有结构的，这些结构反映了事物及事物之间的联系。而数据模型是一种表示实体类型及实体间联系的模型。每一个数据库管理系统都是基于某种数据模型的，它不仅管理数据的值，而且要按照数据模型对数据间的联系进行管理。

数据模型是严格定义的概念的集合。一个数据库系统的数据模型至少包含数据结构、数据操作和完整性规则三部分。其中数据结构是数据模型最基本的部分，它将确定数据库的逻辑结构，是对实体类型和实体间联系的表达和实现。数据操作提供对数据库的操纵手段，主要有检索和更新两大类操作。完整性规则是对数据库有效状态的约束。设计数据模型应该满足三方面的要求：一是能比较真实地模拟现实世界；二是容易为人所理解；三是便于在计算机上实现。一种数据模型要很好地满足这三方面的要求目前还很困难，在数据库系统中针对不同的使用对象和应用目的，可以采用不同的数据模型。在设计数据库系统时，一般有两类不同层次的模型：概念模型和数据模型。概念模型，也称为信息模型，它是按照用户的观点来对数据和信息建模，主要用于数据库设计，例如实体-联系模型，即 E-R 模型。数据模型，主要包括网状模型、层次模型、关系模型和面向对象模型，它是按计算机系统的观点对数据建模，主要用于 DBMS 的实现。

4.2.1　概念模型

从现实生活中事物特性到计算机数据库里数据的具体表示一般要经历三个世界。即现实世界—概念世界—机器世界。有时也将概念世界称为信息世界，将机器世界称为存储世界或数据世界。

一、基本概念

（1）现实世界

事实上，人们管理和使用的对象是处在现实世界中的。在现实世界里，事物与事物之间存在着一定的联系，这种联系是客观存在的。例如，公司和员工，员工在公司中工作，而公司为职员提供工作的环境；应用程序和用户，用户运行和使用应用程序；教师、学生、课程，教师为学生授课，学生选修课程并取得成绩；商品和购买者，购买者购买商品；图书和读者，读者借阅图书；储户与存折，储户将钱存入银行，得到相应的存折等。如果人们管理和使用的对象较多，或者是一些特殊的事物，那么事物与事物之间的联系就可能更为复杂。

（2）信息世界

信息世界是现实世界在人们头脑中的反映，是对客观事物及其联系的一种抽象描述。它不是现实世界的简单映象，而是经过对现实世界的选择、命名、分类等抽象过程而产生的。在信息世界里，对客观事物及其联系的描述一般都涉及实体、实体集、属性、主码和联系等术语。

• 实体（Entry）

客观存在并可相互区别的事物称为实体。实体可以是实际事物，也可以是抽象事件。比如，一个员工、一个公司属于实际事物；教师的一次授课、应用程序的一次运行、储户的一次存款都是比较抽象的事件。

• 实体集（Entry Set）

同一类实体的集合称为实体集。例如，全体员工、全部教师等。

• 属性（Attribute）

实体的具体特性称为属性。例如，学生实体可以用“学号”、“姓名”、“性别”、“出生日期”等属性来描述。属性的具体取值称为属性值，用以刻画一个具体实体的某一方面，这样适当属性的组合就可以具体地表示某一实体。例如，属性组合“980111”、“李萍”、“女”、“12/01/80”在学生名册中就表征了一个具体人。又如，图书实体由属性“总编号”、“分类号”、“书名”、“作者”、“单价”等来描述。属性组合“098765”、“TP298”、“数据库导论”、“Tom”、“12.50”则具体代表一本书。

• 主码（Primary Key）

如果某个属性或属性的值能够唯一地标识出每一个实体，可以将它选作主码。用作标识的主码，也称为主关键字。上例中的“学号”可作为主码，由于可能有重名者存在，“姓名”不宜作主码。图书实体的“总编号”为主码，“分类号”则不宜作为主码。

• 联系（Relation）

实体集之间的对应关系称为联系，它反映现实世界事物之间的相互关联。联系分为两种，一种是实体内部各属性之间的联系。例如，“出生日期”相同的有很多人，但一个学生只能对应一个“出生日期”；另一种是实体之间的联系，例如，一个读者可以借阅若干本图书，同一本书可以相继被几个读者借阅。

（3）机器世界

数据库中的数据一般都是有组织、有结构的，用数据模型来表示这些组织结构。根据数据模型，可以将概念世界中的实体及实体间的联系抽象为便于计算机处理的方式。任何一个数据库管理系统都是基于某种数据模型的。

二、表示方法——E-R模型

E-R 模型即实体-联系模型，是由 P.P.S.Chen 于 1976 年提出的，又称 E-R 图。利用 E-R 图，可以方便地描述概念世界，建立概念模型。它可以进一步转换为任何一种 DBMS 所支持的数据模型。E-R 图一般有实体、属性以及实体间的相互联系三个要素。

• 实体（型）——用矩形框表示，框内标注实体名称。

• 属性——用椭圆形表示，并用连线与实体连接起来。如果属性较多，为使图形更加简明，有时也将实体与其相应的属性另外用列表来表示。

• 实体间的联系——用菱形框表示，框内标注联系名称，并将菱形框与有关实体用连线连接起来，在连线上注明联系类型。

联系类型是指一个实体集合中的每一个实体与另一个实体集合中多少个实体存在联系，并非指一个矩形框通过菱形框与另外几个框画连线。

实体间的联系虽然复杂，但都可以分解到几个实体间的联系，而最基本的是两个实体间的联系。联系一般有如下三种类型：

（1）1:1（一对一联系）

设 A、B 为两个实体集。若 A 中的每个实体最多和 B 中的一个实体有联系，反过来，B 中的每个实体最多和 A 中的一个实体有联系，称 A 对 B 或 B 对 A 是 1:1 联系，如图 4.7 所示。例如，飞机的座位和乘客之间的联系就是一对一联系。

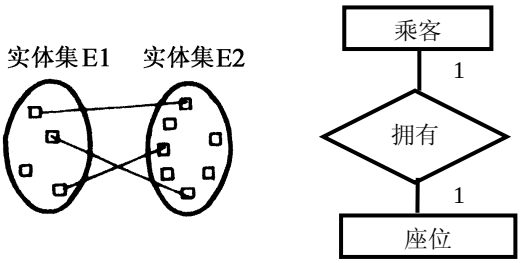


图4.7　一对一联系

（2）1:N（一对多联系）

如果 A 中的每个实体可以和 B 中的几个实体有联系，而 B 中的每个实体最多和 A 中的一个实体有联系，那么 A 对 B 属于 1:N 联系，如图 4.8 所示。这类联系比较普遍，例如，学校与学生是一对多联系，显然一个学校可以有许多学生。又如，学校的系和教师之间也是一对多的联系，因为一个系有许多教师，而一位教师只能属于某一系。

一对一的联系可以看成是一对多联系的一个特殊情况，即 N＝1 时的特例。

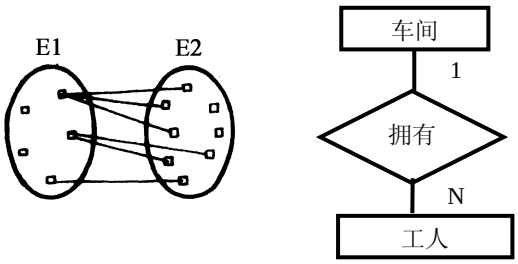


图4.8　一对多联系

（3）M:N（多对多联系）

若 A 中的每个实体可以和 B 中的多个实体有联系，反过来，B 中的每个实体也可以与 A 中的多个实体有联系，称 A 对 B 或 B 对 A 是 M:N 联系，如图 4.9 所示。

例如，一个学生可以选修多门课程，一门课程由多名学生选修。学生和课程间存在多对多联系。又如，商店和顾客之间也是多对多联系，顾客可以到多个商店去购买东西，而一个商店同时又有多名顾客光顾。图书与读者之间也是多对多联系，一位读者可以借阅若干本图书，同一本书可以相继被几个读者借阅。研究人员和科研课题之间是多对多联系，一个人可以参加多个课题，一个课题由多个人参加。

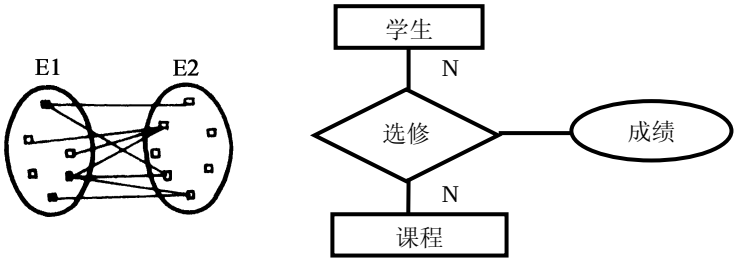


图4.9　多对多联系

当涉及三个实体同时发生联系时，应进行认真分析，使它能真实地反映现实世界。图 4.10 给出了三个实体间联系的 E-R 图。

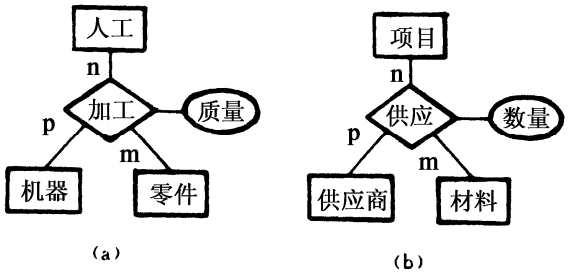


图4.10　三个实体间联系

在图 4.10（a）中，一台机器可以由若干个工人操作加工若干种零件，某一个工人加工某一种零件是在一台机器上完成这道工序的。但一个零件需要多道工序才能完成。因此，零件、机器、工人三个实体之间存在着多对多的联系。

在图 4.10（b）中，一个供应商可以为若干个项目供应多种材料，每个项目可以使用不同供应商的材料，每种材料可由不同供应商提供。因此，供应商、材料、项目三个实体之间存在着多对多的联系。

有时联系也有属性，这类属性不属于任一实体。例如，图 4.9 中“选修”联系可以有属性“成绩”。图 4.10（a）中，“加工”联系有属性“质量”；图 4.10（b）中，“供应”联系有属性“数量”。

E-R 图为抽象地描述现实世界提供了一种有力工具，它所表示的概念模型是各种数据模型的共同基础，进行数据库设计时必然要用到它。

4.2.2 常见数据模型

目前，数据库领域中最常用的数据模型有四种，它们是：

- 层次模型（Hierarchical Model）
- 网状模型（Network Model）
- 关系模型（Relational Model）
- 面向对象模型（Object Oriented Model）

其中层次模型和网状模型统称为非关系模型。非关系模型的数据库关系在 20 世纪 70 年代至 80 年代初非常流行，在数据库系统产品中占据了主导地位，现在已逐渐被关系模型数据库系统取代。20 世纪 80 年代以来，面向对象的方法和技术在计算机各个领域产生了深远影响，也促进了数据库中面向对象数据模型的研究和发展。

下面简要介绍层次模型、网状模型、面向对象模型和关系模型。

一、非关系模型

层次模型和网状模型是非关系模型，它们的共同点是与图论中的图对应。以实体型作为节点，每一个节点代表一个记录类型，节点之间用连线（在图论中称为边）表示联系。任何一个图都可以分解成两个节点和一条边的基本单元，构成数据模型的基本层次联系，如图 4.11 所示。两个实体间的有向边（箭头）代表的基本层次联系表示一对多的联系。通常把表示 1 的实体放在上方，称为父节点；把表示 N 的实体放在下面，称为子节点。具体实现时，父节点的记录类型的一个记录值与子节点的记录类型的多个记录值用指针连接在一起。下面分别对层次模型和网状模型进行介绍。

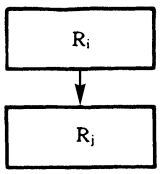


图4.11　数据模型的基本层次联系

1．层次模型

用树形结构表示实体及实体之间联系的模型称为层次模型。它是由若干个基本层次联系组成的一棵树，树的每一个节点代表一个记录类型，如图 4.12 所示。从图（a）可知，系是根节点，相应的树状结构反映了实体与实体之间的结构。该模型的实际存储数据由链接指针来体现联系。图（b）中只给出了学校里一个系的数据，其他系的情况用虚线框表示，并予以省略。由此可见，数据模型反映各组织数据之间的整体逻辑关系。一棵树的具体值是很多棵逻辑结构相同的树。

由图中可以看出，层次模型一般有以下特点：

- 有且仅有一个节点无父节点，此节点为树的根；
- 其他节点有且仅有一个父节点。

层次模型的这些特点，可以非常直接、方便地表达 1:N 联系，但对 M:N 联系的表示相对要麻烦一些，必须设法转换成几个 1:N 的联系才行。

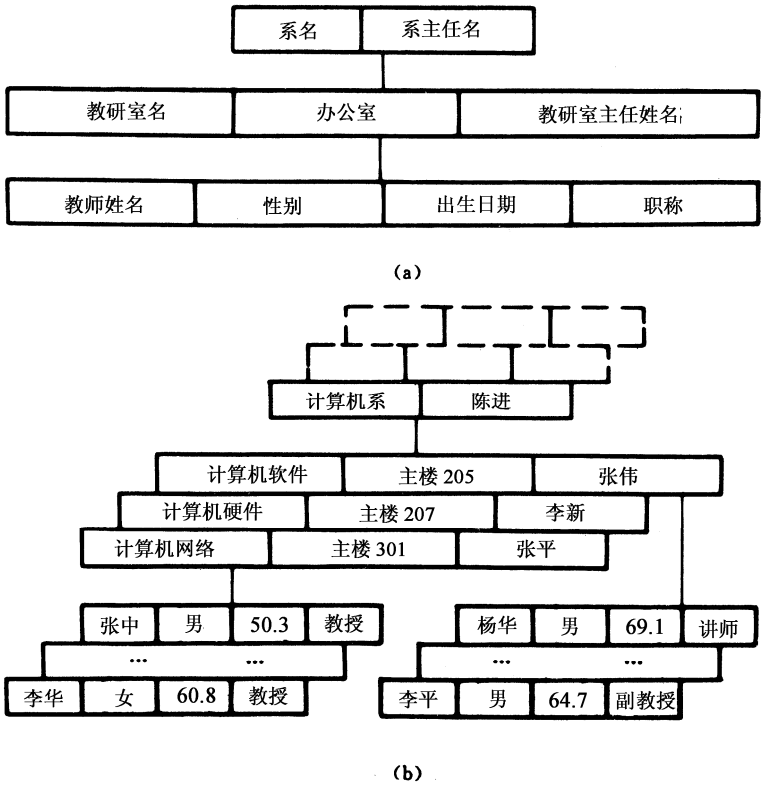


图4.12　层次结构

用支持层次模型的数据库管理系统建立的数据库简称层次数据库。层次数据库管理系统是在数据库技术发展的早期推出的，典型代表是 IBM 公司的大型数据库管理系统 IMS（Information Management System）。

2．网状模型

用网状结构表示实体及实体之间联系的模型称为网状模型。网中的每一个节点代表一个记录类型，联系用链接指针来实现。广义地讲，任何一个连通的基本层次联系的集合都是网状模型。它取消了层次模型的两点限制，允许节点有多于一个的父节点，可以有一个以上的节点没有父节点。图 4.13 给出了一个简单的网状模型，描述的是项目和零件之间的关系。图中假定零件有P_1，P_2，P_3和P_4四种，工程项目有J_1，J_2和J_3三个。

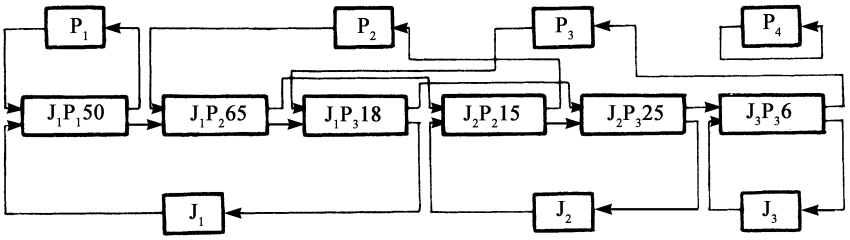


图4.13　网状模型

网状模型比较容易表示 M:N 的联系，记录之间的联系通过指针实现，数据的联系十分密切。网状模型的数据结构在物理上易于实现，效率也较高，但是编写应用程序比较复杂，程序设计人员必须熟悉数据库的逻辑结构。

网状模型和层次模型在本质上是一样的。从逻辑上看，它们都是基本层次联系的集合，用有向边（箭头）表示实体间的联系；从物理上看，它们每一个节点都是一个存储记录，用链接指针来实现记录之间的联系。当存储数据时，这些链接就固定下来了，数据检索时必须考虑存取路径问题；当数据更新时，涉及链接指针的调整，缺乏灵活性，系统扩充相当麻烦。网状模型中的指针更多，纵横交错，从而使数据结构更加复杂。

二、关系模型

关系模型是以二维表格结构作为基础的。与网状模型和层次模型相比，关系模型更容易为初学者所接受。

关系模型是由若干个关系模式组成的集合。每个关系模式实际上是一张二维表。

1．二维表

关系模型是用二维表的形式来表示实体和实体间联系的数据模型。从用户观点来看，关系的逻辑结构是一个二维表，在磁盘上以文件形式存储。表 4-1、表 4-2 和表 4-3 分别代表零件、项目和项目-零件（供应商）三个关系。

关系模型和网状、层次模型的最大差别是关系模型用主码而不是用指针来表示和实现实体间联系。表格简单、易懂，用户只需用简单的查询语句就可以对数据库进行操作，并不涉及存储结构、访问技术等细节。但与网状、层次模型比起来实现较复杂，效率也比较低，因为很多任务都由系统承担。随着硬件的发展，效率已不再成为问题。关系模型是比较数学化的模型，要用到离散数学、集合论等知识，是一种具有严格设计理论的模型，它已成为几种数据模型中最主要的模型。自 20 世纪 80 年代以来，新推出的数据库管理系统几乎都支持关系模型。早期的许多层次和网状模型系统的产品也加上了关系接口。

表 4-1　零件关系

零件编号	零件名称	颜色	重量
P001	螺钉	蓝色	14
P002	螺栓	绿色	17
P003	螺帽	红色	12
P004	螺钉	红色	19

表 4-2　项目关系

项目编号	项目名称	日期
J01	项目 A	95.1
J02	项目 B	97.5
J03	项目 C	97.9

表 4-3　项目-零件关系

项目编号	零件编号	数量
J01	P001	18
J02	P002	25
J03	P003	20
J02	P004	12
J03	P002	21
J01	P004	30
J03	P004	35

2．基本术语

• 关系：一个关系就是一张二维表，每个关系有一个关系名。在计算机里，一个关系可以存储为一个文件。

• 元组：表中的行称为元组。一行为一个元组，对应于存储文件中的一个记录值。例如，表 4-1 的零件关系有四个元组；表 4-2 的项目关系包括三个元组。

• 属性：表中的列称为属性，每一列有一个属性名。这里的属性与前面讲的实体属性相同，属性值相当于记录中的数据项或者字段值。

• 域：属性的取值范围，即不同元组对同一个属性的取值所限定的范围。例如，逻辑型属性只能从逻辑真或逻辑假两个值中取值。

• 分量：元组中的一个属性值

• 主码：属性或属性组合，其值能够唯一地确定一个元组。例如，零件关系中的零件编号；项目关系中的项目编号。

• 目或度：关系模式中属性的数据项数目是关系的目或度。如零件关系是四元关系，项目关系是三元关系。

• 关系模式：对关系的描述称为关系模式，格式为

关系名（属性名 1，属性名 2，…，属性 N）

一个关系模式对应一个关系文件的结构。例如，上述三个关系可描述为：零件（零件编号，零件名称，颜色，重量）；项目（项目编号，项目名称，启动日期）；项目-零件（项目编号，零件编号，数量）分别描述了三个关系模式。

关系模型中基本数据结构就是二维表，因此不用像层次、网状模型那样要用指针来表示记录之间的联系。在关系模型中，记录之间的联系是通过主码来体现的。例如，要查询项目 J03 所用的零件是什么颜色、重量是多少，首先要在项目-零件关系中找到零件号 P003，然后在零件关系中找到对应零件的颜色和重量，分别是红色和 12。在上述查询过程中，主码“零件号”起到了连接两个关系的作用。

由此可见，关系模型中的各个关系模式不应当孤立起来，不是随意拼凑的一组二维表，它必须满足一定的要求。

3．关系模型的优缺点

（1）关系模型与非关系模型不同，它是建立在严格的数学概念的基础上。

（2）模型概念单一。在关系模型中，无论实体本身还是实体间的联系均用关系表示。对数据的检索结果也是关系（即表）。所以，其数据结构简单、清晰，用户易懂易用。

（3）关系模型的存取路径对用户透明，从而具有更高的数据独立性、更好的安全保密性，也简化了程序员的工作和数据库开发建立的工作。

当然，关系数据模型也有缺点，其中最主要的缺点是，由于存取路径对用户透明，查询效率往往不如关系数据模型。因此，为了提高性能，必须对用户的查询请求进行优化，增加了开发数据库管理系统的负担。

三、面向对象模型

虽然在数据处理领域中普遍使用关系模型数据库，但是随着计算机技术的飞速发展，新的应用领域不断出现，它们对数据处理技术的要求也比一般事务处理环境复杂得多。在很多领域中，一个对象由多个属性来描述，而其中某些属性本身又是另一个对象。例如，计算机辅助设计 CAD 的图形数据；多媒体应用的图像、声音和文档等。应用领域的扩展对数据库技术提出了许多新的要求。面向对象数据库是面向对象的概念与数据库技术相结合的产物。

在面向对象模型中，最基本的要素是对象和类。

（1）对象：现实世界中实体的模型化，它和记录相似，但更加复杂。每个对象有一个唯一的标识符，并把一个状态和一个行为封装在一起。

（2）类：每个类由两部分组成，其一是对象类型，其二是对这个对象类型进行的操作方法。对象的状态是描述该对象属性值的集合，对象的行为是对该对象操作的集合。类的定义使得共享同样属性集合和方法集合的对象组合成一类。类有嵌套结构，即类的对象类型（属性）可以是基本类型，如整型、实型、字符串型，也可以是一般类，具有自己的对象类型和方法。

（3）类层次：一个系统中所有的类和子类组成一个树状的类层次，一个类继承其直接或间接祖先的所有属性和方法。定义新的类时，只需定义特殊的对象类型和方法即可，有利于实现扩充。

由于面向对象模型中不仅包括描述对象状态的属性集，而且包括类的方法及类层次，具有更加丰富的表达能力。因此，面向对象的数据库比层次、网状、关系模型使用方便。但由于模型复杂，系统实现起来难度大。当前已出现了一些面向对象的数据库系统，但尚未被广泛使用。

4.3　关系数据库

关系数据库应用数学方法来处理数据库中的数据。最早将这类方法用于数据处理的是 1962 年 Codasyl 发表的“信息代数”，之后有 1968 年 David Child 在 7090 机上实现的集合论数据结构，但系统地、严格地提出关系模型的是美国 IBM 公司的 E.F.Codd，他在美国计算机学会会刊《Communication of the ACM》上发表了题为“A Relational Model of Data for

Shared Data Banks”的论文，开创了数据库系统的新纪元。以后，他连续发表了多篇论文，奠定了关系数据库的理论基础。

关系数据库系统有如下特点：

（1）关系数据库系统使用方便。关系模型只要求程序员和用户了解关系（即二维表）。关系运算有理论基础，相当成功，也比较容易掌握。关系数据库语言适用于各类计算机用户。正是由于关系数据库使用的方便性，使得关系数据库取代了网状、层次数据库。

（2）关系模型中关系是以文件的形式存储的，数据间联系有时也用指针实现，但只是辅助手段。数据联系主要是由选择、投影和连接等运算实现，因而效率较低，这是关系数据库较迟进入商用的一个原因。但随着计算机硬件的飞速发展，效率已赶上了网状和层次数据库。

（3）非过程性查询。面向过程语言是指程序编制时需说明某个访问应该“怎么做”的计算机语言，如 PASCAL、C 一类的语言，网状模型和层次模型的数据操纵语言都属于面向过程语言。它们对数据库的操作在程序里要反映出“做什么”、“怎么做”，而且一次操作只能针对一个元组，即面向过程语言常由用户指出对数据的访问路径。而关系数据库的某个操作，一般只要说明“做什么”就可以了，至于“怎么做”则是由系统去完成，一次的操作也可以针对一个元组集，查询的路径由系统决定。这也就是非过程化语言的主要特征。

关系数据库系统是支持关系模型的数据库系统。关系模型由关系数据结构、关系操作集合和关系完整性约束三部分组成。

4.3.1 关系数据结构

在关系模型中，无论是实体还是实体之间的联系均由单一的结构类型即关系（表）来表示。下面介绍与此相关的一些基本概念。

1．域（Domain）

定义 1 域是一组具有相同数据类型的值的集合。

例如，自然数、整数、实数、字符串集合、{0，1}，都可以是域。

2．笛卡尔积（Cartesian Product）

定义 2 给定一组域D_1，D_2，…，D_n，这些域中可以有相同的。这组域的笛卡尔积为

$$D_1\times D_2\times\cdots\times D_n = \{(d_1, d_2, \cdots, d_n) \mid d_i \in D_i,\ i=1, 2, \cdots, n\}$$

其中，每一个元素（d_1，d_2，…，d_n）叫做一个n元组。元素中的每个值d_i叫做一个分量。

笛卡尔积可表示为一个二维表。表中的每行对应一个元组，表中的每列对应一个域。

3．关系（Relation）

定义 3 $D_1\times D_2\times\cdots\times D_n$的子集叫做在域$D_1$，$D_2$，…，$D_n$上的关系，表示为

$$R(D_1, D_2, \cdots, D_n)$$

其中，R 表示关系的名字，n 是关系的目或度。关系中的每个元素是关系中的元组，用 t 表示。

关系是笛卡尔积的有限子集，所以，关系也是一个二维表，表的每行对应一个元组，表的每列对应一个域。由于域可以相同，为了加以区分，必须对每列起一个名字，称为属性。若关系中的某一属性组的值能唯一地标识一个元组，则称该属性组为候选码（Candidate

Key）。若一个关系有多个候选码，则选定其中一个为主码（Primary Key）。主码的诸属性称为主属性（Prime Attribute）。不包含在任何候选码中的属性称为非码属性（Non-key Attribute）。

关系可以有三种类型：基本关系（通常又称为基本表或基本表）、查询表和视图表。基本表是实际存在的表，是实际存储数据的逻辑表示。查询表是查询结果对应的表。视图表是由基本表或其他视图表导出的表，是虚表，不对应实际存储的数据。

基本关系具有以下六条性质：

（1）列是同质的，即每一列中的分量是同一类型的数据，来自同一域；

（2）不同的列可出自同一个域，称其中的每一列为一个属性，不同的属性要给予不同的属性名；

（3）列的顺序无所谓，即列的次序可以任意交换；

（4）任意两个元组不能完全相同；

（5）行的顺序无所谓，即行的次序可以任意交换；

（6）分量必须取原子值，即每一个分量都必须是不可分的数据项。

关系模型要求关系必须是规范化的，即要求关系模型必须满足一定的规范条件。这些规范条件中最基本的一条就是：关系的每一个分量必须是一个不可分的数据项。规范化的关系简称为范式（Normal Form）。上述规范条件即是第一范式。

4.3.2 关系的完整性

关系模型的完整性规则是对关系的某种约束条件。关系模型中可以有三类完整性约束：实体完整性、参照完整性和用户定义完整性。以学生选课管理为例来说明关系的完整性。

例 1 定义学生、课程、学生与课程之间的三个关系表示如下：

学生（学号，姓名，性别，出生日期）

课程（课程号，课程名，学分）

选修（学号，课程号，成绩）

在上述关系中，用下划线表示主码。

1．实体完整性

规则 1 实体完整性规则

若属性 A 是基本关系 R 的主属性，则属性 A 不能取空值。

实体完整性规则规定基本关系的所有主属性都不能取空值，而不仅是主码整体不能取空值。在例 1 学生选修关系中，“学号、课程号”为主码，则“学号”和“课程号”两个属性都不能取空值。

2．参照完整性

定义 3 设F是基本关系R的一个或一组属性，但不是关系R的码。如果F与基本关系S的主码K_s相对应，则称F是基本关系R的外码，并称基本关系R为参照关系，基本关系S为被参照关系。

规则 2 参照完整性规则

若属性（或属性组）F是基本关系R的外码，它与基本关系S的主码K_s相对应（基本关系R和S不一定是不同的关系），则对于R中每个元组在F上的值必须为：

• 或者取空值（F 的每个属性值均为空值）；

• 或者等于 S 中某个元组的主码值。

在例 1 中，选修关系的“学号”属性与学生关系的主码“学号”相对应，“课程号”属性与课程关系的主码“课程号”相对应，因此，“学号”和“课程号”属性是选修关系的外码，学生关系和课程关系是被参照关系，选修关系是参照关系。这样，选修关系中的“学号”和“课程号”可以取两类值：空值或相应被参照关系中已经存在的主码值。但由于“学号”和“课程号”是选修关系的主属性，按照实体完整性规则，它们均不能取空值。所以，选修关系中的“学号”和“课程号”属性实际上只能取相应被参照关系中已经存在的主码值。

3．用户定义的完整性

用户定义的完整性就是针对某一具体关系数据库的约束条件。它反映某一具体应用所涉及的数据必须满足的语义要求。例如选修关系中的“成绩”属性的取值范围常在 0~100 之间。关系模型应提供定义和检验这类完整性的机制，以便用统一的系统的方法处理它们，而不需要由应用程序来实现这一功能。

4.3.3 关系代数

任何一种运算都是将一定的运算符作用于一定的运算对象上，得到预期的运算结果。所以，运算对象、运算符、运算结果是运算的三大要素。

关系代数的运算对象是关系，运算结果也是关系。关系运算是以集合代数为基础发展起来的。从集合论的观点来定义关系，关系是一个目为 K 的元组集合，即这个关系有若干个元组，每个元组有 K 个属性值。对关系数据库进行查询时，需要找到用户感兴趣的数据，这就需要对关系进行特定的运算操作。关系的基本运算有两类：一类是一般的集合运算（并、差、交等），另一类是专门的关系运算（选择、投影、连接等）。

一、一般集合运算

（1）并（Union）

设有两个关系 R 和 S，它们具有相同的目。R 和 S 的并是由属于 R 或属于 S 的元组组成的集合。

（2）差（Difference）

设有两个关系 R 和 S，它们具有相同的目。R 和 S 的差是由属于 R 但不属于 S 的元组组成的集合。

（3）交（Intersection）

设有两个关系 R 和 S，它们具有相同的目。R 和 S 的交是由既属于 R 又属于 S 的元组组成的集合。

二、专门的集合运算

1．选择（Selection）

从关系中找出满足给定条件的元组的操作称为选择，其中条件是以逻辑表达式给出的。选择运算是从行的角度进行的运算，即水平方向抽取元组。经过选择运算得到的结果元组

可以形成新的关系，其关系模式不变，但其中元组的数目不会多于原来的关系中元组的个数，它是原关系的一个子集。

不同的数据库管理系统所用的语言语法格式各不相同，一般以下列方法表示：

SELECT 〈关系名〉 WHERE 〈条件〉

例 2 从表 4-3 所示的“项目-零件”关系中，找出项目 J02 所用的零件。

SELECT 项目-零件 WHERE 项目编号＝"J02"

查询结果包括两个元组。

例 3 从表 4-3 所示的“项目-零件”关系中，找出项目 J02 或项目 J03 所用的零件。

SELECT 项目-零件 WHERE 项目编号＝"J02" or 项目编号＝"J03"

查询结果包括四个元组。

2．投影（Projection）

从关系模式中选取若干属性组成新的关系称为投影。投影运算是从列的角度进行的运算，相当于对关系进行垂直分解。经过投影运算可以得到一个新关系，其关系模式所包含的属性个数往往比原关系少，或者属性的排列顺序不同。因此，运算提供了垂直调整关系的手段。投影运算可以使用如下方法表示：

PROJECT 〈关系名〉（属性 1，属性 2，…）

例 4 设有“读者”关系，读者（借书证号，姓名，性别，单位，职称，地址）。按照单位在前、姓名在后的次序列出读者名单。

PROJECT 读者（单位，姓名）

此运算结果的元组数目不变，但减少了列的数目，而且列的顺序也颠倒过来了。

例 5 从表 4-1 的“零件”关系中找出所有可能的颜色。

PROJECT 零件（颜色）

查询结果包括三个元组。

投影之后不仅减少了某些列，也可能减少了某些元组。因为取消了某些属性之后，其余属性可能有相同的值，造成重复元组，而投影运算将删除完全相同的元组。

3．连接（Join）

选择和投影运算都属于单目运算，它们运算的对象只是一个关系，连接运算是二目运算，需要两个关系作为操作对象。

连接是将两个关系模式组合成一个属性更多的关系模式，运算过程是通过连接条件控制的，生成的新关系中包含满足连接条件的元组。在连接条件中一般都含有不同关系模式中的公共属性，或者具有相同语义、可以进行比较的属性。连接是由两个关系的组合生成一个新关系。

连接的表示方法如下：

JOIN 〈关系 1〉 AND 〈关系 2〉 WHERE 〈条件〉

连接运算中有两种最为重要也最为常用的连接：等值连接和自然连接。在连接条件中，如果使用“=”的连接运算，则称为等值连接。自然连接是一种特殊的等值连接，要求两个关系中进行比较的分量必须是相同的属性组，并且去掉运算结果中重复的属性列。

例 6 对表 4-1 和表 4-3 所示的关系进行连接运算，以便查看项目编号、项目使用的零件以及相应零件的情况。

JOIN 项目-零件 AND 零件 WHERE 项目-零件（零件编号）＝零件（零件编号）

运算结果如表 4-4 所示。

表 4-4

项目编号	零件编号	数量	零件名称	颜色	重量
J01	P001	18	螺钉	蓝色	14
J02	P002	25	螺栓	绿色	17
J03	P003	20	螺帽	红色	12
J02	P004	12	螺钉	红色	19
J03	P002	21	螺栓	绿色	17
J01	P004	30	螺钉	红色	19
J03	P004	25	螺钉	红色	19

例 7　查看使用“螺钉”的项目编号以及零件的使用情况。

JOIN 项目-零件 AND 零件　WHERE　项目-零件（零件编号）＝零件（零件编号）AND 零件（零件名称）＝"螺钉"

运算结果如表 4-5 所示。

表 4-5

项目编号	零件编号	数量	零件名称	颜色	重量
J01	P001	18	螺钉	蓝色	14
J02	P004	12	螺钉	红色	19
J01	P004	30	螺钉	红色	19
J03	P004	25	螺钉	红色	19

连接运算可能会花费大量的系统时间，耗费大量的系统资源，尤其是包括许多元组的连接运算更是如此。设关系 R 和 S 分别有 M 和 N 个元组，R 与 S 的连接过程要访问 M×N 个元组。先从 R 关系中的第一个元组开始，依次与 S 关系的每个元组进行比较，符合条件的两元组首尾相连纳入新关系，这一轮共进行 N 次比较；再用 R 关系的第二个元组对 S 关系的每个元组进行比较扫描……这样的比较运算共需进行 M 次。如果 M=1000，N=1000，那么 R、S 的连接过程需要进行 100 万次数据的比较。由此可见，查询时应考虑优化，以便提高查询效率。如果有可能，应当首先进行选择运算，使关系中元组个数尽量少；之后，能进行投影运算的用投影运算进行处理，使关系中属性个数较少，然后再进行连接。

例 8　设有三个关系

学生（学号，姓名，性别，民族，出生日期，系别，籍贯，住址）

课程（课程编号，课程名称，课程类型，学分，学时）

成绩（学号，课程编号，分数）

查询成绩 90 分以上的学生姓名：

R1 = SELECT 成绩 WHERE 分数＞＝90

R2 = PROJECT 学生（学号，姓名）

R3 = JOIN R1 AND R2 WHERE R1（学号）＝R2（学号）

如果用以下方式查询，效率则低得多：

S1 = JOIN 成绩 AND 学生 WHERE 成绩（学号）＝学生（学号）

S2 = SELECT S1 WHERE 分数＞＝90

例 9 查看选修了“操作系统”课程的学生详细情况。

S1 = SELECT 课程 WHERE 课程名称＝"操作系统"

S2 = PROJECT S1（课程编号，课程名称）

S3 = JOIN 成绩 AND S2 WHERE 成绩（课程编号）＝S2（课程编号）

S3 = JOIN 学生 AND S3 WHERE 学生（学号）＝S3（学号）

通过上述例子可见，不同关系中的公共属性是自然连接的“纽带”。利用关系的投影、选择和自然连接可以方便地构造新的关系。

小　　结

这一节里，主要讨论了以下内容：

• 关系数据结构：在关系数据库中只有单一的数据结构——关系，数据库中的实体与实体之间的联系均可用关系加以描述。

• 关系的完整性：实体完整性、参照完整性和用户定义的完整性。

• 关系代数：关系数据库的数学基础是关系代数，是集合运算，主要介绍了一般集合运算：并、差、交；专门的集合运算：选择、投影和连接。

4.4　SQL 结构化查询语言

SQL（Structure Query Language）语言是一种通用的、功能极强的关系数据库语言，是结构化查询语言，它实际上包括查询、定义、操纵和控制四个部分。1974 年 Boyce 和 Chamberlin 提出了 SQL 语言，并在大型计算机上实现它。由于 SQL 语言具有语言简洁、方便实用、功能齐全等突出的特点，很快得到推广和应用，随着关系数据库的流行，SQL 语言在计算机界和广大用户中得到了认可。1986 年 10 月美国国家标准局（ANSI）的数据库委员会颁布了 SQL 语言的美国标准，作为标准关系数据库语言。1987 年 6 月，国际标准化组织（ISO）采纳 SQL 语言为关系数据库语言的国际标准。我国也在 1990 年制定了 SQL 语言标准。实际系统中实现的 SQL 语言往往对标准版本有所扩充。

当前各种数据库管理系统几乎都支持 SQL 语言，或者提供 SQL 语言的接口，从而使得无论是大型机、中型机，或者小型机以至微机上的各种数据库系统都有了共同的存取语言标准接口，为更广泛的数据共享开创了乐观的前景。例如，Oracle、DB2、Sybase、SQL Sever 等都支持 SQL 语言。

SQL 语言的使用方法一般有两种，一种是以与用户交互的方式联机使用，称为交互式 SQL 语言；另一种是作为子语言嵌入到其他程序设计语言中使用，称为嵌入式 SQL 语言，

这种方式适合于程序设计人员用高级语言编写应用程序，通过嵌入在宿主语言中的 SQL 语句对数据库中的数据进行存取。两种方式的语法结构基本一致。由于嵌入式 SQL 语言涉及宿主语言，所以，本书仅介绍交互式 SQL 语言。

4.4.1 SQL 语言概述

在 SQL 数据库中使用的有些术语和传统的关系数据库术语不同。在 SQL 语言中，关系模式称为“基本表”，存储模式称为“存储文件”，子模式称为“视图”，“元组”称为“行”，属性称为“列”。

（1）一个 SQL 数据库是表（Table）的汇集，它用一个或若干个 SQL 模式定义。

（2）一个 SQL 表由行集构成，一行是列的序列，每列对应一个数据项。

（3）一个 SQL 数据库有基本表（Base Table）和视图（View）。基本表是实际存储在数据库中的表，而视图是若干个基本表或其他视图构成的集合的子集。

（4）一个基本表与一个存储文件相对应，一个存储文件与外部存储器上一个物理文件相对应。在大型机上以及一些新的关系 DBMS 产品中，一个基本表可以放在多个存储文件中，而一个存储文件也可存放多个基本表。

（5）用户可以用 SQL 语句对基本表和视图进行查询等操作。

（6）SQL 用户可以是应用程序用户，也可以是终端用户。

下面介绍常用的 SQL 语言。

4.4.2 数据定义

数据定义是指定义数据库的逻辑结构，包括定义基本表、视图、索引等，也包括对它们的相应修改和删除操作等。

一、定义、修改和删除基本表

用 SQL 可以定义、修改和删除基本表。定义一个基本表相当于建立一个新的关系模式，也就是定义了一个关系的基本框架。此时基本表中还没有数据。

1．定义基本表

定义基本表就是对基本表的名称，以及基本表中的各个字段以及数据类型作出具体规定。

使用 CREATE TABLE 语句定义基本表，其格式如下：

```
CREATE TABLE <表名>（<列名><数据类型>[列级完整性约束条件]
                    [，<列名><数据类型>[列级完整性约束条件]]…
                    [，<表级完整性约束条件>]）；
```

其中，<表名>是所要定义的基本表的名称，它可以由一个或多个属性（列）组成。创建表的同时通常还可以定义与该表有关的完整性约束条件。

例 10 创建“学生”基本表。

```
CREATE TABLE 学生
        （学号 char（6） not null，
        姓名 char（8） not null，
```

系别 char（10），
性别 char（2），
籍贯 char（20），
住址 char（20）
）；

执行上述语句将在数据库中建立一个名为“学生”的基本表，它有六个列，也就是关系中指的属性，分别是“学号”、“姓名”、“系别”、“性别”、“籍贯”和“住址”，它们都是字符型，只是长度不一样，“学号”长度为 6，“姓名”长度为 8，“系别”长度为 10，“性别”长度为 2，“籍贯”长度为 20，“住址”长度为 20。

SQL 语言的常见数据类型如表 4-6 所示。

表 4-6　SQL 语言的常见数据类型

数据类型	含义说明
INTEGER	4 字节整数
SMALLINT	2 字节整数
DECIMAL（m，n）	十进制数，共 m 个数字位，小数点后有 n 位，可带正负符号
FLOAT	4 字节浮点数，数据大小从 0.1e−307 到 0.9e+308，以指数形式表示
CHAR（n）	固定长度的字符串，长度为 n
VARCHAR（n）	可变长度的字符串，长度最大为 n
DATATIME	8 字节日期型数据

SQL 语言支持空，即 NULL 的概念，空值是不知道或不能用的值，如果基本表中某一列的定义不允许出现空值，则要加上 NOT NULL。例中的 NOT NULL 指出“学号”和“姓名”两列在输入数据时不允许出现空值，而其他字段可以为空。

2．修改基本表

SQL 语言用 ALTER TABLE 语句修改基本表，其格式如下：

ALTER TABLE <表名>
[ADD <新列名><数据类型>[完整性约束]]
[DROP <完整性约束名>]
[MODIFY <列名><数据类型>];

其中，<表名>是要修改的基本表；ADD 子句用于增加新列和新完整性约束条件；DROP 子句用于删除指定的完整约束条件；MODIFY 子句用于修改原有的列定义，包括修改列名和数据类型。

例 11　为“学生”基本表增加“班级”字段。

ALTER TABLE 学生 ADD （班级 CHAR（6））；

新增加的列将处于表的最右面，而且这列不能指定为 NOT NULL，应允许空值，因为如果基本表中原来已经有数据，各记录的新增列必然是空值，以后可以用更新语句来修改。

3．删除基本表

当基本表没有用时，可以用 DROP TABLE 语句将基本表删除，其格式如下：

DROP TABLE <表名>；

基本表定义一旦删除，表中的数据、其上的索引以及以基本表为基础所建立的所有的视图将全部被删除，并释放出所占用的存储空间。

例 12　删除“学生”基本表。

DROP TABLE 学生；

二、定义和删除视图

数据库系统中一般都有若干个基本表。在基本表中保存着多个用户共享的数据。某一个具体应用可能只使用其中一部分数据，基本表的字段有时也不能直接满足用户的具体要求。这时，可以从一个或几个基本表以及现有的视图导出适合具体应用的视图。用户对视图的查询与基本表一样。基本表和视图都是关系，但视图是虚表，不对应于一个存储的数据文件，因此通过视图对数据的修改也要受到一些限制。

显然，建立和使用视图，可以简化查询语句。

通过对用户授以对视图的访问权限可以限制不同用户的查询范围。未授权的用户不能访问任何基本表和视图。将视图授权给用户，可以避免暴露全部的基本表。

1．定义视图

在 SQL 语言中，用 CREATE VIEW 语句建立视图，其格式如下：

CREATE VIEW <视图名>[（<列名>[，<列名>]…）]

AS <子查询>[WITH CHECK OPTION]；

其中，子查询可以是任意复杂的 SELECT 语句，但通常不允许含有 ORDER BY 子句和 DISTINCT 短语。WITH CHECK OPTION 表示对视图进行 UPDATE、INSERT 和 DELETE 操作时要保证更新、插入或删除的行满足视图定义中的子查询中的条件表达式。

例 13　建立计算机系的学生视图，视图名为 V_STUDENT。

```
CREATE VIEW V_STUDENT AS
    SELECT 学号，姓名，性别，班级
        FROM 学生
        WHERE 系别="计算机系"；
```

由于建立的视图的字段名和 SELECT 子句中所列出字段相同，所以可以省略不写。

例 14　建立各班选修课程情况的简单统计视图，名称为 V_SELECTION。

```
CREATE VIEW V_SELECTION（班级，选课人数）
    SELECT 班级，COUNT（DISTINCT 学号）
        FROM 学生，选课
        WHERE 学生.学号=选课.学号
        GROUP BY 班级；
```

这样，对该视图的直接查询便可得到各班级当前选修课程的情况。由于学生和选课两张基本表可能不断地发生变化，因此每次对视图的查询可能结果不同。

2．删除视图

删除视图语句是 DROP VIEW，其格式如下：

DROP VIEW <视图名>；

视图被删除之后，它的定义和在它基础上所建立的其他视图将自动被删除。

例 15　删除名为 V_STUDENT 的视图。

DROP VIEW V_STUDENT；

3．视图的作用

视图最终是定义在基本表之上的，对视图的一切操作最终也要转换为对基本表的操作，合理地使用视图会带来以下好处：

（1）视图能够简化用户的操作

视图机制使用户将注意力集中在所关心的数据上。如果这些数据不是直接来自基本表，则可以通过定义视图，使数据库看起来结构简单、清晰，并且可以简化用户的数据查询操作。

（2）视图使用户能以多种角度看待同一数据

视图机制能使不同的用户以不同的方式看待同一数据，当许多不同种类的用户共享一个数据库时，这种灵活性非常重要。

（3）视图对重构数据库提供了一定程度的逻辑独立性

（4）视图能够对机密数据提供安全保护

有了视图机制，就可以在设计数据库应用系统时，对不同的用户定义不同的视图，使机密数据不出现在不应看到这些数据的用户的视图上，这样视图机制就自动提供了对机密数据的安全保护功能。

三、建立和删除索引

基本表或视图中可能存放有大量的记录，此时查找满足条件的记录可能要花很长时间，为了提高数据的检索速度，可以根据实际应用情况为一个基本表建立若干个索引。

1．建立索引

在 SQL 语言中，建立索引使用 CREATE INDEX 语句，其格式如下：

CREATE [UNIQUE][CLUSTER] INDEX <索引名>

ON <表名>（<列名>[<次序>][，<列名>[<次序>]]…）；

其中，<表名>是要建索引的基本表的名称。索引可以建立在该表的一列或多列上，各列名之间用逗号分隔。<次序>表示排序方式，可以是 ASC（升序）或 DESC（降序），缺省值是 ASC。UNIQUE 表明此索引的每一个索引值只对应唯一的数据记录。CLUSTER 表示建立的索引是聚簇索引，它是指索引项的顺序与表中记录的物理顺序一致的索引组织。建立聚簇索引可以提高查询效率，但更新聚簇索引数据开销大，因此，对于经常更新的列不宜建立聚簇索引。

例 16　对基本表“学生”建立以“学号”为关键字的升序索引。

CREATE UNIQUE INDEX STD_NO ON 学生（学号）ASC；

其中，UNIQUE 为可选项，表示唯一索引，即基本表中相应列的值不能相同。

2．删除索引

维护索引会增加系统开销，因此，可以使用 DROP INDEX 删除一些不必要的索引。删除索引语句的格式如下：

DROP INDEX <索引名>；

删除索引时，系统会同时删除有关该索引的描述。

例 17　将索引 STD_NO 删除。

DROP INDEX STD_NO；

4.4.3　数据查询

SQL 语言中最主要、最核心的部分是它的查询功能。所谓查询就是从数据库中提取出满足用户需要的数据，查询是由 SELECT 语句实现的。在 SQL 语言中，许多操作都涉及 SELECT 语句。例如，将 SELECT 语句查询得到的数据插入到另外一个关系中；使用 SELECT 语句用满足条件的数据创建一个视图等。所以，SELECT 语句也是 SQL 语言中最灵活、最复杂的语句。

通常，一个 SELECT 语句可以分解成三个部分：查找什么数据，从哪里查找，查找条件是什么。因此，SELECT 语句可以分成以下几个子句：

SELECT [ALL | DISTINCT] <目标列表达式>[，<目标列表达式>]…

FROM <表名或视图名>[，<表名或视图名>]…

[WHERE <条件表达式>]

[GROUP BY <列名 1>[HAVING <条件表达式>]]

[ORDER BY <列名 2>[ASC | DESC]]；

其中，SELECT 子句中用逗号分开的表达式为查询目标。最常用也是最简单的是用逗号分开的属性名，即二维表中的列。系统对查询结果按照所需的属性进行投影运算。

FROM 子句指出上述查询目标及下面 WHERE 子句的条件中所涉及的所有关系名。

WHERE 子句指出查询目标必须满足的条件，系统根据条件进行选择运算，输出条件为真的元组集合。WHERE 子句常用的查询条件如表 4-7 所示。

表 4-7　常用的查询条件

查询条件	谓词
比较	=，>，<，>=，<=，!=
确定范围	BETWEEN…AND… NOT BETWEEN…AND…
字符匹配	LIKE，NOT LIKE
空值	IS NULL，IS NOT NULL
多重条件	AND，OR

不同系统所提供的功能有所区别，这里只介绍一般支持 SQL 的系统共有的基本功能。

以一个简单的学生选课管理关系为基础，通过示例来介绍 SELECT 语句的使用方法。设学生选课管理关系数据模型包括以下三个关系模式：

学生（学号，姓名，性别，出生日期，班级，所在系）

选课（学号，课程编号，成绩）

课程（课程编号，课程名称，课程类型，总学时，学分，备注）

一、简单查询

例 18　找出学生陈小东所在的班级。

```
SELECT 姓名，班级
    FROM 学生
    WHERE 姓名＝"陈小东"；
```

SELECT 子句中允许有字符串常量出现，如本例可改写为：

```
SELECT "姓名："，姓名，"班级："，班级
    FROM 学生
    WHERE 姓名＝"陈小东"；
```

在例 18 中，"姓名："和"班级："起到提醒的作用，使得查询的结果易于阅读。

例 19　查看所有学生的全部情况。

```
SELECT *
    FROM 学生
```

SELECT 子句里的星号*是表示全部属性的通配符。当不需要进行投影操作时，属性名就没有必要一一列出。

例 20　查询所有已选课的学生的学号。

```
SELECT DISTINCT 学号
    FROM 选课；
```

本例是针对已选课的学生而言的，每一学生都可能选修多门课程，而只要已选了一门课，就要将其学号显示出来，而且选修多门课程的同学只要显示一个就够了，所以在本例中加上了 DISTINCT 选项，目的是为了去掉重复的元组，如不加该选项，则将列出所有的学号，包括重复的学号。

例 21　查询所有学生的学号和英语（课程编号 TA3001）成绩，要求成绩按升序排列。

```
SELECT 学号，成绩
    FROM 选课
    WHERE 课程编号="TA3001"
    ORDER BY 成绩 ASC；
```

用 ORDER BY 对查询结果进行排序。ASC 表示升序，DESC 表示降序。升序是系统默认的，可以省略，因此本例中的 ASC 可以去掉。

例 22　查询英语（课程编号 TA3001）成绩在 90 到 100 分之间的学号和成绩。

```
SELECT 学号，成绩
    FROM 选课
    WHERE 课程编号="TA3001" AND 成绩 BETWEEN 90 AND 100；
```

例 23　查询 980111 班和 980112 班所有的学生。

```
SELECT *
    FROM 学生
    WHERE 班级 IN （"980111"，"980112"）；
```

本例中谓词 IN 表示包含在后面集合中，也可以用 OR 来代替。如：

```
SELECT *
    FROM 学生
    WHERE 班级="980111" OR 班级="980112";
```

如果集合中的元素多于两个，就要用多个 OR。当集合中的元素无法简单地列举时，就不可能再用 OR 进行替换；同时，用 IN 使得查询语句更为简练，减少了出现逻辑错误的几率。

另外，如果本例要求是查询不属于 980111 和 980112 班的学生，查询语句中只需将 IN 替换成 NOT IN 即可。

例 24　查询学号前五位是 98100 的学生的成绩。

```
SELECT *
    FROM 成绩
    WHERE 学号 LIKE "98100%";
```

本例中，谓词 LIKE 后面必须是字符串常量，其中可以使用两个通配符：

下划线“_”，代表任意一个单个字符；

百分号“%”，代表零到任意多个任意字符。

二、连接查询

简单查询一般只涉及一个关系，如果查询目标涉及两个或几个关系，就需要进行连接运算。连接运算一般是在 FROM 子句中列出关系名称，而在 WHERE 子句中则指明连接的条件，这样连接运算就由系统完成并实现优化。

例 25　查询所有选课学生的学号、姓名和班级。

```
SELECT DISTINCT 学号，姓名，班级
    FROM 学生，选课
    WHERE 学生.学号=选课.学号；
```

本例中学生和选课两关系中有相同的属性名，因此在属性前面要加上关系名用以区分不同的关系。

例 26　查询学生“王东平”选修的所有课程的编号和名称。

```
SELECT 姓名，课程.课程编号，课程.课程名称
    FROM 学生，选课，课程
    WHERE 学生.学号=选课.学号 AND 学生.姓名="王东平"
        AND 选课.课程编号=课程.课程编号；
```

例 27　查询选修了课程编号为“TA110101”的课程的学生的所有信息。

```
SELECT *
    FROM 学生，选课
    WHERE 学生.学号=选课.学号 AND 选课.课程编号="TA110101";
```

本例中 SELECT *代表学生和选课两关系连接后的所有属性。

例 28　查询只选修了“计算机网络”和“Java 语言”的学生的学号和选课信息。

```
SELECT *
    FROM 选课 A，选课 B
```

WHERE A.学号=B.学号

AND A.课程名称="计算机网络" AND B.课程名称="Java 语言"；

由于同一关系在查询模块中出现两次，因此必须引入别名机制，在本例中引入了别名 A 和 B，并在 WHERE 子句中用别名加以限制和区别。本例如果不采用别名，查询语句将比较麻烦。

三、嵌套查询

嵌套查询又称为子查询，它是指允许一条 SELECT 语句作为另一条 SQL 语句中的一部分，通常称被嵌套的 SELECT 语句为子查询，其外层的 SELECT 语句为主查询。子查询的作用是，首先查询出一个或多个关系的属性值，其结果并不显示，而是传递给它的外层语句，作为外层语句的查询条件来使用。

由于子查询还可以在它的语句中再嵌入子查询，因此子查询可以多层嵌套。

ORDER BY 子句不能出现在子查询中，这是因为它是对最终查询结果表示顺序的一种要求。

例 29　查询选修了“数据库技术”的学生姓名和班级。

```
SELECT 姓名，班级
    FROM 学生 WHERE 学号 IN
    （SELECT 学号
      FROM 选课
      WHERE 课程编号 IN
        （SELECT 课程编号
        FROM 课程
        WHERE 课程名称="数据库技术"
    ））；
```

当然，也可以用连接的方式来实现上述查询：

```
SELECT 姓名，班级
    FROM 学生，选课，课程
    WHERE 学生.学号=选课.学号
    AND 课程.课程名称="数据库技术" AND 课程.课程编号=选课.课程编号；
```

比较两种方法可知，用嵌套结构，思路更为清晰、自然；而用连接查询则 WHERE 子句中条件比较复杂，输入的过程中很容易出错。

在嵌套查询中最常用的谓词是 IN。在简单查询中，谓词 IN 可以用一系列的 OR 来代替，但嵌套查询中就不能替换，因为查询的外层用到内层的查询结果，用户事先并不知道内层结果。另外，并非所有的嵌套查询都能用连接查询替代，有时结合使用更显得简洁、方便。

例 30　查询与王子平选修课程相同的学生的学号和姓名。

```
SELECT 学号，姓名
    FROM 学生，选课
    WHERE 学生.学号=选课.学号 AND 课程编号 IN
    （SELECT 课程编号
```

```
        FROM 选课，学生
        WHERE 学生.学号=选课.学号 AND 学生.姓名="王子平"）；
```

嵌套查询语句中，还可使用 ALL 和 ANY。

例 31　查询与王子平或李大柱同在一个班的学生。

```
SELECT 学号，姓名
    FROM 学生
    WHERE 班级=ANY
        （SELECT 班级
        FROM 学生
        WHERE 姓名 IN （"王子平"，"李大柱"））；
```

这里，=ANY 亦可用 IN 来代替，但其他比较运算符，如>ANY、>=ANY、<ANY 以及<=ANY 则不能用 IN 代替。

例 32　查询所有学生中高等数学成绩比 980111 班高等数学最低成绩还低的成绩。

```
SELECT *
    FROM 选课
    WHERE 成绩 < ALL
      （SELECT 成绩
        FROM 学生，选课，课程
        WHERE 学号 学生.学号=选课.学号 AND 选课.课程编号=课程.课程编号
              AND 班级="980111" AND 课程名称="高等数学"））；
```

从本例中也可以看出，ALL 表示与所有子查询的结果相比较，而 ANY 则是与任意一个子查询的结果相比较。

四、使用函数查询

为了进一步增强数据的检索功能，SQL 提供了一些常用的统计函数。函数可以接受零个或多个参数，带不同参数的函数完成不同的运算，并产生一个或多个结果。函数的一般格式是：

函数名 （参数 1，参数 2，参数 3，…，参数 n）

其中，参数是用户提供的常量、变量或者是表达式的值。函数是按列计算的，最简单的表达式就是属性。

SQL 的库函数有（e 为数值型表达式）：

COUNT（*）计算元组的个数，即统计数据库中的记录数目，包括重复值和空值。

SUM（e）　对属性 e（列 e）的值进行求和运算。

AVG（e）　对属性 e（列 e）的值进行求平均值运算。

MAX（e）　计算属性 e（列 e）的值的最大值。

MIN（e）　计算属性 e（列 e）的值的最小值。

例 33　查询选课的总人数。

```
SELECT COUNT（DISTINCT 学号） FROM 选课；
```

例 34　查询 980111 班“英语”成绩的平均分数。

```
SELECT AVG（成绩）
    FROM 选课，学生
    WHERE 选课.学号=学生.学号 AND 学生.班级="980111"
        AND 选课.课程编号=课程.课程编号 AND 课程.课程名称="英语"；
```

例 35　查询“英语”成绩的最高分数和最低分数。

```
SELECT "最高分数："，MAX（成绩），"最低分数："，MIN（成绩）
    FROM 选课，学生
    WHERE 选课.学号=学生.学号
        AND 选课.课程编号=课程.课程编号 AND 课程.课程名称="英语"；
```

例 36　查询各班“高等数学”成绩的最高分数、最低分数和平均分数。

```
SELECT 班级，MAX（成绩），MIN（成绩），AVG（成绩）
    FROM 选课，学生
    WHERE 选课.学号=学生.学号
        AND 选课.课程编号=课程.课程编号 AND 课程.课程名称="高等数学"
    GROUP BY 班级；
```

其中，GROUP BY 的作用是按属性的值对元组进行分组，然后对每一组分别使用统计函数。本例中是按班级来求最高分数、最低分数和平均分数，因此学生按班分成组。

例 37　查询没有选修课程的学生。

```
SELECT 学号，姓名，班级
    FROM 学生
    WHERE NOT EXISTS
    （SELECT *
        FROM 选课
        WHERE 学生.学号=选课.学号）；
```

EXISTS 表示存在，而 NOT EXISTS 表示不存在。本例中，子查询查询的是选修课程的学生的记录，如果还有些学生的信息没有包含在这些记录中，则表明这些学生未选修课程。

NOT 称为逻辑非，在 WHERE 子句中可以出现在任何查询条件之前。例如 NOT IN，NOT LIKE，NOT EXISTS，NOT BETWEEN，NOT ANY，NOT ALL 等。

例 38　查询选课人数超过 20 人的班级和学生人数。

```
SELECT 班级，COUNT（DISTINCT 选课.学号）
    FROM 学生，选课
    WHERE 学生.学号=选课.学号
    GROUP BY 班级
    HAVING COUNT（DISTINCT 选课.学号） > 20；
```

在本例中，出现了 HAVING 子句，用它可以限制查询条件。由于查询条件包括“选课人数超过 20 人”，而这个条件要用到统计函数 COUNT，不能把它放到 WHERE 子句中，只能使用 HAVING 子句。如果条件中没有统计函数，就没有必要使用 HAVING 子句，而是将条件直接放到 WHERE 子句中。

HAVING 子句通常跟在 GROUP BY 子句之后，它和 WHERE 子句并不矛盾，在查询中

一般用 WHERE 子句限定元组，然后用 GROUP BY 子句分组，最后用 HAVING 子句限定组。这样，使得查询灵活可变。

五、集合运算

关系是元组的集合，可以进行传统的集合运算。集合运算包括并 UNION、 差 MINUS、交 INTERSECT。用这些集合运算，就可以求一个 SELECT 子查询的结果与另一个 SELECT 子查询的并、差、交。集合运算是以整个元组为单位的运算，因此，这些子查询目标的结构与类型必须互相匹配。集合运算将去除重复元组。

1．并操作 UNION

UNION 操作是将属于一个查询结果的行或者属于另一个查询结果的行组合起来，构成 UNION 操作的结果。如果有一个行既属于一个查询结果，又属于另一个查询结果，则只取出其中一行。

2．交操作 INTERSECT

INTERSECT 操作是将既属于一个查询结果的行又属于另一个查询结果的行组成 INTERSECT 操作的结果。

3．差操作 MINUS

MINUS 操作是从一个查询结果的行中去掉又属于另一个查询结果的行，因而 A MINUS B 和 B MINUS A 的结果往往是不同的。

例 39　查询选修了“大学物理”或者选修了“高等数学”的学生的学号和姓名。

```
SELECT 学号，姓名
    FROM 选课，学生，课程
    WHERE 学生.学号=选课.学号 AND 选课.课程编号=课程.课程编号
        AND 课程名称="大学物理"
UNION
SELECT 学号，姓名
    FROM 选课，学生，课程
    WHERE 学生.学号=选课.学号 AND 选课.课程编号=课程.课程编号
        AND 课程名称="高等数学"
```

4.4.4　数据更新

数据更新是对关系中的具体数据进行插入、修改、删除等操作。

一、插入数据

SQL 的数据插入语句 INSERT 通常有两种形式。一种是插入一个元组，另一种是插入子查询结果。后者可以一次插入多个元组。

插入单个元组的 INSERT 语句的格式为：

```
INSERT INTO <表名> [（<属性列 1>，<属性列 2>…）]
        VALUES（<常量 1>[，<常量 2>]…）；
```

插入子查询结果的 INSERT 语句的格式为：

INSERT INTO <表名> [（<属性列 1>，<属性列 2>…）]

子查询；

例 40　在基本表“学生”中增加一条新的记录。

INSERT INTO 学生（学号，姓名，性别，出生日期，班级，所在系）

VALUES（"980101"，"李小芬"，"女"，"1980"，"980111"，"计算机系"）；

由于是往基本表中插入一条新的记录，因此记录中各字段的值必须满足基本表的定义。如果增加的是一条完整的记录，而且字段的顺序和基本表的定义一致，可以在基本表名称后省略字段名称。

INSERT INTO 学生

VALUES（"980101"，"李小芬"，"女"，"1980"，"980111"，"计算机系"）；

也可以插入这样一条记录，记录中只有若干列有值，其他允许为空的列暂时为空值，如列为非空（NOT NULL）型字段，则必须给它赋值，基本表名称后面的列的名称必须指明。

例 41　向基本表“学生”中插入一条只有部分列有值的记录。

INSERT INTO 学生（学号，姓名）

VALUES（"980101"，"李小芬"）；

例 42　对每一个系，求学生的人数，并把结果存入数据库。

CREATE TABLE 系学生数

（系别 CHAR（15）

人数 SMALLINT）；

然后对学生表按系分组求学生人数，再把系名和人数存入新表中。

INSERT INTO 系学生数（系别，人数）

SELECT 所在系，COUNT（*）

FROM 学生 GROUP BY 所在系；

二、修改数据

对数据进行修改时可在 WHERE 子句中指明具体的条件，以限定修改的范围。修改语句的格式如下：

UPDATE <表名>

SET <列名>=<表达式>[，<列名>=<表达式>]…

[WHERE <条件>]；

其中，SET 子句用给出<表达式>的值取代相应的属性值。

例 43　将“TA010101”课程改为“DA010101”。

UPDATE 课程 SET 课程编号="DA010101"

WHERE 课程编号="TA010101"；

三、删除数据

当基本表中的某些记录不再需要时，可以删除这些数据。删除的对象是记录，而不是记录之中的部分字段。数据删除用 DELETE FROM 语句。删除语句的格式为：

DELETE

FROM <表名>

[WHERE <条件>];

例 44 从基本表“学生”中删除“学号”为“980101”的学生的记录。

DELETE FROM 学生 WHERE 学号="980101";

注意，为了保持数据的一致性，也要将基本表“选课”中学号“980101”所选的课程删除。

DELETE FROM 选课 WHERE 学号="980101";

删除语句一次可以删除一条或几条记录。当然也可以将一张基本表中的记录全部删除，只保留表的结构定义。

例 45 将基本表“选课”中的记录全部删除。

DELETE FROM 选课;

4.4.5 数据控制

通过对数据库各种权限的授予或回收可以对整个数据库进行合理的管理，这就是数据控制。数据库的每个用户在登录时都要输入用户名和口令，通过系统的合法性检验之后才能使用系统。不同的用户对基本表、视图等的权限都不一样。如果具有定义基本表的权限，用户就可以定义基本表，并拥有基本表的所有权限，同时也可以将其中一些权限授予给别人，当然也可以回收所授予的权限。数据库管理员对数据库的所有资源拥有所有权限，包括数据定义、数据查询、数据操纵和数据控制。数据库管理员可以给用户授予各种权限。非基本表的建立者必须具有相应的权限才能使用这些基本表。

一、授权

在 SQL 语言中，用 GRANT 语句向用户授予操作权限，其格式如下：

GRANT <权限>[， <权限>]…

ON <对象类型><对象名>

TO <用户 1>[， <用户 2>]…

[WITH GRANT OPTION];

接受权限的用户可以是一个或多个具体用户，也可以是 PUBLIC，即全体用户。如果指定了 WITH GRANT OPTION 子句，则获得权限的用户可以把这种权限再授予其他用户。

对不同类型的操作对象有不同的操作权限，常见的操作权限如表 4-8 所示。

表 4-8 不同对象类型允许的操作权限

对象	对象类型	操作权限
属性列	table	Select，Insert，Update，Delete，All
视图	table	Select，Insert，Update，Delete，All
基本表	table	Select，Insert，Update，Delete，Alter，Index，All
数据库	database	Createdb

例 46　将查询、更新、删除基本表“选课”的权限授予给所有用户。

```
GRANT SELECT，UPDATE，DELETE
    ON TABLE 选课
        TO PUBLIC；
```

二、回收权限

回收权限的语句是 REVOKE，其格式如下：

```
REVOKE <权限 1>[，<权限 2>…
    ON <对象类型><对象名>
    FROM <用户 1>[，<用户 2>]…；
```

例 47　回收所有用户对基本表选课的全部权限。

```
REVOKE ALL ON TABLE 选课 FROM PUBLIC；
```

授予和回收权限是有层次的，设用户 A 把自己所建立的基本表“通信录”的所有权限授予给用户 B，并用 WITH GRANT OPTION 允许他再给其他用户授权。用户 B 又将对“通信录”的 SELECT 和 UPDATE 权限授予给用户 C。此时，如用户 A 回收了用户 B 的 UPDATE 权限，则用户 C 的这种权限也就自动被取消。

小　　结

本节主要介绍了结构化查询语言 SQL 的基本功能，包括以下内容：

• SQL 的数据定义：主要介绍了基本表、视图和索引的定义、删除，其中基本表还可进行结构的修改。

• SQL 的查询功能：主要是使用 SELECT 语句，结合函数和一些选项，形成形式不同、功能各异的查询语句。

• SQL 的数据更新：对关系中的具体数据进行增加、删除和修改等操作。

• SQL 的数据控制：包括用户权限的授予和回收。

4.5　数据库设计

数据库设计是建立数据库及其应用系统的技术，是信息系统开发和建设中的核心技术，具体地说，数据库设计是指对于一个给定的应用环境，构造最优的数据库模式，建立数据库及其应用系统，使之能够有效地存储数据，满足各种用户的应用需求（信息需求和处理需求）。

按照软件工程的方法，数据库设计可分为以下六个阶段：

1．需求分析

系统需求分析是整个数据库设计过程中最重要的阶段之一，是以后各阶段的基础。

这一阶段是数据库设计人员和用户共同收集数据库所需要的信息内容和处理要求，以及数据库的安全性和完整性要求；在收集数据的基础上，对这些数据进行分析和整理，以

充分理解用户的要求，并把理解的内容精确地表达出来，加以规范化。这一步的主要成果是“系统需求说明书”，它是由分层的数据流图和数据字典两部分组成。

2．概念结构设计

概念结构设计的目标是产生反映信息需求的整体数据库概念结构，能以用户可以理解的形式表达出来，这就是概念结构模式。概念结构模式应独立于计算机的任何数据模型，不受特定的数据库管理系统的约束，也不考虑计算机软硬件的技术细节。

概念结构设计最常用的方法是实体-联系方法，即 E-R 图。用 E-R 图可以将实体集合抽象成实体模型，用实体间联系的类型反映现实世界事物间的内在联系。

3．逻辑结构设计

逻辑结构设计的主要目标是产生一个 DBMS 能处理的模式。实际上就是把概念设计阶段的实体联系模型转换成特定 DBMS 所支持的数据模型，如网状模型、层次模型、关系模型、面向对象模型等。这个转换过程不太复杂，但在转换过程中要根据应用要求和存储要求对模式不断求精。

设计逻辑结构时，一般分为三步：将概念结构向一般的关系等模型转化；将转化的关系等模型向特定 DBMS 支持下的数据模型转换；对数据模型进行优化。

4．物理结构设计

对于给定的逻辑数据模型选取一个最适合应用环境的物理结构的过程，称为数据库物理结构设计。数据库的物理结构主要指数据库的存储记录格式、存储记录安排和存取方法，这些完全依赖于具体的系统。

物理结构设计的任务是为了有效地实现逻辑模式，确定所采取的存储策略。此阶段以逻辑设计的结果作为输入，结合具体 DBMS 的特点与存储设备特性进行设计，选定数据库在物理设备上的存储结构和存取方法。

数据库的物理结构设计与特定硬件系统、特定的 DBMS 及特定的实施环境密切相关，各系统之间差别很大，因此难以形成统一的设计方法。

决定存储结构的主要因素包括存取时间、存储空间和维护代价等，设计时应当根据实际情况从这三个方面综合考虑。

确定了数据库的物理结构之后，要进行评价，重点是时间和空间的效率。如果评价结果满足设计要求，则可进行数据库实施。实际上，往往需要经过反复测试才能优化物理结构设计。

5．数据库实施

在数据库实施阶段，设计人员运用 DBMS 提供的数据语言及其宿主语言，根据逻辑结构设计和物理结构设计的结果建立数据库，组织数据入库，编制与调试应用程序，并进行试运行。

6．数据库运行和维护

数据库应用系统经过试运行后即可投入正式运行。在数据库系统运行过程中必须不断地对其进行评价、调整与修改。数据库的维护是指维护数据库的安全性和完整性，监测并改善数据库性能，并在必要的情况下重新组织数据库。

设计一个完善的数据库应用系统不可能一蹴而就，往往是这六个阶段的不断反复。

小　　结

本节主要介绍了数据库设计的基本过程，使大家对数据库的设计过程有一些基本的了解。数据库设计的基本过程分为六个阶段：需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施、数据库运行和维护。

4.6　数据库保护

数据库管理系统除了提供对数据的查询操作和更新操作以外，还必须对数据库中的各种操作加以监控并对数据进行保护，例如，要保证对数据的操作没有违反完整性规则，防止对数据的非法存取，防范系统故障造成的数据丢失和数据库处于不一致状态等。下面从数据库的恢复技术、并发控制、完整性和安全性等几个方面来说明对数据库的保护措施。

4.6.1　恢复技术

事务是一系列的数据库操作，是数据库应用程序的基本逻辑单元。事务处理技术主要包括数据库恢复技术和并发控制技术。数据库恢复机制和数据库并发控制机制是数据库管理系统的重要组成部分。本节在介绍事务的基础上，讨论数据库恢复技术。

一、事务的基本概念

1．事务

所谓事务是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的基本单位。例如，在关系数据库中，一个事务可以是一条 SQL 语句、一组 SQL 语句或整个程序。

事务和程序是两个概念。一般地讲，一个程序包括多个事务。

事务的开始和结束可以由用户显式控制。如果用户没有显式地定义事务的开始和结束，则由 DBMS 按缺省规定自动划分事务。在 SQL 语言中，定义事务开始和结束的语句有三条：

BEGIN TRANSACTION

COMMIT

ROLLBACK

事务通常以 BEGIN TRANSACTION 开始，以 COMMIT 或 ROLLBACK 结束。COMMIT 表示提交，即提交事务的所有操作。具体地说就是将事务中所有对数据库的更新写回到磁盘上的物理数据库中去，事务正常结束。ROLLBACK 表示回滚，即在事务运行的过程中发生了某种故障，事务不能继续执行，系统将事务中对数据库的所有已完成的操作全部撤销，回滚到事务开始时的状态。这里的操作是指对数据库的更新操作。

2．事务的特性

事务具有四个特性：原子性（Atomicity）、一致性（Consistence）、隔离性（Isolation）和持续性（Durability），这四个特性也简称为 ACID 特性。

（1）原子性

事务是数据库的逻辑工作单元，事务中包括的诸操作要么全做，要么全不做。

（2）一致性

事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态。当数据库中只包含成功事务提交的结果时，就说数据库处于一致性状态。如果数据库系统运行中发生故障，有些事务尚未完成就被迫中断，这些未完成事务对数据库的修改有一部分已写入物理数据库，这时数据库就处于一种不正确的状态，或者说不一致的状态。例如，某公司在银行中有 A、B 两个账户，现在公司想从账户 A 中取出 10 000 元存入账户 B 中，那么就可以定义一个事务，该事务包括两个操作：第一个操作是从账户 A 中减去 10 000 元，第二个操作是向账户 B 中加入 10 000 元。这两个操作要么全做，要么全不做。全做或者全不做，数据库都处于一致性状态，如果只做一个操作则用户账户就会发生错误，少了 10 000 元，这时数据库就处于不一致的状态。

可见，一致性和原子性是密切相关的。

（3）隔离性

一个事务的执行不能受其他事务的干扰，即一个事务内部的操作和使用的数据对其他并发事务来说是隔离的，并发执行的各个事务之间不能互相干扰。

（4）持续性

持续性指一个事务一旦提交，它对数据库中的数据的改变应该是永久性的，接下来的其他操作或故障不应该对其执行结果有任何影响。

事务是恢复和并发控制的基本单元。保证事务 ACID 特性是事务处理的重要任务。事务 ACID 特性可能遭到破坏的因素有：

- 多个事务并发运行时，不同事务的操作交叉执行；
- 事务在运行过程中被强行停止。

在第一种情况下，数据库管理系统必须保证多个事务的交叉运行不影响这些事务的原子性；在第二种情况下，数据库管理系统必须保证被强行终止的事务对数据库和其他事务没有任何影响。

这些就是数据库管理系统中恢复机制和并发控制机制的责任。

二、恢复的实现技术

尽管数据库系统中采取了各种保护措施来防止数据库的安全性和完整性被破坏，保证并发事务的正确执行，但是计算机系统的故障或恶意破坏仍是不可避免的，这些故障轻则造成运行事务非正常中断，影响数据库中数据的正确性，重则破坏数据库，使数据库中全部或部分数据丢失，因此，数据库管理系统必须具有把数据库从错误状态恢复到某一已知的正确状态的功能，这就是数据库的恢复。

恢复的基本原理就是：冗余，即数据库中任何一部分被破坏的或者不正常的数据可以根据存储在系统别处的冗余数据来重建。尽管恢复的原理很简单，但实现技术的细节很复杂。

恢复机制涉及两个关键问题：其一，如何建立冗余数据；其二，如何利用这些冗余数据实现数据库恢复。

建立冗余数据最常用的技术是数据转储和登记日志文件。通常在一个数据库系统中，这两种方法是一起使用的。

1．数据转储

后备副本（或后援副本）是指在故障发生前某一时刻数据库的副本，副本中包含了数据库中所有数据，包括系统数据和用户数据。

制做后备副本的过程称为转储，是由 DBA 定期将数据库复制到磁带或另一个磁盘上，并将这些备用的数据文本妥善地保存起来。

根据转储数据的情况，数据转储可以分为增量转储和海量转储。海量转储是指每次转储全部数据库，而增量转储是指每次转储上次转储后修改过的数据。

根据转储操作时数据库的状态，转储可分为静态转储和动态转储。

静态转储是指系统停止对外服务，不允许用户运行事务，只进行转储操作。即转储操作开始的时刻，数据库处于一致性状态，而转储期间不允许（或不存在）对数据库的任何存取、修改活动。显然，静态转储得到的一定是一个数据一致的后备副本。

静态转储很简单，但是必须等到正运行的事务结束之后才能进行转储，同样，新的事务必须等到转储结束后才能运行，显然，这会降低数据库的可用性。

动态转储是指转储期间允许用户对数据库进行存取操作，即转储和用户事务可以并发执行。

动态转储可以克服静态转储的缺点，它不用等待正在运行的事务结束，也不会影响新事务的运行。但是，转储结束时后备副本上的数据并不能保证正确有效。例如，在转储期间的某个时刻 T，系统把数据 A=50 转储到磁带上，而在下一时刻，某一事务将 A 改为 100。转储结束后，后备副本上的 A 已经为过时数据了。

为此，必须把转储期间各事务对数据库的修改活动记录下来，建立日志文件。这样，后备副本加上日志文件就能把数据库恢复到某一时刻的正确状态。

转储要用 DBMS 提供的专门命令，而不能简单采用复制数据库文件的方法。普通用户也无权进行转储操作，转储操作必须由 DBA 完成。当然 DBA 也可以授权给其他用户代为转储。

2．日志文件机制

日志文件是用来记录事务的每一次对数据库更新操作的文件，包括用户的更新操作以及由此引起的系统内部的更新操作。有了日志文件，DBMS 就可以根据日志文件进行事务故障恢复和系统故障恢复，并结合后备副本进行介质故障恢复。

（一）日志文件的格式和内容

不同数据库系统采用的日志文件格式并不完全一样。一般来说，日志文件有两种格式：以记录为单位的日志文件和以数据块为单位的日志文件。

对于以记录为单位的日志文件，日志文件中需要登记的主要内容包括：

- 各个事务的开始（BEGIN TRANSACTION）标记；
- 各个事务的结束（COMMIT 或 ROLLBACK）标记；
- 各个事务的所有更新操作。

这里每个事务开始、结束标记和每个更新操作均作为日志文件的一个日志记录，每个日志记录的内容主要包括：

- 事务标识（标明是哪个事务）；
- 操作的类型（插入、删除或修改）；

• 操作对象（记录内部标识）；
• 更新前数据的旧值（对插入操作而言，此项为空值）；
• 更新后数据的新值（对删除操作而言，此项为空值）。

对于以数据块为单位的日志文件，日志记录的内容包括事务标识和被更新的数据块。由于将更新前的整个块和更新后的整个块都放入日志文件中，操作的类型和操作对象等信息就不必放入日志记录中

（二）日志文件的作用

日志文件在数据库恢复中起着非常重要的作用，可以用来进行事务故障恢复和系统故障恢复，并协助后备副本进行介质故障恢复。具体作用为：

（1）事务故障恢复和系统故障恢复必须用日志文件。

（2）在动态转储中必须建立日志文件，后备副本和日志文件结合起来才能有效地恢复数据库。

（3）在静态转储中，也可以建立日志文件。当数据库毁坏后可重新装入后备副本把数据库恢复到转储结束时刻的正确状态，然后利用日志文件，把已完成的事务进行重做处理，对故障发生时尚未完成的事务进行撤销处理。这样不必重新运行那些已经完成的事务程序就可把数据库恢复到故障前某一时刻的正确状态。

（三）登记日志文件

为保证数据库是可恢复的，登记日志文件时必须遵循两条原则：

（1）登记的次序严格按并发事务执行的时间次序；

（2）必须先写日志文件，后写数据库。

三、恢复策略

当系统运行过程中发生故障，利用数据库后备副本和日志文件就可将数据库恢复故障前的某一个一致性状态。不同故障其恢复策略和方法也不一样。

1．事务故障恢复

事务故障是指事务未运行至正常终止点前被撤销，这时恢复子系统应对此事务做UNDO 处理。具体做法是：

（1）反向扫描日志文件，找出该事务的所有更新操作。

（2）对该事务的更新操作做它的逆操作。即若记录中是插入操作，则做删除操作；若记录中是删除操作，则做插入操作；若是修改操作，则用修改前的值代替修改后的值。

（3）继续反向扫描日志文件，查找该事务的其他更新操作，并作同样处理。

（4）如此处理直至读到此事务的开始标记，事务故障恢复完成。

2．系统故障恢复

系统故障发生时，造成数据库不一致状态的原因有两个，一是由于一些未完成事务对数据库的更新已写入数据库，二是由于一些已提交事务对数据库的更新还留在缓冲区没来得及写入数据库。因此恢复操作就是要撤销故障发生时未完成的事务，重做已完成的事务。

系统故障的恢复是由系统在重新启动时自动完成的，不需要用户干预。

系统的恢复步骤为：

（1）正向扫描日志文件（即从头到尾扫描日志文件），找出在故障发生前已经提交的

事务（这些事务既有 BEGIN TRANSACTION 标记，也有 COMMIT 标记），将其事务标识记入重做（REDO）队列。同时找出故障发生时尚未完成的事务（这些事务只有 BEGIN TRANSACTION 标记，无相应的 COMMIT 标记），将其事务标识记入撤销（UNDO）队列。

（2）对撤销队列中的各个事务进行撤销（UNDO）处理。进行 UNDO 处理的方法是反向扫描日志文件，对每个 UNDO 事务的更新操作执行逆操作，即将日志记录中“更新前的值”写入数据库。

（3）对重做队列中的各个事务进行重做（REDO）处理。进行 REDO 处理的方法是正向扫描日志文件，对每个 REDO 事务重新执行日志文件登记的操作，即将日志记录中“更新后的值”写入数据库。

3．介质故障的恢复

发生介质故障后，磁盘上的物理数据和日志文件被破坏，这是最严重的一种故障，恢复方法是重装数据库，然后重做已完成的事务，具体说就是：

（1）装入最新的数据库后备副本（离故障发生时刻最近的转储副本），使数据库恢复到最近一次转储时的一致性状态。

（2）对于动态转储的数据库副本，还需同时装入转储开始时刻的日志文件副本，利用恢复系统故障的方法，才能将数据库恢复到一致性状态。

（3）装入相应的日志文件副本（转储结束时刻的日志文件副本），重做已完成的事务。即首先扫描日志文件，找出故障发生时已提交的事务的标识，将其记入重做队列；然后正向扫描日志文件，对重做队列中的所有事务进行重做处理，即将日志记录中“更新后的值写入数据库”。

这样，就可以将数据库恢复至故障前某一时刻的一致性状态了。

4.6.2 并发控制

DBMS 为了有效利用计算机的硬件资源和数据库中的数据，允许多个事务并发执行，但事务的并发执行可能出现诸如丢失修改、读脏数据或不可重复读等问题，使数据库处于不一致状态。为了防止并发执行产生的问题，DBMS 需要具有并发控制的功能。并发控制技术的主要方法是封锁法。

一、常见并发问题

多个事务并发执行常常引起数据库中的数据不一致，常见的并发问题有：丢失修改、不可重复读和读脏数据。

（1）丢失修改

两个事务T_1和T_2读入同一数据并修改，T_2提交的结果破坏了T_1提交的结果，导致T_1的修改被丢失。

（2）不可重复读

指事务T_1读取数据后，事务T_2执行更新操作，使T_1无法再现前一次读取结果。不可重复读包括三种情况：

• 事务T_1读取某一数据后，事务T_2对其进行了修改，当事务T_1再次读该数据时，得到与前一次不同的值。

•事务T_1按一定条件从数据库中读取了某些数据记录后，事务T_2删除了其中部分记录，当T_1再次按相同条件读取数据时，发现某些记录神秘地消失了。

• 事务T_1按一定条件从数据库中读取某些数据记录后，事务T_2插入了一些记录，当T_1再次按相同条件读取数据时，发现多了一些记录。

后两种不可重复读有时也称为幻影（Phantom Row）现象。

（3）读脏数据

指事务T_1修改某一数据，并将其写回磁盘，事务T_2读取同一数据后，T_1由于某种原因被撤销，这时，T_1已将修改过的数据恢复原值，T_2读到的数据就与数据库中的数据不一致，则T_2读到的数据就为脏数据。

二、封锁

封锁是实现并发控制的一个非常重要的技术。所谓封锁就是事务 T 在对某个数据对象操作前，先向系统发出请求，对其加锁。加锁后事务 T 就对该数据对象有了一定的控制权，在事务 T 释放它的锁之前，其他的事务不能更新此数据对象。

确切的控制由封锁的类型决定。基本的封锁类型有两种：排他锁和共享锁。

排他锁又称为写锁或 X 锁。若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A，其他任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁。这就保证了其他事务在 T 释放 A 上的锁之前不能再读取和修改 A。

共享锁又称为读锁或 S 锁。若事务 T 对数据对象 A 加上 S 锁，则事务 T 可以读 A 但不能修改 A，其他事务只能再对 A 加 S 锁，而不能加 X 锁，直到 T 释放 A 上的 S 锁。这就保证了其他事务可以读 A，但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改。

三、封锁协议

在运用 X 锁和 S 锁对数据对象加锁时，还需要约定一些规则，这些规则被称为封锁协议。对封锁方式规定不同的规则，就形成了各种不同的封锁协议。下面介绍三级封锁协议。

1．一级封锁协议

一级封锁协议是：事务 T 在修改数据 R 之前必须先对其加 X 锁，直到事务结束才释放。

一级封锁协议可防止丢失修改，并保证事务 T 是可恢复的。

在一级封锁协议中，如果仅仅是读数据不对其进行修改，是不需要加锁的。所以，它不能保证可重复读和不读脏数据。

2．二级封锁协议

二级封锁协议是：一级封锁协议加上事务 T 在读取数据 R 之前必须对其加 S 锁，读完后即可释放 S 锁。

二级封锁协议除了防止丢失修改外，还可防止读脏数据。

在二级封锁协议中，由于读完数据后即可释放 S 锁，所以，它不能保证可重复读。

3．三级封锁协议

三级封锁协议是：一级封锁协议加上事务 T 在读取数据 R 之前必须先对其加 S 锁，直到事务结束才释放。

三级封锁协议除了防止丢失修改和不读脏数据外，还可防止不可重复读。

四、并发调度的可串行性

计算机系统对并发事务中并发操作的调度是随机的，而不同的调度可能会产生不同的结果，那么哪个结果是正确的，哪个是不正确的呢？

如果一个事务运行过程中没有其他事务同时运行，即它没有受到其他事务的干扰，那么就可认为事务的运行结果是正常的或者预想的。因此，将所有事务串行运行的调度策略一定是正确的调度策略。虽然以不同的顺序串行执行事务可能会产生不同的结果，但由于不会将数据库置于不一致状态，所以，都是正确的。

定义 多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行它们时的结果相同，称这种调度策略为可串行化的调度。

可串行性是并发事务正确性的准则。按这个准则规定，一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度。

为了保证并发操作的正确性，DBMS 的并发控制机制必须提供一定的手段来保证调度是可串行化的。目前 DBMS 普遍采用封锁方法实现并发操作调度的可串行性，从而保证调度的正确性。两段锁协议就是保证并发调度可串行性的封锁协议。

所谓两段锁协议是指所有事务必须分两个阶段对数据项加锁和解锁，这两个阶段是加锁阶段和解锁阶段。

加锁阶段：在该阶段可以进行加锁操作，也称为扩展阶段。在对任何数据进行读操作之前要申请并获得 S 锁，在进行写操作之前要申请并获得 X 锁。加锁不成功，则事务进入等待状态，直到加锁成功才继续执行。

解锁阶段：当事务释放了一个锁以后，事务进入解锁阶段，也称为收缩阶段。在该阶段只能进行解锁操作不能再进行加锁操作。

可以证明，若并发执行的所有事务均遵守两段锁协议，则对这些事务的任何并发调度策略都是可串行化的。

4.6.3 完整性

数据库的完整性控制是指保证数据库中数据的正确性、有效性和相容性，防止错误的数据进入数据库。例如，学生的学号必须是唯一的；性别只能是男或女；学生所在的系必须是学校已设置的系等，数据库是否具有完整性关系到数据库系统能否真实地反应现实世界，因此维护数据库的完整性是非常重要的。

数据库的完整性和安全性是两个不同的概念。前者是为了防止数据库中存在不符合语义的数据，防止错误信息的输入和输出，即所谓垃圾进入所造成的无效操作和错误结果。而后者是保护数据库，防止恶意的破坏和非法的存取，也就是说，安全性措施的防范对象是非法用户和非法操作，完整性措施防范的对象是不符合语义的数据，当然，完整性和安全性是密切相关的。

为了维护数据库的完整性，DBMS 必须提供一种机制来检查数据库中的数据，看其是否满足语义规定的条件。这些加在数据库数据之上的语义约束条件称为数据库完整性约束条件，它们作为模式的一部分存入数据库中，而 DBMS 中检查数据是否满足完整性条件的

机制称为完整性检查。

一、完整性约束条件

完整性检查是围绕完整性约束条件进行的，因此完整性约束条件是完整性控制机制的核心。

完整性约束条件的作用对象可以是关系、元组、列三种。其中列约束主要是列的类型、取值范围、精度、排列等约束条件；元组约束是对元组中各个字段间相互联系的约束；关系约束是若干元组间、关系集合上以及关系之间的联系的约束。

完整性约束条件涉及的这三类对象，其状态可以是静态的，也可以是动态的。所谓静态约束是指数据库每一确定状态时的数据对象所应满足的约束条件，它是反映数据库状态合理性的约束，这是最重要的一类完整性约束。动态约束是指数据库从一种状态变为另一种状态时，新、旧值之间所应满足的约束条件，它是反映数据库状态变迁的约束。

完整性约束条件可分为六类：静态列级约束、静态元组约束、静态关系约束、动态列级约束、动态元组约束、动态关系约束。

1．静态列级约束

静态列级约束是对一个列的取值域的说明，这是最常用的也是最容易实现的一类完整性约束，包括以下几个方面：

（1）对数据类型的约束（包括数据的类型、长度、单位、精度等）

例如，电子科技大学数据库中，学生姓名的数据类型为字符型，长度为 8；西南民族大学数据库中，学生姓名的数据类型为字符型，长度为 20，因为少数民族的姓名较长。

（2）对数据格式的约束

例如，学号的前两位表示入学年份，中间两位表示系的编号，后三位表示顺序编号。

（3）对取值范围或取值集合的约束

例如，学生成绩的范围为 0~100，大学本科学生年龄的取值范围为 14~29，性别的取值集合为{男，女}等。

（4）对空值的约束

空值表示未定义或未知的值，它与零值和空格不同。有的列允许空值，有的列则不允许。例如，学生学号不能取空值，成绩可以取空值。

（5）其他约束

例如，有关列的排序说明，组合列等。

2．静态元组约束

一个元组是由若干个列值组成的，静态元组约束就是规定元组的各个列之间的约束关系。例如，订货关系中的发货量、订货量等列，规定发货量不得超过订货量；又如教师关系中包括职称、工资等列，规定教授的工资不低于 3000 元。

3．静态关系约束

在一个关系的各个元组之间或者若干关系之间常常存在各种联系或约束。常见的静态关系约束有：

（1）实体完整性约束。

（2）参照完整性约束：参照完整性约束和实体完整性约束是关系模型的两个极其重要

的约束，称为关系的两个不变性。

（3）函数依赖约束：大部分函数依赖约束都在关系模式中定义。

（4）统计约束：即字段值与关系中多个元组的统计值之间的约束关系。例如，规定部门经理的工资不得高于本部门职工平均工资的 5 倍，不低于本部门职工平均工资的 2 倍。本部门职工的平均工资是一个统计值。

4．动态列级约束

动态列级约束是修改列定义或列值时应满足的条件，包括下面两个方面：

（1）修改列定义时的约束

例如，将允许空值的列改为不允许空值时，如果该列已经存在空值，则不允许修改。

（2）修改列值时的约束

修改列值有时需要参照其旧值，而且新旧值之间需要满足某种约束条件。例如，职工工资的调整不得低于其原来的工资，学生的年龄只能增长等。

5．动态元组约束

动态元组约束是指修改元组的值时，元组中各个字段之间需要某种约束条件。例如，职工工资调整时，新工资不得低于原工资+工龄*1.5 等。

6．动态关系约束

动态关系约束是加在关系变化前后状态上的限制条件。例如，事务一致性、原子性等约束条件。

二、完整性控制

DBMS 的完整性控制机制应具有三个方面的功能：

（1）定义功能，提供定义完整性约束条件的机制，一般由 SQL 的 DDL 语句来实现。

（2）检查功能，提供检查是否违背完整性约束条件的方法。

（3）若发现用户的操作违背了完整性约束条件，就采取一定的动作来保证数据的完整性，如拒绝用户执行该操作。

完整性控制机制应该允许用户定义所有六类完整性约束条件。

检查是否违背完整性约束的时机可在一条语句执行完后立即检查，称这类约束为立即执行约束；也可在整个事务执行结束后检查，称这类约束为延迟执行约束。

如果发现用户操作请求违背了完整性约束条件，系统将拒绝该操作，但对于延迟执行的约束，系统将拒绝整个事务，把数据库恢复到该事务执行前的状态。

目前许多关系数据库管理系统都提供了定义和检查实体完整性、参照完整性和用户定义的完整性的功能。对于违反实体完整性和用户定义的完整性的操作一般都采用拒绝执行的方式进行处理。而对于违反参照完整性的操作，并不都是简单地拒绝执行，有时要根据应用语义执行一些附加的操作，以保证数据库的正确性。

4.6.4 安全性

数据库的安全性是指保护数据库以防止不合法的使用所造成的数据泄露、更改或破坏。

安全性问题不是数据库系统所独有的，所有的计算机系统都有这个问题。只是在数据库系统中大量数据集中存放，而且为许多最终用户直接共享，从而使得安全性问题更为突

出。系统安全保护措施是否有效是数据库系统的主要指标之一。

在一般的计算机系统中，安全措施是一级一级层层设置的，包含多种安全技术和措施。下面仅介绍与数据库有关的安全技术。

1．用户鉴别

用户鉴别是操作系统提供的最外层安全保护模式。其方法是由操作系统提供一定的方式让用户标识自己的名字或身份。每次用户进入系统时，由操作系统进行核对，通过鉴别后才提供系统的使用权。

对于获得上机权的用户若要使用数据库时，数据库系统还要进行用户鉴别。

用户鉴别的方法有很多种，而且在一个系统中往往是多种方法并举，以获得更强的安全性。常用的方法有：用一个用户名或一个用户标识号来表明用户身份。系统内部记录着所有合法用户的标识，数据库系统鉴别此用户是否是合法用户，若是，则可以进入下一步的核实；若不是，则不能使用数据库系统。

数据库系统常常通过用户名和口令来鉴别用户身份，这种方法简单易行，但用户名和口令容易被人窃取，因此可以使用更复杂的方法。例如，每个用户都预先约定好一个计算过程或者函数，鉴别用户身份时，系统提供一个随机数，用户根据自己预先约定的计算过程或函数进行计算，系统根据用户计算结果是否正确进一步鉴别用户身份。用户可以约定比较简单的计算过程或函数，以便计算起来方便；也可以约定比较复杂的计算过程或函数，以便安全性更好。

2．存取控制

数据库安全性最重要的一点就是确保只授权给有资格的用户访问数据库的权限，同时令所有未被授权的人员无法接近数据，这主要通过数据库系统的存取控制机制实现。

存取控制机制主要包括两部分：

• 定义用户权限，并将用户权限登记到数据字典中。用户权限是指不同的用户对于不同的数据对象允许执行的操作权限。系统必须提供适当的语言定义用户权限，这些定义经过编译后存放在数据字典中，被称作安全规则或授权规则。

• 合法权限检查，每当用户发出存取数据库的操作请求后（请求一般应包括操作类型、操作对象和操作用户等信息），DBMS 查找数据字典，根据安全规则进行合法权限检查，若用户的操作请求超出了定义的权限，系统将拒绝执行此操作。

当前大型的 DBMS 一般都支持两种存取控制机制：自主存取控制机制（DAC）和强制存取控制机制（MAC）。

（1）自主存取控制机制

自主存取控制是基于存取权限的概念，可以概括为：一个主体只能在获得对一个客体的操作权限后才能对客体进行操作；主体对它创建的客体拥有所有权，自然获得对客体的一切操作权限；一个主体可以把对某个客体的操作权限授予给另外一个主体，也可以收回该权限。

自主存取控制方法实现简单，效率高，但存在安全隐患，容易被特洛伊木马（Trojan horse）方法攻破。

（2）强制存取控制机制

强制存取控制克服了自主存取控制的安全漏洞。比较流行的一个模型是 Bell-LaPadula，

这个模型是基于系统元素密级的，每个客体被赋予一个安全级别，每个主体被赋予对一个安全级别的许可证。安全级别由两部分构成：密级和范围。密级有绝密、机密、秘密和公开四种。范围是系统中非分层元素集合的一个子集。

该模型定义了安全规则：

• 简单安全规则。仅当一个主体的安全级别支配一个客体的安全级别时，主体才能读取客体的内容。A 支配 B 的含义是，A 的密级高于或等于 B 的密级，B 的范围是 A 的范围的子集。

• 星（*）规则。一个主体可以向一个客体中写数据，仅当客体的安全级别支配主体的安全级别；星规则可以防止密级高的信息流向密级低的主体。

强制存取控制也有缺点，主要是它太严格，以至于在某些环境下是不能用的。

3．视图机制

进行存取权限控制时，可以为不同的用户定义不同的视图，把数据对象限制在一定的范围内。也就是说，通过视图机制把要保密的数据对无权存取的用户隐藏起来，从而自动地对数据提供一定程度的安全保护。

4．审计

用户鉴别、存取控制仅仅是安全标准的一个重要方面，不是全部。为了使 DBMS 达到一定的安全级别，还需要在其他方面提供相应的支持。“审计”功能就是 DBMS 达到 C2 以上安全级别必不可少的一项指标。

审计功能是在审计日志中自动记录用户对数据库的所有操作。DBA 可利用审计跟踪信息，重现导致数据库现有状况的一系列事件，找出非法存取数据的人、时间和内容等。

审计很耗费时间和空间，所以，DBMS 往往都将其作为可选特征，允许 DBA 根据应用对安全性的要求，灵活地打开或关闭审计功能。审计功能主要用于安全性要求较高的部门。

5．数据加密

对于高度敏感性数据，例如，财务数据、军事数据、国家机密，除采用以上安全性措施以外，还可以采用数据加密技术。

数据加密是防止数据库中数据在存储和传输过程中失密的有效手段。加密的基本思想是根据一定的算法将原始数据（明文）变换为不可直接识别的格式（密文），从而使得不知道解密方法的人无法识别数据的内容。

加密方法主要有两种：一种是替换方法，该方法使用密钥将明文中的每一个字符转换为密文中的一个字符；另一种方法是置换方法，该方法仅将明文中的字符按不同的顺序重新排列。单独使用这两种方法中的任意一种都是不够安全的。但是将两种方法结合起来就能提供相当高的安全程度。采用这种结合方法的例子是美国 1997 年制定的官方加密标准——数据加密标准（Data Encryption Standard，简称 DES）。

由于数据加密与解密是比较费时的操作，而且数据加密与解密程序会占用大量系统资源，因此，数据加密功能通常也作为可选特征，允许用户自由选择是否加密数据。

小　　结

本节主要介绍了与数据库保护有关的技术，它包括数据库的恢复技术、并发控制技术、完整性约束与控制以及安全防范技术等内容。

习　　题

1．数据和信息的关系如何？
2．简述计算机数据管理技术的发展过程。
3．数据库系统有哪些特点？
4．DBMS 有哪些主要功能？
5. 举例说明实体间的各种类型的联系。
6．解释术语：DB，DBA，DBMS。
7．数据模型有哪几种？
8．举例说明层次模型、网状模型和关系模型。
9．关系模型有什么特点？
10．关系数据库系统有哪些特点？
11．关系运算有哪些？
12．简述关系数据库中实体完整性规则和参照完整性规则。
13．使用例 8 的三个关系，试用关系运算查询下列问题：
（1）学生“张华”选修的全部课程；
（2）学生“张华”所在的系和班级；
（3）“大学物理”成绩的最高和最低分数；
（4）没有选修“数据库技术”的学生；
（5）学生姓名以及平均成绩。
14．SQL 语言有哪些功能？
15．根据例 8 中给出的关系（基本表），用 SQL 语言完成以下问题：
（1）计算机系的女同学的“大学物理”的平均成绩、最高分数、最低分数；
（2）980111 班男、女生的人数；
（3）选修“社会主义建设”的学生学号、姓名、班级；
（4）查询与陈若星选修相同课程的学生的学号和姓名；
（5）王老师讲授的课程，包括课程编号和课程名称；
（6）至少选修了两门课程的学生姓名、性别；
（7）“张伟”的系别、班级以及选修的课程和分数；
（8）各门课程的总分、平均分和不及格人数；
（9）请将课程名称“网络计算”修改为“网格计算”；
（10）请在“学生”基本表中插入一条学生的信息（具体值自拟）。
16．数据库设计包含哪几个阶段？各阶段的任务是什么？
17．什么是事务？事务的四个特性是什么？
18．简述数据库恢复技术。
19．简述一级、二级、三级封锁协议。
20．简述数据库的完整性和安全性的区别和联系。

第 5 章　网络技术基础

计算机网络是计算机技术和通信技术密切结合的产物，已成为计算机应用中一个必不可少的方面。信息网络化已成为信息社会的重要标志。本章首先介绍计算机网络的基础知识，然后讨论 Internet 网络及与网络协议相关的问题，最后简要介绍网络编程接口及 HTML 语言与网页制作。

5.1　计算机网络概述

网络是当今计算机应用领域中最热门的话题之一，计算机网络的推广和普及使计算机产生了一次革命。计算机网几乎无所不在，从公共管理系统，到家庭娱乐；从广域网系统，到局域网系统；从信息共享到个人通信，网络使一个个单独的机器组成了一个有机的整体，像一块块砖瓦构建了一座大厦，全球变成了一个紧密的整体。所有这一切，都源于两个主要技术的大发展，这就是计算机技术和通信技术，而这两种技术的紧密结合则形成了计算机网络。

5.1.1　计算机网络概念

一、什么是计算机网络

从概念上讲，计算机网络是指通过数据通信系统把地理上分散的自主计算机系统连接起来，以达到数据通信和资源共享的一种系统。所谓自主计算机系统是指具有独立处理能力的计算机系统。计算机网络是在计算机技术和通信技术高度发展的基础上，两者相互结合的产物。一方面，通信系统为计算机之间的数据传送提供最重要的支持；另一方面，计算机技术渗透到通信领域中，又极大地提高了通信网络的性能。

根据上述定义，从系统的角度看，计算机网络可分为资源子网和通信子网两个部分。资源子网是指连接在网上的所有计算机系统的集合，其主要任务是完成信息的收集、存储和处理。通信子网是指通过通信线路和网络连接设备（如网卡、集线器、网桥、路由器等）把各种计算机系统互相连接起来的数据通信系统，其主要任务是完成不同计算机之间的通信与信息传输，并完成对通信的控制和管理。系统结构示意如图 5.1 所示。

二、计算机网络的功能

计算机网络的主要目的是实现资源共享。它随应用环境和实现条件不同，其用途也有一些不同，但其主要功能有资源共享、数据传输、负荷均衡和分布处理。

（一）资源共享

一般计算机中的资源可分成三大类，即软件资源、硬件资源和数据资源，因此资源共

享也可分为以下三类。

1．硬件共享

为发挥巨型机和特殊外围设备的作用，满足用户要求，计算机网络应具有硬件资源共享的功能。例如，某计算机系统 A 由于无某特殊外围设备而无法处理某些较复杂的问题时，它可将处理该问题的数据连同有关软件一起送到拥有这种特殊外围设备的系统 B 中去，由系统 B 对该数据进行处理，处理完后再把有关软件及其结果返回给计算机系统 A。

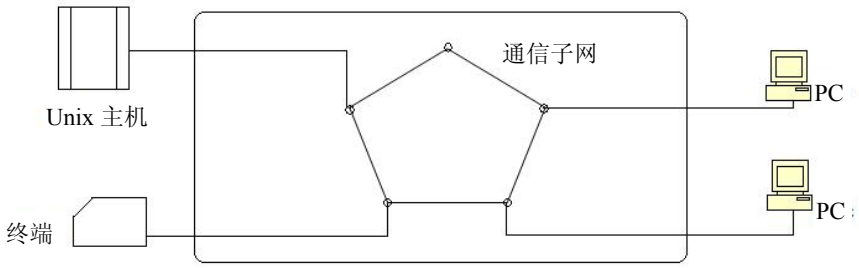


图5.1　计算机网络结构

2．数据共享

随着信息时代的到来，数据资源的重要性也越来越大。各发达国家都已经建立了成千上万个拥有各类资源的大型甚至巨型数据库，供全国乃至全世界的各类不同的用户查询。如产品供求信息数据库、人才库、气象信息库等。事实上，现代计算机网络中是否设置了大型数据库，设置了哪些类型的大型数据库，往往是衡量一个国家计算机网络先进水平的重要标志，尤其是当今发展势头强劲的分布式数据库处理系统，它把计算机网络技术和数据库技术有机地结合起来，使用户能够方便存取几千里之外的数据，使全球成为一个整体。这种数据共享的应用极大地推动了信息社会的发展。我国亦建立了大量的大型数据库，供全国用户查询使用。另外，随着 Internet 在我国的广泛应用，各种各样的在线信息亦在网上发布，用户可上网查询。

计算机网络中有服务端处理和客户端处理两种方式实现数据共享。

（1）当计算机系统甲需要系统乙中的数据时，可将请求信息送至计算机系统乙中，由乙对请求信息进行处理，最后将请求处理的结果通过计算机网络返回计算机系统甲中，这就是客户/服务器模式，即 C/S（Client/Server）模式。这种模式的主要思想是：由客户发出请求，服务器进行处理并只将处理结果返回给客户，这样就大大节约了在网络上传输的信息量，从而大大提高了整个计算机网络的效率。目前还有一种很流行的浏览器/服务器模式，即 B/S（Browser/Server）模式，它是对 C/S 模式应用的扩展。客户端运行浏览器软件，客户端的请求以超文本形式传给 Web 服务器，Web 服务器接受客户端请求后，将这个请求转化为对数据库服务器的访问，数据库服务器得到请求后进行数据处理，然后将处理后的结果返回给 Web 服务器，Web 服务器将得到的结果转变成 HTML 文档形式，转发给客户端浏览器以友好的 Web 页面形式显示出来。（2）与服务端处理方式相反，当计算机系统甲需要计算机系统乙中的数据时，由乙根据甲的请求信息将整个状态下有关的数据内容送至甲，由甲自行处理。这种方式的缺点是占用网络通信量大，要求计算机系统甲应有足够的处理

能力，这种方式已趋于淘汰。

3．软件共享

计算机网络可提供共享的软件，包括各种语言处理程序和各式各样的应用程序，实现软件共享的方法也有两种：

（1）当计算机甲需要计算机乙中的软件 A 时，甲将数据 D 送至乙，由乙利用 A 对 D 进行处理后，再将结果送回甲。

（2）计算机甲请求计算机乙把软件 A 送至甲，由甲自己处理。

（二）数据传输

该功能用于实现计算机与终端，计算机与计算机之间的数据传输，这是计算机网络最基本的功能，也是实现其他几个功能的基础。为实现数据传输，数据通信功能应包含以下几个内容。

（1）连接的建立和拆除。

（2）数据传输控制。

（3）差错检测。

（4）流量控制。

（5）路由选择。

（6）多路复用（即将一条物理链路虚拟为多条虚电路，使一条物理链路能为多个用户同时提供信息传输功能）。

（三）负荷均衡和分布处理

（1）负荷均衡

这是指网络中的工作负荷被均匀地分配给网络中的不同计算机系统。当某系统的负荷过重时，网络能自动将该系统中的一部分负荷移至负荷较轻的系统中去处理。为此，网络必须具有把本地作业传送至其他计算机系统中的批处理系统，待远程计算机系统处理完后又把结果返回该系统的功能。

（2）分布处理

即将一个大型任务分散到网上的多台机器中去进行处理。

5.1.2 计算机网络分类

一、计算机网络分类

计算机网络的分类标准有很多，按网络规模大小和距离可分为局域网、城域网、广域网；按网络的交换功能可分为电路交换、报文交换、分组交换和混合交换（同时采用电路交换和分组交换）4 种；按网络拓扑结构可分为总线型结构、星型结构、环型结构、树型结构、网状结构 5 种；按有无域控制器可分为对等网络和客户/服务器网络；按局域网网络标准可分为以太网、令牌环网、FDDI 及 ATM 等；按信号传递方式又可分为有线网络和无线网络。下面介绍其中主要的几种。

1．局域网

局域网指位于相对有限区域（例如一幢大楼）内的一组计算机、打印机和其他设备连接起来的通信网络，局域网的范围通常在几千米，一间办公室、一座大楼内，甚至家里两

台计算机组建起来的网络也叫做局域网，因此在一般情况下局域网主要是相对地理位置而言的。

2．城域网

顾名思义，城域网就是一个城市或者地区的主干网络，也叫做区域网，如图 5.2 所示。城域网的范围介于局域网与广域网之间，一般覆盖一个地区或一个城市。它对硬件和软件的要求比局域网高，在技术上比较先进。

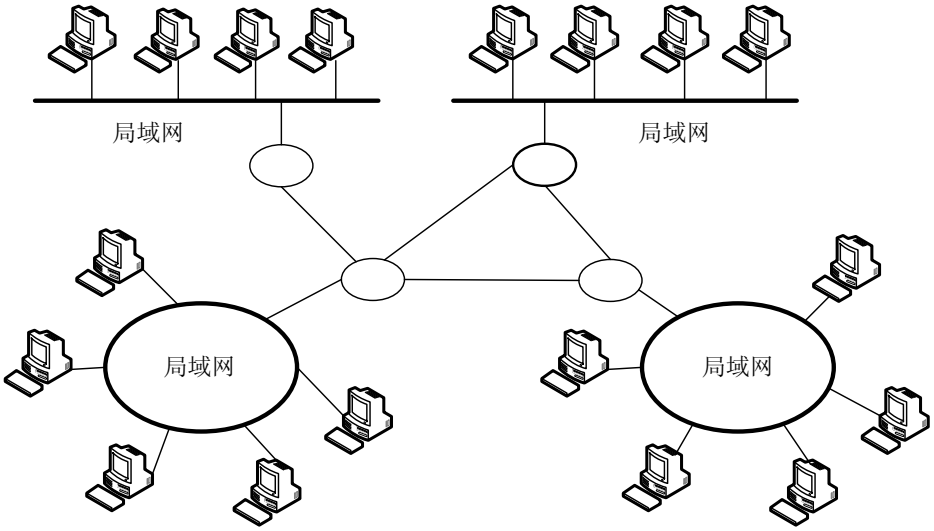


图5.2　城域网

3．广域网

广域网是一种可跨越国家及地区的遍布全球的计算机网络，如图 5.3 所示是连接城市网络的广域网。广域网一般以高速光缆、微波天线或卫星等远程通信形式连接，它是全球计算机网络的主干网络，通过广域网把城域网、局域网连接起来，这样就形成了多样的计算机网络。

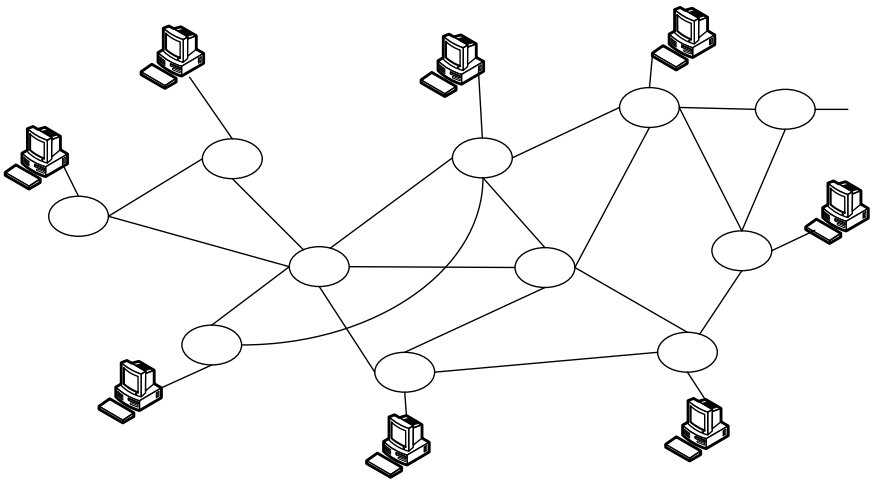


图5.3　广域网

4．无线局域网

无线局域网是计算机网络与无线通信技术相结合的产物，它利用了无线多址信道的一种有效方法来支持计算机之间的通信，并为通信的移动化、个性化和多媒体应用提供了可能。简单地说，无线局域网就是采用无线技术连接的方式来提供有线以太网或者令牌环网的功能。目前实现无线传输的技术有蓝牙传输技术、IEEE 802.11 传输技术以及家庭网络的HomeRF 技术等。

二、网络拓扑结构

在介绍网络结构以前，先对网络的拓扑结构作一介绍。因为要实现计算机之间的通信，必须要有实体的线路把它们连接起来，这里就有一个如何把这些计算机连接起来的问题。在网络中可以把构成网络的每一台计算机及共享设备看成网络上的一个节点。通常把地理位置分散的网络节点互联的几何逻辑布局称为网络的拓扑结构。网络拓扑结构决定了网络的工作原理及信息的传输方式，网络的拓扑结构不同，网络的工作原理及信息的传输方式也不同。按网络拓扑结构，通常可以把网络分为以下几类。

1．总线型网络

由一条高速公用总线连接若干个节点所形成的网络称为总线型网络，如图 5.4 所示。其中一个节点是网络服务器，由它提供网络通信及资源共享服务，其他节点是网络工作站（即用户计算机）。总线型网络采用广播通信方式，即由一个节点发出的信息可被网络上的多个节点所接收。由于多个节点连接到一条公用总线上，因此必须采取某种介质访问控制规程来分配信道，以保证在一段时间内只允许一个节点传送信息。目前最常用的控制规程有CSMA/CD 和总线令牌传送访问控制规程等。

总线型网络具有以下特点：

（1）传输速率高。可利用高速信道来连接多个节点，其传输速率可达 1～100Mb/s，当使用 1000Mb/s 的快速以太网时速率可达 1000Mb/s。

（2）信道利用率高。由于多个节点公用一条传输信道，故信道的利用率较高。

（3）地理覆盖范围小。公用线的长度受到一定的限制，通常小于几千米，节点至总线的连接线也较短，故总线的地理范围一般局限于某个单位。

（4）网络建造容易，成本低。由于网络的物理结构简单，将节点连接到总线上也容易；相应地，传输控制结构也简单，故这是一种较易实现的计算机网络。

（5）可靠性差。一旦总线的某段出了毛病，则整个网络将陷入瘫痪状态。

2．星型网络

每一个远程节点都通过一条单独的通信线路，直接与中心点连接。即中心节点与每一个远程节点之间，都采用点-点的连接方式，中心节点是其他节点的唯一中继节点，如图 5.5 所示。

星型网络具有以下特点。

（1）可靠性好。这种网络的可靠性比总线型高，当一个从中心节点（如 HUB）到终端的线路出现故障时不会影响到其他节点。

（2）功能高度集中。整个网络的处理和控制功能高度地集中于中心节点。

（3）响应时间与终端数目有关。当终端数目较少时，终端的请求能获得及时的响应，

但随着终端数目的增多，响应时间也随之加长。

（4）单信息流通路径。每个终端通常只有一条信息流通路径到达中心节点，反之亦然，因此不存在路径选择问题。

（5）线路利用率低。每条线路只连接一个终端，使该线路利用不充分。

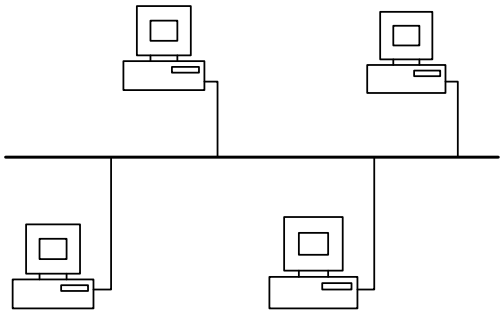


图5.4　总线型网络

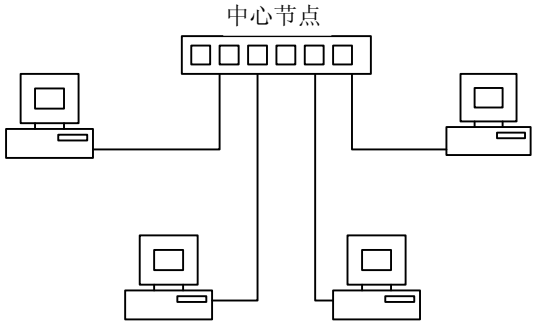


图5.5　星型网络

3．环型网络

在环型网络中，每台入网的计算机都先连接到一个转发器上，再将所有的转发器通过高速信道连成一个环形，如图 5.6 所示。网络中的信息是单向流动的，从任一源转发器所送出的信息，经环路传送一周后，又都返回到源转发器。为了控制各个联网计算机对环路的访问，在环型网络中，也同样可有多种介质访问控制规程。现已列入国际标准的规程有令牌环介质访问控制规程。

环型网络具有以下几个特点：

（1）网络建造容易。由于网络中的每个转发器都只与相邻的两个转发器相连接，这使网络结构简单，且介质访问控制也不复杂，故使网络的建造比较容易。

（2）传输时延的确定性。从某源点发出的信息，能在确定的时间内到达目标节点。基于这一特点，可构成实时性要求较高的网络，如工业控制网络等。

（3）可靠性差。当环路上任何一个转发器或者两个转发器之间的连接发生故障时，都将导致整个网络瘫痪。

（4）灵活性差。无论在增加或减少网络节点时，都需断开原有环路，并对介质访问进行调整。

4．网状型网络

在网状型网络中，计算机系统通过接口信息处理机 IMP（Interface Message Processor）连入网络，IMP 专门用于实现数据通信。将多个 IMP 通过点-点式信道连接成不规则的网状网。由 IMP 构成的网络被称为数据通信网，简称为通信子网，如图 5.7 所示。凡是需要入网的计算机（HOST）都应连接到 IMP 上，而各 HOST 之间必须通过通信子网方能进行通信。通常把通信子网以外的计算机和终端设备等一起称为数据处理子网或资源子网。网状型网络是广域网中最常采用的一种网络形式。

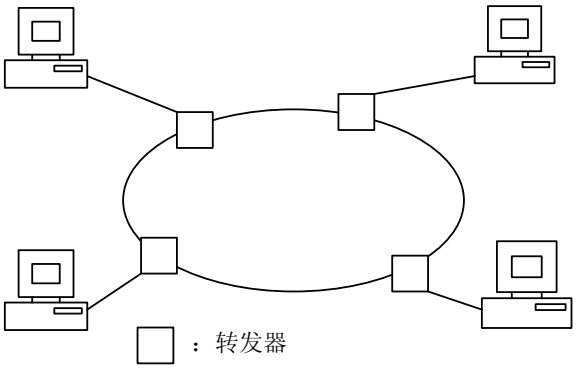


图5.6　环型网络

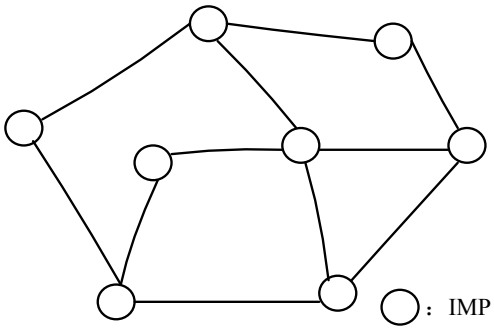


图5.7　网状型网络

网状型网络具有以下特点：

（1）网络可靠性高。由于通信子网中的任意两个 IMP 之间，都存在着两条或两条以上的通信路径，当一条通信路径发生故障时，可以通过另一条路径把信息传送到目标 IMP。

（2）可扩充性好。该网络无论是要增加新的功能，还是要将另一新的计算机入网，以形成更大的或更新的计算机网络时，都可方便地实现。

（3）灵活性好。可组建成各种形状、采用多种通信信道、多种传输方式及传输速率的网络。

（4）两级网络形式。网状型网络在逻辑上可以分为通信子网和资源子网两部分，前者专门用于实现网络通信；后者主要用于数据处理。这种两级网络形式的最大好处是便于将各种类型的计算机连接成异构型计算机网络。

那么，在实际的网络中，网络拓扑结构又是怎样安排的呢？事实上，我们想一下也可以知道它的大概安排，比如说，北京与成都建立了网络连接，我们可以在北京和成都分设两个节点，所有的通信都通过这两个节点，当然在成都内部，也可能有很多的分节点，这与电话网相似，因此，网络的连接方式（或地址）也是分层的。

另外，如果在一个实验室中组建了一个局域网，因为机器不是很多，并且范围较小，可以把每一台都以上面的节点方式来安排，即任何两台机器都用线连接，这种方式虽然可以，但很不实际，并且扩展性也很差。因此，在实际的局域网中，通常都采用共享信道方式来实现。

从上述分析可知，实际网络的拓扑结构一般是分层的网状的节点连接，即同层节点之间是点-点连接的一个网状结构；不同层节点之间构成一个树型结构；同层节点对父节点一般是一个星型结构；叶子节点采用共享线的挂接或星型结构，这要取决于所采用的具体局域网技术。

5.1.3　计算机网络体系结构

计算机和计算机网络技术的发展使网络设计者不仅要考虑计算机系统的体系结构，还必须考虑计算机网络的体系结构。在计算机应用的早期，一台计算机是一个系统，但现在一个网络才算得上是一个系统，一台计算机变为网络上的一台设备，因此从系统的角度，可以把计算机网络看成是一种新型的计算机系统。随着网络规模的扩大，应用的增加，复

杂性的提高，网络的设计和实现变得越来越困难。

计算机网络体系结构是控制和管理网络中各组成部分的互连和互操作的规则的集合，其核心问题是如何确定和实现不同计算机系统之间通信的方法和结构。网络体系结构包括传输的数据格式、协议和网络连接的逻辑结构等内容。现代计算机网络通常采用高度结构化的设计方法，一种有效的方法是把计算机网络按功能划分为若干层，形成层次结构，每一层实现部分通信功能，通信是分层进行的。下面将比较详细地探讨这种结构化技术。

一、协议分层

1．为什么分层

在网络设计中，为什么要引入层的概念呢？

首先，因为网络系统过于庞大复杂，要组建这样的一个系统，必须对任务进行模块化实现，降低其复杂性，这可能是分层的初衷。

其次，层的引入使得网络上的开发更灵活，更具有扩展性和与别的系统的兼容性。因为每一层是向上层提供一定的服务，因此，若需要增加新的功能服务或考虑与别的系统的兼容，只需在相应的层上加入相应的接口模块。而其他各层都不需要修改。

第三，层的概念引入，使得应用更具有独立性，每一层可以集中处理它的任务，不用考虑其他层次的实现。

2．什么是“层”

“层”是与任务和功能联系在一起的，一个层就是软件和可能的硬件的综合体，它用于完成某些特定的功能或任务，对于它的上层而言，它是一些服务的提供者，上层利用它提供的接口来使用这些服务。对上层而言，这些服务都是透明的，即上层不管它的实现，只需要它能完成指定的任务。比如，你是一个经理，有一份报告需要打印出来，你只需把手稿交给秘书，秘书自然会给你完成这一打印任务，而秘书可能自己打印，也可能拿到打字处让专门的打字员打印。这里，作为经理这一层，它根本不管这些具体细节问题。对于层本身来说，它不只是一个服务的提供者完成指定任务，它也可能是利用其下层的服务来完成其自己的任务，实现其对服务的提供能力。

3．服务和接口

服务和接口是与层紧密地联系在一起的。读者都知道服务是什么意思，比如我们到饭店就餐，需要一杯饮料，可以招呼一下服务员，服务员会送来一杯需要的饮料，这就是一种服务。在计算机和网络上也是一样，比如，你需要打印一个文件，你可以发送一条打印命令，打印服务器会为你完成打印任务。对于复杂的系统，这一种服务形式是很重要的。

至于接口的概念，就更容易理解。就拿在饭店就餐来说，首先你要知道谁是服务员，事实上服务员就是一个接口，至于你的招呼则是在“调用”这一服务。在计算机中，接口的概念是很广的，接口就是你调用服务的入口，通过这一接口“服务器”能为你完成指定的任务或功能，如一般的函数调用，函数名就是一个接口。

4．协议

事实上，协议的概念也是广义的，协议就是交互双方的一个规则或协定。因此，协议几乎无所不在，无论在我们的日常生活中，还是国家之间的交往，都有某种协议在协调双方的交流。在计算机网络上，协议是与层分不开的，这里读者不要以为协议是上层与下层

之间的事，上层和下层的合约就是服务和接口，因为在网络中，交互的是两台计算机，每一端都是由相应的层所组成，这里的协议是这些两端对应层的合约。就拿最上层的应用层来说，你发给它的电子邮件，另一端收到的应该是同一个电子邮件。又如，你的网络系统需要一个加密过程，你可以在表示层对从应用层来的信息或数据进行加密，表示层将利用其下层的服务把这一加密后的“内容”发送到另一端的表示层。当然在表示层之间就有一个加密的“协议”，通过这一协议，接收端才能把加密的内容还原成原来的信息或内容，即协议是对等实体之间的合约。

5．对等实体

对等实体也是和层联系在一起的，网络与一般外设不同的是，通信双方是对等的，即通信的两台计算机处于平等的地位，双方都可以给对方发送信息，并且接收的信息应是发送的信息，这两台计算机就是一个对等实体。当然，如果没有层，这样是很合理的，但是，由于每一端都是由一系列层组成，且双方都是一致的，因此，这一实体本身是由几个实体组成，这就是相对应的层。在通信的双方，这些层都是对应的，A 端的 N 层对应于 B 端的 N 层等。这些对应的层就称作对等实体。

对等实体也就是某层的一个实体，因为它需要为上层提供服务，很关键的是定义它的“服务访问点 SAP”。另外，对于网络来说，主要是传送数据，因此，必须有一个用来描述传输的数据格式的基本单位，即需要一个协议传输单元。为什么叫做协议传输单元？因为传输的数据要在对等实体间识别，因此，需要有一定的协议来保证。

二、ISO/OSI开放式网络体系结构

在网络发展的初期，随着网络需求和应用的不断增加，网络产品的数量和种类也不断增加，许多计算机厂家都自行定义了各自的网络体系结构。这些网络体系结构的层次、层次划分的方法、各层具有的功能和遵循的协议等都不尽相同，只适用于它们各自公司的计算机联网。面对众多不同的计算机和网络生产厂家，以及用户要求的灵活方便的联网需求，必须建立一个大家公认的计算机联网标准和网络体系结构，以实现任意计算机系统和任意网络系统的互连。国际标准化组织（ISO）提出了开放系统互连 OSI（Open System Interconnection），确定了开放式网络体系结构标准框架，即 ISO/OSI 参考模型，并在此框架的基础上提供了参考模型中各层的协议和服务标准，以支持不同系统的互连互通。ISO/OSI 参考模型可简单地用图 5.8 表示，它把网络分为七个层次：物理层、数据链路层、网络层、传输层、会话层、表示层和应用层。对网络应用软件设计者来说，应用层是很关键的，这就是如何利用表示层提供的服务来开发网络应用。下面对各层作一简单介绍。

1．物理层

物理层涉及在通信信道上传送实际的比特信息流。设计本层的目的是要确保一方发送二进制数字“1”时，另一方收到的是“1”而不是“0”。这里的典型问题有：使用多少伏特的电压代表“1”，多少伏特的电压代表“0”；一个比特持续多少微秒；传输是否在两个方向上进行；最初的连接如何建立和完成通信后连接如何终止；网络插座有多少针及各针的作用等。物理层主要是处理机械的、电气的、过程的接口，以及物理层下面的物理传输介质等问题。物理层的设计完全可以作为电气工程领域内的问题来处理。

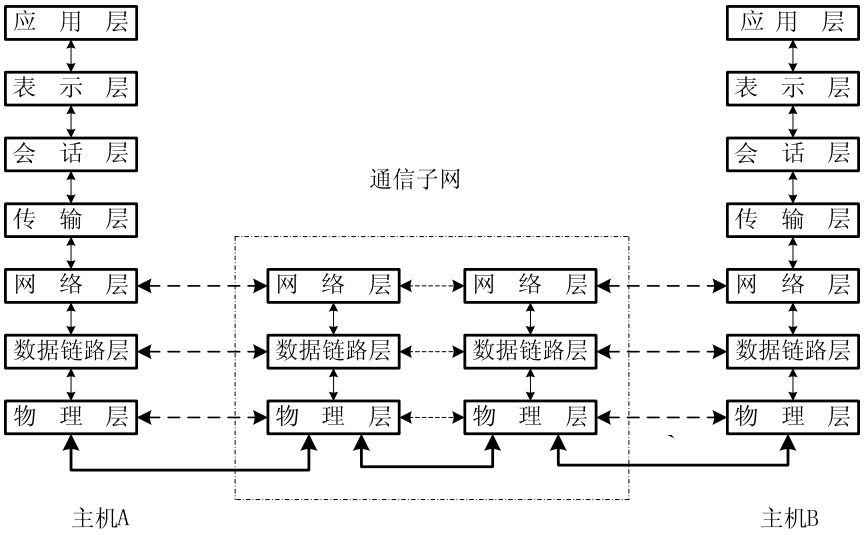


图5.8　ISO/OSI模型

2．数据链路层

数据链路层的主要任务是加强物理层传输原始比特的功能，使之对网络层表现为一条无错线路。发送方把输入数据分装在数据帧（Data Frames）里（典型数据帧可装几百个字节），按顺序传送各帧，并处理接收方回送的确认帧（Acknowledgement Frames）。因为物理层仅仅接收和传送比特流，并不关心它的意义和结构，所以只能依靠数据链路层来产生和识别各帧之间的边界。为此，可在帧的开始和末尾附加上特殊的二进制编码来达到这一目的。如果这些二进制编码偶然在数据中出现，还必须采用特别的措施以避免混淆。传输线上突发的噪声干扰可能把帧完全破坏掉，在这种情况下，发送方机器上的数据链路层软件必须重新发送该帧。然而，相同帧的多次传送也可能使接收方收到重复帧。比如，接收方给发送方的确认帧被破坏后，就可能收到重复帧。数据链路层要解决由于帧的破坏、丢失和重复所出现的问题。数据链路层可能向网络层提供几类不同的服务，每种类别具有不同的服务质量和价格。

数据链路层要解决的另一个问题（在大多数上层中也存在），是防止高速的发送方的数据把低速的接收方“淹没”。因此，需要某种信息传送流量调节机制，使发送方得知接收方当前还有多少缓存空间。为方便起见，流量调节和出错处理通常是放在一起来实现的。

如果传输线路能用于双向传输数据，数据链路软件还必须解决新的麻烦，即从 A 到 B 数据帧的确认帧将同从 B 到 A 的数据帧竞争传输线路。

3．网络层

网络层关系到子网的运行控制。其中一个关键问题是确定分组从源端到目的端如何选择路径。路径既可以采用网络中固定静态选径表（几乎不变），也可以在每一次会话开始时确定（通过终端对话决定），还可以根据当前网络的负载情况，非常灵活地为每一个分组决定路径。

如果在子网中同时出现过多的分组，它们将堵塞通路，降低传输效率。这种避免拥挤和堵塞的控制也属于网络层的功能。因为拥有子网的人总是希望他们提供的子网服务能得

到报酬，所以网络层常设有计账功能。一般来说，要求网络软件必须对每一位顾客发送了多少分组、多少字符或多少比特计数，以便生成账单。当分组跨过国界时，双方税率不同时，计账就更加复杂。

当分组跨越一个网络到达目的地时，又会产生很多新的问题。第二个网络的寻址方法也许与第一个网络不同；第二个网络也可能因为分组太长而无法接收；两个网络使用的协议也可能不同；等等。网络层必须解决这些问题，使异种网络能够互联。在广播网络中，选径问题很简单，因此网络层的功能常常很弱，甚至根本不存在网络层。

4．传输层

传输层的基本功能是从会话层接收数据，在必要时把它们划分成较小的单元，传递给网络层，并确保到达对方的各段的信息正确无误。而且，这些任务必须高效地完成。从某种意义上讲，传输层使会话层不受硬件技术变化的影响。

通常，会话层每请求建立一个传送连接，传输层就为其建立一个独立的网络连接。如果传送连接需要较高的信息吞吐量，传输层也可能为之创建多个网络连接，让数据在这些网络连接上分流，以改善吞吐量；另一方面，如果创建或维持一个网络连接不合算，传输层能使多个会话层连接复用到一个网络连接上来。

传输层也要决定向会话层，最终向网络用户提供什么样的服务。最流行的传送连接是一条无错的、按发送顺序传递信件的点-点通道。其他可能的传送服务有不保证传输顺序的独立报文传输和多目标广播报文。采用哪种服务在建立连接时决定。

传输层是真正的“源到目的地”或称为端-端的层。换言之，源端机上的某程序，利用报文头和控制报文，与目的机上类似的程序进行对话。在传输层下面的子层中，协议是每台机器与它的直接相邻机器之间的协议，而不是最终的源端机和目的机之间的协议。在它们中间，可能还隔着多个 IMP。在七层协议中第 1～3 层间是链接起来的，第 4～7 层是端到端的。很多主机有多道程序，这意味着多条连接将进出于主机，因此需要以某种方式区别报文属于哪条连接。识别这些连接的信息可以放入传输层的报文头。

除了将几个报文流多路复用到一条通道上，传输层还必须管理跨网连接的建立和拆除。这就需要某种命名机制，使机器内的进程能够表明与谁交谈。另外，还需要有一种机制来调节信息流，使高速主机不会过快地向低速主机传送数据。

5．会话层

会话层允许不同机器上的用户之间建立会话关系。会话层允许进行类似传输层的普通数据的传送，在某些场合还提供了一些有用的增强型服务，并允许用户利用一次会话在远端的分时系统上登录，或者在两台机器间传递文件。

6．表示层

表示层完成某些特定的功能，对这些功能人们常常希望找到普遍的解决办法，而不必由每个用户自己来实现。值得一提的是，表示层以下各层只关心从这里到那里可靠地传送比特，而表示层关心的是所传送的信息的语法和语义。

7．应用层

应用层包含大量人们普遍需要的协议。例如，世界上有很多种不兼容的终端类型，假定网络上有不同类型的终端，各种终端表示的屏幕格式、插入、删除和光标移动的字符等都可能不同，全屏幕编辑时的困难便可想而知了。解决这个问题的办法之一是定义一个抽

象网络虚拟终端，所有的编辑程序和其他程序都由虚拟终端处理。再通过一个应用软件，把网络虚拟终端的功能映射到实际终端上去。

应用层另一个重要的功能是文件传输。不同的文件系统有不同的文件命令规则，如文本的行有不同的表示方式等。不同的系统间传送文件时所需要处理的各种不兼容问题，也同样属于应用层的工作。此外还有电子邮件、远程作业录入、目录查寻和各种其他通用和专用的功能等。

5.1.4 网络交换技术

交换技术是网络发展史上的一个里程碑，它使人们对网络的低三层看法发生了根本性的改变。这一小节将介绍有关交换技术，以及它与共享信道技术的不同及优势。

交换技术的实质是“通信代理”的引入。用户与“通信代理”之间可以是独享或独占的信道，“通信代理”与“通信代理”之间同样也可以是共享或独占的信道，但由于“通信代理”的引入，使得要管理的竞争信道用户数减少，从而解决信道的分配也就变得简单了。由于半导体技术的高速发展，使得这种“通信代理”成为可能。

交换技术利用硬件对数据处理的高速性，通过多端口来扩展信道的总带宽。比如，一个 8 端口的交换以太网集线器，若每个端口有 100Mb/ s 的带宽（也可供多个主机来共享），因此它提供了一个 800Mb/ s 的总带宽。

交换技术已广泛应用于局域网和广域网。在以前，交换技术（如分组交换）主要用来构建广域网的主干网络，现在交换技术已不局限于这一领域，它已同样应用于局域网，并且，它可实现局域网和广域网的互联。

一、电路交换

当计算机发出一个电话呼叫时，电话系统中的交换设备在你的电话和接收者之间找寻出一条实际的铜线通路，被叫做电路交换。

电路交换的一个重要特点是：数据传输前，需要建立一条端-端的路径。从拨号到开始响铃，要让你等上几秒，国际和长途需要的时间更长。在这段时间中，电话系统不断搜寻一条物理路径。注意：在数据传输开始之前，呼叫请求信号必须沿到达目的地的所有路径传播，并得到确认。然而许多计算机应用（例如，销售点卖主信誉核实等）不希望有较长的建立连接时间。

通话双方建立一条实际线路的好处是，一旦通路建立起来，则数据的唯一延迟就只有电磁信号的传播时间，大约是每一千公里 6 毫秒。这种方式的另一个优点是，没有拥塞的危险，即一旦话路接通，就不再听到忙音。

二、分组交换

另外一种经改进的交换策略是报文交换。使用这种交换技术，在接收者和发送者之间事先并不建立实际通路。当发方有数据块要发送时，先将其存在第一个交换局，随后再传到下一个交换局，每次一步（一个链路）。每个数据可以作为一个整体接收、查错和重传。使用这种技术的网络叫做存储转发网络。

另一个可能的方式是分组交换。报文交换对信息块的大小没有限制，这意味着交换局

必须具有足够大的存储器对长数据块进行缓存，这也意味着单个数据块就有可能较长时间占用一条交换局到交换局的线路。由此可见报文交换不适合于交互式通信。与之相反，分组交换网有严格的数据块的上限，使分组能在 IMP（消息处理机）的主存储器而不是在外存储器中缓存。由于分组交换网能够保证任何用户都不能长时间独占任何传输线路，因而它很适合交互式通信。分组交换的确比报文交换更优越，在具有多个分组的报文中，第二个分组尚未完成之前，已经收到的第一个分组就可以往前送了，这样就减小了传输时延，提高了吞吐量。由于这些原因，计算机网络往往采用分组交换，在少数情况下也使用电路交换，但不使用报文交换。

电路交换和分组交换在许多方面有所不同，关键的区别是电路交换预先静态地保留所有的带宽，而分组交换却是根据需要动态地获得和释放带宽。在电路交换中，已分配线路上的无用带宽只有浪费掉。而在分组交换中，因为线路不是为一路通话专用，未用的带宽却可以为别的源站到目的站间的其他分组所利用。然而，也正是由于这种线路的非专用性，一个突然涌来的输入可能“淹没”一个 IMP，超出它的存储能力，导致丢失分组。

与电路交换不同，当使用分组交换时，IMP 很容易提高速度和代码转换，也可提高一定程度的纠错功能。然而，在某些分组交换网中，分组到达目的站时可能不按原顺序，需要在接收方重新按序组装，而在电路交换网中，却不会发生错序问题。

两种方法的最后一点区别是计费方法的不同。分组交换通信公司是根据所传送的字节（或分组）数和连接时间这些方面计费的，除了某些国际间的通信外，传输距离通常不影响收费。然而线路交换的收费只计距离和时间，而不考虑通信量。

三、混合交换

随着计算机和通信技术日益密切地结合，出现了电路交换和分组交换的变形和混合形式，下面简要地介绍它们中的几种。

分组交换的出现主要是为了解决现存电话系统中接通时间太长这个问题。一个虽然昂贵，但却直接得多的方法是建立一个新的电话系统，话路在几毫秒内而不是几秒内被接通。因而，这种方法叫做快速连接电路交换，在终端上每输入一行，立刻导致终端内的微处理机“拨通”计算机，发送这行，然后挂断。

正如快速连接电路交换网络是电路交换的变种一样，分组交换也有变种。一种特别有趣的变种是时分交换，这种交换方式是每个 IMP 按严格的规律循环扫描输入线，然后将收到的分组立即送到合适的输出线路，常常是一读到分组头就往前送。通过固定分组大小和严格同步，不再需要缓冲空间，因而整个 IMP 可以缩小到几块芯片。其主要优点是使用现有电话系统中的技术，能以低费用提供高性能的吞吐量。

四、帧中继交换

要建设信息高速公路，就必须建设高速网络，目前已有多种不同的高速网络技术在同时发展，如帧中继、异步传输模式 ATM（Asynchronous Transfer Mode）及综合业务数字网 ISDN 等。

早期的分组交换网主要采用模拟信道，其质量较差，误码率较高。到 20 世纪 80 年代后期，通信的主要干线已逐步采用光缆，光缆不仅大幅度地提高了传输速率，而且使传输

误码率降低了几个数量级。此外，网络中所用的通信设备的可靠性也显著提高。这些都使信息在传输过程中发生差错的几率减小，因此在 X.25 网络的基础上，通过简化其差错控制（包括检测、重发和确认）、流量控制和路由选择功能，而形成了一种新型的交换技术。由于 X.25 分组网和帧中继网很相似，因而很容易从 X.25 升迁到帧中继。

五、异步传输方式ATM

前面介绍的报文交换是以报文为单位，分组交换是以分组为单位进行交换，两者都是在网络层上进行的；而帧（中继）交换则是以帧为单位进行的交换，它是在数据链路层上进行的。异步传输方式又称为信元（中继）交换，以信元为单位，同样是在数据链路层上进行的。异步传输方式，即 ATM 是建立在大容量光纤传输介质的基础上，它比帧中继具有更高的传输速率，在短距离时高达 2.2Gb/s；在中、长距离时也可达每秒几十兆位（Mb/s）到每秒几百兆位。ATM 技术既吸取了分组传输模式中标记复用方式的优点来提高信道利用率，又吸取了同步传输模式中位置复用的优点来简化传输控制机制，将两者结合而形成的一种崭新的高速交换技术。下面简单说明“标记复用”和“位置复用”两个概念。

1．标记复用

ATM 以信元（cell）为基本传输单位。信元由信头和信息段组成。ATM 通过信头来识别通路。在这种方式中，只要信道空闲，便将信元投入信道，这有利于提高信道利用率。由于信元的发送无固定周期，因而将这种传输模式称为异步传输方式。

2．位置复用

为了简化信元的传输控制，在 ATM 中采用了固定长度的信元，规定信元长度为 53 个字节，其中信头 5 个字节、信息段 48 个字节。这样，每个信元都花费同样的传输时间，因而可以把信道的时间划分为一个个的时间片序列，每个时间片用来传输一个信元。当交换器有信元发送时，便逐个时间片地把信元投入信道。接收时，若信道不空，也将逐个时间片地取得信元，时间片和信元一一对应。通过这种对应关系，可大大简化对信元的传输控制。在实现上可采用高速硬件对信头进行识别和交换处理。

由于 ATM 的高速性，使声音、图像和数据能够同时在 ATM 信道中传输。但应在传输前，将它们都分解成固定长度的信元序列，然后再送入信道中传输。但它们的信元发送频率各不相同，语音信元的发送频率低、视频信元的发送频率高。ATM 通过采用不同的信元发送频率来实现各种信息的多路复用。

小　　结

这一小节主要从整体上介绍了网络的有关概念，以及在网络中采用的有关低层技术（OSI 的下三层）。具体包括以下内容：

1．网络的概念，以及什么是通信子网、什么是资源子网。

2．网络的功能，包括资源共享、数据传输、负荷均衡以及分布处理等。

3．网络的分类。从地域范围上看，可把网络分为局域网、广域网、国际网等。

4．网络的拓扑结构。介绍了网络的拓扑结构的概念以及主要的拓扑结构，包括总线型结构、星型结构、环型网络、网状结构。

5．ISO/OSI 体系结构，介绍了网络分层结构，主要内容包括：为什么分层、什么是层、

服务和接口、协议、对等实体等，另外，重点介绍了 ISO/OSI 七层模型。

6．交换技术及其特点，重点介绍了几种常见的交换技术：电路交换、分组交换、帧中继以及 ATM 等。

5.2 Internet 网络及应用

5.2.1 Internet 概述

Internet 产生于世界处于冷战最激化的 20 世纪 60 年代。当时美国认为，一旦爆发世界大战，由于核武器的攻击，一般的通信方式将被破坏而无法使用，需要寻求一种先进的通信手段，来解决地理位置分散的最高军事首脑集团（RAND）中各个成员间的通信问题。为此建立了一个计算机网络，并成立了相应的项目研究组。

当时的网络采用闭合的环状结构，整个网络可靠性很低。为此该项目研究组的一位名叫 Paul Baran 的年轻技术人员提出应使用网状结构的通信网络代替原来的闭合的环状结构。采用这种结构，即使网络的某个部分遭到破坏，但可以寻找另一条通信路径，而使得整个网络仍然可以运行。他将该方案写成了厚厚的 12 册，提交给五角大楼，但是该方案在当时没有被重视。几年后的 1969 年，Paul Baran 的设想才得以实施，以此构筑了一个连接美国 4 所大学的小规模的分散型的计算机网络 ARPAnet（Advanced Research Project Agency：美国国防部高级计划研究局网络）。

ARPAnet 在建立的初期是一个规模很小的研究网络，但其发展速度非常迅速。到 1971 年它连接了 23 台主机。1973 年 ARPAnet 连接到了英国伦敦的一所大学和挪威的皇家雷达部队，开始成为真正的国际性的跨国家的网络。到 1981 年网络中的站点数量已经增加到 213 个，新的主机数量大约以每 20 天增加一台的速度增长。1982 年开始使用 Internet 这一术语，但当时还不是指现在的因特网，它表示将不同的网络连接到一起的技术。到 1984 年，入网的主机数量已经超过 1000 台。而到了 1989 年主机数量已超过十万台。1990 年，ARPAnet 宣告解体，代之的是一个真正的全球性的网络 Internet。1992 年入网的主机数量突破一百万台。到 1996 年，大约有 150 个国家的超过一千万台主机加入到 Internet 中。

网络的使用目的从早期的主要用于军事研究，发展到一般企业、教育科研、政府机构乃至家庭个人使用。Internet 能够得到如此迅速的发展和广泛的应用，其中具有各种各样的原因。而特别是进入 20 世纪 90 年代以来，其发展速度更是惊人。其主要原因是在 Internet 中产生了 WWW 技术。由于该技术的引入，使得用户可以方便地实现在各种系统平台的 Internet 主机上进行操作，并且对客户计算机的硬件资源和软件资源要求非常低。

5.2.2 TCP/IP 协议

由于 ISO/OSI 体系是一个非常庞大的系统，至今采用 ISO/OSI 体系的产品还很少。实际的网络一般采用非 ISO/OSI 的其他几种网络体系结构（TCP/IP、XNS、DNA、SNA 等）。目前广泛采用的 TCP/IP 协议（Transmission Control Protocol/Internet Protocol：传输控制协议/网络互连协议），已成为事实上的工业标准，得到网络厂家和用户的大力支持。下面将介绍

TCP/IP 网络体系结构，并与 ISO/OSI 参考模型相比较。

一、TCP/IP体系结构

TCP/IP 的历史要追溯到 20 世纪 70 年代中期，它最初出现于 ARPAnet 中。1983 年，美国加利福尼亚大学伯克利分校推出了内含 TCP/IP 的第一个 BSD UNIX 操作系统。该协议软件可谓生逢其时，因为当时许多大学的院系正缺乏一种他们迫切需要的联网手段，以建造各自的局域网。BSD UNIX 在网络方面的成功有以下原因：首先，除提供标准的 TCP/IP 应用程序外，它还支持一组网络服务工具程序。这些工具的调用格式与 UNIX 命令调用格式相似，深受 UNIX 用户欢迎；第二，BSD UNIX 提供一种供应用程序访问通信协议的操作系统调用——socket。socket 是一种进程间通信机制，是 UNIX 输入 / 输出机制的推广。

TCP/IP 本身是 TCP 层和 IP 层的合称，前者也就相当于七层协议的传输层，后者也就是网络层。但现在一般所说的 TCP/IP 不仅仅包含这两层，它是一个协议簇，其中包括很多协议，如 UDP、ICMP、ARP 以及一些路由协议和应用层协议等。因此，TCP/IP 定义了一种网络，即 TCP/IP 网络，比如我们最熟悉的 Internet 就是一个 TCP/IP 网络。

这一部分将介绍有关 TCP/IP 网络概念，以及重点把 IP 和 TCP 与网络层和传输层对应进行分析和介绍。

TCP/IP 体系结构与 ISO / OSI 不完全一致，TCP/IP 主要定义了四层，如图 5.9 所示。从下到上分别是网络接口层、IP 层、TCP/UDP 层以及应用层。与 OSI 相比，网络接口层对应了 OSI 中的下两层（物理层和数据链路层），它没有定义表示层和会话层，这都包含在应用层之中。在这四层中，最关键的是 IP 层、TCP/UDP 层，它们对应于 OSI 的网络层和传输层，在传输层有两个协议 TCP 和 UDP，应用最广的是 TCP，它提供了可靠的面向数据字节流服务。在下面几个部分，将分别介绍 IP 和 TCP 的有关内容。

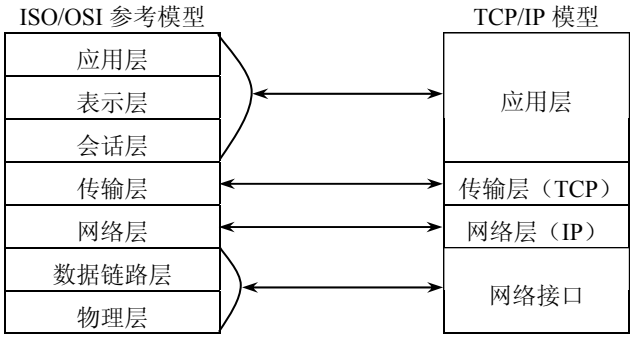


图5.9　TCP/IP模型

二、网络层协议（IP）

1．IP 层的特点

首先，IP 层位于通信子网的最上层，提供无连接的数据报服务；其次，IP 协议是点到点的；第三，IP 层对上层屏蔽了下层物理网的复杂性和多样性，因此，使得 TCP/IP 网得以运行于各种不同的物理网中。

2．IP 数据报

IP 数据报有两层含义，一个是指 IP 层是无连接的，它不保证传输的可靠性；另一个是 IP 数据报的格式。

IP 数据报的报头最短为 20 字节，它主要包括以下一些字段：信源和信宿地址（都是一个互联网地址，各 32 位）、标识、标志、片偏移、总长、头标长、TTL 字段、协议等。

三、传输层协议（TCP和UDP）

传输层是计算机网络（包括多个网络互联的互联网）体系结构中至关重要的一层。传输层的作用是向信源机（源主机）与信宿机（目的主机）提供端-端数据传输，而传输层以下各层只提供相邻节点的点-点传输。从“点-点”到“端-端”协议改变，传输层为此要引入许多新的概念和机制，传输层协议也要为此做许多工作。

在 TCP/IP 协议簇中，定义了 TCP 和 UDP 两个传输层协议。其中，TCP（Transport Control Protocol：传输控制协议）是面向连接的；UDP（User Datagram Protocol：用户数据报协议）是无连接的。

一般情况下，TCP 和 UDP 共存于一个互联网中，前者提供高可靠性服务，后者提供高效率服务。高可靠性的 TCP 用于一次传输要交换大量报文的情形（比如，文件传输、远程登录等）；高效率的 UDP 用于一次传输交换少量报文（尤其是交互型应用，比如，数据库查询）的情形，其可靠性由应用程序提供，因为交换次数不多，即使发生传输错误而重传，应用程序也不会为此付出太高的代价。

为了包容子网的各种不可靠因素，TCP 协议要做大量的工作，解决有关传输层涉及的各种可靠性问题，因此 TCP 协议非常复杂。而 UDP 几乎直接建立在 IP 协议之上，相对简单得多。UDP 的价值在于其效率，当通信子网相当可靠的时候，UDP 更是大有用武之地。

在 TCP/IP 网络中既提供 TCP 又提供 UDP，这从侧面告诉我们一个重要的道理，在计算机网络中，可靠性并不比效率更重要，光有可靠性而无效率的协议在某些场合是根本没有用的，对二者的选择取决于应用的环境和需求。

1．传输层端口

传输层与网络层在功能上的最大区别是前者提供进程通信能力，后者不提供进程通信能力。在进程通信的意义上，网络通信的最终地址就不仅仅是主机地址了，还包括可以描述进程的某种标识符。

为此，TCP/UDP 提出协议端口（Protocol Port，简称端口）的概念，用于标识通信的进程。端口相当于 OSI 的传输层服务访问点（TSAP），它是一种抽象的软件结构（包括一些数据结构和 I/O 缓冲区）。应用程序（即进程）通过系统调用与某（些）端口建立联编后，传输层传给该端口的数据都被相应进程所接收。

从另一个角度说，端口又是进程访问传输服务的入口点。在分时操作系统中，一台机器同时可运行若干并行进程，这些进程要同时访问传输服务怎么办？端口机制为此提供了一种应用程序员熟悉的途径。在 TCP/IP 实现中，端口操作类似于一般的 I/O 操作，进程获取一个端口，相当于获取一个本地唯一的 I/O 文件，可以用一般的读写原语访问端口。类似于文件描述符，每个端口都拥有一个叫做端口号的整数标识符，用于区分不同端口。由于 TCP 和 UDP 是完全独立的两个软件模块，因此各自的端口号也相互独立，比如，TCP 有一个 266 号端口，UDP 也有一个 266 号端口，但这两个端口并不冲突。

按照TCP和UDP协议规定，二者均允许长达 16 比特的端口值，所以TCP和UDP软件分别可以提供 2^{16}=64K个不同端口。

端口号分配是一个重要的问题。我们考虑不同机器上两个进程相互通信的情形。假如 A 进程要与 B 进程通信，A 必须知道 B 的地址（包括主机地址和端口号）。主机地址是全网唯一的，很容易获知。端口号不一样，每台机器都有一套自己的端口号，只能做到本地唯一，而不能做到全局唯一，那么 A 如何知道 B 对应的端口号呢？有关这一问题，TCP/IP 采用了客户/服务器模型等有关机制来解决。

2．TCP 协议

TCP 的特点是高可靠性，但它本身是建立在不可靠的 IP 协议之上，所以 TCP 的可靠性需要由其自身来保证。为此，在 TCP 协议中主要采用了以下技术：

- 确认和超时重传
- 滑动窗口协议
- 三次握手的连接管理

（1）TCP 协议传输单元

从数据流来看，TCP 是一个无结构的字节流，流中的数据是一个个字节构成的序列，而无任何可供解释的结构，这一特点使得 TCP 协议传输单元是不定长的。其协议传输单元主要包括以下几个部分：源和目的端口（各占 16 位）、序号、确认号、头标长度（因为 TCP 头是可变的，它包含可选的“选项字段”）、码位和窗口等。

（2）滑动窗口和流量控制

在面向连接的传输过程中，发送方每发出一个分组都需要得到收方的确认。对于面向连接的数据传输的最简单协议是：每发一个分组，等待确认；收到确认后，再发下一个分组。这种协议存在一个很大的缺点就是效率太低，在等待确认的过程中，本连接拥有的网络资源没有充分利用起来。这种协议叫做简单停等协议。

与上述简单停等协议相对的另一个极端是无确认数据报传输。发送方一味地向网络注入数据，而不管网络是否拥塞，对方是否收到。当然这种方式可靠性就很难保证。

介于上述二者之间，提出了一种关于面向连接传输的折衷方案，既可充分利用连接所提供的网络能力，又能保证可靠性。这种方案允许一次连续传输若干数据报文而不必等待其中任何一个的确认，但连续发送的报文数不得超过一定限制。

与这种方案密切相关的机制称为滑动窗口协议。滑动窗口内含一组顺序排列的报文序号。在发送端，窗口内的报文序号对应的报文是可以连续发送的，具体地说，这些报文有三种情况：已发送但尚未得到确认；未发送但可连续发送；已发送且已得到确认，但本窗口中前面尚有未确认报文。各报文按序发送出去，但确认不一定按序返回。一旦窗口前面部分（不管多少）报文得到确认，则窗口向前滑动相应位置，落入窗口的后续报文又可连续发送。

在接收端，窗口内的序号对应于允许接收的帧。窗口前的帧是已收到且已发回确认的帧，是不允许接收的，窗口后的帧要等待窗口滑动后才能接收。

发送窗口的大小限制发送端一次可发送的最大分组数，它会影响滑动窗口协议的性能。如果发送窗口大小为 1，便是简单停等协议。从表面上看，接收窗口的大小似乎无关紧要。事实并非如此，跟接收窗密切相关的是分组重组的问题。当接收窗大小为 1 时，意味着接

收端只允许顺序接收分组，无分组重组问题；当接收窗无限大（除已确认报文外）时，意味着接收端可以接收任意顺序的分组，需要做完全的重组工作。

接收窗大小反映了接收端缓冲区的大小。假如接收窗口大小为 n，则接收方至少要有 n 个接收缓冲区，否则会引起不必要的报文重传。

滑动窗口协议的一个重要用途是流量控制，网关和接收端可以通过某种方式（比如 ICMP：Internet Control Message Protocol 报文）通知发方改变其窗口大小，以限制发方报文注入网络的速度，达到流量控制和拥塞控制的目的。

（3）三次握手的连接管理

所谓建立传输连接，就是在两个传输用户之间建立一种逻辑联系，彼此达成共识，承认对方为自己的传输连接端点。连接（Connection）在内部表现为一些缓冲区和一组协议机制，在外部则表现出比无连接高的可靠性。

建立一个连接，只需按序交互发送“连接请求、指示、响应、确认原语”就可以了。比如，在客户与服务器之间建立连接时，客户发送一个请求报文，服务器若愿意建立连接，则发回一个响应报文。这样连接就建立起来。这是一个合情合理的过程，很容易理解。

然而，问题并非如此简单。因为通信子网不总是那么理想的，并不是所有子网都能保证分组正确及时地传到信宿端。假如分组丢失怎么办？解决分组丢失很简单，常用的是超时重传机制。客户发出一连接请求时，同时启动一个定时器，不管请求或者响应丢失，定时器总会超时溢出。一旦定时器超时，客户再次发起连接请求，并再启动定时器。直到成功建立连接，或当重传次数到达一定限度时，认为连接不可建立而放弃。

最难解决的问题是，请求根本就没有丢失，而是在子网中存储起来，过一段时间后，又突然出现在服务器端。在无连接数据报子网中，尤其可能出现这种情况。这就是所谓延迟重复问题。

为此，TCP/IP 协议采用了一系列机制来保证可靠性。

1）消除重复分组

其基本思想是删除在网络中滞留的陈旧分组。方法是给每一分组赋予一生存时间（Time To Live，简称 TTL）。分组一进入网络，立刻开始生存时间记数，到规定的时间若还未到达目的机，则一律被抛弃，从互联网中清除。

生存时间一般取决子网中的最长路径延时，其取值能否反映网络延时的变化，对网络的性能有极大影响。假如生存时间值取得过长，可能导致大量重复报文；反之，又可能导致大量重传。

生存时间不一定用时分秒来定义，也可以是允许报文经过的转发站点数等其他可以限制报文滞留的因素。在诸因素中，时间（真正的时间）还是最科学的。转发站点数可以限制不好的路径，尤其是循环路径，却不能限制在转发站点处的延时。

但消除重复连接请求往往依赖于子网来完成，因为报文在传输过程中是不会到达传输层的。IP 数据报的 TTL 域及有关机制便是常用的例子。

2）基于本地时间的“初始序号”

一般情况下，一个连接只需通过两端的“TASP”（传输层服务访问点），这是在连接建立后。但是，若由于重复延时分组的存在，避免重复连接，采用了基于本地时间的“初始序号”，这样，在建立连接时，它的初始序号是基于本地时间的，因此，这样就可以根据初

始序号与本地时间的关系来判断一个连接是否是过时的。但由于计算机上的时间本身不是线性的（按步增长的），以及其他一些原因，初始序号的确定也只在一个非线性的区域内，为了保证初始序号的安全性，还采用了下面三次握手的技术。

3）三次握手

三次握手方法首先要求对本次连接的所有报文进行编号，常用的方法是取当前时钟的最低 n 位作为初始序号。由于序号域具有足够的长度，可以绝对保证序号循环一周回来时，使用同一序号的旧报文早已传输完毕。这样，网络中不会出现关于同一连接、同一序号的两个不同报文。

三次握手解决的问题是：假设 A 机向 B 机发出连接请求，但请求报文丢失，而一个延迟的重复的旧请求报文到达 B，如果 B 接受此请求，连接就会错误地建立起来。

面向连接的传输要求 A 机和 B 机以同步方式进行报文传输，双方发出的第一个报文均给出一个本地独立的初始序号。在双方一来一回的报文交换过程中，“回”报文要对收到的上一个“来”报文加以确认。

在三次握手方法中，第一次，A 机向 B 机发出连接请求（简称 CR），其中包含 A 机端的初始报文序号（比如 X）；第二次，B 机收到 CR 后，发回连接确认（简称 CC），其中，包含 B 机端的初始报文序号（比如 Y），以及 B 机对 A 机初始序号 X 的确认；第三次，A 机向 B 机发送 X 序号数据，其中包含对 B 机初始序号 Y 的确认。

5.2.3 IP 地址与域名

一、IP地址

我们知道，不同的物理网络技术有不同的编址方式。不同物理网络中的主机，有不同的物理网络地址。互联网技术是将不同物理网络技术统一起来的高层软件技术，在统一的过程中，首先要解决的就是地址的统一问题。互联网采用一种全局通用的地址格式，为全网的每一网络和每一主机都分配一个互联网地址，以此屏蔽物理网络地址的差异，从而保证互联网以一个一致性实体的形象出现。

关于地址，首要的要求是唯一性，即同一系统中一个地址只能对应一台主机。这很容易理解，也很容易做到，但互联网地址远非如此简单。

地址实际上是一种标识符，用于标识系统中的某个对象。通常标识符被分为三类：名字（name）、地址（address）和路径（route）。名字标识对象是什么，地址标识对象在哪里，路径指出怎么去访问对象。

事实上，严格说起来，许多地址并没有指出对象的位置。比如，以太网地址就不含位置信息，它只是将不同网络站点区别开来的简单标识符，但对于局域网（尤其是广播型局域网）来说，这已经足够了。

无位置信息的地址是简单的，能力也是有限的。试考虑广域网环境中的情形。我们知道广域网结构复杂而无规则，一般采用点到点连线构成总体结构，且主机数目非常庞大。假设在广域网中采用无位置信息的简单标识符作为地址，那么从一台机器中发出的报文如何才能到达目的机？有两种办法：第一是像以太网一样采取广播方式；第二是逐台主机进行试探。可以想象，这两种方法都是很笨拙的，开销巨大。事实上，正如 ARPAnet 一样，

广域网都采用有结构的地址标识符，不仅标识主机，还要指出主机的位置。

在 ARPAnet 中，主机地址形如（P，N），其中 P 代表分组交换节点（PSN）号，N 代表与该 PSN 相连的某主机号。这样的层次型地址模式真实全面地反映了 ARPAnet 的网络结构，因而极为有效，极大地提高了寻径的效率。

TCP/IP 协议规定，每个互联网地址是一个 32 位的无符号整数，但为了提高寻径的效率，互联网采用了层次式的地址，把一台主机地址分为两部分：网络号和主机号。并且，在 IP 协议中，为了更好地体现这一机制，它还提供了子网的概念。现在广泛使用的 IP 地址表记法为十进制分段法。它是将 32 位二进制地址划分成 4 个 8 位，各个部分用一个十进制数表示，它们之间用圆点分开。例如，用二进制表示的地址为 10000101 00000100 00000010 00000001 的网络地址，用十进制表示为 133.4.2.1。在互联网中 IP 地址被分为 A、B、C、D、E 五类。

A 类：最高位（二进制表示）为 0 的地址，其后紧接着的 7 位为网络部分，其余的 24 位为主机部分，如图 5.10 所示。A 类地址的范围为 0.x.x.x～127.x.x.x。

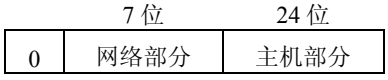


图5.10　A类IP地址

由于网络部分和主机部分全部为 0 或为 1 时具有特殊意义，因此实际可以分配的网络数为 126，一个网络内的最大主机数为 16777214。

B 类：最高两位为 10，其后的 14 位为网络部分，余下的 16 位为主机部分，如图 5.11 所示。B 类的地址范围为 128.0.x.x～191.255.x.x。可分配的网络数为 16383，一个网络内的最大主机数为 65534。

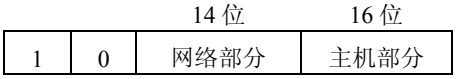


图5.11　B类IP地址

C 类地址：最高 3 位为 110 开始的地址，其后的 21 位为网络部分，余下的 8 位为主机部分，如图 5.12 所示。C 类地址的范围为 192.0.0.x～223.255.255.x。可分配的网络地址数为 2097151，一个网络内的最大主机数为 254。

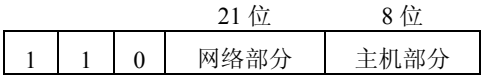


图5.12　C类IP地址

另外，以 1110 开始的地址为 D 类，以 11110 开始的地址为 E 类。D 类地址用于 IGMP 的 IP 多点通信中识别主机组。E 类地址被预留为试验研究用。为寻径方便和对互联网地址的充分使用，TCP/IP 协议中采用了子网编码技术，通过这种方法，可以把一个子网（网络号相同）进一步分成多个独立的网，它通过把主机号进一步划分成子网号和主机号来实现

的，其具体方法是由子网掩码来决定。子网掩码是一个 32 位的数，与互联网地址相对应，子网掩码中为 1 的位，构成子网号。

上面介绍的 IP 地址方案称为 IPv4 版本，是目前采用的 IP 地址分配协议。随着加入互联网的计算机和设备越来越多，这个方案所分配的地址数将被用尽，无法满足计算机网络发展的要求。为此，国际 IETF（Internet Engineering Task Force：互联网工程任务组）设计了用于替代现行的 IPv4 的下一代 IP 地址分配协议 IPv6 方案。IPv6 中使用 128 位地址，具有更大的地址范围。目前，我们国家已经开始在进行 IPv6 网络的试验，预计在不远的时间内将推广使用 IPv6 地址方案。

二、域名系统

虽然 Internet 上每个主机都有一个唯一标识自己的 IP 地址，但是一长串数字很难记忆，因此在 Internet 系统中使用了符号名字来命名每一个主机，它是由域名系统 DNS（Domain Name System）来实现的。DNS 是一个分层定义和分布式管理的命名系统。它由两个部分构成：域名的命名规则和域名服务系统。

DNS 采用了分层次的命名方法，每一层为一个子域名。子域名之间使用点号分隔。一个域名从右向左分别为最高层域名、机构名、网络名、主机名。其中，网络名又可以分成多级子网名。例如，域名 nitl.ccse.uestc.edu.cn 表示中国（cn）教育科研网（edu）电子科技大学（uestc）计算机学院（ccse）的一台主机（nitl）。

域名服务系统实现域名与 IP 地址之间的转换，它是一个分布式的数据库系统，由域名空间（Domain Space）、域名服务器（Domain Server）和地址转换请求程序（Resolver）三部分组成。域名空间成一个倒立的分层树型结构。每一层组成一个子域名空间，结构中每个节点定义了主机的域名与对应的 IP 地址等信息。Internet 在每个子域设置域名服务器，服务器中包含有该子域中的域名与 IP 地址信息。Internet 的每台主机都有地址转换请求程序，负责将地址转换请求传送到本地子域名服务器。当域名服务器接收到地址转换请求后，如果是本地子域中的域名，则直接从数据库中取出对应地址发回给主机；否则将请求传到上一层域名服务器，并等待接收上一层发回来的对应地址。

最高层域名由 Internet 的最高管理中心授权分配登记，在美国被用来区分不同的机构，在美国以外的国家或地区被用作国家或地区识别标识，如表 5-1 所示。

表 5-1　最高层域名的分配示例

域名	意义	域名	意义	域名	意义
com	商业组织	au	澳大利亚	fr	法国
mil	军事组织	br	巴西	gb	英国
edu	教育机构	ca	加拿大	hk	中国香港
net	网络机构	cn	中国	in	印度
gov	政府机构	de	德国	it	意大利
int	国际机构	dk	丹麦	jp	日本
org	其他机构	es	西班牙	kr	韩国

5.2.4 Internet 服务

Internet 是信息的海洋，人们可以从 Internet 获得各种各样的信息。其中可以是文字和数字的信息，也可以是声音和图形图像信息。除了一般的资源收集外，在 Internet 上还可以做许多其他事情，如网上购物、网上电影院、网上寻医、网上对弈、网络红娘、网上学校、网上交谈等，它给人们的生活、工作和学习带来了莫大的方便。

针对不同的大量的应用，Internet 提供的服务种类非常多，包括电子邮件、文件传输、远程登录、交互式服务、超文本服务 WWW、搜索引擎、广域网信息服务 WAIS、新闻群组 Usenet、电子广告牌 BBS 等。下面简要介绍几个常用的服务。

一、WWW服务

WWW（World Wide Web）是一个基于超文本的信息检索工具，其用户界面友好。WWW 将所有 Internet 数据都作为超文本进行组织，用户可以非常方便地找到想要的信息。WWW 除了可以浏览传统的文本信息外，还可以显示各种图形图像、影视、声音等信息。WWW 是目前 Internet 上最受欢迎和最先进的服务方式之一。

WWW 技术的出现是为了解决 Internet 上的信息传递问题。早期的信息传递主要是通过 E-mail（电子邮件）和 FTP（文件传输）等方式来实现。E-mail 主要用作团体或个人之间的信息交换，而 FTP 被用来从一台计算机向另一台计算机传输文件。

由于Internet是一个巨大的分散的网络，用户为了获取信息必须知道信息所处的位置（所在的主机名及访问路径）。这就大大限制了用户对网上资源的利用。针对这个问题，美国 Minnesota 大学开发出了 Gopher 系统。Gopher 系统采用具有体系结构的菜单方式进行信息的查询。用户通过选择查询主题，逐渐缩小信息的搜索范围。即从一般信息限制到具体要搜索的信息。Gopher 一问世就受到广大 Internet 用户热烈欢迎，被公认为是当时查找网上信息最有效的方法，因此被广泛使用。但是，由于其功能有限，对于要访问不同位置的信息还不是很方便；提供的内容格式还不丰富，只局限于字符文本；提供的用户接口界面也不是很友好；还有它对工作平台的依赖性也很大。

由于 Gopher 在使用上存在的一些问题和限制。引发了必须开发出更实用的信息查询系统。当时 Internet 在欧洲最大的站点，欧洲高能粒子物理研究所（European High-Energy Particle Physics Lab）开发出了一种超文本标识语言 HTML（Hyper Text Markup Language）。

这种语言具有与平台的无关性，并且简单易学。使用 HTML 来开发应用程序不仅可以融图形、声音和文本信息于一体，还有最重要的一点，就是可以实现超文本链接。

1990 年 CERN 的 Tim Berners-Lee 创建了 World Wide Web。World Wide Web 也称为 WWW、W3 或 Web，中文也叫做万维网。它基于 Internet，提供一个超文本多媒体的信息检索系统，它以一种快速、强大、通用、便捷的方式，提供访问不同格式信息的方法。WWW 采用了 HTML 语言来写，信息的传递采用 HTTP（Hypertext Transfer Protocol：超文本传输协议）协议。

在 WWW 世界中，主页 Home Page 是被广泛使用的一个术语。当启动了一个浏览器后，所显示的页面也叫做主页。每个浏览器都有一个缺省的主页。多数情况下是开发该浏览器的组织所在的主页。浏览器的缺省主页可以由用户随意改变。

主页的另一个含义是在 Web 上提供信息服务的人，为读者提供访问各种信息的入口所组成的一个集成页面。也就是说，主页是表示所提供信息的概要显示，并且提供了访问每个项目的 URL（Uniform Resource Locator：统一资源定位指针），用户只需要用鼠标简单地点击一下，立即就可以获得相应的信息资源。

二、远程登录

我们首先讨论远程登录协议。大家知道，任何一个多用户系统都有用户账号（user account）的概念。用户账号规定了用户对本系统的使用权限，用户登录进入自己的账号，便可以访问系统的部分或全部资源；无用户账号的人员不能访问系统。

一个远地用户像本地用户一样，在远地机上建立一个用户账号，并通过 TCP/IP 进入该远地账号，访问远地机资源，这就是远程登录。

本节讨论 TCP/IP 协议簇中的 TELNET 远程登录协议。

1．远程登录概念的提出

首先我们约定，将进行远程登录的用户叫做本地用户，本地用户登录进入的系统叫做远地系统。

远程登录的根本目的是访问远地系统的资源，而且像远地机的当地用户一样访问远地用户的资源。一个本地用户通过远程登录进入远地系统后，远地系统内核并不将它与本地登录区别开，因此远程登录和远地系统的本地登录一样可以访问远地系统的、权限允许的资源。反之，假如我们不采用远程登录的方式，则如何访问远地资源呢？显然可以采用单纯的客户/服务器方式，但是单纯的客户/服务器方式要求在远地系统上为每一种服务创建一个服务器。当远地系统提供一种很专门的服务时，这种方式是最佳的，但像用户登录这样的服务，单纯的客户/服务器方式就不适用了。因为用户登录后，可访问的资源（即服务）很多，比如 Unix 系统中，仅常用的 Shell 命令就有几十上百条，假如对每一种可访问资源都建立一个服务器，毫无疑问远地系统会很快被服务器进程阻塞，而且假如一些小型的普通的服务也用客户/服务器方式来实现，明显地是小题大做。

远程登录很好地解决了这个问题，它不要求远地系统创建众多的服务器，只需为每个远程登录用户建立一个进程（如 Shell 进程），这个进程再通过创建子进程为远程登录用户提供各种允许的服务。这样一组少量的动态进程代替了大量的静态的服务器进程，其效率是可想而知的。

远程登录的思想也体现了层次结构概念。远程登录用户所需要的是远地系统普遍的服务，服务类型可能是大量的（如各种 Shell 命令）且变化的（如随机动态调用）。远程登录的实现，使本地用户并不直接面对远地系统的各种资源，相当于在服务客户与具体服务之间加入一个中间层次，这个中间层是远程登录服务器。我们知道，分层结构是处理大量的、变化对象的常用方式，远程登录服务器的引入，代替了各种大量的具体服务器，避免了远地系统服务器进程数的膨胀。

远程登录的另外一个优点是，它提供与本地登录几乎完全相同的用户界面，使用非常方便。

2．TELNET 协议

TELNET 是一个简单的远程登录协议，其远程登录的服务过程分为以下三个步骤：

第一步，本地用户在本地终端上对远地系统进行远程登录。该远程登录的内部实际上是一个 TCP 连接。

第二步，将本地终端上的键盘输入逐一传到远地机。

第三步，将远地机的输出送回本地终端。

在以上过程中，输入/输出均对远地系统内核透明，远程登录服务本身对用户也透明，本地用户好像直接连入远地系统。这种透明性是 TELNET 的重要特点，也是 TELNET 内部机制的重要部分。

TELNET 提供三种基本服务。第一，网络虚终端服务。这是一个面向客户程序的标准接口，提供对远地系统的抽象描述。客户程序员不必了解各种远地机的内部细节，在此接口上可以建立通用的客户程序。第二，一组标准选项及客户、服务器之间的选项协商机制。第三，远程登录连接两端的对等性。比如连接的双方都可以发起选项协商，连接的双方都可以是程序（尤其是客户一端不一定是用户终端，任何程序都可以做客户）。TELNET 远程登录在连接的双方各由一个应用程序实现，本地应用程序是客户，远地应用程序是服务器。

远程登录的工作过程是：当本地用户调用 TELNET 时，本地机上的相应应用程序成为客户。然后客户与远地机上的远程登录服务器建立 TCP 连接。在此 TCP 连接基础上，客户将从用户终端接收的键盘输入传给服务器，同时接收从服务器返回的字符，通过本地操作系统将它显示在用户终端上。

上述过程中服务器在整个过程中究竟扮演什么角色？首先作为 TCP 连接的一端，它是必不可少的。另外，服务器的作用是在 TCP 连接与远地操作系统之间作为数据传输的中继器。我们知道，服务器实际上不能直接处理本地用户键入的字符，它收到这些字符后，必须将它交给远地操作系统处理。客户收到的返回数据也是来自于远地操作系统，而非服务器本身。

因此客户相当于远地系统中的一个终端，它跟远地操作系统打交道是通过一个操作系统入口进行的，这个操作系统入口叫做“伪终端”（pseudo terminal）。任何远程登录服务器应用程序必须建立在伪终端入口上，否则无法实现。伪终端在远地系统中是一个虚拟的字符设备文件，是对物理字符设备的抽象。从这个角度，我们可以说，服务器的作用是将远程登录的 TCP 连接从一个客户连到一个特定的伪终端上。

三、文件传输

文件是计算机系统中信息存储、处理和传输的主要形式，几乎所有的计算机系统都非常重视文件操作，计算机网络系统更不例外。

大多数计算机网络系统都支持网络文件访问功能，网络协议簇如 TCP/IP、ISO/OSI、XNS、SNA、DNA 等均提供各自专门的网络文件访问协议。可以说，网络文件访问是网络系统必备的基本功能，也是一般用户对网络系统的基本需求。

对网络文件访问的需求来自四个方面，第一，不同应用程序之间的数据共享，比如对中央数据库的共享；第二，利用远程存储介质存储后备文件，以防数据丢失；第三，无盘体系结构对文件服务器的依赖，无盘体系结构是指网络上有若干无硬盘站点（通常是工程工作站），由一个带大容量硬盘的有盘站点（叫做文件服务器）为它们提供文件存储功能，无盘站点通过网络访问服务器上的文件信息；第四，以文件形式进行的数据交换，比如上

级机关向下级传达指示，下级机关向上级上传报表等。

网络文件访问包括在线访问和全文拷贝两种独立的形式。前者简称文件访问，后者简称文件传输。

1．文件传输与访问

作为不同形式的网络文件访问手段，文件传输和文件访问的用户视图和内部实现都是截然不同的。在描述具体的文件传输和访问协议之前，有必要对这两个概念本身做一介绍。

文件传输提供的用户服务相对简单一些，用户直接将远程文件全盘拷入本地系统，或将本地文件全盘拷入远地系统。典型的例子是 Unix 的远程拷贝命令 rcp。

远地文件一旦拷入本地系统，便属于本地文件，与远地系统无关，用户可以对它进行任何权限内的操作。文件传输通常用于只读数据，不过也可以对远地拷贝过来的文件进行修改，再通过文件传输将它拷贝回远地系统。

在线文件访问则允许多个（远地）程序同时访问单个文件，任何一个程序都不需要被访问文件的拷贝，直接在原文件上进行操作，某程序对原文件的修改立即在原文件上表现出来，并为访问它的其他程序所感知。

文件传输与访问的最直接区别在于前者在实际读写文件之前，需将远地文件（原文件）拷贝到本地文件系统，然后对本地拷贝进行操作，应用程序与对象文件（本地拷贝）之间是一对一的关系。而文件访问没有拷贝过程，若干程序可同时在原文件上进行操作（原文件同时被若干程序打开并读写），应用程序与对象文件之间是多对一的关系。

2．TCP/IP 文件传输协议 FTP

文件传输是 TCP／IP 中使用最广泛的应用之一，早在 TCP/IP 出现之前，ARPAnet 中就有了文件传输的标准规范，该规范后来发展成为目前众所周知的文件传输协议 FTP（File Transfer Protocol）。

（1）FTP 的用户界面

在 BSD Unix 中，客户端 FTP 由一组 Shell 命令实现，其中最重要的命令是 ftp。客户端用户调用 ftp 命令后，便与服务器建立连接，这个连接叫做控制连接。用于双方传输控制信息，而非数据。一旦建立控制连接，双方进入交互式会话状态。然后，客户端服务器每调用一个 FTP 命令（除 ftp 外），客户与服务器之间再建立一个数据连接，双方可以传输数据（比如文件拷贝等）。该命令执行完后，再回到交互会话状态，可继续执行别的 FTP 命令。最后，用户键入 close 和 quit 命令，退出 FTP 会话。

FTP 的命令达 60 个之多，这里我们不做一一解释。它们的命名很接近 DOS 命令的命名风格。除了基本的文件传输外，这些命令还提供了许多附加的功能。FTP 的用户视图有以下特点：

l）采用交互式用户界面

客户端用户进入 FTP 后，便进入交互状态，就可以利用命令方便地与服务器对话。比如可以列出远地机上某目录下的文件，可以进入远地机某工作目录等。同时用户也可以调用本地 Shell 命令与客户进程对话。

2）可以指定文件格式

FTP 允许客户指定存储数据的类型和格式，比如客户端用户可以指定某文件是以文本还是二进制整数的格式存放。文本文件是使用 ASCII 字符集还是使用 EBCDIC 字符集等。

3）具有权限控制

在请求文件传输之前，FTP 要求客户必须首先向服务器提交正确的注册名和口令，否则服务器拒绝客户的访问。

（2）FTP 的客户/服务器模型

FTP 是基于客户/服务器模型而设计的，客户和服务器之间利用 TCP 建立连接。与其他客户/服务器模型不同，FTP 客户与服务器之间要建立双重连接，一个是控制连接，一个是数据连接。建立双重连接的原因在于 FTP 是一个交互式会话系统，某客户每次调用 FTP，便与服务器建立一个会话，会话以控制连接来维持，直至退出 FTP。控制连接负责传输控制信息，尤其是客户命令（比如文件传输命令等）。利用控制命令，客户可以向服务器提出无限次的请求（一个请求就是一个客户命令）。客户每提出一个请求，服务器就会与客户建立一个数据连接，进行实际的数据（比如文件）传输。一旦数据传输结束，数据连接相应地被撤销，但控制连接依然存在，客户可以继续发出命令，直到客户键入 Close 命令撤销控制连接，再键入 Quit 命令退出 FTP 会话（此时双方控制进程死亡）。

四、电子邮件

同文件传输应用一样，电子邮件（Electronic Mail，简称 E-mail）最早也是出现在 ARPAnet 中，是传统邮件的电子化。它的诱人之处在于传递迅速，在 ARPAnet 上，几秒钟就可以完成美国东西海岸间的邮件传递，在速度上是人工邮件无可比拟的。

跟另一种日常通信手段电话系统相比，电子邮件在速度上虽然不占优势，但它不要求通信双方同时在场就可完成信息的传递。

另外，电子邮件还可以进行一对多的邮件传递，同一邮件可以一次发送给许多人。最重要的是，电子邮件是整个互联网以至所有其他网络系统中直接面向人与人之间信息交流的系统，它的数据发送方和接收方都是人，所以极大地满足了大量存在的人与人通信的需求。

由于上述优点，电子邮件深受用户欢迎。出乎 ARPAnet 设计者的意料，人与人之间电子邮件的通信量一开始就大大超出进程间的通信量，使电子邮件成为 ARPAnet 上最繁忙的业务。因此，后来出现的通用的网络体系结构，几乎无一例外地都把电子邮件作为一个重要的应用，纳入自己的协议簇。

下面先简单介绍 ISO/OSI 的电子邮件系统，然后详细讨论 TCP/IP 电子邮件系统的原理，最后介绍 TCP/IP 的电子邮件协议。

1．电子邮件系统体系结构

表面上看来，电子邮件类似于文件传输，也是利用网络通信设施，将数据传到接收方。但是，与一般文件传输相比，电子邮件有两大特点。

第一，电子邮件的最终发送者和接收者都是人而非机器，于是电子邮件系统在结构上分为人机界面与邮件传输两大部分。人机界面用于发送者和接收者写作、编辑和读取邮件；邮件传输负责将邮件从发送端传到接收端。电子邮件的操作及其向用户所提供的信息非常接近于日常生活习惯。

第二，电子邮件是高度结构化的文件，因此对电子邮件的处理比对一般文件的处理复杂。可见，电子邮件与文件传输是极不相同的。不过就单纯的信息传输这个意义而言，前

者是后者的一个特例。那么，能否依赖于一般的文件传输建立电子邮件系统呢？理论上是可以的，但这样做必然大大降低电子邮件系统的效率。由于电子邮件与文件传输同属应用层，将它们区别开来，各自独立是最好的选择。

2．ISO／OSI 电子邮件系统

ISO/OSI 的电子邮件协议叫做 MOTIS（Message Oriented Text Interchange Systems，即面向报文的文本交换系统）。所谓报文（message）就是高度结构化的电子邮件。文本（text）类似于文本文件的概念，MOTIS 只处理文本型的电子邮件，文本交换相当于文件传输。

像所有别的电子邮件系统一样，MOTIS 要解决以下六个基本问题：

（l）报文的生成

这是电子邮件系统中用户界面的重要内容。它帮助用户写作和编辑邮件，并为邮件加入地址和大量别的控制信息。

（2）传输

这是电子邮件系统中独立于用户的部分，解决报文的传输问题。在 ISO/OSI 体系结构中，报文传输建立在表示层之上，它的具体操作包括建立连接、输出报文和释放连接等。

（3）报告

负责向发送者报告报文发送进展（是否送到、是否被拒绝、是否丢失等）。这一功能在许多需要确认的场合是至关重要的。

（4）转换

在发送端将信息转换成适合于在接收者终端上显示或打印的格式。

（5）格式化

解决报文在接收者终端上的格式化显示问题。报文显示格式的最直接处理方式是由电子邮件系统传来未格式化报文，由用户调用格式化程序进行处理，再调用显示程序（如编辑器）对格式化文件进行阅读。这种处理方式不适合无经验的用户。因此，最好是电子邮件系统能直接显示格式化报文，这样操作就大大简化了。

（6）报文处置

对应于报文生成，它是电子邮件系统用户界面的另一重要方面。它帮助接收者处理所收到的报文，包括立即扔掉、读完扔掉、读完后保存、阅读旧报文及转发报文等。

MOTIS 电子邮件系统分为两部分，一部分叫做用户代理 UA（User Agent），另一部分叫做报文传输代理 MTA（Message Transfer Agent）。UA 就是前面谈到的用户界面，负责报文生成与报文处置。MTA 是所谓“电子化邮局”，主要负责报文传输。

在电子邮件系统的具体实现中，UA 与 MTA 往往不在同一机器上。UA 一般放在个人计算机内，MTA 一般放在通信部门所管理的大型主机中，一个 MTA 可以带若干 UA。

在许多 MTA 上，都有一个叫做报文库（Message Store，MS）的设施，是 MTA 所在机器上的一个专用存储器。MS 为每一用户开设一个电子邮箱，到来的报文可以存放在邮箱中，直到用户登录后来处置它。

正是由于 MS 的存在，电子邮件系统允许发送任意长度的报文，并且允许通信的对方不在场，甚至允许通信对方 UA 所在主机不必随时处于开机状态。

MOTIS 的电子邮件传输是存储转发型的，其中的 MTA 扮演网关的角色，邮件经逐个

MTA 传输，直至信宿用户所在的 MTA。

3．TCP／IP 电子邮件系统

在概念上，TCP／IP 的电子邮件系统也分为用户界面和报文传输两部分，但报文传输部分并未独立出来，形成一个类似于 MOTIS 中 MTA 的概念。其中的原因在于 TCP/IP 自始至终坚持端-端的思想，它的电子邮件系统也不例外地采用端-端传输方式，不存在邮件的存储转发问题，因此也就没有必要引入像 MTA 那样的负责存储转发的机构。

在端-端方式中，虽然初始主机要参与邮件传输的全过程，但由于 TCP／IP 下层协议的简洁性，其效率反而比存储转发方式要高。

同 MOTIS 一样，TCP/IP 电子邮件系统要解决的首要问题也是收方主机（称为远地机）暂时不能访问（比如未开机、出故障、不能建立网络连接等）时，如何发送电子邮件的问题。在 TCP/IP 的术语中，这叫做延迟传递（delayed delivery）。为此 TCP/IP 采用 Spooling 缓冲技术，将用户收发报文与实际的报文传输区别开来，这种划分实际是 UA 和 MTA 划分的原型。

用户界面相当于 UA，客户/服务器相当于 MTA。用于发送邮件的 Spooling 区和接收邮件的邮箱相当于 MS，起缓冲作用。客户和服务器所在机器叫做邮件交换机（mail exchanger），用户发送邮件时，首先利用用户界面生成邮件，然后把它传给发送邮件 Spooling 区，相当于报文从 UA 到 MS。然后整个发送过程用户都不必关心，等待关于发送结果的报告就可以了。负责发送邮件的客户（相当于 MTA 的发送部分）是一个后台进程，它一发现发送缓冲区有邮件待发，立刻将其取出，并把信宿机名映射成 IP 地址，然后请求与对方的服务器（相当于 MTA 的接收部分）建立 TCP 连接。如果连接成功，便发送邮件，对方服务器将接收到的邮件存放在接收邮件邮箱（相当于 MS）中；发送完后，客户将相应邮件从发送缓冲区删除。假如连接不成功，客户记下发送时间。客户进程周期性地检查发送缓冲区，每当它发现未发邮件，或用户传来一个新邮件，客户立即着手发送。当发现某邮件很长时间（比如 3 天）都发不出去，客户将它返回发送者。

4．Internet 与电子邮件

Internet 作为异种网互联的产物，对于应用程序来说，它是一个统一的、一致性的单一网络，因此它可以提供全网上任意机器间的电子邮件交换。

Internet 电子邮件的另一特点是可靠性极高。Internet 电子邮件建立在 TCP 基础上，而 TCP 是能提供端-端可靠连接的。假如客户和服务器之间未成功建立 TCP 连接，并将邮件成功发送到服务器邮箱中，客户就不会将待发邮件从发送缓冲区删除。

5．TCP/IP 电子邮件地址

在 Internet 中，电子邮件地址格式如下：

Local-part@domain-name

其中，domain-name 是信宿邮件交换机的域名。在 Internet 中，一般是一个单位，比如一所大学或一个公司拥有一台邮件交换机。Local-part 是信宿用户在其邮件交换机上的邮箱名，可以是用户本身的姓名，或其注册名，或其他代号，只要在本单位内唯一就行。邮件交换机的域名在整个电子邮件系统中是唯一的。

电子邮件的寻径过程也是一个层次型的过程。发送方所在邮件交换机发现一个未发送

邮件，从信宿地址中取出收方交换机域名，将它映射成 IP 地址，据此与收方交换机建立 TCP 连接，并将邮件发送过去。收方邮件交换机收到邮件后，取出其信宿地址中的本地部分，据此将邮件放入合适的邮箱。

5.2.5 接入 Internet 方法

一、概述

这一节将介绍目前比较常见的 Internet 接入方法。对于企业级用户而言，多是采用专线入网的方式以局域网或广域网规模接入到 Internet。目前各地电信部分和 ISP 为企业级用户提供了分组网、帧中继、DDN 专线、微波无线、光纤接入等方式。而对于个人用户，一般都采用调制解调器拨号上网，还可以使用 ISDN 线路、ADSL 技术等方式。这里，我们将侧重于对个人用户的 Internet 接入进行阐述。

随着 Internet 的发展，就用户而言，在全国大部分城市都可以有多家 ISP 作为选择，所以，我们在选择 ISP 时候可以从以下几个方面去考虑。首先是网络拓扑结构，在选择 ISP 时，网络拓扑结构是至关重要的一个因素，因为 ISP 的网络拓扑结构直接影响到该 ISP 是否有自己的主干线路，网络负荷过重时的用户承载容量以及与其他网络的扩展连接能力、是否有多条出口线路，这样当某条线路处于瘫痪时，不致造成最终用户的连接失败；然后需要考虑的是主干线传输速率，ISP 的主干线传输速率将直接影响用户的连接速度；接下来是网络技术力量，要看 ISP 是否采用先进的网络设备，是否拥有足够多的电话中继线路以满足众多用户同时拨入的需要；最后，我们还要考虑到 ISP 的服务和价格，好的 ISP 不仅技术上具有先进性，价格上有优势，而且还应该提供良好的连接服务。

二、几种常用的网络接入方法

虽然有各种各样的 Internet 接入方式，我们这里侧重介绍几种当前在个人用户接入时比较常见的方式：调制解调器 MODEM 接入、ISDN 接入以及 ADSL 接入。

（一）调制解调器 MODEM 接入方式

调制解调器一般分为内置式调制解调器（调制解调器卡）和外置式调制解调器。内置式调制解调器价格较低，安装时需要将计算机主机箱打开后，才能安装；外置式调制解调器安装很方便，但价格相对较贵。

如果是一个内置式调制解调器，打开计算机主机箱，在空插槽中插入调制解调器卡。

如果是外置串口调制解调器，将外置式调制解调器连接到没有使用的 COM 端口上，并在启动 Windows 之前打开调制解调器的电源。

现在多数调制解调器都具有即插即用性能，将其连接到计算机，Windows 启动后会自动检测到，并提示安装过程。但是，老式的调制解调器和外部串行调制解调器可能不会被 Windows 自动检测到。如果调制解调器不能自动安装，可使用“控制面板”中的“电话和调制解调器选项”手动安装调制解调器。

当系统中正确安装了调制解调器后，从“控制面板”中的“电话和调制解调器选项”的“调制解调器”选项卡中可以看到所安装的调制解调器，如图 5.13 所示。



图5.13 “调制解调器”选项卡

1．建立拨号网络连接

（1）从“控制面板”或“附件”的“通讯”中进入“网络和拨号连接”窗口，如图 5.14 所示。

（2）双击“新建连接”图标，将启动“网络连接向导”。按照系统提示输入相应的信息，在“网络连接类型”菜单中选择“拨号到 Internet（D）”选项，使用调制解调器连接到 Internet，并单击“下一步”按钮，如图 5.15 所示。



图5.14 “网络和拨号连接”窗口

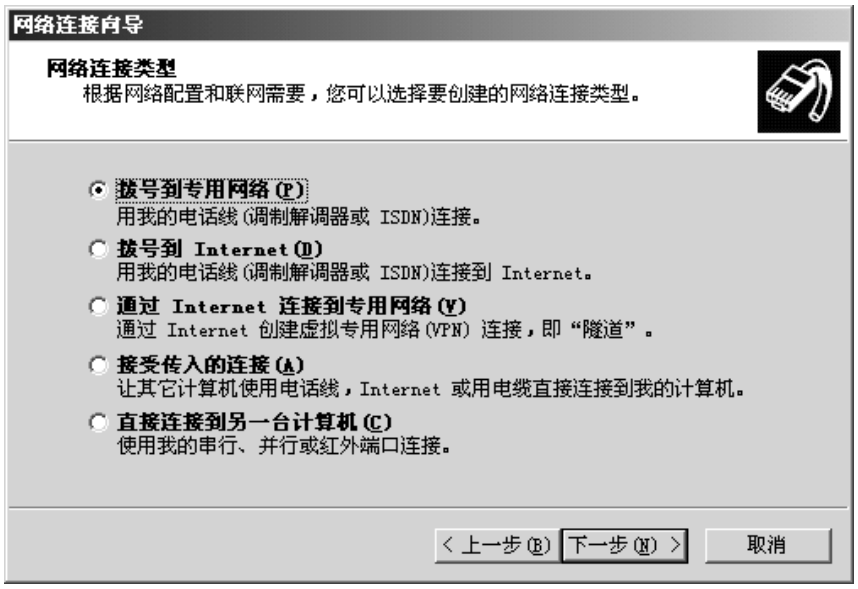


图5.15　选择“拨号到Internet（D）”

（3）在显示的电话号码输入栏中输入指定的网络服务商电话，如中国电信的 16300。然后单击“下一步”。在随后显示的“可用连接”窗口中可选择让所有的用户使用此连接，或只是自己使用，然后单击“下一步”按钮。

（4）在显示的“完成网络连接向导”窗口中，为该拨号连接取一个名字，还可以让系统在桌面上添加一快捷方式，便于以后使用。然后单击“完成”按钮，如图 5.16 所示。

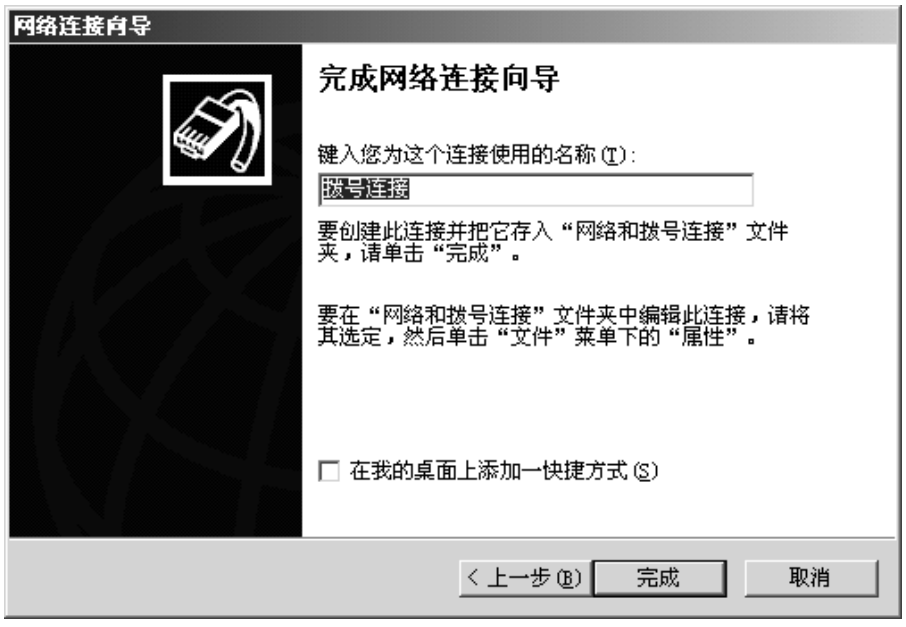


图5.16　拨号连接建立完成显示

（5）系统显示“连接拨号连接”窗口，如图 5.17 所示。输入申请的用户名和密码（注：在成都地区可以直接使用用户名为 163，密码为 163，拨号号码为 16300，进行网络连接。其他地区可向当地的网络服务提供商咨询）。如果需要立即进行网络连接可直接单击“拨号”按钮，否则单击“取消”按钮。



图5.17　“连接拨号连接”窗口

经过上述建立连接的各操作步骤后，将在“网络和拨号连接”窗口显示中增加一个所建立的拨号连接，如图 5.18 所示。



图5.18　增加了“拨号连接”的网络和拨号连接窗口

2．使用拨号网络连接

一旦在 Windows 中建立了拨号网络连接后，就可以通过它进行拨号，建立一条到某个

Internet 服务提供商的网络连接。用户使用这个连接可实现各种网络操作，如 Internet 浏览、文件共享、收发电子邮件等。

从“控制面板”或“附件”的“通讯”中选择“网络和拨号连接”，在窗口中双击已经建立的“拨号连接”图标，系统将显示如图 5.17 所示的“连接拨号连接”窗口。单击“拨号”按钮，系统开始连接拨号，在连接过程中将会从调制解调器中发出拨号声音和与服务器进行信息交互的声音，同时将显示用户登录系统的过程。连接完成后将弹出“连接完成”窗口，如图 5.19 所示。

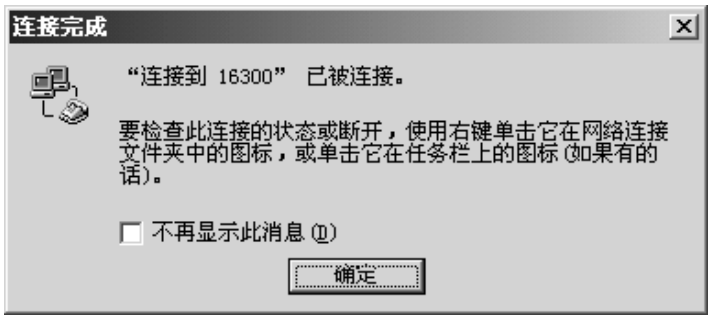


图5.19 “连接完成”窗口

（二）ISDN 接入方式

ISDN（Integrated Service Digital Network）的中文名称是综合业务数字网，俗称“一线通”。它除了可以用来打电话，还可以提供诸如可视电话、数据通信、会议电视等多种业务，从而将电话、传真、数据、图像等多种业务综合在一个统一的数字网络中进行传输和处理。这也就是“综合业务数字网”名字的来历。ISDN 和电话一样按时间收费，所以对于某些上网时间比较少的用户要比使用 ADSL 便宜很多。

根据不同连接方法，会需要不同的硬件。这里会涉及标准 ISDN 终端和非标准 ISDN 终端的概念，像数字电话机、四类传真机都属于标准 ISDN 终端，可以直接连接 ISDN 线路使用。而我们熟悉的电脑、普通电话、普通传真机都属于非标准 ISDN 终端，它们需要通过适配器与网络终端设备 NT1 连接后才能连上 ISDN 线路使用，而另一种称为 NT1+的设备则综合了适配器和 NT1 的功能，使得非标准 ISDN 终端可以方便地连接 ISDN。

各地的电信服务商基本上都开通了 ISDN 的服务。由于各地的情况不同，使得其申请手续和资费标准也有所不同。个人客户携带身份证，单位客户需在申请表上加盖公章，到电信局各营业厅均可办理 ISDN 业务。

在设备安装前需要做些必要的准备工作，通常电信局会把设备一起卖或者租给用户，建议选择外置连接设备，其安装比较方便，并且面板上有指示灯，可以从这些指示灯来判断设备的使用状态。下面介绍外置 ISDNTA 在 Windows 环境下的安装。

1．硬件连接方法

首先关闭电脑及 ISDNTA 电源，将 RS-232 电缆的针型连接器与 ISDNTA 后面板上的“RS-232”接口相连，上好紧固螺丝；电缆的另一头连接到电脑的 RS-232 串行口上。然后将电话线一头连接到 ISDN 入户线，另一头与 ISDNTA 后面板上的“U”接口相连，ISDNTA 后面板上的“S/T”接口可连接数字话机、四类传真机等数字设备。将电源的一端接至 ISDNTA

后面板标有 PWR 的插槽，另一端接上电源。接着开启 ISDNTA 电源，当机器在进行开机自我测试时，DATA 灯、128K 灯、ANA1 灯、ANA2 灯会全亮后全暗，然后四个灯来回循环亮，如果所有自检测试正确，便会全部暗，此时便可使用 ISDNTA。最后打开电脑电源，安装驱动程序。

2．软件设置

为了能通过 ISDNTA 上网，需要对 Windows 的“拨号网络”进行配置。其步骤如下：首先安装网络协议，然后在“控制面板”的“网络”中安装网络组件。请确认已经安装了“拨号网络适配器”和“TCP/IP”协议。接着配置拨号网络，双击桌面上“我的电脑”图标，打开“拨号网络”，双击“新建连接”，输入连接的名称，并选择调制解调器为相应的 ISDN 设备。最后单击“下一步”按钮，输入对方电话号码。进入下一个窗口，单击“下一步”按钮，最后单击“完成”按钮。至此，拨号网络就配置完毕，“拨号网络”中将增加一个新的连接，这时即可上网了。

（三）ADSL 接入方式

ADSL（Asymmetric Digital Subscriber Loop，非对称数字用户环路）能够在现有的双绞线，既普通电话线上根据当地线路状况提供 8Mbps 的下行速率和 1Mbps 的上行速率。这种下行速率远大于上行速率的非对称结构特别适合浏览 Internet，宽带视频点播等下行速率需求大于上行速率需求的应用。ADSL 充分利用了现有电话线路，不需要改造和重新建设网络，在电话线两端加装 ADSL 设备即可，降低了成本，减少了用户上网费用。ADSL 传输距离可达 3～5 公里。

ADSL 技术特点如下：具有很高的传输速率，最高下行速率 8Mbps、上行速率 1Mbps，为普通拨号 Modem 的百倍以上；上网和打电话互不干扰，ADSL 数据信号和电话音频信号以频分复用原理调制于各自频段互不干扰；独享带宽安全可靠，Cablemodem 下行速率虽然可达到 20Mbps，但由于是一种粗糙的总线型广播网络，节点下的几千上万用户共用 20Mbps 的带宽，其效果就可想而知了。更为严重的是由于总线型网络先天的广播特性，造成了信息传输的不安全性。ADSL 利用深入千家万户的电话网络，形成星型结构的网络拓扑构造，骨干网络采用遍布全城全国的光纤传输，各节点采用 ATM 宽带交换机处理交换信息，独享最高 8Mbps 的带宽。

ADSL 的申请方式和 ISDN 类似，个人用户带上电话机主的身份证复印件到当地电信营业厅申请即可。根据各个城市的不同，其资费方式有按在线时间计算以及包月甚至包年计费的方式。虽然电话线同时传递电话语音和数据，但数据并不通过电话交换机，因此不用通过电话拨号进行连接，它可以一直在线，属于专线上网方式。这意味着使用 ADSL 上网不需要缴纳拨号上网的电话费用。实际上，在现有电话线上安装 ADSL，只需在用户侧安装一台 ADSL Modem 和一只电话分离器（一块香皂大小），用户线路不用改动，极其方便。ADSL Modem 的硬件连接也相当地快捷方便。按照 ADSL Modem 背后的插口标示，插上电源，把 LAN 口与计算机网卡 LAN 口使用网线相连，最后插上电话线即可。软件设置也相当方便。

在 Windows XP 以上系统中，新建网络连接中已经有了 PPPOE 的网络接入选择项，在里面填入自己从 ISP 那里获得的账号、密码，然后，运行这个新建立的网络连接就可以顺利地连入 Internet 了。

小　结

本小节从“Internet 网络”这一概念着手，介绍了 Internet 网络的概况，并且对当前网络上运用最广泛的 TCP/IP 协议、IP 地址、常见的 Internet 服务等相关知识进行了讲述。

1．Internet 概述。这一部分主要介绍了 Internet 网络的产生、发展、演变的过程，让大家有一个全局的了解。

2．TCP/IP 介绍。这一部分通过对 TCP/IP 的介绍，让读者对网络层、传输层有一个具体的认识。主要内容包括：TCP/IP 体系结构、IP 协议、IP 协议的特点、互联网地址、传输层协议 TCP，以及 TCP 所采用的主要技术（滑动窗口、三次握手等）。

3. IP 地址及域名介绍。这一部分主要讲述了 Internet 网络中起支柱作用之一的 IP 地址。让大家了解到，IP 地址各个字段的意义以及在整个现行的 IPv4 结构下，几类 IP 地址的分类。简单介绍了 Internet 域名系统的原理。

4. Internet 服务。这一部分讲述了 Internet 上提供的几种常用服务。实际上，现在 Internet 上流行的其他很多服务，大部分都是基于这一节我们所讲述的几种基本服务的。

5．接入 Internet 方法。这一部分介绍了几种目前比较常见的 Internet 网络接入方式：电话线拨号、ISDN 以及 ADSL。

5.3　基于 TCP/IP 协议的网络编程

上一节重点介绍了 Internet 网络及应用，并从宏观上了解 Internet 网络的运作模式和提供的服务。从这一节开始，我们将重点介绍如何开发网络应用程序的问题。这是传输层以上的事，它调用系统提供的传输层服务来开发应用程序。当然不同的系统以及不同的网络这一边界也有所不同，因此，这一节主要介绍有关环境下常用的网络编程接口（特别是 Socket）以及开发网络应用很重要的一个模型——客户/服务器模型。

5.3.1　客户/服务器模型

我们知道，进程是一个动态的过程，一个进程对外界来说是透明的。当位于不同的主机上的进程相互进行通信时，无法知道另一主机上运行了哪些进程，各进程在干什么，因此，这为进程间通信带来了困难。解决这一问题需要通过两种技术来实现：客户/服务器模型和端口号。

客户/服务器模型为端-端通信的建立提供了一种有效的方法，它的基本思想就是服务器向外表明它的服务，以及与它通信所需的信息（地址及端口号），这样网上的其他进程就可以通过地址及端口号与它进行通信。服务器通常必须处于运行中，正如一个商场一般是开放的一样。另外，客户和服务器之间是面向连接还是非连接的，这在建立一个客户/服务器模型时也是很关键的。因此，客户/服务器模型就相当于为通信先建立一个代理服务器，进行通信时，只要与代理服务器进行交互即可。

端口号是用来代表通信的代理服务器，端口号相对比较固定。事实上它只是一个进程，一般在后台运行。

客户/服务器模型主要包括以下几点：

• 客户/服务器模型为在网络环境下进程通信提供了一个基本的方法，客户和服务器是对通信的进程的称呼，主动发起通信请求的进程称为客户，而等待通信请求并作出响应的进程称为服务器，因此服务器一般也称为侍者或守护神，它一般运行在某一主机，处于“Sleep”状态以监听客户对它的请求。

• 通信进程通过一个网络地址+本机端口号来唯一标识。一般来说，这正好对应于传输层服务访问点。一般端口号用一个整数来表示，比如，一个 16 位的整数。另外，端口号与硬件地址端口不是一回事，它是操作系统管理的一个逻辑概念。

• 服务器进程向外表明其服务，这是通过它的网络地址和本机端口号来申明的，这样，其他进程就能根据这一信息来与它通信。

• 客户进程在通信前通过动态申请一个本机端口号与主机网络地址构成一个传输层服务访问点来与服务器通信。

在客户/服务器模型中，从通信本身来说，客户和服务器是没有什么区别的，正如打电话时通话的双方是对等的。它们的区别是如何开始一个通信或会话过程即建立连接过程。比如，家里的电话可看成是一个服务器，别人可以通过你家的电话号码来与你联系，但与你通话的人使用的电话可以是任意的，可以是在一台公用电话上，也可能是通过其自家的电话，因为这时你不关心打电话给你的人的电话号码，因此，一般客户的端口号是可以任选的（理论上来说只要不冲突）。另外，客户与服务器也是相对的，如上面某人通过其自家的电话给你打电话，如相反，则客户与服务器的位置就交换了。

客户/服务器模型为对等通信建立了一个理想的模型。从理论上来说，通过这一模型建立通信是完全可以的，但是，这里还存在一些问题。

1．如上所述，网络地址和端口号等都是数字形式，如 IP 地址是一个 32 位的数值，端口号是一个 16 位的整数，因此，要人们去记住一大堆数字很困难。

2．服务形式很多，哪一个端口号提供什么服务，这种对应关系是很多的，如用一对应的函数来说明，这种函数个数是一个很大的组合数。

因此，为了解决这些问题，提出了一些规范和标准。

第一，预留标准化服务类型，这与电话系统很类似，比如，一般用户电话不能用 0 和 1 开头，而将其留作特殊用途（如火警、急救电话等）。在网络中，也预留了一些端口号及一些标准化服务类型端口，如文件传输、远程登录。而预留了一些端口号是为了将来标准化服务类型扩展等用途。

第二，采用了名字服务。这也类似电话号码管理的方法，如通过名字或单位对电话号码进行查询。在这里，名字服务本身也是通过网络来实现的，因此，它也与电话号码查询一样，需要占用一个端口。

通过这两种方式，客户/服务器模型解决了建立通信的问题，它已成为当前网络通信的一个既成事实的标准，如现在 Internet 上提供的所有服务都是通过这一模型实现的。

最后，将涉及如何在网络环境中创建服务器应用程序和客户应用程序，由于它们在通信的建立过程中所起的作用不同，因此，它们的实现也有所差别。在下一小节中将详细介绍这些内容。

5.3.2 Socket 及其调用的一般流程

Socket 是现在 TCP/IP 网使用最广的系统边界，它最初是由 Berkeley 分校在 Unix 系统上设计的，因此，这里称其为 BSD Socket，这一部分将着重对 Socket 的概念以及在 Unix 下有关这一接口的信息（一组系统调用）进行介绍。

一、什么是Socket

Socket 编程界面由 4BSD Unix 首先提出，目的是解决互联网进程通信问题。Socket 的英文原义是“孔”或“插座”，在这里作为 4BSD Unix 的进程通信机制，取其后一种意义。日常生活中常见的插座，有的是信号插座，有的是电源插座，有的可以接收信号（或能量），有的可以发送信号（或能量）。假如在电话线与电话机之间安放一个插座（相当于二者之间的接口，这一部分装置物理上是存在的，不过往往不是以具体的插座形式表现出来，而是两根简单的接线而已），则 Socket 非常类似于电话插座，而这一“电话插座”的使用者是一个进程。

将电话系统与面向连接的 Socket 机制相比，有惊人相似的地方。以一个国家级电话网为例，电话的通话双方相当于相互通信的两个进程，通话双方所在的地区（享有一个全局唯一的区号）相当于一个网络，区号是它的网络地址。区内一个单位的交换机相当于一台主机，主机分配给每个用户的局内号码相当于 Socket 号。

任何用户在通话之前，首先要占有一部电话机，相当于申请一个 Socket 号，同时要知道对方的号码，相当于对方有一个固定的 Socket。然后向对方拨号呼叫，相当于发出连接请求（假如对方不在同一区内，还要拨对方区号，相当于给出网络地址）。对方假如在场并空闲（相当于通信的另一主机开机且可以接受连接请求），拿起电话话筒，双方就可以正式通话，相当于连接成功。双方通话的过程，是向电话机发出信号和从电话机接收信号的过程，相当于向 Socket 发送数据和从 Socket 接收数据。通话结束后，一方挂上电话机，相当于关闭 Socket，撤销连接。

在电话系统中，一般用户只能感觉到本地电话机和对方电话号码的存在，建立通话的过程、话音传输的过程以及整个电话系统的技术细节对他都是透明的，这也与 Socket 机制非常相似。Socket 利用互联网通信设施实现进程通信，但它对通信设施的细节毫不关心，通信设施能提供足够的通信能力，它就满足了。

至此，我们对 Socket 进行了直观的描述。抽象出来，Socket 实质上提供了进程通信的端点。进程通信之前，双方首先必须各自创建一个端点，否则是没有办法建立联系并相互通信的。正如打电话之前，双方必须各自拥有一台电话机一样。

在互联网内部，每一个 Socket 用一个半相关描述：

〈协议，本地地址，本地端口〉

一个完整的 Socket 连接则用一个相关描述，它标识了两个 Socket 的一个连接，相当于一对通信的电话对，其描述如下：

〈协议，本地地址，本地端口，远地地址，远地端口〉

每一个 Socket 有一个本地唯一的 Socket 号（端口号 port），这是由本地的操作系统分配的。

最重要的是 Socket 是面向客户/服务器模型而设计的，针对客户和服务器程序提供不同的 Socket 系统调用。客户随机申请一个 Socket（相当于一个想打电话的人可以在任何一台入网电话上拨号呼叫），系统为之分配一个 Socket 号。服务器拥有全局公认的 Socket，任何客户都可以向它发出连接请求和信息请求（相当于一个被呼叫的电话拥有一个呼叫方知道的电话号码）。

Socket 利用客户/服务器模式巧妙地解决了进程之间建立通信连接的问题。服务器 Socket 半相关为全局所公认非常重要。读者不妨考虑一下，两个完全随机的用户进程之间如何建立通信？假如通信双方没有任何一方的 Socket 固定，就好比打电话的双方彼此不知道对方的电话号码，要通话是不可能的。

二、Unix的I/O系统

Unix 系统可以说是最精简、层次最清晰的一个操作系统。最具影响的是它的文件系统、流机制等高级 I/O 系统功能（核心上部，主要是一组系统调用），它对用户隐蔽了许多硬件设备的特征，为上层提供了一致的接口。

首先，从用户的角度来看，I/O 系统对用户是一组系统调用以及一些逻辑设备名，设备名包括主设备号和子设备号两部分，主设备号标识了某种类型的设备，子设备号用来标识某种设备和某个具体设备。

其次，对于程序员级的用户来说，还可看到另一个很重要的东西——文件描述符，对于一个打开的文件、设备以及一个网络连接（一个 Socket 句柄），都是通过一个描述符来控制的。从数据结构上来看，描述符本身是一个 16 位的整数，它用来索引一个控制和描述某个逻辑设备的控制块。对用户来说，这一控制块是不用关心的。

另外，在核心下部，还有三个重要部分：快速缓存系统、通用设备驱动程序代码以及访问特定设备的驱动程序。

在 Unix 系统中，有文件系统、字符设备接口、块设备接口以及网络连接接口四种基本的 I/O。因为在 Unix 系统中的设备是在文件系统的名字空间中，通过一个文件名指向一个描述设备的文件 iNode。因此，Unix 系统对前三种 I/O 系统调用接口是一致的，在用户眼中读写设备与读写文件没有什么差别。这一组系统调用如 Open( )、Read( ) 以及 Write( )、Send( )、Close( ) 等，其基本使用过程如下：

第一步，打开设备（或文件），这是由 Open( ) 完成的，若成功则返回一个打开设备的描述符。

第二步，通过返回的描述符来读写设备，如 Read( )、Write( ) 系统调用。

第三步，关闭设备。设备使用结束，要调用 Close( ) 系统调用来关闭这一设备。这是很重要的，因为这里的设备是逻辑上的，关闭这一设备可以让系统释放有关管理和控制这一逻辑设备的内存空间。

另外，对于第四种 I/O——网络连接接口，它采用了另一套系统调用，这在不同版本的 Unix 中也有所不同，如在以前的系统 V 版本中，它采用了系统 V 传输层接口。不过，现在很流行的一种是 Socket，这将在下面介绍。

三、BSD Socket系统调用

下面我们来看一下 Unix 下是如何通过 Socket 开发网络应用的。

我们先不管 Socket 的内部实现机制，只讨论 Socket 的使用方法。在 Unix 中 Socket 是通过向开发网络应用的程序员提供了一组系统调用实现的，并且采用了与文件 I/O 相似的方式（描述符）。下面我们将看到，Socket 系统调用与 Unix 的文件访问有许多相似之处，事实上二者确实有某种联系。文件访问与 Socket 操作的相似之处在于对应用程序来说都是一些输入/输出。不同的是文件访问是本地输入/输出，文件号对应于一个具体的块文件或字符文件。而 Socket 访问是网络输入/输出，Socket 号除了对应于某个网络设备外，还跟一个信宿地址（对方 Socket 半相关）联系在一起。从这个意义上来说，Socket 是对 Unix 输入/输出的扩充。

如上所述，在 Unix 中 Socket 与文件 I/O 类似，它们都采用了描述符的概念，只是提供的系统调用不同。对于一个文件操作，一般过程是打开文件，读写文件，最后关闭文件。对于 Socket 接口，也借用了这种风格，尤其是读写时，它使用完全与文件 I/O 相同的系统调用（read/write）。下面具体介绍它的使用过程。

第一步，创建 Socket。

应用程序在使用 Socket 之前，必须先创建一个 Socket，它的功能是返回一个 Socket 描述符，这与文件操作是类似的。不过这里所使用的系统调用不同，文件是 Open( ) 调用，而这里是 socket( ) 调用。下面是 socket( ) 系统调用的基本格式：

int sockId=socket（af，type，protocol）；

说明：

1．sockId 是一个正数，它用来接收新建的 socket 的描述符。

2．af：地址簇，它用来指明本 Socket 所使用的地址类型，当前 Unix 主要支持以下的地址簇：

AF —UNIX	UNIX 内部地址
AF —INET	TCP/IP 地址
AF —NS	Xerox NS 地址
AF —IMPLINK	ARPANET IMP 地址
AF —APPLETALK	Apple 公司 AppleTalk 地址

3．type：用来指明新建 Socket 将采用的网络通信服务类型，因此，这与所采用的地址簇（因为这标识了所采用的网络协议簇）有关。Socket 主要支持以下几种通信服务类型：

sock —STREAM	流 Socket（对于 TCP/IP 网，采用 TCP 通信协议）
sock —DGRAM	数据报 Socket（对于 TCP/IP 网，采用 UDP 通信协议）
sock —RAW	原始 Socket
sock —SEQPACKET	有序分组 Socket
sock —RDM	可靠发送消息

4．protocol

第二步，指出本 Socket 所使用的协议。

第三步，绑定。

新建一个 Socket 后，必须把它绑定到一个传输层地址，这是通过系统调用 Bind( ) 完成的，其原型如下：

int Bind（sockId，localAddr，int addreLen）

说明：

1．sockId 就是新建的 socket 描述符号。

2．localAddr 是一个结构，对于不同的地址簇，这一结构也不同，主要有下面三种类型的地址结构：

struct sockaddr —in TCP/IP 网

struct sockaddr —ns Xerox NS 网

struct sockaddr —un UNIX Socket 地址结构

3.指明地址参数的字节长度，比如 sizeof（sockaddr —in）。

第四步，建立 Socket 连接。

为建立一个 Socket 连接，涉及两个系统调用，它们分别在客户和服务器中，一个是 connect( )，它让客户发起一个 Socket 连接；另一个是 accept( )，它让服务器接收这一连接请求。另外，还有一个监听系统调用，用来让服务器监听对它的连接请求，这一系统调用是 listen( )。

第五步，数据的收发。

与此相关的系统调用有：

用于发送数据：write( )、writev( )和 send( )。

用于接收数据：read( )、readv( )、recv( )和 recvmsg( )。

第六步，关闭。

Socket 的关闭采用了文件操作的系统调用 close( )。

另外，对于网络应用，还有一些辅助函数，比如，用来转换字节序的函数、地址转换等。

1．字节序转换例程（h 代表 host，即主机；n 代表 net，即网络）

u —long htonl（u —long hostlong）；

u —short htons（u —short hostshort）；

u —long ntohl（u —long netlong）；

u —short ntohl（u —short netshort）；

2．地址转换例程

对于 Internet，其网络地址是一个 32 位的值，但为了方便记忆，它采用了点分十进制来表示，比如，202.115.15.102（另外，与点分十进制地址等同的还有域名，域名仅是一种协议，它把一个点分十进制地址或加上端口号对应到一个域名地址），因此，对于这种情况，必须有一个地址转换，在 Socket 接口中，有下面两个例程：

unsigned long inet —addr（char* ptr）；

char* inet —ntoa（struct in —addr inaddr）；

第一个例程是将一个点分十进制地址转换为一个 32 位的 Internet 地址，第二个例程功能正好相反。

对于其他非 TCP/IP 网络，如 XNS，也有相似的地址转换函数。这里很关键的是对网络

地址的理解。

四、客户/服务器操作的基本过程

由于客户和服务器属性存在一定的差别（主要表现在建立连接上），因此，实现过程也有所不同，如上面所说，在 Socket 中建立连接时，分别为客户/服务器设置了相应的系统调用。

对于这一差别，关键是对客户/服务器模型的理解。

1．在该模型中，等待对它的通信请求的一方即称为服务器，另一个发起对通信的请求，称为客户。这就决定了通信建立过程在两端是不同的。

2．在通信过程中，数据的交换等是完全对等的。

3．服务器要监听对它的请求，这里就有一个问题，假使服务器刚接收了一个通信请求，并建立了通信连接，那么，若这时有另一个客户发起了对它的一个通信请求，服务器该怎么办，正如，电话的另一端正在使用，打不进去一样。当然，对于只能提供单请求的服务器，也未尝不可，正如电话系统一样，但要使服务器面向大众服务（往往是这样），这时就需要一个处理多重通信的服务器，一般称为并发服务器。

4．网络通信所采用的协议，特别是要区分面向连接的还是非面向连接的通信。

下面介绍服务器操作的基本过程。

1．面向连接的服务器

其基本过程如下所示：

第一步，新建一个 Socket，调用 Socket( )；

第二步，绑定 Socket，调用 Bind( )；

第三步，端口监听，若有并接收请求（下面是这一过程）。

```
if（listen（sockId，5）<0）
error（"listen error!"）；
for（；；）
{
    newSockID=accept（sockId，..）；
    if（newSockID<0）
  error（"accept error!"）；
    if（fork（）==0）{/*子进程
  /*子进程关闭继承的描述符*/
  close（sockId）；
  Do（newSockID）；
  close（newSockID）；
  continue；
    }else{/*parent process*/
  continue；/*continue to listen*/；
    }
}
```

2．面向非连接的服务器（UDP）

第一步，新建一个 Socket，调用 Socket( )；

第二步，绑定 Socket，调用 Bind( )；

第三步，因为没有连接过程，因此，它可直接接收，可描述为两者的发送和接收的交互过程，主要是通过系统调用 recvfrom( ) 和 sendto( ) 进行的。

5.3.3 Windows 环境下的网络编程应用

一、Windows Socket

• 什么是 Windows Socket

Windows Socket 为 Microsoft Windows 系列平台定义了一个二进制兼容的编程接口，它是基于 Unix 的套接字（Socket）而建立的，其定义包括 BSD 格式的套接字例程，及有关专门对于 Windows 的扩展。使用 Windows Socket 可以使你的应用实现网络的通信能力，并且，只要遵循 Windows Socket 的定义，应用可以在不同的平台进行通信。在 Windows 平台上，Windows Sockets 是线程安全的。

许多网络开发商在网络协议中都支持 Windows Socket，包括 TCP/IP、XNS，以及 Novell 公司的 IPX/SPX 等，虽然，Windows Socket 现在只定义了对 TCP/IP 协议的抽象说明，但任何协议簇都可以遵循 Windows Socket，只要它提供了其有关 Windows Socket 实现的动态连接库。

另外，Windows Socket 的目的是实现对网络细节的屏蔽，你不必知道有关网络的具体实现，对于具体的协议不用关心，就像电话系统提供的服务一样，你只要知道电话号码及要找的人就可以了。

在 Windows Socket 中，一个 Socket 对象就是通信的一个终点，通过它可以在网络上与别的机器上的应用进行通信，也就是说，通过这一个对象，你可以在网络上发送或接收数据包。这与 BSD 的套接字是一样的。

• Socket 的分类

Windows Socket 定义了两种类型的套接字（Socket），一种是流套接字（Stream Sockets）；另一种是数据块套接字（Datagram Sockets）。

流套接字传输的信息是没有记录边界的，它是一个字节流，这种方式必须保证以发送的顺序来接收信息，它适用于大量的数据传输应用中。这里需要注意的是，虽然在应用层看来是一个流，即一个整体，但网络的传输层会分成一个个包的形式进行发送。另外，这种类型的套接字必须是有明确的建立连接过程，即通信的两端必须是面向连接的。首先是套接字 A 向套接字 B 发送一个连接请求，套接字 B 可以接受或拒绝，若接受，便在两个套接字之间建立了一个连接。这与电话系统很相似。

数据块套接字传输的信息是面向记录的，它不需要保证接收的顺序，即第一个数据记录可以比第二个数据记录后到达，并且，它也不用保证发送的每个记录都必须到达。数据块套接字是面向无连接，它不需要明确的连接建立，有一个指定给某个数据块套接字的消息被发送给它，因此，你可以从这个数据块套接字接收这一消息并进行必要的处理。这与电报系统很相似。

• 端口和套接字（Socket）地址

在上面已说过，套接字是由计算机的网络地址和端口号来准确定义的。那么，端口号和网络地址是怎样来确定的呢？

一个端口是用来标识一个唯一的进程，这一进程就是一个服务的提供者。在 Windows Socket 中，一个端口是与一个应用相关联的，通过不同的端口号来支持多个不同的 Windows Socket 应用，另外，某些端口号是为一些公共的服务保留，比如 FTP、E-Mail 等服务都有其保留端口号。对于端口号的选择，你可以自己在 1024～5000 之间选择一个，但这也不保险，因为可能有别的应用已选取了这一端口号，因此创建套接字会失败，你也可以传一个 0，让系统为你分配一个端口号。

至于套接字地址，在 Windows Socket 中，每一个套接字对象都与一个 IP 地址相关联，可以是机器名（域名），比如，ftp.Elc.EDU，也可以是直接的 IP 地址（在连接中更快），比如，202.115.14.108。在一般情况下，在创建套接字时，你不需要指定自己的 IP 地址，但也有例外，如果你的机器有多个网卡，每一个都代表了不同的网络，这时，需要指定 IP 地址来确定使用哪一个网卡。

二、MFC对Windows Socket的支持

MFC 支持两种模型的 Windows Socket 网络通信，这些都包含在 MFC 的两个类中：

一个是 CAsyncSocket，这个类描述了基本的 Windows Socket 编程接口，主要为那些对网络编程比较熟悉及需要编程具有更大的灵活性的程序员而设计的。

另一个是 CSocket，它是从 CAsyncSocket 派生而来，是在一个更高层次上对套接字的抽象，使用 MFC 类 CArchive，相当于 MFC 对文件进行系列化，因此，在使用上，它比 CAsyncSocket 更简单一些，因为 Socket 也从 CAsyncSocket 继承了很多的成员函数用来对 Windows Socket 的支持，你必须利用它们中的一些函数来实现网络编程功能。

CAsyncSocket 是在一个很低的层次上的 API，它主要是为那些需要利用低层的一些基本功能而设计的，使用 CAsyncSocket 一般包括以下几个步骤：

第一步，创建一个套接字，首先是构造一个 CAsyncSocket 对象，然后再调用其 Create( ) 成员函数。

例 1

```
CAsyncSocket  sock;
sock.Create（）;
//利用缺省参数，这时系统为你选择一个端口号，Socket 类型是流套接字。
```

例 2

```
CAsyncSocket*  pSock=new CAsyncSocket ;
int nPort=27;
pSock->Create（nPort，SOCK  —DGRAM）;
```

第二步，建立连接，连接也分为客户和服务器两种情况，因为客户是连接的发起者，服务器是连接的接受者。对于客户端，调用成员函数 Connect( )；而服务器需要两步，先是监听是否有对它的访问，这是通过调用成员函数 Listen( ) 来实现的，若发现有对它的访问，且不冲突，调用成员函数 Accept( ) 接受这一连接请求，它为这一请求另建一个套接字来处

理该连接上的具体通信，而它自己继续去监听网络连接请求。在实际应用中，一般是通过创建一个子进程来完成通信任务的，当然也可以创建一个新线程来实现。

第三步，在建立好连接以后，调用有关的通信成员函数进行实际的通信。

最后，终断连接，析构套接字对象。下面看一个较完整的例子：

例 3

```
//客户端应用程序
        //      LPSTR   lpszMachineName;
        //      SOCKADDR*   lpIPAddress;
        //      int     nHostPort; 为服务器端的端口号;
        CAsyncSocket* pSock=new CAsyncSocket;
         //int nPort=27;
        pSock->Create ();
        int Length=strlen (*lpIPAddress);
        if (lpIPAddress!=NULL)
        {
        if (pSock->connect (lpIPAddress, Length))
             return CONNECT   —ERROR;
        }
        else if (lpszMachineName!=NULL) {
        if (pSock->connect (lpszMachineName, nHostPort)) {
             return CONNECT   —ERROR;
        }
        }
        //      LPSTR lpBuf=new char [120];
        //      int nSize=120;
        //有关发送内容处理
        int nPort;
        CString lpszLocalName;
        pSock->GetSockName (lpszLocalName, nPort);
        pSock->SendTo (lpBuf, nSize, nPort);
        pSock->Close ();
        delete pSock;
```

例 4　服务器一端

```
CAsyncSocket* pSock=new CAsyncSocket;
        int nPort=27;
        pSock->Create (nPort, SOCK   —STREAM);
        while (TRUE) {
        while (pSock->Listen ()) {}
        CAsyncSocket NewSock=new CAsyncSocket;
```

```
int nPort=0;
pSock->Create（0，SOCK　—STREAM）；//创建一个 Socket
if（pSock->Accept（NewSock））　//继续去监听
　　AfxMessage（“Accept Error!”）；
continue；
}
delete pSock；
```

三、其他类

Socket 只是在网络通信中建立一个连接的作用，但连接后，信息的发送与接收还必须有其他类的支持，有关这样的类在 VC 中主要有 CSocketFile、CArchive。

对于网络信息的发送和接收，VC 把连接的另一端作为一个设备或文件方式来处理，一个 Windows Socket 只是代表了这一设备的信息，通过它，就可以建立一个类似于文件句柄的结构，下面看看上述两个类的基本用法：

1．CSocketFile

CSocketFile 的用法很简单，只需构造一个 CSocketFile 对象，在构造函数中传递一个 CSocket 的对象指针，如下所示：

```
CSocketFile *　pFile=new CSocketFile（pSock）；
```

当有了这样的一个文件对象后，你就可以对它进行读写。在 VC 中，与一般的文件一样，提供了一个归档功能的类 CArChive，用它可方便地对文件进行读写。

2．CArChive

CArChive 类在序列化部分已经介绍过，在网络环境中，它的使用是一样的，它的构造数如下：

```
CArChive（CFile* pFile，UINT nFlags）；
```

只是在网络环境中，pFile 是一个网络的“文件”，比如，在上面，它是一个 CSocketFile 的对象指针。例如：

```
CSocketFile *　pFile=new CSocketFile（pSock）；
pArIn=CArChive（pFile，CArChive::load）；
```

有关网络编程接口本节仅做了简单介绍，对于网络编程，关键的是要掌握客户/服务器模型，而对于不同系统所提供的接口，虽然有所不同，但只要能熟悉一种，其他都大同小异。

小　　结

本节主要介绍了基于 TCP/IP 协议的网络编程。作为其基础是客户/服务器（C/S）模式和 Socket。主要内容包括：

1．客户/服务器模型

2．Socket 及其调用的一般流程

3．Windows 环境下的网络编程应用

5.4 HTML 语言与网页制作

5.4.1 HTTP 和 Web 服务器

一、HTTP协议

自 WWW 诞生以来，一个多姿多彩的资讯和虚拟的世界便出现在我们眼前。当决定使用超文本作为 WWW 文档的标准格式后，在 1990 年，科学家们立即制定了能够快速查找这些超文本文档的协议，即 HTTP 协议。超文本传输协议是互联网上应用最为广泛的一种网络传输协议。所有的 WWW 文件都必须遵守这个标准。设计 HTTP 最初的目的是为了提供一种发布和接收 HTML 页面的方法。目前它的应用除了 HTML 网页外还被用来传输超文本数据，基本上只要是文件数据均可以利用 HTTP 进行传输。

Web 的应用层协议 HTTP 是 Web 的核心，HTTP 在 Web 的客户程序和服务器程序中得以实现。运行在不同端系统上的客户程序和服务器程序通过交换 HTTP 消息彼此交流。HTTP 定义这些消息的结构以及客户和服务器如何交换这些消息。由于 HTTP 协议是基于请求/响应范式的（相当于客户/服务器）。一个客户与服务器建立连接后，发送一个请求给服务器，请求方式的格式为：统一资源标识符（URL）、协议版本号，后边是 MIME 信息包括请求修饰符、客户信息和可能的内容。服务器接到请求后，给予相应的响应信息，其格式为一个状态行，包括信息的协议版本号、一个成功或错误的代码，后边是 MIME 信息包括服务器信息、实体信息和可能的内容。

二、Web服务器

Web 服务器可以解析 HTTP 协议，当 Web 服务器接收到一个 HTTP 请求，会返回一个 HTTP 响应，例如，送回一张图片。为了处理一个请求，Web 服务器可以响应一个静态页面或图片，进行页面跳转，或者把动态响应的产生委托给一些其他的程序，例如 CGI 脚本、JSP 脚本、ASP 脚本、服务器端 JavaScript，或者一些其他的服务器端技术。无论这些脚本的作用如何，服务器端程序通常产生一个 HTML 的响应来让浏览器可以浏览。例如 http://www.uestc.edu.cn/就是一个典型的 Web 服务器。

当前，网络上的 Web 服务器大多架设在 Windows 和 Unix 环境下。在这两个阵营中，都有很杰出的 Web 服务器软件，比如 Windows 下的 IIS 和 Unix/Linux 下的 Apache（实际上，Apache 也有能在 Windows 下运行的版本）。下面，我们以 Windows 2003 环境和 Linux 环境为例，分别介绍 IIS 和 Apache 这两种 Web 服务器的架设和设置办法。

1．Windows 2003 下 IIS 的架设

Windows Server 2003 中 Internet 信息服务（IIS）被升级为 IIS 6.0，为我们带来了更高的安全性。在默认情况下，Windows Server 2003 没有安装 IIS 6.0，我们可以通过控制面板来安装。具体做法如下：进入“控制面板”→ 双击“添加或删除程序”→ 单击“添加/删

除 Windows 组件”→ 在“组件”列表框中，双击“应用程序服务器”→ 双击“Internet 信息服务（IIS）”→ 从中选择“万维网服务”及“文件传输协议（FTP）服务”→ 双击“万维网服务”，从中选择“Active Server Pages”及“万维网服务”等。之后，可能会要求插入 Windows 安装光盘。

安装好 IIS 后，接着设置 Web 服务器，具体做法为：在“开始”菜单中选择“管理工具→ Internet 信息服务（IIS）管理器”→ 在“Internet 信息服务（IIS）管理器”中双击“本地计算机”→ 右击“网站”，在弹出菜单中选择“新建→ 网站”，打开“网站创建向导”→ 依次填写“网站描述”、“IP 地址”、“端口号”、“路径”和“网站访问权限”等。最后，为了便于访问还应设置默认文档（Index.asp、Index.htm）。

上述设置和 Windows 2000 Server 网站设置基本相同，但此时 Web 服务还仅适用于静态内容，即静态页面能正常浏览，常用 Active Server Pages（ASP）功能没有被启用。所以还应在“Internet 信息服务（IIS）管理器”的“Web 服务扩展”中选择允许“Active Server Pages”。

2．Linux 下 Apache 的架设

在 Linux 下，Apache 软件包有两种：一种是源代码，下载后需要自己重新编译才能使用；另一种是可执行文件，下载后只需解压就可以使用。我们可以通过以下三种方法安装 Apache 服务器。

如果安装的 Linux 版本中带有 Apache，就在选择所要安装的服务器时，将 httpd 这个服务选上，Linux 安装程序将自动完成 Apache 的安装工作，并做好基本的配置。如果我们的 Linux 系统中本身不带有 Apache，对编译工作不是太熟悉的初级用户，我们使用可执行文件软件包。下载软件包 apache_1.2.4.e.tar.gz，然后执行命令：tar xvzf apache_1.2.4.e.tar.gz 就可完成安装工作。如果想把 Apache 服务器充分利用起来，就一定要自己编译 Apache 定制其功能。下载包含 Apache 源代码的软件包 apache_1.2.4.tar.gz；然后用 tar 命令将它解开，将当前目录改变为 Apache 源代码发行版的 src 目录，将配置样本文件 Configuration.tmpl 复制为 Configuration 文件，编辑 Configuration 文件中的配置选项。做好这些工作之后，我们通过使用模块配置，可以自定义在 Apache 服务器中需要什么功能，这个部分也是 Apache 灵活性的表现。如果你需要 Apache 服务器具备什么功能，就将那个模块用 AddModule 语句加到配置文件 Configuration 中去。在 src 目录下执行：“ /configure”；编译 Apache：执行命令“make”；将编译好的可执行文件 httpd 复制到/etc/httpd/bin 目录下；将 Apache 发行版的配置文件：access.conf、httpd.conf、mime.types、srm.conf 复制到/etc/httpd/conf 目录下。到此为止，Apache 的安装和设置工作得以完成。

5.4.2 HTML 和 XML

一、什么是HTML

HTML 是 HyperText Markup Language（超文本标记语言）的缩写，它是构成 Web 页面（Page）的主要工具，是用来表示网上信息的符号标记语言。它与 BASIC、FORTRAN、C 等程序设计语言不一样，严格地说它不能称为是一种程序设计语言。HTML 是用来在文本文档中加上一些标注，指出所标注的文本应该使用什么格式（或类型）。就像我们广泛使用

的文字处理程序 WPS 那样，当需要让某些文字按照黑体字显示时，就在它的前面和后面加上一定的标记符号。学习这种语言就是要知道它提供了哪些标记功能，弄清每种标记功能的使用方法。当然这些只是 HTML 的基本功能，HTML 还在不断发展，新的功能在不断增加，并且还融入了一些编程语言具有的功能，包括图形处理、交互式处理等。

HTML 编写的文件（文档）的扩展名是.html 或.htm，它们是可供浏览器解释浏览的文件格式。可以使用记事本、写字板或 FrontPage Editor 等编辑工具来编写 Html 文件。Html 语言使用标志对的方法编写文件，既简单又方便。

对于一个普通文本文档应该包含有一些基本的要素，如标题、段落、项目列表等。另外对于不同的内容可以使用不同的文字来显示，如使用黑体字、斜体字、不同尺寸的字体等。

在 HTML 文档中，一般包含有两种类型的数据。

• 文档的文本内容；

• HTML 的标记符号，简称标记符。标记符不仅用于表示文档的组成元素、结构、整形方法，还可以表示指向其他文档或其他文档中包含的媒体信息（如声音、图像、动画和视频等）的超文本链接。

HTML 标记符的一般使用格式为：

<标记符>标记对象文本</标记符>

前面的<标记符>表示标记开始，后面的</标记符>表示标记结束。一般标记符都是成对出现，一个标记符对也叫做一个容器。

例如：<TITLE>EXAMPLE</TITLE>

它表示将“EXAMPLE”作为一个标题显示。

开始标记符表示打开某种格式功能（如标题、黑体字、斜体字等），结束标记符表示将此功能关闭。结束标记符是在开始标记符的前面加上一个斜杠“/”。在 HTML 中，并不是所有的标记符都是成对出现的，有的标记符就没有结束标记符。另外在尖括号“< >”中，除了有标记符外，有时还要加入一些附加的信息。在 HTML 的标记符表示中不区分字母的大小写。

下面看一个简单的完整的实例。

首先必须选择一个文本编辑器，如 WORD、WPS 以及 Windows 2000 的记事本和写字板等。一般来说，任何一种文本编辑器都可以，但要求必须将编辑好的文件按文本格式保存。

下面我们使用 Windows 2000 的写字板将下面的内容输入并存为一个文本文件，取名为 EXAMPLE.HTM，文件的扩展名一般用.HTM，在 Windows 2000 系统下由于可以使用三个字以上的扩展名，因此可以使用 HTML 作为扩展名。虽然文件的内容是文本格式，与使用.TXT 扩展名存放是一样的。但是多数浏览器在读入文件时，要查看其扩展名，以区分是 HTML 文件还是纯文本文件。

```
<HTML>
<HEAD>
<TITLE>This is a simple HTML document.</TITLE>
```

```
</HEAD>
<BODY>
<H1>Example</H1>
</BODY>
</HTML>
```

然后启动一个浏览器，如 Microsoft Internet Explorer，使用文件菜单中的打开命令将上述文件调入浏览器中。如图 5.20 所示是在 Microsoft Internet Explorer 浏览器上显示的结果。

二、XML语言概述

XML 为 Extensible Markup Language 的缩写，即可扩充标注语言。它是由 SGML（Standard for General Markup Language，通用标记语言标准）所精简而来的一种通用标注语言，主要是简化了 SGML 繁杂的结构，强化 HTML 过于简单而不够严谨的语法。微软是 XML 技术的推动者之一，它希望能够建立一个可以为 WWW 广泛使用的语言环境，推动程序的兼容与协同，从而降低成本，刺激增长。

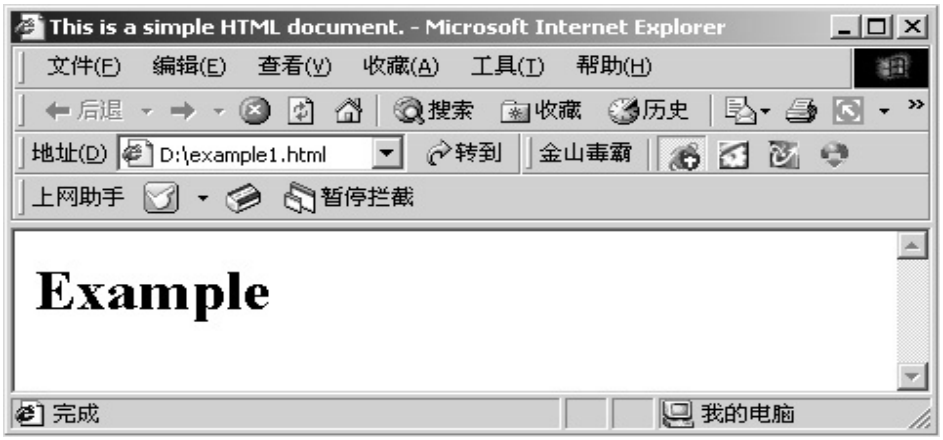


图5.20　浏览器上显示的一个简单的HTML文档

虽然 XML 创立之初只是被当成一项基础技术，但其发展早已超出设计者原先的构想。不论是学术界还是商业界都将其视为下一代网络的基石。XML 现在已经成为一股不可抵挡的技术潮流。

现有的 XML 主要应用在四个方面：一是应用于具有不同复杂格式的不同数据源间的交互；二是应用于大量运算负荷分布在客户端的情况，用户可以根据自己的需求选择和制作不同的应用程序以处理数据，而服务器只需发出同一个 XML 文件；三是应用于将同一数据以不同的形式表现出来；四是应用于网络代理对所取得的信息进行编辑、增减以适应个人用户的需要，形成具有个人特色的数据文件。

5.4.3　网页设计与制作

上面第一节里面我们介绍了HTTP协议和相关软件运行平台，第二节我们介绍了Html和

设计和
制作。

Dreamweaver是美国MACROMEDIA公司开发的集网页制作和管理网站于一身的所见即所得网页编辑器，它是一套针对专业网页设计师而开发的视觉化网页开发工具，利用它可以轻而易举地制作出跨越平台限制和跨越浏览器限制的充满动感的网页，如图 5.21 所示。

随着互联网（Internet）的家喻户晓，HTML技术的不断发展和完善，随之而产生了众多网页编辑器，按网页编辑器基本性质可以分为所见即所得网页编辑器和非所见即所得网页编辑器（即原始代码编辑器），两者各有千秋。所见即所得网页编辑器的优点就是直观性好，使用方便，容易上手，在所见即所得网页编辑器进行网页制作和在WORD中进行文本编辑不会感到有什么区别，但它同时也存在着致命的弱点：



图5.21　Dreamweaver启动界面

• 难以精确达到与浏览器完全一致的显示效果。也就是说将在所见即所得网页编辑器中制作的网页放到浏览器中，很难完全达到真正想要的效果，这一点在结构复杂一些的网页（如分帧结构、动态网页结构）中便可以体现出来；

• 页面原始代码的难以控制性，比如在所见即所得编辑器中制作一张表格也要几分钟，但要它完全符合要求可能需要几十分钟，甚至更多时间。

而相比之下，非所见即所得的网页编辑器，就不存在这个问题，因为所有的HTML代码都在监控下产生，但是由于非所见即所得编辑器的先天条件就注定了它的工作低效率。

如何实现两者的完美结合，则既产生干净、准确的HTML代码，又具备所见即所得的高效率、直观性，这一直是网页设计师的梦想。在Dreamweaver之前，FrontPage 98 一直被人们认为是最好的所见即所得网页编辑器，但是它同样继承了所见即所得的种种劣性。

2．Dreamweaver特点

• 最佳的制作效率

Dreamweaver可以用最快速的方式将Fireworks、FreeHand或Photoshop等档案移至网页上。使用检色吸管工具选择显示屏上的颜色设定最接近的网页安全色。对于选单、快捷键与格式控制，都只需一个简单步骤便可完成。Dreamweaver能与喜爱的设计工具，如Playback Flash，Shockwave和外挂模组等搭配，不需离开Dreamweaver便可完成，整体运用流程自然顺畅。除此之外，只要单击便可使Dreamweaver自动开启Firework或Photoshop来进行编辑与设定图片的最佳化。

• 网站管理

使用网站地图可以快速制作网站雏形，设计、更新和重组网页。如果改变了网页位置或档案名称，Dreamweaver会自动更新所有连接。使用支持文字、HTML码、HTML属性标签和一般语法的搜寻及置换功能可以使复杂的网站更新变得迅速又简单。

• 无可比拟的控制能力

Dreamweaver是唯一提供Roundtrip HTML、视觉化编辑与原始码编辑同步的设计工具。它包含HomeSite和BBEdit等主流文字编辑器。帧（frames）和表格的制作速度快得令你无法想象。进阶表格编辑功能使你简单地选择单格、行、栏或作未连续之选取。甚至可以排序或格式化表格群组，Dreamweaver支援精准定位，利用可轻易转换成表格的图层以拖拉置放的方式进行版面配置。

• 所供即所得

Dreamweaver成功整合动态式出版视觉编辑及电子商务功能，提供超强的支援能力给第三方厂商，包含ASP，Apache，BroadVision，Cold Fusion，iCAT，Tango与自行发展的应用软件。当你正使用Dreamweaver在设计动态网页时，所供即所得的功能让你不需要透过浏览器就能预览网页。

• 梦幻样板和XML

Dreamweaver将内容与设计分开，应用于快速网页更新和团队合作网页编辑。建立网页外观的样板，指定可编辑或不可编辑的部分，内容提供者可直接编辑以样式为主的内容却不会因不小心而改变既定之样式。你也可以使用样板正确地汇入或输出XML内容。

• 全方位的呈现

利用Dreamweaver设计的网页，可以全方位地呈现在任何平台的热门浏览器上。对于cascading style sheets的动态HTML支援和鼠标换图效果，声音和动画的DHTML效果资料库可在Netscape和Microsoft浏览器上执行。使用不同浏览器检示功能，Dreamweaver可以告知你在不同浏览器上执行的成效如何。当有新的浏览器上市时，只要从Dreamweaver的网站上下载它的描述档，便可得知详尽的成效报告。

小　　结

本节从HTTP协议和Web服务器架设入手，全面地介绍了与之相关的HTML，XML。最后，我们简略地介绍了基础的HTML网页的开发。通过这一节的学习，让我们对HTTP协议、Web服务器的概况和架设方法以及基本的HTML网页的制作有一个基本的了解。

5.5 计算机网络安全

5.5.1 网络安全概述

Internet 是将分布于不同地点的不同局域网络进行互联而形成的巨大网络。由于 Internet 是一个面向大众的开放系统，对于信息的保密和系统的安全考虑得并不周到，而且随着计算机网络技术的飞速发展，安全问题日益突出，要求提高 Internet 安全性的呼声也日益高涨。

计算机的安全性主要指的是：网络服务的可用性、网络信息的保密性和网络信息的完整性。

由于计算机系统本身就存在着种种安全性问题，互联后的计算机系统的安全问题就更是复杂。目前计算机 Internet 网络面临的威胁主要有以下几个方面：

• 非法访问和破坏

非法访问和破坏也就是所谓的黑客攻击。各种操作系统，诸如 UNIX、Windows、VM、VMS 以及 MVS 等，总会存在这样和那样的漏洞，一些人就利用系统的漏洞，进行网络攻击，其主要目的就是对系统数据进行非法访问和破坏。当然会有利用诸如特洛伊木马、活板门等后门程序，通过驻留在远程计算机中的客户端程序来达到非法访问和窃取加密文件的目的。

• 计算机病毒

计算机病毒程序很容易做出，有着巨大的破坏性。单机病毒已经危害不浅，网络病毒无论是传播速度、破坏性和传播范围等方面都是单机病毒所不能比拟的，其危害性十分巨大。

• 隐蔽通道

这是一种允许违背合法的安全策略的方式进行操作系统进程间通信的通道，又分为隐蔽存储通道和隐蔽时间通道。

• 拒绝服务攻击

拒绝服务攻击是一种破坏性较强的攻击手段，最早的拒绝服务攻击是“电子邮件炸弹”，它能使用户在很短的时间内收到大量的电子邮件，使用户系统不能处理正常业务，严重时会使系统崩溃、网络瘫痪。

5.5.2 数据安全技术

随着网络应用的领域越来越多，对于数据安全的关注也成为当今网络安全领域研究的热点之一。不论从个人隐私还是信息保密的角度出发，对数据安全的研究，都有其广阔的应用前景。下面介绍几种常见的数据安全技术。

• 身份验证

身份验证是一致性验证的一种，验证是建立一致性证明的一种手段。身份验证主要包括验证依据、验证系统和安全要求。身份验证技术是在计算机中最早应用的安全技术，现在也仍然在广泛应用，它是 Internet 上信息安全的第一道屏障。在实际实现上，身份验证又

有一些不同的具体方式，比如表单身份验证、证书身份验证等。在 Windows 环境下，还可以采用 Windows 身份验证。

表单身份验证是指接受用户凭据的自定义用户界面组件，例如，一个用户名和密码。现在使用的许多 Internet 应用程序具有这种供用户登录的表单。值得注意的是，表单本身并不执行身份验证，它仅仅是一种获得用户凭据的方法。身份验证是通过使用自定义代码访问用户名和密码数据库来实现的。验证用户身份后，服务器一般会通过某种方式向客户端表明其已经通过身份验证，可以进行后续请求。如果需要，你可以强制用户验证每个请求，但这样会影响性能和可伸缩性。在 Web 上，解决这个问题，一般可以采用 COOKIE 的方式。

证书身份验证是指用户通过一些安全的途径从需要验证的对象那里获得一个能够标示用户身份的电子证书。然后，在下次与需要验证对象进行通信时，只要证书还在有效期内，用户“出示”自己的电子证书就可以证明自己的身份。

Windows 身份验证是指通过利用 IIS 的身份验证功能，当 IIS 完成身份验证后，ASP .NET 使用已验证标识的标记来授权访问。当使用该方法验证时，ASP .NET 根据已验证用户在应用程序环境中构造并附加一个 Windows Principal 对象。这样，ASP .NET 线程就能够作为已验证用户运行，并可获得用户的组成员身份。当浏览器向 Web 服务器发出请求，Web 服务器将身份验证交给 Windows，此时 IIS 给浏览器提供登录对话框，Windows 完成身份验证后，ASP .NET 程序获得用户账户，然后根据系统的权限数据库分配用户权限，并将结果返回浏览器，完成用户的登录。

• 存取控制

存取控制规定何种主机对何种客体具有何种操作权利。存取控制是网络安全理论的重要内容，主要包括人员限制、数据标识、权限控制、类型控制和风险分析。存取控制也是最早采用的安全技术之一，它一般与身份验证技术一起使用，赋予不同身份的用户不同的操作权限，以实现不同安全级别的信息分级管理。

• 数据完整保证

完整性证明是在数据传输过程中，验证收到的数据和原来数据之间保持完全一致的证明手段。基偶效验是最早采用的数据完整性验证的方法，它虽不能保证数据的完整性，只起到基本的验证作用，但由于它的实现非常简单（一般都由硬件实现），现在仍然广泛应用于网络数据的传输和保护中。近几年来研究比较多的是数字签名算法，它们虽然可以保证数据的完整性，但由于实现起来比较复杂，系统开销比较大，一般只用于完整性要求较高的领域，特别是商业、金融业等领域。

5.5.3 通信安全技术

• 防火墙技术

防火墙是在内部网与外部网之间实施安全防范的系统，可被认为是一种访问控制机制，用于确定哪些内部服务允许外部访问，以及允许访问哪些外部服务。

Internet 使用的是 TCP/IP 协议，这种协议在设计时没有考虑到安全的问题。由于操作系统和应用程序越来越复杂，开发人员不可能测试出其中所有的安全漏洞，而且用户和网络管理员没有足够的精力关注网络的安全控制，此时的网络布满了各种各样的安全隐患，连接在 Internet 上的计算机或其他网络系统随时都有可能受到外界的恶意攻击或被窃取信息。

防火墙就像一把保护伞一样，保护单机用户和局域网用户。

1．防火墙的作用

防火墙的英文名称为 Firewall，它是一个或一组网络设备，诸如计算机系统或路由器等，在两个或多个网络间加强访问控制，以保护一个网络不受来自另一个网络的攻击的安全技术。防火墙系统决定了哪些内部服务可以被外界访问，外界的哪些人可以访问内部的哪些服务，以及内部的人员可以访问哪些资源。所有往来的信息都必须经过防火墙，只有被授权的数据才能通过。

总的来说，防火墙具有下列功能：

• 禁止不安全服务

网络中很多服务是不安全的，例如，NIS 和 NFS 服务，通过禁止这些服务，防火墙可以极大地提高网络安全和减少子网中主机的风险，同时它还可以禁止某些主机上的不安全的端口服务。

• 网络地址转换

用以转换对外的真正 IP 地址与局域网内部的虚拟 IP 地址。NAT 防火墙至少具有以下两个优点：一是解决合法 IP 地址不足的问题；二是隐藏内部的 IP 地址，保护内部网络的安全。一般 NAT 应能够提供一对一与多对一的 IP 地址转换。

• 使用者认证

防火墙可以实现对系统的访问控制，控制授权非特定 IP 地址使用者的联机，如允许从外部访问某些资源，或禁止访问某些资源，并通过 IP 地址和 MAC 地址的绑定难的方式来加强安全管理。

• 预警功能

当入侵事件或异常情况发生时，防火墙应能提供各式的预警方式以通知防火墙管理者前来处理，一个好的防火墙应至少提供如 E-mail、Pager、SNMPTrap 等预警方式。

• 记录和分析工具

多数的入侵事件无法被发现或被追踪，主要在于记录的遗失甚至没有记录文件，而这正是发现与追踪入侵事件最重要的线索与最直接的证据。一般 Log 包含联机记录、事件记录与系统记录等，这些记录至少要保存 3 个月以上。记录分析工具帮助管理者从大量的 Log 中提取出所需的资料，提供图形与表格接口，用于判断各种入侵事件与不当使用网络的状况，而不少的防火墙没有提供这样的功能，企业需要另外采购记录分析工具。

早期的防火墙主要起屏蔽主机和加强访问控制的作用，现在的防火墙则逐渐集成了信息安全技术中的最新研究成果，一般都具有加密解密和压缩解压等功能，这些技术增强了信息在 Internet 的安全性。尽管目前的防火墙功能比较强大，能够防范一些网络攻击，但是也有它的局限性：

• 防火墙不能防范人为因素的攻击。防火墙不能防止用户误操作造成的威胁，以及由于口令泄露而受到攻击。

• 防火墙不能防止受病毒感染的软件或文件的传输。由于操作系统、病毒和二进制文件类型（加密、压缩）的种类太多且更新很快，所以防火墙无法逐个检查每个文件以查找病毒。

• 防火墙不能防止数据驱动式的攻击。当有些表面看来无害的数据邮寄或拷贝到内部

网的主机上并被执行时，可能会发生数据驱动式的攻击。例如，一种数据驱动式的攻击可以使主机修改与系统安全有关的配置文件，从而使入侵者下一次更容易攻击该系统。

2．防火墙的类型

目前，各种各样的防火墙充斥于市场上，有的以软件形式运行在普通计算机之上，有的以硬件形式设计在路由器之中。但是，总的来说目前大家所见到的防火墙可以分为以下几种类型：

• 包过滤防火墙

包过滤防火墙的英文名称为 IP Filting Firewall，它通常也被称为“网络级防火墙”。包过滤防火墙的功能是在网络层中对数据包实施有选择的通过，依据系统事先设定好的过滤逻辑，检查数据流中的每个数据包，根据数据包的源地址、目标地址以及包所使用端口确定是否允许该类数据包通过。包过滤式的防火墙会检查所有通过信息包里的 IP 地址，并按照系统管理员所给定的过滤规则过滤信息包。如果防火墙设定某一 IP 为危险的话，从这个地址而来的所有信息都会被防火墙屏蔽掉。

包过滤路由器的最大的优点就是它对于用户来说是透明的，也就是说不需要用户名和密码来登录。这种防火墙速度快而且易于维护，通常作为第一道防线。包过滤路由器的弊端也是很明显的，通常它没有用户的使用记录，这样用户就不能从访问记录中发现黑客的攻击记录。此外，配置繁琐也是包过滤防火墙的一个缺点。它阻挡别人进入内部网络，但也不告诉用户何人进入用户的系统，或者何人从内部进入网际网路。它可以阻止外部对私有网络的访问，却不能记录内部的访问。包过滤另一个关键的弱点就是，不能在用户级别上进行过滤，即不能鉴别不同的用户和防止 IP 地址盗用。

• 代理服务器

代理服务器的英文名称为 Proxy Server，它通常也被称为“应用级网关防火墙”。所谓代理服务，即防火墙内外的计算机系统应用层的链接是通过两个终止于代理服务的链接来实现的，这样便成功地实现了防火墙内外计算机系统的隔离。代理服务是设置在 Internet 防火墙网关上的应用程序，是由网管员允许或拒绝的特定的应用程序或者特定服务。

代理服务器拥有较强的数据流监控、过滤、记录和报告等功能。一般情况下可应用于特定的 Internet 服务，如 HTTP（超文本传输）和 FTP（远程文件传输）等。代理服务器通常拥有高速缓存，缓存中存有用户经常访问的站点的内容，在下一个用户要访问同样的站点时，服务器就不必重复地去搜取同样的内容，极大地节约了用户的宝贵时间和网络资源。

代理服务器像一堵墙挡在内部用户和外界之间，从外面来的访问者只能看到代理服务器而看不到任何内部资源，例如用户的 IP 地址，而内部用户感觉不到它的存在，可以自由访问外部站点。代理服务器可以提供极好的访问控制、登录能力以及地址转换功能，对进出防火墙的信息进行记录，便于管理员监视和管理系统。然而，代理服务器同时也存在一些缺点，例如，它会使网络的访问速度变慢，因为它不允许用户直接访问网络，同时又要处理进入和流出的数据流，因此每增加一种新的媒体应用，就必须对代理服务器进行设置。

• 电路级网关

电路级网关用来监控受信任的客户或服务器与不受信任的主机间的 TCP 握手信息，这样来决定该会话是否合法，电路级网关是在 OSI 模型中会话层上来过滤数据包，这样比包过滤防火墙要高两层。另外，电路级网关还提供一个重要的安全功能——代理服务器。代

理服务器防火墙上运行着“地址转换”的过程，将所有局域网内部的 IP 地址映射到一个“安全”的 IP 地址，这个地址是由防火墙使用的。

电路级网关也存在着一些缺陷，由于电路级网关是在会话层工作的，它无法检查应用层级的数据包。它起着一定的代理服务作用，监视两主机建立连接时的握手信息，判断该会话请求是否合法。一旦会话连接有效，该网关仅复制、传递数据。它在 IP 层代理各种高层会话，具有隐藏内部网络信息的能力，且透明性高。但由于其对会话建立后所传输的具体内容不再做进一步的分析，因此安全性低。

• 规则检查防火墙

该防火墙结合了包过滤防火墙、电路级网关和代理服务器的特点，它同包过滤防火墙一样，能够在 OSI 网络层上通过 IP 地址和端口号，过滤进出的数据包。它也像电路级网关一样，能够检查 SYN 和 ACK 标记和序列数字是否逻辑有序。当然它也像应用级网关一样，可以在 OSI 应用层上检查数据包的内容，查看这些内容是否能符合公司网络的安全规则。

规则检查防火墙虽然集成前三者的特点，但是它并不打破客户/服务器模式来分析应用层的数据，它允许受信任的客户机和不受信任的主机建立直接连接。规则检查防火墙不依靠与应用层有关的代理，而是依靠某种算法来识别进出的应用层数据，这些算法通过已知合法数据包的模式来比较进出数据包，这样从理论上就能比应用级代理在过滤数据包上更有效。它将动态记录、维护各个连接的协议状态，并在网络层对通信的各个层次进行分析、检测，以决定是否允许通过防火墙。因此它兼备了较高的效率和安全性，可以支持多种网络协议和应用，且可以方便地扩展实现多种非标准服务的支持。

• 状态监视器

状态监视器的英文名称为 Statefull Inspection。应该说，就状态监视器作为防火墙技术而言，它的安全特性最佳。状态监视器采用了一个网关上执行网络安全策略的软件引擎——检测模块。它在不影响网络正常工作的前提下，采用抽取相关数据的方法对网络通信的各层实施监测，抽取部分数据，即状态信息，并动态地保存起来作为以后制定安全决策的参考。

检测模块支持多种协议和应用程序，并可以很容易地实现应用和服务的扩充。与其他安全方案不同，当用户访问到达网关的操作系统前，状态监视器要抽取有关数据进行分析，结合网络配置和安全规定作出接纳、拒绝、鉴定或给该通信加密等决定。一旦某个访问违反安全规定，安全报警器就会拒绝该访问，并作下记录向系统管理器报告网络状态。状态监视器的另外一个优点就是可以监测 RemoteProcedureCall 类和 UserDatagramProtocol 类的端口信息。状态监视器的最大一个缺点就是它的配置很复杂，并且还会降低网络的速度。

• 数据加密

现在金融系统和商界普遍使用的算法是美国数据加密标准 DES。Internet 免费提供 PGP 系统。实际上，前面提到的 DES 算法，属于加密体系中的“对称加密”。目前，新兴的 RSA 算法由于其算法函数强度大、破译困难等优点也越来越成为人们选择的目标。RSA 公钥密码算法是目前网络上进行保密通信和数字签名的最有效的安全算法。这套密码技术能够广受大家信赖并深获好评，除了运用上的便利性之外，最重要的原因在于：RSA 公钥密码技术所运用的数学原理，足以保障通信双方的信息安全。RSA 算法属于“非对称加密”。近几年对加密算法的研究主要集中在密码强度分析和实用化研究上。

• 安全协议

安全协议的建立和完善是安全保密系统走上规范化、标准化道路的基本因素。一个较完善的内部网和安全保密系统，至少要实现加密机制、验证机制和保护机制。传统的网络服务程序，例如，ftp、pop 和 telnet 在本质上都是不安全的，因为它们在网络上用明文传送口令和数据，别有用心的人非常容易就可以截获这些口令和数据。而且，这些服务程序的安全验证方式也是有其弱点的，就是很容易受到“中间人”（man-in-the-middle）这种方式的攻击。所谓“中间人”的攻击方式，就是“中间人”冒充真正的服务器接收你的传给服务器的数据，然后再冒充你把数据传给真正的服务器。服务器和你之间的数据传送被“中间人”一转手做了手脚之后，就会出现很严重的问题。下面简要的介绍其中的一部分安全协议：

SSH 安全 Shell 协议集

它具有以下特点：支持远程安全登录传输；自动加密数据提供用户鉴别和数据压缩；以 SSH 监控程序保证安全传输；它采用 TCP-22 端口建立连接；支持多种加密算法 RSA，3DES，IDEA，Blowfish，CAST128，AES256 等。

IPSec 协议

它是 IETF 为 IP 网络层制订的安全标准，与 RFC2401-2412 相关，它只能与 IP 网络一起使用。在 IPv6 中强制使用而 IPv4 版本选择使用。IPSec 不是一个单独的协议，而是一套协议包和协议集合。它能够保证网络层数据包交换安全性。它具有以下特点：

• 安全服务

——数据起源地验证

——无连接数据完整性验证

——数据内容的机密性

——抗重播保护

——有限的数据流机密性保证

• 工作模式

——传输模式与隧道模式

• 连接方式

——主机到服务器、服务器到服务器

——主机到主机

SSL 协议

SSL 是 Secure Socket Layer & Security Socket Layer 的简称，下面简要描述 SSL 是怎样工作的，我们以网上购物为例子。当顾客想从 Web 站点购买某个产品时，顾客和 Web 站点都要进行认证。顾客通常是以提供名字和密码的方式来认证他自己。另一方面，Web 站点通过交换一块签名数据和一个有效的 X.509 证书（作为 SSL 握手的一部分）来认证它自己。顾客的浏览器验证该证书并用所附的公用密钥验证签名数据。一旦双方都认证了，则交易就可以开始了。

小　　结

本节主要介绍以下内容：

1．网络安全概述中描述了当前在网络应用中面临的一些主要的安全问题，以及这些问题可能带来的巨大危害。

2．数据安全技术中从数据安全的角度简单地介绍了保证网络安全的一些基本技术，包括了身份验证、数据完整性等内容。

3．通信安全技术中从通信安全的角度介绍了当前所采取的保证通信安全的技术，通信协议、防火墙等是通信安全技术的基础。

习　　题

1．局域网和广域网各有什么特点？

2．网络为什么要分层？ISO／OSI 把网络体系结构分为哪几层？各层的功能如何？

3．什么是对等实体？什么是服务访问点？什么是协议数据单元？

4．什么是交换技术？其特点有哪些？现在主要存在哪些交换技术？

5．电路交换与分组交换有哪些区别？为什么电路交换一般不应用于网络中?

6．什么是 ATM？其特点有哪些？

7．网络层的主要功能是什么？它要解决哪些问题？

8．什么是拥塞？解决拥塞有哪些方法？

9．传输层所要解决的问题是什么？

10．传输层提供有哪些服务？

11．TCP/IP 网的基本结构是什么？

12．IP 层有哪些特点？

13．什么是互联网地址？它有哪些特点？什么是子网掩码？

14．TCP/IP 在传输层提供了哪两个协议？各有什么特点？

15．TCP 层是如何来进行流控的？

16．传输层建立连接的关键问题是什么？

17．三次握手的基本思想是什么？

18．客户/服务器模型的基本思想是什么？

19．因为在一台主机中，其进程是一个不确定的东西，那么，在网络环境中，通信进程是如何来标识的？

20．Socket 最初是在什么平台上开发的？

21．在 BSD Unix 中，有哪几种类型的 Socket？各 Socket 的网络含义是什么？

22．在 Windows 操作系统中，提供的网络编程接口是什么？

23．什么是 HTML？试参考文中示例，用 HTML 语言编写一网页，将网页的标题和内容全改成“MY FIRST TEST”。

24．Internet 网络地址是什么？它采用了哪些方式来表示一个网络地址？

25．Unix 中描述符是一个怎样的概念？一个进程可以通过哪些方式来获得？

26．什么是远程登录？远程登录的协议有哪些？

27．什么是文件传输？
28．文件传输与文件访问的区别是什么？
29．电子邮件系统体系结构是怎样的？
30．TCP/IP 电子邮件系统与 MOTIS 有什么区别？
31．Internet 网络面临的威胁主要有哪几个方面？
32．网络攻击常用的方法是哪些？
33．防火墙的功能和局限性是什么？